

Solutions Manual

Theory of Finite Automata with an Introduction to Formal Languages

John Carroll and Darrell Long

October 6, 2026

Contents

Introduction	3
0 Preliminaries — Solutions	5
1 Introduction and Basic Definitions — Solutions	16
2 Nerode's Theorem — Solutions	49
3 Minimization — Solutions	73
4 Nondeterministic Finite Automata — Solutions	99
5 Closure Properties — Solutions	123
6 Regular Expressions — Solutions	144
7 Finite State Transducers — Solutions	165
8 Regular and Context-Free Grammars — Solutions	202
9 Normal Forms and Pumping Lemmas — Solutions	225
10 Pushdown Automata — Solutions	242
11 Turing Machines — Solutions	261
12 Decidability — Solutions	272

Introduction

This solutions manual follows the same progression as the text itself. The early chapters build the mathematical language needed for automata theory; the middle chapters develop finite automata, regular languages, and equivalent formalisms; and the later chapters move on to grammars, pushdown automata, Turing machines, and decidability. The chapter-level themes are:

1. **Chapter 0: Preliminaries.** The book begins with the mathematical tools used everywhere else: logic, sets, relations, functions, cardinality, induction, recursion, and Backus-Naur Form.
2. **Chapter 1: Introduction and Basic Definitions.** This chapter introduces alphabets, words, and finite automata, then develops examples, implementation ideas, and practical applications.
3. **Chapter 2: Characterization of Finite Automaton Definable Languages.** The central ideas are right congruences, Nerode's Theorem, and pumping arguments that explain exactly which languages finite automata can recognize.
4. **Chapter 3: Minimization of Finite Automata.** The emphasis here is on homomorphisms, isomorphisms, and algorithms for reducing automata to their essential states.
5. **Chapter 4: Nondeterministic Finite Automata.** This chapter shows how nondeterminism extends the model, how NDFAs compare with DFAs, and how lambda-transitions and circuit realizations fit into the picture.
6. **Chapter 5: Closure Properties.** The main theme is that regular languages are stable under many operations, and the chapter develops the constructions that prove those closure results.
7. **Chapter 6: Regular Expressions.** The book studies the algebra of regular expressions, shows how regular expressions describe the same class of languages as finite automata, and uses language equations to connect the two views.
8. **Chapter 7: Finite-State Transducers.** Here automata become input-output devices, with attention to their definitions, minimization, Moore machines, and concrete applications such as circuit-style implementations.
9. **Chapter 8: Regular Grammars.** This chapter places regular grammars in the larger grammar hierarchy and relates right-linear grammars to automata and regular expressions.
10. **Chapter 9: Context-Free Grammars.** The focus shifts to parse trees, ambiguity, canonical forms, pumping results, and closure properties for context-free languages.

11. **Chapter 10: Pushdown Automata.** The main ideas are the PDA model, its equivalence with context-free grammars, alternative acceptance conventions, and the special behavior of deterministic PDAs.
12. **Chapter 11: Turing Machines.** This chapter develops the Turing machine model, compares its variants, relates it to other language-defining systems, and begins to measure computational power.
13. **Chapter 12: Decidability.** The final chapter studies what can and cannot be decided algorithmically, starting with regular-language questions and ending with undecidability and Turing-decidable languages.

Chapter 0

Preliminaries — Solutions

0.1. Construct truth tables for:

(a) $\neg r \vee (\neg p \downarrow \neg q)$

Recall that $p \downarrow q$ (NOR) is $\neg(p \vee q)$.

p	q	r	$\neg p$	$\neg q$	$\neg p \downarrow \neg q$	$\neg r$	$\neg r \vee (\neg p \downarrow \neg q)$
0	0	0	1	1	0	1	1
0	0	1	1	1	0	0	0
0	1	0	1	0	0	1	1
0	1	1	1	0	0	0	0
1	0	0	0	1	0	1	1
1	0	1	0	1	0	0	0
1	1	0	0	0	1	1	1
1	1	1	0	0	1	0	1

(b) $(p \wedge \neg q) \vee \neg(p \uparrow q)$

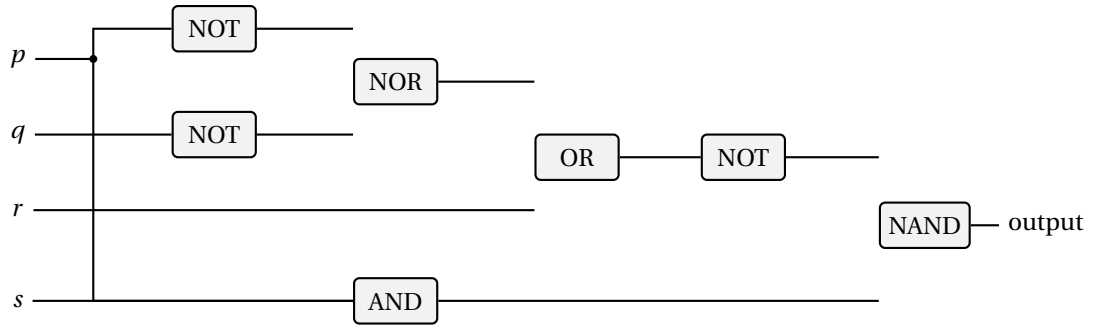
Recall that $p \uparrow q$ (NAND) is $\neg(p \wedge q)$.

p	q	$p \wedge \neg q$	$p \uparrow q$	$\neg(p \uparrow q)$	$(p \wedge \neg q) \vee \neg(p \uparrow q)$
0	0	0	1	0	0
0	1	0	1	0	0
1	0	1	1	0	1
1	1	0	0	1	1

0.2. Draw circuit diagrams for:

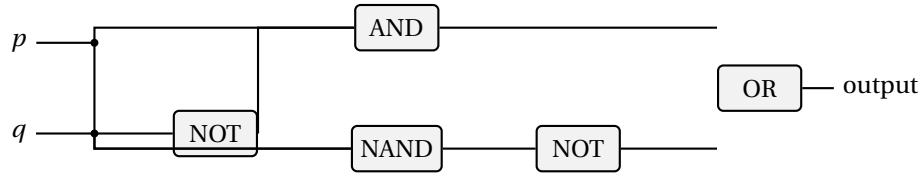
(a) $\neg(r \vee (\neg p \downarrow \neg q)) \uparrow (s \wedge p)$

One corresponding circuit is:



(b) $(p \wedge \neg q) \vee \neg(p \uparrow q)$

One corresponding circuit is:



0.3. **Show that $\{1, 2\} \times \{a, b\}$ and $\{a, b\} \times \{1, 2\}$ are not equal.**

$$\{1, 2\} \times \{a, b\} = \{\langle 1, a \rangle, \langle 1, b \rangle, \langle 2, a \rangle, \langle 2, b \rangle\}$$

$$\{a, b\} \times \{1, 2\} = \{\langle a, 1 \rangle, \langle a, 2 \rangle, \langle b, 1 \rangle, \langle b, 2 \rangle\}$$

Since $\langle 1, a \rangle \in \{1, 2\} \times \{a, b\}$ but $\langle 1, a \rangle \notin \{a, b\} \times \{1, 2\}$ (the first component of any element of $\{a, b\} \times \{1, 2\}$ is a letter, not a number), the two sets are not equal.

0.4. **Let $X = \{1, 2, 3, 4\}$.**

(a) $<$ is $\{\langle 1, 2 \rangle, \langle 1, 3 \rangle, \langle 1, 4 \rangle, \langle 2, 3 \rangle, \langle 2, 4 \rangle, \langle 3, 4 \rangle\}$

(b) $=$ is $\{\langle 1, 1 \rangle, \langle 2, 2 \rangle, \langle 3, 3 \rangle, \langle 4, 4 \rangle\}$

(c) $= \cup <$ is $\{\langle 1, 1 \rangle, \langle 2, 2 \rangle, \langle 3, 3 \rangle, \langle 4, 4 \rangle, \langle 1, 2 \rangle, \langle 1, 3 \rangle, \langle 1, 4 \rangle, \langle 2, 3 \rangle, \langle 2, 4 \rangle, \langle 3, 4 \rangle\}$

(d) $\leq$ is $= \cup <$, which is the set computed in part (c).

0.5. **Show that $\equiv_n$ is an equivalence relation.**

We must show reflexivity, symmetry, and transitivity.

Reflexivity: $a - a = 0 = 0 \cdot n$, so $a \equiv_n a$.

Symmetry: If $a \equiv_n b$, then $n \mid (a - b)$, so $a - b = kn$ for some $k \in \mathbb{Z}$. Then $b - a = (-k)n$, so $n \mid (b - a)$, giving $b \equiv_n a$.

Transitivity: If $a \equiv_n b$ and $b \equiv_n c$, then $a - b = kn$ and $b - c = mn$ for integers k, m . Then $a - c = (a - b) + (b - c) = (k + m)n$, so $a \equiv_n c$.

0.6. **Equivalence classes of $\equiv_0$ on $\mathbb{N}$.**

$a \equiv_0 b$ means $0 \mid (a - b)$, i.e., $a - b = 0$, i.e., $a = b$. So the equivalence classes are all singletons: $\{0\}, \{1\}, \{2\}, \dots$

0.7. Equivalence classes of $\equiv_1$ on $\mathbb{N}$.

$a \equiv_1 b$ means $1 \mid (a - b)$, which is always true. So there is exactly one equivalence class: $\mathbb{N}$ itself.

0.8. Equivalence classes of $\equiv_1$ on $\mathbb{R}$.

Here $x \equiv_1 y$ means $1 \mid (x - y)$, i.e., $x - y \in \mathbb{Z}$ (the difference is an integer). Two reals are equivalent iff they have the same fractional part $\{x\} = x - \lfloor x \rfloor$. The equivalence classes are the cosets of $\mathbb{Z}$ in $(\mathbb{R}, +)$:

$$[r]_{\equiv_1} = \{r + n \mid n \in \mathbb{Z}\}, \quad r \in [0, 1).$$

There are uncountably many such classes, one for each $r \in [0, 1)$.

0.9. Prove the distinct equivalence classes of R form a partition of X .

Let $[x]_R = \{y \in X \mid yRx\}$.

Coverage: For any $x \in X$, reflexivity gives $x \in [x]_R$, so every element of X belongs to at least one class.

Disjointness: Suppose $[x]_R \cap [y]_R \neq \emptyset$; pick z with zRx and zRy . By symmetry xRz , and by transitivity xRy . For any $w \in [x]_R$ we have $wRxRy$, so $w \in [y]_R$; thus $[x]_R \subseteq [y]_R$. By symmetry $[y]_R \subseteq [x]_R$, so $[x]_R = [y]_R$.

Hence the distinct classes are disjoint and cover X , forming a partition.

0.10. Prove X equals the union of the sets in P .

Let $P = \{A_1, \dots, A_n\}$ be a partition of X .

By definition of partition, each $A_i \subseteq X$, so $\bigcup_i A_i \subseteq X$.

Also by definition, every $x \in X$ belongs to some A_i , so $X \subseteq \bigcup_i A_i$.

Therefore $X = \bigcup_{i=1}^n A_i$.

0.11. Prove $R(P)$ is an equivalence relation.

Define $xR(P)y$ iff x and y belong to the same block of P .

Reflexivity: x is in the same block as itself.

Symmetry: If x and y are in the same block, so are y and x .

Transitivity: If x, y are in block A_i and y, z are in block A_j , then since $y \in A_i \cap A_j$ and blocks are disjoint, $A_i = A_j$. So x and z are in the same block.

0.12. Let $X = \{1, 2, 3, 4\}$.

(a) Example where $R(P)$ is a function: $P = \{\{1\}, \{2\}, \{3\}, \{4\}\}$. Then $R(P) = \{\langle x, x \rangle \mid x \in X\}$, which is the identity function.

(b) Example where $R(P)$ is not a function: $P = \{\{1, 2\}, \{3, 4\}\}$. Then $\langle 1, 1 \rangle, \langle 1, 2 \rangle \in R(P)$, so 1 maps to two elements; it is not a function.

0.13. Find the flaw in the “proof” that symmetric and transitive implies reflexive.

The flaw is in the first step. Symmetry says: if xRy then yRx . This requires the hypothesis that xRy . If x is not related to any y at all, we cannot conclude xRx .

Counterexample: Let $X = \{1, 2, 3\}$ and $R = \{\langle 1, 1 \rangle, \langle 2, 2 \rangle, \langle 1, 2 \rangle, \langle 2, 1 \rangle\}$. Then R is symmetric and transitive but $3 \not R 3$, so R is not reflexive.

0.14. **Prove equality refines R .**

We need: $x = y \Rightarrow xRy$. If $x = y$, then by reflexivity of R we have xRx ; since x and y are the same element, this gives xRy . $\square$

0.15. **The “function” $t : \mathbb{R} \rightarrow \mathbb{R}$, pairing x with a real number whose cosine is x .**

- (a) t is not well defined. A function must assign *exactly one* output to each input in its domain. But if $x \in [-1, 1]$, there are generally many real numbers whose cosine is x . For example,

$$\cos\left(\frac{\pi}{3}\right) = \frac{1}{2} = \cos\left(\frac{5\pi}{3}\right),$$

so the input $x = \frac{1}{2}$ would have more than one output. Also, if $x \notin [-1, 1]$, there is no real number whose cosine is x , so the proposed rule is not even defined everywhere on $\mathbb{R}$.

- (b) Restrict domain to $[-1, 1]$ and range to $[0, \pi]$: $t : [-1, 1] \rightarrow [0, \pi]$, $t(x) = \arccos(x)$, is a valid function.

0.16. **Show the converse of $s' : \mathbb{R} \rightarrow \mathbb{R}$, $s'(x) = x^2$, is not a function.**

s'^{-1} (converse) is $\{\langle y, x \rangle \mid x^2 = y\}$. For $y = 1$, both $x = 1$ and $x = -1$ satisfy $x^2 = 1$, so $\langle 1, 1 \rangle$ and $\langle 1, -1 \rangle$ are both in the converse, which is therefore not a function (1 maps to two values).

0.17. **Show s^{-1} exists for $s : \mathbb{P} \rightarrow \mathbb{P}$, $s(x) = x^2$, where $\mathbb{P}$ is nonneg. reals.**

s is injective: if $x^2 = y^2$ and $x, y \geq 0$ then $x = y$. s is surjective onto $\mathbb{P}$: for any $y \geq 0$, $x = \sqrt{y} \geq 0$ satisfies $s(x) = y$. Since s is a bijection, $s^{-1}(y) = \sqrt{y}$ exists.

0.18. **Prove: converse of $f : X \rightarrow Y$ is a function iff f is a bijection.**

($\Rightarrow$) Suppose $\tilde{f} = \{\langle y, x \rangle \mid \langle x, y \rangle \in f\}$ is a function (from Y to X). Then every $y \in Y$ has a unique pre-image under f . *Injectivity*: if $f(x_1) = f(x_2) = y$, then $\tilde{f}(y)$ must equal both x_1 and x_2 ; since $\tilde{f}$ is a function, $x_1 = x_2$. *Surjectivity*: since $\tilde{f}$ is a function with domain Y , every $y \in Y$ is in the domain of $\tilde{f}$, meaning there exists x with $f(x) = y$. Hence f is surjective. (Note: totality of f does not by itself imply surjectivity; surjectivity follows here from $\tilde{f}$ being defined on *all* of Y .)

($\Leftarrow$) If f is injective, each y has at most one pre-image, so $\tilde{f}$ assigns at most one value. If f is surjective, each $y \in Y$ has at least one pre-image, so $\tilde{f}$ is defined everywhere on Y . Together, $\tilde{f}$ is a function.

- 0.19. (a) $\sim(\sim A) = A$: By definition, $\sim A = \{x \mid x \notin A\}$. Then $\sim(\sim A) = \{x \mid x \notin \sim A\} = \{x \mid x \in A\} = A$.

- (b) $\sim(\sim R) = R$ for a relation R (where $\sim R$ is the converse): $\sim R = \{\langle y, x \rangle \mid \langle x, y \rangle \in R\}$. Then $\sim(\sim R) = \{\langle x, y \rangle \mid \langle y, x \rangle \in \sim R\} = \{\langle x, y \rangle \mid \langle x, y \rangle \in R\} = R$.

0.20. **If $f : X \rightarrow Y$ and $g : Y \rightarrow Z$ are one-to-one, prove $g \circ f$ is one-to-one.**

Suppose $(g \circ f)(x_1) = (g \circ f)(x_2)$. Then $g(f(x_1)) = g(f(x_2))$. Since g is injective, $f(x_1) = f(x_2)$. Since f is injective, $x_1 = x_2$.

0.21. **If $f : X \rightarrow Y$ and $g : Y \rightarrow Z$ are onto, prove $g \circ f$ is onto.**

Let $z \in Z$. Since g is surjective, $\exists y \in Y$ with $g(y) = z$. Since f is surjective, $\exists x \in X$ with $f(x) = y$. Then $(g \circ f)(x) = g(f(x)) = g(y) = z$.

0.22. **Define two functions for which $f \circ g = g \circ f$.**

Let $f, g: \mathbb{R} \rightarrow \mathbb{R}$, $f(x) = x + 1$, $g(x) = x + 2$. Then $(f \circ g)(x) = f(x + 2) = x + 3 = g(x + 1) = (g \circ f)(x)$.

0.23. **Define bijections.**

(a) $\mathbb{N}$ and $\mathbb{Z}$: $f: \mathbb{N} \rightarrow \mathbb{Z}$ defined by

$$f(n) = \begin{cases} n/2 & \text{if } n \text{ is even} \\ -(n+1)/2 & \text{if } n \text{ is odd} \end{cases}$$

maps $0 \mapsto 0, 1 \mapsto -1, 2 \mapsto 1, 3 \mapsto -2, 4 \mapsto 2, \dots$

(b) $\mathbb{N}$ and $\mathbb{N} \times \mathbb{N}$: Use the Cantor pairing function $f(m, n) = \frac{(m+n)(m+n+1)}{2} + n$. Its inverse gives a bijection $\mathbb{N} \rightarrow \mathbb{N} \times \mathbb{N}$.

(c) $\mathbb{N}$ and $\mathbb{Q}$: The positive rationals can be arranged in a grid and enumerated by diagonals (Cantor's enumeration). Combined with the bijection $\mathbb{N} \rightarrow \mathbb{Z}$ from part (a), one obtains a bijection $\mathbb{N} \rightarrow \mathbb{Q}$.

(d) $\mathbb{N}$ and $\{a, b, c\}$: No bijection exists, since $\mathbb{N}$ is infinite and $\{a, b, c\}$ has only 3 elements.

0.24. **Use induction to prove $\sum_{k=1}^n k^3 = \frac{n^2(n+1)^2}{4}$.**

Base case ($n = 1$): $1^3 = 1 = \frac{1 \cdot 4}{4}$. ✓

Inductive step: Assume $\sum_{k=1}^n k^3 = \frac{n^2(n+1)^2}{4}$. Then

$$\begin{aligned} \sum_{k=1}^{n+1} k^3 &= \frac{n^2(n+1)^2}{4} + (n+1)^3 \\ &= \frac{n^2(n+1)^2 + 4(n+1)^3}{4} \\ &= \frac{(n+1)^2(n^2 + 4(n+1))}{4} \\ &= \frac{(n+1)^2(n^2 + 4n + 4)}{4} \\ &= \frac{(n+1)^2(n+2)^2}{4}. \end{aligned}$$

This is the formula with n replaced by $n+1$. □

0.25. **Prove $\sum_{k=1}^n k < n^2$ for $n > 1$.**

Base case ($n = 2$): $1 + 2 = 3 < 4 = 2^2$. ✓

Inductive step: Assume $\sum_{k=1}^n k < n^2$. Then

$$\sum_{k=1}^{n+1} k = \sum_{k=1}^n k + (n+1) < n^2 + n + 1.$$

We need $n^2 + n + 1 \leq (n+1)^2 = n^2 + 2n + 1$, which holds since $n \geq 1$. □

0.26. **Prove $n! > n^2$ for $n > 3$.**

Base case ($n = 4$): $4! = 24 > 16 = 4^2$. ✓

Inductive step: Assume $n! > n^2$ for some $n \geq 4$. Then

$$(n+1)! = (n+1) \cdot n! > (n+1) \cdot n^2.$$

We need $(n+1)n^2 \geq (n+1)^2$, i.e., $n^2 \geq n+1$. This holds for $n \geq 4$ because

$$n^2 - n - 1 \geq 4^2 - 4 - 1 = 11 > 0.$$

Hence $n^2 \geq n+1$, as required. □

0.27. **Prove $n! > 2^n$ for $n > 3$.**

Base case ($n = 4$): $24 > 16$. ✓

Inductive step: Assume $n! > 2^n$ for $n \geq 4$. Then

$$(n+1)! = (n+1) \cdot n! > (n+1) \cdot 2^n \geq 5 \cdot 2^n > 2 \cdot 2^n = 2^{n+1}. \quad \square$$

0.28. **Prove $1^2 + 2^2 + \dots + n^2 = \frac{n(n+1)(2n+1)}{6}$.**

Base case ($n = 1$): $1 = \frac{1 \cdot 2 \cdot 3}{6} = 1$. ✓

Inductive step: Assume the formula holds for n . Then

$$\begin{aligned} \sum_{k=1}^{n+1} k^2 &= \frac{n(n+1)(2n+1)}{6} + (n+1)^2 \\ &= \frac{n(n+1)(2n+1) + 6(n+1)^2}{6} \\ &= \frac{(n+1)[n(2n+1) + 6(n+1)]}{6} \\ &= \frac{(n+1)(2n^2 + 7n + 6)}{6} \\ &= \frac{(n+1)(n+2)(2n+3)}{6}. \quad \square \end{aligned}$$

0.29. **Prove $X \cap (X_1 \cup \dots \cup X_n) = (X \cap X_1) \cup \dots \cup (X \cap X_n)$ by induction.**

Base case ($n = 1$): $X \cap X_1 = X \cap X_1$. ✓

Inductive step: Assume the result for n . Then

$$\begin{aligned} X \cap \left(\bigcup_{i=1}^{n+1} X_i \right) &= X \cap \left(\left(\bigcup_{i=1}^n X_i \right) \cup X_{n+1} \right) \\ &= \left(X \cap \bigcup_{i=1}^n X_i \right) \cup (X \cap X_{n+1}) \quad (\text{distributivity for } n=2) \\ &= \left(\bigcup_{i=1}^n (X \cap X_i) \right) \cup (X \cap X_{n+1}) \quad (\text{inductive hypothesis}) \\ &= \bigcup_{i=1}^{n+1} (X \cap X_i). \quad \square \end{aligned}$$

0.30. **Prove** $\sim (X_1 \cup \dots \cup X_n) = (\sim X_1) \cap \dots \cap (\sim X_n)$ **by induction.**

Base case ($n = 2$): De Morgan's law: $\sim (X_1 \cup X_2) = (\sim X_1) \cap (\sim X_2)$. ✓

Inductive step: Assume for n . Then

$$\sim \left(\bigcup_{i=1}^{n+1} X_i \right) = \sim \left(\bigcup_{i=1}^n X_i \cup X_{n+1} \right) = \sim \left(\bigcup_{i=1}^n X_i \right) \cap (\sim X_{n+1}) = \left(\bigcap_{i=1}^n \sim X_i \right) \cap (\sim X_{n+1}) = \bigcap_{i=1}^{n+1} \sim X_i. \quad \square$$

0.31. **Prove** $|\wp(A)| = 2^{|A|}$ **by induction.**

Base case ($|A| = 0$): $\wp(\emptyset) = \{\emptyset\}$, so $|\wp(\emptyset)| = 1 = 2^0$. ✓

Inductive step: Assume $|\wp(A)| = 2^n$ for all sets with $|A| = n$. Let $|B| = n + 1$ and pick any $b \in B$. Set $A = B \setminus \{b\}$. Every subset of B either contains b or not. Those not containing b are exactly $\wp(A)$ (2^n of them). Those containing b are in bijection with $\wp(A)$ via $S \mapsto S \setminus \{b\}$ (2^n more). Total: $2^n + 2^n = 2^{n+1}$.
□

0.32. **Prove strong induction is equivalent to ordinary induction.**

Let $P(n)$ be a statement and $Q(n) = (\forall i \leq n) P(i)$.

($\Rightarrow$) Assume ordinary induction. The strong induction hypotheses are $P(0)$ and $(\forall m)((\forall i \leq m) P(i) \Rightarrow P(m+1))$. Define $Q(n) = (\forall i \leq n) P(i)$. Clearly $Q(0) \Leftrightarrow P(0)$. The strong hypothesis says $Q(m) \Rightarrow P(m+1)$, which means $Q(m) \Rightarrow Q(m+1)$ (since $Q(m+1) = Q(m) \wedge P(m+1)$). By ordinary induction on Q , $(\forall n) Q(n)$, hence $(\forall n) P(n)$.

($\Leftarrow$) Ordinary induction is the special case of strong induction where the hypothesis uses only $P(m)$ instead of $(\forall i \leq m) P(i)$.

0.33. **Determine what strings the BNF defines.**

The grammar defines exactly the strings of the following four forms:

$$\langle \text{integer} \rangle, \quad \langle \text{integer} \rangle., \quad \langle \text{integer} \rangle.\langle \text{natural} \rangle, \quad \langle \text{integer} \rangle.\langle \text{natural} \rangle E \langle \text{integer} \rangle.$$

Here $\langle \text{natural} \rangle$ is any nonempty string of decimal digits, and $\langle \text{integer} \rangle$ is either a natural number or a sign followed by a natural number. Thus the BNF defines signed or unsigned decimal numerals, with an optional fractional part and an optional exponent part.

0.34. **Cofinite sets with universal set $\mathbb{Z}$.**

(a) A finite set: $\{-5, 0, 3, 7\}$.

(b) A cofinite set: $\mathbb{Z} \setminus \{0\} = \{n \in \mathbb{Z} \mid n \neq 0\}$ (complement is $\{0\}$, which is finite).

(c) Neither: $2\mathbb{Z} = \{\dots, -4, -2, 0, 2, 4, \dots\}$, the even integers. Its complement is the set of odd integers, which is infinite; so it is not cofinite. It is infinite, so not finite.

0.35. **Prove the equipotent relation is an equivalence relation.**

Recall: $A \sim B$ iff there exists a bijection $f: A \rightarrow B$.

(a) *Reflexive*: $\text{id}_A: A \rightarrow A$ is a bijection, so $A \sim A$.

(b) *Symmetric*: If $f: A \rightarrow B$ is a bijection, then $f^{-1}: B \rightarrow A$ is also a bijection.

(c) *Transitive*: If $f : A \rightarrow B$ and $g : B \rightarrow C$ are bijections, then $g \circ f : A \rightarrow C$ is a bijection.

0.36. **Define a function showing $|\mathbb{N}| = |\mathbb{N} \times \mathbb{N}|$.**

Let $T_k = \frac{k(k+1)}{2}$ be the k th triangular number. For each $n \in \mathbb{N}$ there are unique integers k, m with

$$n = T_k + m, \quad 0 \leq m \leq k.$$

Define $f : \mathbb{N} \rightarrow \mathbb{N} \times \mathbb{N}$ by

$$f(n) = \langle k - m, m \rangle.$$

Thus

$$f(0) = \langle 0, 0 \rangle, \quad f(1) = \langle 1, 0 \rangle, \quad f(2) = \langle 0, 1 \rangle, \quad f(3) = \langle 2, 0 \rangle, \quad f(4) = \langle 1, 1 \rangle, \quad f(5) = \langle 0, 2 \rangle, \dots$$

so f lists the pairs by diagonals. Every pair $\langle a, b \rangle$ occurs exactly once, namely when $k = a + b$ and $m = b$, so f is a bijection.

0.37. **Show $\mathbb{N}$ is equipotent to $\mathbb{Z}$.**

One bijection $f : \mathbb{N} \rightarrow \mathbb{Z}$ is

$$f(n) = \begin{cases} n/2, & \text{if } n \text{ is even,} \\ -(n+1)/2, & \text{if } n \text{ is odd.} \end{cases}$$

Thus

$$0 \mapsto 0, \quad 1 \mapsto -1, \quad 2 \mapsto 1, \quad 3 \mapsto -2, \quad 4 \mapsto 2, \quad 5 \mapsto -3, \dots$$

so every integer appears exactly once. Hence $\mathbb{N}$ and $\mathbb{Z}$ are equipotent.

0.38. **Show $\mathbb{N}$ is equipotent to $\mathbb{Q}$.**

Let

$$S = \{ \langle a, b \rangle \mid a, b \in \{1, 2, 3, \dots\} \text{ and } \gcd(a, b) = 1 \}.$$

Order the pairs in S first by increasing $a + b$, and among pairs with the same sum, by increasing a . This gives a list

$$\langle a_1, b_1 \rangle, \langle a_2, b_2 \rangle, \langle a_3, b_3 \rangle, \dots$$

Define $h : \{1, 2, 3, \dots\} \rightarrow \mathbb{Q}^+$ by

$$h(n) = \frac{a_n}{b_n}.$$

Every positive rational number has a unique reduced form a/b with $\gcd(a, b) = 1$, so h is a bijection from the positive integers onto $\mathbb{Q}^+$.

Now define $f : \mathbb{N} \rightarrow \mathbb{Q}$ by

$$f(0) = 0, \quad f(2n-1) = h(n), \quad f(2n) = -h(n) \quad (n \geq 1).$$

Then f lists 0, followed by the pairs $h(n), -h(n)$ of positive and negative rationals:

$$0, h(1), -h(1), h(2), -h(2), \dots$$

Hence f is a bijection from $\mathbb{N}$ onto $\mathbb{Q}$, so $\mathbb{N}$ and $\mathbb{Q}$ are equipotent.

0.39. **Show $\wp(\mathbb{N})$ is equipotent to $\{f : \mathbb{N} \rightarrow \{\text{Yes}, \text{No}\}\}$.**

Define $\phi : \wp(\mathbb{N}) \rightarrow \{f : \mathbb{N} \rightarrow \{\text{Yes}, \text{No}\}\}$ by $\phi(A) = \chi_A$, where $\chi_A(n) = \text{Yes}$ if $n \in A$ and No otherwise. This is clearly a bijection (its inverse maps f to $\{n \mid f(n) = \text{Yes}\}$).

0.40. **Show $\wp(\mathbb{N})$ is equipotent to $\mathbb{R}$.**

One explicit bijection is as follows.

By Exercise 0.39, $\wp(\mathbb{N})$ is equipotent to the set

$$B = \{(a_0, a_1, a_2, \dots) \mid a_i \in \{0, 1\}\}$$

of infinite binary sequences. It therefore suffices to show that B is equipotent to $\mathbb{R}$.

Let $E \subseteq B$ be the set of sequences that are eventually all 1s. This set is countable, because such a sequence is determined by a finite initial segment and the position after which only 1s occur. Let

$$D = \left\{ \frac{k}{2^n} \mid n \in \mathbb{N}, 0 \leq k \leq 2^n \right\}$$

be the set of dyadic rationals in $[0, 1]$; D is also countable. Choose a bijection $\eta : E \rightarrow D$.

Now define $\Phi : B \rightarrow [0, 1]$ by

$$\Phi(a_0, a_1, a_2, \dots) = \begin{cases} \sum_{i=0}^{\infty} \frac{a_i}{2^{i+1}}, & \text{if } (a_0, a_1, a_2, \dots) \notin E, \\ \eta(a_0, a_1, a_2, \dots), & \text{if } (a_0, a_1, a_2, \dots) \in E. \end{cases}$$

If a sequence is not eventually all 1s, its binary expansion is the unique binary expansion of a nondyadic real in $[0, 1]$. Thus the first clause is a bijection from $B \setminus E$ onto $[0, 1] \setminus D$. The second clause is a bijection from E onto D . Therefore Φ is a bijection from B onto $[0, 1]$.

It remains to show that $[0, 1]$ is equipotent to $\mathbb{R}$. First define a bijection $j : [0, 1] \rightarrow (0, 1)$ by

$$j(x) = \begin{cases} \frac{1}{2}, & x = 0, \\ \frac{1}{4}, & x = 1, \\ \frac{1}{2^{n+2}}, & x = \frac{1}{2^n} \text{ for } n \geq 1, \\ x, & \text{otherwise.} \end{cases}$$

This simply shifts the countable set $\{1, 1/2, 1/4, 1/8, \dots\}$ inward and fixes all other points. Next define

$$g : (0, 1) \rightarrow \mathbb{R}, \quad g(x) = \begin{cases} 2 - \frac{1}{x}, & 0 < x < \frac{1}{2}, \\ 0, & x = \frac{1}{2}, \\ \frac{1}{1-x} - 2, & \frac{1}{2} < x < 1. \end{cases}$$

The first branch maps $(0, \frac{1}{2})$ onto $(-\infty, 0)$, the middle point maps to 0, and the third branch maps $(\frac{1}{2}, 1)$ onto $(0, \infty)$. Hence g is a bijection from $(0, 1)$ onto $\mathbb{R}$, and $g \circ j$ is a bijection from $[0, 1]$ onto $\mathbb{R}$.

Since $\wp(\mathbb{N}) \sim B$, $B \sim [0, 1]$, and $[0, 1] \sim \mathbb{R}$, the equipotent relation is transitive, so $\wp(\mathbb{N}) \sim \mathbb{R}$.

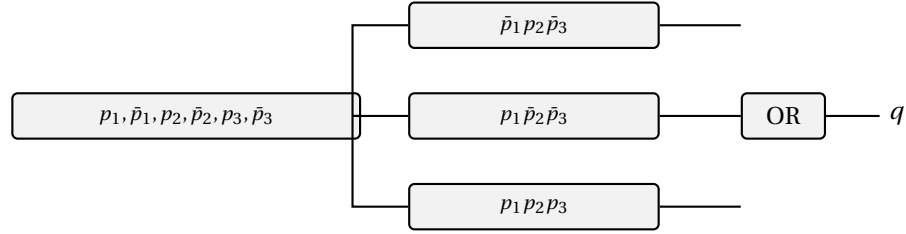
0.41. **Draw a circuit for q (Figure 0.8).**

The truth table shows $q = 1$ when $(p_1, p_2, p_3) \in \{(0, 1, 0), (1, 0, 0), (1, 1, 1)\}$.

Using the rows with $q = 1$, the PDNF is

$$q = (\bar{p}_1 p_2 \bar{p}_3) \vee (p_1 \bar{p}_2 \bar{p}_3) \vee (p_1 p_2 p_3).$$

A clear AND-OR block diagram is:



0.42. **Draw circuits for q_1, q_2, q_3 (Figure 0.9).**

Reading the truth table row by row (rows indexed 0–15, with $p_1 p_2 p_3 p_4$ in binary), we have

$q_1 = 1$ at rows 0, 2, 3, 5, 6, 8, 9, 12,
namely (0000), (0010), (0011), (0101),
(0110), (1000), (1001), (1100).

$q_2 = 1$ at rows 0, 1, 3, 4, 6, 10, 12, 13, 14,
namely (0000), (0001), (0011), (0100), (0110),
(1010), (1100), (1101), (1110).

$q_3 = 1$ at rows 0, 4, 8, 12, 14,
namely (0000), (0100), (1000), (1100), (1110).

Using the truth table and grouping equivalent terms, we can use the following clearer formulas:

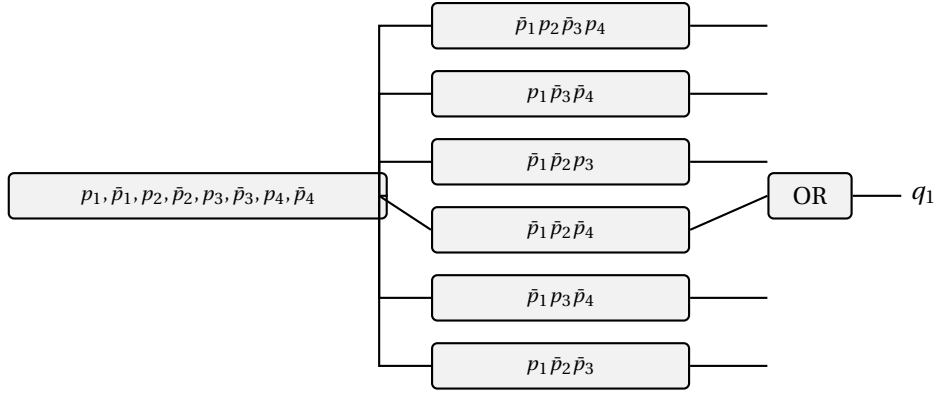
$$q_1 = \bar{p}_1 p_2 \bar{p}_3 p_4 \vee p_1 \bar{p}_3 \bar{p}_4 \vee \bar{p}_1 \bar{p}_2 p_3 \\ \vee \bar{p}_1 \bar{p}_2 \bar{p}_4 \vee \bar{p}_1 p_3 \bar{p}_4 \vee p_1 \bar{p}_2 \bar{p}_3$$

$$q_2 = \bar{p}_1 \bar{p}_2 \bar{p}_3 \vee p_1 p_2 \bar{p}_3 \vee p_2 \bar{p}_4 \vee \bar{p}_1 \bar{p}_2 p_4 \vee p_1 p_3 \bar{p}_4$$

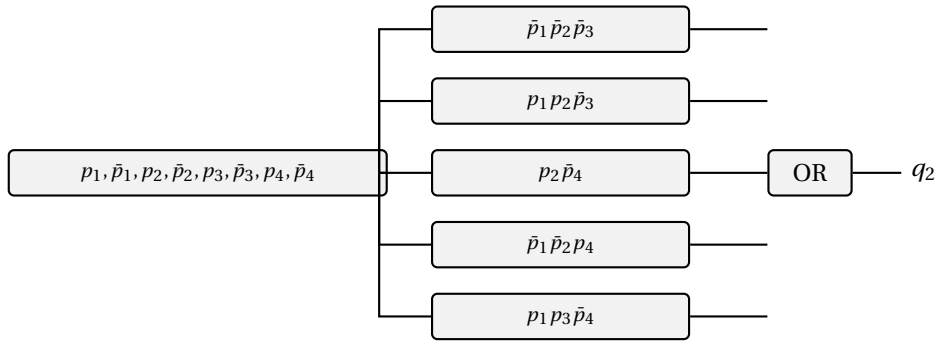
$$q_3 = \bar{p}_3 \bar{p}_4 \vee p_1 p_2 \bar{p}_4$$

Each block diagram below uses one product-term block for each AND subcircuit, fed from the literal bank on the left, and a final OR gate on the right.

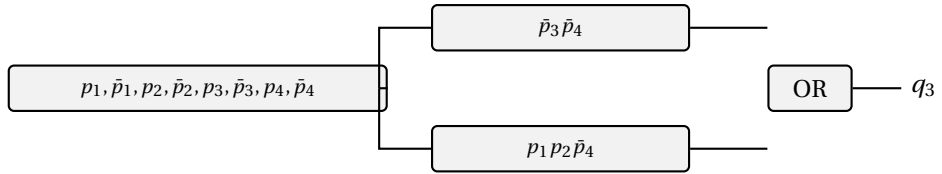
(a) For q_1 :



(b) For q_2 :



(c) For q_3 :



Chapter 1

Introduction and Basic Definitions — Solutions

1.1. Show $\bar{\delta}_t = \bar{\delta}_h$.

Let $P(n)$: $(\forall s \in S)(\forall x \in \Sigma^n)(\bar{\delta}_t(s, x) = \bar{\delta}_h(s, x))$.

Base: $n = 0$: $\bar{\delta}_t(s, \lambda) = s = \bar{\delta}_h(s, \lambda)$. For $n = 1$: $\bar{\delta}_t(s, a) = \delta(s, a) = \bar{\delta}_h(s, a)$.

Inductive step: Let $x = ya$ where $|y| = n$ and $a \in \Sigma$. Assume $\bar{\delta}_t(s, y) = \bar{\delta}_h(s, y)$ for all s, y with $|y| \leq n$.

We prove $\bar{\delta}_t(s, wa) = \delta(\bar{\delta}_t(s, w), a)$ for all $w \in \Sigma^*$, $a \in \Sigma$, $s \in S$, by induction on $|w|$.

If $|w| = 0$: $\bar{\delta}_t(s, a) = \delta(s, a)$. ✓

If $|w| = n+1$, write $w = bv$ with $b \in \Sigma$: $\bar{\delta}_t(s, bva) = \bar{\delta}_t(\delta(s, b), va) = \delta(\bar{\delta}_t(\delta(s, b), v), a)$ [IH] $= \delta(\bar{\delta}_t(s, bv), a)$.

This establishes the claim. Now:

$$\bar{\delta}_h(s, wa) = \delta(\bar{\delta}_h(s, w), a) = \delta(\bar{\delta}_t(s, w), a) \text{ [IH]} = \bar{\delta}_t(s, wa). \quad \square$$

1.2. Modify vending machine to include coin-return r .

Add a new input symbol $\mathbf{r}$ (coin return). When $\mathbf{r}$ is pressed, transition back to the initial state s_0 (0 cents deposited) from any state. All other transitions remain unchanged. Formally, for every state s_i (representing i cents), $\delta(s_i, \mathbf{r}) = s_0$.

1.3. The DFA of Figure 1.26.

(a) The quintuple is

$$A = \langle \{\mathbf{0}, \mathbf{1}\}, \{q_0, q_1\}, q_0, \delta, \{q_1\} \rangle,$$

where

$$\delta(q_0, \mathbf{0}) = q_1, \quad \delta(q_0, \mathbf{1}) = q_1, \quad \delta(q_1, \mathbf{0}) = q_1, \quad \delta(q_1, \mathbf{1}) = q_0.$$

(b) The language is

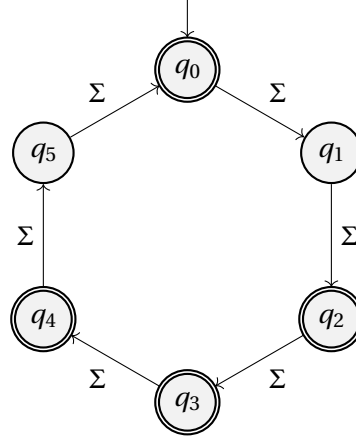
$$L(A) = \Sigma^* \mathbf{0}(\mathbf{11})^* \cup \mathbf{1}(\mathbf{11})^*, \quad \Sigma = \{\mathbf{0}, \mathbf{1}\}.$$

Equivalently, a word is accepted iff it either ends in $\mathbf{0}$ followed by an even number of trailing $\mathbf{1}$ s, or consists entirely of an odd number of $\mathbf{1}$ s.

1.4. **Build a DFA accepting $L = \{x \mid |x| \text{ is a multiple of 2 or 3} \text{ over } \{a, b, c\}$.**

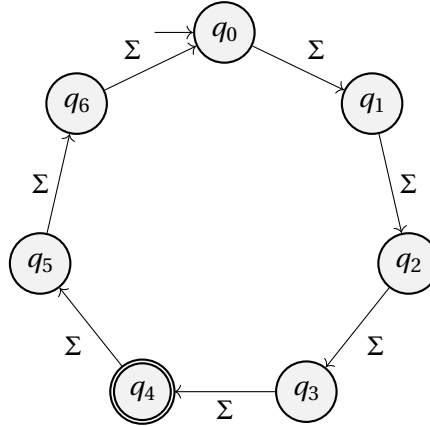
We need $|x| \equiv 0 \pmod{2}$ or $|x| \equiv 0 \pmod{3}$, i.e., $|x| \in \{0, 2, 3, 4, 6, 8, 9, \dots\}$. The complement is $\{|x| \equiv 1 \pmod{6} \text{ or } |x| \equiv 5 \pmod{6}\}$. Use a 6-state DFA tracking $|x| \pmod{6}$, with final states $\{0, 2, 3, 4\}$ (i.e., everything except 1 and 5 mod 6).

$S = \{q_0, q_1, q_2, q_3, q_4, q_5\}$, $s_0 = q_0$, $F = \{q_0, q_2, q_3, q_4\}$, $\delta(q_i, a) = q_{(i+1) \bmod 6}$ for all $a \in \Sigma$.



1.5. **DFA for L_1, L_2, L_3 over $\{0, 1\}$.**

(a) $L_1 = \{x \mid |x| \bmod 7 = 4\}$: 7-state DFA with states $q_0, \dots, q_6$, $F = \{q_4\}$, $\delta(q_i, a) = q_{(i+1) \bmod 7}$.



(b) $L_2 = \Sigma^* \setminus \{w \mid w \text{ ends in } 1\}$: The complement is all words ending in **1**. So L_2 accepts words not ending in **1**: either λ , or words ending in **0**. Two-state DFA: s_0 (start, final – not ending in 1, or empty), s_1 (ending in 1, nonfinal). $\delta(s_0, 0) = s_0$, $\delta(s_0, 1) = s_1$, $\delta(s_1, 0) = s_0$, $\delta(s_1, 1) = s_1$.

(c) $L_3 = \{y \mid |y|_0 = |y|_1\}$: not FAD. Suppose some DFA with n states accepted it. Consider the $n+1$ strings

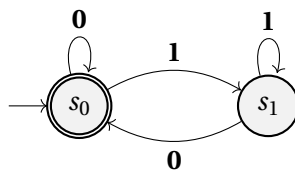
$$\lambda, 0, 00, \dots, 0^n.$$

Two of them, say 0^i and 0^j with $i < j$, must lead from the start state to the same state. Then appending 1^i would give the same final outcome for both strings. But

$$0^i 1^i \in L_3 \quad \text{while} \quad 0^j 1^i \notin L_3,$$

since the second string has more **0**s than **1**s. This contradiction shows that no DFA can accept L_3 .

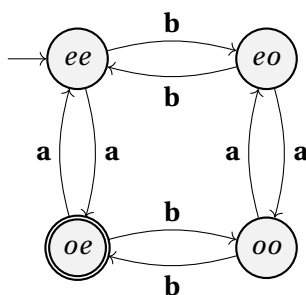
The 2-state DFA for L_2 :



1.6. DFAs A_1, A_2, A_3, A_4 over $\{a, b\}$.

- (a) Use a 4-state DFA with states $\{ee, eo, oe, oo\}$ (parity of #a, parity of #b). $A_1: F = \{oe\}$ (#a odd, #b even). $A_2: F = \{eo, oe, oo\}$ (not ee). $A_3: F = \{oe, eo\}$ (exactly one count even). $A_4: F = \{ee, oe\}$ (#a even).
- (b) Each machine is a quotient or sub-machine of the 4-state product machine from Example 1.10, which tracks parities of #a and #b.

The common 4-state machine (shown here for A_1 with $F = \{oe\}$; change F for the other sub-machines):



1.7. Modify the machine from Example 1.10 to accept $L(M) - \{\lambda\}$.

In Example 1.10,

$$S = \{\langle 0, 0 \rangle, \langle 0, 1 \rangle, \langle 1, 0 \rangle, \langle 1, 1 \rangle\}, \quad s_0 = \langle 0, 0 \rangle, \quad F = \{\langle 0, 0 \rangle\},$$

and the transition function toggles the first coordinate on input **a** and the second coordinate on input **b**.

Add a fresh start state s'_0 that is not final. Let

$$S' = S \cup \{s'_0\}, \quad F' = \{\langle 0, 0 \rangle\},$$

and keep the old transitions on the old states. Define the two new transitions by

$$\delta'(s'_0, \mathbf{a}) = \langle 1, 0 \rangle, \quad \delta'(s'_0, \mathbf{b}) = \langle 0, 1 \rangle.$$

Thus

$$M' = \langle \{\mathbf{a}, \mathbf{b}\}, S', s'_0, \delta', F' \rangle.$$

The word λ is rejected because $s'_0 \notin F'$. For any nonempty word, the first input letter moves M' to exactly the state that M would reach after reading that same first letter from $\langle 0, 0 \rangle$, and all later transitions are unchanged. Hence every nonempty word is accepted by M' iff it was accepted by M , so $L(M') = L(M) - \{\lambda\}$.

1.8. General procedure to modify M to accept $L(M) - \{\lambda\}$.

If $s_0 \notin F$ then $\lambda \notin L(M)$ and M already accepts $L(M) \setminus \{\lambda\} = L(M)$; nothing to do.

If $s_0 \in F$, add a fresh nonfinal start state s'_0 and set

$$\delta'(s'_0, a) = \delta(s_0, a) \quad \text{for all } a \in \Sigma,$$

leaving all other transitions and the final-state set F unchanged. The new machine $M' = (\Sigma, S \cup \{s'_0\}, s'_0, \delta', F)$ accepts exactly $L(M) \setminus \{\lambda\}$.

Correctness: λ is rejected because $s'_0 \notin F$. For any nonempty $x = a_1 \cdots a_k$, the computation in M' follows $s'_0 \xrightarrow{a_1} \delta(s_0, a_1) \xrightarrow{a_2} \cdots$, which mirrors the computation of M from s_0 on x , so $x \in L(M')$ iff $x \in L(M)$.

Why not remove s_0 from F ? That would reject nonempty strings x with $\bar{\delta}(s_0, x) = s_0$, which belong to $L(M)$ and should remain accepted.

1.9. General procedure to modify M to accept $L(M) \cup \{\lambda\}$.

If $s_0 \in F$ already, then M already accepts λ , so nothing needs to be changed.

If $s_0 \notin F$, add a fresh state s'_0 and make it the new start state. Let

$$S' = S \cup \{s'_0\}, \quad F' = F \cup \{s'_0\}.$$

Define the new transition function by

$$\delta'(s, a) = \delta(s, a) \quad (s \in S, a \in \Sigma),$$

and

$$\delta'(s'_0, a) = \delta(s_0, a) \quad (a \in \Sigma).$$

Then $M' = \langle \Sigma, S', s'_0, \delta', F' \rangle$ accepts exactly $L(M) \cup \{\lambda\}$.

Correctness: λ is accepted because $s'_0 \in F'$. For any nonempty word $x = a_1 \cdots a_k$, the computation in M' is

$$s'_0 \xrightarrow{a_1} \delta(s_0, a_1) \xrightarrow{a_2} \cdots,$$

which is exactly the same computation that M performs from s_0 on x . Thus $x \in L(M')$ iff $x \in L(M)$.

Why not simply add s_0 to F ? That could also accept nonempty strings x for which $\bar{\delta}(s_0, x) = s_0$, not just λ .

1.10. DFA for $\Psi = \{x \mid (x \text{ begins with } \mathbf{d}) \vee (x \text{ contains } \mathbf{bb})\}$.

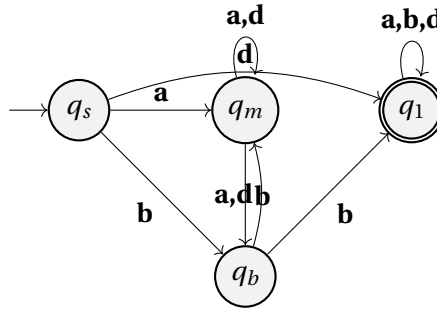
The machine must distinguish “first character is **d**” (which makes the string accepted regardless of what follows) from “**d** appears later”. A 3-state machine conflates these; the minimal DFA has **4 states**:

- q_s : start (no characters read yet).
- q_m : at least one character read; string does not start with **d** and no **bb** seen; last char was not **b**.
- q_b : same as q_m but last char was **b** (pending **bb**).
- q_1 : accepting sink (string starts with **d**, or **bb** has been seen).

$F = \{q_1\}$.

	a	b	d
q_s	q_m	q_b	q_1
q_m	q_m	q_b	q_m
q_b	q_m	q_1	q_m
q_1	q_1	q_1	q_1

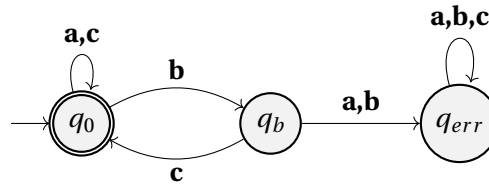
Spot-check: **bd** — $q_s \xrightarrow{b} q_b \xrightarrow{d} q_m$, rejected. ✓ **bb** — $q_s \xrightarrow{b} q_b \xrightarrow{b} q_1$, accepted. ✓ **d** — $q_s \xrightarrow{d} q_1$, accepted. ✓



1.11. **DFA for $\Phi = \{x \mid \text{every } b \text{ in } x \text{ is immediately followed by } c\}$.**

States: q_0 (ok, last char not **b**), q_b (last char was **b**), q_{err} (dead state). $F = \{q_0\}$.

$$\begin{aligned} \delta(q_0, \mathbf{a}) &= q_0, & \delta(q_0, \mathbf{b}) &= q_b, & \delta(q_0, \mathbf{c}) &= q_0, \\ \delta(q_b, \mathbf{c}) &= q_0, & \delta(q_b, \mathbf{a}) &= q_{err}, & \delta(q_b, \mathbf{b}) &= q_{err}, \\ \delta(q_{err}, \mathbf{a}) &= \delta(q_{err}, \mathbf{b}) = \delta(q_{err}, \mathbf{c}) &= q_{err}. \end{aligned}$$



1.12. **DFA for Ω (numbers divisible by 7).**

The key insight: if the current state represents $n \bmod 7$ (the value of the number read so far), then reading digit d gives the new value $(10n + d) \bmod 7$.

States $q_0, \dots, q_6$ (representing $0, \dots, 6 \bmod 7$). $s_0 = q_0$, $F = \{q_0\}$.

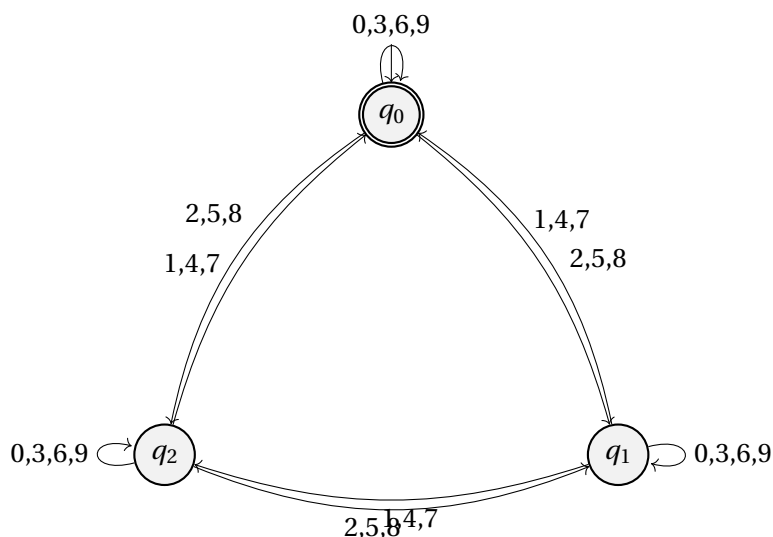
$$\delta(q_i, d) = q_{(10i+d) \bmod 7}.$$

1.13. **DFA for Γ (divisible by 3).**

Three states q_0, q_1, q_2 (representing 0, 1, 2 mod 3). As in Exercise 1.11, we use the chapter's convention that λ represents 0, so q_0 is both the start state and a final state. Thus $F = \{q_0\}$.

$$\delta(q_i, d) = q_{(i+d) \bmod 3}$$

since $(10i + d) \bmod 3 = (i + d) \bmod 3$ (because $10 \equiv 1 \pmod{3}$).



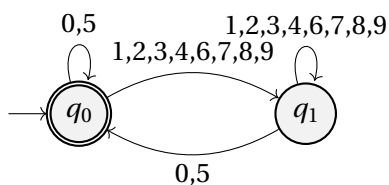
If one wants λ rejected instead, insert a fresh nonfinal start state whose outgoing transitions copy those of q_0 .

1.14. **DFA for K (divisible by 5).**

Since $10 \equiv 0 \pmod{5}$, we have $(10i + d) \bmod 5 = d \bmod 5$. So only the last digit matters.

Five-state DFA: $\delta(q_i, d) = q_{d \bmod 5}$; $F = \{q_0\}$.

Two-state DFA: q_0 (final, last digit in $\{0,5\}$), q_1 (nonfinal, last digit not in $\{0,5\}$). $\delta(q_i, d) = q_0$ if $d \in \{0,5\}$, else q_1 .



Again, this follows the convention from Exercise 1.11 that λ represents 0. If one prefers to exclude λ , add a fresh nonfinal start state that copies the outgoing transitions of q_0 .

1.15. **DFA for the first eight primes:** $\{2, 3, 5, 7, 11, 13, 17, 19\}$.

This language is finite, so it is FAD. A concrete DFA is the prefix-tree machine for the set

$$P = \{\lambda, \mathbf{1}, \mathbf{11}, \mathbf{13}, \mathbf{17}, \mathbf{19}, \mathbf{2}, \mathbf{3}, \mathbf{5}, \mathbf{7}\}.$$

Thus the full state set is

$$S = \{q_\lambda, q_1, q_{11}, q_{13}, q_{17}, q_{19}, q_2, q_3, q_5, q_7, q_{dead}\},$$

that is, the states q_x for $x \in P$ together with q_{dead} . The start state is q_λ , and the final states are

$$F = \{q_2, q_3, q_5, q_7, q_{11}, q_{13}, q_{17}, q_{19}\}.$$

For any state q_x and any digit $\mathbf{d}$, define

$$\delta(q_x, \mathbf{d}) = \begin{cases} q_{x\mathbf{d}} & \text{if } x\mathbf{d} \in P, \\ q_{dead} & \text{otherwise.} \end{cases}$$

Also $\delta(q_{dead}, \mathbf{d}) = q_{dead}$ for every digit $\mathbf{d}$. Thus q_λ branches to q_2, q_3, q_5, q_7 , and to q_1 on input $\mathbf{1}$; then q_1 branches to $q_{11}, q_{13}, q_{17}, q_{19}$ on inputs $\mathbf{1}, \mathbf{3}, \mathbf{7}, \mathbf{9}$, respectively. Every other transition goes to q_{dead} , so the machine accepts exactly the first eight primes.

1.16. Combinations of u, v, w with $uvw = \mathbf{cab}$.

- (a) The string $\mathbf{cab}$ has length 3. We must split it as uvw with $u, v, w \in \Sigma^*$. The split points are $i = |u|, j = |u| + |v|$ with $0 \leq i \leq j \leq 3$. There are $\binom{n+2}{2} = \binom{5}{2} = 10$ ways for length 3:
 $(u, v, w): (\lambda, \lambda, \mathbf{cab}), (\lambda, \mathbf{c}, \mathbf{ab}), (\lambda, \mathbf{ca}, \mathbf{b}), (\lambda, \mathbf{cab}, \lambda), (\mathbf{c}, \lambda, \mathbf{ab}), (\mathbf{c}, \mathbf{a}, \mathbf{b}), (\mathbf{c}, \mathbf{ab}, \lambda), (\mathbf{ca}, \lambda, \mathbf{b}), (\mathbf{ca}, \mathbf{b}, \lambda), (\mathbf{cab}, \lambda, \lambda).$
- (b) For a string of length n : the number of ways to choose split points $0 \leq i \leq j \leq n$ is $\binom{n+2}{2} = \frac{(n+1)(n+2)}{2}$.

1.17. DFA for $\Xi = \{x \mid x \text{ contains } \geq 2 \text{ consecutive bs and does not contain aa}\}$.

The DFA tracks two conditions simultaneously: whether $\mathbf{aa}$ has ever appeared, and whether $\mathbf{bb}$ has ever appeared. Once $\mathbf{aa}$ appears the string is rejected regardless; once $\mathbf{bb}$ appears we continue watching for $\mathbf{aa}$.

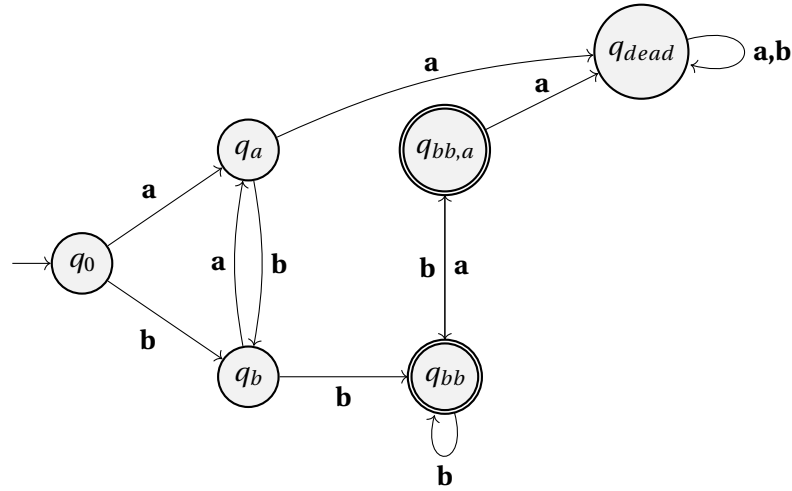
States (6 total):

- q_0 : start; no $\mathbf{bb}$, no $\mathbf{aa}$, last char none.
- q_a : no $\mathbf{bb}$, no $\mathbf{aa}$, last char = $\mathbf{a}$.
- q_b : no $\mathbf{bb}$, no $\mathbf{aa}$, last char = $\mathbf{b}$.
- q_{bb} : seen $\mathbf{bb}$, no $\mathbf{aa}$, last char = $\mathbf{b}$.
- $q_{bb,a}$: seen $\mathbf{bb}$, no $\mathbf{aa}$, last char = $\mathbf{a}$.
- q_{dead} : seen $\mathbf{aa}$ (trap).

$$F = \{q_{bb}, q_{bb,a}\}.$$

Transitions:

	a	b
q_0	q_a	q_b
q_a	q_{dead}	q_b
q_b	q_a	q_{bb}
q_{bb}	$q_{bb,a}$	q_{bb}
$q_{bb,a}$	q_{dead}	q_{bb}
q_{dead}	q_{dead}	q_{dead}



1.18. Modify Figure 1.11 to reject DO and DATA.

Keep the structure of Example 1.9, but split the ordinary identifier states so that the machine can remember the exceptional prefixes **D**, **DO**, **DA**, **DAT**, and **DATA** until it is clear whether the identifier is exactly one of the reserved words.

One convenient design uses the states

$$s_0, d_1, do_2, da_2, dat_3, data_4, g_1, g_2, g_3, g_4, g_5, g_6, s_7,$$

where g_i means “a legal identifier of length i whose prefix is no longer one of the exceptional words,” and s_7 is the dead state. Embedded blanks behave exactly as in Example 1.9: they loop in every nondead state.

The key transitions are:

- From s_0 , the letter **D** goes to d_1 , every other letter goes to g_1 , digits and characters from Γ go to s_7 , and $\diamond$ loops to s_0 .
- From d_1 , the letter **O** goes to do_2 , **A** goes to da_2 , and every other letter or digit goes to g_2 .
- From do_2 , any further letter or digit goes to g_3 ; if the identifier ends while the machine is still in do_2 , the word is rejected.
- From da_2 , the letter **T** goes to dat_3 ; every other letter or digit goes to g_3 .

- From dat_3 , the letter **A** goes to $data_4$; every other letter or digit goes to g_4 .
- From $data_4$, any further letter or digit goes to g_5 ; if the identifier ends while the machine is still in $data_4$, the word is rejected.
- The states $g_1, \dots, g_6$ behave exactly like the length-counting states $s_1, \dots, s_6$ of Example 1.9.

The final states are

$$\{d_1, da_2, dat_3, g_1, g_2, g_3, g_4, g_5, g_6\}.$$

So identifiers such as **D**, **DA**, **DAT**, **DOG**, and **DATAI** are accepted, while exactly **DO** and exactly **DATA** are rejected.

1.19. Modify machine for real-number constants (extended BNF).

The new BNF allows leading decimal points like **.5** and **+ .5**. Modify the DFA from Figure 1.13 to add transitions that accept the input immediately after a sign symbol or at the start by going to a decimal-point state, from which digits are read as the fractional part.

(a) A concrete modified DFA is obtained by using the states

$$s_0 = \text{start}, \quad s_1 = \text{sign}, \quad s_2 = \text{integer part}, \quad s_3 = \text{dot after integer},$$

$$s_4 = \text{E just read}, \quad s_5 = \text{sign after E}, \quad s_6 = \text{exponent digits},$$

$$s_8 = \text{fractional digits}, \quad s_9 = \text{leading dot before any digit}, \quad s_7 = \text{dead}.$$

The accepting states are $\{s_2, s_3, s_8, s_6\}$. The important transitions are:

$$\begin{aligned} \delta(s_0, \text{digit}) &= s_2, & \delta(s_0, +) &= s_1, & \delta(s_0, -) &= s_1, & \delta(s_0, \cdot) &= s_9, \\ \delta(s_1, \text{digit}) &= s_2, & \delta(s_1, \cdot) &= s_9, \\ \delta(s_2, \text{digit}) &= s_2, & \delta(s_2, \cdot) &= s_3, \\ \delta(s_3, \text{digit}) &= s_8, \\ \delta(s_8, \text{digit}) &= s_8, & \delta(s_8, E) &= s_4, \\ \delta(s_9, \text{digit}) &= s_8, \\ \delta(s_4, \text{digit}) &= s_6, & \delta(s_4, +) &= s_5, & \delta(s_4, -) &= s_5, \\ \delta(s_5, \text{digit}) &= s_6, \\ \delta(s_6, \text{digit}) &= s_6. \end{aligned}$$

All unspecified transitions go to s_7 , and s_7 loops to itself. This accepts strings such as **.5**, **+ .5**, and **- .5E7**, while still rejecting strings such as **8.E-10**, because there must be at least one digit after the decimal point if an exponent is present.

(b) Here is a Pascal implementation in the same style as Figures 1.4–1.8.

```

program RealConst(input, output);
type
  Sigma = (digit, plus, minus, dot, emark, other);
  State = (start, sign, intpart, dotafterint, expmark,
           expsign, expdigits, dead, fracpart, leadingdot);
var

```

```

    TransitionTable : array [State, Sigma] of State;
    FinalState : set of State;

function ClassOf(c : char) : Sigma;
begin
    if c in ['0'..'9'] then
        ClassOf := digit
    else if c = '+' then
        ClassOf := plus
    else if c = '-' then
        ClassOf := minus
    else if c = '.' then
        ClassOf := dot
    else if c = 'E' then
        ClassOf := emark
    else
        ClassOf := other
end; { ClassOf }

function Delta(s : State; a : Sigma) : State;
begin
    Delta := TransitionTable[s, a]
end; { Delta }

function DeltaBar(s : State) : State;
var
    t : State;
begin
    t := s;
    while not eoln(input) do
        begin
            t := Delta(t, ClassOf(input^));
            get(input)
        end;
    DeltaBar := t
end; { DeltaBar }

function Accept : boolean;
begin
    Accept := DeltaBar(start) in FinalState
end; { Accept }

procedure Initialize;
var
    s : State;

```

```

    a : Sigma;
begin
    FinalState := [intpart, dotafterint, fracpart, expdigits];

    for s := start to leadingdot do
        for a := digit to other do
            TransitionTable[s, a] := dead;

        TransitionTable[start, digit] := intpart;
        TransitionTable[start, plus] := sign;
        TransitionTable[start, minus] := sign;
        TransitionTable[start, dot] := leadingdot;

        TransitionTable[sign, digit] := intpart;
        TransitionTable[sign, dot] := leadingdot;

        TransitionTable[intpart, digit] := intpart;
        TransitionTable[intpart, dot] := dotafterint;

        TransitionTable[dotafterint, digit] := fracpart;
        TransitionTable[leadingdot, digit] := fracpart;

        TransitionTable[fracpart, digit] := fracpart;
        TransitionTable[fracpart, emark] := expmark;

        TransitionTable[expmark, digit] := expdigits;
        TransitionTable[expmark, plus] := expsign;
        TransitionTable[expmark, minus] := expsign;

        TransitionTable[expsign, digit] := expdigits;

        TransitionTable[expdigits, digit] := expdigits
    end; { Initialize }

begin
    Initialize;
    if Accept then
        writeln(output, 'Accepted')
    else
        writeln(output, 'Rejected')
    end.
end.

```

1.20. **Show Definition 1.11(i) is implied by (ii) and (iii).**

In Definition 1.11, part (i) is the statement

$$(\forall s \in S)(\forall \mathbf{a} \in \Sigma) \bar{\delta}(s, \mathbf{a}) = \delta(s, \mathbf{a}).$$

Assume only parts (ii) and (iii). Then for any state s and any letter $\mathbf{a}$,

$$\bar{\delta}(s, \mathbf{a}) = \bar{\delta}(s, \mathbf{a}\lambda)$$

because $\mathbf{a}\lambda = \mathbf{a}$. Now apply part (iii) with $x = \lambda$:

$$\bar{\delta}(s, \mathbf{a}\lambda) = \bar{\delta}(\delta(s, \mathbf{a}), \lambda).$$

Finally, by part (ii),

$$\bar{\delta}(\delta(s, \mathbf{a}), \lambda) = \delta(s, \mathbf{a}).$$

Therefore $\bar{\delta}(s, \mathbf{a}) = \delta(s, \mathbf{a})$ for every $s \in S$ and every $\mathbf{a} \in \Sigma$, which is exactly part (i).

1.21. Develop a succinct description of the transition function from Example 1.9.

Let Γ be the set defined in Example 1.9, so that the alphabet is partitioned into letters, digits, blanks $\diamond$, and the remaining symbols Γ . Then the transition function can be described uniformly as follows:

$$\begin{aligned} \delta(s_0, \text{letter}) &= s_1, & \delta(s_0, \text{digit}) &= s_7, & \delta(s_0, \diamond) &= s_0, & \delta(s_0, \gamma) &= s_7 \quad (\gamma \in \Gamma), \\ \delta(s_i, \text{letter or digit}) &= s_{i+1} \quad (1 \leq i \leq 5), \\ \delta(s_i, \diamond) &= s_i \quad (1 \leq i \leq 6), \\ \delta(s_i, \gamma) &= s_7 \quad (1 \leq i \leq 6, \gamma \in \Gamma), \\ \delta(s_6, \text{letter or digit}) &= s_7, \\ \delta(s_7, \sigma) &= s_7 \quad (\sigma \in \Sigma). \end{aligned}$$

So for $1 \leq i \leq 6$, s_i means “a legal identifier of length i has been seen.” The state s_0 is the blank-start state, blanks do not change the current length, and any illegal character or seventh nonblank character forces the machine into the dead state s_7 .

1.22. Finite, cofinite, neither over $\{\mathbf{a}, \mathbf{b}\}^*$.

- (a) Finite: $\{\mathbf{a}, \mathbf{ab}\}$.
- (b) Cofinite: $\{\mathbf{a}, \mathbf{b}\}^* \setminus \{\mathbf{a}\}$ (complement is $\{\mathbf{a}\}$, finite).
- (c) Neither: $\{\mathbf{a}^n \mid n \text{ even}\}$. Infinite, and its complement $\{\mathbf{a}^n \mid n \text{ odd}\} \cup \{x \mid x \text{ contains } \mathbf{b}\}$ is also infinite.

1.23. DFA of Figure 1.27.

- (a) The quintuple is

$$\langle \{\mathbf{a}, \mathbf{b}\}, \{q_0, q_1, q_2, q_3, q_4\}, q_0, \delta, \{q_2, q_3\} \rangle,$$

where

$$\begin{aligned} \delta(q_0, \mathbf{a}) &= q_1, & \delta(q_0, \mathbf{b}) &= q_0, \\ \delta(q_1, \mathbf{a}) &= q_2, & \delta(q_1, \mathbf{b}) &= q_1, \\ \delta(q_2, \mathbf{a}) &= q_2, & \delta(q_2, \mathbf{b}) &= q_4, \\ \delta(q_3, \mathbf{a}) &= q_3, & \delta(q_3, \mathbf{b}) &= q_2, \\ \delta(q_4, \mathbf{a}) &= q_4, & \delta(q_4, \mathbf{b}) &= q_3. \end{aligned}$$

- (b) The machine first waits for two occurrences of **a**. The state q_2 is reached as soon as the second **a** is seen. After that, only the number of **b**s modulo 3 matters:

$$q_2 \equiv 0, \quad q_4 \equiv 1, \quad q_3 \equiv 2 \pmod{3}.$$

Since q_2 and q_3 are final and q_4 is not, the language is

$$\{b^i a b^j a x \mid i, j \geq 0, x \in \{a, b\}^*, |x|_b \equiv 0 \text{ or } 2 \pmod{3}\}.$$

Equivalently: the string must contain at least two **a**s, and after the second **a** the number of **b**s must not be congruent to 1 (mod 3).

1.24. Names of everyone in China – FAD?

The set is finite (China has a finite population), hence regular (every finite language is FAD). Yes.

1.25. Legal infix arithmetic expressions – FAD?

Yes. Without parentheses, normal precedence rules affect evaluation, not syntactic well-formedness. A DFA only has to remember whether the next symbol should be an operand or an operator. The symbol **–** is special because it can serve as unary negation while the machine is expecting an operand, and as subtraction while the machine is expecting an operator.

A suitable DFA has three states:

- q_E : expecting an operand,
- q_O : expecting an operator or the end of the string,
- q_D : dead state.

The start state is q_E , the only final state is q_O , and the transitions are

	A	B	+	–	×	/
q_E	q_O	q_O	q_D	q_E	q_D	q_D
q_O	q_D	q_D	q_E	q_E	q_E	q_E
q_D	q_D	q_D	q_D	q_D	q_D	q_D

Thus **A**, **–A**, **A – B**, and **A × –B** are accepted, while strings such as **+A**, **AB**, and **A/** are rejected.

1.26. Properties of a DFA M .

- (a) $\lambda \in L(M)$ iff $s_0 \in F$.
- (b) $L(M) = \Sigma^*$ iff every *reachable* state is a final state. (Unreachable non-final states do not affect the language: a DFA with one reachable final start state and one unreachable non-final state still accepts Σ^* .)
- (c) $L(M) = \emptyset$ iff no final state is reachable from s_0 .

1.27. DFAs for substring recognition.

- (a) For “**abc** is a substring,” use the states q_0, q_1, q_2, q_3 , where q_i means that the longest suffix seen so far that is also a prefix of **abc** has length i . Let q_3 be an accepting sink. The transition table is

	a	b	c
q_0	q_1	q_0	q_0
q_1	q_1	q_2	q_0
q_2	q_1	q_0	q_3
q_3	q_3	q_3	q_3

- (b) For “**acaba** is a substring,” use the states $p_0, p_1, p_2, p_3, p_4, p_5$, where p_5 is an accepting sink. The transition table is

	a	b	c
p_0	p_1	p_0	p_0
p_1	p_1	p_0	p_2
p_2	p_3	p_0	p_0
p_3	p_1	p_4	p_2
p_4	p_5	p_0	p_0
p_5	p_5	p_5	p_5

Again, p_i records the longest suffix that is also a prefix of **acaba**.

1.28. Add pennies to vending machine.

The original machine tracks deposited amounts in multiples of 5¢, needing 7 states (0, 5, 10, 15, 20, 25, 30¢). Adding pennies requires tracking every cent from 0 to 30, so the minimum number of states is **31**: states $q_0, q_1, \dots, q_{30}$ representing 0, 1, $\dots$, 30 cents deposited. State q_{30} is the accepting (dispensing) state; from q_{30} all additional input is ignored (or loops back). Each extra penny input on state q_i (for $i < 30$) transitions to q_{i+1} .

1.29. The DFA (t_0, t_1) of Exercise 1.29.

- (a) The language: $L = \{x \in \{\mathbf{a}, \mathbf{b}\}^* \mid |x|_{\mathbf{b}} \text{ is odd}\}$, i.e., strings with an odd number of **bs**. (t_0 = even #b, t_1 = odd #b, $F = \{t_1\}$.)

- (b) *Proof*: By induction on $|x| = n$. Let $P(n)$: $(\forall x \in \Sigma^n)(\bar{\delta}(t_0, x) = t_0 \Leftrightarrow |x|_{\mathbf{b}} \text{ even}) \wedge (\bar{\delta}(t_0, x) = t_1 \Leftrightarrow |x|_{\mathbf{b}} \text{ odd})$.

Base ($n = 0$): $\bar{\delta}(t_0, \lambda) = t_0$, $|\lambda|_{\mathbf{b}} = 0$ (even). ✓

Step: Let $x = ya$, $|y| = n$. Case $a = \mathbf{a}$: $\bar{\delta}(t_0, ya) = \delta(\bar{\delta}(t_0, y), \mathbf{a})$. Since $\delta(t_0, \mathbf{a}) = t_0$ and $\delta(t_1, \mathbf{a}) = t_1$, the state after ya equals the state after y , and $|ya|_{\mathbf{b}} = |y|_{\mathbf{b}}$. Case $a = \mathbf{b}$: δ flips the state; $|ya|_{\mathbf{b}} = |y|_{\mathbf{b}} + 1$ changes parity. Both cases follow from the IH. □

1.30. Vending machine: pennies, nickels, dimes, quarters, 10¢ bars.

- (a) States $q_0, q_1, \dots, q_{10}$ where $q_i = i$ cents deposited so far, q_{10} = accepting (dispensing) sink. $\delta(q_i, p) = q_{\min(i+1, 10)}$, $\delta(q_i, n) = q_{\min(i+5, 10)}$, $\delta(q_i, d) = q_{\min(i+10, 10)}$, $\delta(q_i, q) = q_{\min(i+25, 10)} \rightarrow q_{10}$. $F = \{q_{10}\}$.
- (b) 11 states.
- (c) The minimum is 11 states (must track exact amount from 0 to 10 cents).

1.31. Vending machine: nickels, dimes, quarters, 10¢ bars.

- (a) States q_0, q_5, q_{10} (representing 0, 5, 10+ cents). $\delta(q_0, n) = q_5, \delta(q_0, d) = \delta(q_5, n) = q_{10}, \delta(q_0, q) = \delta(q_5, d) = \delta(q_5, q) = q_{10}, \delta(q_{10}, -) = q_{10}$. $F = \{q_{10}\}$.
- (b) Minimum: 3 states (0, 5, ≥ 10 cents).
- (c) Encode the states by

$$q_0 = 00, \quad q_5 = 01, \quad q_{10} = 10,$$

with 11 unused. Encode the input alphabet $\{<EOS>, n, d, q\}$ by

$$<EOS> = 00, \quad n = 01, \quad d = 10, \quad q = 11.$$

Let the state bits be t_1, t_2 , and the input bits be a_1, a_2 . One corresponding circuit uses two D flip-flops with inputs

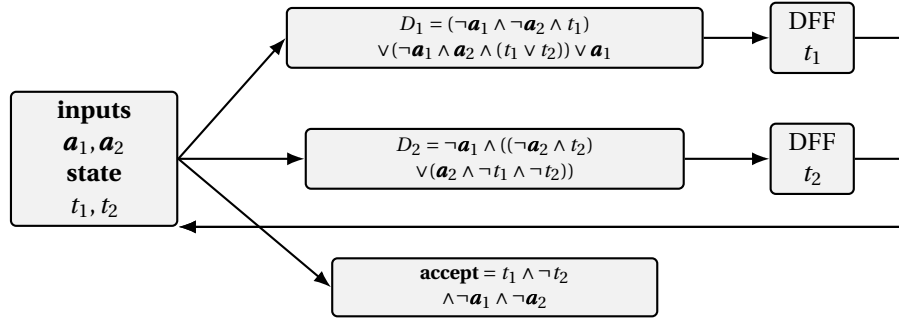
$$t_1' = (\neg a_1 \wedge \neg a_2 \wedge t_1) \vee (\neg a_1 \wedge a_2 \wedge (t_1 \vee t_2)) \vee a_1,$$

$$t_2' = \neg a_1 \wedge ((\neg a_2 \wedge t_2) \vee (a_2 \wedge \neg t_1 \wedge \neg t_2)).$$

The accept light should turn on exactly when $<EOS>$ is scanned in state q_{10} , so

$$\text{accept} = t_1 \wedge \neg t_2 \wedge \neg a_1 \wedge \neg a_2.$$

One corresponding circuit diagram is:



1.32. Circuit for Exercise 1.29 DFA.

- (a) Use one state bit t_1 , with $t_1 = 0$ for state t_0 and $t_1 = 1$ for state t_1 . Encode

$$<EOS> = 00, \quad a = 01, \quad b = 10, \quad <SOS> = 11.$$

Then $<EOS>$ and a leave the parity unchanged, b toggles it, and $<SOS>$ resets to the start state. Thus one D flip-flop with input

$$t_1' = (t_1 \wedge \neg a_1) \vee (\neg t_1 \wedge a_1 \wedge \neg a_2)$$

implements the next-state logic. The accept circuitry is

$$\text{accept} = t_1 \wedge \neg a_1 \wedge \neg a_2,$$

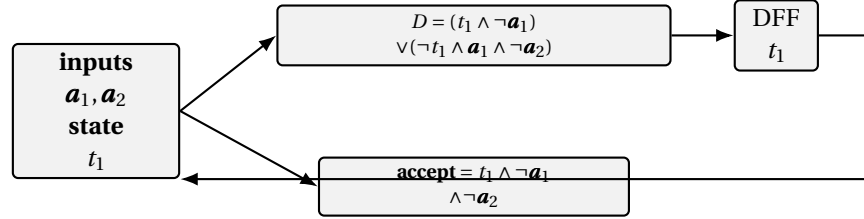
since the string is accepted exactly when $<EOS>$ is scanned in the odd- b state.

- (b) Without <SOS> and <EOS>, only one input bit is needed; let $x = 0$ represent a and $x = 1$ represent b . The next state is just

$$t_1' = t_1 \oplus x.$$

So the circuit is a single D flip-flop whose input is the exclusive-or of the current state bit and the input bit.

A corresponding circuit diagram for the version with <SOS> and <EOS> is:



1.33. Circuit for Exercise 1.7 DFA.

It is convenient to implement the language of Exercise 1.7 by keeping the parity bits from Example 1.10 together with one extra bit u that records whether at least one input symbol has been read. Let

$$p = \text{parity of } |x|_a, \quad q = \text{parity of } |x|_b, \quad u = \text{"have seen input"}.$$

Then the machine accepts exactly when $p = q = 0$ and $u = 1$.

- (a) With <SOS> and <EOS>, use the encoding

$$\langle \text{EOS} \rangle = 00, \quad a = 01, \quad b = 10, \quad \langle \text{SOS} \rangle = 11.$$

One corresponding circuit uses three D flip-flops with inputs

$$p' = (\neg a_1 \wedge \neg a_2 \wedge p) \vee (\neg a_1 \wedge a_2 \wedge \neg p) \vee (a_1 \wedge \neg a_2 \wedge p),$$

$$q' = (\neg a_1 \wedge \neg a_2 \wedge q) \vee (\neg a_1 \wedge a_2 \wedge \neg q) \vee (a_1 \wedge \neg a_2 \wedge q),$$

$$u' = (\neg a_1 \wedge \neg a_2 \wedge u) \vee (\neg a_1 \wedge a_2) \vee (a_1 \wedge \neg a_2).$$

The accept light is

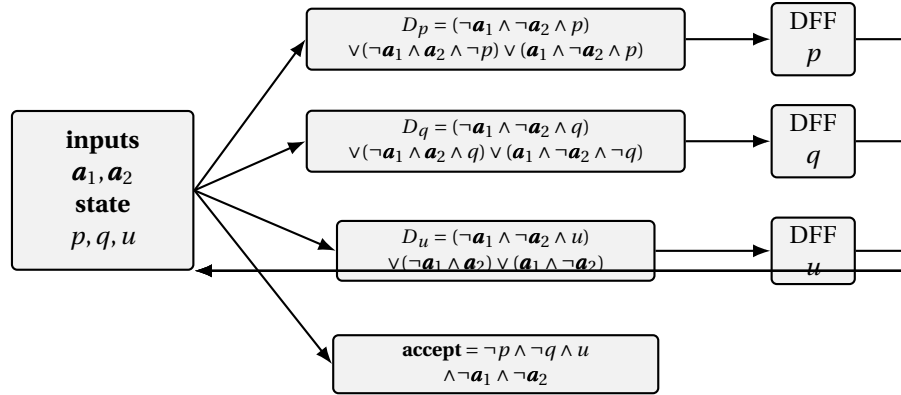
$$\text{accept} = \neg p \wedge \neg q \wedge u \wedge \neg a_1 \wedge \neg a_2.$$

- (b) Without <SOS> and <EOS>, use one input bit x with $x = 0$ for a and $x = 1$ for b . Then

$$p' = p \oplus \neg x, \quad q' = q \oplus x, \quad u' = 1.$$

After the last clock pulse, the machine accepts iff $\neg p \wedge \neg q \wedge u$.

One corresponding circuit diagram for the <SOS>/<EOS> version is:



1.34. **Modify Example 1.12 to correctly handle <SOS>.**

Use the same one-bit state encoding as Example 1.12: $s_0 = 0$ and $s_1 = 1$. Keep the book's input convention and encode

$$\langle \text{EOS} \rangle = 00, \quad b = 01, \quad c = 10, \quad \langle \text{SOS} \rangle = 11.$$

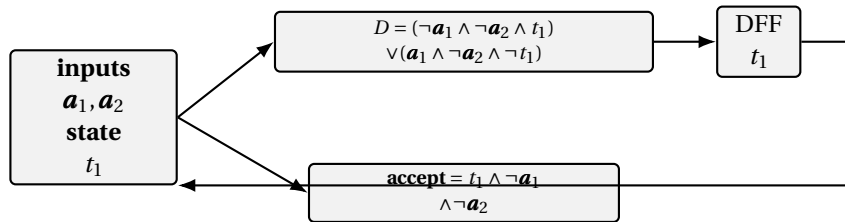
The only extra requirement is that scanning <SOS> from either state must send the machine to s_0 . The resulting next-state function is

$$t'_1 = (\neg a_1 \wedge \neg a_2 \wedge t_1) \vee (a_1 \wedge \neg a_2 \wedge \neg t_1).$$

This agrees with Example 1.12 on b , c , and <EOS>, and forces a reset to the start state on <SOS>. The accept circuitry is unchanged:

$$\text{accept} = t_1 \wedge \neg a_1 \wedge \neg a_2.$$

So the corrected circuit is exactly the Example 1.12 circuit with the input combination **11** wired to force the next state to **0**.



1.35. **Circuit for Example 1.6 machine.**

(a) Encode the states by

$$s_0 = 00, \quad s_1 = 01, \quad s_2 = 10,$$

with **11** unused. Use the standard input encoding

$$\langle \text{EOS} \rangle = 00, \quad a = 01, \quad b = 10, \quad \langle \text{SOS} \rangle = 11.$$

If t_1, t_2 are the state bits, then a corresponding pair of D inputs is

$$t'_1 = (\neg a_1 \wedge \neg a_2 \wedge t_1) \vee (a_1 \wedge \neg a_2 \wedge \neg t_1),$$

$$t'_2 = (\neg a_1 \wedge \neg a_2 \wedge t_2) \vee (\neg a_1 \wedge a_2).$$

The machine accepts in state s_1 , so the accept circuitry is

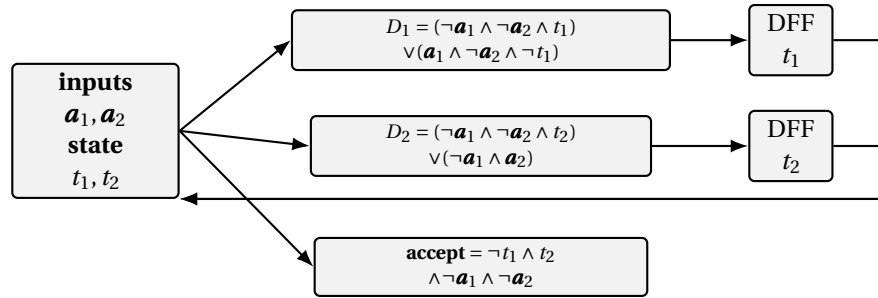
$$\text{accept} = \neg t_1 \wedge t_2 \wedge \neg a_1 \wedge \neg a_2.$$

- (b) Without <SOS> and <EOS>, only one input bit x is needed, with $x = 0$ for a and $x = 1$ for b . Then

$$t'_1 = x \wedge \neg t_1, \quad t'_2 = \neg x.$$

Thus two D flip-flops together with the above combinational logic implement the machine.

One corresponding circuit diagram for the <SOS>/<EOS> version is:



1.36. Circuit for Example 1.10 machine.

Let p and q be the parity bits for $|x|_a$ and $|x|_b$, respectively. Then the four states of Example 1.10 are exactly the four combinations of (p, q) .

- (a) With <SOS> and <EOS>, use

$$\langle \text{EOS} \rangle = 00, \quad a = 01, \quad b = 10, \quad \langle \text{SOS} \rangle = 11.$$

Then <EOS> leaves the state unchanged, a flips p , b flips q , and <SOS> resets both bits to 0. One corresponding circuit uses two D flip-flops with

$$p' = (\neg a_1 \wedge \neg a_2 \wedge p) \vee (\neg a_1 \wedge a_2 \wedge \neg p) \vee (a_1 \wedge \neg a_2 \wedge p),$$

$$q' = (\neg a_1 \wedge \neg a_2 \wedge q) \vee (\neg a_1 \wedge a_2 \wedge \neg q) \vee (a_1 \wedge \neg a_2 \wedge \neg q).$$

The accept light turns on exactly at <EOS> when $p = q = 0$:

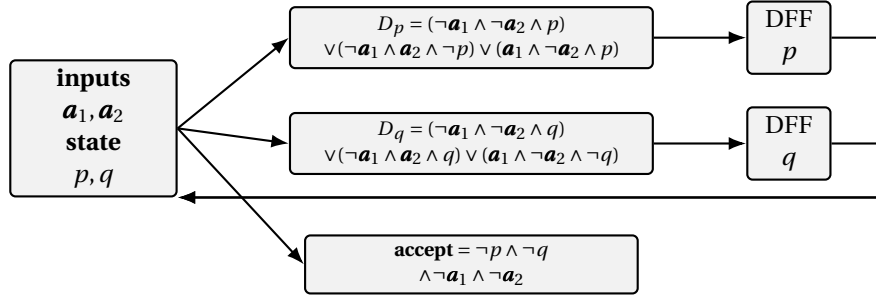
$$\text{accept} = \neg p \wedge \neg q \wedge \neg a_1 \wedge \neg a_2.$$

- (b) Without <SOS> and <EOS>, let $x = 0$ encode a and $x = 1$ encode b . Then

$$p' = p \oplus \neg x, \quad q' = q \oplus x.$$

So the circuit is just two D flip-flops fed by these two exclusive-or subcircuits.

One corresponding circuit diagram for the <SOS>/<EOS> version is:



1.37. Circuit for Example 1.14 (vending machine), with EOS.

Encode the seven states of Example 1.14 by the binary codes for the numbers 0 through 6:

$$s_0 = \mathbf{000}, s_5 = \mathbf{001}, s_{10} = \mathbf{010}, s_{15} = \mathbf{011}, s_{20} = \mathbf{100}, s_{25} = \mathbf{101}, s_{30} = \mathbf{110},$$

leaving **111** unused. Encode the input alphabet by

$$\langle \text{EOS} \rangle = \mathbf{00}, \quad n = \mathbf{01}, \quad d = \mathbf{10}, \quad q = \mathbf{11}.$$

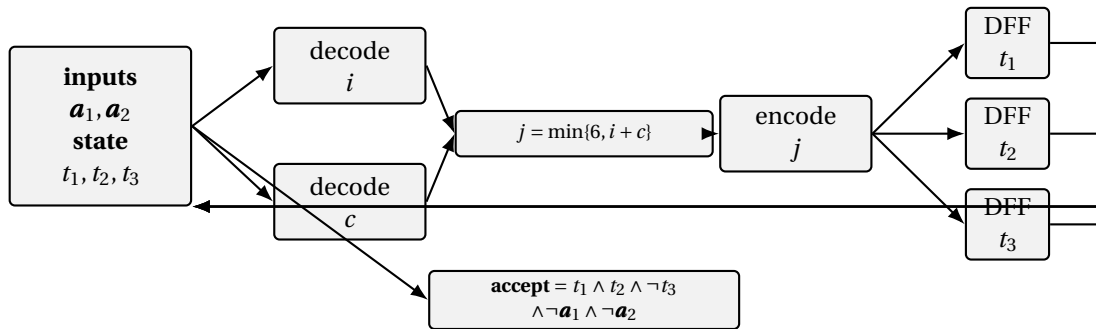
One clean circuit design uses three D flip-flops together with a combinational subcircuit that does the following:

- decode the current state bits as the number $i \in \{0, 1, \dots, 6\}$,
- decode the input bits as the increment c , where $c = 0$ for <EOS>, $c = 1$ for n , $c = 2$ for d , and $c = 5$ for q ,
- compute $j = \min\{6, i + c\}$,
- write the three-bit encoding of j into the D inputs.

The accept light is on exactly when <EOS> is scanned in state s_{30} :

$$\text{accept} = t_1 \wedge t_2 \wedge \neg t_3 \wedge \neg a_1 \wedge \neg a_2,$$

if $t_1 t_2 t_3$ is the chosen encoding of the current state and $s_{30} = \mathbf{110}$.



1.38. **Circuit for Example 1.16, with SOS and EOS.**

The protocol machine has four states, so two state bits suffice. For example, use

$$A = 00, \quad R = 01, \quad RF = 10, \quad RD = 11.$$

The input alphabet is $\{B, D, Z, H, S, \langle EOS \rangle, \langle SOS \rangle\}$, so use three input bits and encode

$$\langle EOS \rangle = 000, \quad B = 001, \quad D = 010, \quad H = 011,$$

$$S = 100, \quad Z = 101, \quad \langle SOS \rangle = 111,$$

leaving **110** unused.

The next-state combinational block should implement exactly the protocol described in Example 1.16:

- scanning $\langle SOS \rangle$ from any state sends the machine to R ,
- scanning $\langle EOS \rangle$ leaves the current state unchanged,
- from R , only S is legal and sends the machine to RF ,
- from RF , only H is legal and sends the machine to RD ,
- from RD , D stays in RD , Z goes to RF , and B goes to R ,
- every unexpected packet sends the machine to A ,
- from A , every packet except $\langle SOS \rangle$ leaves the machine in A .

Let e, b, d, h, s, z, r denote the decoded input lines for $\langle EOS \rangle, B, D, H, S, Z$, and $\langle SOS \rangle$, respectively. Let

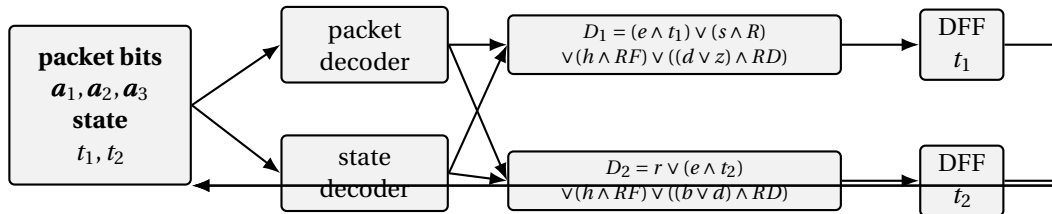
$$R = \neg t_1 \wedge t_2, \quad RF = t_1 \wedge \neg t_2, \quad RD = t_1 \wedge t_2$$

be the decoded current-state lines. One convenient choice of D inputs is then

$$D_1 = (e \wedge t_1) \vee (s \wedge R) \vee (h \wedge RF) \vee ((d \vee z) \wedge RD),$$

$$D_2 = r \vee (e \wedge t_2) \vee (h \wedge RF) \vee ((b \vee d) \wedge RD).$$

Since Example 1.16 is being used as a control automaton rather than as a language acceptor, there is no separate accept light; the outputs of interest are simply the state bits themselves.



1.39. **DFA for $L = \{x \in \{a, b, c\}^* \mid |x|_b = 2\}$.**

- (a) Three states q_0, q_1, q_2 (counting $\#b$ seen so far, capped at 2), plus a dead state q_3 . $F = \{q_2\}$.
 $\delta(q_i, a) = \delta(q_i, c) = q_i$; $\delta(q_0, b) = q_1$; $\delta(q_1, b) = q_2$; $\delta(q_2, b) = q_3$; $\delta(q_3, -) = q_3$.

(b) The quintuple is

$$\langle \{\mathbf{a}, \mathbf{b}, \mathbf{c}\}, \{q_0, q_1, q_2, q_3\}, q_0, \delta, \{q_2\} \rangle,$$

where

$$\delta(q_0, \mathbf{a}) = q_0, \quad \delta(q_0, \mathbf{b}) = q_1, \quad \delta(q_0, \mathbf{c}) = q_0,$$

$$\delta(q_1, \mathbf{a}) = q_1, \quad \delta(q_1, \mathbf{b}) = q_2, \quad \delta(q_1, \mathbf{c}) = q_1,$$

$$\delta(q_2, \mathbf{a}) = q_2, \quad \delta(q_2, \mathbf{b}) = q_3, \quad \delta(q_2, \mathbf{c}) = q_2,$$

$$\delta(q_3, \mathbf{a}) = q_3, \quad \delta(q_3, \mathbf{b}) = q_3, \quad \delta(q_3, \mathbf{c}) = q_3.$$

1.40. **DFA for every b eventually followed by c.**

Two states track whether there is an outstanding **b** with no subsequent **c**: q_0 (no unresolved **b**), q_b (at least one unresolved **b**). $F = \{q_0\}$.

$$\delta(q_0, \mathbf{a}) = q_0, \quad \delta(q_0, \mathbf{b}) = q_b, \quad \delta(q_0, \mathbf{c}) = q_0,$$

$$\delta(q_b, \mathbf{a}) = q_b, \quad \delta(q_b, \mathbf{b}) = q_b, \quad \delta(q_b, \mathbf{c}) = q_0.$$

Reading **c** in q_b resolves all pending **bs**; **a** or **b** keeps them pending. Accept iff input ends in q_0 .

1.41. **DFA for no consecutive as and no consecutive bs.**

(a) States: q_0 (start/even alternation), q_a (last char **a**), q_b (last char **b**), q_{dead} . $F = \{q_0, q_a, q_b\}$.
 $\delta(q_0, \mathbf{a}) = q_a$, $\delta(q_0, \mathbf{b}) = q_b$, $\delta(q_a, \mathbf{a}) = q_{dead}$, $\delta(q_a, \mathbf{b}) = q_b$, $\delta(q_b, \mathbf{a}) = q_a$, $\delta(q_b, \mathbf{b}) = q_{dead}$, $\delta(q_{dead}, -) = q_{dead}$.

(b) The quintuple is

$$\langle \{\mathbf{a}, \mathbf{b}\}, \{q_0, q_a, q_b, q_{dead}\}, q_0, \delta, \{q_0, q_a, q_b\} \rangle,$$

where

$$\delta(q_0, \mathbf{a}) = q_a, \quad \delta(q_0, \mathbf{b}) = q_b,$$

$$\delta(q_a, \mathbf{a}) = q_{dead}, \quad \delta(q_a, \mathbf{b}) = q_b,$$

$$\delta(q_b, \mathbf{a}) = q_a, \quad \delta(q_b, \mathbf{b}) = q_{dead},$$

$$\delta(q_{dead}, \mathbf{a}) = q_{dead}, \quad \delta(q_{dead}, \mathbf{b}) = q_{dead}.$$

1.42. **DFA for $L = \{x \in \{\mathbf{a}, \mathbf{b}, \mathbf{c}\}^* \mid |x|_{\mathbf{a}} \equiv 0 \pmod{3}\}$.**

(a) Use three states r_0, r_1, r_2 , where r_i means that the number of **a**s seen so far is congruent to $i \pmod{3}$. The start state is r_0 , the only final state is r_0 , and

$$\delta(r_i, \mathbf{a}) = r_{i+1 \pmod{3}}, \quad \delta(r_i, \mathbf{b}) = r_i, \quad \delta(r_i, \mathbf{c}) = r_i.$$

(b) The quintuple is

$$\langle \{\mathbf{a}, \mathbf{b}, \mathbf{c}\}, \{r_0, r_1, r_2\}, r_0, \delta, \{r_0\} \rangle,$$

where

$$\delta(r_0, \mathbf{a}) = r_1, \quad \delta(r_0, \mathbf{b}) = r_0, \quad \delta(r_0, \mathbf{c}) = r_0,$$

$$\delta(r_1, \mathbf{a}) = r_2, \quad \delta(r_1, \mathbf{b}) = r_1, \quad \delta(r_1, \mathbf{c}) = r_1,$$

$$\delta(r_2, \mathbf{a}) = r_0, \quad \delta(r_2, \mathbf{b}) = r_2, \quad \delta(r_2, \mathbf{c}) = r_2.$$

1.43. DFAs for Pascal comments.

- (a) For “exactly one valid Pascal comment and no illegal comment fragments,” use

$$A = \langle \Sigma, S, P_0, \delta, F \rangle,$$

where

$$\Sigma = \{a, b, (, *,)\},$$

$$S = \{P_0, P_(), P_*, I_0, I_*, Q_0, Q_(), Q_*, D\}, \quad F = \{Q_0, Q_(), Q_*\}.$$

Here P means “before the unique comment,” I means “inside the comment,” Q means “after the unique comment,” and D is dead. The full transition function is:

	<i>a</i>	<i>b</i>	(	*	)
P_0	P_0	P_0	$P_()$	P_*	P_0
$P_()$	P_0	P_0	$P_()$	I_0	P_0
P_*	P_0	P_0	$P_()$	P_*	D
I_0	I_0	I_0	I_0	I_*	I_0
I_*	I_0	I_0	I_0	I_*	Q_0
Q_0	Q_0	Q_0	$Q_()$	Q_*	Q_0
$Q_()$	Q_0	Q_0	$Q_()$	D	Q_0
Q_*	Q_0	Q_0	$Q_()$	Q_*	D
D	D	D	D	D	D

- (b) For “zero or more valid unnested Pascal comments,” use

$$B = \langle \Sigma, T, O_0, \delta, F' \rangle,$$

where

$$T = \{O_0, O_(), O_*, I_0, I_*, D\}, \quad F' = \{O_0, O_(), O_*\}.$$

Here O means “outside any comment.” The full transition function is:

	<i>a</i>	<i>b</i>	(	*	)
O_0	O_0	O_0	$O_()$	O_*	O_0
$O_()$	O_0	O_0	$O_()$	I_0	O_0
O_*	O_0	O_0	$O_()$	O_*	D
I_0	I_0	I_0	I_0	I_*	I_0
I_*	I_0	I_0	I_0	I_*	O_0
D	D	D	D	D	D

This machine accepts exactly the strings that end outside a comment and never produce an illegal delimiter fragment.

1.44. Postfix expressions.

- (a) Yes. A convenient DFA for the case “at most 2 operators” uses states

$$q_{d,m} \quad (0 \leq d \leq 3, 0 \leq m \leq 2),$$

together with a dead state q_{dead} . Here d is the current stack surplus

$$d = (\# \text{operands read}) - (\# \text{operators read}),$$

and m is the number of operators used so far. The start state is $q_{0,0}$. On an operand A or B ,

$$\delta(q_{d,m}, A) = \delta(q_{d,m}, B) = \begin{cases} q_{d+1,m} & d < 3, \\ q_{dead} & d = 3, \end{cases}$$

and on any operator $\odot \in \{+, -, \times, /\}$,

$$\delta(q_{d,m}, \odot) = \begin{cases} q_{d-1,m+1} & d \geq 2 \text{ and } m < 2, \\ q_{dead} & \text{otherwise.} \end{cases}$$

The final states are the states $q_{1,0}, q_{1,1}, q_{1,2}$, because a complete postfix expression leaves exactly one value on the stack. This is the required machine.

- (b) For four or fewer operators, use exactly the same construction with

$$q_{d,m} \quad (0 \leq d \leq 5, 0 \leq m \leq 4),$$

again plus a dead state. The same transition rules apply, with the bounds 3 and 2 replaced by 5 and 4, respectively. The final states are all $q_{1,m}$ with $0 \leq m \leq 4$.

- (c) For eight or fewer operators, use the same DFA schema once more, now with

$$q_{d,m} \quad (0 \leq d \leq 9, 0 \leq m \leq 8),$$

plus a dead state. On operands the machine increments d ; on operators it checks that $d \geq 2$ and then decrements d while incrementing m . The final states are all $q_{1,m}$ with $0 \leq m \leq 8$.

- (d) The set of *all* postfix expressions is not FAD. Intuitively, a recognizer must keep track of how many partial results are currently on the stack, and that number is unbounded. More formally, the strings

$$A, AA, AAA, \dots, A^n, \dots$$

have pairwise different futures: from A^n , the continuation $+^{n-1}$ completes a legal postfix expression, but the same continuation does not complete a legal expression from A^m when $m \neq n$. Therefore infinitely many distinct states would be required.

1.45. DFA for words that begin and end with different letters.

- (a) Use one start state s , and for each ordered pair $(u, v) \in \{a, b, c\}^2$ a state q_{uv} meaning “the first letter seen was u and the current last letter is v .” From the start state,

$$\delta(s, a) = q_{aa}, \quad \delta(s, b) = q_{bb}, \quad \delta(s, c) = q_{cc}.$$

Thereafter,

$$\delta(q_{uv}, w) = q_{uw} \quad \text{for all } u, v, w \in \{a, b, c\}.$$

The accepting states are precisely the six states q_{uv} with $u \neq v$.

(b) The quintuple is

$$\langle \{\mathbf{a}, \mathbf{b}, \mathbf{c}\}, S, s, \delta, F \rangle,$$

where

$$S = \{s\} \cup \{q_{uv} \mid u, v \in \{\mathbf{a}, \mathbf{b}, \mathbf{c}\}\},$$

and

$$F = \{q_{uv} \mid u \neq v\}.$$

The transition function is

$$\delta(s, \mathbf{a}) = q_{aa}, \quad \delta(s, \mathbf{b}) = q_{bb}, \quad \delta(s, \mathbf{c}) = q_{cc},$$

and for every $u, v \in \{\mathbf{a}, \mathbf{b}, \mathbf{c}\}$,

$$\delta(q_{uv}, \mathbf{a}) = q_{ua}, \quad \delta(q_{uv}, \mathbf{b}) = q_{ub}, \quad \delta(q_{uv}, \mathbf{c}) = q_{uc}.$$

1.46. DFA that rejects words whose last two letters match.

(a) A convenient DFA uses the seven states

$$s, a, b, c, aa, bb, cc.$$

The start state is s . The states a, b, c mean that the current last letter is $\mathbf{a}, \mathbf{b}, \mathbf{c}$ and the last two letters do *not* match; the states aa, bb, cc mean that the last two letters are equal. The transitions are:

$$\delta(s, \mathbf{a}) = a, \quad \delta(s, \mathbf{b}) = b, \quad \delta(s, \mathbf{c}) = c,$$

$$\delta(a, \mathbf{a}) = aa, \quad \delta(a, \mathbf{b}) = b, \quad \delta(a, \mathbf{c}) = c,$$

$$\delta(b, \mathbf{a}) = a, \quad \delta(b, \mathbf{b}) = bb, \quad \delta(b, \mathbf{c}) = c,$$

$$\delta(c, \mathbf{a}) = a, \quad \delta(c, \mathbf{b}) = b, \quad \delta(c, \mathbf{c}) = cc,$$

and from aa, bb, cc the same rule applies: a repeated last letter stays in the corresponding double-letter state, while a different last letter moves to a, b , or c . The accepting states are s, a, b, c .

(b) The quintuple is

$$\langle \{\mathbf{a}, \mathbf{b}, \mathbf{c}\}, \{s, a, b, c, aa, bb, cc\}, s, \delta, \{s, a, b, c\} \rangle,$$

where

$$\delta(s, \mathbf{a}) = a, \quad \delta(s, \mathbf{b}) = b, \quad \delta(s, \mathbf{c}) = c,$$

$$\delta(a, \mathbf{a}) = aa, \quad \delta(a, \mathbf{b}) = b, \quad \delta(a, \mathbf{c}) = c,$$

$$\delta(b, \mathbf{a}) = a, \quad \delta(b, \mathbf{b}) = bb, \quad \delta(b, \mathbf{c}) = c,$$

$$\delta(c, \mathbf{a}) = a, \quad \delta(c, \mathbf{b}) = b, \quad \delta(c, \mathbf{c}) = cc,$$

$$\delta(aa, \mathbf{a}) = aa, \quad \delta(aa, \mathbf{b}) = b, \quad \delta(aa, \mathbf{c}) = c,$$

$$\delta(bb, \mathbf{a}) = a, \quad \delta(bb, \mathbf{b}) = bb, \quad \delta(bb, \mathbf{c}) = c,$$

$$\delta(cc, \mathbf{a}) = a, \quad \delta(cc, \mathbf{b}) = b, \quad \delta(cc, \mathbf{c}) = cc.$$

1.47. **DFA that rejects words whose first two letters match.**

(a) Use the six states

$$s, a, b, c, q_{acc}, q_{rej}.$$

After the first input symbol, the machine remembers that symbol in one of the states a, b, c .
On the second symbol it decides permanently:

$$\delta(a, \mathbf{a}) = q_{rej}, \quad \delta(a, \mathbf{b}) = \delta(a, \mathbf{c}) = q_{acc},$$

and similarly for b and c . Both q_{acc} and q_{rej} are sink states. The accepting states are s, a, b, c, q_{acc} , since words of length 0 or 1 do not have matching first two letters.

(b) The quintuple is

$$\langle \{\mathbf{a}, \mathbf{b}, \mathbf{c}\}, \{s, a, b, c, q_{acc}, q_{rej}\}, s, \delta, \{s, a, b, c, q_{acc}\} \rangle,$$

where

$$\begin{aligned} \delta(s, \mathbf{a}) &= a, & \delta(s, \mathbf{b}) &= b, & \delta(s, \mathbf{c}) &= c, \\ \delta(a, \mathbf{a}) &= q_{rej}, & \delta(a, \mathbf{b}) &= q_{acc}, & \delta(a, \mathbf{c}) &= q_{acc}, \\ \delta(b, \mathbf{a}) &= q_{acc}, & \delta(b, \mathbf{b}) &= q_{rej}, & \delta(b, \mathbf{c}) &= q_{acc}, \\ \delta(c, \mathbf{a}) &= q_{acc}, & \delta(c, \mathbf{b}) &= q_{acc}, & \delta(c, \mathbf{c}) &= q_{rej}, \end{aligned}$$

and

$$\delta(q_{acc}, x) = q_{acc}, \quad \delta(q_{rej}, x) = q_{rej}$$

for every $x \in \{\mathbf{a}, \mathbf{b}, \mathbf{c}\}$.

1.48. **Prove that the empty word is unique.**

Suppose x and y are both empty words. By definition, this means $|x| = 0$ and $|y| = 0$. The definition of equality of strings says that two strings are equal iff they have the same length and agree in every position. Here the lengths are equal, and there are no positions at all to check because the common length is 0. Thus the positional condition is vacuously satisfied, so $x = y$.

1.49. **Show that $|xy| = |x| + |y|$.**

We prove this by induction on $|y|$.

Base case: If $|y| = 0$, then $y = \lambda$, so

$$|xy| = |x\lambda| = |x| = |x| + 0 = |x| + |y|.$$

Inductive step: Assume that $|xz| = |x| + |z|$ whenever $|z| = n$. Let $|y| = n + 1$, so $y = \mathbf{a}z$ for some letter $\mathbf{a}$ and some string z of length n . Then

$$|xy| = |x\mathbf{a}z| = |(x\mathbf{a})z|.$$

By the induction hypothesis,

$$|(x\mathbf{a})z| = |x\mathbf{a}| + |z| = (|x| + 1) + |z| = |x| + (1 + |z|) = |x| + |y|.$$

Hence $|xy| = |x| + |y|$ for all strings x and y .

1.50. The DFA C.

- (a) The state transition diagram has two states, t_0 and t_1 , with t_0 as start state and t_1 final. The transitions are

$$\delta(t_0, \mathbf{a}) = t_0, \quad \delta(t_0, \mathbf{b}) = t_1, \quad \delta(t_0, \mathbf{c}) = t_1,$$

$$\delta(t_1, \mathbf{a}) = t_0, \quad \delta(t_1, \mathbf{b}) = t_1, \quad \delta(t_1, \mathbf{c}) = t_0.$$

- (b) The letter $\mathbf{a}$ always resets the machine to t_0 . After the last $\mathbf{a}$, the state depends only on the remaining suffix over $\{\mathbf{b}, \mathbf{c}\}$. Starting from t_0 , a $\mathbf{b}$ moves to t_1 and then further $\mathbf{b}$ s leave the machine there, while each $\mathbf{c}$ toggles the state. Therefore $L(C)$ consists exactly of those strings for which, after the last $\mathbf{a}$, either

- there is no $\mathbf{b}$ and the number of $\mathbf{c}$ s is odd, or
- there is a last $\mathbf{b}$ and it is followed by an even number of $\mathbf{c}$ s.

- (c) Use one state bit t , with $t = 0$ for t_0 and $t = 1$ for t_1 . Encode

$$\langle \text{EOS} \rangle = 00, \quad \mathbf{a} = 01, \quad \mathbf{b} = 10, \quad \mathbf{c} = 11.$$

Then $\langle \text{EOS} \rangle$ leaves the state unchanged, $\mathbf{a}$ sends the machine to t_0 , $\mathbf{b}$ sends it to t_1 , and $\mathbf{c}$ toggles the state. So one D flip-flop with input

$$t' = (\neg t \wedge \mathbf{a}_1) \vee (t \wedge \neg \mathbf{a}_2)$$

implements the next-state logic. The accept circuitry is

$$\text{accept} = t \wedge \neg \mathbf{a}_1 \wedge \neg \mathbf{a}_2.$$

1.51. Roman numerals.

- (a) For strict-order Roman numerals, a DFA is

$$\langle \{\mathbf{I}, \mathbf{V}, \mathbf{X}, \mathbf{L}, \mathbf{C}, \mathbf{D}, \mathbf{M}\}, \{q_s, q_M, q_D, q_C, q_L, q_X, q_V, q_I, q_{dead}\}, q_s, \delta, F \rangle,$$

where

$$F = \{q_M, q_D, q_C, q_L, q_X, q_V, q_I\}.$$

The states represent the highest-valued symbol class that may still appear. The transitions are:

$$\delta(q_s, \mathbf{M}) = q_M,$$

$$\delta(q_s, \mathbf{D}) = q_D, \quad \delta(q_s, \mathbf{C}) = q_C,$$

$$\delta(q_s, \mathbf{L}) = q_L,$$

$$\delta(q_s, \mathbf{X}) = q_X, \quad \delta(q_s, \mathbf{V}) = q_V, \quad \delta(q_s, \mathbf{I}) = q_I,$$

$$\delta(q_M, \mathbf{M}) = q_M,$$

$$\delta(q_M, \mathbf{D}) = q_D, \quad \delta(q_M, \mathbf{C}) = q_C,$$

$$\delta(q_M, \mathbf{L}) = q_L,$$

$$\delta(q_M, \mathbf{X}) = q_X, \quad \delta(q_M, \mathbf{V}) = q_V, \quad \delta(q_M, \mathbf{I}) = q_I,$$

$$\delta(q_D, \mathbf{C}) = q_C,$$

$$\delta(q_D, \mathbf{L}) = q_L, \quad \delta(q_D, \mathbf{X}) = q_X, \quad \delta(q_D, \mathbf{V}) = q_V, \quad \delta(q_D, \mathbf{I}) = q_I,$$

$$\delta(q_C, \mathbf{C}) = q_C,$$

$$\delta(q_C, \mathbf{L}) = q_L, \quad \delta(q_C, \mathbf{X}) = q_X, \quad \delta(q_C, \mathbf{V}) = q_V, \quad \delta(q_C, \mathbf{I}) = q_I,$$

$$\delta(q_L, \mathbf{X}) = q_X,$$

$$\delta(q_L, \mathbf{V}) = q_V, \quad \delta(q_L, \mathbf{I}) = q_I,$$

$$\delta(q_X, \mathbf{X}) = q_X,$$

$$\delta(q_X, \mathbf{V}) = q_V, \quad \delta(q_X, \mathbf{I}) = q_I,$$

$$\delta(q_V, \mathbf{I}) = q_I, \quad \delta(q_I, \mathbf{I}) = q_I.$$

Every unspecified transition goes to q_{dead} , and q_{dead} loops to itself. This accepts exactly the strings of the form

$$M^* D^{0,1} C^* L^{0,1} X^* V^{0,1} I^*,$$

excluding the empty word.

(b) For ordinary Roman numerals, one explicit DFA is

$$A = \langle \Sigma, S, q_s, \delta, F \rangle,$$

where

$$\Sigma = \{I, V, X, L, C, D, M\},$$

$$S = \{q_s, q_M, q_T, q_{HC}, q_{HCC}, q_{HCCC}, q_{HD}, q_{HDC}, q_{HDCC}, q_{HDCCC},$$

$$q_{TX}, q_{TXX}, q_{TXXX}, q_{TL}, q_{TLX}, q_{TLXX}, q_{TLXXX}, q_O, q_{OI}, q_{OII}, q_{OIII}, q_{OV}, q_{OVI}, q_{OVII}, q_{OVIII}, q_{done}, q_{dead}\},$$

$$F = S - \{q_s, q_{dead}\}.$$

The state names indicate the phase that has been reached: q_T means that the thousands and hundreds phases are complete and the tens phase has not yet begun, while q_O means that

the tens phase is complete and the ones phase has not yet begun. The transition function is:

$$\begin{array}{lll}
\delta(q_s, \mathbf{M}) = q_M, & \delta(q_s, \mathbf{C}) = q_{HC}, & \delta(q_s, \mathbf{D}) = q_{HDC}, \\
\delta(q_s, \mathbf{X}) = q_{TX}, & \delta(q_s, \mathbf{L}) = q_{TL}, & \delta(q_s, \mathbf{V}) = q_{OV}, \\
\delta(q_M, \mathbf{M}) = q_M, & \delta(q_M, \mathbf{C}) = q_{HC}, & \delta(q_M, \mathbf{D}) = q_{HDC}, \\
\delta(q_M, \mathbf{X}) = q_{TX}, & \delta(q_M, \mathbf{L}) = q_{TL}, & \delta(q_M, \mathbf{V}) = q_{OV}, \\
\delta(q_{HC}, \mathbf{M}) = q_T, & \delta(q_{HC}, \mathbf{D}) = q_T, & \delta(q_{HC}, \mathbf{L}) = q_{TL}, \\
\delta(q_{HC}, \mathbf{X}) = q_{TX}, & \delta(q_{HC}, \mathbf{L}) = q_{TL}, & \delta(q_{HC}, \mathbf{V}) = q_{OV}, \\
\delta(q_{HCC}, \mathbf{C}) = q_{HCCC}, & \delta(r, \mathbf{X}) = q_{TX}, & \delta(r, \mathbf{V}) = q_{OV}, \\
\delta(r, \mathbf{I}) = q_{OI}, & & \\
\delta(q_{HD}, \mathbf{C}) = q_{HDC}, & \delta(q_{HDC}, \mathbf{C}) = q_{HDCC}, & \delta(q_{HDCC}, \mathbf{C}) = q_{HDCCC}, \\
\delta(q_T, \mathbf{X}) = q_{TX}, & \delta(q_T, \mathbf{L}) = q_{TL}, & \delta(q_T, \mathbf{V}) = q_{OV}, \\
\delta(q_{TX}, \mathbf{C}) = q_O, & \delta(q_{TX}, \mathbf{L}) = q_O, & \delta(q_{TX}, \mathbf{V}) = q_{OV}, \\
\delta(q_{TX}, \mathbf{I}) = q_{OI}, & \delta(q_{TX}, \mathbf{V}) = q_{OV}, & \\
\delta(q_{TXX}, \mathbf{X}) = q_{TXXX}, & \delta(q_{TXX}, \mathbf{I}) = q_{OI}, & \delta(q_{TXX}, \mathbf{V}) = q_{OV}, \\
\delta(q_{TXXX}, \mathbf{I}) = q_{OI}, & \delta(q_{TXXX}, \mathbf{V}) = q_{OV}, & \\
\delta(q_{TL}, \mathbf{X}) = q_{TLX}, & \delta(q_{TL}, \mathbf{I}) = q_{OI}, & \delta(q_{TL}, \mathbf{V}) = q_{OV}, \\
\delta(q_{TLX}, \mathbf{X}) = q_{TLXX}, & \delta(q_{TLX}, \mathbf{I}) = q_{OI}, & \delta(q_{TLX}, \mathbf{V}) = q_{OV}, \\
\delta(q_{TLXX}, \mathbf{X}) = q_{TLXXX}, & \delta(q_{TLXX}, \mathbf{I}) = q_{OI}, & \delta(q_{TLXX}, \mathbf{V}) = q_{OV}, \\
\delta(q_{TLXXX}, \mathbf{I}) = q_{OI}, & \delta(q_{TLXXX}, \mathbf{V}) = q_{OV}, & \\
\delta(q_O, \mathbf{I}) = q_{OI}, & \delta(q_O, \mathbf{V}) = q_{OV}, & \\
\delta(q_{OI}, \mathbf{I}) = q_{OII}, & \delta(q_{OI}, \mathbf{V}) = q_{done}, & \delta(q_{OI}, \mathbf{V}) = q_{done}, \\
\delta(q_{OII}, \mathbf{I}) = q_{OIII}, & & \\
\delta(q_{OV}, \mathbf{I}) = q_{OVI}, & \delta(q_{OVI}, \mathbf{I}) = q_{OVII}, & \delta(q_{OVII}, \mathbf{I}) = q_{OVIII}.
\end{array}$$

For every state $q \in S$ and every input letter not covered by the equations above,

$$\delta(q, \sigma) = q_{dead},$$

and q_{dead} loops to itself on every input letter. This machine accepts exactly the standard Roman numerals.

- (c) The following Pascal program recognizes exactly the language of part (b), using a transition table and a clean δ function.

```

program Roman(input, output);
type
  Sigma = (symI, symV, symX, symL, symC, symD, symM, other);
  State = (qs, qM, qT, qHC, qHCC, qHCCC, qHD, qHDC, qHDCC, qHDCCC,

```

```

        qTX, qTXX, qTXXX, qTL, qTLX, qTLXX, qTLXXX,
        qOI, qOII, qOIII, qOV, qOVI, qOVII, qOVIII, qO, qdone, qdead);
var
    TransitionTable : array [State, Sigma] of State;
    FinalState : set of State;

function ClassOf(c : char) : Sigma;
begin
    case c of
        'I': ClassOf := symI;
        'V': ClassOf := symV;
        'X': ClassOf := symX;
        'L': ClassOf := symL;
        'C': ClassOf := symC;
        'D': ClassOf := symD;
        'M': ClassOf := symM
    else
        ClassOf := other
    end
end; { ClassOf }

function Delta(s : State; a : Sigma) : State;
begin
    Delta := TransitionTable[s, a]
end; { Delta }

function DeltaBar(s : State) : State;
var
    t : State;
begin
    t := s;
    while not eoln(input) do
        begin
            t := Delta(t, ClassOf(input^));
            get(input)
        end;
    DeltaBar := t
end; { DeltaBar }

function Accept : boolean;
begin
    Accept := DeltaBar(qs) in FinalState
end; { Accept }

procedure Initialize;

```

```

var
  s : State;
  a : Sigma;
begin
  FinalState := [qM, qT, qHC, qHCC, qHCCC, qHD, qHDC, qHDCC, qHDCCC,
                 qTX, qTXX, qTXXX, qTL, qTLX, qTLXX, qTLXXX,
                 qOI, qOII, qOIII, qOV, qOVI, qOVII, qOVIII, qO, qdone];

  for s := qs to qdead do
    for a := symI to other do
      TransitionTable[s, a] := qdead;

  for a := symI to other do
    TransitionTable[qdead, a] := qdead;

  TransitionTable[qs, symM] := qM;
  TransitionTable[qM, symM] := qM;

  TransitionTable[qs, symC] := qHC;
  TransitionTable[qM, symC] := qHC;
  TransitionTable[qs, symD] := qHD;
  TransitionTable[qM, symD] := qHD;
  TransitionTable[qs, symX] := qTX;
  TransitionTable[qM, symX] := qTX;
  TransitionTable[qs, symL] := qTL;
  TransitionTable[qM, symL] := qTL;
  TransitionTable[qs, symI] := qOI;
  TransitionTable[qM, symI] := qOI;
  TransitionTable[qs, symV] := qOV;
  TransitionTable[qM, symV] := qOV;

  TransitionTable[qHC, symM] := qT;
  TransitionTable[qHC, symD] := qT;
  TransitionTable[qHC, symC] := qHCC;
  TransitionTable[qHC, symX] := qTX;
  TransitionTable[qHC, symL] := qTL;
  TransitionTable[qHC, symI] := qOI;
  TransitionTable[qHC, symV] := qOV;

  TransitionTable[qHCC, symC] := qHCCC;
  TransitionTable[qHD, symC] := qHDC;
  TransitionTable[qHDC, symC] := qHDCC;
  TransitionTable[qHDCC, symC] := qHDCCC;

  TransitionTable[qHCC, symX] := qTX;

```

```

TransitionTable[qHCC, symL] := qTL;
TransitionTable[qHCC, symI] := qOI;
TransitionTable[qHCC, symV] := qOV;
TransitionTable[qHCCC, symX] := qTX;
TransitionTable[qHCCC, symL] := qTL;
TransitionTable[qHCCC, symI] := qOI;
TransitionTable[qHCCC, symV] := qOV;
TransitionTable[qHD, symX] := qTX;
TransitionTable[qHD, symL] := qTL;
TransitionTable[qHD, symI] := qOI;
TransitionTable[qHD, symV] := qOV;
TransitionTable[qHDC, symX] := qTX;
TransitionTable[qHDC, symL] := qTL;
TransitionTable[qHDC, symI] := qOI;
TransitionTable[qHDC, symV] := qOV;
TransitionTable[qHDCC, symX] := qTX;
TransitionTable[qHDCC, symL] := qTL;
TransitionTable[qHDCC, symI] := qOI;
TransitionTable[qHDCC, symV] := qOV;
TransitionTable[qHDCCC, symX] := qTX;
TransitionTable[qHDCCC, symL] := qTL;
TransitionTable[qHDCCC, symI] := qOI;
TransitionTable[qHDCCC, symV] := qOV;

TransitionTable[qT, symX] := qTX;
TransitionTable[qT, symL] := qTL;
TransitionTable[qT, symI] := qOI;
TransitionTable[qT, symV] := qOV;

TransitionTable[qTX, symC] := q0;
TransitionTable[qTX, symL] := q0;
TransitionTable[qTX, symX] := qTXX;
TransitionTable[qTXX, symX] := qTXXX;
TransitionTable[qTX, symI] := qOI;
TransitionTable[qTXX, symI] := qOI;
TransitionTable[qTXXX, symI] := qOI;
TransitionTable[qTX, symV] := qOV;
TransitionTable[qTXX, symV] := qOV;
TransitionTable[qTXXX, symV] := qOV;

TransitionTable[qTL, symX] := qTLX;
TransitionTable[qTLX, symX] := qTLXX;
TransitionTable[qTLXX, symX] := qTLXXX;
TransitionTable[qTL, symI] := qOI;
TransitionTable[qTLX, symI] := qOI;

```

```

TransitionTable[qTLXX, symI] := q0I;
TransitionTable[qTLXXX, symI] := q0I;
TransitionTable[qTL, symV] := q0V;
TransitionTable[qTLX, symV] := q0V;
TransitionTable[qTLXX, symV] := q0V;
TransitionTable[qTLXXX, symV] := q0V;

TransitionTable[q0, symI] := q0I;
TransitionTable[q0, symV] := q0V;
TransitionTable[q0I, symX] := qdone;
TransitionTable[q0I, symV] := qdone;
TransitionTable[q0I, symI] := q0II;
TransitionTable[q0II, symI] := q0III;
TransitionTable[q0V, symI] := q0VI;
TransitionTable[q0VI, symI] := q0VII;
TransitionTable[q0VII, symI] := q0VIII;
end; { Initialize }

begin
  Initialize;
  if Accept then
    writeln(output, 'Accepted')
  else
    writeln(output, 'Rejected')
  end.

```

1.52. **Describe the set of words accepted by the DFA in Figure 1.9.**

Let y be the suffix of the input after the last occurrence of **b**; if no **b** occurs, let y be the entire string. The DFA accepts exactly those words for which

$$|y|_a + 2|y|_c \equiv 2 \pmod{3}.$$

Equivalently, **b** resets the machine to state s_0 , while **a** adds 1 modulo 3 and **c** adds 2 modulo 3, so acceptance depends only on the contribution of the suffix after the last **b**.

1.53. **The languages L_7 , L_3 , and L_n for digit sums.**

- (a) For L_7 , use seven states $q_0, \dots, q_6$, where q_i means that the sum of the digits seen so far is congruent to $i \pmod{7}$. The start state is q_0 , the only final state is q_0 , and

$$\delta(q_i, d) = q_{i+d \bmod 7}.$$

- (b) The corresponding quintuple is

$$\langle \{0, 1, \dots, 9\}, \{q_0, \dots, q_6\}, q_0, \delta, \{q_0\} \rangle,$$

with $\delta(q_i, d) = q_{i+d \bmod 7}$.

- (c) For L_3 , use three states r_0, r_1, r_2 , where r_i records the digit sum modulo 3. Again r_0 is both the start state and the only final state, and

$$\delta(r_i, \mathbf{d}) = r_{i+d \bmod 3}.$$

- (d) The corresponding quintuple is

$$\langle \{\mathbf{0}, \mathbf{1}, \dots, \mathbf{9}\}, \{r_0, r_1, r_2\}, r_0, \delta, \{r_0\} \rangle,$$

$$\text{with } \delta(r_i, \mathbf{d}) = r_{i+d \bmod 3}.$$

- (e) In general,

$$\langle \{\mathbf{0}, \mathbf{1}, \dots, \mathbf{9}\}, \{s_0, \dots, s_{n-1}\}, s_0, \delta, \{s_0\} \rangle$$

recognizes L_n if

$$\delta(s_i, \mathbf{d}) = s_{i+d \bmod n}$$

for every residue class i and every digit $\mathbf{d}$. The state always records the current digit sum modulo n .

1.54. Explain the last row of Table 1.3.

The last row corresponds to the case where the circuitry has somehow been initialized to the unused state **11** and then scans $\langle \text{SOS} \rangle$, which is also encoded by **11**. Unlike the other rows containing unused state/input combinations, this one is *not* a don't-care, because the text explicitly requires $\langle \text{SOS} \rangle$ to reset the machine to the real start state. Therefore the outputs in that row must be fixed so that the circuit moves from the bogus state back to s_0 rather than behaving arbitrarily.

Chapter 2

Nerode's Theorem — Solutions

2.1. Show Ψ is not a right congruence.

$x\Psi y \Leftrightarrow |x| - |y|$ is odd. Take $x = \mathbf{a}$, $y = \mathbf{b}$ (so $|x| - |y| = 0$, not odd, thus $x \not\Psi y$). Also check: is Ψ even an equivalence relation? $|x| - |x| = 0$ (even), so $x \not\Psi x$ (not reflexive). Hence Ψ is not an equivalence relation and a fortiori not a right congruence.

2.2. Q defined by $|x|_{\mathbf{a}} - |y|_{\mathbf{a}} \equiv 0 \pmod{3}$.

- (a) *Equivalence*: Reflexive: $|x|_{\mathbf{a}} - |x|_{\mathbf{a}} = 0 \equiv 0$. Symmetric: if $3 \mid (|x|_{\mathbf{a}} - |y|_{\mathbf{a}})$ then $3 \mid (|y|_{\mathbf{a}} - |x|_{\mathbf{a}})$. Transitive: if $3 \mid (a - b)$ and $3 \mid (b - c)$ then $3 \mid (a - c)$.
- (b) *Right congruence*: Suppose xQy , i.e., $|x|_{\mathbf{a}} \equiv |y|_{\mathbf{a}} \pmod{3}$. For any $z \in \Sigma^*$: $|xz|_{\mathbf{a}} = |x|_{\mathbf{a}} + |z|_{\mathbf{a}}$ and $|yz|_{\mathbf{a}} = |y|_{\mathbf{a}} + |z|_{\mathbf{a}}$. So $|xz|_{\mathbf{a}} - |yz|_{\mathbf{a}} = |x|_{\mathbf{a}} - |y|_{\mathbf{a}} \equiv 0 \pmod{3}$, i.e., $xzQyz$.

2.3. All languages L over $\{\mathbf{a}, \mathbf{b}, \mathbf{c}\}$ with $\text{rk}(R_L) = 1$.

$\text{rk}(R_L) = 1$ means R_L has exactly one equivalence class, namely all of Σ^* . This occurs iff $R_L = \Sigma^* \times \Sigma^*$, i.e., $xR_L y$ for all x, y . This happens iff $(\forall z)(xz \in L \Leftrightarrow yz \in L)$ for all x, y, z , which holds iff either $L = \emptyset$ or $L = \Sigma^*$.

Justification: If $L \neq \emptyset$ and $L \neq \Sigma^*$, pick $w \in L$ and $u \notin L$. Then $wR_L u$ fails (take $z = \lambda$), so R_L has at least 2 classes.

2.4. Show P (with 3 equivalence classes as given) is a right congruence.

The classes are $[\lambda]_P = \{\mathbf{0}, \mathbf{1}\}^0$, $[\mathbf{1}]_P = \{\mathbf{0}, \mathbf{1}\}^1$, $[\mathbf{00}]_P = \bigcup_{n \geq 2} \{\mathbf{0}, \mathbf{1}\}^n$.

We must check: if xPy then $xaPya$ for all $a \in \{0, 1\}$.

Case $x, y \in [\lambda]_P$: $|x| = |y| = 0$, so $|xa| = |ya| = 1$, thus $xa, ya \in [\mathbf{1}]_P$.

Case $x, y \in [\mathbf{1}]_P$: $|x| = |y| = 1$, so $|xa| = |ya| = 2$, thus $xa, ya \in [\mathbf{00}]_P$.

Case $x, y \in [\mathbf{00}]_P$: $|x|, |y| \geq 2$, so $|xa|, |ya| \geq 3 \geq 2$, thus $xa, ya \in [\mathbf{00}]_P$.

In every case $xaPya$, so P is a right congruence. $\square$

2.5. All languages L with $R_L = P$ (from Exercise 2.4).

Recall $A_0 = [\lambda]_P = \{\lambda\}$, $A_1 = [\mathbf{1}]_P = \Sigma^1$, $A_2 = [\mathbf{00}]_P = \Sigma^{\geq 2}$. For $R_L = P$ we need L to be a union of P -classes and no two distinct P -classes to be merged by R_L . Two classes A_i, A_j are merged (identified)

by R_L iff for every $z \in \Sigma^*$: $A_i \cdot z \subseteq L \Leftrightarrow A_j \cdot z \subseteq L$ (more precisely, for representative elements $u \in A_i$, $v \in A_j$: $uz \in L \Leftrightarrow vz \in L$ for all z).

Check each of the 6 nonempty proper unions:

- $L = A_0 = \{\lambda\}$: For any $z \neq \lambda$, both $\mathbf{0}z \notin L$ (length ≥ 2) and $\mathbf{00}z \notin L$; for $z = \lambda$, $\mathbf{0} \notin L$ and $\mathbf{00} \notin L$. So A_1 and A_2 are *not* distinguished: $R_L \neq P$.
- $L = A_1 = \Sigma^1$: $z = \lambda$: $\lambda \notin L$ but $\mathbf{0} \in L$ (distinguishes A_0 from A_1). $\mathbf{0} \in L$ but $\mathbf{00} \notin L$ (distinguishes A_1 from A_2). All three classes remain distinct: $R_L = P$. ✓
- $L = A_2 = \Sigma^{\geq 2}$: $z = \mathbf{0}$: $\lambda\mathbf{0} = \mathbf{0} \notin L$ but $\mathbf{0} \cdot \mathbf{0} = \mathbf{00} \in L$ (distinguishes A_0 from A_1). $z = \lambda$: $\mathbf{0} \notin L$ but $\mathbf{00} \in L$ (distinguishes A_1 from A_2). So $R_L = P$. ✓
- $L = A_0 \cup A_1 = \Sigma^{\leq 1}$: $z = \mathbf{0}$: $\lambda \cdot \mathbf{0} = \mathbf{0} \in L$ but $\mathbf{0} \cdot \mathbf{0} = \mathbf{00} \notin L$ (distinguishes A_0 from A_1). $z = \lambda$: $\mathbf{0} \in L$ but $\mathbf{00} \notin L$ (distinguishes A_1 from A_2). So $R_L = P$. ✓
- $L = A_0 \cup A_2$: $z = \lambda$: $\lambda \in L$ but $\mathbf{0} \notin L$ (distinguishes A_0, A_1). $z = \lambda$: $\mathbf{0} \notin L$ but $\mathbf{00} \in L$ (distinguishes A_1, A_2). So $R_L = P$. ✓
- $L = A_1 \cup A_2 = \Sigma^+$: For any z , $\mathbf{0}z \in \Sigma^+$ and $\mathbf{00}z \in \Sigma^+$, both in L . So A_1 and A_2 are not distinguished: $R_L \neq P$.

Therefore exactly 4 languages satisfy $R_L = P$: A_1 , A_2 , $A_0 \cup A_1$, and $A_0 \cup A_2$.

2.6. All languages L with $R_L = Q$ (from Exercise 2.2).

Q has three equivalence classes: $[x]_Q$ depends on $|x|_a \bmod 3$. So Q -classes are $C_0 = \{x : |x|_a \equiv 0\}$, $C_1 = \{x : |x|_a \equiv 1\}$, $C_2 = \{x : |x|_a \equiv 2\}$. Any nonempty proper union of these classes gives $R_L = Q$.

2.7. Analysis of Q on $\{a, b\}^*$.

- Right congruence, classes*: $\lambda Q \lambda$ by definition; for non-empty strings, xQy iff $(|x|, |y|)$ have the same parity. Classes: $[\lambda]_Q$ (just λ) and $C_e = \{x \neq \lambda : |x| \text{ even}\}$ and $C_o = \{x : |x| \text{ odd}\}$. This is a right congruence because appending any a flips parity of length for non-empty strings, consistently within each class.
- $L = [\lambda]_Q \cup [\mathbf{aa}]_Q = \{\lambda\} \cup C_e$. Simple description: $L = \{x : |x| \text{ is even}\}$. R_L has two classes: even-length strings and odd-length strings.
- A_Q : 3 states ($[\lambda]_Q, C_e, C_o$). Final states for L : $\{[\lambda]_Q, C_e\}$.
- A_{R_L} : 2 states (even, odd), final: even.
- The original choice of final states, $[\lambda]_Q \cup C_e$, does not give a three-state minimal machine: it collapses $[\lambda]_Q$ and C_e and therefore gives the two-state machine A_{R_L} . To make A_{R_K} look like the three-state machine A_Q , relabel the final states so that

$$K = C_e.$$

Then $z = \lambda$ distinguishes $[\lambda]_Q$ from C_e , since $\lambda \notin K$ but $\mathbf{aa} \in K$, and it also distinguishes C_e from C_o , since $\mathbf{aa} \in K$ but $\mathbf{a} \notin K$. Thus the three Q -classes remain distinct, so $R_K = Q$ and A_{R_K} has the same state structure as this relabeled A_Q .

- (f) The eight languages are the $2^3 = 8$ unions of subsets of $\{[\lambda]_Q, C_e, C_o\}$. A language K satisfies the criterion of part (e) iff $R_K = Q$, i.e., A_Q (reabeled with new final states for K) is already minimal.

Checking all eight: $\emptyset$ and Σ^* give $\text{rk}(R_K) = 1$; $\{\lambda\}$, all-even ($\{\lambda\} \cup C_e$), C_o , and Σ^+ ($= C_e \cup C_o$) give $\text{rk}(R_K) = 2$ (two of the three Q -classes merge in each case). Only two choices keep all three Q -classes distinct:

- $K = C_e$: $z = \lambda$ distinguishes $[\lambda]$ from C_e (since $\lambda \notin K$ but $aa \in K$); $z = \lambda$ distinguishes C_e from C_o (since $aa \in K$ but $a \notin K$). So $R_K = Q$.
- $K = \{\lambda\} \cup C_o$: $z = aa$ distinguishes $[\lambda]$ from C_o ($\lambda \cdot aa = aa \notin K$ but $a \cdot aa = aaa \in K$); $z = \lambda$ distinguishes C_o from C_e ($a \in K$ but $aa \notin K$). So $R_K = Q$.

These are the only two of the eight languages satisfying part (e).

2.8. **$I = \text{identity on } \{\mathbf{a}\}^*$.**

- (a) $xIy \Leftrightarrow x = y$. Right congruence: if $x = y$, then $xa = ya$ for any a . Yes.
(b) Each equivalence class is a singleton: $\{\mathbf{a}^n\}_I = \{\mathbf{a}^n\}$.

2.9. **$L = \{\lambda\} \cup \{\mathbf{a}\} \cup \{\mathbf{aa}\}$, I has infinite rank. Is L FAD?**

L is a finite set, hence FAD. Nerode's theorem says L is FAD iff R_L has finite rank, not iff I (or any other right congruence) has finite rank. Here R_L has rank equal to the number of R_L -classes, which is finite (since L is finite and FAD). The fact that I has infinite rank is irrelevant – we need I to *refine* R_L (which it does, trivially, since I is the finest relation), but Nerode's theorem requires R_L itself to have finite rank. $\square$

2.10. **Machine M with $\text{rk}(R_{L(M)}) \neq |S_M|$.**

Let M be the two-state DFA: $\Sigma = \{\mathbf{a}\}$, $S = \{q_0, q_1\}$, $s_0 = q_0$, $F = \{q_0, q_1\}$ (all states final), $\delta(q_0, \mathbf{a}) = q_1$, $\delta(q_1, \mathbf{a}) = q_0$. Then $L(M) = \Sigma^*$, so $R_{L(M)}$ has rank 1, but $|S_M| = 2$.

2.11. **Show F_{R_L} is well defined.**

$F_{R_L} = \{[x]_{R_L} \mid x \in L\}$. We must show: if $[x]_{R_L} = [y]_{R_L}$, then $x \in L \Leftrightarrow y \in L$ (so each R_L -class is either entirely inside L or entirely outside L , making the assignment of classes to F_{R_L} unambiguous). Since $[x]_{R_L} = [y]_{R_L}$, we have $xR_L y$, meaning $(\forall z \in \Sigma^*)(xz \in L \Leftrightarrow yz \in L)$. Taking $z = \lambda$: $x \in L \Leftrightarrow y \in L$. $\square$

2.12. **Show δ_{R_L} is well defined.**

$\delta_{R_L}([x]_{R_L}, a) = [xa]_{R_L}$. We must show: if $[x]_{R_L} = [y]_{R_L}$, then $[xa]_{R_L} = [ya]_{R_L}$. Since $xR_L y$, for any z : $xaz \in L \Leftrightarrow x(az) \in L \Leftrightarrow y(az) \in L \Leftrightarrow yaz \in L$. Thus $xaR_L ya$. $\square$

2.13. **Prove $\bar{\delta}_{R_L}([x]_{R_L}, y) = [xy]_{R_L}$ by induction on $|y|$.**

The step below uses the right-end form of the recursion, justified by the Chapter 1 concatenation theorem for $\bar{\delta}$.

Base $|y| = 0$: $\bar{\delta}_{R_L}([x]_{R_L}, \lambda) = [x]_{R_L} = [x\lambda]_{R_L}$. $\checkmark$

Step: $y = wa$, $|w| = n$. $\bar{\delta}_{R_L}([x]_{R_L}, wa) = \delta_{R_L}(\bar{\delta}_{R_L}([x]_{R_L}, w), a) = \delta_{R_L}([xw]_{R_L}, a) = [xwa]_{R_L} = [xy]_{R_L}$. $\square$

2.14. **Find R^P for automaton P in Example 2.6, check $R^P = R_L$.**

R^P is defined by $xR^P y \Leftrightarrow \bar{\delta}(s_0, x) = \bar{\delta}(s_0, y)$. Since the machine P in Example 2.6 is the minimal machine for L , each state is reachable and distinguishable. The relation R^P has the same classes as R_L (each state corresponds to one R_L -class). Hence $R^P = R_L$.

2.15. **Find R^A for machines from Chapter 1 exercises.**

For each DFA A built in Chapter 1: R^A is defined by $xR^A y \Leftrightarrow \bar{\delta}(s_0, x) = \bar{\delta}(s_0, y)$ (same state reached). Compare to $R_{L(A)}$: by definition of R^A , it always refines $R_{L(A)}$ (since if same state, same future acceptance). For minimal machines they coincide; for non-minimal ones R^A may be strictly finer.

2.16. **Prove the pumping lemma statement by induction.**

Let M be a DFA with states S , $|S| = n$. For a word $w = uvw'$ with $|uv| \leq n$, $|v| \geq 1$, and $uvw' \in L$. Let $s = \bar{\delta}(s_0, u)$, so $\bar{\delta}(s, v) = s$ (since the path through v loops back to s).

Claim: $(\forall i \in \mathbb{N})(\bar{\delta}(s_0, uv^i w') = \bar{\delta}(s_0, uv w') = f \in F)$.

Proof by induction on i :

$i = 0$: $\bar{\delta}(s_0, uw') = \bar{\delta}(\bar{\delta}(s_0, u), w') = \bar{\delta}(s, w')$. And $\bar{\delta}(s_0, uvw') = \bar{\delta}(\bar{\delta}(s_0, uv), w') = \bar{\delta}(s, w')$ (since $\bar{\delta}(s, v) = s$). Equal. ✓

$i \rightarrow i+1$: By the inner induction, $\bar{\delta}(s_0, uv^i) = s$ (base: $\bar{\delta}(s_0, u) = s$; step: $\bar{\delta}(s_0, uv^{i+1}) = \bar{\delta}(\bar{\delta}(s_0, uv^i), v) = \bar{\delta}(s, v) = s$). Therefore

$$\bar{\delta}(s_0, uv^{i+1} w') = \bar{\delta}(\bar{\delta}(s_0, uv^i), v w') = \bar{\delta}(s, v w') = \bar{\delta}(\bar{\delta}(s, v), w') = \bar{\delta}(s, w').$$

So $\bar{\delta}(s_0, uv^i w') = \bar{\delta}(s, w') = f$ for all i . □

2.17. **Prove Theorem 2.1.**

Theorem 2.1 states: L is FAD iff R_L has finite rank.

($\Rightarrow$) If $L = L(M)$ for a DFA M with n states, define $\mu: \Sigma^* \rightarrow S$ by $\mu(x) = \bar{\delta}(s_0, x)$. Then $xR^M y \Leftrightarrow \mu(x) = \mu(y)$, so R^M has at most n classes. Since R^M refines R_L and R_L is coarser, $\text{rk}(R_L) \leq \text{rk}(R^M) \leq n < \infty$.

($\Leftarrow$) If R_L has finite rank k , construct the minimal machine $A_{R_L} = \langle \Sigma, [\cdot]_{R_L}, [\lambda]_{R_L}, \delta_{R_L}, F_{R_L} \rangle$ with k states (shown well-defined in Exercises 2.11–2.12). Show $L(A_{R_L}) = L$: $x \in L(A_{R_L}) \Leftrightarrow \bar{\delta}_{R_L}([\lambda], x) \in F_{R_L} \Leftrightarrow [x] \in F_{R_L} \Leftrightarrow x \in L$.

2.18. **Find a language giving rise to I (identity on $\{a\}^*$); show it cannot be FAD.**

(a) Take

$$L = \{a^{2^m} \mid m \geq 0\} \subseteq \{a\}^*.$$

We claim $R_L = I$. Let $i < j$ and $d = j - i \geq 1$. Choose m so that $2^m > d$, and set $z = a^{2^m - i}$. Then

$$a^i z = a^{2^m} \in L,$$

but

$$a^j z = a^{2^m + d}$$

with

$$2^m < 2^m + d < 2^{m+1},$$

so $2^m + d$ is not a power of 2. Hence $a^j z \notin L$. Thus $a^i \not R_L a^j$ for all $i \neq j$, so every class is a singleton and $R_L = I$.

(b) Such a language cannot be FAD: $R_L = I$ has infinite rank (one class per string in $\{a\}^*$), so by Nerode's theorem L is not FAD. $\square$

2.19. Prove Theorem 2.4 from Theorem 2.3.

Theorem 2.3: If L is FAD then R_L has finite rank. Theorem 2.4 (Nerode): L is FAD $\Leftrightarrow R_L$ has finite rank.

We need the converse: finite rank $\Rightarrow$ FAD. Given R_L has k classes, the machine A_{R_L} has k states and accepts L (by Exercises 2.11–2.13). Hence L is FAD.

2.20. Prove Theorem 2.5.

Theorem 2.5: L is FAD $\Leftrightarrow$ there exists a right congruence R of finite rank such that L is a union of R -classes.

($\Rightarrow$) Take $R = R^M$ for a DFA M accepting L : finite rank (at most $|S|$), and $L = \bigcup_{s \in F} [s]_{R^M}$.

($\Leftarrow$) Given such R , build A_R with states = R -classes, $F_R = \text{classes} \subseteq L$. Then $L = L(A_R)$, so L is FAD.

2.21. Prove Theorem 2.6.

Theorem 2.6 states:

$$(\exists n)(\forall x \notin L, |x| \geq n)(\exists u, v, w)[x = uvw, |uv| \leq n, |v| \geq 1, (\forall i \geq 0) uv^i w \notin L].$$

Assume L is FAD, accepted by DFA $M = \langle \Sigma, S, s_0, \delta, F \rangle$, with $|S| = n$. Take any $x \notin L$ with $|x| \geq n$. Look at the first $n + 1$ states on the run of x :

$$s_0, \bar{\delta}(s_0, a_1), \bar{\delta}(s_0, a_1 a_2), \dots, \bar{\delta}(s_0, a_1 \cdots a_n).$$

By pigeonhole, two are equal. So for some decomposition $x = uvw$ with $|uv| \leq n, |v| \geq 1$,

$$\bar{\delta}(s_0, u) = \bar{\delta}(s_0, uv) = q.$$

Hence v is a loop at q , so for every $i \geq 0$,

$$\bar{\delta}(s_0, uv^i w) = \bar{\delta}(q, w) = \bar{\delta}(s_0, uvw) = \bar{\delta}(s_0, x).$$

Since $x \notin L$, $\bar{\delta}(s_0, x) \notin F$. Therefore $\bar{\delta}(s_0, uv^i w) \notin F$, i.e., $uv^i w \notin L$ for all $i \geq 0$. This is exactly Theorem 2.6. $\square$

2.22. Prove Theorem 2.7.

Theorem 2.7 states: if M is an n -state DFA accepting L , then every $x = a_1 a_2 \cdots a_m \in L$ with $m \geq n$ has a subsequence $a_{i_1} a_{i_2} \cdots a_{i_j} \in L$ with $j < n$.

Let $x \in L, |x| = m \geq n$. By Theorem 2.3 (or Exercise 2.16), we can write

$$x = u_1 v_1 w_1, \quad |u_1 v_1| \leq n, |v_1| \geq 1, \text{ and } u_1 v_1^i w_1 \in L \forall i \geq 0.$$

In particular, with $i = 0$, $x_1 = u_1 w_1 \in L$ and $|x_1| < |x|$.

If $|x_1| < n$, stop. Otherwise apply the same argument to x_1 . Each step removes at least one symbol and keeps membership in L , so after finitely many steps we obtain

$$y \in L, \quad |y| < n,$$

and y is obtained from x by deleting some blocks. Therefore y is a subsequence of x :

$$y = a_{i_1} a_{i_2} \cdots a_{i_j}$$

for an increasing index sequence $i_1 < i_2 < \cdots < i_j$, with $j = |y| < n$. This is exactly Theorem 2.7. $\square$

2.23. **Show $L = \{x \mid |x|_a < |x|_b\}$ is not FAD.**

We use Theorem 2.5. Suppose, for contradiction, that L is FAD, and let n be the number supplied by that theorem.

Take

$$x = \mathbf{a}^n \mathbf{b}^{n+1}.$$

Then $x \in L$, because $|x|_a = n < n+1 = |x|_b$.

By Theorem 2.5 there should be a decomposition

$$x = uvw, \quad |vw| \leq n, \quad |v| \geq 1,$$

such that

$$uv^i w \in L \quad \text{for every } i \geq 0.$$

Since the last n symbols of x are all $\mathbf{b}$'s, the substring v must be of the form $\mathbf{b}^k$ for some $k \geq 1$. Now take $i = 0$. Then

$$uw = \mathbf{a}^n \mathbf{b}^{n+1-k}.$$

Because $k \geq 1$, we have $n+1-k \leq n$, so

$$|uw|_b \leq |uw|_a.$$

Hence $uw \notin L$, contradicting Theorem 2.5.

Therefore L is not FAD. $\square$

2.24. **Show $G = \{x \mid |x|_a \geq |x|_b\}$ is not FAD.**

We use Theorem 2.6. Suppose, for contradiction, that G is FAD, and let n be the number supplied by that theorem.

Take

$$x = \mathbf{a}^n \mathbf{b}^{n+1}.$$

Then $x \notin G$, because $|x|_a = n$ and $|x|_b = n+1$.

By Theorem 2.6 there should be a decomposition

$$x = uvw, \quad |uv| \leq n, \quad |v| \geq 1,$$

such that

$$uv^i w \notin G \quad \text{for every } i \geq 0.$$

Since the first n symbols of x are all $\mathbf{a}$'s, the substring v must be of the form $\mathbf{a}^k$ for some $k \geq 1$. Now pump once more than usual, taking $i = 2$. Then

$$uv^2 w = \mathbf{a}^{n+k} \mathbf{b}^{n+1}.$$

Because $k \geq 1$, we have $n + k \geq n + 1$, so

$$|uv^2w|_a \geq |uv^2w|_b.$$

Hence $uv^2w \in G$, contradicting Theorem 2.6.

Therefore G is not FAD. $\square$

2.25. **Show $P = \{\mathbf{d}^p \mid p \text{ prime}\}$ is not FAD.**

Suppose P is FAD with n states. Take a prime $p > n$ and $w = \mathbf{d}^p \in P$. By pumping: $w = uvw'$ with $|uv| \leq n$, $|v| = k \geq 1$, so $v = \mathbf{d}^k$ and $u = \mathbf{d}^j$, $w' = \mathbf{d}^{p-j-k}$. Then $uv^i w' = \mathbf{d}^{j+ik+(p-j-k)} = \mathbf{d}^{p+(i-1)k}$ must be in P for all $i \geq 0$. So $p + (i-1)k$ must be prime for all $i \geq 0$. Take $i = p+1$: $p + pk = p(1+k)$, which is composite (both factors > 1 since $p \geq 2$ and $k \geq 1$). Contradiction. $\square$

2.26. **Show $\Gamma = \{w \cdot \mathbf{2} \cdot w \mid w \in \{\mathbf{0}, \mathbf{1}\}^*\}$ is not FAD.**

We use Theorem 2.6. Suppose Γ were FAD, and let n be the number supplied by that theorem.

Take

$$x = \mathbf{0}^{n!} \mathbf{2} \mathbf{0}^{2n!}.$$

This string is *not* in Γ , because the block to the left of $\mathbf{2}$ has length $n!$ while the block to the right has length $2n!$, so the two sides are unequal.

By Theorem 2.6 there should be a decomposition

$$x = uvw, \quad |uv| \leq n, \quad |v| \geq 1,$$

such that

$$uv^i w \notin \Gamma \quad \text{for every } i \geq 0.$$

Since the first n symbols of x are all $\mathbf{0}$'s, the substring v must be of the form $\mathbf{0}^k$ for some $1 \leq k \leq n$. Because $k \leq n$, the integer k divides $n!$. Choose

$$i = \frac{n!}{k} + 1.$$

Then pumping v changes only the block of $\mathbf{0}$'s to the left of $\mathbf{2}$, and its new length is

$$n! + (i-1)k = n! + \frac{n!}{k} k = 2n!.$$

Therefore

$$uv^i w = \mathbf{0}^{2n!} \mathbf{2} \mathbf{0}^{2n!} \in \Gamma.$$

This contradicts Theorem 2.6.

Hence Γ is not FAD. $\square$

2.27. **Show $\Psi = \{ww \mid w \in \{\mathbf{0}, \mathbf{1}\}^*\}$ is not FAD.**

For each $n \geq 0$, let $x_n = \mathbf{0}^n \mathbf{1}$. We show the x_n are pairwise in distinct R_Ψ -classes.

Fix $i \neq j$ and take suffix $z_i = \mathbf{0}^i \mathbf{1}$. Then

$$x_i z_i = \mathbf{0}^i \mathbf{1} \mathbf{0}^i \mathbf{1} = ww \in \Psi$$

with $w = \mathbf{0}^i \mathbf{1}$.

Now consider

$$x_j z_i = \mathbf{0}^j \mathbf{10}^i \mathbf{1}.$$

Suppose this were in Ψ , so $x_j z_i = tt$ for some t . The string has exactly two **1**s, so each half must contain exactly one **1**. Hence the position of the second **1** must be “half-length” after the first. Its first **1** is at position $j + 1$, second at $j + i + 2$, so the half-length must be $i + 1$. But half-length is also $\frac{i+j+2}{2}$, giving $\frac{i+j+2}{2} = i + 1$, hence $j = i$, contradiction. Therefore $x_j z_i \notin \Psi$.

So $x_i R_\Psi x_j$ for $i \neq j$, giving infinitely many Nerode classes. Thus Ψ is not FAD. $\square$

2.28. **Show $K = \{w \mid w = w^r\}$ (palindromes) is not FAD.**

We use Theorem 2.3. Suppose, for contradiction, that K is FAD, and let n be the number supplied by that theorem.

Take

$$x = \mathbf{0}^n \mathbf{10}^n.$$

This word is in K , since it is the same read from left to right and right to left.

By Theorem 2.3 there should be a decomposition

$$x = uvw, \quad |uv| \leq n, \quad |v| \geq 1,$$

such that

$$uv^i w \in K \quad \text{for every } i \geq 0.$$

Since the first n symbols of x are all **0**'s, the substring v must be of the form $\mathbf{0}^k$ for some $k \geq 1$. Now take $i = 0$. Then

$$uw = \mathbf{0}^{n-k} \mathbf{10}^n.$$

The block of **0**'s to the left of **1** has length $n - k$, while the block to the right has length n . These are unequal, so uw is not a palindrome. Thus $uw \notin K$, contradicting Theorem 2.3.

Therefore K is not FAD. $\square$

2.29. **Show $\Phi = \{\mathbf{a}^j \mathbf{b}^k \mathbf{c}^m \mid j \geq 3, k = m\}$ is not FAD (second pumping lemma).**

We use Theorem 2.5. Suppose, for contradiction, that Φ is FAD. Then there is a right congruence R of finite rank such that Φ is a union of R -classes.

For each $i \in \mathbb{N}$, let

$$x_i = \mathbf{a}^3 \mathbf{b}^i.$$

We claim that the x_i lie in distinct R -classes. If $i \neq j$ and $x_i R x_j$, then by the right-congruence property,

$$x_i \mathbf{c}^i R x_j \mathbf{c}^i.$$

But

$$x_i \mathbf{c}^i = \mathbf{a}^3 \mathbf{b}^i \mathbf{c}^i \in \Phi,$$

while

$$x_j \mathbf{c}^i = \mathbf{a}^3 \mathbf{b}^j \mathbf{c}^i \notin \Phi$$

because $j \neq i$. Since Φ is a union of R -classes, two R -equivalent strings cannot disagree about membership in Φ . Contradiction.

So $x_0, x_1, x_2, \dots$ belong to infinitely many distinct R -classes, contradicting the finite rank of R . Therefore Φ is not FAD. $\square$

Why first pumping lemma is hard: Any long word in Φ starts with ≥ 3 **as**. Pumping within the **a**-block may still satisfy $j \geq 3$; pumping within **b** or **c** block destroys $k = m$.

2.30. **Show $C = \{\mathbf{d}^q \mid q \text{ non-prime}\}$ is not FAD.**

We use Theorem 2.6. Suppose C were FAD, and let n be the number supplied by that theorem.

Choose a prime $p > n$ and let

$$x = \mathbf{d}^p.$$

Since p is prime, $x \notin C$. By Theorem 2.6 there should be a decomposition

$$x = uvw, \quad |uv| \leq n, \quad |v| \geq 1,$$

such that

$$uv^i w \notin C \quad \text{for every } i \geq 0.$$

Because x is unary, $v = \mathbf{d}^k$ for some $k \geq 1$. Then

$$uv^i w = \mathbf{d}^{p+(i-1)k}.$$

Now choose $i = p + 1$. Its length is

$$p + pk = p(1 + k),$$

which is composite. Hence

$$uv^{p+1} w \in C,$$

contradicting Theorem 2.6.

Therefore C is not FAD. $\square$

2.31. **R_L with 3 classes: $\{\lambda\}$, odd-length, even-length $\setminus \{\lambda\}$.**

(a) Why $L = \{x \mid |x| \text{ odd}\}$ can't give this R_L : Compute $R_{\{x: |x| \text{ odd}\}}$. We need $xR_L y \Leftrightarrow (\forall z)(|xz| \text{ odd} \Leftrightarrow |yz| \text{ odd})$, i.e., $|x| \equiv |y| \pmod{2}$. So R_L has only 2 classes (even-length, odd-length), not 3. λ is not distinguished from other even-length strings.

(b) Let

$$A = \{\lambda\}, \quad B = \{x : |x| \text{ odd}\}, \quad C = \{x : |x| \text{ even and } x \neq \lambda\}.$$

Checking all 8 unions of A, B, C , exactly two give this same 3-class relation:

$$L = C \quad \text{and} \quad L = A \cup B.$$

For each of these, A, B, C are pairwise distinguishable:

- A vs C : suffix $z = \lambda$.
- A vs B : suffix z of odd length.
- B vs C : suffix $z = \lambda$.

All other unions merge at least one pair of classes.

2.32. **Describe R_Ψ and draw machine for $\Psi = \{x \mid |x|_a \text{ even, ends with } b\}$.**

R_Ψ classes: we need to determine when x, y have the same future w.r.t. Ψ . The future of x depends on: (1) parity of $|x|_a$, and (2) whether x ends with b .

Thus R_Ψ has four classes:

- $C_{e,b} = \{x \mid |x|_a \text{ even and } x \text{ ends with } b\}$.
- $C_{e,\neg b} = \{x \mid |x|_a \text{ even and } x \text{ does not end with } b\}$.
- $C_{o,b} = \{x \mid |x|_a \text{ odd and } x \text{ ends with } b\}$.
- $C_{o,\neg b} = \{x \mid |x|_a \text{ odd and } x \text{ does not end with } b\}$.

Here $\lambda \in C_{e,\neg b}$ (even number of a 's, and it does not end with b).

So the minimal DFA has 4 states, start state $C_{e,\neg b}$, and unique final state $C_{e,b}$.

2.33. **Show $\Xi = \{\lambda, a, a^4, a^9, \dots\}$ (perfect square lengths) is not FAD.**

We use Theorem 2.3. Suppose Ξ were FAD, and let n be a pumping length for Ξ . Take

$$w = a^{n^2} \in \Xi.$$

For any decomposition

$$w = uvw, \quad |uv| \leq n, \quad |v| \geq 1,$$

the string v must be of the form a^k with $1 \leq k \leq n$.

Pump once upward:

$$uv^2w = a^{n^2+k}.$$

Since $1 \leq k \leq n$, we have

$$n^2 < n^2 + k < n^2 + (2n + 1) = (n + 1)^2.$$

So $n^2 + k$ is strictly between two consecutive squares, hence is not itself a square. Therefore

$$uv^2w \notin \Xi,$$

contradicting Theorem 2.3.

Thus Ξ is not FAD. $\square$

2.34. **Show $\Phi = \{b, b^2, b^4, b^8, \dots\}$ (powers of 2) is not FAD.**

We use Theorem 2.3. Suppose Φ were FAD, and let n be a pumping length for Φ . Choose m so that $2^m > n$, and let

$$w = b^{2^m} \in \Phi.$$

For any decomposition

$$w = uvw, \quad |uv| \leq n, \quad |v| \geq 1,$$

we have $v = b^k$ for some $1 \leq k \leq n < 2^m$.

Pump once upward:

$$uv^2w = b^{2^m+k}.$$

Because $0 < k < 2^m$,

$$2^m < 2^m + k < 2^{m+1}.$$

So $2^m + k$ lies strictly between two consecutive powers of 2, and therefore is not itself a power of 2. Hence

$$uv^2w \notin \Phi,$$

contradicting Theorem 2.3.

Therefore Φ is not FAD. $\square$

2.35. Five-class R_L over $\{a, b\}$.

(a) Let

$$C_0 = \{\lambda\}, C_1 = \{a\}, C_2 = \{aa\}, C_3 = \{a^k : k \geq 3\}, C_4 = \{x : x \text{ contains a } b\}.$$

If L is exactly one class and still has R_L equal to this 5-class partition, the only possibility is

$$L = C_3 = \{a^3, a^4, \dots\}.$$

Choosing any other single class merges some classes under R_L .

(b) The other languages L that still give this same R_L are exactly:

$$C_0 \cup C_3, \quad C_1 \cup C_3, \quad C_0 \cup C_1 \cup C_3,$$

$$C_2 \cup C_4, \quad C_0 \cup C_2 \cup C_4, \quad C_1 \cup C_2 \cup C_4, \quad C_0 \cup C_1 \cup C_2 \cup C_4.$$

Together with part (a), there are 8 such languages.

2.36. Find R_Ω for $\Omega = \{y \mid (y \text{ has exactly one } 0) \vee (y \text{ has even \#1s})\}$.

$xz \in \Omega$ iff $(|x|_0 + |z|_0 = 1)$ or $(|x|_1 + |z|_1 \equiv 0 \pmod{2})$. The future of x (i.e., which z satisfy $xz \in \Omega$) depends only on $c_0 = |x|_0$ (capped at 2) and $p = |x|_1 \pmod{2}$. Writing (c_0, p) for these two pieces of information, the six cases yield the following future descriptions:

- $(0, 0)$: $xz \in \Omega \Leftrightarrow |z|_0 = 1$ or $|z|_1$ even.
- $(0, 1)$: $xz \in \Omega \Leftrightarrow |z|_0 = 1$ or $|z|_1$ odd.
- $(1, 0)$: $xz \in \Omega \Leftrightarrow |z|_0 = 0$ or $|z|_1$ even.
- $(1, 1)$: $xz \in \Omega \Leftrightarrow |z|_0 = 0$ or $|z|_1$ odd.
- $(2, 0)$: $xz \in \Omega \Leftrightarrow |z|_1$ even. ($|xz|_0 \geq 2$ so the first disjunct can never hold.)
- $(2, 1)$: $xz \in \Omega \Leftrightarrow |z|_1$ odd.

All six futures are distinct (for example, $(0, 0)$ and $(2, 0)$ differ on $z = \mathbf{01}$: the first accepts it since $|z|_0 = 1$, the second rejects since $|z|_1 = 1$ is odd; and so on for the other pairs). Hence R_Ω has exactly **6** equivalence classes:

$$C_{c_0, p} = \{x \in \{0, 1\}^* \mid |x|_0 = c_0 \text{ (or } \geq 2 \text{ if } c_0 = 2) \text{ and } |x|_1 \equiv p \pmod{2}\},$$

for $(c_0, p) \in \{0, 1, 2\} \times \{0, 1\}$. The minimal DFA for Ω has 6 states. Final states are those in Ω : $(1, 0), (1, 1), (0, 0), (2, 0)$ — i.e., strings with exactly one **0** (states $(1, \cdot)$) or with even **#1s** (states $(\cdot, 0)$).

2.37. Which of $L_1, L_2, L_1 \cap L_2, \sim L_2, L_1 \cup L_2$ are FAD?

$L_1 = \{x \mid |x|_a > |x|_b\}$: **not FAD**. If it were FAD, let n be a pumping length from Theorem 2.3. Take

$$w = a^{n+1}b^n \in L_1.$$

Any split $w = uvw'$ with $|uv| \leq n$ and $|v| \geq 1$ has $v = a^k$ for some $k \geq 1$. Pumping down gives

$$uw' = a^{n+1-k}b^n,$$

and now $n+1-k \leq n$, so this pumped string is not in L_1 . Contradiction.

$L_2 = \{x \mid |x|_a < 3\}$: **FAD**. A DFA only needs to remember whether it has seen 0, 1, 2, or at least 3 copies of **a**.

$L_1 \cap L_2$: **FAD**. In fact this language is finite:

$$L_1 \cap L_2 = \{a, aa, aab, aba, baa\}.$$

Every finite language is FAD.

$\sim L_2 = \{x \mid |x|_a \geq 3\}$: **FAD**. The same 4-state machine used for L_2 works here too; we simply make the “at least 3 a’s” state final.

$L_1 \cup L_2$: **not FAD**. Suppose it were FAD, and let n be a pumping length. Consider

$$w = a^{n+4}b^{n+3} \in L_1 \cup L_2$$

because it has more **a**’s than **b**’s. Again any split $w = uvw'$ with $|uv| \leq n$ and $|v| \geq 1$ has $v = a^k$ for some $k \geq 1$. Pumping down gives

$$uw' = a^{n+4-k}b^{n+3}.$$

Here $n+4-k \geq 4$, so the pumped string is not in L_2 ; and also $n+4-k \leq n+3$, so it is not in L_1 . Thus $uw' \notin L_1 \cup L_2$, contradicting Theorem 2.3.

2.38. R_m defined by $|x| - |y|$ is a multiple of m .

- (a) *Equivalence relation and right congruence*: Reflexive, symmetric, and transitive are immediate from divisibility by m . If $m \mid (|x| - |y|)$, then

$$|xz| - |yz| = |x| - |y|,$$

so $m \mid (|xz| - |yz|)$ as well. Thus R_m is a right congruence.

- (b) $R_2 \cap R_3 = R_6$: $xR_2 \cap R_3 y \Leftrightarrow 2 \mid |x| - |y|$ and $3 \mid |x| - |y| \Leftrightarrow 6 \mid |x| - |y| \Leftrightarrow xR_6 y$. So every R_6 -class is contained in an R_2 -class (and also in an R_3 -class); in particular, $R_6 \subseteq R_2$.
- (c) In general: since both R and S are equivalence relations, $R \cap S$ is also an equivalence relation. And if $xR \cap Sy$, then xRy and xSy . For any z , right congruence of R and S gives $xzRyz$ and $xzSyz$, hence $xz(R \cap S)yz$. So $R \cap S$ is a right congruence. Also $R \cap S \subseteq R$ and $R \cap S \subseteq S$, so its classes refine the classes of both R and S .
- (d) $R_2 \cup R_3$: $\lambda R_2 aa$ and $a R_3 aaaa$. But is $\lambda R_2 \cup R_3 a$? $|\lambda| - |a| = -1$, not divisible by 2 or 3. No. Is $\lambda R_2 \cup R_3 aaa$? $0 - 3 = -3$, divisible by 3. Yes. Now $\lambda R_2 \cup R_3 aaa$ and $aaa R_2 \cup R_3 aaaaa$ (length diff=2). But is $\lambda R_2 \cup R_3 aaaaa$? Length diff=5, not divisible by 2 or 3. No. So transitivity fails: not an equivalence relation.

- (e) If $R \cup S$ is an equivalence relation: for any x, y with $xR \cup Sy$, we have xRy or xSy . Either way, for any z : $xzRyz$ or $xzSyz$ (by right congruence of R or S), hence $xzR \cup Syz$.

2.39. **Example of two right congruences whose union is not a right congruence.**

Let $R = R_2$ and $S = R_3$. As shown above, $R_2 \cup R_3$ is not even an equivalence relation, hence not a right congruence.

2.40. **Analysis of $\Gamma = \{\lambda, \mathbf{a}, \mathbf{ab}, \mathbf{ba}, \mathbf{bb}, \mathbf{bbb}\} \cup \{x : |x| \geq 4\}$.**

- (a) **$\mathbf{ab}R_\Gamma\mathbf{ba}$** : For all z : $\mathbf{ab}z \in \Gamma \Leftrightarrow \mathbf{ba}z \in \Gamma$. If $|z| = 0$: $\mathbf{ab}, \mathbf{ba} \in \Gamma$ (both listed). $\checkmark$ If $|z| = 1$: $|\mathbf{ab}z| = 3$. Since Γ contains only $\mathbf{bbb}$ at length 3, and $\mathbf{ab}$ cannot be extended to $\mathbf{bbb}$ by appending one letter, $\mathbf{ab}z \notin \Gamma$; likewise $\mathbf{ba}z \notin \Gamma$ (since $\mathbf{baa}, \mathbf{bab} \neq \mathbf{bbb}$). Both outside Γ . $\checkmark$ If $|z| \geq 2$: $|\mathbf{ab}z|, |\mathbf{ba}z| \geq 4$, so both in Γ . $\checkmark$ Hence **$\mathbf{ab}R_\Gamma\mathbf{ba}$** .
- (b) **$\mathbf{ab}$ is not related to $\mathbf{bb}$** : take $z = \mathbf{b}$. Then

$$\mathbf{ab}z = \mathbf{abb} \notin \Gamma$$

because among length-3 strings only $\mathbf{bbb}$ is in Γ . But

$$\mathbf{bb}z = \mathbf{bbb} \in \Gamma.$$

So **$\mathbf{ab} \not R_\Gamma \mathbf{bb}$** . $\square$

- (c) The 8 classes are exactly:

$$C_0 = \{\lambda\}, C_1 = \{\mathbf{a}\}, C_2 = \{\mathbf{b}\}, C_3 = \{\mathbf{aa}\}, C_4 = \{\mathbf{bb}\},$$

$$C_5 = \{\mathbf{ab}, \mathbf{ba}\}, C_6 = \{x : |x| = 3, x \neq \mathbf{bbb}\}, C_7 = \{\mathbf{bbb}\} \cup \{x : |x| \geq 4\}.$$

These are pairwise distinct by suitable suffixes (for instance, C_4 vs C_5 by suffix $\mathbf{b}$: $\mathbf{bbb} = \mathbf{bbb} \in \Gamma$ but $\mathbf{abb} = \mathbf{abb} \notin \Gamma$).

- (d) Minimal DFA has these 8 states, start C_0 , final states

$$F = \{C_0, C_1, C_4, C_5, C_7\}.$$

Transition function:

$$\delta(C_0, \mathbf{a}) = C_1, \delta(C_0, \mathbf{b}) = C_2,$$

$$\delta(C_1, \mathbf{a}) = C_3, \delta(C_1, \mathbf{b}) = C_5,$$

$$\delta(C_2, \mathbf{a}) = C_5, \delta(C_2, \mathbf{b}) = C_4,$$

$$\delta(C_3, \mathbf{a}) = C_6, \delta(C_3, \mathbf{b}) = C_6,$$

$$\delta(C_4, \mathbf{a}) = C_6, \delta(C_4, \mathbf{b}) = C_7,$$

$$\delta(C_5, \mathbf{a}) = C_6, \delta(C_5, \mathbf{b}) = C_6,$$

$$\delta(C_6, \mathbf{a}) = C_7, \delta(C_6, \mathbf{b}) = C_7, \delta(C_7, \mathbf{a}) = C_7, \delta(C_7, \mathbf{b}) = C_7.$$

2.41. **Prove R^M is a right congruence.**

$$R^M: xR^M y \Leftrightarrow \bar{\delta}(s_0, x) = \bar{\delta}(s_0, y).$$

Since this relation is defined by equality of destination state, it is immediately an equivalence relation: reflexive, symmetric, and transitive all follow from equality.

To check the right-congruence property, assume $xR^M y$, so $\bar{\delta}(s_0, x) = \bar{\delta}(s_0, y) = s$. Then for any $z \in \Sigma^*$,

$$\bar{\delta}(s_0, xz) = \bar{\delta}(s, z) = \bar{\delta}(s_0, yz),$$

so $xzR^M yz$. Therefore R^M is a right congruence. $\square$

2.42. **Is Pascal (set of all valid Pascal programs) FAD?**

No. We use a Chapter 2 distinct-futures argument: matching nested `begin/end` blocks requires arbitrarily many different continuations.

For each $n \geq 1$, let x_n be the ASCII string

```
program p; begin begin ... begin
```

with exactly n occurrences of the keyword `begin`, and let z_n be the ASCII string

```
end end ... end.
```

with exactly n occurrences of the keyword `end`.

Then $x_n z_n$ is a valid Pascal program: after the header `program p;`, the body is a nested compound statement with n occurrences of `begin` followed by the matching n occurrences of `end`, and then the final period.

If $m \neq n$, then $x_m z_n$ is not a valid Pascal program. If $m < n$, there are too many ends; if $m > n$, there are unmatched begins left over before the final period.

Therefore x_n and x_m have different futures with respect to the language Pascal, since

$$x_n z_n \in \text{Pascal} \quad \text{but} \quad x_m z_n \notin \text{Pascal} \quad (m \neq n).$$

So the strings $x_1, x_2, x_3, \dots$ lie in distinct R_{Pascal} -classes. Hence R_{Pascal} has infinitely many equivalence classes, and Pascal is not FAD.

2.43. **Is Short Pascal (programs with $< 10^6$ characters) FAD?**

Yes. Short Pascal is a finite language (finitely many programs of length $< 10^6$ over the ASCII alphabet). Every finite language is FAD. The DFA would have a huge but finite number of states (roughly exponential in 10^6 , but finite).

2.44. $L = \{a^n b^k c^j \mid n < 3 \text{ or } (n \geq 3 \text{ and } k = j)\}$.

(a) **Hypothesis false, conclusion true.**

First, the hypothesis “ L is FAD” is false. For each $r \in \mathbb{N}$, let

$$x_r = a^3 b^r.$$

If $r \neq s$, append the suffix $z = \mathbf{c}^r$. Then

$$x_r z = \mathbf{a}^3 \mathbf{b}^r \mathbf{c}^r \in L,$$

but

$$x_s z = \mathbf{a}^3 \mathbf{b}^s \mathbf{c}^r \notin L$$

because the $\mathbf{b}$ - and $\mathbf{c}$ -counts are unequal. So the strings $x_0, x_1, x_2, \dots$ lie in distinct R_L -classes. Hence R_L has infinite rank, and by Theorem 2.1 the language L is not FAD.

Now show the pumping *conclusion* of Theorem 2.3 does hold for L . Take pumping length $N = 3$. Let

$$x = \mathbf{a}^n \mathbf{b}^k \mathbf{c}^j \in L, \quad |x| \geq 3.$$

We choose a decomposition $x = uvw$ with $|uv| \leq 3$, $|v| \geq 1$, and show $uv^m w \in L$ for all $m \geq 0$.

Case 1: $n \geq 3$ (so $k = j$). Choose

$$u = \lambda, \quad v = \mathbf{a}, \quad w = \mathbf{a}^{n-1} \mathbf{b}^k \mathbf{c}^k.$$

Then $uv^m w = \mathbf{a}^{n-1+m} \mathbf{b}^k \mathbf{c}^k$. If $n-1+m < 3$, it is in L by the $n < 3$ clause. If $n-1+m \geq 3$, it is in L because $k = j$ still holds.

Case 2: $n < 3$. Since $|x| \geq 3$, there is at least one non- $\mathbf{a}$ symbol, and its position is at most 3. Let v be that first non- $\mathbf{a}$ symbol (either $\mathbf{b}$ or $\mathbf{c}$), with u the preceding prefix. Then $|uv| \leq 3$, $|v| = 1$. Pumping changes only the number of $\mathbf{b}$'s or $\mathbf{c}$'s, not the number of $\mathbf{a}$'s, so $n < 3$ remains true for all m . Therefore $uv^m w \in L$ for all $m \geq 0$.

So the pumping conclusion holds even though L is not FAD.

- (b) The contrapositive of Theorem 2.3: “if pumping fails then L is not FAD.” This is trivially true (it’s logically equivalent to Theorem 2.3). But here pumping doesn’t fail, so the contrapositive doesn’t give us information. The converse (“if pumping holds then L is FAD”) is *false*, as this example shows.

2.45. Show F_Q in Definition 2.5 is well defined.

$$F_Q = \{[x]_Q \mid x \in L\}.$$

First, this is defined everywhere it needs to be: if $x \in L$, then certainly $x \in \Sigma^*$, so its equivalence class $[x]_Q$ exists and is an element of

$$S_Q = \{[y]_Q \mid y \in \Sigma^*\}.$$

Second, it is not multiply defined. Suppose $[x]_Q = [y]_Q$ and $x \in L$. Then x and y lie in the same Q -class. Since L is assumed to be a union of Q -classes, the whole class $[x]_Q = [y]_Q$ is either contained in L or disjoint from L . Because $x \in L$, that class must be contained in L , so $y \in L$ as well.

Thus the choice of representative does not matter, and F_Q is a well-defined subset of S_Q . $\square$

2.46. Show δ_Q in Definition 2.5 is well defined.

$$\delta_Q([x]_Q, a) = [xa]_Q.$$

First, it is defined for every state and input letter: if $[x]_Q \in S_Q$ and $a \in \Sigma$, then $xa \in \Sigma^*$, so $[xa]_Q \in S_Q$.

Second, it is independent of the chosen representative of the class. If $[x]_Q = [y]_Q$, then xQy . Since Q is a right congruence, $xaQya$, and therefore

$$[xa]_Q = [ya]_Q.$$

So δ_Q is well defined. $\square$

2.47. **Prove** $\bar{\delta}_Q([x]_Q, y) = [xy]_Q$ **by induction.**

As above, the right-end decomposition is legitimate by the Chapter 1 concatenation theorem for $\bar{\delta}$.

Base: $\bar{\delta}_Q([x]_Q, \lambda) = [x]_Q = [x\lambda]_Q$. $\checkmark$

Step: $y = wa$. $\bar{\delta}_Q([x]_Q, wa) = \delta_Q(\bar{\delta}_Q([x]_Q, w), a) = \delta_Q([xw]_Q, a) = [xwa]_Q = [xy]_Q$. $\square$

2.48. **Prove** $L(A_Q) = L$.

$x \in L(A_Q) \Leftrightarrow \bar{\delta}_Q([\lambda]_Q, x) \in F_Q \Leftrightarrow [x]_Q \in F_Q \Leftrightarrow x \in L$ (by definition of F_Q). $\square$

2.49. **Prove or disprove** $Q = R^{A_Q}$.

$R^{A_Q}: xR^{A_Q}y \Leftrightarrow \bar{\delta}_Q([\lambda]_Q, x) = \bar{\delta}_Q([\lambda]_Q, y) \Leftrightarrow [x]_Q = [y]_Q \Leftrightarrow xQy$. So $R^{A_Q} = Q$. $\square$

2.50. **Prove** $R_L = R^{A_{R_L}}$.

By Exercise 2.43 applied with $Q = R_L$: $R^{A_{R_L}} = R_L$. $\square$

2.51. **Show the converse of Theorem 2.3 is false.**

The converse would say: if a language satisfies the conclusion of Theorem 2.3, then it must be FAD. The immediately preceding exercise already provides a counterexample:

$$L = \{\mathbf{a}^n \mathbf{b}^k \mathbf{c}^j \mid n < 3 \text{ or } (n \geq 3 \text{ and } k = j)\}.$$

Part (a) showed that the pumping conclusion of Theorem 2.3 does hold for this language, while the hypothesis fails because L is not FAD.

So a language can satisfy the pumping conclusion without being FAD. Therefore the converse of Theorem 2.3 is false. $\square$

2.52. **Show** $L = \{x : |x|_{\mathbf{a}} = 2|x|_{\mathbf{b}}\}$ **is not FAD.**

For any n : strings $\mathbf{a}^0, \mathbf{a}^2, \mathbf{a}^4, \dots, \mathbf{a}^{2n}$ are in distinct R_L -classes. Indeed: $\mathbf{a}^{2i} R_L \mathbf{a}^{2j}$ requires for all z : $\mathbf{a}^{2i} z \in L \Leftrightarrow \mathbf{a}^{2j} z \in L$. Take $z = \mathbf{b}^i$: $\mathbf{a}^{2i} \mathbf{b}^i \in L$ (yes). $\mathbf{a}^{2j} \mathbf{b}^i \in L$ iff $2j = 2i$, i.e., $j = i$. So distinct $i \neq j$ give distinct classes. $\square$

2.53. **K palindromes: $[\mathbf{110}]_{R_K}$ and description of R_K .**

Let $K = \{w \mid w = w^r\}$.

To describe R_K , suppose $xR_K y$. Take suffixes

$$z_0 = \mathbf{0}x^r, \quad z_1 = \mathbf{1}x^r.$$

Both $xz_0 = x\mathbf{0}x^r$ and $xz_1 = x\mathbf{1}x^r$ are palindromes, so by $xR_K y$, both yz_0 and yz_1 must be palindromes.

Now:

- If a palindrome ends with $0x^r$, it begins with $x0$.
- If a palindrome ends with $1x^r$, it begins with $x1$.

Hence if $|y| \geq |x| + 1$, the first $|x| + 1$ symbols of y would have to be both $x0$ and $x1$, impossible. So $|y| \leq |x|$. By symmetry (since equivalence relations are symmetric), also $|x| \leq |y|$. Therefore $|x| = |y|$. With equal lengths, from yz_0 palindrome we get that the first $|x|$ symbols of yz_0 equal x ; those first $|x|$ symbols are exactly y . So $y = x$.

Thus $xR_K y \Rightarrow x = y$, and the converse is trivial. Therefore R_K is the identity relation on $\{0, 1\}^*$.

So

$$[110]_{R_K} = \{110\}.$$

2.54. Prove or disprove: $R_{L_1} \cap R_{L_2} = R_{L_1 \cap L_2}$.

This is false in general.

However, $R_{L_1} \cap R_{L_2}$ always refines $R_{L_1 \cap L_2}$. Indeed, if $xR_{L_1} y$ and $xR_{L_2} y$, then for every z ,

$$xz \in L_1 \cap L_2 \Leftrightarrow (xz \in L_1 \text{ and } xz \in L_2) \Leftrightarrow (yz \in L_1 \text{ and } yz \in L_2) \Leftrightarrow yz \in L_1 \cap L_2,$$

so $xR_{L_1 \cap L_2} y$.

Sometimes equality does happen. For example, over $\{a, b\}$ let

$$L_1 = \{x : |x| \text{ even}\}, \quad L_2 = \{x : |x|_a \text{ even}\}.$$

Then both $R_{L_1} \cap R_{L_2}$ and $R_{L_1 \cap L_2}$ have the same four classes, determined by the parity of $|x|$ and the parity of $|x|_a$.

But in general the equality can fail. Let $L_1 = \{\lambda\}$ and $L_2 = \{a\}$ over $\{a\}^*$. Then R_{L_1} has 2 classes, R_{L_2} has 3 classes, and so $R_{L_1} \cap R_{L_2}$ has 3 classes. On the other hand,

$$L_1 \cap L_2 = \emptyset,$$

so $R_{L_1 \cap L_2} = R_\emptyset$ has just one class, namely all of Σ^* .

Therefore $R_{L_1} \cap R_{L_2} = R_{L_1 \cap L_2}$ is false in general. $\square$

2.55. Prove: each R -class has a representative of length $< n$.

R is a right congruence of rank n . Consider the DFA A_R with n states. By the pigeonhole principle, the BFS tree of a DFA with n states reaches every state by a path of length at most $n - 1$: any path of length $\geq n$ must revisit a state. Hence every reachable state has a witness string of length $\leq n - 1 < n$, so every R -class has a representative of length $< n$. $\square$

2.56. Find all L with $R_L = R^N$ for R^N from Example 2.6.

Let the four R^N -classes from Example 2.6 be

$$A_0 = [\lambda], \quad A_1 = [1], \quad A_2 = [11], \quad A_3 = [000].$$

Not every union of these classes preserves $R_L = R^N$; many choices collapse states. The unions that keep all four classes pairwise Nerode-distinct are exactly:

$$L = A_0 \cup A_1, \quad L = A_1 \cup A_2, \quad L = A_0 \cup A_3, \quad L = A_2 \cup A_3.$$

These are precisely the languages with $R_L = R^N$.

2.57. **Right congruences for languages of Exercise 1.6.**

In Exercise 1.6(a), let

$$p(x) = (|x|_a \bmod 2, |x|_b \bmod 2) \in \{0, 1\}^2.$$

Appending a suffix adds parities mod 2, so each R_L is determined by which parity information must be remembered.

(a) $L_1 = \{x : (|x|_a \text{ odd}) \wedge (|x|_b \text{ even})\}.$

$$xR_{L_1}y \iff p(x) = p(y).$$

So there are 4 classes (one per parity pair).

(b) $L_2 = \{x : (|x|_a \text{ even}) \vee (|x|_b \text{ odd})\}.$ Again all 4 parity pairs are distinguishable, so

$$xR_{L_2}y \iff p(x) = p(y).$$

(c) $L_3 = \{x : (|x|_a \text{ even}) \vee (|x|_b \text{ even})\}.$ This depends only on whether the two parities are equal or different:

$$xR_{L_3}y \iff (|x|_a + |x|_b \bmod 2) = (|y|_a + |y|_b \bmod 2).$$

So there are 2 classes: “same parity” and “different parity”.

(d) $L_4 = \{x : |x|_a \text{ even}\}.$ Only parity of **a** matters:

$$xR_{L_4}y \iff |x|_a \equiv |y|_a \pmod{2}.$$

So there are 2 classes.

2.58. $L \subseteq \{a\}^*, \lambda R_L a.$

$\lambda R_L a$ means: for all $z \in \{a\}^*, z \in L \iff az \in L$. This means L is closed under left-prepending **a** and left-removing **a** (when applicable). Combined, $a^n \in L \iff a^{n+1} \in L$ for all n . So either $L = \emptyset$ or $L = \{a\}^*$.

2.59. $aR_L aa$ over $\{a\}^*$.

$aR_L aa$: $(\forall z)(az \in L \iff aaz \in L)$. So $a^{n+1} \in L \iff a^{n+2} \in L$. This means membership in L is the same at all lengths ≥ 1 except possibly at length 0 (where λ may or may not be in L). Also $aR_L aa$ means $[a] = [aa]$, so $L \cap \{a^n : n \geq 1\}$ is either $\emptyset$ or $\{a^n : n \geq 1\}$. With the possible inclusion of λ : 4 possibilities: $\emptyset, \{\lambda\}, \{a^n : n \geq 1\}, \{a\}^*$.

2.60. $\lambda R_L aa$ over $\{a\}^*$.

$(\forall z)(z \in L \iff aaz \in L)$. So $a^n \in L \iff a^{n+2} \in L$: membership depends on parity of length. Possibilities: $\emptyset, \{a^{2k} : k \geq 0\}$ (even lengths), $\{a^{2k+1} : k \geq 0\}$ (odd lengths), $\{a\}^*$.

2.61. **Examples with $R^M = R_{L(M)}$ or $R^M \neq R_{L(M)}$.**

(a) $R^M = R_{L(M)}$: any minimal DFA. E.g., the 2-state DFA for $\{a^{2k} : k \geq 0\}$.

(b) $R^M \neq R_{L(M)}$: any non-minimal DFA. E.g., a 3-state DFA for the same language, where two states are equivalent. R^M has 3 classes (one per state) but $R_{L(M)}$ has 2.

- (c) For every DFA M , R^M refines $R_{L(M)}$: if $\bar{\delta}(s_0, x) = \bar{\delta}(s_0, y) = s$, then for any z : $xz \in L(M) \Leftrightarrow \bar{\delta}(s_0, xz) \in F \Leftrightarrow \bar{\delta}(s, z) \in F \Leftrightarrow \bar{\delta}(s_0, yz) \in F \Leftrightarrow yz \in L(M)$.

2.62. **Find R^M and $R_{L(M)}$ for machine of Example 1.5.**

In Example 1.5 (vending machine), states track deposited amount in

$$T = \{0, 5, 10, 15, 20, 25, 30, 35, 40, 45, 50\}$$

(capped at 50), and final states are amounts ≥ 30 .

R^M .

$$xR^M y \iff \bar{\delta}(s_0, x) = \bar{\delta}(s_0, y).$$

So R^M has one class for each amount in T (11 classes).

$R_{L(M)}$. All classes for amounts < 30 remain distinct, while all amounts ≥ 30 merge into one Nerode class.

Why amounts < 30 are distinct: if $t_1 < t_2 < 30$, append coins totaling $30 - t_2$ (possible with $\mathbf{n}, \mathbf{d}, \mathbf{q}$); then from t_2 the result is accepted (reaches 30), while from t_1 it is still < 30 and rejected.

Why amounts ≥ 30 are equivalent: once at least 30 cents has been deposited, any further input keeps the machine in accepting territory.

Hence $R_{L(M)}$ has 7 classes:

$$[0], [5], [10], [15], [20], [25], [\geq 30].$$

So R^M is strictly finer than $R_{L(M)}$.

2.63. **Is $A_{R_{L(A)}}$ always equivalent to A ?**

Yes. $A_{R_{L(A)}}$ is the minimal DFA for $L(A)$, hence $L(A_{R_{L(A)}}) = L(A)$. So they are always equivalent (accept the same language). They need not be isomorphic (if A is not minimal).

2.64. $L_4 = \{y : |y|_0 = |y|_1\}$: **does the example u, v, w show L_4 is FAD?**

No. The pumping lemma says: for each sufficiently long word in a regular language, there exists *some* decomposition that pumps. Finding one convenient decomposition for one word does not prove regularity. To prove non-regularity, we choose one long word and show that *every* allowed decomposition fails. For L_4 , a standard choice is $y = \mathbf{0}^n \mathbf{1}^n$: any v with $|uv| \leq n$ lies in the $\mathbf{0}$ -block, and pumping changes the $\mathbf{0}/\mathbf{1}$ balance, leaving L_4 .

2.65. **Find R_{L_4} .**

$L_4 = \{y : |y|_0 = |y|_1\}$. R_{L_4} is determined by: $xR_{L_4} y \iff |x|_0 - |x|_1 = |y|_0 - |y|_1$ (the “balance” of 0s over 1s must be equal). This gives infinitely many classes (one per integer balance $\in \mathbb{Z}$), confirming L_4 is not FAD.

2.66. **Show postfix expressions over $\{A, B, +, -\}$ are not FAD.**

For any n : A^n (n copies of A) is a prefix of postfix expressions. The strings A^i for $i = 0, 1, \dots, n$ are in distinct R_L -classes: A^i needs i more operands consumed (by operators) to complete a valid postfix expression. Specifically: $A^i z \in L$ iff z is a sequence of $i - 1$ binary operators (plus the right structure). So the future of A^i differs from A^j for $i \neq j$. Infinite rank. Not FAD. $\square$

2.67. **Show parenthesized infix expressions are not FAD.**

Similar: (n is a prefix; each n gives a distinct future (requiring n closing parens eventually). Infinite rank. Not FAD. $\square$

2.68. R_L vs. $R_{\sim L}$.

$R_L = R_{\sim L}$. Proof: $xR_L y \Leftrightarrow (\forall z)(xz \in L \Leftrightarrow yz \in L) \Leftrightarrow (\forall z)(xz \notin L \Leftrightarrow yz \notin L) \Leftrightarrow (\forall z)(xz \in \sim L \Leftrightarrow yz \in \sim L) \Leftrightarrow xR_{\sim L} y$. $\square$

2.69. **Show no L has $R_L = Q$ where Q has classes $\{\lambda\}, \{\mathbf{a}\}, \{\mathbf{b}\}, \{x : |x| \geq 2\}$.**

Q is a right congruence: every class maps consistently under each letter (e.g. $\{\mathbf{a}\} \cdot \sigma$ and $\{\mathbf{b}\} \cdot \sigma$ both land in $\{|x| \geq 2\}$ for any σ). Nevertheless, no language L satisfies $R_L = Q$.

Proof. If $R_L = Q$, then $\mathbf{a}$ and $\mathbf{b}$ must lie in different R_L -classes. For $R_L = Q$ to hold, any candidate L must be a union of Q -classes. Consider $L = \{x : |x| \geq 2\}$. Then $\mathbf{a}R_L \mathbf{b}$: for all z , $\mathbf{a}z \in L \Leftrightarrow |\mathbf{a}z| \geq 2 \Leftrightarrow |z| \geq 1$, and $\mathbf{b}z \in L \Leftrightarrow |z| \geq 1$; the futures are identical. So R_L merges $\{\mathbf{a}\}$ and $\{\mathbf{b}\}$ into one class — contradicting Q . A case analysis over all unions of Q -classes shows the same merging occurs in every case. Hence $R_L \neq Q$ for any L . $\square$

2.70. **Q with classes $\{\lambda, \mathbf{a}, \mathbf{aa}\}$ and $\{\mathbf{a}^3, \mathbf{a}^4, \dots\}$ is not a right congruence.**

- (a) $\mathbf{aa}Q\lambda$ (same class). Append $\mathbf{a}$: $\mathbf{aaa} \in$ second class, but $\lambda \cdot \mathbf{a} = \mathbf{a} \in$ first class. So $\mathbf{aaa} \not Q \mathbf{a}$, violating right congruence.
- (b) Building A_Q : states are the two Q -classes. $\delta([\lambda]_Q, \mathbf{a}) = [?]$. For λ : $\lambda \cdot \mathbf{a} = \mathbf{a} \in [\lambda]_Q$. For $\mathbf{a}$: $\mathbf{a} \cdot \mathbf{a} = \mathbf{aa} \in [\lambda]_Q$. For $\mathbf{aa}$: $\mathbf{aa} \cdot \mathbf{a} = \mathbf{aaa} \in [\mathbf{a}^3]_Q$. So from the same class $[\lambda]_Q$, different elements lead to different successor classes. δ is not well-defined. This is the failure.
- (c) Part (a) says Q is not a right congruence. Part (b) shows δ fails to be a function, which is precisely because right congruence is the condition needed for δ to be well-defined.

2.71. **Show $\{\mathbf{a}^i \mathbf{b}^j \mathbf{c}^k \mid i, j, k \in \mathbb{N}, i + j = k\}$ is not FAD.**

For any n : consider $\mathbf{a}^m$ for $m = 0, 1, \dots, n$. $\mathbf{a}^m R_L \mathbf{a}^{m'}$: need $(\forall z) \mathbf{a}^m z \in L \Leftrightarrow \mathbf{a}^{m'} z \in L$. Take $z = \mathbf{b}^0 \mathbf{c}^m = \mathbf{c}^m$: $\mathbf{a}^m \mathbf{c}^m \in L \Leftrightarrow m + 0 = m$ (yes). $\mathbf{a}^{m'} \mathbf{c}^m \in L \Leftrightarrow m' + 0 = m \Leftrightarrow m' = m$. So for $m \neq m'$, $\mathbf{a}^m \not R_L \mathbf{a}^{m'}$. Infinite rank. Not FAD. $\square$

2.72. **Show Q with classes $[\lambda] = \{\lambda\}, [\mathbf{a}] = \{\mathbf{a}\}, [\mathbf{aa}] = \{\mathbf{a}\}^{\geq 2}$ is a right congruence on $\{\mathbf{a}\}^*$.**

Case $x, y \in [\lambda]$: $x = y = \lambda$, $xa = ya = \mathbf{a} \in [\mathbf{a}]$.

Case $x, y \in [\mathbf{a}]$: $x = y = \mathbf{a}$, $xa = ya = \mathbf{aa} \in [\mathbf{aa}]$.

Case $x, y \in [\mathbf{aa}]$: $|x|, |y| \geq 2$, so $|xa|, |ya| \geq 3 \geq 2$, thus $xa, ya \in [\mathbf{aa}]$.

In each case $xaQya$. $\square$

2.73. **R_2 on $\{\mathbf{a}, \mathbf{b}, \mathbf{c}\}^*$: same last letter.**

- (a) Equivalence: Reflexive (x ends with same letter as x). Symmetric (same). Transitive (same). But for λ : λ has no last letter. By convention, λ is in its own class (or we restrict to Σ^+). Assuming λ is its own class: $\lambda R_2 \lambda$, and for $x, y \in \Sigma^+$, $xR_2 y \Leftrightarrow \text{last}(x) = \text{last}(y)$.
- (b) Right congruence: If $xR_2 y$ (same last letter, say c), then for any $z \in \Sigma^+$: $\text{last}(xz) = \text{last}(z) = \text{last}(yz)$, so $xzR_2 yz$. For $z = \lambda$: $xR_2 y$. $\square$

2.74. R_3 on $\{a, b, c\}^*$: same first letter.

- (a) Equivalence: reflexive, symmetric, transitive by same argument (λ its own class).
- (b) Right congruence: Suppose xR_3y , i.e., x and y have the same first letter (or both are λ). For any $z \in \Sigma^*$: if $z = \lambda$ then $xz = x$ and $yz = y$, so xzR_3yz trivially. If $z \neq \lambda$ then $\text{first}(xz) = \text{first}(x) = \text{first}(y) = \text{first}(yz)$, so xzR_3yz . Hence R_3 is a right congruence. $\square$

2.75. Which of $L_1, \dots, L_5$ are FAD?

- (a) L_1 = last letter matches first letter: FAD. Use 5 states: q_ϵ (no input yet), and $(a, a), (a, b), (b, a), (b, b)$ where the first component is the first symbol seen and the second is the current last symbol. Final states are (a, a) and (b, b) (and include/exclude q_ϵ depending on the convention for λ).
- (b) L_2 = odd-length words whose first letter equals the center letter: Not FAD. Suppose L_2 were FAD, and let p be a pumping length. Take

$$w = \mathbf{a}^{p+1}\mathbf{b}^p.$$

This word has odd length $2p + 1$, its first letter is $\mathbf{a}$, and its center letter is also $\mathbf{a}$, so $w \in L_2$.

For any split $w = uvw'$ with $|uv| \leq p$ and $|v| \geq 1$, the substring v lies entirely in the initial $\mathbf{a}$ -block. So $v = \mathbf{a}^k$ for some $k \geq 1$. Pumping down gives

$$uw' = \mathbf{a}^{p+1-k}\mathbf{b}^p.$$

If k is odd, then $|uw'| = 2p + 1 - k$ is even, so $uw' \notin L_2$. If k is even, then $|uw'|$ is odd, but its center position is

$$\frac{|uw'| + 1}{2} = p + 1 - \frac{k}{2},$$

which is strictly larger than the length $p + 1 - k$ of the remaining $\mathbf{a}$ -block. So the center letter lies in the $\mathbf{b}$ -block, while the first letter is still $\mathbf{a}$. Hence again $uw' \notin L_2$.

This contradicts Theorem 2.3. Therefore L_2 is not FAD.

- (c) L_3 = last letter does not appear in any earlier position: **FAD**. The DFA tracks (i) the set of characters seen in positions $1, \dots, n - 1$ and (ii) the current last character. Since Σ is fixed and finite, there are at most $2^{|\Sigma|} \times |\Sigma|$ states (plus a start state for λ). Accept when the last character is not in the set of characters seen before it. Both components update in one step: reading a sets last-char = a and adds the previous last-char to the "seen" set. This is a finite machine.
- (d) L_4 = even-length, two center letters match: Not FAD. Suppose L_4 were FAD, and let p be a pumping length. Consider

$$w = \mathbf{a}^{2p-1}\mathbf{b}(\mathbf{ba})^p.$$

This word has length $4p$, so it is even. Its two center letters are the two consecutive $\mathbf{b}$'s, so $w \in L_4$.

For any split $w = uvw'$ with $|uv| \leq p$ and $|v| \geq 1$, we have $v = \mathbf{a}^k$ for some $k \geq 1$, because the first p symbols are all $\mathbf{a}$'s. Pumping down gives

$$uw' = \mathbf{a}^{2p-1-k}\mathbf{b}(\mathbf{ba})^p.$$

If k is odd, then $|uw'| = 4p - k$ is odd, so $uw' \notin L_4$. If $k = 2r$ is even, then the center positions of uw' are

$$2p - r \quad \text{and} \quad 2p - r + 1.$$

After the initial block $\mathbf{a}^{2p-1-2r}$, the remaining suffix is

$$\mathbf{b}(\mathbf{ba})^p = \mathbf{bbaba} \dots$$

Those two center positions become positions $r + 1$ and $r + 2$ within this suffix, and these consecutive symbols are always different. So the two center letters of uw' do not match, and $uw' \notin L_4$.

This contradicts Theorem 2.3. Therefore L_4 is not FAD.

- (e) L_5 = odd-length, center letter matches none of the others: Not FAD. Suppose L_5 were FAD, and let p be a pumping length. Take

$$w = \mathbf{a}^p \mathbf{ba}^p.$$

This word has odd length, its center letter is $\mathbf{b}$, and that $\mathbf{b}$ appears nowhere else, so $w \in L_5$.

For any split $w = uvw'$ with $|uv| \leq p$ and $|v| \geq 1$, again $v = \mathbf{a}^k$ for some $k \geq 1$. Pumping down gives

$$uw' = \mathbf{a}^{p-k} \mathbf{ba}^p.$$

If k is odd, then $|uw'| = 2p + 1 - k$ is even, so $uw' \notin L_5$. If $k = 2r$ is even, then the center position of uw' is

$$\frac{|uw'| + 1}{2} = p + 1 - r.$$

But the unique $\mathbf{b}$ is at position $p - 2r + 1$, so the center lies strictly to the right of that $\mathbf{b}$, inside the final $\mathbf{a}$ -block. Thus the center letter is $\mathbf{a}$, which certainly matches other letters of the word. Hence $uw' \notin L_5$.

This contradicts Theorem 2.3. Therefore L_5 is not FAD.

2.76. Complete proof of case 1 in (2) $\Rightarrow$ (3) of Nerode's theorem.

- (a) Case 1: $x = \lambda$. We need $\lambda R_L y$ (for some y with $\bar{\delta}(s_0, y) = \bar{\delta}(s_0, \lambda) = s_0$). Since $\bar{\delta}(s_0, \lambda) = s_0 = \bar{\delta}(s_0, y)$, for all z : $\bar{\delta}(s_0, \lambda z) = \bar{\delta}(s_0, z) = \bar{\delta}(s_0, yz)$ (using $\bar{\delta}(s_0, y) = s_0$ and the homomorphism property of $\bar{\delta}$). So $\lambda z \in L \Leftrightarrow yz \in L$, i.e., $\lambda R_L y$.
- (b) Case 1 *can* be included under case 2 if we allow x to be a prefix of y with $|x| = 0$: the proof of case 2 (induction on $|x|$) can start at $|x| = 0$ (the λ case), so yes.

2.77. Right congruence property can use $\Leftrightarrow$ instead of $\Rightarrow$.

The original right congruence (RC): $xRy \Rightarrow xuRyu$.

Claim: $xRy \Leftrightarrow xuRyu$ (for all u).

The $\Rightarrow$ direction is given. For $\Leftarrow$: assume $xuRyu$ for all u ; take $u = \lambda$: $x\lambda R y\lambda$, i.e., xRy (since $x\lambda = x$ and $y\lambda = y$). $\square$

2.78. Show $\bar{\delta}(s, uv^i) = q$ by induction.

Given $\bar{\delta}(s, u) = q$ and $\bar{\delta}(q, v) = q$.

Base ($i = 0$): $\bar{\delta}(s, uv^0) = \bar{\delta}(s, u) = q$. $\checkmark$

Step: $\bar{\delta}(s, uv^{i+1}) = \bar{\delta}(\bar{\delta}(s, uv^i), v) = \bar{\delta}(q, v) = q$ (by IH). $\square$

2.79. $L = \text{strings agreeing with } 0^1 10^2 10^3 1 \dots \text{ is not FAD.}$

The strings in L are all prefixes of this infinite sequence. Consider prefix $0^1 10^2 1 \dots 0^n 1$ (length $n(n+3)/2$) and the prefix $0^1 10^2 1 \dots 0^n$ (length $n(n+3)/2 - n = n(n+1)/2$). The futures of these two strings are different (one expects a 1 next, the other expects $n+1$ zeros followed by 1). So different-length prefixes that cut at different points in the pattern have different futures. An unbounded number of distinct futures exist. Not FAD. $\square$

2.80. **Show the “simple” BNF language is not FAD.**

$\langle \text{simple} \rangle ::= \mathbf{a} \mid (\langle \text{simple} \rangle)$. The language is $L = \{(\mathbf{a})^n \mid n \geq 0\}$. By Myhill-Nerode: the strings $(^0, (^1, \dots, (^n$ are pairwise in distinct R_L -classes. The distinguishing suffix for $(^j$ vs. $(^k$ (with $j \neq k$) is $z = \mathbf{a})^j$: $(^j \mathbf{a})^j \in L$ but $(^k \mathbf{a})^j \notin L$ (since $k \neq j$). Hence R_L has infinite rank and L is not FAD. $\square$

2.81. L_2 (binary powers of 2) is FAD; L_{10} (decimal powers of 2) is not.

- (a) $L_2 = \{1, 10, 100, 1000, \dots\}$: these are exactly the strings whose first symbol is 1 and whose remaining symbols, if any, are all 0. This is FAD: use a 2-state DFA with $q_0 \xrightarrow{1} q_1$, $q_1 \xrightarrow{0} q_1$, $F = \{q_1\}$, and a dead state for all other transitions.
- (b) $L_{10} = \{1, 2, 4, 8, 16, 32, \dots\}$ (decimal powers of 2) is not FAD. *Proof (by pumping lemma and modular arithmetic).* Assume L_{10} is FAD with pumping length p . Choose n so large that if w is the decimal representation of 2^n , then

$$|w| = d > p + 2$$

and

$$4 \cdot 5^{d-p-1} > (p+1) \log_2 10.$$

Write $w = xyz$ with $|xy| \leq p$, $|y| = t \geq 1$, and let $|z| = r$. Then $r = d - |xy| \geq d - p$.

By pumping, $xy^2z \in L_{10}$, so for some $m > n$,

$$\text{val}(xy^2z) = 2^m, \quad \text{val}(xyz) = 2^n.$$

Because pumping duplicates a block before the final z , both numbers end with the same r digits z . Hence

$$2^m \equiv 2^n \pmod{10^r},$$

so

$$(*) \quad 2^{m-n} \equiv 1 \pmod{5^r}.$$

Lemma. If $2^q \equiv 1 \pmod{5^r}$, then $4 \cdot 5^{r-1} \mid q$.

Why: the order of 2 modulo 5^r is $4 \cdot 5^{r-1}$. Base $r = 1$: order mod 5 is 4. Induction step: if the order mod 5^r is $4 \cdot 5^{r-1}$, then

$$2^{4 \cdot 5^{r-1}} = 1 + u5^r, \quad 5 \nmid u,$$

so

$$2^{4 \cdot 5^r} = (1 + u5^r)^5 \equiv 1 + u5^{r+1} \not\equiv 1 \pmod{5^{r+2}},$$

which shows the order at the next level is multiplied by 5. Thus $\text{order}(2 \bmod 5^r) = 4 \cdot 5^{r-1}$ for all $r \geq 1$.

Applying the lemma to (*) gives

$$(1) \quad m - n \geq 4 \cdot 5^{r-1} \geq 4 \cdot 5^{d-p-1}.$$

Now get an upper bound for $m - n$ from decimal length. The pumped string xy^2z has length $d + t \leq d + p$, so

$$2^m = \text{val}(xy^2z) < 10^{d+t} \leq 10^{d+p}.$$

Also $2^n = \text{val}(w) \geq 10^{d-1}$. Therefore

$$2^{m-n} < 10^{p+1},$$

so

$$(2) \quad m - n < (p + 1) \log_2 10.$$

(1) and (2) contradict our choice of n . Hence L_{10} is not FAD. $\square$

Chapter 3

Minimization — Solutions

3.1. **Prove** $\mu(\bar{\delta}(s, x)) = \bar{\delta}_{R_L}(\mu(s), x)$ **by induction for the mapping μ in Theorem 3.1.**

$\mu(s) = [s]_{R^M}$ (the R^M -class of state s , identified with an R_L -class via the homomorphism). Let $P(n)$: $(\forall s \in S)(\forall x \in \Sigma^n)(\mu(\bar{\delta}(s, x)) = \bar{\delta}_{R_L}(\mu(s), x))$.

Base ($n = 0$): $\mu(\bar{\delta}(s, \lambda)) = \mu(s) = \bar{\delta}_{R_L}(\mu(s), \lambda)$. ✓

Step: Let $x = ay$ where $a \in \Sigma$ is a single letter on the left and $y \in \Sigma^*$ is a string of length n on the right. This is the tail-recursive form of Definition 1.11iii. Using that definition at the beginning and end, together with the induction hypothesis for the state $\delta(s, a)$ and the string y , and with the homomorphism property $\mu(\delta(t, a)) = \delta_{R_L}(\mu(t), a)$,

$$\mu(\bar{\delta}(s, ay)) = \mu(\bar{\delta}(\delta(s, a), y)) = \bar{\delta}_{R_L}(\mu(\delta(s, a)), y) = \bar{\delta}_{R_L}(\delta_{R_L}(\mu(s), a), y) = \bar{\delta}_{R_L}(\mu(s), ay).$$

□

3.2. **Show** $\bar{\delta}_{E_A}([s]_{E_A}, x) = [\bar{\delta}(s, x)]_{E_A}$ **by induction.**

Base ($x = \lambda$): $\bar{\delta}_{E_A}([s]_{E_A}, \lambda) = [s]_{E_A} = [\bar{\delta}(s, \lambda)]_{E_A}$. ✓

Step: Let $x = ay$ where $a \in \Sigma$ is a single letter and $y \in \Sigma^*$ is the remaining string. By the tail-recursive clause in Definition 1.11iii,

$$\bar{\delta}_{E_A}([s]_{E_A}, ay) = \bar{\delta}_{E_A}(\bar{\delta}_{E_A}([s]_{E_A}, a), y) = \bar{\delta}_{E_A}([\bar{\delta}(s, a)]_{E_A}, y) = [\bar{\delta}(\bar{\delta}(s, a), y)]_{E_A} = [\bar{\delta}(s, ay)]_{E_A}. \quad \square$$

3.3. **Prove** $E_A = \bigcap_{j=0}^{\infty} E_{jA}$.

Define E_{0A} : $sE_{0A}t \Leftrightarrow (s \in F \Leftrightarrow t \in F)$.

Define $E_{i+1,A}$: $sE_{i+1,A}t \Leftrightarrow sE_{iA}t \wedge (\forall a \in \Sigma)(\delta(s, a)E_{iA}\delta(t, a))$.

E_A (state equivalence): $sE_At \Leftrightarrow L(A^s) = L(A^t)$ where A^s is A with start state s .

($\subseteq$): We show $E_A \subseteq E_{jA}$ for every j by induction on j .

Base ($j = 0$): if sE_At , then $L(A^s) = L(A^t)$. In particular,

$$\lambda \in L(A^s) \Leftrightarrow \lambda \in L(A^t),$$

so

$$s \in F \Leftrightarrow t \in F.$$

Hence $sE_{0A}t$.

Step: Clearly, if $sE_A t$, then $L(A^s) = L(A^t)$. For every $a \in \Sigma$,

$$L(A^{\delta(s,a)}) = \{x \in \Sigma^* : ax \in L(A^s)\} = \{x \in \Sigma^* : ax \in L(A^t)\} = L(A^{\delta(t,a)}),$$

so $\delta(s,a)E_A \delta(t,a)$. By the induction hypothesis,

$$sE_{jA}t \quad \text{and} \quad \delta(s,a)E_{jA}\delta(t,a) \quad (\forall a \in \Sigma).$$

Therefore $sE_{j+1,A}t$. Thus $E_A \subseteq E_{jA}$ for every j , and so

$$E_A \subseteq \bigcap_{j=0}^{\infty} E_{jA}.$$

($\supseteq$): Suppose $sE_{jA}t$ for every j . We claim that for every $n \geq 0$ and every string x with $|x| \leq n$,

$$\bar{\delta}(s,x) \in F \Leftrightarrow \bar{\delta}(t,x) \in F.$$

We prove this by induction on n .

Base ($n = 0$): the only string of length at most 0 is λ . Since $sE_{0A}t$, we have

$$s \in F \Leftrightarrow t \in F,$$

that is,

$$\bar{\delta}(s,\lambda) \in F \Leftrightarrow \bar{\delta}(t,\lambda) \in F.$$

Step: Clearly, assume the claim holds for n . Let $|x| \leq n+1$. If $x = \lambda$, then $sE_{n+1,A}t$ implies $sE_{nA}t$, so the induction hypothesis already gives the result. If $x = ay$ with $a \in \Sigma$ and $|y| \leq n$, then $sE_{n+1,A}t$ implies

$$\delta(s,a)E_{nA}\delta(t,a).$$

Applying the induction hypothesis to the pair $\delta(s,a), \delta(t,a)$ and the string y , we get

$$\bar{\delta}(\delta(s,a), y) \in F \Leftrightarrow \bar{\delta}(\delta(t,a), y) \in F.$$

Since

$$\bar{\delta}(\delta(s,a), y) = \bar{\delta}(s, ay) \quad \text{and} \quad \bar{\delta}(\delta(t,a), y) = \bar{\delta}(t, ay),$$

the claim follows.

Now let $x \in \Sigma^*$ be arbitrary and take $n = |x|$. Then

$$\bar{\delta}(s,x) \in F \Leftrightarrow \bar{\delta}(t,x) \in F,$$

which means

$$x \in L(A^s) \Leftrightarrow x \in L(A^t).$$

Therefore $L(A^s) = L(A^t)$, i.e., $sE_A t$. So

$$\bigcap_{j=0}^{\infty} E_{jA} \subseteq E_A.$$

Hence $E_A = \bigcap_{j=0}^{\infty} E_{jA}$. $\square$

3.4. Show δ_{E_A} in Definition 3.8 is well defined.

For every class $[s]_{E_A}$ and letter $a \in \Sigma$, the state $\delta(s, a)$ exists, so the class $[\delta(s, a)]_{E_A}$ exists as well. Thus $\delta_{E_A}([s]_{E_A}, a) = [\delta(s, a)]_{E_A}$ is defined everywhere. To show it is not multiply-defined, suppose $[s]_{E_A} = [t]_{E_A}$, i.e., $sE_A t$. Then $\delta(s, a)E_A \delta(t, a)$ (proved in Exercise 3.3, $\supseteq$ direction): $L(A^{\delta(s, a)}) = \{x : ax \in L(A^s)\} = \{x : ax \in L(A^t)\} = L(A^{\delta(t, a)})$. So $[\delta(s, a)]_{E_A} = [\delta(t, a)]_{E_A}$. $\square$

3.5. Show F_{E_A} is well defined.

Every E_A -class has at least one representative, so the description $F_{E_A} = \{[s]_{E_A} \mid s \in F\}$ addresses every class. To show membership is not multiply-defined, suppose $[s]_{E_A} = [t]_{E_A}$ and $s \in F$. Then $sE_A t \Rightarrow L(A^s) = L(A^t) \Rightarrow \lambda \in L(A^s) \Leftrightarrow \lambda \in L(A^t) \Rightarrow s \in F \Leftrightarrow t \in F$. So $t \in F$, and the definition is independent of the chosen representative. $\square$

3.6. Show δ^c maps into S^c .

$S^c = \text{connected states} = \{\bar{\delta}(s_0, x) \mid x \in \Sigma^*\}$. δ^c is δ restricted to S^c . For any $s \in S^c$ and $a \in \Sigma$: $s = \bar{\delta}(s_0, x)$ for some x , so $\delta(s, a) = \delta(\bar{\delta}(s_0, x), a) = \bar{\delta}(s_0, xa) \in S^c$. $\square$

3.7. Prove Lemma 3.1.

Suppose $sE_{m+1A} t$. By definition, this means

$$(\forall x \in \Sigma^* \ni |x| \leq m+1)(\bar{\delta}(s, x) \in F \Leftrightarrow \bar{\delta}(t, x) \in F).$$

In particular, the same equivalence holds for every x with $|x| \leq m$. Hence

$$(\forall x \in \Sigma^* \ni |x| \leq m)(\bar{\delta}(s, x) \in F \Leftrightarrow \bar{\delta}(t, x) \in F),$$

which is exactly $sE_{mA} t$. Therefore $E_{m+1A} \subseteq E_{mA}$. $\square$

3.8. Prove Lemma 3.3.

By definition,

$$sE_{0A} t \Leftrightarrow (\forall x \in \Sigma^* \ni |x| \leq 0)(\bar{\delta}(s, x) \in F \Leftrightarrow \bar{\delta}(t, x) \in F).$$

The only string of length 0 is λ , so

$$sE_{0A} t \Leftrightarrow \bar{\delta}(s, \lambda) \in F \Leftrightarrow \bar{\delta}(t, \lambda) \in F \Leftrightarrow s \in F \Leftrightarrow t \in F.$$

Therefore the equivalence classes of E_{0A} are exactly F and $S - F$, unless one of these sets is empty, in which case there is only the single class S . $\square$

3.9. Prove Theorem 3.9.

Assume $E_{mA} = E_{m+1A}$. We prove by induction on k that $E_{m+k,A} = E_{mA}$.

Base case ($k = 0$): $E_{m+0,A} = E_{mA}$ trivially.

Inductive step: assume $E_{m+k,A} = E_{mA}$. Let $s, t \in S$. By Theorem 3.8,

$$\begin{aligned} sE_{m+k+1,A} t &\Leftrightarrow sE_{m+k,A} t \wedge (\forall a \in \Sigma)(\delta(s, a)E_{m+k,A} \delta(t, a)) \\ &\Leftrightarrow sE_{mA} t \wedge (\forall a \in \Sigma)(\delta(s, a)E_{mA} \delta(t, a)) \end{aligned}$$

by the induction hypothesis. Applying Theorem 3.8 again,

$$sE_{m+k+1,A}t \Leftrightarrow sE_{m+1,A}t.$$

Since $E_{m+1A} = E_{mA}$ by hypothesis, this gives

$$sE_{m+k+1,A}t \Leftrightarrow sE_{mA}t.$$

Therefore $E_{m+k+1,A} = E_{mA}$. By induction, $(\forall k \in \mathbb{N})(E_{m+k,A} = E_{mA})$. $\square$

3.10. Prove $\mu(\bar{\delta}_A(s, x)) = \bar{\delta}_B(\mu(s), x)$ by induction for a homomorphism μ .

Base: $\mu(\bar{\delta}_A(s, \lambda)) = \mu(s) = \bar{\delta}_B(\mu(s), \lambda)$. $\checkmark$

Step: Let $x = ay$ where $a \in \Sigma$ is a single letter and $y \in \Sigma^*$ is the remaining string. By the tail-recursive clause in Definition 1.11iii,

$$\mu(\bar{\delta}_A(s, ay)) = \mu(\bar{\delta}_A(\delta_A(s, a), y)) = \bar{\delta}_B(\mu(\delta_A(s, a)), y) = \bar{\delta}_B(\delta_B(\mu(s), a), y) = \bar{\delta}_B(\mu(s), ay),$$

where the third equality is the induction hypothesis, the fourth is the homomorphism property, and the last is again Definition 1.11iii. $\square$

3.11. Prove $L(A) = L(B)$ for a homomorphism $\mu : A \rightarrow B$.

For any $x \in \Sigma^*$,

$$\begin{aligned} x \in L(A) &\Leftrightarrow \bar{\delta}_A(s_{0A}, x) \in F_A \\ &\Leftrightarrow \mu(\bar{\delta}_A(s_{0A}, x)) \in F_B \quad (\text{because } \mu(s) \in F_B \Leftrightarrow s \in F_A) \\ &\Leftrightarrow \bar{\delta}_B(\mu(s_{0A}), x) \in F_B \quad (\text{by Exercise 3.10}) \\ &\Leftrightarrow \bar{\delta}_B(s_{0B}, x) \in F_B \quad (\text{since } \mu(s_{0A}) = s_{0B}) \\ &\Leftrightarrow x \in L(B). \end{aligned}$$

Therefore $L(A) = L(B)$. $\square$

3.12. Examples with connectivity and A/E_A .

(a) A not connected, A/E_A not connected: take

$$A = \langle \{\mathbf{a}\}, \{s_0, s_1, s_2\}, s_0, \delta, \{s_0, s_1\} \rangle,$$

where

$$\delta(s_0, \mathbf{a}) = s_1, \quad \delta(s_1, \mathbf{a}) = s_1, \quad \delta(s_2, \mathbf{a}) = s_2.$$

The state s_2 is unreachable, so A is not connected. Also $s_0 E_A s_1$, because from either state every continuation is accepted, while s_2 is not equivalent to them because from s_2 every continuation is rejected. Thus A/E_A has exactly the two states $[s_0] = [s_1]$ and $[s_2]$, and $[s_2]$ is still unreachable. Hence A/E_A is not connected.

(b) A not connected, A/E_A connected: take

$$A = \langle \{\mathbf{a}\}, \{t_0, t_1, t_2\}, t_0, \delta, \{t_1, t_2\} \rangle,$$

where

$$\delta(t_0, \mathbf{a}) = t_1, \quad \delta(t_1, \mathbf{a}) = t_1, \quad \delta(t_2, \mathbf{a}) = t_2.$$

Again t_2 is unreachable, so A is not connected. But now $t_1 E_A t_2$, because from either state every continuation is accepted. Therefore A/E_A has just the two states $[t_0]$ and $[t_1] = [t_2]$, and both are reachable from the start class: $[t_0]$ is the start state and

$$\delta([t_0], \mathbf{a}) = [t_1].$$

Hence A/E_A is connected.

3.13. Show there exists a homomorphism from A to A/E_A .

Define $\mu : S_A \rightarrow S_{E_A}$ by $\mu(s) = [s]_{E_A}$. Then:

- $\mu(s_{0A}) = [s_{0A}]_{E_A} = s_{0,E_A}$. ✓
- $\mu(\delta_A(s, a)) = [\delta_A(s, a)]_{E_A} = \delta_{E_A}([s]_{E_A}, a) = \delta_{E_A}(\mu(s), a)$. ✓
- $s \in F_A \Leftrightarrow [s]_{E_A} \in F_{E_A}$, i.e., $\mu(s) \in F_{E_A} \Leftrightarrow s \in F_A$. ✓

So μ is a homomorphism. $\square$

3.14. Show there exists a homomorphism from connected A to $A_{R_{L(A)}}$.

(a) Define $\psi : S \rightarrow \{R_{L(A)}\text{-classes}\}$ by $\psi(s) = [x_s]_{R_{L(A)}}$ where x_s is any string with $\bar{\delta}(s_{0A}, x_s) = s$ (well-defined by part (b)).

More precisely: since A is connected, for each $s \in S$ there exists $x_s \in \Sigma^*$ with $\bar{\delta}(s_{0A}, x_s) = s$. Define $\psi(s) = [x_s]_{R_{L(A)}}$.

(b) Well-definedness: it is defined everywhere because connectedness gives, for every $s \in S$, at least one string x_s with $\bar{\delta}(s_{0A}, x_s) = s$. It is not multiply-defined because if $\bar{\delta}(s_0, x) = \bar{\delta}(s_0, y) = s$, then $xR^A y$ (same state), and R^A refines $R_{L(A)}$, so $xR_{L(A)} y$, giving $[x]_{R_{L(A)}} = [y]_{R_{L(A)}}$. $\square$

(c) Homomorphism:

- $\psi(s_{0A}) = [\lambda]_{R_{L(A)}} = s_{0,R_{L(A)}}$. ✓
- $\psi(\delta(s, a)) = [x_s a]_{R_{L(A)}} = \delta_{R_{L(A)}}([x_s]_{R_{L(A)}}, a) = \delta_{R_{L(A)}}(\psi(s), a)$. ✓
- $s \in F \Leftrightarrow \lambda \in L(A^s) \Leftrightarrow x_s \in L(A) \Leftrightarrow [x_s]_{R_{L(A)}} \in F_{R_{L(A)}} \Leftrightarrow \psi(s) \in F_{R_{L(A)}}$. ✓

3.15. Show there may not be a homomorphism $A \rightarrow A_{R_{L(A)}}$ if A is not connected.

Take $\Sigma = \{\mathbf{0}\}$ and let

$$A = \langle \{\mathbf{0}\}, \{s_0, s_1\}, s_0, \delta, \{s_1\} \rangle$$

where $\delta(s_0, \mathbf{0}) = s_0$ and $\delta(s_1, \mathbf{0}) = s_1$. The state s_1 is unreachable, so A is not connected. Since the reachable part is just the nonfinal loop at s_0 , we have $L(A) = \emptyset$. Therefore $A_{R_{L(A)}}$ is the one-state DFA for $\emptyset$, with a single nonfinal state q and $\delta(q, \mathbf{0}) = q$.

If there were a homomorphism $\psi : A \rightarrow A_{R_{L(A)}}$, then $\psi(s_0) = q$ because s_0 is the start state. But a homomorphism must also preserve finality:

$$s_1 \in F_A \Leftrightarrow \psi(s_1) \in F_{R_{L(A)}}.$$

This is impossible, because q is the only state of $A_{R_{L(A)}}$ and q is not final. Hence no such homomorphism exists.

3.16. Show there may still be a homomorphism $A \rightarrow A_{R_{L(A)}}$ if A is not connected.

Take $\Sigma = \{0\}$ and let

$$A = \langle \{0\}, \{s_0, s_1\}, s_0, \delta, \{s_0, s_1\} \rangle$$

where $\delta(s_0, 0) = s_0$ and $\delta(s_1, 0) = s_1$. The state s_1 is unreachable, so A is not connected, but $L(A) = \Sigma^*$ because every string keeps the start state s_0 in a final state.

Therefore $A_{R_{L(A)}}$ is the one-state DFA

$$\langle \{0\}, \{q\}, q, \delta', \{q\} \rangle$$

with $\delta'(q, 0) = q$. Define $\psi : A \rightarrow A_{R_{L(A)}}$ by

$$\psi(s_0) = q, \quad \psi(s_1) = q.$$

This is a homomorphism: the start state is preserved, finality is preserved because all three states involved are final, and

$$\psi(\delta(s_i, 0)) = \psi(s_i) = q = \delta'(q, 0) = \delta'(\psi(s_i), 0)$$

for $i = 0, 1$.

3.17. Show there need not be a homomorphism from A_{R_L} to A_R .

Let $\Sigma = \{a\}$ and let Q be the right congruence with two classes

$$[\lambda]_Q = \{\lambda\}, \quad [a]_Q = \{x \in \Sigma^* : |x| \geq 1\}.$$

Let R refine Q by splitting the nonempty strings according to parity:

$$[\lambda]_R = \{\lambda\}, \quad [a]_R = \{x : |x| \text{ is odd}\}, \quad [aa]_R = \{x : |x| \text{ is even and } |x| \geq 2\}.$$

Take $L = \{\lambda\}$, which is a union of Q -classes. Then A_Q has two states,

$$q_0 = [\lambda]_Q \quad (\text{start and final}), \quad q_1 = [a]_Q$$

with $\delta_Q(q_0, a) = q_1$ and $\delta_Q(q_1, a) = q_1$.

The DFA A_R has three states,

$$r_0 = [\lambda]_R \quad (\text{start and final}), \quad r_1 = [a]_R, \quad r_2 = [aa]_R,$$

with transitions

$$\delta_R(r_0, a) = r_1, \quad \delta_R(r_1, a) = r_2, \quad \delta_R(r_2, a) = r_1.$$

Assume there is a homomorphism $\psi : A_Q \rightarrow A_R$. Since start states must map to start states, $\psi(q_0) = r_0$. Then

$$\psi(q_1) = \psi(\delta_Q(q_0, \mathbf{a})) = \delta_R(\psi(q_0), \mathbf{a}) = \delta_R(r_0, \mathbf{a}) = r_1.$$

But now

$$\psi(q_1) = \psi(\delta_Q(q_1, \mathbf{a})) = \delta_R(\psi(q_1), \mathbf{a}) = \delta_R(r_1, \mathbf{a}) = r_2,$$

which says $r_1 = r_2$, a contradiction. Therefore there need not be a homomorphism from A_Q to A_R .

3.18. Is $\cong$ appropriate to ask if it's a right congruence?

$\cong$ is a relation on DFAs, not on Σ^* . Right congruences are relations on Σ^* . So asking whether $\cong$ (between DFAs) is a right congruence (on strings) is a category error – it makes no sense. One could ask whether the relation induced on strings by $\cong$ is a right congruence, but the question itself is ill-formed in the original sense.

3.19. Is E_A a right congruence?

E_A is a relation on states S , not on Σ^* . So strictly speaking, E_A is not a right congruence (which is a relation on Σ^*). However, R^A (induced on strings) is a right congruence, and E_A induces R^A via $xR^A y \Leftrightarrow \bar{\delta}(s_0, x)E_A\bar{\delta}(s_0, y)$.

3.20. If A is reduced then A^c is reduced.

A^c = connected part of A . States of A^c are $S^c \subseteq S$. If A is reduced, no two distinct states in S are equivalent; in particular, no two distinct states in $S^c \subseteq S$ are equivalent. So A^c is reduced. $\square$

3.21. For a homomorphism $\mu : S_A \rightarrow S_B$, prove $\mu(s)E_B\mu(t) \Leftrightarrow sE_A t$.

($\Rightarrow$) Suppose $\mu(s)E_B\mu(t)$, i.e., $L(B^{\mu(s)}) = L(B^{\mu(t)})$. By Exercise 3.10: $L(A^s) = L(B^{\mu(s)})$ and $L(A^t) = L(B^{\mu(t)})$ (since μ is a homomorphism from A^s to $B^{\mu(s)}$). So $L(A^s) = L(A^t)$, i.e., $sE_A t$.

($\Leftarrow$) Suppose $sE_A t$. Then $L(A^s) = L(A^t)$. By the homomorphism property: $L(B^{\mu(s)}) = L(A^s) = L(A^t) = L(B^{\mu(t)})$, so $\mu(s)E_B\mu(t)$. $\square$

3.22. Define $\psi : M^c \rightarrow A_{R^M}$ and prove it is an isomorphism.

(a) For each state $s \in S^c$, choose a string $x_s \in \Sigma^*$ with

$$\bar{\delta}(s_0, x_s) = s,$$

and define

$$\psi(s) = [x_s]_{R^M}.$$

(b) *Well-defined*: this is defined everywhere on S^c because, by definition of the connected part, each $s \in S^c$ has at least one string reaching it. If x and y both satisfy

$$\bar{\delta}(s_0, x) = \bar{\delta}(s_0, y) = s,$$

then by definition of R^M we have $xR^M y$. Hence

$$[x]_{R^M} = [y]_{R^M},$$

so $\psi(s)$ does not depend on which string reaching s was chosen.

(c) *Homomorphism*: first,

$$\psi(s_0) = [\lambda]_{R^M},$$

because λ reaches s_0 . Now let $s \in S^c$ and $a \in \Sigma$. Since

$$\bar{\delta}(s_0, x_s a) = \delta(\bar{\delta}(s_0, x_s), a) = \delta(s, a),$$

the string $x_s a$ reaches $\delta(s, a)$, so

$$\psi(\delta(s, a)) = [x_s a]_{R^M} = \delta_{R^M}([x_s]_{R^M}, a) = \delta_{R^M}(\psi(s), a).$$

Also,

$$s \in F \Leftrightarrow \bar{\delta}(s_0, x_s) \in F \Leftrightarrow x_s \in L(M) \Leftrightarrow [x_s]_{R^M} \in F_{R^M} \Leftrightarrow \psi(s) \in F_{R^M}.$$

Thus ψ preserves start state, transitions, and finality.

(d) *Bijection*: to show injectivity, suppose $\psi(s) = \psi(t)$. Then

$$[x_s]_{R^M} = [x_t]_{R^M},$$

so $x_s R^M x_t$. By definition of R^M ,

$$\bar{\delta}(s_0, x_s) = \bar{\delta}(s_0, x_t),$$

hence $s = t$. For surjectivity, let $[x]_{R^M}$ be any state of A_{R^M} . Then

$$s = \bar{\delta}(s_0, x) \in S^c,$$

and by definition of ψ ,

$$\psi(s) = [x]_{R^M}.$$

(e) $M^c \cong A_{R^M}$. $\square$

3.23. **For the machine A in Figure 3.10(a), find E_A , $L(A)$, $A_{R_{L(A)}}$, $R_{L(A)}$, and A/E_A .**

(a)

$$E_{0A} = \{\{q_0, q_1, q_3, q_6\}, \{q_2, q_4, q_5\}\}.$$

Refining once gives

$$E_{1A} = \{\{q_0, q_1\}, \{q_3, q_6\}, \{q_2\}, \{q_4, q_5\}\}.$$

Refining again gives

$$E_{2A} = \{\{q_0\}, \{q_1\}, \{q_2\}, \{q_3, q_6\}, \{q_4, q_5\}\}.$$

This is already stable, so $E_A = E_{2A}$.

(b) The language is

$$L(A) = \{\lambda, 1\} \cup 00\Sigma^* \cup 11\Sigma^*.$$

Indeed, λ is accepted at q_0 , the string 1 is accepted at q_1 , and every string of length at least 2 is accepted exactly when its first two symbols are equal.

- (c) $A_{R_{L(A)}}$ has five states, represented by the classes of λ , 1, 0, 00, and 01. Writing these as $[\lambda], [1], [0], [00], [01]$, the transitions are

	0	1
$[\lambda]$	$[0]$	$[1]$
$[1]$	$[01]$	$[00]$
$[0]$	$[00]$	$[01]$
$[00]$	$[00]$	$[00]$
$[01]$	$[01]$	$[01]$

with final states $[\lambda], [1], [00]$. Here $[11] = [00]$ and $[10] = [01]$.

- (d) The right-congruence classes of $R_{L(A)}$ may be written as

$$[\lambda], \quad [1], \quad [0], \quad [00] = [11], \quad [01] = [10].$$

Equivalently, they are

$$\{\lambda\}, \quad \{1\}, \quad \{0\}, \quad \{x : |x| \geq 2 \text{ and the first two symbols are equal}\},$$

and

$$\{x : |x| \geq 2 \text{ and the first two symbols are different}\}.$$

- (e) A/E_A has states

$$[q_0], [q_1], [q_2], [q_3] = [q_6], [q_4] = [q_5],$$

with $[q_0]$ initial and final states $[q_0], [q_1], [q_3] = [q_6]$. Its transition table is

	0	1
$[q_0]$	$[q_2]$	$[q_1]$
$[q_1]$	$[q_4] = [q_5]$	$[q_3] = [q_6]$
$[q_2]$	$[q_3] = [q_6]$	$[q_4] = [q_5]$
$[q_3] = [q_6]$	$[q_3] = [q_6]$	$[q_3] = [q_6]$
$[q_4] = [q_5]$	$[q_4] = [q_5]$	$[q_4] = [q_5]$

so $A/E_A \cong A_{R_{L(A)}}$.

3.24. Figure 3.10(b).

- (a)

$$E_{0B} = \{\{t_0\}, \{t_1, t_2, t_3\}\}.$$

All three nonfinal states have the same 0- and 1-transitions into the nonfinal class, so $E_{1B} = E_{0B}$. Hence $E_B = E_{0B}$.

- (b) $L(B) = \{\lambda\}$.

- (c) $A_{R_{L(B)}}$ is the two-state DFA with states $[\lambda]$ and $[0]$ (equivalently, $[1]$), where $[\lambda]$ is initial and final, $[0]$ is nonfinal, and both letters send $[0]$ to itself.

- (d) $R_{L(B)}$ has exactly two classes:

$$[\lambda] = \{\lambda\}, \quad [0] = \Sigma^+.$$

(e) B/E_B has states $[t_0]$ and $[t_1] = [t_2] = [t_3]$, with $[t_0]$ initial and final, and

$$\delta([t_0], 0) = \delta([t_0], 1) = [t_1] = [t_2] = [t_3],$$

while both letters fix $[t_1] = [t_2] = [t_3]$. Thus $B/E_B \cong A_{R_{L(B)}}$.

3.25. **Figure 3.10(c).**

(a)

$$E_{0C} = \{\{q_1, q_4\}, \{q_0, q_2, q_3, q_5\}\}.$$

Refining once gives

$$E_{1C} = \{\{q_1\}, \{q_4\}, \{q_0, q_2\}, \{q_3, q_5\}\}.$$

This is already stable, so $E_C = E_{1C}$.

(b) The language is

$$L(C) = 1(1 \cup 01)^*,$$

that is, the strings that begin with 1, end with 1, and never contain the substring 00.

(c) $A_{R_{L(C)}}$ has three states, represented by the classes of λ , 1, and 0. Writing them as $[\lambda]$, $[1]$, $[0]$, the transition table is

	0	1
$[\lambda]$	$[0]$	$[1]$
$[1]$	$[\lambda]$	$[1]$
$[0]$	$[0]$	$[0]$

with only $[1]$ final. Here $[10] = [\lambda]$, while $[11] = [1]$ and $[00] = [01] = [0]$.

(d) The classes of $R_{L(C)}$ can be represented by $[\lambda]$, $[1]$, $[0]$. Concretely,

$$[1] = 1(1 \cup 01)^*,$$

$$[\lambda] = \{\lambda\} \cup 1(1 \cup 01)^*0,$$

and

$$[0] = \Sigma^* - ([\lambda] \cup [1]).$$

(e) C/E_C has the four states

$$[q_1], [q_4], [q_0] = [q_2], [q_3] = [q_5].$$

The initial state is $[q_0] = [q_2]$, the final states are $[q_1]$ and $[q_4]$, and the transition table is

	0	1
$[q_0] = [q_2]$	$[q_3] = [q_5]$	$[q_1]$
$[q_1]$	$[q_0] = [q_2]$	$[q_1]$
$[q_3] = [q_5]$	$[q_3] = [q_5]$	$[q_3] = [q_5]$
$[q_4]$	$[q_3] = [q_5]$	$[q_3] = [q_5]$

The state $[q_4]$ is disconnected, so C/E_C is not connected; its connected part is isomorphic to $A_{R_{L(C)}}$.

3.26. **Figure 3.10(d).**

(a)

$$E_{0D} = \{\{S_1, S_4, S_7\}, \{S_0, S_2, S_3, S_5, S_6, S_8, S_9\}\}.$$

Refining once gives

$$E_{1D} = \{\{S_1, S_4\}, \{S_7\}, \{S_0, S_3\}, \{S_2, S_5, S_6\}, \{S_8, S_9\}\}.$$

Refining again gives

$$E_{2D} = \{\{S_1, S_4\}, \{S_7\}, \{S_0, S_3\}, \{S_2, S_5\}, \{S_6\}, \{S_8\}, \{S_9\}\}.$$

This is already stable, so $E_D = E_{2D}$.

- (b) From the initial state S_0 , only the classes $[S_0] = [S_3]$, $[S_1] = [S_4]$, and $[S_2] = [S_5]$ are reachable. Therefore $L(D)$ consists exactly of those strings over $\{a, b\}$ that contain at least one a and either end in b or are of the form b^*a . Equivalently,

$$L(D) = b^*a(\{a, b\}^*b \cup \{\lambda\}).$$

- (c) $A_{R_{L(D)}}$ has three states represented by $[\lambda]$, $[a]$, and $[aa]$, with transition table

	a	b
$[\lambda]$	$[a]$	$[\lambda]$
$[a]$	$[aa]$	$[a]$
$[aa]$	$[aa]$	$[a]$

and only $[a]$ final.

- (d) The classes of $R_{L(D)}$ may be described as

$$[\lambda] = b^*,$$

$$[a] = L(D),$$

and

$$[aa] = \{x \in \{a, b\}^* : x \text{ ends with } a \text{ and contains at least two } a\text{'s}\}.$$

- (e) D/E_D has the seven states

$$[S_0] = [S_3], [S_1] = [S_4], [S_2] = [S_5], [S_6], [S_7], [S_8], [S_9].$$

Its initial state is $[S_0] = [S_3]$, its final states are $[S_1] = [S_4]$ and $[S_7]$, and its transition table is

	a	b
$[S_0] = [S_3]$	$[S_1] = [S_4]$	$[S_0] = [S_3]$
$[S_1] = [S_4]$	$[S_2] = [S_5]$	$[S_1] = [S_4]$
$[S_2] = [S_5]$	$[S_2] = [S_5]$	$[S_1] = [S_4]$
$[S_6]$	$[S_6]$	$[S_7]$
$[S_7]$	$[S_7]$	$[S_7]$
$[S_8]$	$[S_0] = [S_3]$	$[S_9]$
$[S_9]$	$[S_2] = [S_5]$	$[S_2] = [S_5]$

The connected part of this quotient is isomorphic to the three-state machine $A_{R_{L(D)}}$ above.

3.27. **Prove: if A, B are reduced, connected, equivalent DFAs then $A \cong B$.**

(a) Since A is connected, for each $s \in S_A$ choose a string x_s with $\bar{\delta}_A(s_{0A}, x_s) = s$. Define

$$\psi(s) = \bar{\delta}_B(s_{0B}, x_s).$$

(b) To show this is well defined, first note that it is defined everywhere because A is connected, so every state $s \in S_A$ has at least one string reaching it. It is not multiply-defined: suppose x and y both satisfy $\bar{\delta}_A(s_{0A}, x) = \bar{\delta}_A(s_{0A}, y) = s$. Then for every $z \in \Sigma^*$,

$$xz \in L(A) \Leftrightarrow yz \in L(A),$$

because xz and yz continue from the same state s in A . Since $L(A) = L(B)$, it follows that

$$xz \in L(B) \Leftrightarrow yz \in L(B)$$

for every z . Therefore the states $\bar{\delta}_B(s_{0B}, x)$ and $\bar{\delta}_B(s_{0B}, y)$ have the same future language in B . Because B is reduced, these states must be equal. Hence ψ is well defined.

(c) First, take the candidate string for the start state to be $x_{s_{0A}} = \lambda$. Then

$$\psi(s_{0A}) = \bar{\delta}_B(s_{0B}, x_{s_{0A}}) = \bar{\delta}_B(s_{0B}, \lambda) = s_{0B}.$$

Now let $s \in S_A$ and $a \in \Sigma$. Since

$$\bar{\delta}_A(s_{0A}, x_s a) = \delta_A(\bar{\delta}_A(s_{0A}, x_s), a) = \delta_A(s, a),$$

we may use $x_s a$ as a string reaching $\delta_A(s, a)$. Therefore

$$\psi(\delta_A(s, a)) = \bar{\delta}_B(s_{0B}, x_s a) = \delta_B(\bar{\delta}_B(s_{0B}, x_s), a) = \delta_B(\psi(s), a).$$

Also,

$$s \in F_A \Leftrightarrow x_s \in L(A) \Leftrightarrow x_s \in L(B) \Leftrightarrow \psi(s) \in F_B.$$

So ψ is a homomorphism.

(d) *Injective*: suppose $\psi(s) = \psi(t)$. Then for every $z \in \Sigma^*$,

$$x_s z \in L(B) \Leftrightarrow x_t z \in L(B),$$

because x_s and x_t reach the same state in B . Since $L(A) = L(B)$, we also have

$$x_s z \in L(A) \Leftrightarrow x_t z \in L(A)$$

for every z . Thus s and t have the same future language in A . Because A is reduced, $s = t$.

Surjective: let $u \in S_B$. Since B is connected, there exists $y \in \Sigma^*$ such that $\bar{\delta}_B(s_{0B}, y) = u$. Let $s = \bar{\delta}_A(s_{0A}, y)$. Then y is one of the strings reaching s , so by the definition of ψ and well-definedness,

$$\psi(s) = \bar{\delta}_B(s_{0B}, y) = u.$$

Hence ψ is surjective.

3.28. Show the transition from line 3 to 4 in Theorem 3.1 (6) is not $\Leftrightarrow$.

In that step, we used $\bar{\delta}_A(s, x) \in F_A \Rightarrow \mu(\bar{\delta}_A(s, x)) \in F_B$. The converse would require $\mu(s) \in F_B \Rightarrow s \in F_A$, which is the definition of a homomorphism (both directions). If μ is only required to satisfy $s \in F_A \Rightarrow \mu(s) \in F_B$ (not $\Leftarrow$), then we only get $L(A) \subseteq L(B)$, not equality. Example: A has states $\{q_0, q_1\}$, $F_A = \{q_1\}$; B has $\{p_0\}$, $F_B = \{p_0\}$ (all final); $\mu(q_i) = p_0$. Then $L(A) \subset L(B) = \Sigma^*$.

3.29. Supply reasons for equivalences in proof of Theorem 3.8.

Reading down the proof of Theorem 3.8:

(a)

$$sE_{i+1A}t \Leftrightarrow (\forall x \in \Sigma^* \ni |x| \leq i+1)(\bar{\delta}(s, x) \in F \Leftrightarrow \bar{\delta}(t, x) \in F)$$

is just the definition of E_{i+1A} .

(b) Splitting this into the strings of length at most i and the strings of length exactly $i+1$ is valid because those two cases exhaust the condition $|x| \leq i+1$.

(c) Replacing “ $|y| = i+1$ ” by “ $1 \leq |y| \leq i+1$ ” is valid because the strings of lengths $1, \dots, i$ are already handled by the first conjunct.

(d) Passing from nonempty y with $|y| \leq i+1$ to strings of the form $\mathbf{a}x$ with $\mathbf{a} \in \Sigma$ and $|x| \leq i$ uses the fact that every nonempty string can be written uniquely in that form.

(e)

$$(\forall x \in \Sigma^* \ni |x| \leq i)(\bar{\delta}(s, x) \in F \Leftrightarrow \bar{\delta}(t, x) \in F) \Leftrightarrow sE_{iA}t$$

is the definition of E_{iA} .

(f)

$$\bar{\delta}(s, \mathbf{a}x) = \bar{\delta}(\bar{\delta}(s, \mathbf{a}), x)$$

and

$$\bar{\delta}(t, \mathbf{a}x) = \bar{\delta}(\bar{\delta}(t, \mathbf{a}), x)$$

are both instances of the definition of the extended transition function $\bar{\delta}$.

(g) Finally,

$$(\forall x \in \Sigma^* \ni |x| \leq i)(\bar{\delta}(\bar{\delta}(s, \mathbf{a}), x) \in F \Leftrightarrow \bar{\delta}(\bar{\delta}(t, \mathbf{a}), x) \in F) \Leftrightarrow \delta(s, \mathbf{a})E_{iA}\delta(t, \mathbf{a})$$

is again the definition of E_{iA} .

3.30. Minimize the machine of Figure 3.3.

Let

$$E_{0A} = \{\{S_0, S_5\}, \{S_1, S_2, S_3, S_4\}\}.$$

Refining once gives

$$E_{1A} = \{\{S_0, S_5\}, \{S_1, S_2\}, \{S_3, S_4\}\},$$

and this is already stable. Thus the minimal machine has the three states

$$A = [S_0] = [S_5], \quad B = [S_1] = [S_2], \quad C = [S_3] = [S_4],$$

with A initial and final, and transition table

	a	b
A	B	B
B	C	C
C	A	A

So the machine minimizes to a 3-state DFA in which both letters move one step forward around the cycle $A \rightarrow B \rightarrow C \rightarrow A$. $\square$

3.31. Examples where A not reduced, A^c not/is reduced.

(a) A not reduced, A^c not reduced: take

$$A = \langle \{\mathbf{a}\}, \{q_0, q_1\}, q_0, \delta, \{q_0, q_1\} \rangle,$$

where

$$\delta(q_0, \mathbf{a}) = q_1, \quad \delta(q_1, \mathbf{a}) = q_1.$$

Both states are reachable, so $A^c = A$. Also $q_0 E_A q_1$, because from either state every continuation is accepted. Thus A is not reduced, and neither is A^c .

(b) A not reduced, A^c is reduced: take

$$A = \langle \{\mathbf{a}\}, \{q_0, q_1, q_2\}, q_0, \delta, \{q_1, q_2\} \rangle,$$

where

$$\delta(q_0, \mathbf{a}) = q_1, \quad \delta(q_1, \mathbf{a}) = q_1, \quad \delta(q_2, \mathbf{a}) = q_2.$$

The state q_2 is unreachable, and $q_1 E_A q_2$ because from either state every continuation is accepted. So A is not reduced. However,

$$A^c = \langle \{\mathbf{a}\}, \{q_0, q_1\}, q_0, \delta|_{S^c \times \Sigma}, \{q_1\} \rangle,$$

and q_0 and q_1 are not equivalent in A^c because λ is rejected from q_0 but accepted from q_1 . Hence A^c is reduced.

3.32. Prove $\cong$ is an equivalence relation on DFAs.

- (a) If $g : A \rightarrow B$ and $f : B \rightarrow C$ are isomorphisms, then $f \circ g : A \rightarrow C$ satisfies all isomorphism conditions (since both are bijective homomorphisms and composition of bijections is bijective, composition of homomorphisms is a homomorphism).
- (b) Symmetry: if $f : A \rightarrow B$ is an isomorphism (bijective homomorphism), then $f^{-1} : B \rightarrow A$ is also an isomorphism: $f^{-1}(s_{0B}) = s_{0A}$ (since $f(s_{0A}) = s_{0B}$); $f^{-1}(\delta_B(t, a)) = f^{-1}(\delta_B(f(s), a)) = f^{-1}(f(\delta_A(s, a))) = \delta_A(s, a) = \delta_A(f^{-1}(t), a)$; $t \in F_B \Leftrightarrow f^{-1}(t) \in F_A$.
- (c) Reflexivity: $\text{id} : A \rightarrow A$ is an isomorphism.
- (d) From (a),(b),(c): $\cong$ is an equivalence relation. $\square$

3.33. Homomorphism is not an equivalence relation on DFAs.

Homomorphism is not symmetric. Let $\Sigma = \{a\}$, and let A be the one-state DFA with state p (initial, nonfinal) and $\delta_A(p, a) = p$, so $L(A) = \emptyset$. Let B have two states q_0, q_1 , with q_0 initial and nonfinal, q_1 final and unreachable, and both states looping on a .

There is a homomorphism $\mu : A \rightarrow B$ given by $\mu(p) = q_0$: initials match, transitions are preserved, and $p \in F_A \Leftrightarrow q_0 \in F_B$.

However, there is no homomorphism $\nu : B \rightarrow A$, because $q_1 \in F_B$ would have to imply $\nu(q_1) \in F_A$, and A has no final state. Therefore homomorphism is not an equivalence relation on DFAs. $\square$

3.34. For R and R_L from Theorem 2.2, show homomorphism from A_R to A_{R_L} .

R refines R_L : each R -class is contained in some R_L -class. Define

$$\mu : S_{A_R} \rightarrow S_{A_{R_L}} \quad \text{by} \quad \mu([x]_R) = [x]_{R_L}.$$

This is well defined: every R -class has at least one representative $x \in \Sigma^*$, so the definition applies everywhere on S_{A_R} . If $[x]_R = [y]_R$, then xRy , and since R refines R_L , also xR_Ly , so $[x]_{R_L} = [y]_{R_L}$.

Now

$$\mu(s_{0_{A_R}}) = \mu([\lambda]_R) = [\lambda]_{R_L} = s_{0_{R_L}},$$

so μ sends the start state of A_R to the start state of A_{R_L} . Also, for any $a \in \Sigma$,

$$\mu(\delta_R([x]_R, a)) = \mu([xa]_R) = [xa]_{R_L} = \delta_{R_L}([x]_{R_L}, a) = \delta_{R_L}(\mu([x]_R), a).$$

Finally,

$$[x]_R \in F_R \Leftrightarrow x \in L \Leftrightarrow [x]_{R_L} \in F_{R_L} \Leftrightarrow \mu([x]_R) \in F_{R_L}.$$

Therefore μ is a homomorphism from A_R to A_{R_L} . $\square$

3.35. Prove: if there is a homomorphism $A \rightarrow B$ then R^A refines R^B .

$\mu : A \rightarrow B$ homomorphism. If $xR^A y$: $\bar{\delta}_A(s_0, x) = \bar{\delta}_A(s_0, y)$. Then $\bar{\delta}_B(s_0, x) = \mu(\bar{\delta}_A(s_0, x)) = \mu(\bar{\delta}_A(s_0, y)) = \bar{\delta}_B(s_0, y)$, so $xR^B y$. $\square$

3.36. Prove: if $A \cong B$ then $R^A = R^B$.

(a) Let $\mu : A \rightarrow B$ be an isomorphism. If $xR^A y$, then

$$\bar{\delta}_A(s_{0_A}, x) = \bar{\delta}_A(s_{0_A}, y).$$

Applying μ to both sides gives

$$\mu(\bar{\delta}_A(s_{0_A}, x)) = \mu(\bar{\delta}_A(s_{0_A}, y)).$$

Because μ preserves start states and transitions, the left side is exactly $\bar{\delta}_B(s_{0_B}, x)$ and the right side is exactly $\bar{\delta}_B(s_{0_B}, y)$. Hence $xR^B y$. Therefore $R^A \subseteq R^B$.

Now let $xR^B y$. Since $\mu^{-1} : B \rightarrow A$ is also an isomorphism, the inclusion proved above with A and B interchanged gives $xR^A y$. Hence $R^B \subseteq R^A$. Therefore $R^A = R^B$.

(b) Equivalently, for every word x ,

$$\mu(\bar{\delta}_A(s_{0_A}, x)) = \bar{\delta}_B(s_{0_B}, x),$$

and this can be proved by induction on $|x|$. The base case $x = \lambda$ is

$$\mu(\bar{\delta}_A(s_{0_A}, \lambda)) = \mu(s_{0_A}) = s_{0_B} = \bar{\delta}_B(s_{0_B}, \lambda).$$

For the step, write $x = \mathbf{a}w$. Then

$$\mu(\bar{\delta}_A(s_{0_A}, \mathbf{a}w)) = \mu(\bar{\delta}_A(\bar{\delta}_A(s_{0_A}, \mathbf{a}), w)) = \bar{\delta}_B(\mu(\bar{\delta}_A(s_{0_A}, \mathbf{a})), w),$$

by the induction hypothesis, and since μ preserves transitions this equals

$$\bar{\delta}_B(\bar{\delta}_B(s_{0_B}, \mathbf{a}), w) = \bar{\delta}_B(s_{0_B}, \mathbf{a}w).$$

Because μ is injective,

$$\bar{\delta}_A(s_{0_A}, x) = \bar{\delta}_A(s_{0_A}, y) \Leftrightarrow \bar{\delta}_B(s_{0_B}, x) = \bar{\delta}_B(s_{0_B}, y),$$

so again $R^A = R^B$.

3.37. **A not homomorphic to B**, $R^A = R^B$.

- (a) $L(A) = L(B)$: let $\Sigma = \{a\}$. Let B be the minimal 2-state DFA for even-length strings: q_0 is initial and final, q_1 is nonfinal, and $\delta(q_0, a) = q_1$, $\delta(q_1, a) = q_0$. Let A have the same reachable part, plus an unreachable final state r with $\delta(r, a) = r$.

Then $L(A) = L(B)$ and $R^A = R^B$, because R^A only depends on the states reached from the initial state, and A and B have the same reachable part. However, there is no homomorphism $A \rightarrow B$: since r is final, its image would have to be the final state q_0 , but then

$$\mu(\delta_A(r, a)) = \mu(r) = q_0$$

while

$$\delta_B(\mu(r), a) = \delta_B(q_0, a) = q_1,$$

a contradiction.

- (b) $L(A) \neq L(B)$: let A again be the even-length machine, and let B be the same transition graph but with final state q_1 instead of q_0 , so $L(B)$ is the odd-length strings. Then $R^A = R^B$ (both relations simply partition Σ^* into even and odd lengths), but A is not homomorphic to B because the initial state of A is final while the initial state of B is not.
- (c) No. Suppose A and B are connected, $L(A) = L(B)$, and $R^A = R^B$. Define

$$\psi(\bar{\delta}_A(s_{0_A}, x)) = \bar{\delta}_B(s_{0_B}, x).$$

Because $R^A = R^B$, this is well defined: if $\bar{\delta}_A(s_{0_A}, x) = \bar{\delta}_A(s_{0_A}, y)$, then xR^Ay , hence xR^By , and therefore $\bar{\delta}_B(s_{0_B}, x) = \bar{\delta}_B(s_{0_B}, y)$. Since both machines are connected, the map ψ is onto: given $u \in S_B$, choose x with $\bar{\delta}_B(s_{0_B}, x) = u$, and then

$$\psi(\bar{\delta}_A(s_{0_A}, x)) = u.$$

It is also one-to-one: if $\psi(s) = \psi(t)$, choose strings x_s, x_t with

$$\bar{\delta}_A(s_{0A}, x_s) = s, \quad \bar{\delta}_A(s_{0A}, x_t) = t.$$

Then

$$\bar{\delta}_B(s_{0B}, x_s) = \psi(s) = \psi(t) = \bar{\delta}_B(s_{0B}, x_t),$$

so $x_s R^B x_t$. Since $R^A = R^B$, also $x_s R^A x_t$, hence

$$s = \bar{\delta}_A(s_{0A}, x_s) = \bar{\delta}_A(s_{0A}, x_t) = t.$$

For transitions,

$$\psi(\delta_A(\bar{\delta}_A(s_{0A}, x), a)) = \psi(\bar{\delta}_A(s_{0A}, xa)) = \bar{\delta}_B(s_{0B}, xa) = \delta_B(\bar{\delta}_B(s_{0B}, x), a) = \delta_B(\psi(\bar{\delta}_A(s_{0A}, x)), a).$$

Finally, for any $x \in \Sigma^*$,

$$\bar{\delta}_A(s_{0A}, x) \in F_A \Leftrightarrow x \in L(A) \Leftrightarrow x \in L(B) \Leftrightarrow \bar{\delta}_B(s_{0B}, x) \in F_B,$$

so ψ preserves finality. Thus ψ is an isomorphism $A \cong B$, so in particular there is a homomorphism from A to B . Such examples therefore cannot exist.

- (d) Yes. The example from part (a) already works if we note that both machines are reduced: B is minimal and therefore reduced, and the extra state r in A has language Σ^* , which is different from the state languages of q_0 and q_1 . So A and B are both reduced, $L(A) = L(B)$, and $R^A = R^B$, but A is still not homomorphic to B .

3.38. Disprove: if A homomorphic to B then $R^A = R^B$.

Counterexample: $A = 2$ -state DFA for $\{a^{2k}\}$, $B = 1$ -state DFA for Σ^* (all states final). There is a homomorphism $\mu: A \rightarrow B$ mapping both states to the one state of B . R^A has 2 classes (even/odd), R^B has 1 class. $R^A \neq R^B$. $\square$

3.39. Prove or give counterexamples (homomorphisms involving A_{R^M}).

- (a) Homomorphism $\psi: A_{R^M} \rightarrow M$: Yes. Define

$$\psi([x]_{R^M}) = \bar{\delta}_M(s_0, x).$$

This is well defined: if $[x]_{R^M} = [y]_{R^M}$, then $xR^M y$, so by definition of R^M ,

$$\bar{\delta}_M(s_0, x) = \bar{\delta}_M(s_0, y).$$

It is a homomorphism because

$$\psi([\lambda]_{R^M}) = \bar{\delta}_M(s_0, \lambda) = s_0,$$

$$\psi(\delta_{R^M}([x]_{R^M}, a)) = \psi([xa]_{R^M}) = \bar{\delta}_M(s_0, xa) = \delta_M(\bar{\delta}_M(s_0, x), a) = \delta_M(\psi([x]_{R^M}), a),$$

and

$$[x]_{R^M} \in F_{R^M} \Leftrightarrow x \in L(M) \Leftrightarrow \bar{\delta}_M(s_0, x) \in F_M \Leftrightarrow \psi([x]_{R^M}) \in F_M.$$

- (b) Isomorphism $\psi : A_{R^M} \rightarrow M$: Not always. Only if M is connected (since $A_{R^M} \cong M^c$, isomorphism only if $M = M^c$, i.e., M connected).
- (c) Homomorphism $\psi : M \rightarrow A_{R^M}$: Not always. The reachable part of M maps naturally to A_{R^M} (send each reachable state s to $[x_s]_{R^M}$ where x_s is any string reaching s), but unreachable states may obstruct extension.

Counterexample. Let $\Sigma = \{\mathbf{a}\}$, $S = \{s_0, s_1, s_d\}$, s_0 initial, $F = \{s_1\}$, with

$$\delta(s_0, \mathbf{a}) = s_1, \quad \delta(s_1, \mathbf{a}) = s_1, \quad \delta(s_d, \mathbf{a}) = s_0.$$

State s_d is unreachable. The accessible quotient A_{R^M} has two states q_0, q_1 with

$$\delta'(q_0, \mathbf{a}) = q_1, \quad \delta'(q_1, \mathbf{a}) = q_1,$$

q_0 initial, $F' = \{q_1\}$.

A homomorphism $\psi : M \rightarrow A_{R^M}$ must satisfy $\psi(s_0) = q_0$ (initial states correspond) and $\delta'(\psi(s), \mathbf{a}) = \psi(\delta(s, \mathbf{a}))$ for every s . From s_d : $\delta'(\psi(s_d), \mathbf{a}) = \psi(\delta(s_d, \mathbf{a})) = \psi(s_0) = q_0$. But $\delta'(q_0, \mathbf{a}) = q_1 \neq q_0$ and $\delta'(q_1, \mathbf{a}) = q_1 \neq q_0$, so no state in A_{R^M} can serve as $\psi(s_d)$. The homomorphism does not exist. $\square$

3.40. **Prove: if A minimal then $R^A = R_{L(A)}$.**

Let

$$A = \langle \Sigma, S, s_0, \delta, F \rangle$$

be minimal.

First, A must be connected. If not, then its connected part A^c accepts the same language as A but has fewer states, contradicting minimality.

Second, A must be reduced. If not, then two distinct states are E_A -equivalent, so the quotient A/E_A accepts the same language as A and has fewer states, again contradicting minimality.

Because A is connected, every state is reached by some string, so the R^A -classes are in one-to-one correspondence with the states of A . Hence

$$rk(R^A) = \|S\|.$$

Also, Exercise 2.57(c) says that R^A refines $R_{L(A)}$, so

$$rk(R^A) \geq rk(R_{L(A)}).$$

On the other hand, Corollary 3.2 says that $A_{R_{L(A)}}$ is minimal for $L(A)$. Since A is also minimal for $L(A)$, the two minimal DFAs have the same number of states:

$$\|S\| = rk(R_{L(A)}).$$

Therefore

$$rk(R^A) = \|S\| = rk(R_{L(A)}).$$

A refinement of finite rank cannot be proper if the two ranks are equal. Hence $R^A = R_{L(A)}$. $\square$

3.41. Example where Exercise 3.40 fails if A not minimal.

Take $\Sigma = \{a\}$ and let

$$A = \langle \{a\}, \{q_0, q_1, q_2\}, q_0, \delta, \{q_1, q_2\} \rangle,$$

where

$$\delta(q_0, a) = q_1, \quad \delta(q_1, a) = q_2, \quad \delta(q_2, a) = q_2.$$

This DFA is connected, but it is not reduced, because $q_1 E_A q_2$: from either state, every continuation is accepted. Hence A is not minimal. From the initial state, the machine accepts exactly

$$L(A) = a^+.$$

Therefore $R_{L(A)}$ has exactly two classes, represented by λ and a . On the other hand, λ , a , and aa reach the three distinct states q_0 , q_1 , and q_2 , so R^A has three classes. Hence

$$R^A \neq R_{L(A)}.$$

3.42. Example where Exercise 3.40 holds even if A not minimal.

Take the minimal 2-state DFA for the language of even-length strings over $\{a\}$, with states q_0 (initial and final) and q_1 (nonfinal), and add an extra unreachable state q_2 with $\delta(q_2, a) = q_2$. The resulting machine A is not minimal, because it is not connected. However, R^A is unchanged by the presence of q_2 , since no string from the initial state ever reaches q_2 . Therefore R^A still has exactly the two classes “even length” and “odd length,” which are precisely the classes of $R_{L(A)}$. So in this example A is not minimal but $R^A = R_{L(A)}$. $\square$

3.43. Quotient by arbitrary relation R .

- (a) $R = E_{0A}$: A/E_{0A} partitions into final and nonfinal. $\delta_{E_{0A}}$ maps $[s]_{E_{0A}} \times \Sigma \rightarrow S_{E_{0A}}$: need $[s] \ni s'$ with $\delta(s, a)$ and $\delta(s', a)$ in the same E_{0A} -class... not necessarily. So A/E_{0A} is not always well-defined. Example: $A = 3$ -state DFA, $F = \{q_1\}$, $\delta(q_0, a) = q_1$, $\delta(q_2, a) = q_0$. E_{0A} -classes: $\{q_0, q_2\}$ (nonfinal) and $\{q_1\}$ (final). $\delta_{E_{0A}}(\{q_0, q_2\}, a)$: $\delta(q_0, a) = q_1 \in \{q_1\}$ but $\delta(q_2, a) = q_0 \in \{q_0, q_2\}$: two different classes! Not well-defined.
- (b) If R refines E_A , then A/R is well defined exactly when R is stable under the transitions:

$$sRt \Rightarrow \delta(s, a)R\delta(t, a)$$

for every $a \in \Sigma$.

For such well-defined refinements R , the analogues of Theorems 3.4, 3.5, and 3.6 behave as follows.

Analogue of Theorem 3.4: false in general. Take R to be the identity relation on the states of a DFA A that is not reduced. Then R certainly refines E_A , and $A/R = A$, which is not reduced.

Analogue of Theorem 3.5: true. If A is connected and $[s]_R$ is a state of A/R , choose x with $\bar{\delta}(s_0, x) = s$. Since A/R is well defined,

$$\bar{\delta}_R([s_0]_R, x) = [\bar{\delta}(s_0, x)]_R = [s]_R,$$

so every state class is reachable.

Analogue of Theorem 3.6: true. Because R refines E_A , every R -class is contained in a single E_A -class, so no R -class mixes final and nonfinal states; hence F_R is well defined. Compute directly:

$$x \in L(A/R) \Leftrightarrow \bar{\delta}_R([s_0]_R, x) \in F_R \Leftrightarrow [\bar{\delta}(s_0, x)]_R \in F_R \Leftrightarrow \bar{\delta}(s_0, x) \in F \Leftrightarrow x \in L(A).$$

Thus $L(A/R) = L(A)$.

3.44. Homomorphisms between M/E_M and $A_{R_{L(M)}}$.

(a) Not always. Consider the DFA

$$M = \langle \{a\}, \{s_0, s_1, s_d\}, s_0, \delta, \{s_1\} \rangle$$

with

$$\delta(s_0, a) = s_1, \quad \delta(s_1, a) = s_1, \quad \delta(s_d, a) = s_0.$$

The state s_d is unreachable. The three states have distinct future languages:

$$L(M^{s_0}) = a^+, \quad L(M^{s_1}) = a^*, \quad L(M^{s_d}) = a^{\geq 2},$$

so E_M is the identity relation and $M/E_M = M$.

Now $L(M) = a^+$, and $A_{R_{L(M)}}$ is the 2-state DFA with initial nonfinal state q_0 , final state q_1 , and transitions $\delta(q_0, a) = q_1, \delta(q_1, a) = q_1$.

If there were a homomorphism $\psi : M/E_M \rightarrow A_{R_{L(M)}}$, then $\psi(s_0) = q_0$ and $\psi(s_1) = q_1$. Since s_d is nonfinal, $\psi(s_d)$ would have to be q_0 . But then

$$\psi(\delta(s_d, a)) = \psi(s_0) = q_0$$

while

$$\delta(\psi(s_d), a) = \delta(q_0, a) = q_1,$$

a contradiction. So the statement is false.

(b) Yes. Define

$$\psi([x]_{R_{L(M)}}) = [\bar{\delta}_M(s_0, x)]_{E_M}.$$

This is well defined: if $xR_{L(M)}y$, then for every $z \in \Sigma^*$,

$$xz \in L(M) \Leftrightarrow yz \in L(M),$$

so the states $\bar{\delta}_M(s_0, x)$ and $\bar{\delta}_M(s_0, y)$ have the same future language and are therefore E_M -equivalent.

Now

$$\psi([\lambda]_{R_{L(M)}}) = [s_0]_{E_M},$$

and for any $a \in \Sigma$,

$$\psi(\delta_{R_{L(M)}}([x]_{R_{L(M)}}, a)) = \psi([xa]_{R_{L(M)}}) = [\bar{\delta}_M(s_0, xa)]_{E_M} = [\delta(\bar{\delta}_M(s_0, x), a)]_{E_M} = \delta_{E_M}(\psi([x]_{R_{L(M)}}), a).$$

Finally,

$$[x]_{R_{L(M)}} \in F_{R_{L(M)}} \Leftrightarrow x \in L(M) \Leftrightarrow \bar{\delta}_M(s_0, x) \in F \Leftrightarrow [\bar{\delta}_M(s_0, x)]_{E_M} \in F_{E_M}.$$

So ψ is a homomorphism from $A_{R_{L(M)}}$ to M/E_M . $\square$

3.45. **Prove:** $\exists m < \|S\|$ with $E_{mA} = E_{m+1,A}$.

The sequence $E_{0A} \supseteq E_{1A} \supseteq \dots$ is decreasing (each refinement removes some equivalences). Since S is finite, the number of equivalence classes is bounded by $\|S\|$. Each strict refinement step increases the number of classes by at least 1. Starting from E_{0A} (which has at most 2 classes: final/nonfinal), we can have at most $\|S\| - 1$ strict refinement steps before reaching E_A . So $E_{\|S\|-1,A} = E_{\|S\|,A} = E_A$, meaning $m \leq \|S\| - 1 < \|S\|$. $\square$

3.46. **Equivalence vs. isomorphism.**

(a) False. Let B be the minimal DFA for the even-length strings over

$$\Sigma = \{a\},$$

with states r_0, r_1 , start state r_0 , final set $\{r_0\}$, and transitions

$$\delta_B(r_0, a) = r_1, \quad \delta_B(r_1, a) = r_0.$$

Now let A be the DFA obtained by adjoining an unreachable state r_2 :

$$A = \langle \{a\}, \{r_0, r_1, r_2\}, r_0, \delta_A, \{r_0\} \rangle,$$

where

$$\delta_A(r_0, a) = r_1, \quad \delta_A(r_1, a) = r_0, \quad \delta_A(r_2, a) = r_2.$$

Then $L(A) = L(B)$, so A and B are equivalent, but they are not isomorphic because one has three states and the other has two.

(b) True: if $A \cong B$ then in particular $L(A) = L(B)$ (by Exercise 3.11).

3.47. **Prove** $C_i \subseteq C_{i+1}$.

$C_0 = \{s_0\}$, $C_{i+1} = C_i \cup \{\delta(s, a) : s \in C_i, a \in \Sigma\}$. Clearly $C_i \subseteq C_{i+1}$ by definition. $\square$

3.48. **Prove** C_i = states reachable from s_0 by strings of length $\leq i$.

Let $R_i = \{\bar{\delta}(s_0, x) : |x| \leq i\}$.

Base: $R_0 = \{s_0\} = C_0$. $\checkmark$

Step: $R_{i+1} = \{\bar{\delta}(s_0, x) : |x| \leq i+1\} = R_i \cup \{\bar{\delta}(\bar{\delta}(s_0, x), a) : |x| \leq i, a \in \Sigma\} = C_i \cup \{\delta(s, a) : s \in C_i, a \in \Sigma\} = C_{i+1}$. $\square$

3.49. **Prove:** $C_i = C_{i+1} \Rightarrow (\forall k)(C_i = C_{i+k})$.

Base $k = 0$: trivial. $k = 1$: given.

Step: Assume $C_i = C_{i+k}$. Then $C_{i+k+1} = C_{i+k} \cup \{\delta(s, a) : s \in C_{i+k}\} = C_i \cup \{\delta(s, a) : s \in C_i\} = C_{i+1} = C_i$. $\square$

3.50. **Prove:** for a DFA A , $C_i = C_{i+1} \Rightarrow C_i = S^c$.

$S^c = \bigcup_{k \geq 0} C_k$. If $C_i = C_{i+1}$, then by Exercise 3.49, $C_i = C_{i+k}$ for all $k \geq 0$. So $S^c = \bigcup_k C_k = C_i$ (since $C_k \subseteq C_i$ for $k \leq i$ by $C_i \supseteq C_{i-1} \supseteq \dots$, and $C_k = C_i$ for $k \geq i$). $\square$

3.51. **Prove:** $\exists i$ with $C_i = C_{i+1}$.

Since S is finite, the sequence $C_0 \subseteq C_1 \subseteq \dots$ is non-decreasing and bounded above by S . So it must stabilize: $\exists i$ with $C_i = C_{i+1}$. $\square$

3.52. **Prove:** $\exists i \leq \|S\|$ with $C_i = S^c$.

Each strict inclusion $C_i \subsetneq C_{i+1}$ adds at least one new state. Starting from $\|C_0\| = 1$, after at most $\|S\| - 1$ strict inclusions we reach $C_{\|S\|-1} \supseteq \dots$, which must equal $C_{\|S\|}$ (no room to grow beyond $\|S\|$ states). So $i \leq \|S\| - 1 \leq \|S\|$ and $C_i = S^c$. $\square$

3.53. **Argue the C_i algorithm is correct and terminates.**

Correctness: by Exercise 3.48, C_i is exactly the reachable states within depth i . By Exercise 3.51, $C_i = S^c$ when stable. Termination: by Exercise 3.51 and 3.52, stability occurs by step $\|S\|$. $\square$

3.54. **Give two DFAs with homomorphisms in both directions but no isomorphism.**

Let

$$L = \{a^{2k} : k \geq 0\}.$$

Let

$$A = \langle \{a\}, \{q_0, q_1\}, q_0, \delta_A, \{q_0\} \rangle,$$

where

$$\delta_A(q_0, a) = q_1, \quad \delta_A(q_1, a) = q_0.$$

This is the minimal 2-state DFA for L .

Let

$$B = \langle \{a\}, \{p_0, p_1, p_2\}, p_0, \delta_B, \{p_0, p_2\} \rangle,$$

where

$$\delta_B(p_0, a) = p_1, \quad \delta_B(p_1, a) = p_0, \quad \delta_B(p_2, a) = p_1.$$

Here p_2 is unreachable, and $p_0 E_B p_2$, so B is a non-minimal DFA for L .

There is a homomorphism $\mu : B \rightarrow A$ collapsing $p_0, p_2 \mapsto q_0$ and $p_1 \mapsto q_1$: this satisfies $\delta_A(\mu(p), a) = \mu(\delta_B(p, a))$ and $p \in F_B \Leftrightarrow \mu(p) \in F_A$.

There is also a homomorphism $\nu : A \rightarrow B$ sending $q_0 \mapsto p_0$ and $q_1 \mapsto p_1$ (the natural inclusion into the connected part of B): $\nu(\delta_A(q, a)) = \delta_B(\nu(q), a)$ since $\delta_A(q_0, a) = q_1$ and $\delta_B(p_0, a) = p_1 = \nu(q_1)$, and $q_0 \in F_A \Leftrightarrow p_0 \in F_B$.

But $A \not\cong B$ since they have different numbers of states.

3.55. **R and Q right congruences of finite rank, R refines Q , L union of Q -classes.**

(a) L is a union of R -classes: since R refines Q , each Q -class is a union of R -classes. So $L = \bigcup(\text{some } Q\text{-classes}) = \bigcup(\text{corresponding } R\text{-classes})$.

(b) Homomorphism $A_R \rightarrow A_Q$: Define $\mu([x]_R) = [x]_Q$ (well-defined since R refines Q : $[x]_R = [y]_R \Rightarrow xRy \Rightarrow xQy \Rightarrow [x]_Q = [y]_Q$). Check: $\mu(s_{0,R}) = \mu([\lambda]_R) = [\lambda]_Q = s_{0,Q}$. $\mu(\delta_R([x]_R, a)) = \mu([xa]_R) = [xa]_Q = \delta_Q([x]_Q, a) = \delta_Q(\mu([x]_R), a)$. $[x]_R \in F_R \Leftrightarrow x \in L \Leftrightarrow [x]_Q \in F_Q \Leftrightarrow \mu([x]_R) \in F_Q$.

(c) Not in general. Take $\Sigma = \{a\}$, let

$$[\lambda]_Q = \{\lambda\}, \quad [a]_Q = \{x \in \Sigma^* : |x| \geq 1\},$$

and let R refine Q by splitting the nonempty strings according to parity:

$$[\lambda]_R = \{\lambda\}, \quad [a]_R = \{x : |x| \text{ is odd}\}, \quad [aa]_R = \{x : |x| \text{ is even and } |x| \geq 2\}.$$

Let $L = \{\lambda\}$, which is a union of Q -classes. Then A_Q has two states, $q_0 = [\lambda]_Q$ (start and final) and $q_1 = [a]_Q$, with

$$\delta_Q(q_0, a) = q_1, \quad \delta_Q(q_1, a) = q_1.$$

The DFA A_R has three states, $r_0 = [\lambda]_R$ (start and final), $r_1 = [a]_R$, and $r_2 = [aa]_R$, with

$$\delta_R(r_0, a) = r_1, \quad \delta_R(r_1, a) = r_2, \quad \delta_R(r_2, a) = r_1.$$

Assume there is a homomorphism $\psi : A_Q \rightarrow A_R$. Since start states must map to start states, $\psi(q_0) = r_0$. Then

$$\psi(q_1) = \psi(\delta_Q(q_0, a)) = \delta_R(\psi(q_0), a) = \delta_R(r_0, a) = r_1.$$

But also

$$\psi(q_1) = \psi(\delta_Q(q_1, a)) = \delta_R(\psi(q_1), a) = \delta_R(r_1, a) = r_2,$$

which is impossible. Therefore there need not be a homomorphism from A_Q to A_R .

3.56. Prove A_Q is connected.

Let $[x]_Q$ be any state of A_Q ; such an x exists because equivalence classes are nonempty. Then $\bar{\delta}_Q([\lambda]_Q, x) = [x]_Q$, so every state of A_Q is reachable. $\square$

3.57. Prove: if $A \cong B$ and A connected then B connected.

Let $\mu : A \rightarrow B$ be an isomorphism. μ is surjective (bijection), so every state of B is $\mu(s)$ for some $s \in S_A$. Since A is connected, $s = \bar{\delta}_A(s_0, x)$ for some x . Then $\mu(s) = \mu(\bar{\delta}_A(s_0, x)) = \bar{\delta}_B(\mu(s_0), x) = \bar{\delta}_B(s_0, x)$ (by Exercise 3.10). So every state of B is reachable. $\square$

3.58. Prove: if $A \cong B$ and B connected then A connected.

$\mu^{-1} : B \rightarrow A$ is also an isomorphism. By Exercise 3.56, A is connected. $\square$

3.59. Disprove: homomorphism $A \rightarrow B$, A connected $\Rightarrow B$ connected.

Take

$$A = \langle \{0\}, \{s\}, s, \delta_A, \{s\} \rangle$$

with $\delta_A(s, 0) = s$. This DFA is connected. Let

$$B = \langle \{0\}, \{t, u\}, t, \delta_B, \{t\} \rangle$$

where $\delta_B(t, 0) = t$ and $\delta_B(u, 0) = u$. The state u is unreachable from t , so B is not connected.

Define $\mu : A \rightarrow B$ by $\mu(s) = t$. Then μ preserves the start state, it preserves finality because s and t are both final, and

$$\mu(\delta_A(s, 0)) = \mu(s) = t = \delta_B(t, 0) = \delta_B(\mu(s), 0).$$

Hence μ is a homomorphism even though B is not connected.

3.60. **Disprove: homomorphism $A \rightarrow B$, B connected $\Rightarrow A$ connected.**

Take

$$A = \langle \{\mathbf{0}\}, \{s_0, s_1\}, s_0, \delta_A, \{s_0, s_1\} \rangle$$

with $\delta_A(s_0, \mathbf{0}) = s_0$ and $\delta_A(s_1, \mathbf{0}) = s_1$. The state s_1 is unreachable, so A is not connected. Let

$$B = \langle \{\mathbf{0}\}, \{t\}, t, \delta_B, \{t\} \rangle$$

with $\delta_B(t, \mathbf{0}) = t$; this DFA is connected.

Define $\mu: A \rightarrow B$ by

$$\mu(s_0) = t, \quad \mu(s_1) = t.$$

Then μ preserves the start state, it preserves finality because all these states are final, and

$$\mu(\delta_A(s_i, \mathbf{0})) = \mu(s_i) = t = \delta_B(t, \mathbf{0}) = \delta_B(\mu(s_i), \mathbf{0})$$

for $i = 0, 1$. So there is a homomorphism from A to connected B even though A is not connected.

3.61. $A_{R^A} \cong A^c$: **define ψ and prove it is an isomorphism.**

(a) For each R^A -class $[x]_{R^A}$, choose a representative x from the class; this is possible because equivalence classes are nonempty. Define $\psi([x]_{R^A}) = \bar{\delta}(s_0, x)$, which sends each class to the state it reaches.

(b) Well-defined: $[x]_{R^A} = [y]_{R^A} \Rightarrow \bar{\delta}(s_0, x) = \bar{\delta}(s_0, y)$. ✓

(c) Homomorphism: $\psi([x]_{R^A}) = s_0$. Also

$$\psi([xa]_{R^A}) = \bar{\delta}(s_0, xa) = \delta(\bar{\delta}(s_0, x), a) = \delta(\psi([x]_{R^A}), a).$$

Finally, since $F^c = F \cap S^c$ for the connected part of A ,

$$[x] \in F_{R^A} \Leftrightarrow x \in L(A) \Leftrightarrow \bar{\delta}(s_0, x) \in F \Leftrightarrow \psi([x]) \in F^c.$$

✓

(d) Isomorphism: Injective: $\psi([x]) = \psi([y]) \Rightarrow \bar{\delta}(s_0, x) = \bar{\delta}(s_0, y) \Rightarrow [x] = [y]$. Surjective (onto S^c): if $s \in S^c$, then connectedness gives a string x_s with $s = \bar{\delta}(s_0, x_s)$, and therefore $s = \psi([x_s])$.

3.62. **Prove $\psi = \mu^{-1}$ when A, B connected and $\psi: A \rightarrow B$, $\mu: B \rightarrow A$ isomorphisms.**

$(\mu \circ \psi)(s) = \mu(\psi(s))$. For connected A : $s = \bar{\delta}_A(s_0, x)$ for some x . $\psi(s) = \bar{\delta}_B(s_{0B}, x)$ (by Exercise 3.10 applied to ψ). $\mu(\psi(s)) = \mu(\bar{\delta}_B(s_{0B}, x)) = \bar{\delta}_A(s_{0A}, x) = s$ (by Exercise 3.10 applied to μ). So $\mu \circ \psi = \text{id}_A$, i.e., $\psi = \mu^{-1}$. □

3.63. **Give example for which $\psi \neq \mu^{-1}$ (not connected).**

Take $A = B$ to be the DFA

$$\langle \{\mathbf{0}\}, \{q_0, q_1, q_2\}, q_0, \delta, \{q_0\} \rangle$$

where

$$\delta(q_0, \mathbf{0}) = q_0, \quad \delta(q_1, \mathbf{0}) = q_1, \quad \delta(q_2, \mathbf{0}) = q_2.$$

This machine is not connected because q_1 and q_2 are unreachable.

Let $\psi : A \rightarrow B$ be the identity isomorphism:

$$\psi(q_i) = q_i \quad (i = 0, 1, 2).$$

Let $\mu : B \rightarrow A$ swap the two unreachable states:

$$\mu(q_0) = q_0, \quad \mu(q_1) = q_2, \quad \mu(q_2) = q_1.$$

Because q_1 and q_2 have exactly the same final/nonfinal status and the same loop transition on **0**, μ is also an isomorphism. But

$$\mu^{-1} = \mu \neq \psi,$$

since, for example, $\mu^{-1}(q_1) = q_2$ while $\psi(q_1) = q_1$.

3.64. Three-state DFA with E_{0A} having one class. Can $E_{0A} \neq E_{1A}$?

E_{0A} has one class iff all states are equivalent under E_{0A} , i.e., all are final or all are nonfinal. Example: $F = S = \{q_0, q_1, q_2\}$ (all final). Then $E_{0A} = S \times S$ (one class). Can $E_{1A} \neq E_{0A}$? E_{1A} : $qE_{1A}r \Leftrightarrow qE_{0A}r \wedge (\forall a)\delta(q, a)E_{0A}\delta(r, a)$. Since $E_{0A} = S \times S$, all δ -targets are in the same class, so $E_{1A} = E_{0A}$. So no, if E_{0A} has one class then $E_{1A} = E_{0A}$.

3.65. If A, B reduced and connected and $\psi : A \rightarrow B$ is a homomorphism, must ψ be an isomorphism?

Yes. Since ψ is a homomorphism, it already satisfies the start-state, final-state, and transition conditions from Definition 3.5. We only need to show that ψ is one-to-one and onto.

To prove injectivity, assume $\psi(s) = \psi(t)$. Then certainly $\psi(s)E_B\psi(t)$, and Exercise 3.20 gives

$$\psi(s)E_B\psi(t) \Leftrightarrow sE_A t.$$

Since A is reduced, $sE_A t$ implies $s = t$. Thus ψ is one-to-one.

To prove surjectivity, let $u \in S_B$. Because B is connected, there exists $x_u \in \Sigma^*$ such that $\bar{\delta}_B(s_{0B}, x_u) = u$. Let $s = \bar{\delta}_A(s_{0A}, x_u) \in S_A$. By Exercise 3.10,

$$\psi(s) = \psi(\bar{\delta}_A(s_{0A}, x_u)) = \bar{\delta}_B(\psi(s_{0A}), x_u) = \bar{\delta}_B(s_{0B}, x_u) = u.$$

So every state of B lies in the image of ψ .

Therefore ψ is a bijective homomorphism, hence an isomorphism. $\square$

3.66. Prove Corollary 3.2.

By Theorem 2.2, A_{R_L} accepts L . The discussion immediately preceding the corollary shows that if $M = \langle \Sigma, S, s_0, \delta, F \rangle$ is any DFA with $L(M) = L$, then

$$\|S\| \geq rk(R^M) \geq rk(R_L) = \|S_{R_L}\|.$$

Thus every DFA accepting L has at least as many states as A_{R_L} , so A_{R_L} is a minimal DFA for L . $\square$

3.67. Prove $S^c = \{\bar{\delta}(s_0, x) : |x| \leq \|S\|\}$.

Let

$$C_0 = \{s_0\}, \quad C_{i+1} = C_i \cup \{\delta(s, \mathbf{a}) \mid s \in C_i, \mathbf{a} \in \Sigma\}.$$

We prove by induction on i that

$$C_i = \{\bar{\delta}(s_0, x) \mid |x| \leq i\}.$$

For $i = 0$, both sides are $\{s_0\}$. If the claim holds for i , then a state is reachable by some word of length at most $i + 1$ exactly when either it is already reachable by a word of length at most i , or it has the form $\delta(\bar{\delta}(s_0, x), \mathbf{a})$ for some $|x| \leq i$ and $\mathbf{a} \in \Sigma$. That is exactly the definition of C_{i+1} .

The chain

$$C_0 \subseteq C_1 \subseteq C_2 \subseteq \dots \subseteq S^c$$

can gain a new state at most $\|S\|$ times, since there are only $\|S\|$ states altogether. Hence for some $i \leq \|S\|$ we have $C_i = C_{i+1}$, and then no new reachable states ever appear. Therefore $C_i = S^c$ for some $i \leq \|S\|$.

Thus every connected state has the form $\bar{\delta}(s_0, x)$ for some word with $|x| \leq \|S\|$, so

$$S^c \subseteq \{\bar{\delta}(s_0, x) \mid |x| \leq \|S\|\}.$$

The reverse inclusion is immediate, because every state reached from s_0 by a word is connected. Therefore

$$S^c = \{\bar{\delta}(s_0, x) \mid |x| \leq \|S\|\}. \quad \square$$

3.68. Prove $sE_A t \Leftrightarrow L(A^s) = L(A^t)$.

This is the definition of E_A . Formally: $sE_A t$ is defined as $L(A^s) = L(A^t)$ where $A^s = \langle \Sigma, S, s, \delta, F \rangle$. The equivalence is by definition. (The substantive content is showing E_A satisfies the right congruence property and the iterative characterization from Exercises 3.3–3.4.) $\square$

3.69. Properties of initial and terminal sets.

- (a) $\{I(A, t) \mid t \in S\}$ partitions Σ^* : Each $x \in \Sigma^*$ reaches exactly one state $\bar{\delta}(s_0, x) = t$. So $x \in I(A, t)$ for exactly one t . The sets are disjoint and cover Σ^* . $\square$
- (b) Terminal sets need not partition Σ^* . For example, let $A = \langle \{\mathbf{a}\}, \{q_0, q_1\}, q_0, \delta, \{q_0, q_1\} \rangle$ with $\delta(q_0, \mathbf{a}) = q_0$ and $\delta(q_1, \mathbf{a}) = q_1$. Then

$$T(A, q_0) = \Sigma^* = T(A, q_1).$$

Thus the terminal sets are not disjoint, so they do not form a partition.

- (c) Terminal sets can partition Σ^* . Let $A = \langle \{\mathbf{a}, \mathbf{b}\}, \{q_0, q_1\}, q_0, \delta, \{q_0\} \rangle$ where both letters toggle the states:

$$\delta(q_0, \mathbf{a}) = \delta(q_0, \mathbf{b}) = q_1, \quad \delta(q_1, \mathbf{a}) = \delta(q_1, \mathbf{b}) = q_0.$$

Then $T(A, q_0)$ is the set of even-length strings, while $T(A, q_1)$ is the set of odd-length strings. These two sets are disjoint and their union is all of Σ^* , so in this example the terminal sets do form a partition.

Chapter 4

Nondeterministic Finite Automata — Solutions

4.1. Using the local Figure 4.25, the connected deterministic equivalents are as follows. In each table, $\emptyset$ is the dead state and all missing subsets from $\wp(S)$ are disconnected.

(a) Figure 4.25(a): start state $\{t\}$, final state $\{t\}$.

	<i>a</i>	<i>b</i>	<i>c</i>
$\{t\}$	$\{t\}$	$\emptyset$	$\{t\}$
$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$

(b) Figure 4.25(b): start state $\{q_1\}$, final state $\{q_1\}$.

	<i>a</i>	<i>b</i>	<i>c</i>
$\{q_1\}$	$\{q_0\}$	$\{q_1\}$	$\emptyset$
$\{q_0\}$	$\emptyset$	$\{q_1\}$	$\{q_0\}$
$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$

(c) Figure 4.25(c): start state $\{r_0, r_1\}$, final states $\{r_0\}$ and $\{r_0, r_1\}$.

	<i>a</i>	<i>b</i>	<i>c</i>
$\{r_0\}$	$\{r_0, r_1\}$	$\emptyset$	$\{r_1\}$
$\{r_0, r_1\}$	$\{r_0, r_1\}$	$\{r_0\}$	$\{r_1\}$
$\{r_1\}$	$\emptyset$	$\{r_0\}$	$\emptyset$
$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$

(d) Figure 4.25(d): start state $\{s_0\}$, final state $\{s_2\}$.

	<i>a</i>	<i>b</i>	<i>c</i>
$\{s_0\}$	$\{s_1\}$	$\emptyset$	$\emptyset$
$\{s_1\}$	$\{s_2\}$	$\{s_3\}$	$\emptyset$
$\{s_2\}$	$\emptyset$	$\emptyset$	$\emptyset$
$\{s_3\}$	$\emptyset$	$\{s_1\}$	$\emptyset$
$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$

(e) Figure 4.25(e): start state $\{s_1, s_2\}$, final state $\{s_1, s_2\}$.

	a	b	c
$\{s_1, s_2\}$	$\{s_1, s_2\}$	$\{s_1\}$	$\emptyset$
$\{s_1\}$	$\{s_1, s_2\}$	$\emptyset$	$\emptyset$
$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$

(f) Figure 4.25(f): after taking the λ -closures, the connected DFA has start state $\{s_0\}$ and final state $\{s_0, s_1, s_3\}$.

	a	b	c
$\{s_0\}$	$\{s_0, s_1\}$	$\emptyset$	$\emptyset$
$\{s_0, s_1\}$	$\{s_0, s_1\}$	$\emptyset$	$\{s_2\}$
$\{s_2\}$	$\emptyset$	$\emptyset$	$\{s_0, s_1, s_3\}$
$\{s_0, s_1, s_3\}$	$\{s_0, s_1\}$	$\emptyset$	$\{s_2\}$
$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$

(g) Figure 4.25(g): start state $\{s_0\}$, final state $\{s_0\}$.

	a	b	c
$\{s_0\}$	$\{s_1\}$	$\emptyset$	$\emptyset$
$\{s_1\}$	$\{s_0\}$	$\{s_3\}$	$\{s_2\}$
$\{s_2\}$	$\emptyset$	$\emptyset$	$\{s_1\}$
$\{s_3\}$	$\emptyset$	$\{s_1\}$	$\emptyset$
$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$

4.2. Consider Example 4.17, that is, the automaton E of Figure 4.23. Reading the figure gives

$$E = \langle \{\mathbf{a}, \mathbf{b}\}, \{s_1, s_2\}, \{s_1\}, \delta_\lambda, \{s_2\} \rangle$$

with ordinary transitions

$$\delta_\lambda(s_1, \mathbf{a}) = \{s_1, s_2\}, \quad \delta_\lambda(s_1, \mathbf{b}) = \emptyset,$$

$$\delta_\lambda(s_2, \mathbf{a}) = \{s_1\}, \quad \delta_\lambda(s_2, \mathbf{b}) = \{s_1\},$$

and one λ -transition $\delta_\lambda(s_1, \lambda) = \{s_2\}$.

(a) *Remove λ -transitions (Definition 4.9).* The λ -closures are:

$$\Lambda(s_1) = \{s_1, s_2\}, \quad \Lambda(s_2) = \{s_2\}.$$

Using the λ -closure of the original start state, the corresponding λ -free NDFA has start set $\{s_1, s_2\}$. Thus

$$E_\lambda^\sigma = \langle \{\mathbf{a}, \mathbf{b}\}, \{s_1, s_2\}, \{s_1, s_2\}, \delta_\lambda^\sigma, \{s_2\} \rangle$$

with

$$\delta_\lambda^\sigma(s_1, \mathbf{a}) = \{s_1, s_2\}, \quad \delta_\lambda^\sigma(s_1, \mathbf{b}) = \{s_1, s_2\},$$

$$\delta_\lambda^\sigma(s_2, \mathbf{a}) = \{s_1, s_2\}, \quad \delta_\lambda^\sigma(s_2, \mathbf{b}) = \{s_1, s_2\}.$$

- (b) *Convert this NDEFA into a DFA (Definition 4.5).* The full deterministic machine has $2^2 = 4$ subset states, but only one is reachable from the start set $\{s_1, s_2\}$:

$$Q_0 = \{s_1, s_2\}.$$

This state is final because it contains s_2 . Its transitions are

	a	b
Q_0	Q_0	Q_0

so the connected part of the DFA is a one-state accepting machine.

- 4.3. Consider the NDEFA A from Example 4.4: $A = \langle \{\mathbf{a}, \mathbf{b}\}, \{r, s, t\}, \{r, s\}, \delta, \{t\} \rangle$ with

$$\begin{aligned} \delta(r, \mathbf{a}) &= \{s\}, & \delta(r, \mathbf{b}) &= \emptyset, & \delta(s, \mathbf{a}) &= \emptyset, & \delta(s, \mathbf{b}) &= \{r, t\}, \\ \delta(t, \mathbf{a}) &= \emptyset, & \delta(t, \mathbf{b}) &= \{t\}. \end{aligned}$$

- (a) *Circuit diagram (no <SOS> or <EOS>).* Using one activity bit for each state, the next-state equations are

$$r' = s \wedge \mathbf{b}, \quad s' = r \wedge \mathbf{a}, \quad t' = (s \wedge \mathbf{b}) \vee (t \wedge \mathbf{b}).$$

Initially the active states are r and s , and t is inactive.

- (b) *Circuit diagram (with <SOS> and <EOS>).* <SOS> resets the activity bits to $r = s = 1$, $t = 0$. The accept line is $t \wedge \text{<EOS>}$.
- (c) *DFA via subset construction (connected portion only).* The start state of A^d is $\{r, s\}$. The reachable states are

$$\{r, s\}, \quad \{s\}, \quad \{r, t\}, \quad \{t\}, \quad \emptyset.$$

Their transition table is

State	a	b
$\{r, s\}$	$\{s\}$	$\{r, t\}$
$\{s\}$	$\emptyset$	$\{r, t\}$
$\{r, t\}$	$\{s\}$	$\{t\}$
$\{t\}$	$\emptyset$	$\{t\}$
$\emptyset$	$\emptyset$	$\emptyset$

The final states are $\{r, t\}$ and $\{t\}$.

- (d) *DFA including disconnected portion.* The full state set is

$$\wp(\{r, s, t\}) = \{\emptyset, \{r\}, \{s\}, \{t\}, \{r, s\}, \{r, t\}, \{s, t\}, \{r, s, t\}\}.$$

Its transition table is

State	a	b
$\emptyset$	$\emptyset$	$\emptyset$
$\{r\}$	$\{s\}$	$\emptyset$
$\{s\}$	$\emptyset$	$\{r, t\}$
$\{t\}$	$\emptyset$	$\{t\}$
$\{r, s\}$	$\{s\}$	$\{r, t\}$
$\{r, t\}$	$\{s\}$	$\{t\}$
$\{s, t\}$	$\emptyset$	$\{r, t\}$
$\{r, s, t\}$	$\{s\}$	$\{r, t\}$

Final states are exactly those subsets containing t .

- 4.4. Consider the NDFA from Example 4.2, which has multiple transitions on **0**: from s_0 to both s_1 and s_2 .

- (a) *Circuit diagram (with <SOS> and <EOS>).* Use one activity flip-flop for each state $s_0, \dots, s_5$. The next-state equations are

$$s'_0 = \text{<SOS>}, \quad s'_1 = (s_0 \wedge \mathbf{0}) \vee (s_1 \wedge \mathbf{0}) \vee (s_5 \wedge \mathbf{1}),$$

$$s'_2 = (s_0 \wedge \mathbf{0}) \vee (s_0 \wedge \mathbf{1}) \vee (s_4 \wedge \mathbf{0}) \vee (s_4 \wedge \mathbf{1}),$$

$$s'_3 = (s_0 \wedge \mathbf{1}) \vee (s_1 \wedge \mathbf{1}) \vee (s_3 \wedge \mathbf{0}),$$

$$s'_4 = (s_2 \wedge \mathbf{0}) \vee (s_2 \wedge \mathbf{1}), \quad s'_5 = (s_3 \wedge \mathbf{1}) \vee (s_5 \wedge \mathbf{0}).$$

Since the final states in Figure 4.3 are s_0 , s_1 , and s_4 , the accept line is

$$(s_0 \vee s_1 \vee s_4) \wedge \text{<EOS>}.$$

- (b) *DFA via subset construction.* Starting from the set of start state $\{s_0\}$, the connected portion of the subset DFA has the following reachable states:

$$Q_0 = \{s_0\}, \quad Q_1 = \{s_1, s_2\}, \quad Q_2 = \{s_2, s_3\}, \quad Q_3 = \{s_1, s_4\}, \quad Q_4 = \{s_3, s_4\},$$

$$Q_5 = \{s_4, s_5\}, \quad Q_6 = \{s_2, s_5\}.$$

Their transitions are

	0	1
Q_0	Q_1	Q_2
Q_1	Q_3	Q_4
Q_2	Q_4	Q_5
Q_3	Q_1	Q_2
Q_4	Q_2	Q_6
Q_5	Q_6	Q_3
Q_6	Q_5	Q_3

The final states are exactly those subsets containing one of the original final states s_0 , s_1 , or s_4 , namely

$$Q_0, Q_1, Q_3, Q_4, Q_5.$$

- 4.5. Consider the NDFA from Example 4.3, which accepts the same language as Example 4.2 but uses multiple start states instead of multiple transitions.

- (a) *Circuit diagram (no <SOS> or <EOS>).* Again use one activity flip-flop per state. For the automaton of Figure 4.4,

$$s'_1 = (s_1 \wedge \mathbf{0}) \vee (s_5 \wedge \mathbf{0}), \quad s'_2 = (s_4 \wedge \mathbf{0}) \vee (s_4 \wedge \mathbf{1}),$$

$$s'_3 = (s_1 \wedge \mathbf{1}) \vee (s_3 \wedge \mathbf{0}), \quad s'_4 = (s_2 \wedge \mathbf{0}) \vee (s_2 \wedge \mathbf{1}),$$

$$s'_5 = s_3 \wedge \mathbf{1}.$$

The initial configuration has $s_1 = s_4 = 1$ and the other activity bits equal to 0. Since the final states are s_1 and s_4 , the accept line is $s_1 \vee s_4$.

- (b) *Circuit diagram (with <SOS> and <EOS>).* Keep the equations for s'_2 , s'_3 , and s'_5 from part (a), keep $s'_0 = 0$, and modify only the two start-state inputs by OR-ing in the start pulse:

$$s'_1 = \text{<SOS>} \vee (s_1 \wedge \mathbf{0}) \vee (s_5 \wedge \mathbf{0}), \quad s'_4 = \text{<SOS>} \vee (s_2 \wedge \mathbf{0}) \vee (s_2 \wedge \mathbf{1}),$$

with the equations for s'_2 , s'_3 , and s'_5 unchanged. The accept output is

$$(s_1 \vee s_4) \wedge \text{<EOS>}.$$

- (c) *DFA (connected portion only).* The start state of A^d is the set of all start states, namely $\{s_1, s_4\}$. The reachable subsets are

$$P_0 = \{s_1, s_4\}, \quad P_1 = \{s_1, s_2\}, \quad P_2 = \{s_2, s_3\}, \quad P_3 = \{s_3, s_4\}, \quad P_4 = \{s_4, s_5\}, \quad P_5 = \{s_2, s_5\},$$

$$P_6 = \{s_2\}, \quad P_7 = \{s_4\}.$$

Their transitions are

	0	1
P_0	P_1	P_2
P_1	P_0	P_3
P_2	P_3	P_4
P_3	P_2	P_5
P_4	P_1	P_6
P_5	P_0	P_7
P_6	P_7	P_7
P_7	P_6	P_6

The final states are

$$P_0, P_1, P_3, P_4, P_7,$$

since these are exactly the reachable subsets containing s_1 or s_4 .

- (d) *Is this DFA isomorphic to any of those in Exercise 4.4?* No. The DFA from Exercise 4.4(b) has seven reachable states

$$Q_0, \dots, Q_6,$$

while the DFA from part (c) above has eight reachable states

$$P_0, \dots, P_7.$$

Since isomorphic automata must have the same number of states, these two subset-construction DFAs are not isomorphic. They do accept the same language, but that only implies that their *minimal* DFAs are isomorphic, not these unreduced machines.

- 4.6. Consider the λ -NDEFA from Example 4.14, with states $\{s_0, s_1, s_2, s_3\}$ and λ -transitions $s_0 \rightarrow s_1$ and $s_1 \rightarrow s_2$.

- (a) *Circuit diagram for the original λ -NDEFA (no SOS/EOS).* Figure 4.19 uses four activity bits, one for each state. Ignoring the λ -moves for a moment, the ordinary transitions give

$$s'_0 = 0, \quad s'_1 = (s_0 \wedge \mathbf{a}) \vee (s_3 \wedge \mathbf{b}),$$

$$s'_2 = (s_1 \wedge \mathbf{c}) \vee (s_2 \wedge \mathbf{d}), \quad s'_3 = s_1 \wedge \mathbf{b}.$$

Because a λ -move consumes no input symbol, whenever s_0 is active the next-state logic must also make s_1 active, and whenever s_1 is active it must also make s_2 active. This is achieved by OR-ing the source activity line directly into the destination input. Thus the circuit can be specified by

$$\begin{aligned} D_0 &= 0, & D_1 &= s_0 \vee (s_0 \wedge \mathbf{a}) \vee (s_3 \wedge \mathbf{b}), \\ D_2 &= s_1 \vee (s_1 \wedge \mathbf{c}) \vee (s_2 \wedge \mathbf{d}), & D_3 &= s_1 \wedge \mathbf{b}. \end{aligned}$$

Initially only s_0 is active; the accept output is simply s_2 .

- (b) *Circuit diagram for the NDFA from Definition 4.9 (part (b) of Example 4.14, no SOS/EOS).* The corresponding λ -free automaton is the one shown in Figure 4.20. Use one activity bit for each of q_0, s_0, s_1, s_2, s_3 ; then

$$\begin{aligned} q'_0 &= 0, & s'_0 &= 0, \\ s'_1 &= (s_0 \wedge \mathbf{a}) \vee (s_3 \wedge \mathbf{b}), \\ s'_2 &= (s_0 \wedge \mathbf{a}) \vee (s_0 \wedge \mathbf{c}) \vee (s_0 \wedge \mathbf{d}) \vee (s_1 \wedge \mathbf{c}) \vee (s_1 \wedge \mathbf{d}) \vee (s_3 \wedge \mathbf{b}) \vee (s_2 \wedge \mathbf{d}), \\ s'_3 &= (s_0 \wedge \mathbf{b}) \vee (s_1 \wedge \mathbf{b}). \end{aligned}$$

The initial activity bits are $q_0 = s_0 = 1$ and $s_1 = s_2 = s_3 = 0$, and the accept output is $q_0 \vee s_2$.

- 4.7. Consider the second NDFA of Example 4.16, which accepts the language $L^+ = \{x_1 x_2 \cdots x_k \mid k \geq 1, x_i \in L\}$ where L is the language of strings over $\{\mathbf{a}, \mathbf{b}, \mathbf{c}\}$ containing exactly one $\mathbf{b}$ immediately followed by $\mathbf{c}$.

- (a) *Circuit diagram for A_λ^σ (with EOS but no SOS).* For the λ -NDFA of Figure 4.22, use one activity bit for each of s_0, s_1, s_2 . The ordinary transitions give

$$\begin{aligned} s'_0 &= s_0 \wedge (\mathbf{a} \vee \mathbf{c}), & s'_1 &= s_0 \wedge \mathbf{b}, \\ s'_2 &= (s_1 \wedge \mathbf{c}) \vee (s_2 \wedge (\mathbf{a} \vee \mathbf{c})). \end{aligned}$$

Because Figure 4.22 also has the λ -transition $s_2 \rightarrow s_0$, OR the activity line for s_2 directly into the input for s_0 . Thus

$$\begin{aligned} D_0 &= s_2 \vee (s_0 \wedge (\mathbf{a} \vee \mathbf{c})), & D_1 &= s_0 \wedge \mathbf{b}, \\ D_2 &= (s_1 \wedge \mathbf{c}) \vee (s_2 \wedge (\mathbf{a} \vee \mathbf{c})). \end{aligned}$$

The accept line is $s_2 \wedge \langle \text{EOS} \rangle$.

- (b) *NDFA without λ -transitions (Definition 4.9).* For Figure 4.22 the λ -closures are

$$\Lambda(s_0) = \{s_0\}, \quad \Lambda(s_1) = \{s_1\}, \quad \Lambda(s_2) = \{s_0, s_2\}.$$

Since $\lambda \notin L(A_\lambda)$, Definition 4.9 adds q_0 as an extra disconnected start state but does not make it final. Thus

$$A_\lambda^\sigma = \langle \{\mathbf{a}, \mathbf{b}, \mathbf{c}\}, \{q_0, s_0, s_1, s_2\}, \{q_0, s_0\}, \delta_\lambda^\sigma, \{s_2\} \rangle,$$

where

$$\begin{aligned} \delta_\lambda^\sigma(q_0, \mathbf{a}) &= \delta_\lambda^\sigma(q_0, \mathbf{b}) = \delta_\lambda^\sigma(q_0, \mathbf{c}) = \emptyset, \\ \delta_\lambda^\sigma(s_0, \mathbf{a}) &= \{s_0\}, & \delta_\lambda^\sigma(s_0, \mathbf{b}) &= \{s_1\}, & \delta_\lambda^\sigma(s_0, \mathbf{c}) &= \{s_0\}, \\ \delta_\lambda^\sigma(s_1, \mathbf{a}) &= \emptyset, & \delta_\lambda^\sigma(s_1, \mathbf{b}) &= \emptyset, & \delta_\lambda^\sigma(s_1, \mathbf{c}) &= \{s_0, s_2\}, \\ \delta_\lambda^\sigma(s_2, \mathbf{a}) &= \{s_0, s_2\}, & \delta_\lambda^\sigma(s_2, \mathbf{b}) &= \{s_1\}, & \delta_\lambda^\sigma(s_2, \mathbf{c}) &= \{s_0, s_2\}. \end{aligned}$$

- (c) *Circuit diagram for the λ -free NDEFA (with EOS, no SOS).* Use one activity bit for each of q_0, s_0, s_1, s_2 . The next-state equations are

$$q'_0 = 0,$$

$$s'_0 = (s_0 \wedge \mathbf{a}) \vee (s_0 \wedge \mathbf{c}) \vee (s_1 \wedge \mathbf{c}) \vee (s_2 \wedge \mathbf{a}) \vee (s_2 \wedge \mathbf{c}),$$

$$s'_1 = (s_0 \wedge \mathbf{b}) \vee (s_2 \wedge \mathbf{b}),$$

$$s'_2 = (s_1 \wedge \mathbf{c}) \vee (s_2 \wedge \mathbf{a}) \vee (s_2 \wedge \mathbf{c}).$$

The initial configuration has $q_0 = s_0 = 1$ and $s_1 = s_2 = 0$, and the accept line is $s_2 \wedge \langle \text{EOS} \rangle$.

- (d) *DFA (connected portion only).* Starting from $\{q_0, s_0\}$, the reachable states are

$$Q_0 = \{q_0, s_0\}, \quad Q_1 = \{s_0\}, \quad Q_2 = \{s_1\}, \quad Q_3 = \emptyset, \quad Q_4 = \{s_0, s_2\}.$$

Their transitions are

	$\mathbf{a}$	$\mathbf{b}$	$\mathbf{c}$
Q_0	Q_1	Q_2	Q_1
Q_1	Q_1	Q_2	Q_1
Q_2	Q_3	Q_3	Q_4
Q_3	Q_3	Q_3	Q_3
Q_4	Q_4	Q_2	Q_4

The only final state is Q_4 .

4.8. Complementing the final states of an NDEFA does not yield a machine accepting the complement language.

For a DFA, $L(\bar{A}) = \Sigma^* \setminus L(A)$ because every word follows exactly one path and is accepted iff it ends in a final state. Flipping final and nonfinal states reverses acceptance on that unique path.

For an NDEFA, a word w may follow *multiple* paths, some ending in final states and some not. Under Definition 4.4, $w \in L(A)$ iff *at least one* path ends in a final state. If we complement the final states, a word is accepted by the modified machine iff at least one path ends in a *formerly nonfinal* state. This is *not* the same as $w \notin L(A)$, because a word can simultaneously have a path to a (formerly) final state and a path to a (formerly) nonfinal state; the complemented machine would accept it, but it was already in $L(A)$.

Example. Let $\Sigma = \{\mathbf{a}\}$, $S = \{s_0, s_1\}$, $S_0 = \{s_0\}$, $F = \{s_1\}$, $\delta(s_0, \mathbf{a}) = \{s_0, s_1\}$, $\delta(s_1, \mathbf{a}) = \emptyset$. Then $\mathbf{a} \in L(A)$ because $s_1 \in \delta(s_0, \mathbf{a})$. After complementing, $F' = \{s_0\}$, and $\mathbf{a}$ is still accepted (via the path $s_0 \rightarrow s_0 \in F'$), yet $\mathbf{a} \notin \Sigma^* \setminus L(A)$ since $\mathbf{a} \in L(A)$. Hence the complement-of-final-states trick fails for NDFAs.

4.9. Given an NDEFA A without λ -transitions, construct a λ -NDEFA A' with exactly one start state, exactly one final state, and $L(A') = L(A)$.

Construction. Let $A = \langle \Sigma, S, S_0, \delta, F \rangle$. Define $A' = \langle \Sigma, S', \{q_0\}, \delta', \{q_f\} \rangle$ where:

- $S' = S \cup \{q_0, q_f\}$ (two new states).
- q_0 is the unique start state.
- $\delta'(q_0, \lambda) = S_0$ (a λ -transition from q_0 to every start state of A).
- For all $s \in S$ and $\mathbf{a} \in \Sigma$: $\delta'(s, \mathbf{a}) = \delta(s, \mathbf{a})$.

- $\delta'(f, \lambda) \ni q_f$ for every $f \in F$ (λ -transitions from each old final state to the new unique final state q_f).
- $\delta'(q_f, \mathbf{a}) = \emptyset$ for all $\mathbf{a}$ (and no λ -transitions out of q_f).

Correctness. A word $w \in \Sigma^+$ is accepted by A iff there is a path $q_0 \xrightarrow{\lambda} s_i \xrightarrow{w} f$ for some $s_i \in S_0$ and $f \in F$; by the λ -transition $f \xrightarrow{\lambda} q_f$, this same path exists in A' ending at q_f . Conversely, any accepting path in A' uses the initial λ -step to some $s_i \in S_0$, processes w within S (by identical transitions), then uses a final λ -step to q_f , which corresponds to an accepting path in A .

If $\lambda \in L(A)$, add the extra λ -transition $\delta'(q_0, \lambda) \ni q_f$. Then λ is accepted while A' still has exactly one start state (q_0) and exactly one final state (q_f). Hence $L(A') = L(A)$ in all cases.

4.10. Can part (ii) of Definition 4.8 be deduced from parts (i) and (iii)?

Definition 4.8 defines $\bar{\delta}$ for NDFAs:

- (i) $\bar{\delta}(s, \lambda) = \{s\}$,
- (ii) $(\forall \mathbf{a} \in \Sigma) \bar{\delta}(s, \mathbf{a}) = \delta(s, \mathbf{a})$,
- (iii) $(\forall x \in \Sigma^*) (\forall \mathbf{a} \in \Sigma) \bar{\delta}(s, x\mathbf{a}) = \bigcup_{t \in \bar{\delta}(s, x)} \delta(t, \mathbf{a})$.

Part (ii) concerns single-letter strings. Setting $x = \lambda$ in part (iii):

$$\bar{\delta}(s, \lambda\mathbf{a}) = \bigcup_{t \in \bar{\delta}(s, \lambda)} \delta(t, \mathbf{a}) = \bigcup_{t \in \{s\}} \delta(t, \mathbf{a}) = \delta(s, \mathbf{a}),$$

where we used part (i) to obtain $\bar{\delta}(s, \lambda) = \{s\}$ and the fact that $\lambda\mathbf{a} = \mathbf{a}$. Since $\lambda\mathbf{a} = \mathbf{a}$, this gives $\bar{\delta}(s, \mathbf{a}) = \delta(s, \mathbf{a})$, which is exactly part (ii).

Conclusion. Yes, part (ii) can be derived from parts (i) and (iii). It is included explicitly to make the base case of inductive proofs more transparent, but it is logically redundant.

4.11. Alternative construction for removing λ -transitions.

Define $S' = S$, $S'_0 = \Lambda(S_0)$, $F' = F$, and $\delta'(s, \mathbf{a}) = \bar{\delta}_\lambda(s, \mathbf{a})$ for all $s \in S$, $\mathbf{a} \in \Sigma$.

This construction does work. Let

$$A' = \langle \Sigma, S, \Lambda(S_0), \delta', F \rangle.$$

We show $L(A') = L(A_\lambda)$.

First, for every $s \in S$ and every nonempty word $x \in \Sigma^+$,

$$\bar{\delta}'(s, x) = \bar{\delta}_\lambda(s, x).$$

We prove this by induction on $|x|$.

Base case. If $x = \mathbf{a} \in \Sigma$, then by definition of δ' ,

$$\bar{\delta}'(s, \mathbf{a}) = \delta'(s, \mathbf{a}) = \bar{\delta}_\lambda(s, \mathbf{a}).$$

Inductive step. Assume $\bar{\delta}'(s, x) = \bar{\delta}_\lambda(s, x)$ for some $x \in \Sigma^+$. Let $\mathbf{a} \in \Sigma$. Then

$$\begin{aligned}\bar{\delta}'(s, x\mathbf{a}) &= \bigcup_{t \in \bar{\delta}'(s, x)} \delta'(t, \mathbf{a}) \\ &= \bigcup_{t \in \bar{\delta}_\lambda(s, x)} \bar{\delta}_\lambda(t, \mathbf{a}) \quad (\text{induction hypothesis and definition of } \delta').\end{aligned}$$

Now $\bar{\delta}_\lambda(s, x)$ is λ -closed, because every value of $\bar{\delta}_\lambda$ is obtained by applying Λ . Hence if $t \in \bar{\delta}_\lambda(s, x)$, then $\Lambda(t) \subseteq \bar{\delta}_\lambda(s, x)$, and so

$$\bar{\delta}_\lambda(t, \mathbf{a}) = \Lambda\left(\bigcup_{q \in \Lambda(t)} \delta_\lambda(q, \mathbf{a})\right) \subseteq \Lambda\left(\bigcup_{q \in \bar{\delta}_\lambda(s, x)} \delta_\lambda(q, \mathbf{a})\right) = \bar{\delta}_\lambda(s, x\mathbf{a}).$$

This shows

$$\bigcup_{t \in \bar{\delta}_\lambda(s, x)} \bar{\delta}_\lambda(t, \mathbf{a}) \subseteq \bar{\delta}_\lambda(s, x\mathbf{a}).$$

For the reverse inclusion, if $r \in \bar{\delta}_\lambda(s, x\mathbf{a})$, then by definition there is some $q \in \bar{\delta}_\lambda(s, x)$ and some $u \in \Lambda(q) \subseteq \bar{\delta}_\lambda(s, x)$ such that r is reached from u by reading $\mathbf{a}$ followed by λ -moves. Therefore $r \in \bar{\delta}_\lambda(u, \mathbf{a})$ for some $u \in \bar{\delta}_\lambda(s, x)$, so

$$\bar{\delta}_\lambda(s, x\mathbf{a}) \subseteq \bigcup_{t \in \bar{\delta}_\lambda(s, x)} \bar{\delta}_\lambda(t, \mathbf{a}).$$

Thus

$$\bar{\delta}'(s, x\mathbf{a}) = \bar{\delta}_\lambda(s, x\mathbf{a}),$$

completing the induction.

Case 1: $x = \lambda$. Then

$$\lambda \in L(A') \iff \Lambda(S_0) \cap F \neq \emptyset.$$

But this says exactly that some start state of A_λ can reach a final state by λ -moves alone, which is the condition $\lambda \in L(A_\lambda)$.

Case 2: $x \in \Sigma^+$. Using the identity above,

$$x \in L(A') \iff \left(\bigcup_{t \in \Lambda(S_0)} \bar{\delta}'(t, x)\right) \cap F \neq \emptyset \iff \left(\bigcup_{t \in \Lambda(S_0)} \bar{\delta}_\lambda(t, x)\right) \cap F \neq \emptyset.$$

If $t \in \Lambda(S_0)$, then $t \in \Lambda(s_0)$ for some $s_0 \in S_0$, so the initial λ -moves taking s_0 to t may be prefixed to any path counted in $\bar{\delta}_\lambda(t, x)$. Hence

$$\bigcup_{t \in \Lambda(S_0)} \bar{\delta}_\lambda(t, x) = \bigcup_{s_0 \in S_0} \bar{\delta}_\lambda(s_0, x),$$

and therefore

$$x \in L(A') \iff \left(\bigcup_{s_0 \in S_0} \bar{\delta}_\lambda(s_0, x)\right) \cap F \neq \emptyset \iff x \in L(A_\lambda).$$

So this is a valid alternative to Definition 4.9: instead of adding q_0 , it enlarges the start-state set to $\Lambda(S_0)$.

4.12. **Algorithm for union of two FAD languages using λ -NDFAs.**

Let $L_1 = L(A_1)$ and $L_2 = L(A_2)$ where A_1 and A_2 are λ -NDFAs. By Exercise 4.9 we may assume each A_i has exactly one start state q_0^i and exactly one final state q_f^i . Construct A' as follows:

- (1) Introduce a new start state q_0 and a new final state q_f .
- (2) Add λ -transitions: $q_0 \xrightarrow{\lambda} q_0^1$ and $q_0 \xrightarrow{\lambda} q_0^2$.
- (3) Add λ -transitions: $q_f^1 \xrightarrow{\lambda} q_f$ and $q_f^2 \xrightarrow{\lambda} q_f$.
- (4) Keep all existing transitions of A_1 and A_2 .

Then $L(A') = L(A_1) \cup L(A_2)$: a word is accepted iff it is accepted by A_1 or by A_2 , because the λ -transitions route the computation into one of the two machines and collect the result at q_f .

4.13. **Examples of three NDFAs such that the corresponding A^d is respectively not connected, not reduced, and minimal.**

(a) **A^d is not connected.** Any NFA in which some subsets of states are never reachable from the start set under the transition function will produce an A^d with unreachable states. For example, let $\Sigma = \{a\}$, $S = \{s_0, s_1\}$, $S_0 = \{s_0\}$, $F = \{s_1\}$, $\delta(s_0, a) = \{s_1\}$, $\delta(s_1, a) = \emptyset$. Then A^d has states $\{\emptyset, \{s_0\}, \{s_1\}, \{s_0, s_1\}\}$ but $\{s_0, s_1\}$ and $\{s_1\}$ with $\delta^d(\{s_1\}, a) = \emptyset$ are reachable, while $\{s_0, s_1\}$ is not reached from $\{s_0\}$.

(b) **A^d is not reduced.** Use a different example:

$$A = \langle \{a, b\}, \{p, u, v\}, \{p\}, \delta, \{u, v\} \rangle$$

where

$$\delta(p, a) = \{u\}, \quad \delta(p, b) = \{v\},$$

and all other transitions are empty. Then the reachable subset states of A^d are

$$\{p\}, \quad \{u\}, \quad \{v\}, \quad \emptyset.$$

Both $\{u\}$ and $\{v\}$ are final, and from either one every input leads to $\emptyset$. Therefore

$$\{u\} E_{A^d} \{v\},$$

so A^d is not reduced.

(c) **A^d is minimal.** Here we use a different example. Let $A = \langle \{a, b\}, \{s\}, \{s\}, \delta, \{s\} \rangle$ where

$$\delta(s, a) = \{s\}, \quad \delta(s, b) = \emptyset.$$

Then $L(A) = a^*$. The deterministic machine A^d has exactly two states, $\{s\}$ and $\emptyset$, with

$$\delta^d(\{s\}, a) = \{s\}, \quad \delta^d(\{s\}, b) = \emptyset, \quad \delta^d(\emptyset, a) = \delta^d(\emptyset, b) = \emptyset.$$

This is the standard 2-state DFA for a^* over the alphabet $\{a, b\}$, so A^d is minimal.

4.14. **Why was the extra state q_0 included in A_λ^σ ?**

The role of q_0 is to preserve acceptance of λ while leaving the original start states and all ordinary transitions unchanged. In Definition 4.9 the extra state q_0 is disconnected from the rest of the machine, and

$$q_0 \in F^\sigma \iff \lambda \in L(A_\lambda).$$

Thus q_0 serves as a separate witness for the empty word.

Example. Let

$$A_\lambda = \langle \{\mathbf{a}\}, \{s_0, f\}, \{s_0\}, \delta_\lambda, \{f\} \rangle$$

where $\delta_\lambda(s_0, \lambda) = \{f\}$ and all ordinary transitions are empty. Then $\lambda \in L(A_\lambda)$ because $f \in \Lambda(s_0)$.

If we try to remove λ -moves *without* adding q_0 and keep the same start state s_0 , then the new machine has no way to accept λ : no input is read, and s_0 is not final. The disconnected state q_0 fixes exactly this problem. We keep s_0 for all nonempty words, and we make q_0 final so that the empty word is still accepted.

4.15. **Union construction using NDFAs *without* λ -transitions.**

- (a) **Algorithm.** Let $A_1 = \langle \Sigma, S_1, S_0^1, \delta_1, F_1 \rangle$ and $A_2 = \langle \Sigma, S_2, S_0^2, \delta_2, F_2 \rangle$ (assume $S_1 \cap S_2 = \emptyset$). Construct $A' = \langle \Sigma, S_1 \cup S_2, S_0^1 \cup S_0^2, \delta', F_1 \cup F_2 \rangle$ where $\delta'(s, \mathbf{a}) = \delta_i(s, \mathbf{a})$ for $s \in S_i$. The machine A' starts in the combined set of initial states and accepts any word accepted by either A_1 or A_2 . Thus $L(A') = L(A_1) \cup L(A_2)$.
- (b) **Comparison.** This is actually *easier* than the λ -transition approach (Exercise 4.12): no new states are needed at all, since we simply merge the state sets and initial-state sets.
- (c) **Can we ensure exactly one start state and one final state?** Not in full generality. The disjoint-union construction in part (a) gives a correct union machine, but it usually has several start states and several final states. If the resulting language does *not* contain λ , then Exercise 4.22 shows how to convert a no- λ NDFA into another no- λ NDFA with exactly one start state and exactly one final state. But Exercise 4.23 shows that this can fail when λ is in the language. So the correct conclusion is: one start state and one final state can be forced under the hypothesis $\lambda \notin L$, but not uniformly in general.

4.16. Consider the λ -NDFA A_λ^σ from Example 4.15 (the machine without λ -transitions corresponding to the automaton of Example 4.14).

- (a) *Circuit diagram (no SOS or EOS).* The equivalent λ -free machine has the five states q_0, s_0, s_1, s_2, s_3 , so we use one activity bit for each of them. The next-state equations are

$$\begin{aligned} q_0' &= 0, & s_0' &= 0, \\ s_1' &= (s_0 \wedge \mathbf{a}) \vee (s_3 \wedge \mathbf{b}), \\ s_2' &= (s_0 \wedge \mathbf{a}) \vee (s_0 \wedge \mathbf{c}) \vee (s_0 \wedge \mathbf{d}) \vee (s_1 \wedge \mathbf{c}) \vee (s_1 \wedge \mathbf{d}) \vee (s_3 \wedge \mathbf{b}) \vee (s_2 \wedge \mathbf{d}), \\ s_3' &= (s_0 \wedge \mathbf{b}) \vee (s_1 \wedge \mathbf{b}). \end{aligned}$$

The initial configuration has $q_0 = s_0 = 1$ and $s_1 = s_2 = s_3 = 0$. Since q_0 and s_2 are final states in Example 4.15, the accept output is $q_0 \vee s_2$.

(b) *Convert to $A_\lambda^{\sigma d}$ (connected portion only).* In Example 4.15, the start states of A_λ^σ are

$$\{q_0, s_0\},$$

and the final states are q_0 and s_2 . The connected part of the deterministic equivalent has the reachable states

$$R_0 = \{q_0, s_0\}, \quad R_1 = \{s_1, s_2\}, \quad R_2 = \{s_3\}, \quad R_3 = \{s_2\}, \quad R_4 = \emptyset.$$

Their transitions are

	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>
R_0	R_1	R_2	R_3	R_3
R_1	R_4	R_2	R_3	R_3
R_2	R_4	R_1	R_4	R_4
R_3	R_4	R_4	R_4	R_3
R_4	R_4	R_4	R_4	R_4

The final states are

$$R_0, R_1, R_3,$$

because these are exactly the reachable subsets containing q_0 or s_2 .

4.17. (a) **Proof that $\bar{\delta}(s, \mathbf{a}) = \delta(s, \mathbf{a})$ for any NDFA without λ -transitions.**

By Definition 4.8(ii), $\bar{\delta}(s, \mathbf{a}) = \delta(s, \mathbf{a})$ for all $s \in S$ and $\mathbf{a} \in \Sigma$. This is immediate from the definition; alternatively, apply part (iii) with $x = \lambda$:

$$\bar{\delta}(s, \lambda \mathbf{a}) = \bigcup_{t \in \bar{\delta}(s, \lambda)} \delta(t, \mathbf{a}) = \bigcup_{t \in \{s\}} \delta(t, \mathbf{a}) = \delta(s, \mathbf{a}),$$

using part (i): $\bar{\delta}(s, \lambda) = \{s\}$. Since $\lambda \mathbf{a} = \mathbf{a}$, we conclude $\bar{\delta}(s, \mathbf{a}) = \delta(s, \mathbf{a})$. $\square$

(b) **Counterexample for λ -NDFAs.**

Let $\Sigma = \{\mathbf{a}\}$, $S = \{s_0, s_1\}$, $S_0 = \{s_0\}$, $F = \{s_1\}$, $\delta_\lambda(s_0, \lambda) = \{s_1\}$, $\delta_\lambda(s_0, \mathbf{a}) = \emptyset$, $\delta_\lambda(s_1, \mathbf{a}) = \emptyset$.

Then $\Lambda(s_0) = \{s_0, s_1\}$. So

$$\bar{\delta}_\lambda(s_0, \mathbf{a}) = \Lambda\left(\bigcup_{t \in \Lambda(s_0)} \delta_\lambda(t, \mathbf{a})\right) = \Lambda(\emptyset \cup \emptyset) = \emptyset,$$

but $\delta_\lambda(s_0, \mathbf{a}) = \emptyset$ also. Let us try a different example: $\delta_\lambda(s_0, \lambda) = \{s_1\}$, $\delta_\lambda(s_1, \mathbf{a}) = \{s_1\}$. Then $\delta_\lambda(s_0, \mathbf{a}) = \emptyset$, yet

$$\bar{\delta}_\lambda(s_0, \mathbf{a}) = \Lambda\left(\bigcup_{t \in \{s_0, s_1\}} \delta_\lambda(t, \mathbf{a})\right) = \Lambda(\{s_1\}) = \{s_1\} \neq \emptyset = \delta_\lambda(s_0, \mathbf{a}).$$

This demonstrates that $\bar{\delta}_\lambda(s, \mathbf{a}) \neq \delta_\lambda(s, \mathbf{a})$ is possible for λ -NDFAs.

4.18. Consider the NDFA accepting the original language L from Example 4.10: $L = \{x \in \{\mathbf{a}, \mathbf{b}\}^* \mid x \text{ begins with } \mathbf{a} \vee x \text{ contains } \mathbf{ba}\}$.

- (a) *Circuit diagram (no SOS/EOS)*. The NDEFA has a start state q_0 , two final states q_1, q_3 , and the transitions of Figure 4.10. Using one activity bit per state, the next-state equations are

$$\begin{aligned} q'_0 &= 0, & q'_1 &= (q_0 \wedge \mathbf{a}) \vee (q_1 \wedge \mathbf{a}) \vee (q_1 \wedge \mathbf{b}), \\ q'_2 &= (q_0 \wedge \mathbf{b}) \vee (q_2 \wedge \mathbf{b}), & q'_3 &= (q_2 \wedge \mathbf{a}) \vee (q_3 \wedge \mathbf{a}) \vee (q_3 \wedge \mathbf{b}). \end{aligned}$$

Initially only q_0 is active, and the accept line is $q_1 \vee q_3$.

- (b) *DFA (connected portion only)*. Apply the subset construction to Figure 4.10. Starting from $\{q_0\}$, the reachable states are

$$Q_0 = \{q_0\}, \quad Q_1 = \{q_1\}, \quad Q_2 = \{q_2\}, \quad Q_3 = \{q_3\}.$$

Their transitions are

	a	b
Q_0	Q_1	Q_2
Q_1	Q_1	Q_1
Q_2	Q_3	Q_2
Q_3	Q_3	Q_3

The final states are Q_1 and Q_3 .

4.19. Consider the NDEFA accepting $L^r = \{x \in \{\mathbf{a}, \mathbf{b}\}^* \mid x \text{ ends with } \mathbf{a} \vee x \text{ contains } \mathbf{ab}\}$ from Example 4.10.

- (a) *Circuit diagram (no SOS/EOS)*. This NDEFA is obtained by reversing the arrows in Figure 4.10 and swapping the start and final states. Using activity bits q_0, q_1, q_2, q_3 , the next-state equations are

$$\begin{aligned} q'_0 &= (q_1 \wedge \mathbf{a}) \vee (q_2 \wedge \mathbf{b}), \\ q'_1 &= (q_1 \wedge \mathbf{a}) \vee (q_1 \wedge \mathbf{b}), & q'_2 &= (q_3 \wedge \mathbf{a}) \vee (q_2 \wedge \mathbf{b}), \\ q'_3 &= (q_3 \wedge \mathbf{a}) \vee (q_3 \wedge \mathbf{b}). \end{aligned}$$

The initial configuration has $q_1 = q_3 = 1$ and $q_0 = q_2 = 0$, and the accept line is q_0 .

- (b) *DFA (connected portion only)*. From Figure 4.11, the NDEFA has start states $\{q_1, q_3\}$ and only one final state, q_0 . Its nonempty transitions are

$$\begin{aligned} \delta(q_1, \mathbf{a}) &= \{q_1, q_0\}, & \delta(q_1, \mathbf{b}) &= \{q_1\}, \\ \delta(q_2, \mathbf{b}) &= \{q_2, q_0\}, & \delta(q_3, \mathbf{a}) &= \{q_3, q_2\}, & \delta(q_3, \mathbf{b}) &= \{q_3\}. \end{aligned}$$

Starting from $Q_0 = \{q_1, q_3\}$, Definition 4.5 gives

$$\delta^d(Q_0, \mathbf{a}) = \{q_0, q_1, q_2, q_3\} = Q_1, \quad \delta^d(Q_0, \mathbf{b}) = \{q_1, q_3\} = Q_0.$$

From Q_1 we get

$$\delta^d(Q_1, \mathbf{a}) = Q_1, \quad \delta^d(Q_1, \mathbf{b}) = Q_1.$$

Therefore the connected DFA has just two states:

$$Q_0 = \{q_1, q_3\} \quad (\text{nonfinal}), \quad Q_1 = \{q_0, q_1, q_2, q_3\} \quad (\text{final}),$$

with transition table

	a	b
Q_0	Q_1	Q_0
Q_1	Q_1	Q_1

4.20. **Are the pumping lemma arguments still valid for NDFAs?**

Not as *machine* arguments about an NDFA. In Chapter 2, the pumping lemma was proved for DFAs by following the *single* path determined by a long input and finding a repeated state on that path. An NDFA does not behave that way, because one word may have many possible paths.

However, the pumping lemma conclusion *does* still hold for every language accepted by an NDFA. If A is an NDFA, then by Theorem 4.1 there is an equivalent DFA A^d with $L(A) = L(A^d)$. Since $L(A^d)$ is FAD, the Chapter 2 pumping lemma applies to that language. Therefore any language accepted by an NDFA satisfies the pumping lemma, but the proof is obtained by passing to the equivalent DFA, not by reusing the Chapter 2 path argument directly on the NDFA.

4.21. **Does the conclusion of Theorem 2.7 still hold for NDFAs?**

Yes, but again this is a statement about the *language*, not about the internal states of an arbitrary NDFA. Theorem 2.7 says that a language L is FAD iff $\equiv_L$ has finitely many equivalence classes. By Theorem 4.1, a language is accepted by some NDFA iff it is accepted by some DFA. Therefore the conclusion of Theorem 2.7 still holds for languages accepted by NDFAs:

$$L \text{ is accepted by an NDFA} \Leftrightarrow L \text{ is FAD} \Leftrightarrow \equiv_L \text{ has finitely many classes.}$$

What does *not* follow is that the states of a given NDFA correspond to the equivalence classes of $\equiv_L$; that correspondence belongs to the minimal DFA, not to an arbitrary nondeterministic machine.

4.22. **Later no- λ exercise: given an NDFA A without λ -transitions with $\lambda \notin L(A)$, construct A'' with exactly one start state, one final state, and $L(A'') = L(A)$.**

Construction. Let $A = \langle \Sigma, S, S_0, \delta, F \rangle$ with $\lambda \notin L(A)$, so $S_0 \cap F = \emptyset$. Define

$$A'' = \langle \Sigma, S \cup \{q_0, q_f\}, \{q_0\}, \delta'', \{q_f\} \rangle$$

where q_0 and q_f are new states and:

1. For each $\mathbf{a} \in \Sigma$, let

$$\delta''(q_0, \mathbf{a}) = \left(\bigcup_{s \in S_0} \delta(s, \mathbf{a}) \right) \cup \begin{cases} \{q_f\} & \text{if } (\bigcup_{s \in S_0} \delta(s, \mathbf{a})) \cap F \neq \emptyset, \\ \emptyset & \text{otherwise.} \end{cases}$$

2. For each old state $s \in S$ and each $\mathbf{a} \in \Sigma$, let

$$\delta''(s, \mathbf{a}) = \delta(s, \mathbf{a}) \cup \begin{cases} \{q_f\} & \text{if } \delta(s, \mathbf{a}) \cap F \neq \emptyset, \\ \emptyset & \text{otherwise.} \end{cases}$$

3. $\delta''(q_f, \mathbf{a}) = \emptyset$ for every $\mathbf{a} \in \Sigma$.

This machine has exactly one start state (q_0) and exactly one final state (q_f).

Why this preserves the language. Since $\lambda \notin L(A)$, every accepted word has positive length. Suppose $w = \mathbf{a}_1 \cdots \mathbf{a}_n \in L(A)$. Then there is a path in A from some start state to some final state that reads all of w . In A'' , use the same path for the first $n - 1$ letters. On the last letter, when the old path would move into a final state, choose the extra target q_f instead. Thus $w \in L(A'')$.

Conversely, any accepting path in A'' must end in q_f . The only way to enter q_f is on some actual input symbol $\mathbf{a}$ from q_0 or from an old state s , and this happens only when the corresponding old transition set contains at least one state of F . Replacing that last step by a move into one of those original final states yields an accepting path in A . Hence $L(A'') = L(A)$.

4.23. If $\lambda \in L(A)$, the three conditions of Exercise 4.22 may be incompatible (without λ -transitions).

Example. Let

$$A = \langle \{\mathbf{a}\}, \{s_0, s_1\}, \{s_0\}, \delta, \{s_0, s_1\} \rangle$$

where $\delta(s_0, \mathbf{a}) = \{s_1\}$ and $\delta(s_1, \mathbf{a}) = \emptyset$. Then

$$L(A) = \{\lambda, \mathbf{a}\}.$$

We claim there is no NDFA without λ -transitions that has exactly one start state, exactly one final state, and also accepts exactly $\{\lambda, \mathbf{a}\}$.

Assume to the contrary that

$$B = \langle \{\mathbf{a}\}, S, \{q\}, \delta_B, \{f\} \rangle$$

has those three properties and $L(B) = \{\lambda, \mathbf{a}\}$. Since $\lambda \in L(B)$, the unique start state must also be the unique final state:

$$q = f.$$

Since $\mathbf{a} \in L(B)$ and there are no λ -transitions, there must be a transition on $\mathbf{a}$ from q to the only final state, namely to q itself. Thus $q \in \delta_B(q, \mathbf{a})$.

But then the same move can be used repeatedly, so every positive power of $\mathbf{a}$ is accepted:

$$\mathbf{a}^n \in L(B) \quad \text{for all } n \geq 1.$$

In particular, $\mathbf{a}\mathbf{a} \in L(B)$, contradicting $L(B) = \{\lambda, \mathbf{a}\}$. Therefore the three conditions may indeed be incompatible.

4.24. **Prove Lemma 4.2.**

Lemma 4.2: For any $s \in S$ and any $x \in \Sigma^+$, $\overline{\delta}_\lambda^\sigma(s, x) = \overline{\delta}_\lambda(s, x)$.

Proof. We proceed by induction on $|x| = k \geq 1$.

Base case ($k = 1$). Let $x = \mathbf{a} \in \Sigma$. By definition of δ_λ^σ :

$$\delta_\lambda^\sigma(s, \mathbf{a}) = \Lambda \left(\bigcup_{t \in \Lambda(s)} \delta_\lambda(t, \mathbf{a}) \right) = \overline{\delta}_\lambda(s, \mathbf{a}).$$

So $\overline{\delta}_\lambda^\sigma(s, \mathbf{a}) = \delta_\lambda^\sigma(s, \mathbf{a}) = \overline{\delta}_\lambda(s, \mathbf{a})$.

Inductive step. Assume the result holds for all words of length k . Let $y = x\mathbf{a}$ with $|x| = k$ and $\mathbf{a} \in \Sigma$. Then:

$$\begin{aligned} \overline{\delta}_\lambda^\sigma(s, x\mathbf{a}) &= \bigcup_{t \in \overline{\delta}_\lambda^\sigma(s, x)} \delta_\lambda^\sigma(t, \mathbf{a}) \\ &= \bigcup_{t \in \overline{\delta}_\lambda(s, x)} \delta_\lambda^\sigma(t, \mathbf{a}) \quad (\text{induction hypothesis}) \\ &= \bigcup_{t \in \overline{\delta}_\lambda(s, x)} \overline{\delta}_\lambda(t, \mathbf{a}) \quad (\text{base case}). \end{aligned}$$

Let $D = \bar{\delta}_\lambda(s, x)$. Since every value of $\bar{\delta}_\lambda$ is obtained by applying Λ , the set D is λ -closed. Therefore

$$\begin{aligned} \bigcup_{t \in D} \bar{\delta}_\lambda(t, \mathbf{a}) &= \bigcup_{t \in D} \Lambda \left(\bigcup_{q \in \Lambda(t)} \delta_\lambda(q, \mathbf{a}) \right) \\ &= \Lambda \left(\bigcup_{t \in D} \bigcup_{q \in \Lambda(t)} \delta_\lambda(q, \mathbf{a}) \right) \\ &= \Lambda \left(\bigcup_{q \in D} \delta_\lambda(q, \mathbf{a}) \right) \quad (\text{because } \Lambda(t) \subseteq D \text{ for every } t \in D) \\ &= \bar{\delta}_\lambda(s, x\mathbf{a}). \end{aligned}$$

By induction, the result holds for all $x \in \Sigma^+$. □ □

4.25. **A DFA can be viewed as an NDFA, and $L(A^n) = L(A)$.**

Construction. Given a DFA $A = \langle \Sigma, S, s_0, \delta, F \rangle$ (with $\delta : S \times \Sigma \rightarrow S$ a total function), define the NDFA $A^n = \langle \Sigma, S, \{s_0\}, \delta^n, F \rangle$ where $\delta^n : S \times \Sigma^+ \rightarrow \wp(S)$ is given by $\delta^n(s, \mathbf{a}) = \{\delta(s, \mathbf{a})\}$.

A^n is a valid NDFA: S is finite, $S_0^n = \{s_0\}$ is a nonempty set of start states, and δ^n maps to subsets of S .

Claim: $L(A^n) = L(A)$.

Proof. By induction on $|x|$, we show $\bar{\delta}^n(s_0, x) = \{\bar{\delta}(s_0, x)\}$ (i.e., the nondeterministic extended transition function applied to the singleton start always yields the singleton $\{\bar{\delta}(s_0, x)\}$).

Base: $\bar{\delta}^n(s_0, \lambda) = \{s_0\} = \{\bar{\delta}(s_0, \lambda)\}$.

Step: $\bar{\delta}^n(s_0, x\mathbf{a}) = \bigcup_{t \in \bar{\delta}^n(s_0, x)} \delta^n(t, \mathbf{a}) = \bigcup_{t \in \{\bar{\delta}(s_0, x)\}} \{\delta(t, \mathbf{a})\} = \{\delta(\bar{\delta}(s_0, x), \mathbf{a})\} = \{\bar{\delta}(s_0, x\mathbf{a})\}$.

Hence $x \in L(A^n)$ iff $\bar{\delta}^n(s_0, x) \cap F \neq \emptyset$ iff $\bar{\delta}(s_0, x) \in F$ iff $x \in L(A)$. □ □

4.26. **Show that $\bar{\delta}_\lambda^\sigma(s, \lambda) \neq \bar{\delta}_\lambda(s, \lambda)$ in general.**

By definition:

- $\bar{\delta}_\lambda^\sigma(s, \lambda) = \{s\}$ (definition of the extended transition function for zero-length input).
- $\bar{\delta}_\lambda(s, \lambda) = \Lambda(s)$ (definition of $\bar{\delta}_\lambda$ for λ).

These differ whenever $\Lambda(s) \neq \{s\}$, i.e., whenever there exists at least one λ -transition out of s .

Example. In Example 4.14, $\Lambda(s_0) = \{s_0, s_1, s_2\}$ but $\bar{\delta}_\lambda^\sigma(s_0, \lambda) = \{s_0\}$. Hence $\bar{\delta}_\lambda^\sigma(s_0, \lambda) \neq \bar{\delta}_\lambda(s_0, \lambda)$.

This is why Lemma 4.2 is stated only for $x \in \Sigma^+$ (words of length ≥ 1) and cannot be extended to λ .

4.27. **Prove:** $(\forall A, B \in \wp(S))(\forall \mathbf{a} \in \Sigma) \delta^d(A \cup B, \mathbf{a}) = \delta^d(A, \mathbf{a}) \cup \delta^d(B, \mathbf{a})$.

Proof. By Definition 4.5, $\delta^d(Q, \mathbf{a}) = \bigcup_{q \in Q} \delta(q, \mathbf{a})$ for any $Q \in \wp(S)$. Therefore:

$$\begin{aligned} \delta^d(A \cup B, \mathbf{a}) &= \bigcup_{q \in A \cup B} \delta(q, \mathbf{a}) \\ &= \left(\bigcup_{q \in A} \delta(q, \mathbf{a}) \right) \cup \left(\bigcup_{q \in B} \delta(q, \mathbf{a}) \right) \\ &= \delta^d(A, \mathbf{a}) \cup \delta^d(B, \mathbf{a}). \quad \square \end{aligned}$$

□

4.28. **Acceptance requiring *every* reachable state to be final.**

(a) **New acceptance definition.** Define

$$L_{\forall}(A) = \left\{ w \in \Sigma^* \mid \bigcup_{q \in S_0} \bar{\delta}(q, w) \neq \emptyset \wedge \bigcup_{q \in S_0} \bar{\delta}(q, w) \subseteq F \right\}.$$

That is, w is accepted iff (i) at least one state is reachable and (ii) *every* reachable state is final.

(b) **Modified A^d .** Define

$$A^{\forall} = \langle \Sigma, \wp(S), S_0, \delta^d, F^{\forall} \rangle,$$

where

$$\delta^d(Q, \mathbf{a}) = \bigcup_{q \in Q} \delta(q, \mathbf{a})$$

for each $Q \subseteq S$ and $\mathbf{a} \in \Sigma$, and where the final-state set is

$$F^{\forall} = \{Q \in \wp(S) \mid Q \neq \emptyset \wedge Q \subseteq F\}.$$

That is, Q is a final state of $A^{\forall}$ iff Q is nonempty and all states in Q are final states of A .

For any word w , the subset construction yields

$$\bar{\delta}^d(S_0, w) = \bigcup_{q \in S_0} \bar{\delta}(q, w).$$

Therefore

$$\begin{aligned} w \in L(A^{\forall}) &\Leftrightarrow \bar{\delta}^d(S_0, w) \in F^{\forall} \\ &\Leftrightarrow \bigcup_{q \in S_0} \bar{\delta}(q, w) \neq \emptyset \wedge \bigcup_{q \in S_0} \bar{\delta}(q, w) \subseteq F \Leftrightarrow w \in L_{\forall}(A). \end{aligned}$$

Hence $L_{\forall}(A) = L(A^{\forall})$.

Note: this “universal” acceptance is still FAD (the modified $A^{\forall}$ is a DFA), so $L_{\forall}(A)$ is always a FAD language.

4.29. **Draw the connected part of T^d , the deterministic equivalent of the NDEFA T in Example 4.7.**

Let

$$Q_0 = \{s_0\}, \quad Q_1 = \{s_0, s_1\}, \quad Q_2 = \{s_0, s_2\}, \quad Q_3 = \{s_0, s_1, s_3\},$$

$$Q_4 = \{s_0, s_1, s_4\}, \quad Q_5 = \{s_0, s_2, s_5\}, \quad Q_6 = \{s_0, s_1, s_3, s_6\},$$

$$Q_7 = \{s_0, s_1, s_4, s_6\}, \quad Q_8 = \{s_0, s_2, s_6\}, \quad Q_9 = \{s_0, s_1, s_6\},$$

$$Q_{10} = \{s_0, s_2, s_5, s_6\}, \quad Q_{11} = \{s_0, s_6\}.$$

These are exactly the reachable subsets from $\{s_0\}$ under Definition 4.5. The accepting states are those containing s_6 , namely

$$Q_6, Q_7, Q_8, Q_9, Q_{10}, Q_{11}.$$

Their transitions are:

	0	1
Q_0	Q_1	Q_0
Q_1	Q_1	Q_2
Q_2	Q_3	Q_0
Q_3	Q_4	Q_2
Q_4	Q_1	Q_5
Q_5	Q_6	Q_0
Q_6	Q_7	Q_8
Q_7	Q_9	Q_{10}
Q_8	Q_6	Q_{11}
Q_9	Q_9	Q_8
Q_{10}	Q_6	Q_{11}
Q_{11}	Q_9	Q_{11}

This is the connected part of T^d .

4.30. **Modify T to revert to a nonfinal state when 000111 (EOT) is detected.**

Keep the original start-of-transmission arm for **010010**, but replace the single final sink s_6 by six final “recording” states $r_0, \dots, r_5$ that track the longest suffix of the data seen so far that is also a prefix of the EOT string **000111**. The only nonfinal states are the original $s_0, \dots, s_5$.

	0	1
s_0	$\{s_0, s_1\}$	$\{s_0\}$
s_1	$\emptyset$	$\{s_2\}$
s_2	$\{s_3\}$	$\emptyset$
s_3	$\{s_4\}$	$\emptyset$
s_4	$\emptyset$	$\{s_5\}$
s_5	$\{r_0\}$	$\emptyset$
r_0	$\{r_1\}$	$\{r_0\}$
r_1	$\{r_2\}$	$\{r_0\}$
r_2	$\{r_3\}$	$\{r_0\}$
r_3	$\{r_3\}$	$\{r_4\}$
r_4	$\{r_1\}$	$\{r_5\}$
r_5	$\{r_1\}$	$\{s_0\}$

Here r_i means that recording is on and the last i scanned bits match the first i bits of **000111**. Thus $r_5 \xrightarrow{1} s_0$ turns the recorder off exactly when the full EOT signal **000111** has just been seen, while every other input keeps the machine in one of the final recording states.

4.31. Consider the NDFA A from Example 4.14.

(a) **Diagram of A_λ^σ .** For Example 4.14, $\lambda \in L(A_\lambda)$, so Definition 4.9 yields

$$A_\lambda^\sigma = \langle \{\mathbf{a}, \mathbf{b}, \mathbf{c}, \mathbf{d}\}, \{q_0, s_0, s_1, s_2, s_3\}, \{q_0, s_0\}, \delta_\lambda^\sigma, \{q_0, s_2\} \rangle$$

with transition table

	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>
q_0	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$
s_0	$\{s_1, s_2\}$	$\{s_3\}$	$\{s_2\}$	$\{s_2\}$
s_1	$\emptyset$	$\{s_3\}$	$\{s_2\}$	$\{s_2\}$
s_2	$\emptyset$	$\emptyset$	$\emptyset$	$\{s_2\}$
s_3	$\emptyset$	$\{s_1, s_2\}$	$\emptyset$	$\emptyset$

This table is exactly the state-transition diagram of Figure 4.20.

- (b) **Diagram of $A_\lambda^{\sigma d}$ (connected portion).** Starting from the actual start set $\{q_0, s_0\}$, the reachable subsets are

$$R_0 = \{q_0, s_0\}, \quad R_1 = \{s_1, s_2\}, \quad R_2 = \{s_3\}, \quad R_3 = \{s_2\}, \quad R_4 = \emptyset.$$

Their transitions are

	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>
R_0	R_1	R_2	R_3	R_3
R_1	R_4	R_2	R_3	R_3
R_2	R_4	R_1	R_4	R_4
R_3	R_4	R_4	R_4	R_3
R_4	R_4	R_4	R_4	R_4

The final states are R_0 , R_1 , and R_3 .

4.32. What is wrong with the given “proof” of Lemma 4.2?

The flaw is in the inductive step. The “proof” starts the induction at $k = 1$ (base case for single letters), but the inductive step uses $\overline{\delta}_\lambda^\sigma(s, x)$ for $x \in \Sigma^k$ and then applies the “induction hypothesis” $\overline{\delta}_\lambda^\sigma(s, x) = \overline{\delta}_\lambda(s, x)$.

The critical error is on the line:

$$\delta_\lambda^\sigma(\overline{\delta}_\lambda^\sigma(s, x), \mathbf{a}) = \overline{\delta}_\lambda^\sigma(\overline{\delta}_\lambda(s, x), \mathbf{a}).$$

Here, $\overline{\delta}_\lambda^\sigma(s, x)$ is a *set* of states (since we’re in an NDEFA), but δ_λ^σ is defined on individual states, not sets. The expression $\delta_\lambda^\sigma(Q, \mathbf{a})$ where Q is a set is not directly the same as $\overline{\delta}_\lambda^\sigma(Q, \mathbf{a})$ for a set-valued argument: the extended function $\overline{\delta}_\lambda^\sigma$ on a set-argument must take the union over all states in the set.

More specifically, the step

$$\delta_\lambda^\sigma(\overline{\delta}_\lambda^\sigma(s, x), \mathbf{a}) = \overline{\delta}_\lambda^\sigma(\overline{\delta}_\lambda(s, x), \mathbf{a})$$

conflates applying δ_λ^σ to a *set* with applying the set-union version of $\overline{\delta}_\lambda^\sigma$ to a set argument. These are not the same: the left side applies δ_λ^σ to the entire set at once (which is not how the function is defined), while the right side correctly takes the union over individual states. This makes the step invalid without justification. The same typing problem reappears later if one writes an expression such as

$$\overline{\delta}_\lambda(\overline{\delta}_\lambda(s, x), \mathbf{a}),$$

because the first argument of $\overline{\delta}_\lambda$ must be a single state, not a set of states. The correct way to proceed is to write a union over the states in that set.

4.33. Consider the NDFA T from Example 4.7 (detecting substring **010010**).

- (a) *Circuit diagram (no SOS/EOS)*. T has 8 states (one start state that loops on all input plus the six states that track the pattern **010010**). Using one activity bit per state, the next-state equations are

$$\begin{aligned} s'_0 &= s_0, & s'_1 &= s_0 \wedge \mathbf{0}, & s'_2 &= s_1 \wedge \mathbf{1}, \\ s'_3 &= s_2 \wedge \mathbf{0}, & s'_4 &= s_3 \wedge \mathbf{0}, & s'_5 &= s_4 \wedge \mathbf{1}, \\ s'_6 &= (s_5 \wedge \mathbf{0}) \vee (s_6 \wedge \mathbf{0}) \vee (s_6 \wedge \mathbf{1}). \end{aligned}$$

Initially only s_0 is active, and the accept line is s_6 .

- (b) *DFA T^d (connected portion)*. See Exercise 4.29 for the subset construction. The connected DFA is exactly the 12-state machine listed there, with states $Q_0, \dots, Q_{11}$ and the transition table given in Exercise 4.29.

4.34. Consider the NDFA from Example 4.11, which accepts all words containing **10110**, **1010**, or **01101** as substrings.

- (a) *Circuit diagram (no SOS/EOS)*. The NDFA has a single initial state s_0 that loops on Σ^* , three pattern-matching arms, and the final sink s_{18} from Figure 4.12. Since s_1, s_7 , and s_{12} are only shown for clarity in the text, they can be omitted from the actual circuit. With one activity bit per remaining state, the nonzero next-state equations are

$$\begin{aligned} s'_0 &= s_0, \\ s'_2 &= s_0 \wedge \mathbf{1}, & s'_3 &= s_2 \wedge \mathbf{0}, & s'_4 &= s_3 \wedge \mathbf{1}, & s'_5 &= s_4 \wedge \mathbf{1}, & s'_6 &= s_5 \wedge \mathbf{0}, \\ s'_8 &= s_0 \wedge \mathbf{1}, & s'_9 &= s_8 \wedge \mathbf{0}, & s'_{10} &= s_9 \wedge \mathbf{1}, & s'_{11} &= s_{10} \wedge \mathbf{0}, \\ s'_{13} &= s_0 \wedge \mathbf{0}, & s'_{14} &= s_{13} \wedge \mathbf{1}, & s'_{15} &= s_{14} \wedge \mathbf{1}, & s'_{16} &= s_{15} \wedge \mathbf{0}, & s'_{17} &= s_{16} \wedge \mathbf{1}, \\ s'_{18} &= s_6 \vee s_{11} \vee s_{17} \vee s_{18}. \end{aligned}$$

The accept line is $s_6 \vee s_{11} \vee s_{17} \vee s_{18}$.

- (b) *DFA (connected portion only)*. Since all three patterns are started from the looping state s_0 , the subset construction yields the following 23 reachable states:

$Q_0 = \{s_0\},$	$Q_1 = \{s_0, s_{13}\},$	$Q_2 = \{s_0, s_2, s_8\},$
$Q_3 = \{s_0, s_{14}, s_2, s_8\},$	$Q_4 = \{s_0, s_{13}, s_3, s_9\},$	$Q_5 = \{s_0, s_{15}, s_2, s_8\},$
$Q_6 = \{s_0, s_{10}, s_{14}, s_2, s_4, s_8\},$	$Q_7 = \{s_0, s_{13}, s_{16}, s_3, s_9\},$	$Q_8 = \{s_0, s_{11}, s_{13}, s_3, s_9\},$
$Q_9 = \{s_0, s_{15}, s_2, s_5, s_8\},$	$Q_{10} = \{s_0, s_{10}, s_{14}, s_{17}, s_2, s_4, s_8\},$	$Q_{11} = \{s_0, s_{13}, s_{18}\},$
$Q_{12} = \{s_0, s_{10}, s_{14}, s_{18}, s_2, s_4, s_8\},$	$Q_{13} = \{s_0, s_{13}, s_{16}, s_3, s_6, s_9\},$	$Q_{14} = \{s_0, s_{11}, s_{13}, s_{18}, s_3, s_9\},$
$Q_{15} = \{s_0, s_{15}, s_{18}, s_2, s_5, s_8\},$	$Q_{16} = \{s_0, s_{14}, s_{18}, s_2, s_8\},$	$Q_{17} = \{s_0, s_{10}, s_{14}, s_{17}, s_{18}, s_2, s_4, s_8\},$
$Q_{18} = \{s_0, s_{13}, s_{16}, s_{18}, s_3, s_6, s_9\},$	$Q_{19} = \{s_0, s_{18}, s_2, s_8\},$	$Q_{20} = \{s_0, s_{13}, s_{18}, s_3, s_9\},$
$Q_{21} = \{s_0, s_{15}, s_{18}, s_2, s_8\},$	$Q_{22} = \{s_0, s_{13}, s_{16}, s_{18}, s_3, s_9\}.$	

Their transition table is

	0	1
Q_0	Q_1	Q_2
Q_1	Q_1	Q_3
Q_2	Q_4	Q_2
Q_3	Q_4	Q_5
Q_4	Q_1	Q_6
Q_5	Q_7	Q_2
Q_6	Q_8	Q_9
Q_7	Q_1	Q_{10}
Q_8	Q_{11}	Q_{12}
Q_9	Q_{13}	Q_2
Q_{10}	Q_{14}	Q_{15}
Q_{11}	Q_{11}	Q_{16}
Q_{12}	Q_{14}	Q_{15}
Q_{13}	Q_{11}	Q_{17}
Q_{14}	Q_{11}	Q_{12}
Q_{15}	Q_{18}	Q_{19}
Q_{16}	Q_{20}	Q_{21}
Q_{17}	Q_{14}	Q_{15}
Q_{18}	Q_{11}	Q_{17}
Q_{19}	Q_{20}	Q_{19}
Q_{20}	Q_{11}	Q_{12}
Q_{21}	Q_{22}	Q_{19}
Q_{22}	Q_{11}	Q_{17}

The final states are exactly those containing one of s_6 , s_{11} , s_{17} , or s_{18} :

$$Q_8, Q_{10}, Q_{11}, Q_{12}, Q_{13}, Q_{14}, Q_{15}, Q_{16}, Q_{17}, Q_{18}, Q_{19}, Q_{20}, Q_{21}, Q_{22}.$$

4.35. Consider the NDFA B from Example 4.8 and its deterministic equivalent B^d .

- (a) *Circuit diagram for B (no SOS/EOS).* Encode the states of B using one activity bit for r and one for s . From Figure 4.7,

$$r' = (r \wedge \mathbf{a}) \vee (s \wedge \mathbf{a}), \quad s' = (r \wedge \mathbf{a}) \vee (r \wedge \mathbf{b}),$$

with initial state $r = 1$, $s = 0$. Since s is the only final state, the accept output is s .

- (b) *Circuit diagram for B^d (no SOS/EOS).* The states of B^d are

$$\emptyset, \quad \{r\}, \quad \{s\}, \quad \{r, s\}.$$

Using the binary state assignment

$$\emptyset = 00, \quad \{r\} = 01, \quad \{s\} = 10, \quad \{r, s\} = 11,$$

with present-state bits $x_1 x_0$, the transition table is

$x_1 x_0$	$\mathbf{a}$	$\mathbf{b}$
00	00	00
01	11	10
10	01	00
11	11	10

so the DFA circuit is given by

$$x'_1 = x_0, \quad x'_0 = \mathbf{a} \wedge (x_1 \vee x_0).$$

The start state is 01 and the accept output is x_1 , since the final subset states are $\{s\}$ and $\{r, s\}$.

4.36. Circuit diagrams for each NDFA in Figure 4.25 (no SOS/EOS).

For each NDFA A_i in Figure 4.25, with $\Sigma = \{\mathbf{a}, \mathbf{b}, \mathbf{c}\}$:

1. **(a)** With activity bit t for the single state:

$$t' = t \wedge (\mathbf{a} \vee \mathbf{c}), \quad \text{accept} = t.$$

2. **(b)** With activity bits q_1, q_0 :

$$q'_1 = (q_1 \wedge \mathbf{b}) \vee (q_0 \wedge \mathbf{b}), \quad q'_0 = (q_1 \wedge \mathbf{a}) \vee (q_0 \wedge \mathbf{c}),$$

and the accept output is q_1 .

3. **(c)** With activity bits r_0, r_1 :

$$r'_0 = (r_0 \wedge \mathbf{a}) \vee (r_1 \wedge \mathbf{b}), \quad r'_1 = (r_0 \wedge \mathbf{a}) \vee (r_0 \wedge \mathbf{c}),$$

and the accept output is r_0 .

4. **(d)** With activity bits s_0, s_1, s_2, s_3 :

$$s'_0 = 0, \quad s'_1 = (s_0 \wedge \mathbf{a}) \vee (s_3 \wedge \mathbf{b}),$$

$$s'_2 = s_1 \wedge \mathbf{a}, \quad s'_3 = s_1 \wedge \mathbf{b},$$

and the accept output is s_2 .

5. **(e)** With activity bits s_1, s_2 :

$$s'_1 = (s_1 \wedge \mathbf{a}) \vee (s_2 \wedge \mathbf{a}) \vee (s_2 \wedge \mathbf{b}), \quad s'_2 = s_1 \wedge \mathbf{a},$$

and the accept output is s_2 .

6. **(f)** The λ -machine has states s_0, s_1, s_2, s_3 with ordinary transitions $s_0 \xrightarrow{\mathbf{a}} s_1, s_1 \xrightarrow{\mathbf{b}} s_2, s_2 \xrightarrow{\mathbf{c}} s_3$ and λ -moves $s_1 \rightarrow s_0, s_3 \rightarrow s_1$. The standard circuit therefore uses

$$D_0 = s_1, \quad D_1 = (s_0 \wedge \mathbf{a}) \vee s_3, \quad D_2 = s_1 \wedge \mathbf{b}, \quad D_3 = s_2 \wedge \mathbf{c},$$

with accept output s_3 . Adding the transitive λ -connection $s_3 \rightarrow s_0$ as well is also consistent with the text's discussion.

7. (g) With activity bits s_0, s_1, s_2, s_3 :

$$s'_0 = s_1 \wedge \mathbf{a}, \quad s'_1 = (s_0 \wedge \mathbf{a}) \vee (s_2 \wedge \mathbf{c}) \vee (s_3 \wedge \mathbf{b}),$$

$$s'_2 = s_1 \wedge \mathbf{c}, \quad s'_3 = s_1 \wedge \mathbf{b},$$

and the accept output is s_0 .

4.37. **Circuit diagrams for each NDFAs in Figure 4.25 (with SOS and EOS).**

Extend each circuit from Exercise 4.36 by forcing the start state(s) active on <SOS> and gating the final-state output with <EOS>:

- (a) $t' = \text{<SOS>} \vee (t \wedge (\mathbf{a} \vee \mathbf{c}))$ and $\text{accept} = t \wedge \text{<EOS>}$.
- (b) OR <SOS> into the input for q_1 :

$$q'_1 = \text{<SOS>} \vee (q_1 \wedge \mathbf{b}) \vee (q_0 \wedge \mathbf{b}), \quad q'_0 = (q_1 \wedge \mathbf{a}) \vee (q_0 \wedge \mathbf{c}),$$

and $\text{accept} = q_1 \wedge \text{<EOS>}$.

- (c) OR <SOS> into the input for r_0 and gate the output:

$$r'_0 = \text{<SOS>} \vee (r_0 \wedge \mathbf{a}) \vee (r_1 \wedge \mathbf{b}), \quad r'_1 = (r_0 \wedge \mathbf{a}) \vee (r_0 \wedge \mathbf{c}),$$

with $\text{accept} = r_0 \wedge \text{<EOS>}$.

- (d) OR <SOS> into s'_0 and use $\text{accept} = s_2 \wedge \text{<EOS>}$.
- (e) OR <SOS> into the corrected first equation:

$$s'_1 = \text{<SOS>} \vee (s_1 \wedge \mathbf{a}) \vee (s_2 \wedge \mathbf{a}) \vee (s_2 \wedge \mathbf{b}), \quad s'_2 = s_1 \wedge \mathbf{a},$$

and use $\text{accept} = s_2 \wedge \text{<EOS>}$.

- (f) OR <SOS> into D_0 and use $\text{accept} = s_3 \wedge \text{<EOS>}$.
- (g) OR <SOS> into s'_0 and use $\text{accept} = s_0 \wedge \text{<EOS>}$.

4.38. **Adapting Definition 3.10 for NDFAs and proving that an algorithm exists to find the connected portion.**

- (a) **Adapted definition.** The connected portion of an NDFAs $A = \langle \Sigma, S, S_0, \delta, F \rangle$ is defined as follows. Let

$$\text{Reach}_0 = S_0, \quad \text{Reach}_{k+1} = \text{Reach}_k \cup \bigcup_{\mathbf{a} \in \Sigma} \bigcup_{s \in \text{Reach}_k} \delta(s, \mathbf{a}).$$

The connected portion is $\text{Reach} = \bigcup_{k \geq 0} \text{Reach}_k$, together with all transitions among states in Reach and the restriction of F to Reach .

- (b) **Proof that the algorithm terminates.**

Proof. The sets Reach_k form a non-decreasing chain $\text{Reach}_0 \subseteq \text{Reach}_1 \subseteq \dots \subseteq S$. Since S is finite, the chain must stabilize: there exists k^* such that $\text{Reach}_{k^*} = \text{Reach}_{k^*+1}$. At that point, no new states are added. The algorithm terminates in at most $|S|$ iterations (since at least one new state must be added in each non-stable step, and there are only $|S|$ states).

To compute this algorithmically:

1. Initialize $R := S_0$, $\text{New} := S_0$.
2. While $\text{New} \neq \emptyset$:
 - (a) $\text{Next} := \bigcup_{\mathbf{a} \in \Sigma} \bigcup_{s \in \text{New}} \delta(s, \mathbf{a}) \setminus R$.
 - (b) $R := R \cup \text{Next}$; $\text{New} := \text{Next}$.
3. Output R as the connected portion.

Each iteration of the while loop adds at least one state to R (otherwise $\text{New} = \emptyset$ and we stop). Thus the loop executes at most $|S|$ times. The running time depends on how the transition sets are represented: with adjacency lists it is linear in the number of reachable transition triples $(s, \mathbf{a}, t)$ with $t \in \delta(s, \mathbf{a})$, while a dense subset-valued representation gives a bound of $O(|S|^2 \cdot |\Sigma|)$. In either representation the procedure is finite and effective. $\square$ $\square$

Chapter 5

Closure Properties — Solutions

5.1. Let F_Σ be the collection of all finite languages over Σ .

- (a) **Complementation: No.** Σ^* is infinite, so for any finite L , $\Sigma^* \setminus L$ is infinite (assuming $|\Sigma| \geq 1$), hence not in F_Σ .
- (b) **Union: Yes.** If L_1, L_2 are finite, then $|L_1 \cup L_2| \leq |L_1| + |L_2| < \infty$.
- (c) **Intersection: Yes.** $|L_1 \cap L_2| \leq |L_1| < \infty$.
- (d) **Concatenation: Yes.** $|L_1 \cdot L_2| \leq |L_1| \cdot |L_2| < \infty$.
- (e) **Kleene closure: No.** Let $L = \{a\}$. Then $L^* = \{a\}^* = \{\lambda, a, aa, \dots\}$ is infinite.
- (f) **Relative complement: Yes.** $L_1 \setminus L_2 \subseteq L_1$, so $|L_1 \setminus L_2| \leq |L_1| < \infty$.

5.2. Let C_Σ be the collection of all cofinite languages (complements of finite languages).

- (a) **Complementation: No.** Let $L = \Sigma^* \setminus \{a\}$. Then $L \in C_\Sigma$, but $\sim L = \{a\}$ is finite, hence not in C_Σ .
- (b) **Union: Yes.** If $L_1 = \Sigma^* \setminus F_1$ and $L_2 = \Sigma^* \setminus F_2$, then $L_1 \cup L_2 = \Sigma^* \setminus (F_1 \cap F_2)$. Since $F_1 \cap F_2$ is finite, $L_1 \cup L_2 \in C_\Sigma$.
- (c) **Intersection: Yes.** $L_1 \cap L_2 = \Sigma^* \setminus (F_1 \cup F_2)$. Since F_1 and F_2 are finite, so is $F_1 \cup F_2$, hence $L_1 \cap L_2 \in C_\Sigma$.
- (d) **Concatenation: Yes.** If $L_1, L_2 \in C_\Sigma$, then there exists N_1 such that all words of length $\geq N_1$ are in L_1 , and there exists N_2 such that all words of length $\geq N_2$ are in L_2 . Then any word of length $\geq N_1 + N_2$ can be split so that both parts are in L_1 and L_2 , so $L_1 \cdot L_2$ is cofinite.
- (e) **Kleene closure: Yes.** Since L is cofinite, there is an N such that every word of length at least N lies in L . Hence every word of length at least N also lies in L^* (just take it as a one-factor product). Therefore $\Sigma^* \setminus L^*$ contains only words of length $< N$, so it is finite and $L^* \in C_\Sigma$.
- (f) **Relative complement: No.** $L_1 \setminus L_2 = L_1 \cap \sim L_2$. Here $\sim L_2$ is finite, so $L_1 \setminus L_2 \subseteq \sim L_2$ is finite, not cofinite in general. Counterexample: $L_1 = \Sigma^* \setminus \emptyset = \Sigma^*$ and $L_2 = \Sigma^* \setminus \{a\}$. Then $L_1 \setminus L_2 = \{a\}$, which is finite, not cofinite.

5.3. $B_\Sigma = F_\Sigma \cup C_\Sigma$.

- (a) **Complementation: Yes.** The complement of a finite language is cofinite and vice versa.

- (b) **Union: Yes.** If both L_1, L_2 are finite, $L_1 \cup L_2$ is finite. If one is cofinite, $L_1 \cup L_2$ is cofinite. If both are cofinite, $L_1 \cup L_2$ is cofinite (by 5.2b). **Closed.**
- (c) **Intersection: Yes.** If both finite: finite. If one cofinite and one finite: finite. If both cofinite: cofinite (by 5.2c).
- (d) **Concatenation: It depends on Σ .** If $|\Sigma| = 1$, say $\Sigma = \{a\}$, then B_Σ is closed under concatenation. Finite unary languages correspond to finite sets of lengths, and cofinite unary languages correspond to cofinite sets of lengths. The sum of two finite sets of lengths is finite; the sum of a nonempty finite set and a cofinite set is cofinite; and the sum of two cofinite sets is cofinite. So every concatenation stays in B_Σ .
If $|\Sigma| \geq 2$, then B_Σ is *not* closed under concatenation. For example, let $L_1 = \{a\}$ and $L_2 = \Sigma^* \setminus \{aa\}$. Then $L_1 \in F_\Sigma$ and $L_2 \in C_\Sigma$, but

$$L_1 \cdot L_2 = \{aw \mid w \neq aa\},$$

the set of all words beginning with a except aaa . This language is infinite, and its complement is also infinite, so it is neither finite nor cofinite.

Therefore the correct answer is:

$$|\Sigma| = 1 \Rightarrow \text{closed}, \quad |\Sigma| \geq 2 \Rightarrow \text{not closed}.$$

- (e) **Kleene closure: No.** Pick any symbol $a \in \Sigma$ and let $L = \{aa\}$. Then $L \in F_\Sigma \subseteq B_\Sigma$, but

$$L^* = \{(aa)^n \mid n \geq 0\}.$$

This language is infinite. If $|\Sigma| = 1$, its complement is the set of odd-length words, which is infinite. If $|\Sigma| \geq 2$, its complement contains every word with some symbol other than a , so it is again infinite. Thus L^* is neither finite nor cofinite, so B_Σ is not closed under Kleene closure.

- (f) **Relative complement: Yes.** If $L_1 \in C_\Sigma$ and $L_2 \in C_\Sigma$, then $L_1 \setminus L_2 = L_1 \cap \sim L_2$ where $\sim L_2$ is finite, so the result is finite and therefore belongs to $F_\Sigma \subseteq B_\Sigma$. If $L_1 \in F_\Sigma$, then $L_1 \setminus L_2 \subseteq L_1$ is finite. Hence B_Σ is closed under relative complement.

5.4. $I_\Sigma = \varnothing(\Sigma^*) \setminus F_\Sigma$, the infinite languages.

- (a) **Complementation: No.** $\Sigma^* \setminus L$ may be finite. E.g., $L = \Sigma^* \setminus \{\lambda\}$ is infinite; $\sim L = \{\lambda\}$ is finite.
- (b) **Union: Yes.** Union of two infinite sets is infinite.
- (c) **Intersection: No.** $L_1 = \{a^{2n} \mid n \geq 0\}$ and $L_2 = \{a^{2n+1} \mid n \geq 0\}$ are both infinite, but $L_1 \cap L_2 = \emptyset$, which is finite.
- (d) **Concatenation: Yes.** If L_1, L_2 are infinite, pick any fixed word $y \in L_2$. Then

$$\{xy \mid x \in L_1\} \subseteq L_1 \cdot L_2.$$

The map $x \mapsto xy$ is injective, so $\{xy \mid x \in L_1\}$ is infinite. Therefore $L_1 \cdot L_2$ is infinite.

- (e) **Kleene closure: Yes.** $L^* \supseteq L$ when $\lambda \in L^*$, and L^* is always infinite if L contains any nonempty word.
- (f) **Relative complement: No.** $L_1 \setminus L_2$ could be finite even if L_1, L_2 are infinite (e.g., $L_1 = L_2$).

5.5. $J_\Sigma = \wp(\Sigma^*) \setminus C_\Sigma$, languages with infinite complements.

- (a) **Complementation: No.** $L \in J_\Sigma$ means $\sim L$ is infinite. For $\sim L$ itself to belong to J_Σ , its complement $\sim(\sim L) = L$ would also have to be infinite; this need not hold. Counterexample: $L = \emptyset \in J_\Sigma$ (complement is Σ^* , infinite), but $\sim L = \Sigma^* \notin J_\Sigma$ (complement is $\emptyset$, finite). **Not closed.**
- (b) **Union: No.** Over $\Sigma = \{a\}$, let $L_1 = \{a^{2n} \mid n \geq 0\}$ and $L_2 = \{a^{2n+1} \mid n \geq 0\}$. Both have infinite complements, so both belong to J_Σ . But $L_1 \cup L_2 = \Sigma^*$, whose complement is finite, so $L_1 \cup L_2 \notin J_\Sigma$.
- (c) **Intersection: Yes.** If $\sim L_1$ and $\sim L_2$ are both infinite, then $\sim(L_1 \cap L_2) = \sim L_1 \cup \sim L_2 \supseteq \sim L_1$, which is infinite.
- (d) **Concatenation: No.** Over $\Sigma = \{a\}$, let $L_1 = \{a^n \mid n \text{ odd or } n = 2\}$ and $L_2 = \{a^{2k} \mid k \geq 1\}$. Then $\sim L_1 = \{\lambda, a^4, a^6, \dots\}$ and $\sim L_2 = \{\lambda, a, a^3, a^5, \dots\}$ are both infinite, so $L_1, L_2 \in J_\Sigma$. But $L_1 \cdot L_2 = \{a^n \mid n \geq 3\}$ (any $n \geq 3$ is expressible as odd+even or 2+even), whose complement $\{\lambda, a, a^2\}$ is finite—so $L_1 \cdot L_2 \notin J_\Sigma$.
- (e) **Kleene closure: No.** Over $\Sigma = \{a\}$, let $L = \{a^n \mid n \text{ odd}\}$. Then $\sim L = \{\lambda, a^2, a^4, \dots\}$ is infinite, so $L \in J_\Sigma$. But odd+odd=even, even+odd=odd, so L^k contains all a -strings of the right parity at length $\geq k$; inductively $L^* = \{a\}^* = \Sigma^*$. Then $\sim L^* = \emptyset$ is finite, so $L^* \notin J_\Sigma$.
- (f) **Relative complement: Yes.** $\sim(L_1 \setminus L_2) = \sim(L_1 \cap \sim L_2) = \sim L_1 \cup L_2 \supseteq \sim L_1$. Since $L_1 \in J_\Sigma$, $\sim L_1$ is infinite, so $\sim(L_1 \setminus L_2)$ is infinite, hence $L_1 \setminus L_2 \in J_\Sigma$. $\square$

5.6. E : all languages over $\{a, b\}$ that contain **abba**.

- (a) **Complementation: No.** $\sim L$ contains all words *not* in L ; since L contains **abba**, $\sim L$ does not contain **abba**, so $\sim L \notin E$.
- (b) **Union: Yes.** If **abba** $\in L_1$, then **abba** $\in L_1 \cup L_2$.
- (c) **Intersection: Yes.** If $L_1, L_2 \in E$ then both contain **abba**, so $L_1 \cap L_2$ also contains **abba**. **Closed.**
- (d) **Concatenation: No.** Here E means languages that have the specific word **abba** as an element. Let $L_1 = L_2 = \{\mathbf{abba}\}$. Then $L_1, L_2 \in E$, but $L_1 \cdot L_2 = \{\mathbf{abbaabba}\}$, which does not contain **abba** as a member. So E is not closed under concatenation.
- (e) **Kleene closure: Yes.** L^* contains $L^1 = L$, so **abba** $\in L \subseteq L^*$.
- (f) **Relative complement: No.** $L_1 \setminus L_2$ might not contain **abba** if **abba** $\in L_2$. E.g., $L_1 = L_2 = \{\mathbf{abba}\}$: $L_1 \setminus L_2 = \emptyset \notin E$.

5.7. **Intersection closed does not imply union closed.**

Counterexample. Let $\mathcal{C} = \{\emptyset\} \cup \{\{x\} \mid x \in \Sigma^*\}$ (the empty language together with all singleton languages).

Closed under intersection: $\{x\} \cap \{y\} = \{x\}$ if $x = y$, and $\emptyset$ if $x \neq y$; both are in $\mathcal{C}$.

Not closed under union: $\{x\} \cup \{y\} = \{x, y\}$ for $x \neq y$, which is a two-element set and not in $\mathcal{C}$. $\square$

5.8. **If closed under intersection and complement, then closed under union (by De Morgan).**

If $\mathcal{C}$ is closed under intersection and complement, then for any $L_1, L_2 \in \mathcal{C}$: $\sim L_1, \sim L_2 \in \mathcal{C}$ (by closure under complement), so $\sim L_1 \cap \sim L_2 \in \mathcal{C}$ (by closure under intersection), so $\sim(\sim L_1 \cap \sim L_2) = L_1 \cup L_2 \in \mathcal{C}$.

$L_2) = L_1 \cup L_2 \in \mathcal{C}$ (by De Morgan and closure under complement). Hence $\mathcal{C}$ is also closed under union. $\square$

5.9. Closed under concatenation does not imply closed under Kleene closure.

Let $\mathcal{C} = F_\Sigma$ (all finite languages). Closed under concatenation (5.1d). Not closed under Kleene closure: $\{a\}^*$ is infinite (5.1e).

5.10. Closed under Kleene closure does not imply closed under concatenation.

Let

$$\mathcal{C} = \{\{w\}^* \mid w \in \Sigma^*\}.$$

This includes $\{\lambda\} = \{\lambda\}^*$ (take $w = \lambda$). Each language in $\mathcal{C}$ is closed under Kleene closure, because

$$(\{w\}^*)^* = \{w\}^*.$$

But $\mathcal{C}$ is not closed under concatenation: over $\Sigma = \{a, b\}$,

$$\{a\}^* \cdot \{b\}^* = \{a^m b^n \mid m, n \geq 0\},$$

and this language is not of the form $\{u\}^*$ for any word u .

5.11. Closed under complement does not imply closed under relative complement.

Counterexample. Let

$$K = \{\emptyset, \Sigma^*\} \cup \{\{w\} \mid w \in \Sigma^*\} \cup \{\Sigma^* \setminus \{w\} \mid w \in \Sigma^*\}.$$

This collection is closed under complement by construction. But it is not closed under relative complement: if $u \neq v$, then

$$(\Sigma^* \setminus \{u\}) \setminus \{v\} = \Sigma^* \setminus \{u, v\} \notin K.$$

So closure under complement does not imply closure under relative complement.

5.12. A finite set of numbers closed under $\sqrt{\cdot}$:

$\{0, 1\}$. $\sqrt{0} = 0 \in \{0, 1\}$ and $\sqrt{1} = 1 \in \{0, 1\}$.

5.13. An infinite set of numbers closed under $\sqrt{\cdot}$:

$[0, 1] = \{x \in \mathbb{R} \mid 0 \leq x \leq 1\}$. For any $x \in [0, 1]$, $\sqrt{x} \in [0, 1]$ since $0 \leq x \leq 1 \Rightarrow 0 \leq \sqrt{x} \leq 1$.

5.14. Algorithm for deterministic union machine A^U .

Given $A_1 = \langle \Sigma, S_1, s_0^1, \delta_1, F_1 \rangle$ and $A_2 = \langle \Sigma, S_2, s_0^2, \delta_2, F_2 \rangle$ (both DFAs, $S_1 \cap S_2 = \emptyset$):

1. Build the NDFA $A^U = \langle \Sigma, S_1 \cup S_2, \{s_0^1, s_0^2\}, \delta^U, F_1 \cup F_2 \rangle$ (as in Theorem 5.2). 2. Apply Definition 4.5 (subset construction) to A^U to produce a DFA A^U . Start state: $\{s_0^1, s_0^2\}$. For each reachable state-set $Q \subseteq S_1 \cup S_2$ and each $a \in \Sigma$: $\delta^U(Q, a) = \delta^d(Q, a) = \bigcup_{q \in Q} \delta^U(q, a)$. Final states: all Q with $Q \cap (F_1 \cup F_2) \neq \emptyset$.

5.15. (a) Direct product machine for union. Define $A^u = \langle \Sigma, S_1 \times S_2, \langle s_0^1, s_0^2 \rangle, \delta^u, F^u \rangle$ where $\delta^u(\langle s, t \rangle, a) = \langle \delta_1(s, a), \delta_2(t, a) \rangle$ and $F^u = \{\langle s, t \rangle \mid s \in F_1 \vee t \in F_2\}$.

- (b) **Proof.** By induction, $\overline{\delta^u}(\langle s_0^1, s_0^2 \rangle, x) = \langle \overline{\delta^1}(s_0^1, x), \overline{\delta^2}(s_0^2, x) \rangle$. Then $x \in L(A^u)$ iff $\overline{\delta^u}(\langle s_0^1, s_0^2 \rangle, x) \in F^u$ iff $\overline{\delta^1}(s_0^1, x) \in F_1$ or $\overline{\delta^2}(s_0^2, x) \in F_2$ iff $x \in L(A_1) \cup L(A_2)$.
- (c) **State counts (no minimization).** $A^\cup$ (NFA union): $|S_1| + |S_2|$ states (plus the new start state if added separately, but in Theorem 5.2 no new state is needed). A^U (subset construction of $A^\cup$): up to $2^{|S_1|+|S_2|}$ states, but in practice far fewer. A^u (product): $|S_1| \cdot |S_2| = nm$ states. With $n = |S_1|$ and $m = |S_2|$: $A^\cup$ has $n + m$ states; A^U has up to 2^{n+m} states; A^u has nm states.

5.16. $\mathcal{D}_\Sigma$ is closed under the prefix operator P .

Proof. Let $L \in \mathcal{D}_\Sigma$ and $A = \langle \Sigma, S, s_0, \delta, F \rangle$ be a DFA with $L(A) = L$. Define $A' = \langle \Sigma, S, s_0, \delta, F' \rangle$ where $F' = \{s \in S \mid \exists y \in \Sigma^* \ni \overline{\delta}(s, y) \in F\}$. (A state is final in A' iff some continuation from it leads to acceptance in A .)

Claim: $L(A') = P(L)$.

$x \in P(L)$ iff $\exists y \in \Sigma^* \ni xy \in L$ iff $\exists y \in \Sigma^* \ni \overline{\delta}(s_0, xy) \in F$ iff $\exists y \in \Sigma^* \ni \overline{\delta}(\overline{\delta}(s_0, x), y) \in F$ iff $\overline{\delta}(s_0, x) \in F'$ iff $x \in L(A')$.

Since F' is computable (for each state s , check if any state in F is reachable from s —a finite graph problem), A' is a valid DFA and $P(L) = L(A') \in \mathcal{D}_\Sigma$. □

5.17. $\mathcal{D}_\Sigma$ is closed under the suffix operator S .

- (a) **Construction.** Given DFA $A = \langle \Sigma, S, s_0, \delta, F \rangle$, build NFA $A' = \langle \Sigma, S, S'_0, \delta, F \rangle$ where $S'_0 = \{\overline{\delta}(s_0, y) \mid y \in \Sigma^*\}$, the set of states reachable from s_0 .
- (b) **Proof.** $x \in S(L)$ iff $\exists y \in \Sigma^* \ni yx \in L$ iff $\exists y \in \Sigma^* \ni \overline{\delta}(s_0, yx) \in F$ iff $\exists s = \overline{\delta}(s_0, y) \in S'_0 \ni \overline{\delta}(s, x) \in F$ iff $\overline{\delta'}(\{s\}, x) \cap F \neq \emptyset$ for some $s \in S'_0$ iff $x \in L(A')$.
- (c) Since A' is an NFA over a finite state set, $L(A') \in \mathcal{D}_\Sigma$ by Theorem 4.1. Hence $S(L) \in \mathcal{D}_\Sigma$. □

5.18. $\mathcal{D}_\Sigma$ is closed under the center operator C .

- (a) **Construction.** Let $R = \{\overline{\delta}(s_0, y) \mid y \in \Sigma^*\}$ be the set of reachable states. Build an NFA $A' = \langle \Sigma, S, R, \delta, F' \rangle$ where $F' = \{s \in S \mid \exists y \in \Sigma^* \ni \overline{\delta}(s, y) \in F\}$ (states from which F is reachable).
- (b) **Proof.** $x \in C(L)$ iff $\exists y, z \ni yxz \in L$ iff $\exists y, z \ni \overline{\delta}(\overline{\delta}(\overline{\delta}(s_0, y), x), z) \in F$ iff $\exists s = \overline{\delta}(s_0, y) \in R$ and $\overline{\delta}(s, x) \in F'$ iff $x \in L(A')$.
- (c) Since A' is an NFA, $C(L) \in \mathcal{D}_\Sigma$. □

5.19. $\mathcal{D}_\Sigma$ is closed under F (maximal words not extended in L).

Let $A = \langle \Sigma, S, s_0, \delta, F \rangle$ be a minimal DFA for L . $F(L) = \{x \in L \mid \text{no nonempty word } y \text{ satisfies } xy \in L\}$. Define $A' = \langle \Sigma, S, s_0, \delta, F'' \rangle$ where $F'' = \{s \in F \mid (\forall y \in \Sigma^+) (\overline{\delta}(s, y) \notin F)\}$. Then $x \in L(A')$ iff $\overline{\delta}(s_0, x) \in F''$ iff $x \in L$ and there is no nonempty y with $\overline{\delta}(s_0, xy) \in F$, iff $x \in F(L)$. So $F(L) = L(A') \in \mathcal{D}_\Sigma$. □

5.20. $\mathcal{D}_\Sigma$ is closed under reversal.

- (a) **Construction.** Given DFA $A = \langle \Sigma, S, s_0, \delta, F \rangle$, build NFA $A^r = \langle \Sigma, S, F, \delta^r, \{s_0\} \rangle$ where $\delta^r(s, a) = \{t \in S \mid \delta(t, a) = s\}$ (reverse all arrows).

(b) **Proof.** We show $x \in L(A^r)$ iff $x^r \in L(A)$.

By induction on $|x|$: a path $f \xrightarrow{a_1} s_1 \xrightarrow{a_2} \dots \xrightarrow{a_k} s_0$ in A^r (from some $f \in F$ to s_0) corresponds to the path $s_0 \xrightarrow{a_k} \dots \xrightarrow{a_1} f$ in A . Hence $a_1 a_2 \dots a_k \in L(A^r)$ iff $a_k \dots a_1 \in L(A)$, i.e., $x \in L(A^r)$ iff $x^r \in L(A)$.

(c) $L^r = L(A^r) \in \mathcal{D}_\Sigma$ (since A^r is an NDFA which is equivalent to some DFA). $\square$

5.21. (a) $\mathcal{D}_\Sigma$ **is not closed under** $G(L) = \{w^r \cdot w \mid w \in L\}$.

Let $L = \Sigma^*$ over $\Sigma = \{a, b\}$. Then $G(L) = \{w^r w \mid w \in \Sigma^*\}$, the set of even-length palindromes (of the form $w^r w$). By the pumping lemma, this is not FAD: take $w = a^n b^n$; then $w^r w = b^n a^n a^n b^n \in G(L)$. If $G(L)$ were FAD with n states, pump the string $b^n a^n a^n b^n$ to get a contradiction (pumping the first block of b s destroys the palindrome structure).

(b) $\mathcal{N}_\Sigma$ **is closed under** G . Let $L \in \mathcal{N}_\Sigma$. If $G(L)$ were FAD, then Theorem 5.11 would imply that $G(L)^b$ is FAD. But $G(L)^b = L^r$, and Exercise 5.20 shows that reversal preserves the FAD languages. Hence $L = (L^r)^r$ would be FAD, contradicting $L \in \mathcal{N}_\Sigma$. Therefore $G(L) \in \mathcal{N}_\Sigma$.

(c) $\mathcal{N}_\Sigma$ **is not closed under** b . By Lemma 5.6, $L = \{a^n b^n \mid n \geq 0\} \in \mathcal{N}_\Sigma$, but $L^b = \{a^n \mid n \geq 0\}$ is FAD.

5.22. $\mathcal{N}_\Sigma$ **is closed under reversal**.

Assume $L \in \mathcal{N}_\Sigma$ but $L^r \notin \mathcal{N}_\Sigma$. Then $L^r \in \mathcal{D}_\Sigma$. By Exercise 5.20, $\mathcal{D}_\Sigma$ is closed under reversal, so $L = (L^r)^r \in \mathcal{D}_\Sigma$, a contradiction. Hence $L^r \in \mathcal{N}_\Sigma$ whenever $L \in \mathcal{N}_\Sigma$.

5.23. $\mathcal{N}_\Sigma$ **is not closed under** P (prefix operator).

Let $K = \{x \in \{a, b\}^* \mid |x|_a = |x|_b\}$. This language belongs to $\mathcal{N}_\Sigma$ (it is the language used in Lemma 5.4). But $P(K) = \{a, b\}^*$: given any word x , if $|x|_a - |x|_b = d$, append $|d|$ copies of the deficient symbol to obtain a balanced word. Thus $P(K)$ is FAD, so $\mathcal{N}_\Sigma$ is not closed under P .

5.24. $\mathcal{N}_\Sigma$ **is not closed under** S (suffix operator).

Let

$$K = \{x \in \{a, b\}^* \mid |x|_a = |x|_b\}.$$

Then $K \in \mathcal{N}_\Sigma$. For any word x , prefix enough copies of the deficient symbol to balance the counts; this shows $x \in S(K)$. Hence

$$S(K) = \{a, b\}^*.$$

Since $S(K)$ is FAD, $\mathcal{N}_\Sigma$ is not closed under S .

5.25. $\mathcal{N}_\Sigma$ **is not closed under** C (center operator).

Again let $K = \{x \mid |x|_a = |x|_b\}$. Then $C(K) = \{a, b\}^*$, because for any word x we can choose $y = a^m$ and $z = b^{m+|x|_a-|x|_b}$ with m large enough so that yxz has equally many a s and b s. Hence $C(K)$ is FAD while $K \in \mathcal{N}_\Sigma$.

5.26. $\mathcal{N}_\Sigma$ **is not closed under** F .

Let

$$K = \{x \in \{a, b\}^* \mid |x|_a = |x|_b\}.$$

Every word in K has a proper extension in K : if $x \in K$, then $xab \in K$. Therefore $F(K) = \emptyset$, which is FAD. So $\mathcal{N}_\Sigma$ is not closed under F .

5.27. **Complementing final states of an NFA does not give complement language.**

From Exercise 4.8: if A is an NFA, $L(A^\sim) \neq \sim L(A)$ in general.

Example. Let A have $\Sigma = \{a\}$, $S = \{s_0, s_1\}$, $S_0 = \{s_0\}$, $F = \{s_1\}$, $\delta(s_0, a) = \{s_0, s_1\}$, $\delta(s_1, a) = \{s_1\}$. Then $L(A) = \{a\}^+ = \{a, aa, \dots\}$. After complementing: $F^\sim = \{s_0\}$. a : from s_0 , reach $\{s_0, s_1\}$; $s_0 \in F^\sim$, so $a \in L(A^\sim)$. But $a \in L(A)$, so $a \notin \sim L(A)$. Thus $a \in L(A^\sim)$ but $a \notin \sim L(A)$, so $L(A^\sim) \neq \sim L(A)$.

5.28. **Complete the proof of Lemma 5.7.**

Proof. Let $L = \{x \in \{a, b\}^* \mid |x|_a \neq |x|_b\}$, the language used in Lemma 5.7. We show that $L/L = \Sigma^*$.

Take any $x \in \Sigma^*$ and let $d = |x|_a - |x|_b$. Define $y = a^{|d|+1}$. Then $y \in L$, because it has strictly more a s than b s. Also

$$|xy|_a - |xy|_b = d + (|d| + 1) \neq 0,$$

so $xy \in L$ as well. Hence $x \in L/L$. Since x was arbitrary, $L/L = \Sigma^*$. Therefore $\mathfrak{N}_\Sigma$ is not closed under quotient. $\square$

5.29. **Collection closed under union, concatenation, Kleene closure, but not intersection.**

Take $\mathcal{C} = I_\Sigma$, the collection of all infinite languages from Exercise 5.4. Exercise 5.4 shows that I_Σ is closed under union, concatenation, and Kleene closure, but not under intersection. So I_Σ is the required example.

5.30. **Closed under union does not imply closed under intersection.**

Counterexample. Again use I_Σ , the infinite languages. Exercise 5.4(b) shows it is closed under union, while Exercise 5.4(c) shows it is not closed under intersection.

5.31. **Prove $L(A^\bullet) = L_1 \cdot L_2$ (Theorem 5.4 construction).**

Proof. If $x \in L_1 \cdot L_2$, write $x = uv$ with $u \in L_1$ and $v \in L_2$. Run A_1 on u from s_{0_1} to some state of F_1 .

If $v = \lambda$, then $\lambda \in L_2$, so Theorem 5.4 makes every state of F_1 final in $A^\bullet$; hence the same path accepts x .

If $v = aw$ is nonempty, then from the final state reached after u the definition of $\delta^\bullet$ allows the next input letter a to be processed exactly as it would be from s_{0_2} in A_2 . The rest of v then follows the transitions of A_2 , so some path in $A^\bullet$ accepts x . Thus $L_1 \cdot L_2 \subseteq L(A^\bullet)$.

Conversely, let $x \in L(A^\bullet)$. An accepting path either stays entirely in S_1 and ends in a state of F_1 ; this can happen only when $\lambda \in L_2$, so then $x \in L_1 = L_1 \cdot \lambda \subseteq L_1 \cdot L_2$. Otherwise there is a first transition that enters S_2 . Just before that step, the path is at some state $f \in F_1$, so the already-read prefix u lies in L_1 . The rest of the input, beginning with the symbol that caused the move into S_2 , follows A_2 from s_{0_2} to a final state, so it lies in L_2 . Hence $x = uv \in L_1 \cdot L_2$. $\square$

5.32. **Prove $L(A_*) = L^*$ (Theorem 5.5 construction).**

Proof. Since $q_0 \in F_*$, the empty word is accepted, so $L^0 = \{\lambda\} \subseteq L(A_*)$.

Now let $x = x_1 x_2 \cdots x_k$ with each $x_i \in L$. Run the original machine A on x_1 from some start state in S_0 to a final state in F . Whenever one factor ends and the next factor begins with a letter a , the

definition of δ_* for final states adds the transitions $\bigcup_{t \in S_0} \delta(t, \mathbf{a})$, so the same input letter can also begin a new run of A on the next factor. Repeating this for all factors yields an accepting path for x , so $L^* \subseteq L(A_*)$.

Conversely, consider an accepting path of A_* on some nonempty word x . Because $\delta_*(q_0, \mathbf{a}) = \emptyset$, the path must begin in some state of S_0 . Each time the path uses one of the extra transitions $\bigcup_{t \in S_0} \delta(t, \mathbf{a})$ from a final state, it starts a fresh copy of the original machine on the next factor of the input. Thus the input splits into finitely many consecutive blocks, each of which labels a path in A from a start state to a final state; hence each block lies in L . Therefore $x \in L^*$. $\square$

5.33. Amplify the proof of Theorem 5.2 (union).

Theorem 5.2 uses NDFAs

$$A_1 = \langle \Sigma, S_1, S_{0_1}, \delta_1, F_1 \rangle, \quad A_2 = \langle \Sigma, S_2, S_{0_2}, \delta_2, F_2 \rangle,$$

and the union machine

$$A^\cup = \langle \Sigma, S_1 \cup S_2, S_{0_1} \cup S_{0_2}, \delta^\cup, F_1 \cup F_2 \rangle.$$

The book's chain of equivalences is:

$$\begin{aligned} x \in L(A^\cup) &\Leftrightarrow (\exists s \in S_{0_1} \cup S_{0_2}) [\overline{\delta^\cup}(s, x) \cap (F_1 \cup F_2) \neq \emptyset] \quad (\text{Definition 4.4}) \\ &\Leftrightarrow (\exists s \in S_{0_1}) [\overline{\delta^\cup}(s, x) \cap (F_1 \cup F_2) \neq \emptyset] \\ &\quad \vee (\exists s \in S_{0_2}) [\overline{\delta^\cup}(s, x) \cap (F_1 \cup F_2) \neq \emptyset] \quad (\text{definition of union}) \\ &\Leftrightarrow (\exists s \in S_{0_1}) [\overline{\delta_1}(s, x) \cap (F_1 \cup F_2) \neq \emptyset] \\ &\quad \vee (\exists s \in S_{0_2}) [\overline{\delta_2}(s, x) \cap (F_1 \cup F_2) \neq \emptyset] \quad (\text{by the inductive fact about } \overline{\delta^\cup}) \\ &\Leftrightarrow (\exists s \in S_{0_1}) [\overline{\delta_1}(s, x) \cap F_1 \neq \emptyset] \\ &\quad \vee (\exists s \in S_{0_2}) [\overline{\delta_2}(s, x) \cap F_2 \neq \emptyset] \quad (\text{since } S_1 \cap S_2 = \emptyset \text{ and } F^\cup = F_1 \cup F_2) \\ &\Leftrightarrow x \in L(A_1) \vee x \in L(A_2) \quad (\text{Definition 4.4}) \\ &\Leftrightarrow x \in L_1 \cup L_2. \end{aligned}$$

The missing ingredient in the textbook proof is exactly the inductive claim

$$\overline{\delta^\cup}(s, x) = \begin{cases} \overline{\delta_1}(s, x), & s \in S_1, \\ \overline{\delta_2}(s, x), & s \in S_2, \end{cases}$$

which is proved later in Exercise 5.50.

5.34. Given DFA A , define NFA accepting $L(A)^+$.

Define $A^+ = \langle \Sigma, S, s_0, \delta^+, F \rangle$ where $\delta^+(s, \mathbf{a}) = \{\delta(s, \mathbf{a})\} \cup \{s_0 \mid \delta(s, \mathbf{a}) \in F\}$. That is, whenever a transition would lead to a final state, simultaneously allow the machine to restart from s_0 . The final states remain F . This accepts $L^+ = \bigcup_{k \geq 1} L^k$.

5.35. Given NFA A , define NFA accepting $L(A)^+$.

Define $A^+ = \langle \Sigma, S, S_0, \delta^+, F \rangle$ where for each $s \in S$ and $\mathbf{a} \in \Sigma$: $\delta^+(s, \mathbf{a}) = \delta(s, \mathbf{a}) \cup (S_0 \text{ if } \delta(s, \mathbf{a}) \cap F \neq \emptyset)$. This allows returning to any start state after reaching a final state.

5.36. **If L is FAD, must all subsets of L be FAD?**

No. Let $\Sigma = \{a, b\}$ and $L = \Sigma^*$ (FAD). The subset $\{a^n b^n \mid n \geq 0\}$ is not FAD (by the pumping lemma).

5.37. **If $L \in \mathcal{D}_\Sigma$, must all supersets of L be in $\mathcal{D}_\Sigma$?**

No. Let $L = \emptyset \in \mathcal{D}_\Sigma$. The superset $\{a^n b^n \mid n \geq 0\}$ is not FAD.

5.38. **If $L \in \mathcal{N}_\Sigma$, must all subsets be in $\mathcal{N}_\Sigma$?**

No. Let $K = \{x \in \{a, b\}^* \mid |x|_a = |x|_b\} \in \mathcal{N}_\Sigma$. Then $\{\lambda\} \subseteq K$, but $\{\lambda\} \in \mathcal{D}_\Sigma$.

5.39. **If $L \in \mathcal{N}_\Sigma$, must all supersets be in $\mathcal{N}_\Sigma$?**

No. Let

$$K = \{x \in \{a, b\}^* \mid |x|_a = |x|_b\}.$$

Then $K \in \mathcal{N}_\Sigma$ and $K \subseteq \{a, b\}^*$, but $\{a, b\}^* \in \mathcal{D}_\Sigma$.

5.40. **Why is q_0 needed in the Kleene closure construction (Theorem 5.5)?**

The extra start state q_0 handles the case $\lambda \in L^*$ (which is always true) independently from the rest of the machine. Without q_0 , making s_0 both the start and a final state could create spurious acceptance: words that loop back through the original start states might be accepted even when they are not valid concatenations of words from L . The new state q_0 is needed only to accept λ without changing the behavior of the original start states.

5.41. **Redesign Theorem 5.4 (concatenation) using λ -transitions.**

Given DFAs A_1 and A_2 , construct $A_\lambda^\bullet = \langle \Sigma, S_1 \cup S_2, s_0^\bullet, \delta_\lambda^\bullet, F_2 \rangle$ where: $\delta_\lambda^\bullet(s, a) = \delta_1(s, a)$ for $s \in S_1$, $\delta_\lambda^\bullet(s, a) = \delta_2(s, a)$ for $s \in S_2$, and $\delta_\lambda^\bullet(f, \lambda) \ni s_0^\bullet$ for each $f \in F_1$. The λ -transition from each $f \in F_1$ to $s_0^\bullet$ cleanly separates the two phases.

5.42. **Redesign Theorem 5.5 (Kleene closure) using λ -transitions, without extra q_0 .**

Let

$$A = \langle \Sigma, S, S_0, \delta, F \rangle$$

be the NDFA for L . For each start state $s \in S_0$, introduce a new copy s' of that start state. Let

$$S' = \{s' \mid s \in S_0\},$$

with $S' \cap S = \emptyset$. Define the λ -NDFA

$$A_*^\lambda = \langle \Sigma, S \cup S', S', \delta_*^\lambda, F \cup S' \rangle$$

as follows:

1. For each old state $t \in S$ and each $a \in \Sigma$, keep the old transitions:

$$\delta_*^\lambda(t, a) = \delta(t, a).$$

2. For each copied start state $s' \in S'$ corresponding to $s \in S_0$, let it behave on letters exactly as s did:

$$\delta_*^\lambda(s', a) = \delta(s, a) \quad (a \in \Sigma).$$

3. For each final state $f \in F$, add λ -transitions from f to every copied start state:

$$\delta_*^\lambda(f, \lambda) \supseteq S'.$$

4. No other λ -transitions are added.

This makes the copied start states both start states and final states, so λ is accepted immediately, and every time a copy of a word from L is finished, the machine may jump by a λ -move to a copied start state and begin another copy.

Why it works. If $x = x_1 x_2 \cdots x_n$ with each $x_i \in L$, start in a copied start state corresponding to the start used for x_1 , follow the old transitions while reading x_1 , jump by λ from the reached final state to a copied start state for x_2 , and continue. Thus every word of L^* is accepted.

Conversely, every accepting computation starts in a copied start state, and the only way to enter a copied start state later is by a λ -move from a final state. Therefore the input splits into consecutive blocks, each of which takes the machine from some copied start state through the old machine to a final state. Replacing each copied start state by its original partner in S_0 shows that each block lies in L . Hence the whole input lies in L^* .

So this redesign does make the single extra start state q_0 unnecessary: the role of “accept λ and restart a new factor” is carried by copied start states together with λ -transitions from final states.

5.43. **Redesign Theorem 5.4 for NDFAs A_1 and A_2 .**

Given NDFAs $A_1 = \langle \Sigma, S_1, S_0^1, \delta_1, F_1 \rangle$ and $A_2 = \langle \Sigma, S_2, S_0^2, \delta_2, F_2 \rangle$ with $S_1 \cap S_2 = \emptyset$: Define

$$A^* = \langle \Sigma, S_1 \cup S_2, S_0^1, \delta^*, F^* \rangle,$$

where

$$F^* = \begin{cases} F_2, & \lambda \notin L_2, \\ F_1 \cup F_2, & \lambda \in L_2, \end{cases}$$

and

$$\delta^*(s, \mathbf{a}) = \begin{cases} \delta_1(s, \mathbf{a}), & s \in S_1 - F_1, \\ \delta_1(s, \mathbf{a}) \cup \bigcup_{t \in S_0^2} \delta_2(t, \mathbf{a}), & s \in F_1, \\ \delta_2(s, \mathbf{a}), & s \in S_2. \end{cases}$$

This is the exact NDFA analogue of Theorem 5.4: when the current state is already in F_1 , the same input symbol may also start a run of A_2 from any start state in S_0^2 .

5.44. **Redesign Theorem 5.5 for DFA A .**

If $A = \langle \Sigma, S, s_0, \delta, F \rangle$ is deterministic, the redesigned machine is the NDFA

$$A_* = \langle \Sigma, S \cup \{q_0\}, \{q_0, s_0\}, \delta_*, F \cup \{q_0\} \rangle,$$

where

$$\delta_*(s, \mathbf{a}) = \begin{cases} \{\delta(s, \mathbf{a})\}, & s \in S - F, \\ \{\delta(s, \mathbf{a}), \delta(s_0, \mathbf{a})\}, & s \in F, \\ \emptyset, & s = q_0. \end{cases}$$

This is the DFA-specialized form of Theorem 5.5, with singleton outputs exactly because the old transition function δ is deterministic.

5.45. **Comparing R_L and $R_{\sim L}$.**

The right congruence $\equiv_L$ on Σ^* is defined by $x \equiv_L y$ iff $\forall z \in \Sigma^*, xz \in L \Leftrightarrow yz \in L$. The right congruence $\equiv_{\sim L}$ satisfies $x \equiv_{\sim L} y$ iff $\forall z, xz \in \sim L \Leftrightarrow yz \in \sim L$, i.e., $xz \notin L \Leftrightarrow yz \notin L$, i.e., $xz \in L \Leftrightarrow yz \in L$. So $\equiv_L = \equiv_{\sim L}$: the two right congruences are identical.

This makes sense: if A is a minimal DFA for L , then $A^\sim$ (with complemented final states) is a minimal DFA for $\sim L$, and both have the same state set and transition function. Thus $R_L = R_{\sim L}$.

- 5.46. (a) **Examples where $R_{L_1 \cap L_2} = R_{L_1} \cap R_{L_2}$.** Let $L_1 = L_2$ be any FAD language. Then $L_1 \cap L_2 = L_1$, so $R_{L_1 \cap L_2} = R_{L_1} = R_{L_1} \cap R_{L_1} = R_{L_1} \cap R_{L_2}$.
- (b) **Examples where $R_{L_1 \cap L_2} \neq R_{L_1} \cap R_{L_2}$.** Let $L_1 = \{a\}$ and $L_2 = \{b\}$ over $\Sigma = \{a, b\}$. Then $L_1 \cap L_2 = \emptyset$, so $R_{L_1 \cap L_2} = R_\emptyset$ is the universal relation. But $\lambda R_{L_1} a$, because with suffix λ the word a is in L_1 and λ is not. Hence

$$R_{L_1 \cap L_2} \neq R_{L_1} \cap R_{L_2}.$$

5.47. **$\mathcal{D}_\Sigma$ is closed under relative complement.**

- (a) **By theorems.** $L_1 \setminus L_2 = L_1 \cap \sim L_2$. Since $\mathcal{D}_\Sigma$ is closed under complement (Theorem 5.1) and intersection (Lemma 5.1), $L_1 \setminus L_2 \in \mathcal{D}_\Sigma$.
- (b) **Machine definition.** Define A^- as the product of A_1 and $A_2^\sim$ (complement of A_2), using the intersection machine from Lemma 5.1 with $F^- = F_1 \times (S_2 \setminus F_2)$.
- (c) **Proof.** We prove the stronger claim, by induction on $|x|$, that for all $s_1 \in S_1$ and $s_2 \in S_2$,

$$\overline{\delta^-}(\langle s_1, s_2 \rangle, x) = \langle \overline{\delta^1}(s_1, x), \overline{\delta^2}(s_2, x) \rangle.$$

For $x = \lambda$ both sides are $\langle s_1, s_2 \rangle$. If the claim holds for x , then for any letter a , Definition 1.1.1iii gives

$$\begin{aligned} \overline{\delta^-}(\langle s_1, s_2 \rangle, ax) &= \overline{\delta^-}(\delta^-(\langle s_1, s_2 \rangle, a), x) \\ &= \overline{\delta^-}(\langle \delta_1(s_1, a), \delta_2(s_2, a) \rangle, x) = \langle \overline{\delta^1}(\delta_1(s_1, a), x), \overline{\delta^2}(\delta_2(s_2, a), x) \rangle = \langle \overline{\delta^1}(s_1, ax), \overline{\delta^2}(s_2, ax) \rangle. \end{aligned}$$

In particular, the claim holds for $\langle s_0^1, s_0^2 \rangle$. Therefore $x \in L(A^-)$ iff $\overline{\delta^1}(s_0^1, x) \in F_1$ and $\overline{\delta^2}(s_0^2, x) \notin F_2$, which is exactly the condition $x \in L_1 \setminus L_2$. $\square$

5.48. **$\mathcal{L}_\Sigma$ = languages recognized by λ -NDFAs.** By Theorem 4.2, $\mathcal{L}_\Sigma = \mathcal{D}_\Sigma$. Hence $\mathcal{L}_\Sigma$ has exactly the same closure properties as $\mathcal{D}_\Sigma$ (Theorems 5.1–5.12).

- 5.49. (a) **Example of L with $\lambda \in L^+$.** Let $L = \{\lambda\}$. Then $L^+ = \bigcup_{k \geq 1} L^k = \{\lambda\}$, and $\lambda \in L^+$.
- (b) **Three languages with $L^+ = L$.**
- (i) $L = \Sigma^*$: $(\Sigma^*)^+ = \Sigma^*$.
 - (ii) $L = \{a\}^*$: $(\{a\}^*)^+ = \{a\}^*$.
 - (iii) $L = \Sigma^+$: $(\Sigma^+)^+ = \Sigma^+$ (since $\Sigma^+ \cdot \Sigma^+ \subseteq \Sigma^+$).

- 5.50. (a) **Prove $\overline{\delta^U}(s, x) = \overline{\delta_i}(s, x)$ for $s \in S_i$.**

Proof. By induction on $|x|$.

Base ($|x| = 0$): $\overline{\delta}^\cup(s, \lambda) = \{s\} = \overline{\delta}_i(s, \lambda)$ (by definition of extended transition functions for single start-states, noting NDFAs).

Step: Assume $\overline{\delta}^\cup(s, x) = \overline{\delta}_i(s, x)$ for $s \in S_i$. For $x\mathbf{a}$:

$$\overline{\delta}^\cup(s, x\mathbf{a}) = \bigcup_{t \in \overline{\delta}^\cup(s, x)} \delta^\cup(t, \mathbf{a}) = \bigcup_{t \in \overline{\delta}_i(s, x)} \delta_i(t, \mathbf{a}) = \overline{\delta}_i(s, x\mathbf{a}),$$

where we used the fact that $\overline{\delta}_i(s, x) \subseteq S_i$ (paths from $s \in S_i$ stay in S_i , since $\delta_i : S_i \rightarrow \wp(S_i)$) and $\delta^\cup(t, \mathbf{a}) = \delta_i(t, \mathbf{a})$ for $t \in S_i$. $\square$ $\square$

(b) **Was this used in Theorem 5.2?** Yes, implicitly: the proof that $L(A^\cup) = L_1 \cup L_2$ relies on the fact that paths starting in S_1 stay in S_1 and paths starting in S_2 stay in S_2 , which is exactly this lemma.

- 5.51. (a) $\mathfrak{N}_\Sigma$ **is not closed under relative complement.** Let $K = \{x \in \{\mathbf{a}, \mathbf{b}\}^* \mid |x|_{\mathbf{a}} = |x|_{\mathbf{b}}\} \in \mathfrak{N}_\Sigma$. Then $K - K = \emptyset \in \mathfrak{D}_\Sigma$.
- (b) $\mathfrak{N}_\Sigma$ **is not closed under union.** If $K \in \mathfrak{N}_\Sigma$, then $\sim K \in \mathfrak{N}_\Sigma$ by Theorem 5.8. But $K \cup \sim K = \Sigma^* \in \mathfrak{D}_\Sigma$.
- (c) $\mathfrak{N}_\Sigma$ **is not closed under concatenation.** Let $E = \{\mathbf{a}^n \mathbf{b}^n \mid n \geq 1\}$ and define $L = \{\mathbf{a}, \mathbf{b}\}^* - E$. Since $E \in \mathfrak{N}_\Sigma$, Theorem 5.8 gives $L \in \mathfrak{N}_\Sigma$. Also $\lambda \in L$, and if $x \notin L$ then $x = \mathbf{a}^n \mathbf{b}^n = \mathbf{a} \cdot \mathbf{a}^{n-1} \mathbf{b}^n$ with both factors in L . Hence $L \cdot L = \{\mathbf{a}, \mathbf{b}\}^* \in \mathfrak{D}_\Sigma$.
- (d) $\mathfrak{N}_\Sigma$ **is not closed under Kleene closure.** For

$$L = \{\mathbf{a}, \mathbf{b}\}^* - \{\mathbf{a}^n \mathbf{b}^n \mid n \geq 1\},$$

we have $\lambda \in L$ and $L \cdot L = \Sigma^*$, so $L^* = \Sigma^* \in \mathfrak{D}_\Sigma$.

- (e) **If $L \in \mathfrak{N}_\Sigma$, then $L^+ \in \mathfrak{N}_\Sigma$: No.** For

$$L = \{\mathbf{a}, \mathbf{b}\}^* - \{\mathbf{a}^n \mathbf{b}^n \mid n \geq 1\},$$

because $\lambda \in L$ we have $L^+ = L^* = \Sigma^*$, which is FAD.

5.52. **Why require $S_1 \cap S_2 = \emptyset$ in Theorem 5.4?**

If $S_1 \cap S_2 \neq \emptyset$, the transition function $\delta^\bullet$ would be ambiguous: for a state $s \in S_1 \cap S_2$, which transitions should apply—those of A_1 or A_2 ? The definition of $\delta^\bullet$ would not be well-defined. Moreover, the proof that paths through the concatenation machine correctly simulate A_1 then A_2 relies on the fact that once the computation enters S_2 , it stays in S_2 , and once it begins in S_1 it follows only the A_1 transitions until such a handoff occurs. If the state sets overlap, a path might inadvertently “mix” transitions from A_1 and A_2 .

5.53. $\mathfrak{D}_\Sigma$ **is closed under** $E(L) = \{z \mid \exists y \in \Sigma^+, \exists x \in L \ni z = yx\}$.

Here $E(L) = \Sigma^+ \cdot L$, the set of words obtained by prepending a nonempty prefix to a word of L .

- (a) **Construction.** Let B be the two-state DFA for Σ^+ : start state p_0 (nonfinal), final state p_1 , and for every $\mathbf{a} \in \Sigma$, $\delta_B(p_0, \mathbf{a}) = p_1$ and $\delta_B(p_1, \mathbf{a}) = p_1$. If A accepts L , apply the concatenation construction of Theorem 5.4 to B and A .

(b) **Proof.** A word z is accepted by the new machine iff it can be written as $z = yx$ where y is accepted by B and x is accepted by A . That is equivalent to saying $y \in \Sigma^+$ and $x \in L$, so the new machine accepts exactly $E(L)$.

(c) Since $\Sigma^+ \in \mathcal{D}_\Sigma$ and $\mathcal{D}_\Sigma$ is closed under concatenation, $E(L) = \Sigma^+ \cdot L \in \mathcal{D}_\Sigma$. $\square$

5.54. $\mathcal{D}_\Sigma$ is closed under $B(L) = \{z \mid \exists x \in L, \exists y \in \Sigma^* \ni z = xy\}$.

$B(L)$ is the set of all words that have a prefix in L .

(a) **Construction.** Given $A = \langle \Sigma, S, s_0, \delta, F \rangle$ for L , build

$$A' = \langle \Sigma, S \times \{0, 1\}, \langle s_0, 0 \rangle, \delta', S \times \{1\} \rangle,$$

where the second component records whether some prefix read so far belongs to L . Define

$$\delta'(\langle s, b \rangle, a) = \langle t, b' \rangle,$$

where $t = \delta(s, a)$ and

$$b' = \begin{cases} 1 & \text{if } b = 1 \text{ or } t \in F, \\ 0 & \text{otherwise.} \end{cases}$$

Thus once the flag becomes 1, it stays 1 forever.

(b) **Proof:** By induction on the length of the input, after reading a word z the machine A' is in state

$$\langle \bar{\delta}(s_0, z), 1 \rangle$$

exactly when some prefix of z is in L , and otherwise in

$$\langle \bar{\delta}(s_0, z), 0 \rangle.$$

Therefore A' accepts exactly those words having a prefix in L , that is, exactly the words in $B(L)$.

(c) $\square$

5.55. $\mathcal{D}_\Sigma$ is closed under $M(L) = \{z \mid \exists x \in L, \exists y \in \Sigma^+ \ni z = xy\}$.

$M(L) = \{z \mid z = xy, x \in L, y \in \Sigma^+\} = L \cdot \Sigma^+$. By closure under concatenation, $L \cdot \Sigma^+ \in \mathcal{D}_\Sigma$.

(a) **Construction:** Concatenate A (for L) with a DFA for Σ^+ (which is a two-state machine: start nonfinal, self-loop on Σ to final, final self-loops on Σ).

(b) **Proof:** By Theorem 5.4, $M(L) = L(A^\bullet) \in \mathcal{D}_\Sigma$.

(c) $\square$

5.56. **Prove** $\bar{\delta}^\cap(\langle s, t \rangle, x) = \langle \bar{\delta}_1(s, x), \bar{\delta}_2(t, x) \rangle$.

Proof. We prove the stronger statement, for all state pairs $\langle s, t \rangle$, by induction on $|x|$.

Base: $\bar{\delta}^\cap(\langle s, t \rangle, \lambda) = \langle s, t \rangle = \langle \bar{\delta}_1(s, \lambda), \bar{\delta}_2(t, \lambda) \rangle$.

Step: Let the string be $\mathbf{ax}$, where $\mathbf{a}$ is a single letter and x is the remaining string. Assume the result for x and arbitrary states. Then, using Definition 1.11iii for the first and last equalities,

$$\begin{aligned}
 \overline{\delta^\cap}(\langle s, t \rangle, \mathbf{ax}) &= \overline{\delta^\cap}(\delta^\cap(\langle s, t \rangle, \mathbf{a}), x) \\
 &= \overline{\delta^\cap}(\langle \delta_1(s, \mathbf{a}), \delta_2(t, \mathbf{a}) \rangle, x) \\
 &= \langle \overline{\delta_1}(\delta_1(s, \mathbf{a}), x), \overline{\delta_2}(\delta_2(t, \mathbf{a}), x) \rangle \\
 &= \langle \overline{\delta_1}(s, \mathbf{ax}), \overline{\delta_2}(t, \mathbf{ax}) \rangle. \quad \square
 \end{aligned}$$

□

5.57. **Prove** $L(A^\cap) = L_1 \cap L_2$.

Proof. By Exercise 5.56:

$$\begin{aligned}
 x \in L(A^\cap) &\Leftrightarrow \overline{\delta^\cap}(\langle s_0^1, s_0^2 \rangle, x) \in F^\cap \\
 &\Leftrightarrow \langle \overline{\delta_1}(s_0^1, x), \overline{\delta_2}(s_0^2, x) \rangle \in F_1 \times F_2 \\
 &\Leftrightarrow \overline{\delta_1}(s_0^1, x) \in F_1 \wedge \overline{\delta_2}(s_0^2, x) \in F_2 \\
 &\Leftrightarrow x \in L_1 \wedge x \in L_2 \\
 &\Leftrightarrow x \in L_1 \cap L_2. \quad \square
 \end{aligned}$$

□

5.58. **Prove Theorem 5.6** ($\mathcal{D}_\Sigma$ closed under Z).

Proof. Let $A = \langle \Sigma, S, s_0, \delta, F \rangle$ be a DFA for L . Build a λ -NDEFA A_Z by adding, for every ordinary transition $s \xrightarrow{\mathbf{a}} t$ of A , a matching λ -transition $s \xrightarrow{\lambda} t$. Then a path in A_Z may either read a letter of the input or skip a letter that was present in some word of L .

Thus x is accepted by A_Z iff there exists some $w \in L$ from which x can be obtained by deleting zero or more letters, i.e., iff $x \in Z(L)$. Hence $Z(L) \in \mathcal{D}_\Sigma$. □

5.59. **Prove Theorem 5.7** ($\mathcal{D}_\Sigma$ closed under Y).

Proof. Let $A = \langle \Sigma, S, s_0, \delta, F \rangle$ be a DFA for L . Following the text after Theorem 5.7, build two copies of A . Being in the first copy means that no letter has yet been deleted; being in the second copy means that exactly one letter has been deleted. Ordinary transitions stay within each copy, and for every transition $s \xrightarrow{\mathbf{a}} t$ in the first copy add a λ -transition from the first-copy state s to the second-copy state t .

An accepting path therefore behaves like a path of A on some word of L , with exactly one letter skipped at the moment when the path moves from the first copy to the second. So the resulting machine accepts exactly $Y(L)$, and therefore $Y(L) \in \mathcal{D}_\Sigma$. □

5.60. (a) **Alternative construction for Theorem 5.7 without λ -moves.**

Let $A = \langle \Sigma, S, s_0, \delta, F \rangle$ be a DFA for L . Use state set $S \times \{0, 1\}$, where the second component records whether the one deletion has already occurred. The start state is $\langle s_0, 0 \rangle$. Final states are

$$\{\langle s, 1 \rangle \mid s \in F\} \cup \{\langle s, 0 \rangle \mid (\exists \mathbf{a} \in \Sigma) \delta(s, \mathbf{a}) \in F\}.$$

On input $\mathbf{a}$, define

$$\delta'(\langle s, 0 \rangle, \mathbf{a}) = \{\langle \delta(s, \mathbf{a}), 0 \rangle\} \cup \{\langle \delta(s, \mathbf{b}), 1 \rangle \mid \mathbf{b} \in \Sigma\},$$

and

$$\delta'(\langle s, 1 \rangle, \mathbf{a}) = \{\langle \delta(s, \mathbf{a}), 1 \rangle\}.$$

- (b) **Proof:** In the 0-copy the machine either reads the current input symbol normally, or guesses that one letter $\mathbf{b}$ of the original word was deleted immediately before it and moves into the 1-copy. From that moment on, if the original machine would have been in state s after reading the undeleted word, the new machine is in the state $\langle s, 1 \rangle$ after reading the shortened word. Thus the 1-copy simulates the remainder of the computation of A on the undeleted word. The extra final states in the 0-copy handle the case where the deleted letter was the last letter of the original word. Hence this NDFA accepts exactly $Y(L)$.

5.61. $\mathcal{D}_\Sigma$ is closed under $W(L)$ (delete one or more letters).

- (a) **Construction.** Given DFA A for L , build λ -NDEFA A' by adding, for each transition $s \xrightarrow{\mathbf{a}} t$, an optional λ -transition $s \xrightarrow{\lambda} t$ (allowing deletion of $\mathbf{a}$). Require at least one deletion by adding a flag bit.

More precisely: A' has state set $S \times \{0, 1\}$ where the second component tracks whether at least one letter has been deleted. Initial state: $\langle s_0, 0 \rangle$. Final states: $\{\langle f, 1 \rangle \mid f \in F\}$. For each $c \in \Sigma$,

$$\delta'(\langle s, b \rangle, c) = \{\langle \delta(s, c), b \rangle\}, \quad \delta'(\langle s, b \rangle, \lambda) = \{\langle \delta(s, c), 1 \rangle \mid c \in \Sigma\}.$$

The ordinary transition reads the next input symbol, while the λ -transition simulates deleting one symbol c from the original word and moves into the “already deleted” copy.

- (b) **Proof.** A word z is in $W(L)$ iff z can be obtained by deleting at least one letter from some $w \in L$. A path in A' simulates a path in A but uses λ -transitions to skip letters; the flag ensures at least one skip. Hence $L(A') = W(L)$.

- (c) Since A' is a λ -NDEFA, $W(L) \in \mathcal{D}_\Sigma$. □

5.62. $\mathcal{D}_\Sigma$ is closed under $V(L)$ (delete odd-positioned letters).

- (a) **Construction.** Let $A = \langle \Sigma, S, s_0, \delta, F \rangle$. Build the λ -NDEFA

$$A' = \langle \Sigma, S \times \{0, 1\}, \langle s_0, 0 \rangle, \delta', F \times \{0, 1\} \rangle,$$

where the second component records the parity of the next position in the original word: 0 means odd, 1 means even. For each $s \in S$ and $\mathbf{a} \in \Sigma$,

$$\delta'(\langle s, 1 \rangle, \mathbf{a}) = \{\langle \delta(s, \mathbf{a}), 0 \rangle\},$$

because even-positioned letters are kept, and

$$\delta'(\langle s, 0 \rangle, \lambda) = \{\langle \delta(s, c), 1 \rangle \mid c \in \Sigma\},$$

because an odd-positioned letter is deleted and may be any symbol c .

- (b) **Proof.** A computation of A' alternates between deleting one symbol of the original word at every odd position and reading one input symbol at every even position. Thus the visible input is exactly the word obtained by deleting the odd-positioned letters of some word accepted by A . Conversely, if a word z arises from deleting the odd-positioned letters of some $w \in L$, then A' can guess those deleted odd-positioned letters by λ -moves and read precisely the surviving even-positioned letters. So $L(A') = V(L)$.

- (c) $L(A') \in \mathcal{D}_\Sigma$. □

5.63. $\mathcal{D}_\Sigma$ is closed under $U(L)$ (delete even-positioned letters).

Let

$$A = \langle \Sigma, S, s_0, \delta, F \rangle.$$

Build the λ -NDEFA

$$A' = \langle \Sigma, S \times \{0, 1\}, \langle s_0, 0 \rangle, \delta', F \times \{0, 1\} \rangle,$$

where again 0 means the next position in the original word is odd and 1 means even. This time odd-positioned letters are kept and even-positioned letters are deleted:

$$\delta'(\langle s, 0 \rangle, a) = \{\langle \delta(s, a), 1 \rangle\} \quad (a \in \Sigma),$$

$$\delta'(\langle s, 1 \rangle, \lambda) = \{\langle \delta(s, c), 0 \rangle \mid c \in \Sigma\}.$$

To see what A' accepts, read an input word $x = x_1 x_2 \cdots x_k$. In state $\langle s, 0 \rangle$ the machine is about to read an odd-positioned letter of the original word, so it reads one actual input symbol and moves to $\langle \delta(s, x_i), 1 \rangle$. In state $\langle s, 1 \rangle$ it is about to skip an even-positioned letter of the original word, so it guesses that deleted letter by a λ -move to $\langle \delta(s, c), 0 \rangle$ for some $c \in \Sigma$. Thus every other letter is guessed and deleted, and the machine accepts precisely the words obtained from words of L by deleting the even-positioned letters. Hence $U(L) \in \mathcal{D}_\Sigma$. □

5.64. $\mathcal{D}_\Sigma$ is closed under $T(L)$ (delete every 3rd, 6th, ... letter).

- (a) **Construction.** Let $A = \langle \Sigma, S, s_0, \delta, F \rangle$. Build the λ -NDEFA

$$A' = \langle \Sigma, S \times \{1, 2, 0\}, \langle s_0, 1 \rangle, \delta', F \times \{1, 2, 0\} \rangle,$$

where the second component records the next position modulo 3. For each $s \in S$ and $a \in \Sigma$,

$$\delta'(\langle s, 1 \rangle, a) = \{\langle \delta(s, a), 2 \rangle\}, \quad \delta'(\langle s, 2 \rangle, a) = \{\langle \delta(s, a), 0 \rangle\},$$

because positions 1 and 2 modulo 3 are kept, while

$$\delta'(\langle s, 0 \rangle, \lambda) = \{\langle \delta(s, c), 1 \rangle \mid c \in \Sigma\},$$

because every third letter is deleted and may be any symbol c .

- (b) **Proof.** The machine simulates A on an original word while the counter records which positions are first, second, and third in each block of three. At positions congruent to 1 or 2 modulo 3, it reads the next input symbol and processes it through δ ; at positions congruent to 0 modulo 3, it guesses and deletes one original symbol by a λ -move. Thus the visible input is exactly the word obtained from the original one by deleting every 3rd, 6th, 9th, ... letter. Hence $L(A') = T(L)$.

(c) $L(A') \in \mathcal{D}_\Sigma$. □

5.65. $I(L) = L \cap P$ where $P = \{x \mid |x| \text{ is prime}\}$.

- (a) $\mathcal{D}_\Sigma$ **not closed under I** . Let $\Sigma = \{a\}$ and $L = \Sigma^*$. Then

$$I(L) = P = \{a^q \mid q \text{ prime}\}.$$

We show $P \notin \mathcal{D}_\Sigma$ by the pumping lemma. Assume P were regular, and let p be a pumping length. Choose a prime $q > p$, and consider $a^q \in P$. Write

$$a^q = uvw$$

with $|uv| \leq p$ and $|v| \geq 1$. Since the word is unary, let $|v| = m \geq 1$. Pump with $i = q + 1$. Then

$$|uv^{q+1}w| = q + qm = q(1 + m).$$

This number is composite, because $q > 1$ and $1 + m > 1$. Hence $uv^{q+1}w \notin P$, contradicting the pumping lemma. Therefore $P \notin \mathcal{D}_\Sigma$.

- (b) F_Σ **closed under I** . $I(L) = L \cap P \subseteq L$; if L is finite, so is $I(L)$.
(c) C_Σ **closed under I ? No.** Take $L = \Sigma^*$. Then $L \in C_\Sigma$, but $I(L) = P$, the set of prime-length words. P is neither finite nor cofinite, since there are infinitely many primes and infinitely many non-primes. **Not closed.**
(d) B_Σ **closed under I ? No.** $I(\Sigma^*) = P \notin B_\Sigma$ (not finite, not cofinite).
(e) I_Σ **closed under I ? No.** Let $\Sigma = \{a\}$ and $L = \{a^{2^n} \mid n \geq 1\}$. Then $L \in I_\Sigma$, but

$$I(L) = L \cap P = \{aa\},$$

since 2 is the only even prime. Thus $I(L) \notin I_\Sigma$.

- (f) J_Σ **closed under I ? $I(L) = L \cap P$. If $\sim L$ is infinite, $\sim(L \cap P) \supseteq \sim P$ which is infinite (non-primes). **Yes, closed.****
(g) **E closed under I ? If $abba \in L$ and $|abba| = 4$ is not prime, then $abba \notin I(L)$. So $I(L)$ might not contain $abba$. **Not closed.****
(h) $\mathcal{N}_\Sigma$ **closed under I ? No.** Over $\Sigma = \{a\}$, let $P = \{a^p \mid p \text{ prime}\}$ and $L = \{a\}^* - P$. Since $P \notin \mathcal{D}_\Sigma$, Theorem 5.8 implies $L \in \mathcal{N}_\Sigma$. But

$$I(L) = L \cap P = \emptyset \in \mathcal{D}_\Sigma.$$

Hence $\mathcal{N}_\Sigma$ is not closed under I .

5.66. C : all languages over $\{a, b\}$ not containing λ .

- (a) **Complementation: No.** $\sim L$ contains λ (since $\lambda \notin L$ but $\lambda \in \Sigma^*$), so $\sim L \notin C$.
- (b) **Union: Yes.** If $\lambda \notin L_1$ and $\lambda \notin L_2$, then $\lambda \notin L_1 \cup L_2$.
- (c) **Intersection: Yes.** $\lambda \notin L_1 \cap L_2 \subseteq L_1$.
- (d) **Concatenation: Yes.** If $\lambda \notin L_1$ and $\lambda \notin L_2$: every word in $L_1 \cdot L_2$ is of the form xy with $x \in L_1, y \in L_2$, and $x \neq \lambda, y \neq \lambda$, so $xy \neq \lambda$.
- (e) **Kleene closure: No.** $\lambda \in L^*$ always.
- (f) **Relative complement: Yes.** $\lambda \notin L_1 \setminus L_2 \subseteq L_1$.
- (g) **L^+ : Yes.** $L^+ = \bigcup_{k \geq 1} L^k$. Each L^k contains words of positive length (since $\lambda \notin L$), so $\lambda \notin L^+$.

5.67. (a) $\mathfrak{D}_\Sigma$ **closed under finite union.**

- (i) *By theorems and induction:* Base: $\{L\} \subseteq \mathfrak{D}_\Sigma$ trivially. Step: if $L_1 \cup L_2 \cup \dots \cup L_k \in \mathfrak{D}_\Sigma$ and $L_{k+1} \in \mathfrak{D}_\Sigma$, then $(L_1 \cup \dots \cup L_k) \cup L_{k+1} \in \mathfrak{D}_\Sigma$ by Theorem 5.2.
- (ii) *By construction:* Build $A^{(k)}$ iteratively by the union construction of Exercise 5.14; each step is deterministic.

(b) $\mathfrak{D}_\Sigma$ **is NOT closed under infinite union.** Each singleton $\{a^n b^n\} \in \mathfrak{D}_\Sigma$ (finite languages are regular), but $\bigcup_{n \geq 0} \{a^n b^n\} = \{a^n b^n \mid n \geq 0\} \notin \mathfrak{D}_\Sigma$ (not regular, by pumping).

5.68. (a) **Three homomorphisms under which $\mathfrak{N}_\Sigma$ is not closed** (over $\Sigma = \{a, b\}$): Use the non-FAD language $K = \{x \in \{a, b\}^* \mid |x|_a = |x|_b\}$ from Lemma 5.4.

- (i) $\xi_1(a) = a, \xi_1(b) = a$. By Lemma 5.4, $\overline{\xi_1(K)}$ is the regular language of even-length a -strings.
- (ii) $\xi_2(a) = \lambda, \xi_2(b) = \lambda$. Then $\overline{\xi_2(K)} = \{\lambda\} \in \mathfrak{D}_\Sigma$.
- (iii) $\xi_3(a) = a, \xi_3(b) = \lambda$. For $L = \{a^n b^n \mid n \geq 0\} \in \mathfrak{N}_\Sigma$, we get $\overline{\xi_3(L)} = \{a^n \mid n \geq 0\} \in \mathfrak{D}_\Sigma$.

(b) **Three homomorphisms under which $\mathfrak{N}_\Sigma$ is closed:**

- (i) The identity map: $\psi_1(a) = a, \psi_1(b) = b$.
- (ii) The swap map: $\psi_2(a) = b, \psi_2(b) = a$.
- (iii) The doubling map: $\psi_3(a) = aa, \psi_3(b) = bb$.

For ψ_1 and ψ_2 , if $\overline{\psi_i(L)}$ were FAD then applying the inverse letter permutation would make L FAD. For ψ_3 , the map is injective, so

$$L = \psi_3^{-1}(\overline{\psi_3(L)}).$$

Thus, if $\overline{\psi_3(L)}$ were FAD, then Theorem 5.10 (closure under inverse homomorphism) would imply that L is FAD as well.

5.69. Over $\Sigma = \{a\}$ there is only one homomorphism under which $\mathfrak{N}_\Sigma$ is not closed, namely the erasing homomorphism $\psi(a) = \lambda$. For any non-FAD unary language L , $\overline{\psi(L)} = \{\lambda\} \in \mathfrak{D}_\Sigma$.

For every other unary homomorphism, $\psi(a) = a^k$ with $k \geq 1$. Such a homomorphism is injective, so if $\overline{\psi(L)}$ were FAD, then

$$L = \overline{\psi^{-1}(\overline{\psi(L)})}$$

would be FAD by Theorem 5.10. Therefore $\mathfrak{N}_\Sigma$ is closed under every nonerasing unary homomorphism. So the answer is **no**: there are not two different unary homomorphisms under which $\mathfrak{N}_\Sigma$ is not closed.

5.70. (a) **Prove** $\overline{\delta^\psi}(s, x) = \overline{\delta}(s, \overline{\psi}(x))$.

Proof. We prove the stronger statement, for all states s , by induction on $|x|$.

Base: $\overline{\delta^\psi}(s, \lambda) = s = \overline{\delta}(s, \lambda) = \overline{\delta}(s, \overline{\psi}(\lambda))$ (since $\psi(\lambda) = \lambda$).

Step: Let the string be $\mathbf{b}x$, where $\mathbf{b}$ is a single letter and x is the remaining string. Assume the result for x and arbitrary states. Then, using Definition 1.11iii for the first and last equalities,

$$\overline{\delta^\psi}(s, \mathbf{b}x) = \overline{\delta^\psi}(\delta^\psi(s, \mathbf{b}), x) = \overline{\delta^\psi}(\overline{\delta}(s, \psi(\mathbf{b})), x) = \overline{\delta}(\overline{\delta}(s, \psi(\mathbf{b})), \overline{\psi}(x)) = \overline{\delta}(s, \psi(\mathbf{b})\overline{\psi}(x)) = \overline{\delta}(s, \overline{\psi}(\mathbf{b}x)).$$

This is exactly the desired statement for $\mathbf{b}x$. □ □

(b) **Proof of Theorem 5.10:** $y \in L(A^\psi)$ iff $\overline{\delta^\psi}(s_0, y) \in F$ iff $\overline{\delta}(s_0, \psi(y)) \in F$ iff $\psi(y) \in L$ iff $y \in \psi^{-1}(L)$. □

5.71. (a) **Apparent contradiction resolved.** $\xi(L)$ is the set of even-length $\mathbf{a}$ -strings; $\xi^{-1}(\xi(L))$ is the set of all strings x with $\xi(x) \in \xi(L)$, not L itself. In general, $\xi^{-1}(\xi(L)) \supseteq L$ and can be much larger; it is not necessarily L . The inverse image lands in $\mathcal{D}_\Sigma$ by Theorem 5.10, but it's $\xi^{-1}(\xi(L))$, not L .

(b) **Example where $\overline{\psi}(\psi^{-1}(L)) \neq L$:** Take $\psi(\mathbf{a}) = \mathbf{aa}$, $L = \{\mathbf{a}\}$ (singleton). Then $\psi^{-1}(L) = \emptyset$ (no word maps to $\mathbf{a}$ under ψ , since ψ always produces even-length strings), and $\overline{\psi}(\emptyset) = \emptyset \neq L$.

(c) **Example where $\psi^{-1}(\overline{\psi}(L)) \neq L$:** Same: $\psi^{-1}(\overline{\psi}(\{\mathbf{a}\})) = \psi^{-1}(\{\mathbf{a}\}) = \emptyset \neq \{\mathbf{a}\} = L$.

(d) **Prove $\overline{\psi}(\psi^{-1}(L)) \subseteq L$:** Let $y \in \overline{\psi}(\psi^{-1}(L))$. Then $y = \psi(x)$ for some $x \in \psi^{-1}(L)$. Since $x \in \psi^{-1}(L)$, $\psi(x) \in L$, i.e., $y \in L$. □

(e) **Prove $L \subseteq \psi^{-1}(\overline{\psi}(L))$:** Let $x \in L$. Then $\psi(x) \in \overline{\psi}(L)$ (since $\psi(x)$ is the image of some element of L , namely x itself). Hence $x \in \psi^{-1}(\overline{\psi}(L))$. □

5.72. $\mathcal{D}_\Sigma$ is closed under L^e (last halves of even-length words in L).

$$L^e = \{x \mid \exists y \in \Sigma^* \ni yx \in L \wedge |x| = |y|\}.$$

Proof. Observe that

$$x \in L^e \iff \exists y \in \Sigma^* (yx \in L \wedge |y| = |x|) \iff x^r \in (L^r)^b.$$

Indeed, $yx \in L$ with $|y| = |x|$ iff $x^r y^r \in L^r$ with $|x^r| = |y^r|$, which says exactly that x^r is a first half of some word in L^r .

Therefore

$$L^e = ((L^r)^b)^r.$$

Exercise 5.20 shows that $\mathcal{D}_\Sigma$ is closed under reversal, and Theorem 5.11 shows that it is closed under b . Hence $\mathcal{D}_\Sigma$ is closed under e . □

5.73. **Show that redefining final states to merely “midway” states is incorrect.**

Example 5.25 gives a concrete counterexample. For the highlighted state q ,

$$I(A, q) = \{(\mathbf{abc})^n \mid n \geq 1\} \quad \text{and} \quad T(A, q) = \{\mathbf{a}^{2n} \mid n \geq 1\}.$$

So a word can use q as a midpoint only when its first half has a length that occurs in both sets, namely a length divisible by 6. The proof of Theorem 5.11 therefore keeps only

$$I(A, q) \cap \overline{\psi^{-1}}(\overline{\psi}(I(A, q)) \cap \overline{\psi}(T(A, q))) = \{(\mathbf{abc})^{2n} \mid n \geq 1\}.$$

If we simply declared q to be a final state, the resulting automaton would accept all of $I(A, q)$, including $\mathbf{abc}$. But $\mathbf{abc}$ is not a first half of any word in L , because after reaching q there is no continuation of length 3 to a final state. Hence “make the midway states final” accepts too many words and is incorrect.

5.74. Flaw in the argument that K is not FAD because M is not FAD.

Theorem 5.9 states that $\mathfrak{D}_\Sigma$ is closed under homomorphism: if L is FAD and ψ is a homomorphism, then $\psi(L)$ is FAD. This does *not* mean that if L is not FAD, then $\psi(L)$ is not FAD. A homomorphism can collapse the structure that made M non-FAD, yielding a simpler FAD image $K = \psi(M)$. Hence the fact that $M \notin \mathfrak{D}_\Sigma$ says nothing by itself about whether $K = \psi(M)$ is FAD.

5.75. Prove Theorem 5.9 ($\mathfrak{D}_\Sigma$ closed under homomorphism).

Proof. Let $A = \langle \Sigma, S, s_0, \delta, F \rangle$ be a DFA for L , and let $\psi : \Sigma \rightarrow \Sigma^*$ be a homomorphism. For each transition $s \xrightarrow{a} t$ of A , replace that edge by a path labeled $\psi(a)$. If $\psi(a) = \lambda$, use a λ -transition from s to t ; if $\psi(a) = \mathbf{b}_1 \cdots \mathbf{b}_k$ with $k \geq 1$, insert $k - 1$ new states so that the new path reads $\mathbf{b}_1, \mathbf{b}_2, \dots, \mathbf{b}_k$.

The resulting finite automaton accepts exactly the homomorphic images of words in L , namely $\overline{\psi}(L)$. Hence $\overline{\psi}(L) \in \mathfrak{D}_\Sigma$, proving Theorem 5.9. $\square$

5.76. Homomorphism capitalizing lowercase ASCII letters.

Define $\psi : \text{ASCII}^* \rightarrow \text{ASCII}^*$ by $\psi(c) = \begin{cases} \text{uppercase}(c) & \text{if } c \in \{a, \dots, z\} \\ c & \text{otherwise} \end{cases}$ (extended letter-by-letter to words). This is a homomorphism since $\psi(xy) = \psi(x)\psi(y)$ follows from the letter-by-letter definition.

5.77. (a) Prove $L(A') = L_1/L_2$.

$$L_1/L_2 = \{x \mid \exists y \in L_2 \ni xy \in L_1\}. F' = \{t \in S_1 \mid \exists y \in \Sigma^* \ni y \in L_2 \wedge \overline{\delta}_1(t, y) \in F_1\}.$$

Proof. $x \in L(A')$ iff $\overline{\delta}_1(s_0, x) \in F'$ iff $\exists y \in L_2 \ni \overline{\delta}_1(\overline{\delta}_1(s_0, x), y) \in F_1$ iff $\exists y \in L_2 \ni \overline{\delta}_1(s_0, xy) \in F_1$ iff $\exists y \in L_2 \ni xy \in L_1$ iff $x \in L_1/L_2$. $\square$

(b) Algorithm to compute F' .

For each state $t \in S_1$, determine whether $\exists y \in L_2 \ni \overline{\delta}_1(t, y) \in F_1$:

1. Build the product machine $A_1 \times A_2$: state set $S_1 \times S_2$, start at $\langle t, s_{0_2} \rangle$ for each t .
2. In this product, run A_2 on the input and simultaneously advance A_1 .
3. State $t \in F'$ iff some state $\langle f_1, f_2 \rangle$ with $f_1 \in F_1$ and $f_2 \in F_2$ is reachable from $\langle t, s_{0_2} \rangle$ in the product.

This is a reachability computation in a finite graph, computable in $O(|S_1| \cdot |S_2|)$ time.

5.78. Extend DFA over Σ_1 to handle $\Sigma_1 \cup \Sigma_2$.

- (a) Define $A' = \langle \Sigma_1 \cup \Sigma_2, S \cup \{d\}, s_0, \delta', F \rangle$ where d is a new dead state and $\delta'(s, \mathbf{a}) = \begin{cases} \delta(s, \mathbf{a}) & \mathbf{a} \in \Sigma_1 \\ d & \mathbf{a} \in \Sigma_2 \setminus \Sigma_1 \end{cases}$ and $\delta'(d, \mathbf{a}) = d$ for all $\mathbf{a}$. Final states: F (unchanged).
- (b) **Proof.** If $w \in (\Sigma_1 \cup \Sigma_2)^*$ and w contains any letter from $\Sigma_2 \setminus \Sigma_1$, then A' goes to d and stays there, rejecting w . If $w \in \Sigma_1^*$, then A' behaves identically to A . Hence $L(A') = L(A)$ (viewed as a subset of $(\Sigma_1 \cup \Sigma_2)^*$). $\square$

5.79. **If $\mathcal{S}$ is closed under union, concatenation, Kleene closure, and is infinite, must every language in $\mathcal{S}$ be FAD?**

No. The collection I_Σ of all infinite languages is infinite, and Exercise 5.4 shows that it is closed under union, concatenation, and Kleene closure. But I_Σ contains languages in $\mathfrak{N}_\Sigma$, so not every language in such a collection must be FAD.

5.80. **If $\mathcal{S}$ is closed under union, concatenation, Kleene closure, and is finite, must every language be FAD?**

No. Let

$$K = \{x \in \{\mathbf{a}, \mathbf{b}\}^* \mid |x|_{\mathbf{a}} = |x|_{\mathbf{b}}\},$$

which is in $\mathfrak{N}_\Sigma$ by Lemma 5.4, and define

$$\mathcal{S} = \{\emptyset, \{\lambda\}, K\}.$$

This collection is finite. It is closed under union because $\{\lambda\} \subseteq K$, so every union is still one of the three listed languages. It is closed under concatenation because

$$\emptyset \cdot L = \emptyset, \quad \{\lambda\} \cdot L = L \cdot \{\lambda\} = L, \quad K \cdot K = K$$

(the numbers of $\mathbf{a}$ s and $\mathbf{b}$ s add, and $\lambda \in K$). It is also closed under Kleene closure:

$$\emptyset^* = \{\lambda\}, \quad \{\lambda\}^* = \{\lambda\}, \quad K^* = K.$$

So a finite collection with these closure properties need not consist entirely of FAD languages.

5.81. **If $u \circ u = \text{id}$ and $\mathfrak{D}_\Sigma$ is closed under u , then $\mathfrak{N}_\Sigma$ is closed under u .**

Proof. Let $L \in \mathfrak{N}_\Sigma$. Suppose, toward a contradiction, that $u(L) \in \mathfrak{D}_\Sigma$. Because $\mathfrak{D}_\Sigma$ is closed under u , applying u again gives

$$u(u(L)) \in \mathfrak{D}_\Sigma.$$

But $u \circ u = \text{id}$, so $u(u(L)) = L$. Hence $L \in \mathfrak{D}_\Sigma$, contradicting $L \in \mathfrak{N}_\Sigma$. Therefore $u(L) \in \mathfrak{N}_\Sigma$ whenever $L \in \mathfrak{N}_\Sigma$, so $\mathfrak{N}_\Sigma$ is closed under u . $\square$

Chapter 6

Regular Expressions — Solutions

- 6.1. Even-length words over $\{a, b\}$:

$$((a \cup b)(a \cup b))^*$$

- 6.2. Words of length ≥ 2 over $\{a, b\}$:

$$(a \cup b)(a \cup b)(a \cup b)^*$$

- 6.3. Words with $|x|_a = |x|_b$ over $\{a, b\}$: This language is *not* regular (not FAD). By the pumping lemma, if it were FAD with n states, take $w = a^n b^n$; any pump within the first n characters (all a s) would unbalance the counts. Hence no regular expression exists.

- 6.4. Odd-length words over $\{a, b, c\}$ not ending in b :

A word of odd length not ending in b has an even-length prefix (possibly empty) followed by a non- b letter:

$$((a \cup b \cup c)(a \cup b \cup c))^*(a \cup c)$$

(When the Kleene star produces k pairs, the total length is $2k + 1$, which is odd.)

- 6.5. Words over $\{a, b, c\}$ not containing cc : Words with no two consecutive c s are those where every c is followed by a non- c (or is at the end). Let $N = a \cup b$ (non- c letters).

$$(N \cup cN)^*(\epsilon \cup c)$$

The final factor need only be $\epsilon \cup c$, because any terminal non- c letter is already covered by the last block chosen from $N \cup cN$.

- 6.6. Words over $\{a, b, c\}$ containing cc :

$$(a \cup b \cup c)^* cc (a \cup b \cup c)^*$$

- 6.7. Words over $\{a, b, c\}$ with no cs :

$$(a \cup b)^*$$

- 6.8. Let $\Sigma = \{0, 1\}$.

- (a) $L_1 = \{x \mid |x| \bmod 3 = 2\}$: words whose length is congruent to 2 modulo 3:

$$(0 \cup 1)(0 \cup 1)((0 \cup 1)(0 \cup 1)(0 \cup 1))^*$$

- (b) $L_2 = \Sigma^* \setminus \{w \mid w \text{ ends in } 1\}$: words that do *not* end in 1, i.e., words that end in 0 or equal λ :

$$\epsilon \cup (0 \cup 1)^* 0$$

- (c) $L_3 = \{y \mid |y|_0 > |y|_1\}$: this language is *not* regular. If it were regular, then its intersection with the regular language $0^* 1^*$ would also be regular. But

$$L_3 \cap 0^* 1^* = \{0^m 1^n \mid m > n\},$$

and this language is not regular by the pumping lemma: for pumping length p , the word $0^{p+1} 1^p$ belongs to it, but pumping down any nonempty part from the initial 0-block yields a word with at most as many 0s as 1s. Hence no regular expression exists.

6.9. Let $\Sigma = \{a, b, c\}$. Let $\Sigma^* = (a \cup b \cup c)^*$.

- (a) $L_1 = \{x \mid |x|_a \text{ odd} \wedge |x|_b \text{ even}\}$:

Let $C = c^*$, $A = aC$, and $B = bC$. Then the non- c letters must contribute odd a -parity and even b -parity. One exact regular expression is:

$$C(AA \cup BB \cup (AB \cup BA)(AA \cup BB)^*(AB \cup BA))^*(A \cup (AB \cup BA)(AA \cup BB)^* B)$$

- (b) $L_2 = \{y \mid |y|_c \text{ even} \vee |y|_b \text{ odd}\}$:

$$((a \cup b)^* c (a \cup b)^* c)^* (a \cup b)^* \cup ((a \cup c)^* b (a \cup c)^* b)^* (a \cup c)^* b (a \cup c)^*$$

The first expression gives all words with an even number of cs ; the second gives all words with an odd number of bs .

- (c) $L_3 = \{z \mid |z|_a \text{ even}\}$:

$$((b \cup c)^* a (b \cup c)^* a)^* (b \cup c)^*$$

- (d) $L_4 = \{z \mid |z|_c \text{ prime}\}$: This language is not regular. If it were FAD, let p be a pumping length and choose a prime $q > p$. Then $c^q \in L_4$. Write

$$c^q = uvw$$

with $|uv| \leq p$ and $|v| \geq 1$. Since the word is unary, let $|v| = m \geq 1$. Pump with $i = q + 1$. The pumped word has length

$$q + qm = q(1 + m),$$

which is composite. Hence $uv^{q+1}w \notin L_4$, contradicting the pumping lemma. Therefore L_4 has no regular expression.

- (e) $L_5 = \{x \mid abc \text{ is a substring}\}$:

$$(a \cup b \cup c)^* abc (a \cup b \cup c)^*$$

- (f) $L_6 = \{x \mid acaba \text{ is a substring}\}$:

$$(a \cup b \cup c)^* acaba (a \cup b \cup c)^*$$

- (g) $L_7 = \{x \mid |x|_a \equiv 0 \pmod{3}\}$:

$$(b \cup c)^* (a(b \cup c)^* a(b \cup c)^* a(b \cup c)^*)^*$$

6.10. $\Psi = \{x \mid x \text{ begins with } \mathbf{d} \vee x \text{ contains } \mathbf{bb}\}$ over $\{\mathbf{a}, \mathbf{b}, \mathbf{d}\}$:

$$\mathbf{d}(\mathbf{a} \cup \mathbf{b} \cup \mathbf{d})^* \cup (\mathbf{a} \cup \mathbf{b} \cup \mathbf{d})^* \mathbf{bb}(\mathbf{a} \cup \mathbf{b} \cup \mathbf{d})^*$$

6.11. $\Phi = \{x \mid \text{every } \mathbf{b} \text{ in } x \text{ is immediately followed by } \mathbf{c}\}$ over $\{\mathbf{a}, \mathbf{b}, \mathbf{c}\}$: Words with no isolated $\mathbf{b}$ (every $\mathbf{b}$ must be followed immediately by $\mathbf{c}$). Let $S = \mathbf{a} \cup \mathbf{c} \cup \mathbf{bc}$ (safe building blocks):

$$(\mathbf{a} \cup \mathbf{c} \cup \mathbf{bc})^*$$

6.12. Numbers divisible by 3 over $\{0, \dots, 9\}$:

A number is divisible by 3 iff the sum of its digits is divisible by 3. Let

$$D_0 = (\mathbf{0} \cup \mathbf{3} \cup \mathbf{6} \cup \mathbf{9}), \quad D_1 = (\mathbf{1} \cup \mathbf{4} \cup \mathbf{7}), \quad D_2 = (\mathbf{2} \cup \mathbf{5} \cup \mathbf{8}).$$

Using the 3-state DFA that tracks the running digit sum modulo 3, the terminal-set equations are

$$X_0 = \epsilon \cup D_0 X_0 \cup D_1 X_1 \cup D_2 X_2$$

$$X_1 = D_2 X_0 \cup D_0 X_1 \cup D_1 X_2$$

$$X_2 = D_1 X_0 \cup D_2 X_1 \cup D_0 X_2.$$

Applying Theorems 6.2 and 6.1 yields the following exact regular expression for the language:

$$\begin{aligned} X_0 = & ((D_0)^* \cup (D_0)^* D_1 ((D_0 \cup D_2 (D_0)^* D_1))^* D_2 (D_0)^*) \\ & \cup ((D_0)^* D_2 \cup (D_0)^* D_1 ((D_0 \cup D_2 (D_0)^* D_1))^* (D_1 \cup D_2 (D_0)^* D_2)) \\ & \cdot (((D_0 \cup D_1 (D_0)^* D_2) \cup (D_2 \cup D_1 (D_0)^* D_1) ((D_0 \cup D_2 (D_0)^* D_1))^* (D_1 \cup D_2 (D_0)^* D_2)))^* \\ & \cdot (D_1 (D_0)^* \cup (D_2 \cup D_1 (D_0)^* D_1) ((D_0 \cup D_2 (D_0)^* D_1))^* D_2 (D_0)^*). \end{aligned}$$

6.13. $K = \{x \mid x \text{ is divisible by } 5\}$ over decimal digits:

A nonempty decimal string represents a number divisible by 5 iff it ends in $\mathbf{0}$ or $\mathbf{5}$. Using the same convention as Exercise 6.12, where λ represents zero,

$$K = \epsilon \cup (\mathbf{0} \cup \mathbf{1} \cup \dots \cup \mathbf{9})^* (\mathbf{0} \cup \mathbf{5})$$

6.14. **Build NDEFA for $\mathbf{b} \cup \mathbf{a}^* \mathbf{c}$ using exact constructs of Chapter 5.**

We follow the unsimplified Chapter 5 constructions literally, without merging states that happen to be equivalent.

Step 1: Build $\{\mathbf{a}\}$ and apply Theorem 5.5. Let

$$A_{\mathbf{a}} = \langle \Sigma, \{t_0, t_1\}, \{t_0\}, \delta_{\mathbf{a}}, \{t_1\} \rangle$$

with $\delta_{\mathbf{a}}(t_0, \mathbf{a}) = \{t_1\}$ and all other transitions empty. Then Theorem 5.5 gives an NDEFA for $\mathbf{a}^*$:

$$A_{\mathbf{a}^*} = \langle \Sigma, \{t_0, t_1, q_0\}, \{t_0, q_0\}, \delta_*, \{t_1, q_0\} \rangle,$$

where

$$\delta_*(t_0, \mathbf{a}) = \{t_1\}, \quad \delta_*(t_1, \mathbf{a}) = \{t_1\},$$

and every other transition is empty.

Step 2: Concatenate with $\{c\}$ using the pattern of Example 6.6. Let

$$A_c = \langle \Sigma, \{r_0, r_1\}, \{r_0\}, \delta_c, \{r_1\} \rangle$$

with $\delta_c(r_0, c) = \{r_1\}$ and all other transitions empty. Using the exact concatenation construction, form

$$A_{a^*c} = \langle \Sigma, \{t_0, t_1, q_0, r_0, r_1\}, \{t_0, q_0\}, \delta_\bullet, \{r_1\} \rangle,$$

where $\delta_\bullet$ agrees with the old transitions and, because t_1 and q_0 are final in A_{a^*} , they also mimic the transitions out of r_0 . Thus

$$\delta_\bullet(t_0, a) = \{t_1\}, \quad \delta_\bullet(t_1, a) = \{t_1\}, \quad \delta_\bullet(t_1, c) = \{r_1\},$$

$$\delta_\bullet(q_0, c) = \{r_1\}, \quad \delta_\bullet(r_0, c) = \{r_1\},$$

and every other transition is empty. As in Example 6.6, the state r_0 remains disconnected.

Step 3: Union with $\{b\}$ using Theorem 5.2. Let

$$A_b = \langle \Sigma, \{s_0, s_1\}, \{s_0\}, \delta_b, \{s_1\} \rangle$$

with $\delta_b(s_0, b) = \{s_1\}$ and all other transitions empty. Then Theorem 5.2 yields

$$A = \langle \Sigma, \{t_0, t_1, q_0, r_0, r_1, s_0, s_1\}, \{t_0, q_0, s_0\}, \delta, \{r_1, s_1\} \rangle,$$

where δ agrees with $\delta_\bullet$ on $\{t_0, t_1, q_0, r_0, r_1\}$, agrees with δ_b on $\{s_0, s_1\}$, and has no other transitions. This is the required unsimplified NFA for $b \cup a^*c$.

6.15. Inequalities in Lemma 6.1:

- (a) $R_1 \cup \epsilon \neq R_1$: Let $R_1 = a$. Then $R_1 \cup \epsilon = \{\lambda, a\} \neq \{a\} = R_1$.
- (b) $R_1 \cdot R_2 \neq R_2 \cdot R_1$: Let $R_1 = a$, $R_2 = b$. $ab \neq ba$.
- (c) $R_1 \cdot R_1 \neq R_1$: Let $R_1 = a$. $aa \neq a$.
- (d) $R_1 \cup (R_2 \cdot R_3) \neq (R_1 \cup R_2) \cdot (R_1 \cup R_3)$: Let $R_1 = a$, $R_2 = b$, $R_3 = c$. Then

$$R_1 \cup (R_2 \cdot R_3) = \{a, bc\}, \quad (R_1 \cup R_2) \cdot (R_1 \cup R_3) = \{a, b\} \cdot \{a, c\} = \{aa, ac, ba, bc\}.$$

These two sets are not equal.

- (e) $(R_1 \cdot R_2)^* \neq (R_1^* \cdot R_2^*)^*$: Let $R_1 = a$, $R_2 = b$. $(ab)^* = \{\lambda, ab, abab, \dots\}$. $(a^*b^*)^* = (a \cup b)^*$. Not equal.
- (f) $(R_1 \cdot R_2)^* \neq (R_1^* \cup R_2^*)^*$: Same example. $(R_1^* \cup R_2^*)^* = (a^* \cup b^*)^* = (a \cup b)^*$. Not equal to $(ab)^*$.

Examples where equality holds:

- (g) $R_1 \cup \epsilon = R_1$: let $R_1 = a^*$. Then $a^* \cup \epsilon = a^*$.
- (h) $R_1 \cdot R_2 = R_2 \cdot R_1$: let $R_1 = a$ and $R_2 = aa$. Then $R_1 \neq R_2$, neither is ϵ , and both products equal aaa .

- (i) $R_1 \cdot R_1 = R_1$: let $R_1 = \Sigma^*$. Then $\Sigma^* \Sigma^* = \Sigma^*$.
- (j) $R_1 \cup (R_2 \cdot R_3) = (R_1 \cup R_2) \cdot (R_1 \cup R_3)$: let $R_1 = (\mathbf{a} \cup \mathbf{b})^*$, $R_2 = \mathbf{a}$, and $R_3 = \mathbf{b}$. These three sets are distinct, and both sides equal $(\mathbf{a} \cup \mathbf{b})^*$.
- (k) $(R_1 \cdot R_2)^* = (R_1^* \cdot R_2^*)^*$: let $R_1 = \mathbf{a}^*$ and $R_2 = \mathbf{a}\mathbf{a}^*$. Then $R_1 \neq R_2$, neither is ϵ , and both sides equal $\mathbf{a}^*$.
- (l) $(R_1 \cdot R_2)^* = (R_1^* \cup R_2^*)^*$: with the same choice $R_1 = \mathbf{a}^*$ and $R_2 = \mathbf{a}\mathbf{a}^*$, both sides again equal $\mathbf{a}^*$.

6.16. Prove the equalities of Lemma 6.1.

Let $L_i = L(R_i)$. Each identity follows directly from the definitions.

- (a) $R_1 \cup \emptyset = R_1$; $L_1 \cup \emptyset = L_1$.
- (b) $R_1 \cdot \epsilon = R_1 = \epsilon \cdot R_1$: concatenating with $\{\lambda\}$ does not change a word.
- (c) $R_1 \cdot \emptyset = \emptyset = \emptyset \cdot R_1$: there is no second factor to form a concatenation.
- (d) $R_1 \cup R_2 = R_2 \cup R_1$: union is commutative.
- (e) $R_1 \cup R_1 = R_1$: union is idempotent.
- (f) $R_1 \cup (R_2 \cup R_3) = (R_1 \cup R_2) \cup R_3$: union is associative.
- (g) $R_1 \cdot (R_2 \cdot R_3) = (R_1 \cdot R_2) \cdot R_3$: both sides are all words of the form xyz with $x \in L_1$, $y \in L_2$, $z \in L_3$.
- (h) $R_1 \cdot (R_2 \cup R_3) = (R_1 \cdot R_2) \cup (R_1 \cdot R_3)$: a word on the left is xy with $x \in L_1$ and $y \in L_2 \cup L_3$, so $y \in L_2$ or $y \in L_3$.
- (i) $\epsilon^* = \epsilon$: concatenating any number of empty words still gives λ .
- (j) $\emptyset^* = \epsilon$: by definition, $L^* = \bigcup_{n \geq 0} L^n$, and $\emptyset^0 = \epsilon$ while $\emptyset^n = \emptyset$ for $n \geq 1$.
- (k) $(R_1 \cup R_2)^* = (R_1^* \cup R_2^*)^*$: every factor from $L_1 \cup L_2$ already lies in $L_1^* \cup L_2^*$, so the left side is contained in the right; conversely, each factor from L_1^* or L_2^* is itself a finite concatenation of words from $L_1 \cup L_2$, so the right side is contained in the left.
- (l) $(R_1 \cup R_2)^* = (R_1^* \cdot R_2^*)^*$: a block from $L_1^* L_2^*$ is a finite string of words from L_1 followed by a finite string of words from L_2 , hence belongs to $(L_1 \cup L_2)^*$; the reverse inclusion holds because each factor from $L_1 \cup L_2$ can be viewed as a block with either the L_1^* or L_2^* part empty.
- (m) $(R_1^*)^* = R_1^*$: L_1^* is already closed under finite concatenation.
- (n) $R_1^* \cdot R_1^* = R_1^*$: concatenating two finite concatenations of words from L_1 still yields a finite concatenation of words from L_1 , and every word of L_1^* is obtained by taking the second factor to be λ .

6.17. (a) Non-uniqueness of A^*E when $\lambda \in A$.

Let $A = \{\lambda, \mathbf{a}\} = \epsilon \cup \mathbf{a}$ and $E = \{\mathbf{b}\}$. Then $A^*E = \{\mathbf{a}\}^* \{\mathbf{b}\} = \{\mathbf{a}^n \mathbf{b} \mid n \geq 0\}$. But $X = \Sigma^*$ also satisfies $X = A \cdot X \cup E$: indeed,

$$A \cdot \Sigma^* \cup \{\mathbf{b}\} = (\{\lambda, \mathbf{a}\} \cdot \Sigma^*) \cup \{\mathbf{b}\} = \Sigma^* \cup \{\mathbf{b}\} = \Sigma^*.$$

So Σ^* is also a solution, but $\Sigma^* \neq A^*E$. Hence the solution is not unique.

(b) A^*E as unique solution even if $\lambda \in A$.

Let the alphabet be $\{a\}$, let $A = \{\lambda, a\}$, and let $E = a^*$. Then

$$A^*E = (\epsilon \cup a)^* a^* = a^*.$$

The equation becomes

$$X = E \cup A \cdot X = a^* \cup (\epsilon \cup a)X.$$

Any solution must contain $E = a^*$, but a^* is already the whole universe over this alphabet. Therefore the only possible solution is $X = a^* = A^*E$.

6.18. Solve the system of Exercise 6.18.

The equations are

$$X_0 = (0 \cup 1)X_1, \quad X_1 = \epsilon \cup 1X_0 \cup 0X_1.$$

Substitute the first equation into the second:

$$X_1 = \epsilon \cup 1(0 \cup 1)X_1 \cup 0X_1 = \epsilon \cup (10 \cup 11 \cup 0)X_1.$$

Since $\lambda \notin (10 \cup 11 \cup 0)$, Theorem 6.1 gives

$$X_1 = (10 \cup 11 \cup 0)^*.$$

Therefore

$$X_0 = (0 \cup 1)X_1 = (0 \cup 1)(10 \cup 11 \cup 0)^*.$$

These are the terminal sets for the corresponding two-state DFA. In fact, this is the same state-equation system one obtains from the DFA in Example 3.4, where X_0 and X_1 are the two terminal sets.

6.19. Solve for X_1, X_2, X_3 from the equations.

Simplifying (noting $\emptyset \cdot X_i = \emptyset$):

$$X_1 = (0 \cup 1)X_2$$

$$X_2 = \epsilon \cup 0X_1 \cup 1X_2$$

$$X_3 = (0 \cup 1)X_2$$

Note $X_3 = X_1$.

(a) **Eliminate X_3 , then X_2 :**

Since $X_3 = X_1$, eliminate X_3 . From $X_2 = \epsilon \cup 0X_1 \cup 1X_2$: by Theorem 6.1 ($\lambda \notin \{1\}$): $X_2 = 1^*(\epsilon \cup 0X_1) = 1^* \cup 1^*0X_1$. Substitute into $X_1 = (0 \cup 1)X_2 = (0 \cup 1)(1^* \cup 1^*0X_1) = (0 \cup 1)1^* \cup (0 \cup 1)1^*0X_1$. By Theorem 6.1: $X_1 = ((0 \cup 1)1^*0)^*(0 \cup 1)1^*$. Then

$$X_2 = 1^* \cup 1^*0((0 \cup 1)1^*0)^*(0 \cup 1)1^*, \quad X_3 = X_1.$$

(b) **Eliminate X_3 , then X_1 :**

Same $X_3 = X_1$. From $X_1 = (0 \cup 1)X_2$, substitute into X_2 : $X_2 = \epsilon \cup 0(0 \cup 1)X_2 \cup 1X_2 = \epsilon \cup (0(0 \cup 1) \cup 1)X_2$. Let $A = 0(0 \cup 1) \cup 1 = 00 \cup 01 \cup 1$; $\lambda \notin A$. $X_2 = A^*$. Then $X_1 = (0 \cup 1)A^*$ and $X_3 = X_1$.

- (c) **Comparison:** Both solutions are equivalent: $(0 \cup 1)1^*(0(0 \cup 1)1^*)^* = (0 \cup 1)(00 \cup 01 \cup 1)^*$. The second form is more concise.

6.20. **Prove Lemma 6.2: every regular expression describes a FAD language.**

Proof. Let $P(m)$: “Every regular expression with $\leq m$ operators represents a FAD language.”

Base ($m = 0$): No operators: the expression is $\emptyset$, ϵ , or a single letter $a \in \Sigma$. These represent $\emptyset$, $\{\lambda\}$, and $\{a\}$ respectively—all finite, hence FAD.

Inductive step: Assume $P(m)$. Any regular expression with $m + 1$ operators is either $R_1 \cup R_2$, $R_1 \cdot R_2$, or R_1^* , where R_1 and R_2 each have $\leq m$ operators. By $P(m)$, $L(R_1), L(R_2) \in \mathfrak{D}_\Sigma$. Then:

- $L(R_1 \cup R_2) = L(R_1) \cup L(R_2) \in \mathfrak{D}_\Sigma$ (Theorem 5.2).
- $L(R_1 \cdot R_2) = L(R_1) \cdot L(R_2) \in \mathfrak{D}_\Sigma$ (Theorem 5.4).
- $L(R_1^*) = L(R_1)^* \in \mathfrak{D}_\Sigma$ (Theorem 5.5).

So $P(m + 1)$ holds. By induction, every regular expression represents a FAD language. □ □

6.21. **All solutions to $X = X \cup \{b\}$ over $\{a, b, c\}^*$.**

$X = X \cup \{b\}$ means exactly that $\{b\} \subseteq X$. Hence the solutions are precisely the languages X with $b \in X$. Every such language is a solution, and the smallest one is $\{b\}$.

6.22. **Prove $A^*E = E \cup A \cdot (A^*E)$ for any languages A and E .**

Proof. $A^*E = (\epsilon \cup A \cup A^2 \cup \dots)E = \epsilon E \cup AE \cup A^2E \cup \dots = E \cup A(E \cup AE \cup A^2E \cup \dots) = E \cup A(A^*E)$. □ □

6.23. **Regular expression for $(ab \cup b)^*a \cap (ba \cup a)^*$.**

Both are FAD; their intersection is FAD (Lemma 5.1). We use the product DFA approach.

$L_1 = (ab \cup b)^*a$: strings over $\{a, b\}$ of the form wa where $w \in (ab \cup b)^*$. $L_2 = (ba \cup a)^*$: strings over $\{a, b\}$ that are concatenations of ba or a .

L_1 ends in a . $L_2 = (ba \cup a)^*$ consists of all strings that decompose as concatenations of a and ba .

$L_2 = (ba \cup a)^*$ means every b is immediately followed by a . A word in $L_1 = (ab \cup b)^*a$ has prefix in $(ab \cup b)^*$, so every a in the prefix is immediately followed by b . Combining both constraints on the prefix atoms: each atom (whether ab or b) must be immediately followed by something starting with a (since the atom ends in b , and every b must be followed by a). Consequently, b -atoms can appear only once, at the start (any b -atom followed by another b -atom would put b before b , violating L_2); and ab -atoms must be followed by ab -atoms or the final a . The intersection is therefore:

$$L_1 \cap L_2 = (\epsilon \cup b)(ab)^*a$$

Verification: $(\epsilon \cup b)(ab)^k a$ gives words $a, aba, ababa, \dots$ and $ba, baba, bababa, \dots$, all of which belong to both L_1 and L_2 ; and any word in $L_1 \cap L_2$ has this form by the above analysis.

6.24. **Algorithm for intersection of two regular expressions.**

Given regular expressions R_1 and R_2 (each describing a FAD language):

1. Convert R_1 to an NDFA A_1 using the constructions of Chapter 5.
2. Convert R_2 to an NDFA A_2 so that $L(A_2) = L(R_2)$.
3. Apply the subset construction to get DFAs A_1^d and A_2^d .
4. Build the product DFA $A^\cap$ (Lemma 5.1) with $F^\cap = F_1 \times F_2$.
5. Use the state-equation method of Theorems 6.3 and 6.2 to convert $A^\cap$ to a regular expression for $L(A_1) \cap L(A_2)$.

6.25. **Verify that $X_1 = (a \cup bb)^*$ and $X_2 = b \cdot (a \cup bb)^*$ satisfy:**

$$X_1 = \epsilon \cup a \cdot X_1 \cup b \cdot X_2$$

$$X_2 = b \cdot X_1$$

Verification of $X_2 = b \cdot X_1$:

$$b \cdot X_1 = b \cdot (a \cup bb)^* = X_2. \quad \checkmark$$

Verification of $X_1 = \epsilon \cup a \cdot X_1 \cup b \cdot X_2$:

$$\begin{aligned} \epsilon \cup a \cdot X_1 \cup b \cdot X_2 &= \epsilon \cup a(a \cup bb)^* \cup b \cdot b(a \cup bb)^* \\ &= \epsilon \cup (a \cup bb) \cdot (a \cup bb)^* \\ &= (a \cup bb)^* = X_1. \quad \checkmark \end{aligned}$$

- 6.26. (a) **$L(D)$ for Example 6.10:** Example 6.10 gives $L(B) = X_1 = (a \cup bb)^*$ and $L(C) = X_2 = b(a \cup bb)^*$. Since D has both corresponding start states, its language is

$$L(D) = X_1 \cup X_2 = (\epsilon \cup b)(a \cup bb)^*.$$

- (b) **General formula:** For a machine A with start states $s_{i_1}, \dots, s_{i_m}$:

$$L(A) = X_{i_1} \cup X_{i_2} \cup \dots \cup X_{i_m}$$

where each X_j is the solution to the state equation for state s_j .

6.27. **Regular expression for the complement of $(ab \cup b)^* a$.**

The language $(ab \cup b)^* a$ is exactly the set of words over $\{a, b\}$ that end in a and contain no aa as a substring. Therefore its complement is the set of words that either end in b (or are λ) or contain aa . One regular expression is

$$\epsilon \cup (a \cup b)^* b \cup (a \cup b)^* aa(a \cup b)^*.$$

6.28. **Algorithm to complement a regular expression.**

Given regular expression R :

1. Convert R to a DFA A (via NDFA $\rightarrow$ subset construction).
2. Complement A by swapping F and $S \setminus F$.
3. Use the state-equation method of Theorems 6.3 and 6.2 to convert the complemented DFA back to a regular expression.

6.29. $\mathfrak{R}_\Sigma$ is closed under $E(L)$ using regular expressions.

Here

$$E(L) = \{z \mid (\exists y \in \Sigma^+) (\exists x \in L) z = yx\} = \Sigma^+ \cdot L.$$

If $L = L(R)$ and $\Sigma = \{\mathbf{a}, \mathbf{b}, \mathbf{c}\}$, then

$$E(L) = L\left((\mathbf{a} \cup \mathbf{b} \cup \mathbf{c})(\mathbf{a} \cup \mathbf{b} \cup \mathbf{c})^* R\right),$$

which is again a regular expression. Hence $\mathfrak{R}_\Sigma$ is closed under E .

6.30. $\mathfrak{R}_\Sigma$ is closed under $B(L)$.

$B(L) = L \cdot \Sigma^*$ (words starting with a word from L). If $L = L(R)$, then $B(L) = L(R \cdot \Sigma^*) = L(R \cdot (\mathbf{a} \cup \mathbf{b} \cup \mathbf{c})^*)$. This is a regular expression, so $B(L) \in \mathfrak{R}_\Sigma$.

6.31. $\mathfrak{R}_\Sigma$ is closed under $M(L)$.

$M(L) = L \cdot \Sigma^+$. If $L = L(R)$, then $M(L) = L(R \cdot (\mathbf{a} \cup \mathbf{b} \cup \mathbf{c})^+) = L(R \cdot (\mathbf{a} \cup \mathbf{b} \cup \mathbf{c}) \cdot (\mathbf{a} \cup \mathbf{b} \cup \mathbf{c})^*)$. Regular expression, hence $M(L) \in \mathfrak{R}_\Sigma$.

6.32. (a) **No unique solution to the given system:**

The equations: $X_1 = \mathbf{b} \cup \epsilon \cdot X_1 \cup \mathbf{a} \cdot X_2 = \mathbf{b} \cup X_1 \cup \mathbf{a}X_2$. Since $X_1 \subseteq X_1 \cup \mathbf{b} \cup \mathbf{a}X_2$, the equation $X_1 = \mathbf{b} \cup X_1 \cup \mathbf{a}X_2$ says $X_1 \supseteq \mathbf{b} \cup \mathbf{a}X_2$. Any X_1 that contains $\mathbf{b}$ and $\mathbf{a}X_2$ is a solution; there are infinitely many such X_1 .

$X_2 = \mathbf{c} \cup \emptyset \cdot X_1 \cup \epsilon \cdot X_2 = \mathbf{c} \cup X_2$. Thus any $X_2 \supseteq \{\mathbf{c}\}$ is a solution.

(b) **No contradiction with Theorem 6.2:** Theorem 6.2 assumes $\lambda \notin A_{ij}$ for every coefficient in the system. Here $A_{11} = \epsilon$ and $A_{22} = \epsilon$, so the hypothesis fails. Therefore Theorem 6.2 does not apply, and there is no contradiction.

6.33. **Solve for X_0 and X_1 .**

$$X_0 = \mathbf{0}^* \mathbf{1} \cup (\mathbf{10})^* X_0 \cup \mathbf{0}(\mathbf{0} \cup \mathbf{1})X_1$$

$$X_1 = \epsilon \cup \mathbf{1}^* \mathbf{01} X_0 \cup \mathbf{0} X_1$$

From the X_1 equation (by Theorem 6.1, $\lambda \notin \{\mathbf{0}\}$): $X_1 = \mathbf{0}^* (\epsilon \cup \mathbf{1}^* \mathbf{01} X_0) = \mathbf{0}^* \cup \mathbf{0}^* \mathbf{1}^* \mathbf{01} X_0$.

Substitute into X_0 : $X_0 = \mathbf{0}^* \mathbf{1} \cup (\mathbf{10})^* X_0 \cup \mathbf{0}(\mathbf{0} \cup \mathbf{1})(\mathbf{0}^* \cup \mathbf{0}^* \mathbf{1}^* \mathbf{01} X_0)$.

Let $C = \mathbf{0}(\mathbf{0} \cup \mathbf{1})\mathbf{0}^* \mathbf{1}^* \mathbf{01}$. Then: $X_0 = ((\mathbf{10})^* \cup C)X_0 \cup (\mathbf{0}^* \mathbf{1} \cup \mathbf{0}(\mathbf{0} \cup \mathbf{1})\mathbf{0}^*)$.

Since $(\mathbf{10})^*$ contains λ , Theorem 6.1 does not guarantee a unique solution here. Following the chapter's discussion of ambiguous equations, the least solution is

$$X_0 = ((\mathbf{10})^* \cup C)^* (\mathbf{0}^* \mathbf{1} \cup \mathbf{0}(\mathbf{0} \cup \mathbf{1})\mathbf{0}^*).$$

Back-substitution then gives

$$X_1 = \mathbf{0}^* \cup \mathbf{0}^* \mathbf{1}^* \mathbf{01} ((\mathbf{10})^* \cup C)^* (\mathbf{0}^* \mathbf{1} \cup \mathbf{0}(\mathbf{0} \cup \mathbf{1})\mathbf{0}^*).$$

- 6.34. (a) Words ending with **c** with no **aa**, **bb**, **cc** substrings:

Build the DFA with states q_0 (start), q_a (last letter **a**), q_b (last letter **b**), q_c (last letter **c**, accepting), and dead state q_d ; transitions $q_x \xrightarrow{x} q_d$ and $q_x \xrightarrow{y} q_y$ for $y \neq x$. Write state equations for strings reaching q_c :

$$\begin{aligned} X_a &= bX_b \cup cX_c \\ X_b &= aX_a \cup cX_c \\ X_c &= \epsilon \cup aX_a \cup bX_b \end{aligned}$$

Eliminate X_a : substituting $X_b = aX_a \cup cX_c$ into X_a gives $X_a = baX_a \cup (bc \cup c)X_c$, so $X_a = (ba)^*(b \cup \epsilon)cX_c$. Then $X_b = a(ba)^*(b \cup \epsilon)cX_c \cup cX_c$. Substituting into X_c :

$$X_c = \epsilon \cup ((a \cup ba)(ba)^*(b \cup \epsilon)c \cup bc)X_c$$

Since λ is absent from the coefficient, $X_c = A^*$ where

$$A = (a \cup ba)(ba)^*(b \cup \epsilon)c \cup bc.$$

The language is $L = aX_a \cup bX_b \cup cX_c = (a(ba)^*(b \cup \epsilon)c \cup b(a(ba)^*(b \cup \epsilon)c \cup c) \cup c)A^*$.

- (b) Words beginning with **c** with no **aa**, **bb**, **cc** substrings: Let X_c be the set of suffixes that may follow an initial **c** while preserving the “no equal consecutive letters” condition. Solving the corresponding state equations gives

$$X_c = (((a \cup ba)(ba)^*(\epsilon \cup b) \cup b)c)^*(\epsilon \cup a)(ba)^*(\epsilon \cup b).$$

Therefore one regular expression for the language is

$$c(((a \cup ba)(ba)^*(\epsilon \cup b) \cup b)c)^*(\epsilon \cup a)(ba)^*(\epsilon \cup b).$$

- 6.35. (a) Words with at most two **cs**:

$$(a \cup b)^* \cup (a \cup b)^*c(a \cup b)^* \cup (a \cup b)^*c(a \cup b)^*c(a \cup b)^*$$

- (b) Words that do *not* have exactly one **c** (i.e., zero or ≥ 2 **cs**):

$$(a \cup b)^* \cup (a \cup b)^*c(a \cup b)^*c(a \cup b \cup c)^*$$

6.36. Reversal of regular expressions.

- (a) Equivalences:

- $(R_1 \cup R_2)^r = R_1^r \cup R_2^r$
- $(R_1 \cdot R_2)^r = R_2^r \cdot R_1^r$
- $(R^*)^r = (R^r)^*$
- $\emptyset^r = \emptyset$
- $\epsilon^r = \epsilon$
- $a^r = a$ for each $a \in \Sigma$

(b) To generate a regular expression for L^r from R : apply the reversal operator recursively to R using the rules above.

(c) **Proof by induction on the number of operators m in R :**

Base ($m = 0$): $R \in \{\emptyset, \epsilon, \mathbf{a}\}$. $L(R)^r = L(R^r)$ by direct check.

Step: Assume the result for all expressions with $< m$ operators. If $R = R_1 \cup R_2$, then

$$(L(R_1) \cup L(R_2))^r = L(R_1)^r \cup L(R_2)^r = L(R_1^r) \cup L(R_2^r) = L(R_1^r \cup R_2^r) = L(R^r).$$

If $R = R_1 \cdot R_2$, then

$$L(R)^r = (L(R_1)L(R_2))^r = L(R_2)^r L(R_1)^r = L(R_2^r)L(R_1^r) = L(R_2^r \cdot R_1^r) = L(R^r).$$

If $R = R_1^*$, then

$$L(R)^r = (L(R_1)^*)^r = (L(R_1)^r)^* = (L(R_1^r))^* = L((R_1^r)^*) = L(R^r).$$

Thus the claim holds in all three operator cases. $\square$

(d) Since every FAD language has a regular expression (Corollary 6.2), and the reversal of any regular expression (via the rules above) is also a regular expression, $\mathfrak{R}_\Sigma$ is closed under r .

6.37. Complete details of Theorem 6.4.

Theorem 6.4 is the left-linear analog of Theorem 6.2.

Proof. Focus on the m th equation:

$$Y_m = I_m \cup Y_1 B_{m1} \cup \cdots \cup Y_{m-1} B_{m(m-1)} \cup Y_m B_{mm} \cup Y_{m+1} B_{m(m+1)} \cup \cdots \cup Y_n B_{mn}.$$

Treating every term except $Y_m B_{mm}$ as a constant, the mirror-image of the argument in Theorem 6.1 gives

$$Y_m = (I_m \cup Y_1 B_{m1} \cup \cdots \cup Y_{m-1} B_{m(m-1)} \cup Y_{m+1} B_{m(m+1)} \cup \cdots \cup Y_n B_{mn}) B_{mm}^*.$$

This is part (b).

Substitute this expression for Y_m into each remaining equation

$$Y_k = I_k \cup Y_1 B_{k1} \cup \cdots \cup Y_m B_{km} \cup \cdots \cup Y_n B_{kn} \quad (k \neq m),$$

and distribute B_{km} across the substituted expression. Collecting the constant term and the coefficients of each Y_j yields

$$\hat{I}_k = I_k \cup (I_m B_{mm}^* B_{km})$$

and

$$\hat{B}_{kj} = B_{kj} \cup (B_{mj} B_{mm}^* B_{km}) \quad (j \neq m),$$

which is exactly the reduced system in part (a).

For a single left-linear equation

$$Y_1 = I_1 \cup Y_1 B_{11},$$

the same mirror-image argument gives the unique solution

$$Y_1 = I_1 B_{11}^*.$$

This is part (c). Hence Theorem 6.4 follows. $\square$

$\square$

- 6.38. (a) Words with no **b** immediately preceded by **a** over $\{a, b, c\}$:

This means exactly that **ab** never occurs. Every maximal block of **as** must therefore be followed by **c** or occur at the end of the word. Hence one regular expression is

$$(b \cup c \cup aa^*c)^*(\epsilon \cup aa^*).$$

The right-hand term simplifies, since

$$\epsilon \cup aa^* = a^*.$$

So a shorter equivalent expression is

$$(b \cup c \cup aa^*c)^*a^*.$$

- (b) Words with exactly two **cs** and no **ab**:

Over $\{a, b\}$, avoiding **ab** means that once an **a** appears, only **as** may follow; hence the **a/b** portions are exactly b^*a^* . Therefore the desired language is

$$b^*a^*cb^*a^*cb^*a^*.$$

- 6.39. (a) Words with no **b** immediately preceded by **c** (no **cb** substring):

This means exactly that every maximal block of **cs** is followed by **a** or occurs at the end of the word. Hence one regular expression is

$$(a \cup b \cup cc^*a)^*(\epsilon \cup cc^*)$$

Since $\epsilon \cup cc^* = c^*$, this may also be written more compactly as

$$(a \cup b \cup cc^*a)^*c^*.$$

- (b) Words with exactly one **c** and no **cb**:

Exactly one **c**, and that **c** is not immediately followed by **b**, means that the lone **c** is followed by either **a** and then any string over $\{a, b\}$, or by the end of the word:

$$(a \cup b)^* \cdot c \cdot ((a(a \cup b)^*) \cup \epsilon)$$

- 6.40. (a) For Figure 6.11 the right-linear equations are

$$X_1 = \emptyset \cup aX_1 \cup aX_2,$$

$$X_2 = \epsilon \cup (a \cup b)X_1.$$

- (b) In the notation of Theorem 6.2,

$$E_1 = \emptyset, \quad E_2 = \epsilon, \quad A_{11} = a, \quad A_{12} = a, \quad A_{21} = a \cup b, \quad A_{22} = \emptyset.$$

Eliminating X_2 gives the one-variable equation

$$X_1 = \hat{E}_1 \cup \hat{A}_{11}X_1,$$

where

$$\hat{E}_1 = E_1 \cup A_{12}A_{22}^*E_2 = \emptyset \cup \mathbf{a}\emptyset^*\epsilon = \mathbf{a},$$

and

$$\hat{A}_{11} = A_{11} \cup A_{12}A_{22}^*A_{21} = \mathbf{a} \cup \mathbf{a}\emptyset^*(\mathbf{a} \cup \mathbf{b}) = \mathbf{a} \cup \mathbf{aa} \cup \mathbf{ab}.$$

By Theorem 6.1,

$$X_1 = (\mathbf{a} \cup \mathbf{aa} \cup \mathbf{ab})^* \mathbf{a}.$$

Back-substitution, equivalently Theorem 6.2(c), gives

$$X_2 = \emptyset^*(\epsilon \cup (\mathbf{a} \cup \mathbf{b})X_1) = \epsilon \cup (\mathbf{a} \cup \mathbf{b})(\mathbf{a} \cup \mathbf{aa} \cup \mathbf{ab})^* \mathbf{a}.$$

The same result follows by direct substitution. Substitute the second equation into the first:

$$X_1 = \mathbf{a}X_1 \cup \mathbf{a}(\epsilon \cup (\mathbf{a} \cup \mathbf{b})X_1) = \mathbf{a} \cup (\mathbf{a} \cup \mathbf{aa} \cup \mathbf{ab})X_1.$$

Hence

$$X_1 = (\mathbf{a} \cup \mathbf{aa} \cup \mathbf{ab})^* \mathbf{a}, \quad X_2 = \epsilon \cup (\mathbf{a} \cup \mathbf{b})(\mathbf{a} \cup \mathbf{aa} \cup \mathbf{ab})^* \mathbf{a}.$$

(c) Therefore the NDFA accepts

$$L = (\mathbf{a} \cup \mathbf{aa} \cup \mathbf{ab})^* \mathbf{a}.$$

(d) The left-linear equations are

$$Y_1 = \epsilon \cup Y_1 \mathbf{a} \cup Y_2(\mathbf{a} \cup \mathbf{b}),$$

$$Y_2 = Y_1 \mathbf{a}.$$

Substituting gives

$$Y_1 = \epsilon \cup Y_1(\mathbf{a} \cup \mathbf{aa} \cup \mathbf{ab}),$$

so

$$Y_1 = (\mathbf{a} \cup \mathbf{aa} \cup \mathbf{ab})^*, \quad Y_2 = (\mathbf{a} \cup \mathbf{aa} \cup \mathbf{ab})^* \mathbf{a}.$$

6.41. (a) For Figure 6.12 the right-linear equations are

$$X_0 = \mathbf{a}X_1,$$

$$X_1 = \mathbf{a}X_1 \cup \mathbf{a}X_2,$$

$$X_2 = \epsilon \cup \mathbf{b}X_2 \cup \mathbf{a}X_3,$$

$$X_3 = \mathbf{b}X_0 \cup (\mathbf{a} \cup \mathbf{b})X_3.$$

(b) First apply Theorem 6.2 to eliminate X_3 . The last equation has $E_3 = \emptyset$, $A_{30} = \mathbf{b}$, and $A_{33} = \mathbf{a} \cup \mathbf{b}$, so

$$X_3 = (\mathbf{a} \cup \mathbf{b})^* \mathbf{b}X_0,$$

and the X_2 equation becomes

$$X_2 = \mathbf{b}^*(\epsilon \cup \mathbf{a}X_3) = \mathbf{b}^* \cup \mathbf{b}^* \mathbf{a}(\mathbf{a} \cup \mathbf{b})^* \mathbf{b}X_0,$$

after applying Theorem 6.1 to the coefficient $\mathbf{b}$ of X_2 .

Substituting this expression for X_2 leaves the pair

$$\begin{aligned} X_0 &= \mathbf{a}X_1, \\ X_1 &= \mathbf{ab}^* \cup \mathbf{ab}^* \mathbf{a}(\mathbf{a} \cup \mathbf{b})^* \mathbf{b}X_0 \cup \mathbf{a}X_1. \end{aligned}$$

For this pair,

$$E_0 = \emptyset, \quad E_1 = \mathbf{ab}^*, \quad A_{00} = \emptyset, \quad A_{01} = \mathbf{a}, \quad A_{10} = \mathbf{ab}^* \mathbf{a}(\mathbf{a} \cup \mathbf{b})^* \mathbf{b}, \quad A_{11} = \mathbf{a}.$$

Theorem 6.2 therefore reduces the pair to

$$X_0 = \hat{E}_0 \cup \hat{A}_{00}X_0,$$

where

$$\hat{E}_0 = E_0 \cup A_{01}A_{11}^*E_1 = \mathbf{aa}^*\mathbf{ab}^*$$

and

$$\hat{A}_{00} = A_{00} \cup A_{01}A_{11}^*A_{10} = \mathbf{aa}^*\mathbf{ab}^*\mathbf{a}(\mathbf{a} \cup \mathbf{b})^*\mathbf{b}.$$

By Theorem 6.1,

$$X_0 = (\mathbf{aa}^*\mathbf{ab}^*\mathbf{a}(\mathbf{a} \cup \mathbf{b})^*\mathbf{b})^* \mathbf{aa}^*\mathbf{ab}^*.$$

The remaining unknowns are obtained by back-substitution:

$$X_1 = \mathbf{a}^*(\mathbf{ab}^* \cup \mathbf{ab}^* \mathbf{a}(\mathbf{a} \cup \mathbf{b})^* \mathbf{b}X_0),$$

$$X_2 = \mathbf{b}^*(\epsilon \cup \mathbf{a}(\mathbf{a} \cup \mathbf{b})^* \mathbf{b}X_0), \quad X_3 = (\mathbf{a} \cup \mathbf{b})^* \mathbf{b}X_0.$$

The direct substitution method gives the same equation for X_0 :

$$X_1 = \mathbf{a}^* \mathbf{a}X_2, \quad X_0 = \mathbf{aa}^* \mathbf{a}X_2.$$

Therefore

$$X_0 = \mathbf{aa}^*\mathbf{ab}^* \cup \mathbf{aa}^*\mathbf{ab}^*\mathbf{a}(\mathbf{a} \cup \mathbf{b})^*\mathbf{b}X_0,$$

so

$$X_0 = (\mathbf{aa}^*\mathbf{ab}^*\mathbf{a}(\mathbf{a} \cup \mathbf{b})^*\mathbf{b})^* \mathbf{aa}^*\mathbf{ab}^*.$$

(c) Hence the language of the automaton is

$$L = (\mathbf{aa}^*\mathbf{ab}^*\mathbf{a}(\mathbf{a} \cup \mathbf{b})^*\mathbf{b})^* \mathbf{aa}^*\mathbf{ab}^*.$$

(d) The left-linear equations are

$$\begin{aligned} Y_0 &= \epsilon \cup Y_3\mathbf{b}, \\ Y_1 &= Y_0\mathbf{a} \cup Y_1\mathbf{a}, \\ Y_2 &= Y_1\mathbf{a} \cup Y_2\mathbf{b}, \\ Y_3 &= Y_2\mathbf{a} \cup Y_3(\mathbf{a} \cup \mathbf{b}). \end{aligned}$$

Solving gives

$$\begin{aligned} Y_0 &= (\mathbf{aa}^*\mathbf{ab}^*\mathbf{a}(\mathbf{a} \cup \mathbf{b})^*\mathbf{b})^*, \\ Y_1 &= Y_0\mathbf{aa}^*, \quad Y_2 = Y_0\mathbf{aa}^*\mathbf{ab}^*, \quad Y_3 = Y_0\mathbf{aa}^*\mathbf{ab}^*\mathbf{a}(\mathbf{a} \cup \mathbf{b})^*. \end{aligned}$$

6.42. (a) For Figure 6.13 the right-linear equations are

$$\begin{aligned} X_0 &= \mathbf{0}X_0 \cup \mathbf{1}X_0 \cup \mathbf{0}X_1, \\ X_1 &= \mathbf{1}X_2, \\ X_2 &= \mathbf{0}X_3, \\ X_3 &= \mathbf{0}X_4, \\ X_4 &= \mathbf{1}X_5, \\ X_5 &= \mathbf{0}X_6, \\ X_6 &= \epsilon \cup \mathbf{0}X_6 \cup \mathbf{1}X_6. \end{aligned}$$

(b) These equations collapse immediately:

$$\begin{aligned} X_6 &= (\mathbf{0} \cup \mathbf{1})^*, \quad X_5 = \mathbf{0}(\mathbf{0} \cup \mathbf{1})^*, \quad X_4 = \mathbf{10}(\mathbf{0} \cup \mathbf{1})^*, \\ X_3 &= \mathbf{010}(\mathbf{0} \cup \mathbf{1})^*, \quad X_2 = \mathbf{0010}(\mathbf{0} \cup \mathbf{1})^*, \quad X_1 = \mathbf{10010}(\mathbf{0} \cup \mathbf{1})^*. \end{aligned}$$

Hence

$$X_0 = (\mathbf{0} \cup \mathbf{1})X_0 \cup \mathbf{010010}(\mathbf{0} \cup \mathbf{1})^*,$$

so

$$X_0 = (\mathbf{0} \cup \mathbf{1})^* \mathbf{010010}(\mathbf{0} \cup \mathbf{1})^*.$$

(c) Therefore the language is all binary strings containing **010010** as a substring:

$$L = (\mathbf{0} \cup \mathbf{1})^* \mathbf{010010}(\mathbf{0} \cup \mathbf{1})^*.$$

(d) The left-linear equations are

$$\begin{aligned} Y_0 &= \epsilon \cup Y_0\mathbf{0} \cup Y_0\mathbf{1}, \\ Y_1 &= Y_0\mathbf{0}, \\ Y_2 &= Y_1\mathbf{1}, \\ Y_3 &= Y_2\mathbf{0}, \\ Y_4 &= Y_3\mathbf{0}, \\ Y_5 &= Y_4\mathbf{1}, \\ Y_6 &= Y_5\mathbf{0} \cup Y_6\mathbf{0} \cup Y_6\mathbf{1}. \end{aligned}$$

Solving gives

$$\begin{aligned} Y_0 &= (\mathbf{0} \cup \mathbf{1})^*, \quad Y_1 = (\mathbf{0} \cup \mathbf{1})^*\mathbf{0}, \quad Y_2 = (\mathbf{0} \cup \mathbf{1})^*\mathbf{01}, \\ Y_3 &= (\mathbf{0} \cup \mathbf{1})^*\mathbf{010}, \quad Y_4 = (\mathbf{0} \cup \mathbf{1})^*\mathbf{0100}, \quad Y_5 = (\mathbf{0} \cup \mathbf{1})^*\mathbf{01001}, \end{aligned}$$

and

$$Y_6 = (\mathbf{0} \cup \mathbf{1})^* \mathbf{010010}(\mathbf{0} \cup \mathbf{1})^*.$$

6.43. **Prove: for any A, E, Y , if $E \subseteq Y$ then $A \cdot E \subseteq A \cdot Y$.**

Proof. Let $w \in A \cdot E$. Then $w = xy$ with $x \in A$ and $y \in E$. Since $E \subseteq Y$, $y \in Y$. Hence $w = xy \in A \cdot Y$. □

6.44. (a) **Image of $\mathfrak{N}_\Sigma$ under substitution $\bar{s}$ is in $\mathfrak{N}_\Gamma$.**

Proof. Interpreting s as a regular set substitution in the sense of Definition 6.5, we prove directly by structural induction on a regular expression R that $\bar{s}(L(R))$ is regular over Γ . For a regular expression R over Σ , define $\bar{s}(R)$ by: $\bar{s}(\mathbf{a}) = s(\mathbf{a}) \in \mathfrak{N}_\Gamma$ (by assumption), $\bar{s}(\emptyset) = \emptyset$, $\bar{s}(\epsilon) = \{\lambda\}$, $\bar{s}(R_1 \cup R_2) = \bar{s}(R_1) \cup \bar{s}(R_2) \in \mathfrak{N}_\Gamma$ (closed under union), $\bar{s}(R_1 \cdot R_2) = \bar{s}(R_1) \cdot \bar{s}(R_2) \in \mathfrak{N}_\Gamma$ (closed under concatenation), $\bar{s}(R^*) = \bar{s}(R)^* \in \mathfrak{N}_\Gamma$ (closed under Kleene closure). By induction on the structure of R , $\bar{s}(R) \in \mathfrak{N}_\Gamma$. $\square$

(b) **Image of $\mathfrak{N}_\Sigma$ under $\bar{s}$ need not be in $\mathfrak{N}_\Gamma$.**

Let

$$L = \{x \in \{\mathbf{a}, \mathbf{b}\}^* \mid |x|_{\mathbf{a}} = |x|_{\mathbf{b}}\} \in \mathfrak{N}_\Sigma.$$

Define the substitution by $s(\mathbf{a}) = (\mathbf{a} \cup \mathbf{b})$ and $s(\mathbf{b}) = (\mathbf{a} \cup \mathbf{b})$. Every word in L has even length, so every word in $\bar{s}(L)$ also has even length. Thus $\bar{s}(L)$ is contained in the set of even-length words.

Conversely, let y be any even-length word, say $|y| = 2n$. Choose $x = \mathbf{a}^n \mathbf{b}^n \in L$. Since each letter of x may be replaced by either $\mathbf{a}$ or $\mathbf{b}$, the word y belongs to $\bar{s}(x)$. Hence every even-length word lies in $\bar{s}(L)$.

Therefore $\bar{s}(L)$ is exactly the set of all even-length words over $\{\mathbf{a}, \mathbf{b}\}$, which is regular. So the image of a nonregular language under a substitution need not remain in $\mathfrak{N}_\Gamma$.

6.45. **Proof of Lemma 6.3.**

Lemma 6.3 is the version of Theorem 6.2 that eliminates the m th unknown instead of necessarily the last unknown.

Proof. Proof. Renumber the unknowns so that X_m becomes the last unknown X_n , without changing the actual system except for notation. After this relabeling, Theorem 6.2 applies directly. It gives the elimination formulas

$$\hat{E}_i = E_i \cup (A_{im} A_{mm}^* E_m) \quad \text{and} \quad \hat{A}_{ij} = A_{ij} \cup (A_{im} A_{mm}^* A_{mj}),$$

for every index other than m , together with the back-substitution formula

$$X_m = A_{mm}^* (E_m \cup A_{m1} X_1 \cup \cdots \cup A_{m(m-1)} X_{m-1} \cup A_{m(m+1)} X_{m+1} \cup \cdots \cup A_{mn} X_n).$$

Renaming the variables back proves Lemma 6.3. $\square$

6.46. $\Xi = \{x \in \{\mathbf{a}, \mathbf{b}\}^* \mid x \text{ contains at least two consecutive } \mathbf{b}\mathbf{s} \wedge x \text{ contains no two consecutive } \mathbf{a}\mathbf{s}\}$:

Words over $\{\mathbf{a}, \mathbf{b}\}$ avoiding $\mathbf{a}\mathbf{a}$ are exactly $(\mathbf{b} \cup \mathbf{a}\mathbf{b})^* (\mathbf{a} \cup \epsilon)$. Requiring a $\mathbf{b}\mathbf{b}$ substring gives:

$$(\mathbf{b} \cup \mathbf{a}\mathbf{b})^* (\mathbf{a} \cup \epsilon) \mathbf{b}\mathbf{b} (\mathbf{b} \cup \mathbf{a}\mathbf{b})^* (\mathbf{a} \cup \epsilon)$$

6.47. (a) Every $\mathbf{b}$ eventually followed by $\mathbf{c}$ (not necessarily immediately): A word is in the language iff either it has no $\mathbf{b}$ at all after its last $\mathbf{c}$, or it has no $\mathbf{c}$ and then must contain only $\mathbf{a}\mathbf{s}$. Thus the suffix after the last $\mathbf{c}$ must lie in $\mathbf{a}^*$:

$$\mathbf{a}^* \cup (\mathbf{a} \cup \mathbf{b} \cup \mathbf{c})^* \mathbf{c} \mathbf{a}^*$$

Any $\mathbf{b}$ before the last $\mathbf{c}$ is discharged by that final $\mathbf{c}$, and after the last $\mathbf{c}$ only $\mathbf{a}\mathbf{s}$ may remain.

- (b) Every ***b*** immediately followed by ***c***: Any valid word is a concatenation of blocks from $\{\mathbf{a}, \mathbf{c}, \mathbf{bc}\}$, because every occurrence of ***b*** must be paired immediately with a following ***c***. Conversely, any concatenation of those blocks has the required property. Therefore

$$(\mathbf{a} \cup \mathbf{c} \cup \mathbf{bc})^*$$

6.48. Over $\Sigma = \{\mathbf{a}, \mathbf{b}\}$:

- (a) No consecutive ***as*** and no consecutive ***bs*** (strictly alternating):

$$(\mathbf{ab})^*(\mathbf{a} \cup \epsilon) \cup (\mathbf{ba})^*(\mathbf{b} \cup \epsilon)$$

(Alternating starting with ***a*** or ***b***, or empty word.)

- (b) Words beginning and ending with different letters:

$$\mathbf{a}(\mathbf{a} \cup \mathbf{b})^* \mathbf{b} \cup \mathbf{b}(\mathbf{a} \cup \mathbf{b})^* \mathbf{a}$$

- (c) Words for which the last two letters match (***aa*** or ***bb*** at the end):

$$(\mathbf{a} \cup \mathbf{b})^* \mathbf{aa} \cup (\mathbf{a} \cup \mathbf{b})^* \mathbf{bb}$$

- (d) Words for which the first two letters match (***aa*** or ***bb*** at the start):

$$\mathbf{aa}(\mathbf{a} \cup \mathbf{b})^* \cup \mathbf{bb}(\mathbf{a} \cup \mathbf{b})^*$$

- (e) Words for which the first and last letters match:

$$\mathbf{a} \cup \mathbf{b} \cup \mathbf{a}(\mathbf{a} \cup \mathbf{b})^* \mathbf{a} \cup \mathbf{b}(\mathbf{a} \cup \mathbf{b})^* \mathbf{b}$$

6.49. **The language of all valid regular expressions over $\{\mathbf{a}, \mathbf{b}\}$ is not FAD.**

Proof. Let V be the language of all valid regular expressions over $\{\mathbf{a}, \mathbf{b}\}$. Intersect V with the regular language

$$R = (\mathbf{()^* a ()})^*.$$

The only valid regular expressions in R are the strings

$$(\mathbf{^n a})^n \quad (n \geq 0),$$

because the parentheses must match and simply enclose the one-letter expression ***a***. Therefore

$$V \cap R = \{(\mathbf{^n a})^n \mid n \geq 0\},$$

which is not FAD by the pumping lemma. But the intersection of two FAD languages would be FAD. Hence V is not FAD. $\square$

6.50. Apply Theorem 6.3 (state equations) and Theorem 6.2 (Arden's lemma) to each NDFA in Figure 6.14 to produce a regular expression.

(a) Let X_0 denote the terminal set from t_0 . The equation is

$$X_0 = \epsilon \cup (\mathbf{a} \cup \mathbf{c})X_0.$$

Theorem 6.1 gives

$$X_0 = (\mathbf{a} \cup \mathbf{c})^*.$$

(b) Let X_1 and X_0 denote the terminal sets from q_1 and q_0 , respectively. The equations are

$$X_1 = \epsilon \cup \mathbf{b}X_1 \cup \mathbf{a}X_0, \quad X_0 = \mathbf{c}X_0 \cup \mathbf{b}X_1.$$

The second equation gives $X_0 = \mathbf{c}^*\mathbf{b}X_1$. Hence

$$X_1 = \epsilon \cup (\mathbf{b} \cup \mathbf{ac}^*\mathbf{b})X_1,$$

and

$$X_1 = (\mathbf{b} \cup \mathbf{ac}^*\mathbf{b})^*.$$

(c) With terminal sets X_0 and X_1 from r_0 and r_1 ,

$$X_0 = \epsilon \cup \mathbf{a}X_0 \cup (\mathbf{a} \cup \mathbf{c})X_1, \quad X_1 = \mathbf{b}X_0.$$

Substitution gives

$$X_0 = \epsilon \cup (\mathbf{a} \cup (\mathbf{a} \cup \mathbf{c})\mathbf{b})X_0,$$

so

$$X_0 = (\mathbf{a} \cup (\mathbf{a} \cup \mathbf{c})\mathbf{b})^*.$$

(d) With X_i the terminal set from s_i ,

$$X_0 = \mathbf{a}X_1, \quad X_1 = \mathbf{a}X_2 \cup \mathbf{b}X_3, \quad X_2 = \epsilon, \quad X_3 = \mathbf{b}X_1.$$

Thus $X_1 = \mathbf{a} \cup \mathbf{bb}X_1$, so $X_1 = (\mathbf{bb})^*\mathbf{a}$ and

$$X_0 = \mathbf{a}(\mathbf{bb})^*\mathbf{a}.$$

(e) With terminal sets X_1 and X_2 from the two states,

$$X_1 = \mathbf{a}X_1 \cup \mathbf{a}X_2, \quad X_2 = \epsilon \cup (\mathbf{a} \cup \mathbf{b})X_1.$$

Substitute the second equation into the first:

$$X_1 = \mathbf{a}X_1 \cup \mathbf{a}(\epsilon \cup (\mathbf{a} \cup \mathbf{b})X_1) = \mathbf{a} \cup (\mathbf{a} \cup \mathbf{aa} \cup \mathbf{ab})X_1.$$

Therefore, by Theorem 6.1,

$$X_1 = (\mathbf{a} \cup \mathbf{aa} \cup \mathbf{ab})^*\mathbf{a}.$$

(f) With X_i the terminal set from S_i ,

$$X_0 = \mathbf{a}X_1, \quad X_1 = X_0 \cup \mathbf{b}X_2, \quad X_2 = \mathbf{c}X_3, \quad X_3 = \epsilon \cup X_1.$$

Hence

$$X_1 = X_0 \cup \mathbf{bc} \cup \mathbf{bc}X_1 = \mathbf{a}X_1 \cup \mathbf{bc} \cup \mathbf{bc}X_1,$$

so

$$X_1 = (\mathbf{a} \cup \mathbf{bc})^*\mathbf{bc}.$$

Since $X_0 = \mathbf{a}X_1$,

$$X_0 = \mathbf{a}(\mathbf{a} \cup \mathbf{bc})^*\mathbf{bc} = \mathbf{aa}^*\mathbf{bc}(\mathbf{a}^*\mathbf{bc})^*.$$

(g) With X_i the terminal set from S_i ,

$$X_0 = \epsilon \cup \mathbf{a}X_1, \quad X_1 = \mathbf{a}X_0 \cup \mathbf{c}X_2 \cup \mathbf{b}X_3, \quad X_2 = \mathbf{c}X_1, \quad X_3 = \mathbf{b}X_1.$$

Substituting for X_2 and X_3 gives

$$X_1 = \mathbf{a}X_0 \cup (\mathbf{b}\mathbf{b} \cup \mathbf{c}\mathbf{c})X_1,$$

so

$$X_1 = (\mathbf{b}\mathbf{b} \cup \mathbf{c}\mathbf{c})^* \mathbf{a}X_0.$$

Then

$$X_0 = \epsilon \cup \mathbf{a}(\mathbf{b}\mathbf{b} \cup \mathbf{c}\mathbf{c})^* \mathbf{a}X_0,$$

and therefore

$$X_0 = (\mathbf{a}(\mathbf{b}\mathbf{b} \cup \mathbf{c}\mathbf{c})^* \mathbf{a})^*.$$

6.51. Complete details of Theorem 6.6.

Theorem 6.6 proves closure of $\mathfrak{R}_\Sigma$ under substitution. Below are the three operator cases written out explicitly.

Proof. The basis step is the case in which R has no operators. Then R is either a single letter $\mathbf{a} \in \Sigma$, $\emptyset$, or ϵ . By Definition 6.5, $s(\mathbf{a})$ is a regular expression over Γ representing $\bar{s}(\{\mathbf{a}\})$, while $\emptyset$ represents $\bar{s}(\emptyset)$ and ϵ represents $\bar{s}(\{\lambda\})$. Thus the substituted expression has the required language in each basis case.

Assume inductively that whenever R_1 and R_2 have at most m operators, the expressions obtained by replacing each letter $\mathbf{a}$ with $s(\mathbf{a})$ represent $\bar{s}(R_1)$ and $\bar{s}(R_2)$.

If $R = R_1 \cdot R_2$, then by Definition 6.5,

$$\bar{s}(R_1 \cdot R_2) = \bar{s}(R_1) \cdot \bar{s}(R_2).$$

So the substituted expression $R'_1 R'_2$ represents $\bar{s}(R)$.

If $R = R_1 \cup R_2$, then

$$\bar{s}(R_1 \cup R_2) = \bar{s}(R_1) \cup \bar{s}(R_2),$$

so the substituted expression $R'_1 \cup R'_2$ represents $\bar{s}(R)$.

If $R = (R_1)^*$, then

$$\bar{s}((R_1)^*) = (\bar{s}(R_1))^*,$$

so $(R'_1)^*$ represents $\bar{s}(R)$.

Thus substitution preserves each of the three operators used to build regular expressions. By induction on the number of operators in R , the expression obtained by substituting $s(\mathbf{a})$ for each letter $\mathbf{a}$ represents exactly $\bar{s}(R)$. Therefore $\mathfrak{R}_\Sigma$ is closed under substitution. $\square$ $\square$

6.52. Prove Lemma 6.4.

Proof. Let

$$A = \langle \Sigma, \{s_1, s_2, \dots, s_n\}, S_0, \delta, F \rangle$$

be an NDFA, and for each i let

$$Y_i = I(A, s_i) = \{x \mid \bar{\delta}(S_0, x) \ni s_i\}.$$

Define

$$I_i = \begin{cases} \epsilon & \text{if } s_i \in S_0, \\ \emptyset & \text{if } s_i \notin S_0, \end{cases}$$

and

$$B_{ij} = \bigcup_{a \in \Sigma \wedge s_i \in \delta(s_j, a)} a.$$

We show that

$$Y_k = I_k \cup Y_1 B_{k1} \cup Y_2 B_{k2} \cup \dots \cup Y_n B_{kn}.$$

If $x \in Y_k$, then either $x = \lambda$ and $s_k \in S_0$, so $x \in I_k$, or $x = za$ where z takes some start state to some s_j and a takes s_j to s_k . Then $z \in Y_j$ and $a \in B_{kj}$, so $x \in Y_j B_{kj}$. Hence $Y_k \subseteq I_k \cup Y_1 B_{k1} \cup \dots \cup Y_n B_{kn}$.

Conversely, if $x \in I_k$, then $x = \lambda$ and $s_k \in S_0$, so $x \in Y_k$. If $x \in Y_j B_{kj}$, then $x = za$ for some $z \in Y_j$ and some letter $a \in B_{kj}$. Thus z reaches s_j from a start state, and then a takes s_j to s_k , so $x \in Y_k$. Therefore $I_k \cup Y_1 B_{k1} \cup \dots \cup Y_n B_{kn} \subseteq Y_k$ as well.

So the required equations hold for every k . □ □

6.53. Theorems 5.2, 5.4, 5.5 as corollaries of Theorem 6.6.

By Theorem 6.5, every FAD language has a regular expression. Let $L_1 = L(R_1)$ and $L_2 = L(R_2)$.

For union, work over the alphabet $\{a, b\}$ with the regular expression

$$Q = a \cup b.$$

Define a substitution by

$$s(a) = R_1, \quad s(b) = R_2.$$

Then the substituted expression is

$$\bar{s}(Q) = R_1 \cup R_2,$$

whose language is $L_1 \cup L_2$. So $L_1 \cup L_2$ is regular by Theorem 6.6.

For concatenation, use the regular expression

$$Q = ab$$

with the same substitution. Then

$$\bar{s}(Q) = R_1 R_2,$$

whose language is $L_1 \cdot L_2$. So $L_1 \cdot L_2$ is regular.

For Kleene closure, work over the alphabet $\{a\}$ and use the regular expression

$$Q = a^*.$$

Define

$$s(\mathbf{a}) = R_1.$$

Then

$$\bar{s}(Q) = (R_1)^*,$$

whose language is L_1^* . So L_1^* is regular.

Thus Theorems 5.2, 5.4, and 5.5 follow as corollaries of Theorem 6.6.

6.54. **Class $\mathbf{F}$ generated by the five rules:**

The five rules generate exactly the class of *finite* languages over $\{\mathbf{a}, \mathbf{b}\}$:

- (i) Singletons $\{\mathbf{a}\}$ and $\{\mathbf{b}\}$ are in $\mathbf{F}$.
- (ii) $\emptyset \in \mathbf{F}$.
- (iii) $\{\lambda\} \in \mathbf{F}$.
- (iv) Concatenation: $F_1 \cdot F_2$ is finite if F_1 and F_2 are.
- (v) Union: $F_1 \cup F_2$ is finite if F_1 and F_2 are.

No Kleene closure is allowed, so $\mathbf{F}$ consists only of finite languages. Every finite language over $\{\mathbf{a}, \mathbf{b}\}$ is in $\mathbf{F}$ (can be expressed as a finite union of finite concatenations of singletons), and $\mathbf{F} \subseteq F_{\{\mathbf{a}, \mathbf{b}\}}$ (the finite languages). Thus $\mathbf{F} = F_{\{\mathbf{a}, \mathbf{b}\}}$, the class of all finite languages over $\{\mathbf{a}, \mathbf{b}\}$.

Chapter 7

Finite State Transducers — Solutions

7.1. (a) **Prove** $\bar{\omega}(t, yx) = \bar{\omega}(t, y) \cdot \bar{\omega}(\bar{\delta}(t, y), x)$ for any Mealy machine.

Proof. By induction on $|y| = k$.

Base ($k = 0$): $\bar{\omega}(t, \lambda x) = \bar{\omega}(t, x) = \lambda \cdot \bar{\omega}(t, x) = \bar{\omega}(t, \lambda) \cdot \bar{\omega}(\bar{\delta}(t, \lambda), x)$.

Step: Let $y = \mathbf{a}y'$ with $|y'| = k$. Then:

$$\begin{aligned} \bar{\omega}(t, \mathbf{a}y'x) &= \omega(t, \mathbf{a}) \cdot \bar{\omega}(\delta(t, \mathbf{a}), y'x) \\ &= \omega(t, \mathbf{a}) \cdot \bar{\omega}(\delta(t, \mathbf{a}), y') \cdot \bar{\omega}(\bar{\delta}(\delta(t, \mathbf{a}), y'), x) \\ &= \bar{\omega}(t, \mathbf{a}y') \cdot \bar{\omega}(\bar{\delta}(t, \mathbf{a}y'), x) \\ &= \bar{\omega}(t, y) \cdot \bar{\omega}(\bar{\delta}(t, y), x). \quad \square \end{aligned}$$

□

(b) **Prove** $\bar{\delta}(s, yx) = \bar{\delta}(\bar{\delta}(s, y), x)$.

Proof. By induction on $|y|$.

Base: $\bar{\delta}(s, \lambda x) = \bar{\delta}(s, x) = \bar{\delta}(\bar{\delta}(s, \lambda), x)$.

Step: Let $y = \mathbf{a}y'$ with $|y'| = k$. Then

$$\bar{\delta}(s, \mathbf{a}y'x) = \bar{\delta}(\delta(s, \mathbf{a}), y'x) = \bar{\delta}(\bar{\delta}(\delta(s, \mathbf{a}), y'), x) = \bar{\delta}(\bar{\delta}(s, \mathbf{a}y'), x).$$

Hence $\bar{\delta}(s, yx) = \bar{\delta}(\bar{\delta}(s, y), x)$. □

□

7.2. Let $\mu : S_A \rightarrow S_B$ be a homomorphism between Mealy machines.

(a) **Prove** $\mu(\bar{\delta}_A(s, x)) = \bar{\delta}_B(\mu(s), x)$.

Proof. By induction on $|x|$.

Base: $\mu(\bar{\delta}_A(s, \lambda)) = \mu(s) = \bar{\delta}_B(\mu(s), \lambda)$.

Step: Let $x = \mathbf{a}y$ with $|y| = k$. Then

$$\begin{aligned} \mu(\bar{\delta}_A(s, \mathbf{a}y)) &= \mu(\bar{\delta}_A(\delta_A(s, \mathbf{a}), y)) \\ &= \bar{\delta}_B(\mu(\delta_A(s, \mathbf{a})), y) \\ &= \bar{\delta}_B(\delta_B(\mu(s), \mathbf{a}), y) \\ &= \bar{\delta}_B(\mu(s), \mathbf{a}y). \end{aligned}$$

Thus $\mu(\bar{\delta}_A(s, x)) = \bar{\delta}_B(\mu(s), x)$. □

(b) **Prove** $\bar{\omega}_A(s, x) = \bar{\omega}_B(\mu(s), x)$.

Proof. By induction on $|x|$.

Base: $\bar{\omega}_A(s, \lambda) = \lambda = \bar{\omega}_B(\mu(s), \lambda)$.

Step: Let $x = ay$ with $|y| = k$. Then

$$\begin{aligned}\bar{\omega}_A(s, ay) &= \omega_A(s, a) \cdot \bar{\omega}_A(\delta_A(s, a), y) \\ &= \omega_B(\mu(s), a) \cdot \bar{\omega}_B(\mu(\delta_A(s, a)), y) \\ &= \omega_B(\mu(s), a) \cdot \bar{\omega}_B(\delta_B(\mu(s), a), y) \\ &= \bar{\omega}_B(\mu(s), ay).\end{aligned}$$

Thus $\bar{\omega}_A(s, x) = \bar{\omega}_B(\mu(s), x)$. □

7.3. Prove Corollary 7.3.

If there is a homomorphism $\mu : A \rightarrow B$ between connected Mealy machines, then A and B are equivalent.

Proof. By Exercise 7.2(b), $\bar{\omega}_A(s_0, x) = \bar{\omega}_B(\mu(s_0), x)$ for all $x \in \Sigma^*$. Since $\mu(s_{0_A}) = s_{0_B}$ (by definition of homomorphism), $\bar{\omega}_A(s_{0_A}, x) = \bar{\omega}_B(s_{0_B}, x)$, i.e., $f_A(x) = f_B(x)$ for all x . Hence $A \equiv B$. □

7.4. Prove Corollary 7.4: M is minimal iff M is reduced and connected.

Proof. ($\Rightarrow$) If M is minimal, then M must be connected by Corollary 7.2. Also, M must be reduced by Corollary 7.3. Hence M is both reduced and connected.

($\Leftarrow$) Assume that M is reduced and connected. Let A be any minimal FST equivalent to M . By Corollaries 7.2 and 7.3, A is also reduced and connected. Therefore Theorem 7.5 applies, so $A \cong M$. Isomorphic machines have the same number of states, hence $\|S_A\| = \|S_M\|$. Since A was chosen minimal, no equivalent machine has fewer states than A , and therefore no equivalent machine has fewer states than M . Thus M is minimal. □

7.5. Pumping lemma for FTD functions.

(a) **Statement:** Let $f : \Sigma^* \rightarrow \Gamma^*$ be FTD, implemented by a Mealy machine M with n states. Then for any $x \in \Sigma^*$ with $|x| \geq n$, there exist strings u, v, w with $x = uvw$, $|uv| \leq n$, $|v| \geq 1$, such that for all $k \geq 0$:

$$f(uv^k w) = \bar{\omega}(s_0, u) \cdot \bar{\omega}(\bar{\delta}(s_0, u), v)^k \cdot \bar{\omega}(\bar{\delta}(s_0, uv), w).$$

That is, the output of $uv^k w$ consists of the output while processing u , followed by k repetitions of the output while processing v (each time starting from state $\bar{\delta}(s_0, u)$), followed by the output while processing w from state $\bar{\delta}(s_0, uv) = \bar{\delta}(s_0, u)$.

(b) **Proof:** Since $|x| \geq n$ and M has n states, by the pigeonhole principle the sequence of states $s_0, \bar{\delta}(s_0, u), \bar{\delta}(s_0, uv), \dots$ must repeat a state within the first $n + 1$ steps. Let u, v, w be such that $uvw = x$, v causes a cycle: $\bar{\delta}(s_0, u) = \bar{\delta}(s_0, uv)$. Then for any $k \geq 0$, $\bar{\delta}(s_0, uv^k) = \bar{\delta}(s_0, u)$, so the machine processes w from the same state regardless of k . Thus $f(uv^k w) = \bar{\omega}(s_0, u) \cdot \bar{\omega}(\bar{\delta}(s_0, u), v)^k \cdot \bar{\omega}(\bar{\delta}(s_0, uv), w)$. □

7.6. Prove Lemma 7.3 parts a–e.

(a) **Part (a):** $E_{m+1,M}$ is a refinement of E_{mM} .

Proof. If $sE_{m+1,M}t$, then by definition the states s and t agree on all input strings of length at most $m+1$. In particular, they agree on all strings of length at most m . Hence $sE_{mM}t$. $\square$ $\square$

(b) **Part (b):** E_M is a refinement of E_{mM} .

Proof. If sE_Mt , then $\bar{\omega}(s, x) = \bar{\omega}(t, x)$ for every $x \in \Sigma^*$. In particular, the two states agree on all strings of length at most m , so $sE_{mM}t$. Therefore $E_M \subseteq E_{mM}$. $\square$ $\square$

(c) **Part (c):** If $E_{mM} = E_{m+1,M}$, then $E_{m+k,M} = E_{mM}$ for every $k \in \mathbb{N}$.

Proof. We use induction on k . The case $k = 0$ is trivial, and the case $k = 1$ is the hypothesis. Assume $E_{m+k,M} = E_{mM}$ for some $k \geq 1$. By part (a), $E_{m+k+1,M} \subseteq E_{m+k,M} = E_{mM}$, so only the reverse inclusion needs proof.

Let $sE_{mM}t$. Since $E_{mM} = E_{m+1,M}$, we also have $sE_{m+1,M}t$. Therefore, for every $\mathbf{a} \in \Sigma$, the successor states $\delta(s, \mathbf{a})$ and $\delta(t, \mathbf{a})$ agree on all strings of length at most m , so

$$\delta(s, \mathbf{a})E_{mM}\delta(t, \mathbf{a}).$$

By the induction hypothesis this implies

$$\delta(s, \mathbf{a})E_{m+k,M}\delta(t, \mathbf{a})$$

for every $\mathbf{a}$. Hence s and t agree on all strings of length at most $m+k+1$, so $sE_{m+k+1,M}t$. Thus $E_{mM} \subseteq E_{m+k+1,M}$, and therefore $E_{m+k+1,M} = E_{mM}$. $\square$ $\square$

(d) **Part (d):** There exists $m \leq \|S\|$ such that $E_{mM} = E_{m+1,M}$.

Proof. By part (a), the relations

$$E_{0M}, E_{1M}, E_{2M}, \dots$$

form a descending chain of refinements. If $E_{i+1,M} \neq E_{iM}$, then the containment is proper, so some E_{iM} -class is split into at least two $E_{i+1,M}$ -classes; hence the two adjacent relations are distinct, and the number of equivalence classes increases by at least one. Since there are only $\|S\|$ states, there can be at most $\|S\|$ equivalence classes. Therefore the chain cannot refine properly more than $\|S\| - 1$ times, and for some $m \leq \|S\|$ we must have $E_{mM} = E_{m+1,M}$. $\square$ $\square$

(e) **Part (e):** If $E_{mM} = E_{m+1,M}$, then $E_{mM} = E_M$.

Proof. By part (b), $E_M \subseteq E_{mM}$. For the reverse inclusion, let $sE_{mM}t$ and let $x \in \Sigma^*$ be arbitrary, say $|x| = k$. By part (c),

$$E_{m+k,M} = E_{mM},$$

so $sE_{m+k,M}t$. By definition of $E_{m+k,M}$, the states s and t agree on every input string of length at most $m+k$, and in particular on the string x . Thus $\bar{\omega}(s, x) = \bar{\omega}(t, x)$. Since x was arbitrary, sE_Mt . Hence $E_{mM} \subseteq E_M$, and therefore $E_{mM} = E_M$. $\square$ $\square$

7.7. Prove parts of Lemma 7.4 for FST M .

(a) E_{0M} **has one class consisting of all of S :**

E_{0M} is defined by $sE_{0M}t$ always (no conditions). Hence all states are in one class: $[s]_{E_{0M}} = S$ for all s .

(b) $sE_{1M}t$ **iff $\forall \mathbf{a} \in \Sigma, \omega(s, \mathbf{a}) = \omega(t, \mathbf{a})$:**

E_{1M} refines E_{0M} by requiring that s and t agree on single-letter outputs. $sE_{1M}t$ iff $sE_{0M}t$ (trivially) and $\forall \mathbf{a} \in \Sigma, \omega(s, \mathbf{a}) = \omega(t, \mathbf{a})$.

(c) $sE_{i+1,M}t$ **iff $sE_{iM}t$ and $\forall \mathbf{a} \in \Sigma, \delta(s, \mathbf{a})E_{iM}\delta(t, \mathbf{a})$:**

Proof. Assume $i \geq 1$.

($\Rightarrow$) If $sE_{i+1,M}t$, then s and t agree on every input string of length at most $i+1$. In particular, they agree on all strings of length at most i , so $sE_{iM}t$. Now fix $\mathbf{a} \in \Sigma$ and $y \in \Sigma^*$ with $|y| \leq i$. Since $|\mathbf{a}y| \leq i+1$, Definition 7.2 gives

$$\omega(s, \mathbf{a}) \cdot \bar{\omega}(\delta(s, \mathbf{a}), y) = \bar{\omega}(s, \mathbf{a}y) = \bar{\omega}(t, \mathbf{a}y) = \omega(t, \mathbf{a}) \cdot \bar{\omega}(\delta(t, \mathbf{a}), y).$$

Taking $y = \lambda$ shows $\omega(s, \mathbf{a}) = \omega(t, \mathbf{a})$. Canceling this common first letter from the displayed equality leaves $\bar{\omega}(\delta(s, \mathbf{a}), y) = \bar{\omega}(\delta(t, \mathbf{a}), y)$ for every $|y| \leq i$. Hence $\delta(s, \mathbf{a})E_{iM}\delta(t, \mathbf{a})$.

($\Leftarrow$) Suppose $sE_{iM}t$ and $\delta(s, \mathbf{a})E_{iM}\delta(t, \mathbf{a})$ for every $\mathbf{a} \in \Sigma$. Let $w \in \Sigma^*$ with $|w| \leq i+1$. If $|w| \leq i$, then $\bar{\omega}(s, w) = \bar{\omega}(t, w)$ because $sE_{iM}t$. Otherwise write $w = \mathbf{a}y$, where $|y| \leq i$. Since $i \geq 1$, $sE_{iM}t$ implies $sE_{1M}t$, so $\omega(s, \mathbf{a}) = \omega(t, \mathbf{a})$. Also, $\delta(s, \mathbf{a})E_{iM}\delta(t, \mathbf{a})$ gives $\bar{\omega}(\delta(s, \mathbf{a}), y) = \bar{\omega}(\delta(t, \mathbf{a}), y)$. Therefore, by Definition 7.2,

$$\bar{\omega}(s, \mathbf{a}y) = \omega(s, \mathbf{a}) \cdot \bar{\omega}(\delta(s, \mathbf{a}), y) = \omega(t, \mathbf{a}) \cdot \bar{\omega}(\delta(t, \mathbf{a}), y) = \bar{\omega}(t, \mathbf{a}y).$$

Thus $sE_{i+1,M}t$. □

7.8. Prove Theorem 7.6 for Moore machines.

For a Moore machine $A = \langle \Sigma, \Gamma, S, s_0, \delta, \omega \rangle$:

Proof. We prove that

$$(\forall x \in \Sigma^*)(\forall y \in \Sigma^*)(\forall t \in S) \bar{\omega}(t, yx) = \bar{\omega}(t, y) \cdot \bar{\omega}(\bar{\delta}(t, y), x)$$

by induction on $|y|$.

Base ($|y| = 0$): If $y = \lambda$, then

$$\bar{\omega}(t, \lambda x) = \bar{\omega}(t, x) = \lambda \cdot \bar{\omega}(t, x) = \bar{\omega}(t, \lambda) \cdot \bar{\omega}(\bar{\delta}(t, \lambda), x),$$

since $\bar{\omega}(t, \lambda) = \lambda$ and $\bar{\delta}(t, \lambda) = t$.

Step: Assume the identity holds for a word y . Let $\mathbf{a} \in \Sigma$. Then, by Definition 7.17,

$$\bar{\omega}(t, \mathbf{a}yx) = \omega(\delta(t, \mathbf{a})) \cdot \bar{\omega}(\delta(t, \mathbf{a}), yx).$$

Apply the inductive hypothesis with start state $\delta(t, \mathbf{a})$:

$$\bar{\omega}(t, \mathbf{a}yx) = \omega(\delta(t, \mathbf{a})) \cdot \bar{\omega}(\delta(t, \mathbf{a}), y) \cdot \bar{\omega}(\bar{\delta}(\delta(t, \mathbf{a}), y), x).$$

Again by Definition 7.17, the first two factors are exactly $\bar{\omega}(t, \mathbf{a}y)$, and by Exercise 7.1(b), $\bar{\delta}(\delta(t, \mathbf{a}), y) = \bar{\delta}(t, \mathbf{a}y)$. Hence

$$\bar{\omega}(t, \mathbf{a}yx) = \bar{\omega}(t, \mathbf{a}y) \cdot \bar{\omega}(\bar{\delta}(t, \mathbf{a}y), x).$$

This proves the result. □

7.9. **Prove Theorem 7.7.**

Proof. Let

$$A = \langle \Sigma, \Gamma, S, s_0, \delta, \omega \rangle$$

be a Moore machine, and let

$$M = \langle \Sigma, \Gamma, S, s_0, \delta, \omega' \rangle$$

be the corresponding Mealy machine from Definition 7.19, where $\omega'(s, \mathbf{a}) = \omega(\delta(s, \mathbf{a}))$.

We prove by induction on $|x|$ that

$$(\forall s \in S)(\forall x \in \Sigma^*) \bar{\omega}_M(s, x) = \bar{\omega}_A(s, x).$$

Base: If $x = \lambda$, then both sides equal λ .

Step: Let $x = \mathbf{a}y$. Then

$$\bar{\omega}_M(s, \mathbf{a}y) = \omega'(s, \mathbf{a}) \cdot \bar{\omega}_M(\delta(s, \mathbf{a}), y) = \omega(\delta(s, \mathbf{a})) \cdot \bar{\omega}_M(\delta(s, \mathbf{a}), y).$$

By the inductive hypothesis applied to the state $\delta(s, \mathbf{a})$,

$$\bar{\omega}_M(s, \mathbf{a}y) = \omega(\delta(s, \mathbf{a})) \cdot \bar{\omega}_A(\delta(s, \mathbf{a}), y) = \bar{\omega}_A(s, \mathbf{a}y),$$

by Definition 7.17. Therefore $\bar{\omega}_M(s, x) = \bar{\omega}_A(s, x)$ for all s and x . In particular, $f_M(x) = \bar{\omega}_M(s_0, x) = \bar{\omega}_A(s_0, x) = f_A(x)$, so $M \equiv A$. $\square$ $\square$

7.10. **Prove Theorem 7.8.**

Proof. Let

$$M = \langle \Sigma, \Gamma, S, s_0, \delta, \omega \rangle$$

be a Mealy machine, and let

$$A = \langle \Sigma, \Gamma, S \times \Gamma, \langle s_0, \alpha \rangle, \delta', \omega' \rangle$$

be the corresponding Moore machine from Definition 7.20.

We prove the stronger statement

$$(\forall s \in S)(\forall \mathbf{b} \in \Gamma)(\forall x \in \Sigma^*) \bar{\omega}_A(\langle s, \mathbf{b} \rangle, x) = \bar{\omega}_M(s, x).$$

Base: If $x = \lambda$, then both sides are λ .

Step: Let $x = \mathbf{a}y$. By Definition 7.17 for Moore machines,

$$\bar{\omega}_A(\langle s, \mathbf{b} \rangle, \mathbf{a}y) = \omega'(\delta'(\langle s, \mathbf{b} \rangle, \mathbf{a})) \cdot \bar{\omega}_A(\delta'(\langle s, \mathbf{b} \rangle, \mathbf{a}), y).$$

By Definition 7.20,

$$\delta'(\langle s, \mathbf{b} \rangle, \mathbf{a}) = \langle \delta(s, \mathbf{a}), \omega(s, \mathbf{a}) \rangle$$

and

$$\omega'(\langle \delta(s, \mathbf{a}), \omega(s, \mathbf{a}) \rangle) = \omega(s, \mathbf{a}).$$

Hence

$$\bar{\omega}_A(\langle s, \mathbf{b} \rangle, \mathbf{a}y) = \omega(s, \mathbf{a}) \cdot \bar{\omega}_A(\langle \delta(s, \mathbf{a}), \omega(s, \mathbf{a}) \rangle, y).$$

Apply the inductive hypothesis:

$$\bar{\omega}_A(\langle s, \mathbf{b} \rangle, \mathbf{a}y) = \omega(s, \mathbf{a}) \cdot \bar{\omega}_M(\delta(s, \mathbf{a}), y) = \bar{\omega}_M(s, \mathbf{a}y).$$

Therefore $\bar{\omega}_A(\langle s, \mathbf{b} \rangle, x) = \bar{\omega}_M(s, x)$ for all states and inputs. In particular,

$$f_A(x) = \bar{\omega}_A(\langle s_0, \mathbf{a} \rangle, x) = \bar{\omega}_M(s_0, x) = f_M(x),$$

so $A \equiv M$. □

7.11. Use Lemma 7.6 to find E_C in Example 7.10.

For the Moore machine C of Example 7.10,

$$\omega(r_0) = \omega(r_2) = \mathbf{0}, \quad \omega(r_1) = \omega(r_3) = \mathbf{1}.$$

Hence

$$E_{0C} = \{\{r_0, r_2\}, \{r_1, r_3\}\}.$$

Now apply Lemma 7.6(b). The states r_0 and r_2 are separated at the next stage because

$$\delta(r_0, \mathbf{a}) = r_0, \quad \delta(r_2, \mathbf{a}) = r_1,$$

and r_0 and r_1 are in different E_{0C} -classes. Likewise, r_1 and r_3 are separated because

$$\delta(r_1, \mathbf{a}) = r_0, \quad \delta(r_3, \mathbf{a}) = r_1,$$

again landing in different E_{0C} -classes. Therefore

$$E_{1C} = \{\{r_0\}, \{r_1\}, \{r_2\}, \{r_3\}\}.$$

Since E_{1C} is already the identity relation, the process has stabilized and

$$E_C = E_{1C}.$$

7.12. Homomorphism from machine M (Example 7.11) to machine B (Example 7.2).

Let the states of M be $\{r_0, r_1, r_2, r_3\}$, as in Example 7.11, and let the states of B be $\{s_0, s_1\}$. Define

$$\mu(r_0) = \mu(r_1) = s_0, \quad \mu(r_2) = \mu(r_3) = s_1.$$

This is the natural choice because in Example 7.11 the states r_0, r_1 print only $\mathbf{0}$, while r_2, r_3 print only $\mathbf{1}$, exactly as s_0 and s_1 do in Example 7.2.

We check the homomorphism conditions.

- (i) $\mu(r_0) = s_0$, so start states are preserved.

(ii) For $\mathbf{a}$, Example 7.11 gives

$$\delta_M(r_0, \mathbf{a}) = \delta_M(r_1, \mathbf{a}) = r_0, \quad \delta_M(r_2, \mathbf{a}) = \delta_M(r_3, \mathbf{a}) = r_1.$$

Therefore

$$\mu(\delta_M(r_i, \mathbf{a})) = s_0 = \delta_B(\mu(r_i), \mathbf{a})$$

for every i , since $\delta_B(s_0, \mathbf{a}) = \delta_B(s_1, \mathbf{a}) = s_0$ in Example 7.2.

For $\mathbf{b}$, Example 7.11 gives

$$\delta_M(r_0, \mathbf{b}) = \delta_M(r_1, \mathbf{b}) = r_2, \quad \delta_M(r_2, \mathbf{b}) = \delta_M(r_3, \mathbf{b}) = r_3.$$

Hence

$$\mu(\delta_M(r_i, \mathbf{b})) = s_1 = \delta_B(\mu(r_i), \mathbf{b})$$

for every i , since $\delta_B(s_0, \mathbf{b}) = \delta_B(s_1, \mathbf{b}) = s_1$.

(iii) The output table of Example 7.11 gives

$$\omega_M(r_0, \mathbf{a}) = \omega_M(r_0, \mathbf{b}) = \omega_M(r_1, \mathbf{a}) = \omega_M(r_1, \mathbf{b}) = \mathbf{0},$$

and

$$\omega_M(r_2, \mathbf{a}) = \omega_M(r_2, \mathbf{b}) = \omega_M(r_3, \mathbf{a}) = \omega_M(r_3, \mathbf{b}) = \mathbf{1}.$$

These agree exactly with the outputs of B at s_0 and s_1 .

Thus μ is a homomorphism from M onto B .

7.13. **Prove $\bar{\omega}(t, \mathbf{a}) = \omega(t, \mathbf{a})$ for any FST.**

Proof. By definition of $\bar{\omega}$ for a single letter: $\bar{\omega}(t, \mathbf{a}) = \omega(t, \mathbf{a}) \cdot \bar{\omega}(\delta(t, \mathbf{a}), \lambda) = \omega(t, \mathbf{a}) \cdot \lambda = \omega(t, \mathbf{a})$. (Since $\bar{\omega}(\delta(t, \mathbf{a}), \lambda) = \lambda$ and λ is the identity element for concatenation.) $\square$ $\square$

7.14. **Modified vending machine with coin return.**

Add to the machine:

- New input $\mathbf{r}$ (coin return).
- New output $\mathbf{a}$ (release coins).
- From each state s_k (representing k cents in holding), the transition on $\mathbf{r}$ goes to s_0 (reset to zero) and outputs $\mathbf{a}$.
- The output $\mathbf{a}$ could encode how many coins to return (one per 5 cents, etc.), but if $\mathbf{a}$ is a single symbol meaning “release all,” then it’s a single output.

Formally: add $\delta(s_k, \mathbf{r}) = s_0$ and $\omega(s_k, \mathbf{r}) = \mathbf{a}$ for each state s_k in the vending machine.

7.15. δ_{E_M} **is well defined.**

Proof. $\delta_{E_M}([s]_{E_M}, \mathbf{a}) = [\delta(s, \mathbf{a})]_{E_M}$. Suppose $[s]_{E_M} = [s']_{E_M}$, i.e., $s E_M s'$. By Lemma 7.3c, $\delta(s, \mathbf{a}) E_M \delta(s', \mathbf{a})$. Hence $[\delta(s, \mathbf{a})]_{E_M} = [\delta(s', \mathbf{a})]_{E_M}$, so δ_{E_M} gives the same result regardless of the representative chosen. Also, for every $[s]_{E_M} \in S_{E_M}$ and every $\mathbf{a} \in \Sigma$, δ is defined on $(s, \mathbf{a})$, so $[\delta(s, \mathbf{a})]_{E_M}$ is an element of S_{E_M} . Thus δ_{E_M} is defined on all of $S_{E_M} \times \Sigma$. $\square$ $\square$

7.16. ω_{E_M} is well defined.

Proof. $\omega_{E_M}([s]_{E_M}, \mathbf{a}) = \omega(s, \mathbf{a})$. Suppose $s E_M s'$. By the definition of E_M (states are equivalent iff they produce the same output on all inputs), in particular on the single letter $\mathbf{a}$: $\omega(s, \mathbf{a}) = \omega(s', \mathbf{a})$. Hence ω_{E_M} is independent of the choice of representative. Also, for every $[s]_{E_M} \in S_{E_M}$ and every $\mathbf{a} \in \Sigma$, ω is defined on $(s, \mathbf{a})$, so ω_{E_M} is defined on all of $S_{E_M} \times \Sigma$. $\square$ $\square$

7.17. Reduced is not sufficient for minimality.

Example. A reduced machine may have unreachable states. Let M have states $\{s_0, s_1\}$ where s_1 is unreachable from s_0 . If s_0 and s_1 have distinct output behaviors, M is reduced (no two states are equivalent). But s_1 is useless; removing it gives an equivalent machine with fewer states. Hence M is reduced but not minimal.

7.18. μ in Theorem 7.5 is well defined.

Proof. μ is defined by $\mu(s) = \bar{\delta}_2(s_{0_2}, x)$ where x is any word with $\bar{\delta}_1(s_{0_1}, x) = s$ (such x exists because M_1 is connected). Well-definedness requires: if $\bar{\delta}_1(s_{0_1}, x) = \bar{\delta}_1(s_{0_1}, x') = s$, then $\bar{\delta}_2(s_{0_2}, x) = \bar{\delta}_2(s_{0_2}, x')$.

Since M_1 and M_2 are equivalent, $\bar{\omega}_1(s_{0_1}, wx) = \bar{\omega}_2(s_{0_2}, wx)$ and $\bar{\omega}_1(s_{0_1}, wx') = \bar{\omega}_2(s_{0_2}, wx')$ for all w . In particular, $\bar{\omega}_1(s_{0_1}, xy) = \bar{\omega}_2(s_{0_2}, xy)$ implies $\bar{\omega}_1(s, y) = \bar{\omega}_2(\bar{\delta}_2(s_{0_2}, x), y)$ for all y . Also, $\bar{\omega}_1(s_{0_1}, x'y) = \bar{\omega}_2(s_{0_2}, x'y)$ implies $\bar{\omega}_1(s, y) = \bar{\omega}_2(\bar{\delta}_2(s_{0_2}, x'), y)$ for all y . Since M_2 is reduced, the state $\bar{\delta}_2(s_{0_2}, x)$ is uniquely determined by its output behavior, which equals that of s . Hence $\bar{\delta}_2(s_{0_2}, x) = \bar{\delta}_2(s_{0_2}, x')$, and μ is well defined. $\square$ $\square$

7.19. μ is an isomorphism.

- (a) $\mu(s_{0_1}) = \bar{\delta}_2(s_{0_2}, \lambda) = s_{0_2}$.
- (b) $\mu(\delta_1(s, \mathbf{a})) = \bar{\delta}_2(s_{0_2}, x\mathbf{a}) = \delta_2(\bar{\delta}_2(s_{0_2}, x), \mathbf{a}) = \delta_2(\mu(s), \mathbf{a})$.
- (c) $\omega_1(s, \mathbf{a}) = \bar{\omega}_1(s_{0_1}, x\mathbf{a})|_{\text{last output}} = \bar{\omega}_2(s_{0_2}, x\mathbf{a})|_{\text{last}} = \omega_2(\mu(s), \mathbf{a})$. (The output functions agree since the machines are equivalent and μ commutes with transitions.)
- (d) **Injective:** If $\mu(s) = \mu(t)$, then $\bar{\delta}_2(s_{0_2}, x_s) = \bar{\delta}_2(s_{0_2}, x_t)$, so s and t have the same output behavior from s_{0_1} ; by M_1 being reduced, $s = t$.
- (e) **Surjective:** Since M_2 is connected, every $t \in S_2$ is reachable: $t = \bar{\delta}_2(s_{0_2}, y)$ for some y . Let $s = \bar{\delta}_1(s_{0_1}, y)$; then $\mu(s) = t$.

7.20. One-unit delay Mealy and Moore machines.

$\bar{\omega}(x_1 x_2 \cdots x_m) = \mathbf{x} \cdot x_1 \cdots x_{m-1}$ (output is $\mathbf{x}$ followed by the input shifted by one).

- (a) **Mealy sextuple:** $M = \langle \{\mathbf{a}, \mathbf{b}\}, \{\mathbf{a}, \mathbf{b}, \mathbf{x}\}, \{s_a, s_b, s_0\}, s_0, \delta, \omega \rangle$ where s_0 is the initial state, s_a means “last input was $\mathbf{a}$ ”, s_b means “last input was $\mathbf{b}$ ”. $\delta(s_0, \mathbf{a}) = s_a$, $\delta(s_0, \mathbf{b}) = s_b$, $\delta(s_a, \mathbf{a}) = s_a$, $\delta(s_a, \mathbf{b}) = s_b$, $\delta(s_b, \mathbf{a}) = s_a$, $\delta(s_b, \mathbf{b}) = s_b$. $\omega(s_0, \mathbf{a}) = \mathbf{x}$, $\omega(s_0, \mathbf{b}) = \mathbf{x}$, $\omega(s_a, \mathbf{a}) = \mathbf{a}$, $\omega(s_a, \mathbf{b}) = \mathbf{a}$, $\omega(s_b, \mathbf{a}) = \mathbf{b}$, $\omega(s_b, \mathbf{b}) = \mathbf{b}$.
- (b) **Mealy machine diagram:** Three states; s_0 outputs $\mathbf{x}$ on both inputs; s_a outputs $\mathbf{a}$ on both inputs; s_b outputs $\mathbf{b}$ on both inputs.

- (c) **Moore sextuple:** A Moore machine cannot use the same three states, because under Definition 7.17 the symbol printed after reading the first input is the output attached to the *destination* state. A correct Moore machine is

$$A = \langle \{\mathbf{a}, \mathbf{b}\}, \{\mathbf{a}, \mathbf{b}, \mathbf{x}\}, Q, q_0, \delta', \eta \rangle,$$

where

$$Q = \{q_0, q_{x,a}, q_{x,b}, q_{a,a}, q_{a,b}, q_{b,a}, q_{b,b}\}.$$

Here the first component records the symbol to be printed *now*, and the second component records the most recent input symbol for the next step. The output function is

$$\begin{aligned} \eta(q_0) &= \mathbf{x}, & \eta(q_{x,a}) &= \eta(q_{x,b}) = \mathbf{x}, \\ \eta(q_{a,a}) &= \eta(q_{a,b}) = \mathbf{a}, & \eta(q_{b,a}) &= \eta(q_{b,b}) = \mathbf{b}. \end{aligned}$$

The transition function is

$$\delta'(q_0, \mathbf{a}) = q_{x,a}, \quad \delta'(q_0, \mathbf{b}) = q_{x,b},$$

and, for $u \in \{\mathbf{x}, \mathbf{a}, \mathbf{b}\}$ and $v, \sigma \in \{\mathbf{a}, \mathbf{b}\}$,

$$\delta'(q_{u,v}, \sigma) = q_{v,\sigma}.$$

Thus the first output is $\mathbf{x}$, and each later output is the previous input symbol.

- (d) **Moore machine diagram:** The diagram is the graph of the sextuple in part (c): the start state is $q_0/\mathbf{x}$; it goes to $q_{x,a}/\mathbf{x}$ on input $\mathbf{a}$ and to $q_{x,b}/\mathbf{x}$ on input $\mathbf{b}$; and every state $q_{u,v}$ has arrows

$$q_{u,v} \xrightarrow{\mathbf{a}} q_{v,\mathbf{a}}, \quad q_{u,v} \xrightarrow{\mathbf{b}} q_{v,\mathbf{b}}.$$

7.21. Circuit for vending machine (Example 7.1).

- (a) **EOS:** No. The machine operates continuously, and the candy bar is dispensed on the transition where the button is pushed.
- (b) **SOS:** Yes. As the text points out, a vending machine should reset to its zero-credit state after power is restored.
- (c) **Input encoding:** We want codes for $\{\mathbf{b}, \mathbf{d}, \mathbf{n}, \mathbf{q}, \langle \text{SOS} \rangle\}$, so three bits suffice. One convenient assignment is

$$\mathbf{b} = 000, \quad \mathbf{d} = 001, \quad \mathbf{n} = 010, \quad \mathbf{q} = 011, \quad \langle \text{SOS} \rangle = 111,$$

with 100, 101, 110 unused.

- (d) **Output encoding:** The output alphabet is

$$\{c_0, c_1, c_2, c_3, c_4, \mathbf{d}', \mathbf{n}', \mathbf{q}', \boldsymbol{\varphi}\},$$

so four bits are needed. Let

$$c_0 = 0000, c_1 = 0001, c_2 = 0010, c_3 = 0011, c_4 = 0100, \mathbf{d}' = 0101, \mathbf{n}' = 0110, \mathbf{q}' = 0111, \boldsymbol{\varphi} = 1000.$$

- (e) **State encoding:** Write s_i for the state representing $5i\text{¢}$ for $0 \leq i \leq 10$. Then 11 states require four bits; for example

$$s_0 = 0000, s_1 = 0001, \dots, s_{10} = 1010,$$

with 1011, 1100, 1101, 1110, 1111 unused.

- (f) t_2 **truth table and Boolean function:**

	<i>b</i>	<i>d</i>	<i>n</i>	<i>q</i>	<i><SOS></i>
s_0	0	1	0	0	0
s_1	0	1	1	1	0
s_2	1	0	1	1	0
s_3	1	0	0	0	0
s_4	0	1	0	0	0
s_5	0	1	1	1	0
s_6	0	1	1	1	0
s_7	0	1	1	1	0
s_8	0	0	0	0	0
s_9	0	0	0	0	0
s_{10}	0	1	1	1	0

and one minimized sum-of-products is

$$\begin{aligned} t'_2 = & (\neg t_4 \wedge \neg t_2 \wedge t_1 \wedge \neg a_3 \wedge a_2) \vee (t_3 \wedge t_2 \wedge \neg a_3 \wedge a_2) \vee \\ & (\neg t_4 \wedge \neg t_3 \wedge t_2 \wedge \neg a_2 \wedge \neg a_1) \vee (t_4 \wedge t_2 \wedge \neg a_3 \wedge a_1) \vee \\ & (t_2 \wedge \neg t_1 \wedge \neg a_3 \wedge a_2) \vee (t_3 \wedge \neg a_2 \wedge a_1) \vee (\neg t_4 \wedge \neg t_2 \wedge \neg a_2 \wedge a_1). \end{aligned}$$

- (g) w_2 **truth table and Boolean function:**

	<i>b</i>	<i>d</i>	<i>n</i>	<i>q</i>	<i><SOS></i>
s_0	0	0	0	0	0
s_1	0	0	0	0	0
s_2	0	0	0	0	0
s_3	0	0	0	0	0
s_4	0	0	0	0	0
s_5	0	0	0	0	0
s_6	0	0	1	1	0
s_7	0	0	1	1	0
s_8	1	0	1	1	0
s_9	1	0	1	1	0
s_{10}	0	0	1	1	0

and a minimized form is

$$w_2 = (t_4 \wedge \neg t_2 \wedge \neg a_1) \vee (t_3 \wedge t_2 \wedge \neg a_3 \wedge a_2) \vee (t_4 \wedge \neg a_3 \wedge a_2).$$

- (h) The complete circuit uses four D flip-flops for the state bits t_1, t_2, t_3, t_4 , three input lines a_1, a_2, a_3 , and four output networks w_1, w_2, w_3, w_4 . The full transition/output table is exactly the one read from Figure 7.2: for $0 \leq i \leq 5$, pushing ***b*** leaves s_i fixed with output ***φ***, while

n, d, q send s_i to $s_{i+1}, s_{i+2}, s_{i+5}$ with output φ ; for $6 \leq i \leq 10$, pushing b sends s_i to s_0 with output c_{i-6} , while n, d, q self-loop with outputs n', d', q' ; and $\langle SOS \rangle$ sends every real state to s_0 with output φ . The other next-state bits and output bits are obtained from this same table in exactly the same way.

7.22. **Circuit for modified vending machine (Exercise 7.14).**

(a) **EOS:** No.

(b) **Input encoding:** We again keep a reset code, so use three bits for $\{b, d, n, q, r, \langle SOS \rangle\}$:

$$b = 000, \quad d = 001, \quad n = 010, \quad q = 011, \quad r = 100, \quad \langle SOS \rangle = 111.$$

(c) **Output encoding:** There are now ten outputs $\{a, c_0, c_1, c_2, c_3, c_4, d', n', q', \varphi\}$, so four bits suffice. One convenient code is

$$a = 0000, c_0 = 0001, c_1 = 0010, c_2 = 0011, c_3 = 0100, c_4 = 0101, d' = 0110, n' = 0111, q' = 1000, \varphi = 1001.$$

(d) **State encoding:** Keep the state encoding from Exercise 7.21: s_i means $5i¢$ for $0 \leq i \leq 10$, with $s_0 = 0000, \dots, s_{10} = 1010$.

(e) t_3 **truth table and Boolean function:**

	b	d	n	q	r	$\langle SOS \rangle$
s_0	0	0	0	1	0	0
s_1	0	0	0	1	0	0
s_2	0	1	0	1	0	0
s_3	0	1	1	0	0	0
s_4	1	1	1	0	0	0
s_5	1	1	1	0	0	0
s_6	0	1	1	1	0	0
s_7	0	1	1	1	0	0
s_8	0	0	0	0	0	0
s_9	0	0	0	0	0	0
s_{10}	0	0	0	0	0	0

and one minimized form is

$$\begin{aligned} t'_3 = & (\neg t_4 \wedge t_2 \wedge \neg t_1 \wedge \neg a_3 \wedge a_1) \vee (t_2 \wedge t_1 \wedge a_2 \wedge \neg a_1) \vee \\ & (t_3 \wedge a_2 \wedge \neg a_1) \vee (\neg t_4 \wedge t_2 \wedge \neg a_2 \wedge a_1) \vee (t_3 \wedge t_2 \wedge \neg a_3 \wedge a_2) \vee \\ & (t_3 \wedge \neg t_2 \wedge \neg a_3 \wedge \neg a_2) \vee (\neg t_4 \wedge \neg t_3 \wedge \neg t_2 \wedge \neg a_3 \wedge a_2 \wedge a_1). \end{aligned}$$

(f) w_3 truth table and Boolean function:

	b	d	n	q	r	$\langle SOS \rangle$
s_0	0	0	0	0	0	0
s_1	0	0	0	0	0	0
s_2	0	0	0	0	0	0
s_3	0	0	0	0	0	0
s_4	0	0	0	0	0	0
s_5	0	0	0	0	0	0
s_6	0	1	1	0	0	0
s_7	0	1	1	0	0	0
s_8	0	1	1	0	0	0
s_9	1	1	1	0	0	0
s_{10}	1	1	1	0	0	0

and a minimized form is

$$w_3 = (t_4 \wedge \neg a_2 \wedge a_1) \vee (t_4 \wedge t_1 \wedge \neg a_3 \wedge \neg a_1) \vee (t_3 \wedge t_2 \wedge \neg a_2 \wedge a_1) \vee \\ (t_3 \wedge t_2 \wedge a_2 \wedge \neg a_1) \vee (t_4 \wedge a_2 \wedge \neg a_1) \vee (t_4 \wedge t_2 \wedge \neg a_3 \wedge \neg a_1).$$

(g) The complete circuit is obtained from Exercise 7.21 by adding one new input column for r and changing the output table accordingly. Every real state goes to s_0 on r and produces a ; every real state goes to s_0 on $\langle SOS \rangle$ and produces φ ; and all b, d, n, q entries are unchanged from Exercise 7.21. The other state and output bits are obtained from this same combined table.

7.23. Circuit for Example 7.2.

Let $a_1 = 0$ code a and $a_1 = 1$ code b ; let $t_1 = 0$ code s_0 and $t_1 = 1$ code s_1 ; and let $w_1 = 0, 1$ code the output symbols $0, 1$. Then

t_1	a_1	t'_1	t_1	a_1	w_1
0	0	0	0	0	0
0	1	1	0	1	0
1	0	0	1	0	1
1	1	1	1	1	1

so the minimized equations are

$$t'_1 = a_1, \quad w_1 = t_1.$$

Thus the circuit uses one D flip-flop holding t_1 , the input line a_1 feeding the D input, and the present-state line t_1 feeding the output line w_1 .

7.24. Circuit for Example 7.6.

Encode the four states of Figure 7.4 by

$$t_0 = 00, \quad t_1 = 01, \quad t_2 = 10, \quad t_3 = 11,$$

let $a_1 = 0$ code a and $a_1 = 1$ code b , and let $w_1 = 0, 1$ code the output symbols $0, 1$. To avoid confusion with the state names, write the two state bits as $u_2 u_1$.

The truth tables are

u_2	u_1	a_1	u'_2	u'_1	u_2	u_1	a_1	w_1
0	0	0	0	1	0	0	0	0
0	0	1	0	0	0	0	1	0
0	1	0	1	0	0	1	0	0
0	1	1	0	0	0	1	1	0
1	0	0	1	0	1	0	0	0
1	0	1	1	1	1	0	1	1
1	1	0	0	1	1	1	0	0
1	1	1	0	0	1	1	1	0

and the minimized equations are

$$u'_2 = (u_2 \wedge \neg u_1) \vee (\neg u_2 \wedge u_1 \wedge \neg a_1),$$

$$u'_1 = (\neg u_2 \wedge \neg u_1 \wedge \neg a_1) \vee (u_2 \wedge \neg u_1 \wedge a_1) \vee (u_2 \wedge u_1 \wedge \neg a_1),$$

$$w_1 = u_2 \wedge \neg u_1 \wedge a_1.$$

So the circuit has two D flip-flops holding u_2, u_1 , a combinational network for u'_2, u'_1 , and one output gate computing w_1 .

7.25. Circuit for FST D (Example 7.8).

Encode the states of Figure 7.5 by

$$q_0 = 00, \quad q_1 = 01, \quad q_2 = 10,$$

leaving 11 as a don't-care code. Let $a_1 = 0$ code $\mathbf{a}$ and $a_1 = 1$ code $\mathbf{b}$, and let $u_2 u_1$ again denote the state bits. Then

u_2	u_1	a_1	u'_2	u'_1	u_2	u_1	a_1	w_1
0	0	0	0	1	0	0	0	0
0	0	1	0	0	0	0	1	0
0	1	0	1	0	0	1	0	0
0	1	1	0	0	0	1	1	0
1	0	0	1	0	1	0	0	0
1	0	1	0	0	1	0	1	1

with minimized equations

$$u'_2 = \neg a_1 \wedge (u_1 \vee u_2), \quad u'_1 = \neg u_2 \wedge \neg u_1 \wedge \neg a_1, \quad w_1 = u_2 \wedge a_1.$$

Thus the circuit again has two D flip-flops and one output gate.

7.26. Connected is not sufficient for minimality.

Example. Let M have states $\{s_0, s_1\}$, $\Sigma = \{\mathbf{a}\}$, $\Gamma = \{\mathbf{0}, \mathbf{1}\}$: $\delta(s_0, \mathbf{a}) = s_1$, $\delta(s_1, \mathbf{a}) = s_0$; $\omega(s_0, \mathbf{a}) = \mathbf{0}$, $\omega(s_1, \mathbf{a}) = \mathbf{0}$. Both states output $\mathbf{0}$ on $\mathbf{a}$ and alternate, so $s_0 E_M s_1$. M is connected (both states reachable from s_0) but not reduced (two equivalent states). Hence it is not minimal.

7.27. Two-unit delay Mealy and Moore machines.

$$\overline{\omega}(x_1 \cdots x_m) = \mathbf{x} \mathbf{x} x_1 \cdots x_{m-2}.$$

(a) **Mealy sextuple:**

Let

$$M = \langle \{\mathbf{a}, \mathbf{b}\}, \{\mathbf{a}, \mathbf{b}, \mathbf{x}\}, S, s_{xx}, \delta, \omega \rangle,$$

where

$$S = \{s_{xx}, s_{xa}, s_{xb}, s_{aa}, s_{ab}, s_{ba}, s_{bb}\}.$$

The state remembers the last two input symbols, using $\mathbf{x}$ as a padding symbol before enough input has been seen. The transition function is

$$\delta(s_{uv}, \sigma) = s_{v\sigma},$$

where $u, v \in \{\mathbf{x}, \mathbf{a}, \mathbf{b}\}$ and the only states of this form that occur are the seven listed above. The output function is

$$\omega(s_{uv}, \sigma) = u.$$

Thus the machine prints the symbol seen two steps earlier; from the start state s_{xx} the first two outputs are $\mathbf{x}$.

(b) **Mealy machine diagram:**

The state transition diagram has the seven states above. Every arrow into $s_{v\sigma}$ is labeled σ / u , where u is the first component of the source state s_{uv} . Concretely,

$$\begin{array}{ll} s_{xx} \xrightarrow{\mathbf{a}/\mathbf{x}} s_{xa}, & s_{xx} \xrightarrow{\mathbf{b}/\mathbf{x}} s_{xb}, \\ s_{xa} \xrightarrow{\mathbf{a}/\mathbf{x}} s_{aa}, & s_{xa} \xrightarrow{\mathbf{b}/\mathbf{x}} s_{ab}, \\ s_{xb} \xrightarrow{\mathbf{a}/\mathbf{x}} s_{ba}, & s_{xb} \xrightarrow{\mathbf{b}/\mathbf{x}} s_{bb}, \end{array}$$

and from each state s_{uv} with $u, v \in \{\mathbf{a}, \mathbf{b}\}$,

$$s_{uv} \xrightarrow{\mathbf{a}/u} s_{va}, \quad s_{uv} \xrightarrow{\mathbf{b}/u} s_{vb}.$$

(c) **Moore sextuple:**

A correct Moore machine must distinguish the “first input seen” stage from the “two most recent inputs known” stage. One convenient choice is

$$A = \langle \{\mathbf{a}, \mathbf{b}\}, \{\mathbf{a}, \mathbf{b}, \mathbf{x}\}, Q, q_0, \delta', \eta \rangle,$$

where

$$Q = \{q_0\} \cup \{q_{x,u} \mid u \in \{\mathbf{a}, \mathbf{b}\}\} \cup \{q_{c,uv} \mid c \in \{\mathbf{x}, \mathbf{a}, \mathbf{b}\}, u, v \in \{\mathbf{a}, \mathbf{b}\}\}.$$

The intended meanings are: $q_{x,u}$ means “print $\mathbf{x}$ and remember the one symbol u ,” while $q_{c,uv}$ means “print c and remember the last two input symbols uv .” The output function is

$$\eta(q_0) = \mathbf{x}, \quad \eta(q_{x,u}) = \mathbf{x},$$

$$\eta(q_{c,uv}) = c.$$

The transition function is

$$\delta'(q_0, \sigma) = q_{x,\sigma} \quad (\sigma \in \{\mathbf{a}, \mathbf{b}\}),$$

$$\delta'(q_{x,u}, \sigma) = q_{x,u\sigma} \quad (u, \sigma \in \{\mathbf{a}, \mathbf{b}\}),$$

and, for $c \in \{\mathbf{x}, \mathbf{a}, \mathbf{b}\}$ and $u, v, \sigma \in \{\mathbf{a}, \mathbf{b}\}$,

$$\delta'(q_{c,uv}, \sigma) = q_{u,v\sigma}.$$

This prints $\mathbf{x}$ on the first two inputs and thereafter prints the symbol seen two steps earlier.

(d) **Moore machine diagram:**

The diagram is the graph of the sextuple above. The start state is $q_0/\mathbf{x}$; from it,

$$q_0 \xrightarrow{\mathbf{a}} q_{x,\mathbf{a}}, \quad q_0 \xrightarrow{\mathbf{b}} q_{x,\mathbf{b}}.$$

Next,

$$q_{x,u} \xrightarrow{\sigma} q_{x,u\sigma} \quad (u, \sigma \in \{\mathbf{a}, \mathbf{b}\}),$$

and every state $q_{c,uv}$ has arrows

$$q_{c,uv} \xrightarrow{\mathbf{a}} q_{u,v\mathbf{a}}, \quad q_{c,uv} \xrightarrow{\mathbf{b}} q_{u,v\mathbf{b}}.$$

- 7.28. (a) Take $M_1 = C$ from Example 7.6 and $M_2 = D$ from Example 7.8. These two transducers are equivalent, but C has four states while D has only three, so they cannot be isomorphic. Thus the conclusion of Theorem 7.5 fails if M_1 is not reduced.

- (b) The missing property is that the proposed map μ need not be one-to-one.

- 7.29. (a) Let $M_2 = D$ from Example 7.8. Let M_1 be obtained from D by adding one extra unreachable state r with the same outgoing arrows as q_0 :

$$\delta(r, \mathbf{a}) = q_1, \quad \delta(r, \mathbf{b}) = r, \quad \omega(r, \mathbf{a}) = \omega(r, \mathbf{b}) = \mathbf{0}.$$

Then M_1 and M_2 are equivalent, but they are not isomorphic because M_1 has an extra unreachable state.

- (b) The missing property is that μ need not even be well defined on all of S_1 , since the definition of $\mu(s)$ requires a string reaching s from the start state.

- 7.30. (a) Again take $M_1 = D$ from Example 7.8 and $M_2 = C$ from Example 7.6. They are equivalent, but C is not reduced and has one extra state, so the machines are not isomorphic.

- (b) The missing property is that μ need not be onto S_2 .

- 7.31. (a) Let $M_1 = D$ from Example 7.8, and let M_2 be obtained from D by adjoining one extra unreachable state r with

$$\delta(r, \mathbf{a}) = q_1, \quad \delta(r, \mathbf{b}) = r, \quad \omega(r, \mathbf{a}) = \omega(r, \mathbf{b}) = \mathbf{0}.$$

Then $M_1 \equiv M_2$, but the two machines are not isomorphic because M_2 has an unreachable state that M_1 does not have.

- (b) The missing property is surjectivity of μ onto S_2 .

- 7.32. (a) Let $A = C$ from Example 7.6. The machine C is connected, so $A^c = A$. But C is not reduced, since Example 7.8 shows that it has an equivalent three-state quotient. Hence A is not reduced and A^c is not reduced.

(b) Let A be obtained from D in Example 7.8 by adjoining one extra unreachable state r satisfying

$$\delta(r, \mathbf{a}) = q_1, \quad \delta(r, \mathbf{b}) = r, \quad \omega(r, \mathbf{a}) = \omega(r, \mathbf{b}) = \mathbf{0}.$$

Then r is equivalent to q_0 , so A is not reduced. However, the connected part A^c throws away r and is exactly D , which is reduced.

7.33. Complete proof of Theorem 7.4.

(a) $\bar{\omega}(t, y) = \bar{\omega}_{E_M}([t]_{E_M}, y)$:

Proof. By induction on $|y|$.

Base: If $y = \lambda$, then both sides equal λ .

Step: Let $y = \mathbf{a}x$. Then

$$\bar{\omega}_{E_M}([t]_{E_M}, \mathbf{a}x) = \omega_{E_M}([t]_{E_M}, \mathbf{a}) \cdot \bar{\omega}_{E_M}(\delta_{E_M}([t]_{E_M}, \mathbf{a}), x).$$

By the definitions of ω_{E_M} and δ_{E_M} , this is

$$\omega(t, \mathbf{a}) \cdot \bar{\omega}_{E_M}([\delta(t, \mathbf{a})]_{E_M}, x).$$

Apply the inductive hypothesis to the state $\delta(t, \mathbf{a})$:

$$\bar{\omega}_{E_M}([t]_{E_M}, \mathbf{a}x) = \omega(t, \mathbf{a}) \cdot \bar{\omega}(\delta(t, \mathbf{a}), x) = \bar{\omega}(t, \mathbf{a}x).$$

□

(b) $M /_{E_M} \equiv M$:

Proof. Part (a) with $t = s_0$ gives

$$\bar{\omega}_{E_M}([s_0]_{E_M}, y) = \bar{\omega}(s_0, y)$$

for every input string y . Since $[s_0]_{E_M}$ is the start state of $M /_{E_M}$, the two machines define the same translation. □

(c) $M /_{E_M}$ is reduced:

Proof. Suppose $[s]_{E_M}$ and $[t]_{E_M}$ are equivalent states of $M /_{E_M}$. Then for every $y \in \Sigma^*$,

$$\bar{\omega}_{E_M}([s]_{E_M}, y) = \bar{\omega}_{E_M}([t]_{E_M}, y).$$

By part (a), this implies

$$\bar{\omega}(s, y) = \bar{\omega}(t, y)$$

for every y . Hence $s E_M t$, so $[s]_{E_M} = [t]_{E_M}$. Therefore $M /_{E_M}$ is reduced. □

(d) If M connected, $M /_{E_M}$ connected:

Proof. Let $[t]_{E_M}$ be any state of the quotient. Since M is connected, there exists $x \in \Sigma^*$ with $\bar{\delta}(s_0, x) = t$. We claim that

$$\bar{\delta}_{E_M}([s_0]_{E_M}, x) = [t]_{E_M}.$$

This follows by induction on $|x|$ from the definition of δ_{E_M} . Hence every quotient state is reachable from the quotient start state, so the quotient is connected. □

7.34. (a) f_1 is FTD: Use the Mealy machine

$$M = \langle \{0, 1\}, \{y, n\}, \{\text{start}, \text{read1}, \text{read0}\}, \text{start}, \delta, \omega \rangle,$$

where

$$\delta(\text{start}, 1) = \text{read1}, \quad \delta(\text{start}, 0) = \text{read0},$$

$$\delta(\text{read1}, 0) = \delta(\text{read1}, 1) = \text{read1},$$

$$\delta(\text{read0}, 0) = \delta(\text{read0}, 1) = \text{read0},$$

and

$$\omega(\text{start}, 1) = y, \quad \omega(\text{start}, 0) = n,$$

$$\omega(\text{read1}, 0) = \omega(\text{read1}, 1) = y,$$

$$\omega(\text{read0}, 0) = \omega(\text{read0}, 1) = n.$$

After the first input, the machine enters a state that remembers whether that first symbol was 1 or 0 and keeps printing y or n forever. Hence it computes f_1 .

(b) f_2 is not FTD: Assume f_2 were FTD. Then Exercise 7.36 would apply: for every n and every $x \in \Sigma^n$, the first n letters of $f_2(xy)$ and $f_2(xz)$ must agree for all suffixes y, z .

Take $x = 0^n$. Then

$$f_2(0^n 0) = n^{n+1}, \quad f_2(0^n 1) = y^{n+1}.$$

These two input strings agree on their first n letters, but the first n letters of the outputs are completely different. This contradicts Exercise 7.36. Therefore f_2 is not FTD. $\square$

7.35. f_3 is not FTD.

Assume f_3 were FTD, and let M be a Mealy machine with n states that computes it. Since f_3 depends only on the input length, we may feed M the unary inputs

$$a, a^2, a^3, \dots$$

and examine the resulting infinite output sequence

$$010010001000010^5 10^6 10^7 \dots$$

Among the states reached after reading

$$\lambda, a, a^2, \dots, a^n,$$

some state must repeat. So there exist integers $0 \leq i < j \leq n$ such that

$$\bar{\delta}(s_0, a^i) = \bar{\delta}(s_0, a^j).$$

Let

$$u = a^i, \quad v = a^{j-i}.$$

Then Exercise 7.5 implies that the outputs on the strings

$$u, uv, uv^2, uv^3, \dots$$

are obtained by repeating one fixed output block corresponding to v . Hence the infinite output stream produced while repeatedly reading a would be ultimately periodic.

But the target sequence is not ultimately periodic, because the number of consecutive 0s between successive 1s is

$$1, 2, 3, 4, 5, \dots,$$

which grows without bound. Therefore f_3 cannot be FTD. $\square$

7.36. **Prove: for FTD f , the first n letters of $f(xy)$ equal those of $f(xz)$.**

Proof. Let M be a Mealy machine for f with states S . For any $x \in \Sigma^n$: $\bar{\delta}(s_0, x)$ is some state s . Then $f(xy) = \bar{w}(s_0, xy) = \bar{w}(s_0, x) \cdot \bar{w}(s, y)$ and $f(xz) = \bar{w}(s_0, xz) = \bar{w}(s_0, x) \cdot \bar{w}(s, z)$. The first n letters of both are $\bar{w}(s_0, x)$ (since $|x| = n$ and the first n output letters come from processing x). Hence they agree on the first n letters. $\square$ $\square$

7.37. **Elevator transducer.**

(a) **Input alphabet:** Since several buttons may be active simultaneously, use

$$\Sigma = \{0, 1\}^4,$$

with an input written $\langle u, d, c_1, c_2 \rangle$. Here u is the hall call “up from floor 1,” d is the hall call “down from floor 2,” and c_1, c_2 are the car buttons for floors 1 and 2.

(b) **Output alphabet:**

$$\Gamma = \{\text{close, open, goto1, goto2}\}.$$

(c) **Mealy sextuple:** Let

$$M = \langle \Sigma, \Gamma, S, (1, \text{closed}, \emptyset), \delta, \omega \rangle,$$

where

$$S = \{(f, m, P) \mid f \in \{1, 2\}, m \in \{\text{open, closed}\}, P \subseteq \{1, 2\}\}.$$

Here f is the current floor, m is the door status, and P is the set of pending requests. For an input $x = \langle u, d, c_1, c_2 \rangle$, define

$$R(x) = \{1 \mid u = 1 \text{ or } c_1 = 1\} \cup \{2 \mid d = 1 \text{ or } c_2 = 1\},$$

and let $P^* = P \cup R(x)$. Then:

(i) if $m = \text{open}$, then

$$\delta((f, \text{open}, P), x) = (f, \text{closed}, P^*), \quad \omega((f, \text{open}, P), x) = \text{close};$$

(ii) if $m = \text{closed}$ and $f \in P^*$, then

$$\delta((f, \text{closed}, P), x) = (f, \text{open}, P^* \setminus \{f\}), \quad \omega((f, \text{closed}, P), x) = \text{open};$$

(iii) if $m = \text{closed}$, $f = 1$, and $2 \in P^*$, then

$$\delta((1, \text{closed}, P), x) = (2, \text{closed}, P^* \setminus \{2\}), \quad \omega((1, \text{closed}, P), x) = \text{goto2};$$

(iv) if $m = \text{closed}$, $f = 2$, and $1 \in P^*$, then

$$\delta((2, \text{closed}, P), x) = (1, \text{closed}, P^* \setminus \{1\}), \quad \omega((2, \text{closed}, P), x) = \text{goto1};$$

(v) if $m = \text{closed}$ and $P^* = \emptyset$, then

$$\delta((f, \text{closed}, \emptyset), x) = (f, \text{closed}, \emptyset), \quad \omega((f, \text{closed}, \emptyset), x) = \text{close}.$$

(d) **Mealy machine diagram:** The diagram is exactly the directed graph of the sextuple in part (c): each state is a triple (f, m, P) , and each edge is labeled by the corresponding input/output pair from the five cases above.

(e) **Moore sextuple:** A Moore version can use

$$S' = \{C_{f,P}, O_{f,P} \mid f \in \{1, 2\}, P \subseteq \{1, 2\}\} \cup \{U_P, D_P \mid P \subseteq \{1, 2\}\},$$

where $C_{f,P}$ means “at floor f , doors closed, pending set P ,” $O_{f,P}$ means “doors open,” U_P means “travelling to floor 2,” and D_P means “travelling to floor 1.” The output function is

$$\omega(C_{f,P}) = \text{close}, \quad \omega(O_{f,P}) = \text{open}, \quad \omega(U_P) = \text{goto2}, \quad \omega(D_P) = \text{goto1}.$$

The transition function is the Moore analogue of part (c):

$$\begin{aligned} \delta(O_{f,P}, x) &= C_{f, P \cup R(x)}, \\ \delta(C_{1,P}, x) &= \begin{cases} O_{1, P^* \setminus \{1\}} & \text{if } 1 \in P^*, \\ U_{P^*} & \text{if } 1 \notin P^* \text{ and } 2 \in P^*, \\ C_{1, P^*} & \text{if } P^* = \emptyset, \end{cases} \\ \delta(C_{2,P}, x) &= \begin{cases} O_{2, P^* \setminus \{2\}} & \text{if } 2 \in P^*, \\ D_{P^*} & \text{if } 2 \notin P^* \text{ and } 1 \in P^*, \\ C_{2, P^*} & \text{if } P^* = \emptyset, \end{cases} \end{aligned}$$

$$\delta(U_P, x) = C_{2, (P \cup R(x)) \setminus \{2\}}, \quad \delta(D_P, x) = C_{1, (P \cup R(x)) \setminus \{1\}}.$$

(f) **Moore machine diagram:** The diagram is the directed graph of the Moore sextuple in part (e).

(g) **Circuit:** The Mealy machine in part (c) has 16 states, so four state bits suffice. The input is already given as four bits $\langle u, d, c_1, c_2 \rangle$, and the outputs can be encoded by two bits, for example

$$\text{close} = 00, \quad \text{open} = 01, \quad \text{goto1} = 10, \quad \text{goto2} = 11.$$

The required circuit is therefore the standard four-flip-flop sequential network implementing the transition and output rules from part (c).

7.38. Mealy machine for traffic signal (Example 7.16).

Let

$$\Sigma = \{\langle 0, 0 \rangle, \langle 0, 1 \rangle, \langle 1, 0 \rangle, \langle 1, 1 \rangle\}$$

on the sensor pair $\langle \alpha, \beta \rangle$. A correct Mealy machine is obtained from the Moore machine of Figure 7.15 by placing the output of the destination state on each edge. Use the state set $\{s_0, s_1, s_2, s_3, s_4, s_5, s_6, s_7\}$ with

$$\begin{aligned} o(s_0) &= \langle G, R, G, R \rangle, & o(s_1) &= \langle Y, R, Y, R \rangle, \\ o(s_2) &= \langle R, R, R, G \rangle, & o(s_3) &= \langle G, R, Y, R \rangle, \\ o(s_4) &= \langle R, R, R, Y \rangle, & o(s_5) &= \langle G, Y, R, R \rangle, \\ o(s_6) &= \langle G, G, R, R \rangle, & o(s_7) &= \langle Y, Y, R, R \rangle. \end{aligned}$$

The transition table is

	$\langle 0, 0 \rangle$	$\langle 0, 1 \rangle$	$\langle 1, 0 \rangle$	$\langle 1, 1 \rangle$
s_0	s_0	s_1	s_3	s_1
s_1	s_2	s_2	s_2	s_2
s_2	s_4	s_2	s_4	s_2
s_3	s_6	s_6	s_6	s_6
s_4	s_0	s_0	s_6	s_6
s_5	s_0	s_0	s_0	s_0
s_6	s_5	s_7	s_6	s_6
s_7	s_2	s_2	s_2	s_2

and the Mealy output on any transition (s, x) is

$$\omega(s, x) = o(\delta(s, x)).$$

7.39. Traffic signal with walk signals.

(a) The new alphabets are

$$\Sigma = \{0, 1\}^3$$

on $\langle \alpha, \beta, \gamma \rangle$, and

$$\Gamma = \{R, Y, G\}^4 \times \{D, W\}.$$

(b) **Moore machine:** Use the states

$$s_0, s_1^D, s_1^W, s_2^D, s_2^W, s_3, s_4, s_5, s_6, s_7,$$

with outputs

$$\begin{aligned} \omega(s_0) &= \langle G, R, G, R, D \rangle, & \omega(s_1^D) &= \langle Y, R, Y, R, D \rangle, \\ \omega(s_1^W) &= \langle Y, R, Y, R, D \rangle, & \omega(s_2^D) &= \langle R, R, R, G, D \rangle, \\ \omega(s_2^W) &= \langle R, R, R, G, W \rangle, & \omega(s_3) &= \langle G, R, Y, R, D \rangle, \\ \omega(s_4) &= \langle R, R, R, Y, D \rangle, & \omega(s_5) &= \langle G, Y, R, R, D \rangle, \\ \omega(s_6) &= \langle G, G, R, R, D \rangle, & \omega(s_7) &= \langle Y, Y, R, R, D \rangle. \end{aligned}$$

The transition policy is:

- (i) from s_0 , stay in s_0 on 000; go to s_3 on 100; and whenever $\beta = 1$ or $\gamma = 1$, go to s_1^W if $\gamma = 1$ and to s_1^D otherwise;
- (ii) s_1^D goes to s_2^D on every input, and s_1^W goes to s_2^W on every input;
- (iii) s_2^W goes to s_2^D on every input, so the walk signal reverts to D before the side-street light turns yellow;

- (iv) s_2^D stays in s_2^D while $\beta = 1$ and goes to s_4 when $\beta = 0$;
 - (v) s_4 goes to s_6 if $\alpha = 1$ and to s_0 otherwise;
 - (vi) s_3 goes to s_6 on every input;
 - (vii) s_6 stays in s_6 while $\alpha = 1$; if $\alpha = 0$ and either $\beta = 1$ or $\gamma = 1$, it goes to s_7 ; otherwise it goes to s_5 ;
 - (viii) s_5 goes to s_0 on every input, and s_7 goes to s_2^W when $\gamma = 1$ and to s_2^D otherwise.
- (c) **Mealy machine:** Take the state set and transition graph from part (b), and place on each edge the output of the destination state. If $o(t)$ denotes the five-component output attached to the Moore state t , then

$$\omega(s, x) = o(\delta(s, x)).$$

7.40. Intersection with left-turn signals.

Let

$$\Sigma = \{0, 1\}^4$$

on $\langle \alpha, \beta, \gamma, \delta \rangle$, and let

$$\Gamma = \{R, Y, G\}^8$$

with coordinates ordered by signals 1, 2, 3, 4, 5, 6, 7, 8 in Figure 7.20.

A convenient Moore controller uses the twelve phase states

$$EW_G, EW_Y, NS_G, NS_Y, WB_G, WB_Y, NB_G, NB_Y, EB_G, EB_Y, SB_G, SB_Y,$$

with outputs

$$\begin{aligned} EW_G &= \langle G, R, G, R, R, R, R, R \rangle, & EW_Y &= \langle Y, R, Y, R, R, R, R, R \rangle, \\ NS_G &= \langle R, G, R, G, R, R, R, R \rangle, & NS_Y &= \langle R, Y, R, Y, R, R, R, R \rangle, \\ WB_G &= \langle G, R, R, R, G, R, R, R \rangle, & WB_Y &= \langle Y, R, R, R, Y, R, R, R \rangle, \\ NB_G &= \langle R, G, R, R, R, G, R, R \rangle, & NB_Y &= \langle R, Y, R, R, R, Y, R, R \rangle, \\ EB_G &= \langle R, R, G, R, R, R, G, R \rangle, & EB_Y &= \langle R, R, Y, R, R, R, Y, R \rangle, \\ SB_G &= \langle R, R, R, G, R, R, R, G \rangle, & SB_Y &= \langle R, R, R, Y, R, R, R, Y \rangle. \end{aligned}$$

One explicit transition policy is:

- (i) start in EW_G ; remain there while no sensor is active, and move to EW_Y as soon as some request appears;
- (ii) from EW_Y , give priority to the two sensor-controlled lanes on the north-south side: go to NB_G if $\beta = 1$, else to SB_G if $\delta = 1$, else to NS_G ;
- (iii) NB_G stays in NB_G while $\beta = 1$, then goes to NB_Y , and NB_Y goes to SB_G if $\delta = 1$ and otherwise to NS_G ;
- (iv) SB_G stays in SB_G while $\delta = 1$, then goes to SB_Y , and SB_Y goes to NS_G ;
- (v) NS_G remains in NS_G while $\alpha = \gamma = 0$, and moves to NS_Y once some east-west left-turn request appears;
- (vi) from NS_Y , give priority first to α and then to γ : go to WB_G if $\alpha = 1$, else to EB_G if $\gamma = 1$, else to EW_G ;

- (vii) WB_G stays in WB_G while $\alpha = 1$, then goes to WB_Y , and WB_Y goes to EB_G if $\gamma = 1$ and otherwise to EW_G ;
- (viii) EB_G stays in EB_G while $\gamma = 1$, then goes to EB_Y , and EB_Y goes to EW_G .

This gives a complete Moore machine. The corresponding Mealy machine uses the same graph and places the output of the destination phase on each transition.

7.41. **Base-3 adder.**

- (a) **Input alphabet:**

$$\Sigma = \{0, 1, 2\} \times \{0, 1, 2\}.$$

- (b) **Output alphabet:**

$$\Gamma = \{0, 1, 2\}.$$

- (c) **Mealy machine:** Let

$$M = \langle \Sigma, \Gamma, \{C_0, C_1\}, C_0, \delta, \omega \rangle,$$

where C_0 means “no carry” and C_1 means “carry 1.” For $\langle a, b \rangle \in \Sigma$, define

$$\delta(C_i, \langle a, b \rangle) = \begin{cases} C_1 & \text{if } a + b + i \geq 3, \\ C_0 & \text{if } a + b + i \leq 2, \end{cases}$$

and

$$\omega(C_i, \langle a, b \rangle) \equiv a + b + i \pmod{3}.$$

This is exactly the base-3 analogue of the binary adder in Example 7.15.

- (d) **Moore machine:** A corresponding Moore machine is

$$A = \langle \Sigma, \Gamma, S, q_*, \delta', \eta \rangle,$$

where

$$S = \{q_*\} \cup \{q_{ij} \mid i \in \{0, 1\}, j \in \{0, 1, 2\}\},$$

the output function is

$$\eta(q_*) = 0, \quad \eta(q_{ij}) = j,$$

and the transition function is

$$\delta'(q_*, \langle a, b \rangle) = q_{c,s},$$

$$\delta'(q_{ij}, \langle a, b \rangle) = q_{c',s'},$$

where

$$s \equiv a + b \pmod{3}, \quad c = \begin{cases} 1 & \text{if } a + b \geq 3, \\ 0 & \text{if } a + b \leq 2, \end{cases}$$

and

$$s' \equiv a + b + i \pmod{3}, \quad c' = \begin{cases} 1 & \text{if } a + b + i \geq 3, \\ 0 & \text{if } a + b + i \leq 2. \end{cases}$$

Thus the Moore state records both the current carry and the digit that is to be printed.

- (e) **Circuit with <EOS> and <SOS>:** Encode each trit by two bits, for example 0 = 00, 1 = 01, 2 = 10, and encode a column $\langle a, b \rangle$ by the four bits of the two trits. Reserve extra input codes for <SOS> and <EOS>. One flip-flop stores the carry bit c . The <SOS> input forces $c = 0$. On an ordinary input column $\langle a, b \rangle$, the combinational network computes

$$s \equiv a + b + c \pmod{3}, \quad c' = \begin{cases} 1 & \text{if } a + b + c \geq 3, \\ 0 & \text{if } a + b + c \leq 2, \end{cases}$$

and sends the two-bit code of s to the output device. On <EOS>, the circuit outputs the final carry digit: print 1 if $c = 1$ and 0 if $c = 0$, then reset the flip-flop to 0. This is the required circuit implementation of the Mealy machine in part (c).

7.42. Base-10 adder.

- (a) **Input and output alphabets:**

$$\Sigma = \{0, 1, \dots, 9\} \times \{0, 1, \dots, 9\}, \quad \Gamma = \{0, 1, \dots, 9\}.$$

As in Example 7.15, one input symbol is a column of two digits, and the transducer reads the columns from right to left.

- (b) **Mealy sextuple:**

Let

$$M = \langle \Sigma, \Gamma, \{N, C\}, N, \delta, \omega \rangle,$$

where N means “no carry” and C means “carry.” For $(i, j) \in \Sigma$, define

$$\omega(N, (i, j)) = (i + j) \bmod 10, \quad \delta(N, (i, j)) = \begin{cases} C & \text{if } i + j \geq 10, \\ N & \text{if } i + j < 10, \end{cases}$$

and

$$\omega(C, (i, j)) = (i + j + 1) \bmod 10, \quad \delta(C, (i, j)) = \begin{cases} C & \text{if } i + j + 1 \geq 10, \\ N & \text{if } i + j + 1 < 10. \end{cases}$$

- (c) **Moore sextuple:**

A convenient Moore version uses one start state s together with states

$$q_{c,d} \quad (c \in \{0, 1\}, d \in \{0, 1, \dots, 9\}),$$

where c records the carry to be used on the next column and d is the most recently produced output digit. Let

$$A = \langle \Sigma, \Gamma, \{s\} \cup \{q_{c,d} \mid c \in \{0, 1\}, 0 \leq d \leq 9\}, s, \delta', \eta \rangle.$$

The output function is

$$\eta(q_{c,d}) = d,$$

while the value of $\eta(s)$ is irrelevant because the start state's output is not used on empty input.

For $(i, j) \in \Sigma$, define

$$\delta'(s, (i, j)) = q_{c',d'},$$

where

$$d' = (i + j) \bmod 10, \quad c' = \begin{cases} 1 & \text{if } i + j \geq 10, \\ 0 & \text{if } i + j < 10, \end{cases}$$

and in general

$$\delta'(q_{c,d}, (i, j)) = q_{c',d'},$$

where

$$d' = (i + j + c) \bmod 10, \quad c' = \begin{cases} 1 & \text{if } i + j + c \geq 10, \\ 0 & \text{if } i + j + c < 10. \end{cases}$$

(d) **Circuit:**

Using <EOS> and <SOS>, encode each decimal digit in BCD and use one extra carry flip-flop. The combinational part computes

$$s = i + j + c,$$

outputs the BCD code for $s \bmod 10$, and feeds back the carry bit indicating whether $s \geq 10$. The <SOS> input initializes the carry flip-flop to 0, and the <EOS> input can be used to report whether a final overflow carry is present.

7.43. **Left-to-right addition is not FTD.**

Assume for contradiction that the left-to-right addition function f_4 is FTD. Then Exercise 7.36 applies: for every n , whenever two inputs agree on their first n symbols, the first n output symbols must also agree.

Fix any $n \geq 1$. Consider the following two length- $(n+1)$ input strings of column pairs:

$$x = \langle 0, 0 \rangle \langle 0, 1 \rangle^{n-1} \langle 1, 1 \rangle \quad \text{and} \quad y = \langle 0, 0 \rangle \langle 0, 1 \rangle^{n-1} \langle 0, 1 \rangle.$$

These two inputs agree on their first n symbols. However, their first output symbols are different when the addition is carried out correctly from right to left:

- For x , the last column $\langle 1, 1 \rangle$ generates a carry. Each middle column $\langle 0, 1 \rangle$ propagates that carry, so a carry reaches the first column $\langle 0, 0 \rangle$. Hence the first output bit is 1.
- For y , the last column $\langle 0, 1 \rangle$ generates no carry, so no carry reaches the first column. Hence the first output bit is 0.

Thus the first output symbols of $f_4(x)$ and $f_4(y)$ differ, even though the first n input symbols of x and y are identical. This contradicts Exercise 7.36. Therefore f_4 is not FTD.

7.44. **Prove Theorem 7.9 for MSM.**

(a) $M /_{E_M} \equiv M$:

Proof. As in the Mealy case, one first proves by induction on $|y|$ that for every state t of M ,

$$\bar{\omega}_M(t, y) = \bar{\omega}_{E_M}([t]_{E_M}, y).$$

The base case is immediate since both sides are λ on empty input. For the inductive step, use the Moore-machine definition of $\bar{\omega}$, together with the facts that

$$\delta_{E_M}([s]_{E_M}, \mathbf{a}) = [\delta(s, \mathbf{a})]_{E_M} \quad \text{and} \quad \omega_{E_M}([s]_{E_M}) = \omega(s).$$

Taking $t = s_0$ yields $f_{M /_{E_M}} = f_M$, so the quotient is equivalent to M . □

(b) M/E_M is reduced:

Proof. Suppose $[s]_{E_M}$ and $[t]_{E_M}$ are equivalent states of the quotient. Then

$$\bar{\omega}_{E_M}([s]_{E_M}, y) = \bar{\omega}_{E_M}([t]_{E_M}, y)$$

for all $y \in \Sigma^*$. By part (a), this implies $\bar{\omega}_M(s, y) = \bar{\omega}_M(t, y)$ for all y , so $sE_M t$. Therefore $[s]_{E_M} = [t]_{E_M}$, and the quotient is reduced. $\square$

(c) If M connected, M/E_M connected:

Proof. Let $[t]_{E_M}$ be any state of the quotient. Since M is connected, there exists $x \in \Sigma^*$ with $\bar{\delta}_M(s_0, x) = t$. By induction on $|x|$,

$$\bar{\delta}_{E_M}([s_0]_{E_M}, x) = [\bar{\delta}_M(s_0, x)]_{E_M} = [t]_{E_M}.$$

Thus every quotient state is reachable, so the quotient is connected. $\square$

7.45. Minimal Moore machine is reduced.

Proof. Suppose the minimal Moore machine M for f is not reduced. Then two states $s \neq t$ in M have $sE_M t$. Form M' by merging $[s] = [t]$ in the quotient M/E_M ; this gives an equivalent machine with fewer states. But M was defined as minimal—contradiction. Hence M is reduced. $\square$ $\square$

7.46. Theorem 7.10 for MSM.

(a) M minimal iff M is reduced and connected:

Proof. If M is minimal, then it must be connected, since unreachable states can be deleted without changing the translation. It must also be reduced by Theorem 7.9(d). Hence minimality implies reduced and connected.

Conversely, assume M is reduced and connected. Let B be any Moore machine equivalent to M . Replace B by B^c/E_{B^c} if necessary. By Theorem 7.9(a)–(c), this replacement is equivalent to B , reduced, and connected, and it has no more states than B . So it is enough to prove that every reduced connected Moore machine equivalent to M has at least as many states as M .

Let B now be reduced, connected, and equivalent to M . Since M is connected, for each state $s \in S_M$ choose a string $x_s \in \Sigma^*$ such that

$$\bar{\delta}_M(s_0, x_s) = s.$$

Define

$$\mu(s) = \bar{\delta}_B(s_0, x_s).$$

Since M is connected, such a string x_s exists for every $s \in S_M$; hence this rule assigns a value to every state of M . We show that μ is well defined and one-to-one.

First, suppose x_s and y_s both reach the same state s in M . Put

$$p = \bar{\delta}_B(s_0, x_s), \quad q = \bar{\delta}_B(s_0, y_s).$$

We show that p and q have the same output behavior. Let $z \in \Sigma^*$ be arbitrary. By Theorem 7.6,

$$\bar{\omega}_M(s_{0_M}, x_s z) = \bar{\omega}_M(s_{0_M}, x_s) \cdot \bar{\omega}_M(s, z)$$

and

$$\bar{\omega}_B(s_{0_B}, x_s z) = \bar{\omega}_B(s_{0_B}, x_s) \cdot \bar{\omega}_B(p, z).$$

Since M and B are equivalent, the left sides are equal; applying equivalence also to the prefix x_s gives $\bar{\omega}_M(s_{0_M}, x_s) = \bar{\omega}_B(s_{0_B}, x_s)$. Canceling this common prefix leaves

$$\bar{\omega}_M(s, z) = \bar{\omega}_B(p, z).$$

The same argument with y_s gives

$$\bar{\omega}_M(s, z) = \bar{\omega}_B(q, z).$$

Hence $\bar{\omega}_B(p, z) = \bar{\omega}_B(q, z)$ for every $z \in \Sigma^*$, so $p E_B q$. Since B is reduced, $p = q$. Therefore the value of $\mu(s)$ does not depend on which reaching string is chosen, and μ is well defined.

Next, suppose $\mu(s) = \mu(t)$. Then for every $z \in \Sigma^*$,

$$\bar{\omega}_M(s, z) = \bar{\omega}_B(\mu(s), z) = \bar{\omega}_B(\mu(t), z) = \bar{\omega}_M(t, z),$$

so $s E_M t$. Because M is reduced, $s = t$. Therefore μ is injective.

Since μ is an injective map from S_M into S_B ,

$$\|S_M\| \leq \|S_B\|.$$

Hence no equivalent Moore machine can have fewer states than M , and M is minimal. $\square$

(b) $M^c /_{E_{M^c}}$ is minimal:

Proof. By Theorem 7.9, the quotient $M^c /_{E_{M^c}}$ is equivalent to M^c , reduced, and connected. Since M^c is equivalent to M , the quotient is also equivalent to M . Part (a) now implies that $M^c /_{E_{M^c}}$ is minimal. $\square$

7.47. Prove Lemma 7.5 and Corollary 7.9 for MSMs.

(a) $(\forall s \in S_A)(\forall x \in \Sigma^*)(\mu(\bar{\delta}_A(s, x)) = \bar{\delta}_B(\mu(s), x))$.

Proof. By induction on $|x|$.

Base: If $x = \lambda$, then $\mu(\bar{\delta}_A(s, \lambda)) = \mu(s) = \bar{\delta}_B(\mu(s), \lambda)$.

Step: Let $x = \mathbf{a}y$. Then

$$\mu(\bar{\delta}_A(s, \mathbf{a}y)) = \mu(\bar{\delta}_A(\delta_A(s, \mathbf{a}), y)) = \bar{\delta}_B(\mu(\delta_A(s, \mathbf{a})), y),$$

where the last equality is the inductive hypothesis. By the homomorphism property,

$$\bar{\delta}_B(\mu(\delta_A(s, \mathbf{a})), y) = \bar{\delta}_B(\delta_B(\mu(s), \mathbf{a}), y) = \bar{\delta}_B(\mu(s), \mathbf{a}y).$$

$\square$

(b) $(\forall s \in S_A)(\forall x \in \Sigma^*)(\bar{\omega}_A(s, x) = \bar{\omega}_B(\mu(s), x)).$

Proof. By induction on $|x|$.

Base: If $x = \lambda$, then both sides equal λ .

Step: Let $x = \mathbf{a}y$. By Definition 7.17,

$$\bar{\omega}_A(s, \mathbf{a}y) = \omega_A(\delta_A(s, \mathbf{a})) \cdot \bar{\omega}_A(\delta_A(s, \mathbf{a}), y).$$

Since μ is a Moore homomorphism,

$$\omega_A(\delta_A(s, \mathbf{a})) = \omega_B(\mu(\delta_A(s, \mathbf{a}))) = \omega_B(\delta_B(\mu(s), \mathbf{a})).$$

Applying the inductive hypothesis to the state $\delta_A(s, \mathbf{a})$ gives

$$\bar{\omega}_A(s, \mathbf{a}y) = \omega_B(\delta_B(\mu(s), \mathbf{a})) \cdot \bar{\omega}_B(\delta_B(\mu(s), \mathbf{a}), y) = \bar{\omega}_B(\mu(s), \mathbf{a}y).$$

□

(c) $A \equiv B$:

Proof. Taking $s = s_{0_A}$ in part (b), and using $\mu(s_{0_A}) = s_{0_B}$, we obtain

$$\bar{\omega}_A(s_{0_A}, x) = \bar{\omega}_B(s_{0_B}, x)$$

for every $x \in \Sigma^*$. Hence $f_A = f_B$.

□

7.48. Prove Corollary 7.10.

Proof. Start with E_{0M} , which groups together exactly those states that print the same output symbol. By Lemma 7.6(b), compute successively $E_{1M}, E_{2M}, \dots$ from the transition function. By Lemma 7.3(d), some equality $E_{mM} = E_{m+1,M}$ must occur after finitely many steps, and by Lemma 7.3(e), that stabilized relation is exactly E_M . Thus there is an algorithm for computing E_M . □ □

7.49. Prove Corollary 7.11.

Proof. Compute the connected part M^c by starting with the set containing only the start state and repeatedly adjoining every state reached by one transition from states already found. Then use Corollary 7.10 to compute E_{M^c} . Finally, form the quotient M^c / E_{M^c} . By Theorem 7.10(b), this quotient is the minimal Moore machine equivalent to M . Hence there is an algorithm for computing the minimal machine equivalent to M . □ □

7.50. δ_{E_M} and ω_{E_M} well-defined for FST quotient.

(a) δ_{E_M} is well-defined:

Proof. Define

$$\delta_{E_M}([s]_{E_M}, \mathbf{a}) = [\delta(s, \mathbf{a})]_{E_M}.$$

This rule is defined for every argument: each element of S_{E_M} has the form $[s]_{E_M}$ for some $s \in S$, and the original transition function δ is defined for every pair $(s, \mathbf{a}) \in S \times \Sigma$. If $[s]_{E_M} = [t]_{E_M}$, then $s E_M t$. By Lemma 7.3(c),

$$\delta(s, \mathbf{a}) E_M \delta(t, \mathbf{a}),$$

so

$$[\delta(s, \mathbf{a})]_{E_M} = [\delta(t, \mathbf{a})]_{E_M}.$$

Therefore δ_{E_M} does not depend on the chosen representative. $\square$

(b) ω_{E_M} is well-defined:

Proof. Define

$$\omega_{E_M}([s]_{E_M}, \mathbf{a}) = \omega(s, \mathbf{a}).$$

This rule is defined for every argument because every quotient state has a representative $s \in S$, and the original output function ω is defined for every pair $(s, \mathbf{a}) \in S \times \Sigma$. If $[s]_{E_M} = [t]_{E_M}$, then $s E_M t$, so the states s and t agree on every one-letter input. In particular,

$$\omega(s, \mathbf{a}) = \omega(t, \mathbf{a}).$$

Hence ω_{E_M} is independent of the representative. $\square$

7.51. δ_{E_M} and ω_{E_M} well-defined for MSM quotient.

(a) δ_{E_M} is well-defined:

Proof. Define

$$\delta_{E_M}([s]_{E_M}, \mathbf{a}) = [\delta(s, \mathbf{a})]_{E_M}.$$

This rule is defined for every argument: each quotient state is $[s]_{E_M}$ for some $s \in S$, and the Moore-machine transition function δ is defined on all of $S \times \Sigma$. Suppose $[s]_{E_M} = [t]_{E_M}$, so $s E_M t$. Then s and t produce the same output on every continuation. In particular, for every $x \in \Sigma^*$,

$$\overline{\omega}(s, \mathbf{a}x) = \overline{\omega}(t, \mathbf{a}x).$$

By the Moore-machine definition of $\overline{\omega}$, this implies

$$\omega(\delta(s, \mathbf{a})) \cdot \overline{\omega}(\delta(s, \mathbf{a}), x) = \omega(\delta(t, \mathbf{a})) \cdot \overline{\omega}(\delta(t, \mathbf{a}), x).$$

Taking $x = \lambda$ first shows that $\omega(\delta(s, \mathbf{a})) = \omega(\delta(t, \mathbf{a}))$. Therefore, for arbitrary x , the common first output symbol may be canceled from the two sides of the displayed equality, giving

$$\overline{\omega}(\delta(s, \mathbf{a}), x) = \overline{\omega}(\delta(t, \mathbf{a}), x).$$

Hence $\delta(s, \mathbf{a}) E_M \delta(t, \mathbf{a})$, and so

$$[\delta(s, \mathbf{a})]_{E_M} = [\delta(t, \mathbf{a})]_{E_M}.$$

Therefore δ_{E_M} does not depend on the representative chosen. $\square$

(b) ω_{E_M} is well-defined:

Proof. Define

$$\omega_{E_M}([s]_{E_M}) = \omega(s).$$

This rule is defined for every argument because every quotient state has a representative $s \in S$, and the original Moore output function ω is defined for every state in S . If $[s]_{E_M} = [t]_{E_M}$, then $sE_M t$. By Lemma 7.3(b), the full state equivalence relation E_M is a refinement of E_{0M} , so $sE_{0M} t$. Lemma 7.6(a) says that $sE_{0M} t$ is exactly the statement that the two states have the same Moore output. Hence

$$\omega(s) = \omega(t).$$

Thus ω_{E_M} is independent of the chosen representative. □

7.52. Isomorphism from A to B and A connected $\Rightarrow B$ connected.

- (a) **Mealy machines:** Let $\psi : S_A \rightarrow S_B$ be an isomorphism. Every $t \in S_B$ has a preimage $s = \psi^{-1}(t)$. Since A is connected, $s = \bar{\delta}_A(s_{0_A}, x)$ for some x . Then $t = \psi(s) = \psi(\bar{\delta}_A(s_{0_A}, x)) = \bar{\delta}_B(\psi(s_{0_A}), x) = \bar{\delta}_B(s_{0_B}, x)$. So t is reachable in B .
- (b) **Moore machines:** Let $\psi : S_A \rightarrow S_B$ be a Moore isomorphism. For any state $t \in S_B$, choose $s = \psi^{-1}(t)$. Since A is connected, there exists $x \in \Sigma^*$ with

$$s = \bar{\delta}_A(s_{0_A}, x).$$

Because an isomorphism commutes with the transition function,

$$t = \psi(s) = \psi(\bar{\delta}_A(s_{0_A}, x)) = \bar{\delta}_B(\psi(s_{0_A}), x) = \bar{\delta}_B(s_{0_B}, x).$$

Thus every state of B is reachable, so B is connected.

7.53. Isomorphism from A to B and B connected $\Rightarrow A$ connected.

- (a) **Mealy machines:** Let $\psi : A \rightarrow B$ be an isomorphism, and let $s \in S_A$. Put $t = \psi(s) \in S_B$. Since B is connected, there exists $x \in \Sigma^*$ with

$$t = \bar{\delta}_B(s_{0_B}, x).$$

Because ψ is bijective, ψ^{-1} exists, and because ψ preserves transitions, ψ^{-1} does as well. Hence

$$s = \psi^{-1}(t) = \psi^{-1}(\bar{\delta}_B(s_{0_B}, x)) = \bar{\delta}_A(\psi^{-1}(s_{0_B}), x) = \bar{\delta}_A(s_{0_A}, x).$$

So every state of A is reachable, and A is connected.

- (b) **Moore machines:** Let $\psi : A \rightarrow B$ be a Moore isomorphism. Since ψ is bijective and preserves the start state and all transitions, its inverse $\psi^{-1} : B \rightarrow A$ also preserves the start state and transitions. Applying the reachability argument above to ψ^{-1} shows that every state of A is reachable from s_{0_A} . Thus A is connected.

7.54. Homomorphism from A to B and A connected does NOT imply B connected.

- (a) **Mealy counterexample:** Let A have state $\{s_0\}$ (connected), $\Sigma = \{a\}$, $\delta_A(s_0, a) = s_0$, $\omega_A(s_0, a) = \mathbf{0}$. Let B have states $\{t_0, t_1\}$ where t_1 is unreachable from t_0 ; $\delta_B(t_0, a) = t_0$, $\omega_B(t_0, a) = \mathbf{0}$, $\delta_B(t_1, a) = t_1$, $\omega_B(t_1, a) = \mathbf{0}$. Map $\mu(s_0) = t_0$. Then μ is a homomorphism, A is connected, but B is not connected (t_1 unreachable).
- (b) **Moore machines:** Use the same state graph and the same map $\mu(s_0) = t_0$, but interpret outputs as state outputs rather than transition outputs. Then μ is again a homomorphism from a connected machine onto a machine with an unreachable state.

7.55. **Homomorphism from A to B and B connected does NOT imply A connected.**

- (a) **Counterexample:** Let A have states $\{s_0, s_1\}$ where s_1 is unreachable; B has state $\{t_0\}$ (trivially connected). Map $\mu(s_0) = \mu(s_1) = t_0$. μ is a homomorphism, B is connected, but A is not connected.
- (b) **Moore machines:** Use the same two-state domain and one-state codomain, but interpret outputs as state outputs. The map sending both states of A to the single state of B is again a homomorphism, while A remains disconnected.

7.56. **If $\psi : A \rightarrow B$ and $\mu : B \rightarrow A$ are isomorphisms of connected FSTs, then $\psi = \mu^{-1}$.**

Proof. $\mu \circ \psi : S_A \rightarrow S_A$ is an isomorphism (composition of isomorphisms). For any $s \in S_A$ reachable via x ($s = \bar{\delta}_A(s_0, x)$): $(\mu \circ \psi)(s) = \mu(\psi(\bar{\delta}_A(s_0, x))) = \mu(\bar{\delta}_B(s_0, x)) = \bar{\delta}_A(s_0, x) = s$. So $\mu \circ \psi = \text{id}_{S_A}$, i.e., $\mu = \psi^{-1}$. $\square$

7.57. **If A, B are FSTs (possibly not connected), $\psi = \mu^{-1}$ may fail.**

Example: Let $A = B$ have states $\{s_0, u, v\}$, start state s_0 , alphabet $\{a\}$, and output alphabet $\{\mathbf{0}\}$, with

$$\delta(s_0, a) = s_0, \quad \delta(u, a) = u, \quad \delta(v, a) = v,$$

and every transition printing $\mathbf{0}$. The states u and v are both unreachable from the start state, so the machines are not connected. Now

$$\psi = \text{id}_{\{s_0, u, v\}}$$

is an isomorphism from A to B , and so is

$$\mu(s_0) = s_0, \quad \mu(u) = v, \quad \mu(v) = u.$$

But $\psi^{-1} = \psi \neq \mu$. Thus the conclusion of Exercise 7.56 fails without connectedness.

7.58. **Three-state MSM with E_{0A} having one class; can $E_{0A} \neq E_{1A}$?**

Yes, such a three-state Moore machine exists: take

$$A = \langle \{a\}, \{\mathbf{0}\}, \{r_0, r_1, r_2\}, r_0, \delta, \omega \rangle,$$

where

$$\omega(r_0) = \omega(r_1) = \omega(r_2) = \mathbf{0}.$$

and

$$\delta(r_0, a) = r_1, \quad \delta(r_1, a) = r_2, \quad \delta(r_2, a) = r_0.$$

Then Lemma 7.6(a) gives

$$E_{0A} = \{(r_i, r_j) \mid 0 \leq i, j \leq 2\},$$

so E_{0A} has exactly one equivalence class.

However, in this situation it is *not* possible for E_{0A} to differ from E_{1A} . By Lemma 7.6(b),

$$sE_{1A}t \iff sE_{0A}t \wedge (\forall \mathbf{a} \in \Sigma)(\delta(s, \mathbf{a})E_{0A}\delta(t, \mathbf{a})).$$

If E_{0A} has only one equivalence class, then every pair of successor states is automatically E_{0A} -related. Hence every pair of states is also E_{1A} -related, so in fact

$$E_{1A} = E_{0A}.$$

7.59. (a) Mealy M not connected and M/E_M not connected: take

$$M = \langle \{\mathbf{a}\}, \{\mathbf{0}, \mathbf{1}\}, \{s_0, s_1, s_2\}, s_0, \delta, \omega \rangle$$

where

$$\delta(s_0, \mathbf{a}) = s_1, \quad \delta(s_1, \mathbf{a}) = s_1, \quad \delta(s_2, \mathbf{a}) = s_2,$$

and

$$\omega(s_0, \mathbf{a}) = \mathbf{0}, \quad \omega(s_1, \mathbf{a}) = \mathbf{0}, \quad \omega(s_2, \mathbf{a}) = \mathbf{1}.$$

The state s_2 is unreachable, so M is not connected. Also $s_2 \not\sim_M s_0$ and $s_2 \not\sim_M s_1$, because on the one-letter input $\mathbf{a}$ it outputs $\mathbf{1}$ while the reachable states output $\mathbf{0}$. Hence $[s_2]$ remains a separate unreachable state in the quotient, so M/E_M is not connected.

(b) Mealy M not connected but M/E_M connected: take

$$M = \langle \{\mathbf{a}\}, \{\mathbf{0}\}, \{t_0, t_1, t_2\}, t_0, \delta, \omega \rangle$$

where every state loops on $\mathbf{a}$ and always outputs $\mathbf{0}$:

$$\delta(t_j, \mathbf{a}) = t_j, \quad \omega(t_j, \mathbf{a}) = \mathbf{0} \quad (j = 0, 1, 2).$$

The states t_1 and t_2 are unreachable from t_0 , so M is not connected. But all three states are equivalent, since every input word produces the same output word $\mathbf{0}^{|x|}$ no matter where we start. Therefore

$$[t_0] = [t_1] = [t_2],$$

so M/E_M has just one state and is therefore connected.

7.60. (a) Moore M not connected and M/E_M not connected: take a machine with states $\{r_0, r_1, r_2\}$, start state r_0 , alphabet $\{\mathbf{a}\}$, outputs

$$\omega(r_0) = \mathbf{0}, \quad \omega(r_1) = \mathbf{1}, \quad \omega(r_2) = \mathbf{0},$$

and transitions

$$\delta(r_0, \mathbf{a}) = r_1, \quad \delta(r_1, \mathbf{a}) = r_1, \quad \delta(r_2, \mathbf{a}) = r_2.$$

The state r_2 is unreachable, so M is not connected. Also, r_2 is not equivalent to either reachable state: it outputs $\mathbf{0}$ forever, while r_0 leads to $\mathbf{1}$ after one step and r_1 already outputs $\mathbf{1}$. Hence the quotient still has an unreachable class, so M/E_M is not connected.

- (b) Moore M not connected but M/E_M connected: take states $\{r_0, r_1\}$ with start state r_0 , alphabet $\{\mathbf{a}\}$, outputs

$$\omega(r_0) = \omega(r_1) = \mathbf{0},$$

and transitions

$$\delta(r_0, \mathbf{a}) = r_0, \quad \delta(r_1, \mathbf{a}) = r_1.$$

The state r_1 is unreachable, so M is not connected. But r_0 and r_1 are equivalent, since both produce only $\mathbf{0}$ on every input. Therefore the quotient merges them into one state, and that one-state quotient is connected.

7.61. **Prove $\mu(s)E_B\mu(t) \Leftrightarrow sE_At$ for Mealy homomorphism.**

Proof. ($\Rightarrow$) If $\mu(s)E_B\mu(t)$: $\bar{\omega}_B(\mu(s), x) = \bar{\omega}_B(\mu(t), x)$ for all x . By Exercise 7.2(b), $\bar{\omega}_A(s, x) = \bar{\omega}_B(\mu(s), x) = \bar{\omega}_B(\mu(t), x) = \bar{\omega}_A(t, x)$, so sE_At .

($\Leftarrow$) If sE_At : $\bar{\omega}_A(s, x) = \bar{\omega}_A(t, x)$ for all x . So $\bar{\omega}_B(\mu(s), x) = \bar{\omega}_A(s, x) = \bar{\omega}_A(t, x) = \bar{\omega}_B(\mu(t), x)$, giving $\mu(s)E_B\mu(t)$. $\square$

7.62. **Prove $\mu(s)E_B\mu(t) \Leftrightarrow sE_At$ for Moore homomorphism.**

Proof. ($\Rightarrow$) Suppose $\mu(s)E_B\mu(t)$. Then

$$\bar{\omega}_B(\mu(s), x) = \bar{\omega}_B(\mu(t), x)$$

for every $x \in \Sigma^*$. By Exercise 7.47(b),

$$\bar{\omega}_A(s, x) = \bar{\omega}_B(\mu(s), x) \quad \text{and} \quad \bar{\omega}_A(t, x) = \bar{\omega}_B(\mu(t), x),$$

so $\bar{\omega}_A(s, x) = \bar{\omega}_A(t, x)$ for all x . Hence sE_At .

($\Leftarrow$) Suppose sE_At . Then

$$\bar{\omega}_A(s, x) = \bar{\omega}_A(t, x)$$

for every $x \in \Sigma^*$. Again using Exercise 7.47(b),

$$\bar{\omega}_B(\mu(s), x) = \bar{\omega}_A(s, x) = \bar{\omega}_A(t, x) = \bar{\omega}_B(\mu(t), x).$$

Therefore $\mu(s)E_B\mu(t)$. $\square$

7.63. (a) FST A not reduced and A^c not reduced:

Let

$$A = \langle \{\mathbf{a}\}, \{\mathbf{0}\}, \{s_0, s_1, s_2\}, s_0, \delta, \omega \rangle,$$

where

$$\delta(s_0, \mathbf{a}) = s_1, \quad \delta(s_1, \mathbf{a}) = s_1, \quad \delta(s_2, \mathbf{a}) = s_2,$$

and every transition prints $\mathbf{0}$. The states s_1 and s_2 are equivalent, so A is not reduced. Both s_0 and s_1 are reachable from s_0 , and s_0 and s_1 are also equivalent, so the connected part A^c still has equivalent distinct states. Hence A^c is not reduced.

(b) FST A not reduced and A^c reduced:

Let

$$A = \langle \{\mathbf{a}\}, \{\mathbf{0}, \mathbf{1}\}, \{s_0, s_1, s_2\}, s_0, \delta, \omega \rangle,$$

where

$$\delta(s_0, \mathbf{a}) = s_1, \quad \delta(s_1, \mathbf{a}) = s_1, \quad \delta(s_2, \mathbf{a}) = s_2,$$

and

$$\omega(s_0, \mathbf{a}) = \mathbf{0}, \quad \omega(s_1, \mathbf{a}) = \omega(s_2, \mathbf{a}) = \mathbf{1}.$$

Then $s_1 E_A s_2$, so A is not reduced. But s_2 is unreachable, while the connected part $A^c = \{s_0, s_1\}$ is reduced because s_0 and s_1 have different output behavior.

7.64. (a) Moore machine A not reduced and A^c not reduced:

Let the states be $\{r_0, r_1, r_2\}$ with start state r_0 , alphabet $\{\mathbf{a}\}$, transitions

$$\delta(r_0, \mathbf{a}) = r_1, \quad \delta(r_1, \mathbf{a}) = r_1, \quad \delta(r_2, \mathbf{a}) = r_2,$$

and outputs

$$\omega(r_0) = \omega(r_1) = \omega(r_2) = \mathbf{0}.$$

Then all states are equivalent, so A is not reduced. The connected part A^c consists of r_0 and r_1 , which are still equivalent, so A^c is not reduced.

(b) Moore machine A not reduced and A^c reduced:

Let the states be $\{r_0, r_1, r_2\}$ with start state r_0 , alphabet $\{\mathbf{a}\}$, transitions

$$\delta(r_0, \mathbf{a}) = r_1, \quad \delta(r_1, \mathbf{a}) = r_1, \quad \delta(r_2, \mathbf{a}) = r_2,$$

and outputs

$$\omega(r_0) = \mathbf{0}, \quad \omega(r_1) = \omega(r_2) = \mathbf{1}.$$

Then $r_1 E_A r_2$, so A is not reduced. But r_2 is unreachable, while the connected part $A^c = \{r_0, r_1\}$ is reduced because r_0 and r_1 have different output behavior.

7.65. **Isomorphism $\cong$ is an equivalence relation on Mealy machines.**

(a) **Symmetry:** If $\psi : A \rightarrow B$ is an isomorphism, then ψ is bijective and satisfies

$$\psi(s_{0_A}) = s_{0_B}, \quad \psi(\delta_A(s, \mathbf{a})) = \delta_B(\psi(s), \mathbf{a}), \quad \omega_A(s, \mathbf{a}) = \omega_B(\psi(s), \mathbf{a}).$$

Applying ψ^{-1} to these identities shows that

$$\psi^{-1}(s_{0_B}) = s_{0_A}, \quad \psi^{-1}(\delta_B(t, \mathbf{a})) = \delta_A(\psi^{-1}(t), \mathbf{a}),$$

and

$$\omega_B(t, \mathbf{a}) = \omega_A(\psi^{-1}(t), \mathbf{a}).$$

Hence $\psi^{-1} : B \rightarrow A$ is also an isomorphism.

(b) **Reflexivity:** The identity map id_{S_A} preserves the start state, transition function, and output function, so it is an isomorphism from A to itself.

- (c) **Composition:** If $\psi : A \rightarrow B$ and $\varphi : B \rightarrow C$ are isomorphisms, then $\varphi \circ \psi$ is bijective. Also,

$$(\varphi \circ \psi)(s_{0_A}) = \varphi(s_{0_B}) = s_{0_C},$$

and for every state s and input $\mathbf{a}$,

$$(\varphi \circ \psi)(\delta_A(s, \mathbf{a})) = \varphi(\delta_B(\psi(s), \mathbf{a})) = \delta_C((\varphi \circ \psi)(s), \mathbf{a}),$$

while

$$\omega_A(s, \mathbf{a}) = \omega_B(\psi(s), \mathbf{a}) = \omega_C((\varphi \circ \psi)(s), \mathbf{a}).$$

Thus $\varphi \circ \psi$ is an isomorphism.

- (d) **Equivalence:** Parts (a), (b), and (c) show that $\cong$ is symmetric, reflexive, and transitive. Therefore it is an equivalence relation.
- (e) **Homomorphism is not an equivalence relation:** Homomorphism need not be symmetric. Let A have two distinct states s_0, s_1 with the same transition and output behavior, and let B be the one-state machine obtained by identifying them. Then there is a homomorphism $A \rightarrow B$, but no inverse map $B \rightarrow A$ exists, so homomorphism is not symmetric.

7.66. Isomorphism $\cong$ is an equivalence relation on Moore machines; homomorphism is not.

- (a) **Symmetry:** If $\psi : A \rightarrow B$ is a Moore isomorphism, then ψ^{-1} preserves the start state, transition function, and state-output function, so $\psi^{-1} : B \rightarrow A$ is also an isomorphism.
- (b) **Reflexivity:** The identity map on the state set of a Moore machine is an isomorphism.
- (c) **Composition:** The composition of two Moore isomorphisms again preserves start state, transitions, and outputs, hence is an isomorphism.
- (d) **Equivalence:** Therefore Moore-machine isomorphism is reflexive, symmetric, and transitive.
- (e) **Homomorphism is not an equivalence relation:** It is not symmetric. Let

$$A = \langle \{\mathbf{a}\}, \{\mathbf{0}\}, \{r_0, r_1\}, r_0, \delta_A, \omega_A \rangle$$

with

$$\omega_A(r_0) = \omega_A(r_1) = \mathbf{0}, \quad \delta_A(r_0, \mathbf{a}) = r_1, \quad \delta_A(r_1, \mathbf{a}) = r_1,$$

and let

$$B = \langle \{\mathbf{a}\}, \{\mathbf{0}\}, \{s_0\}, s_0, \delta_B, \omega_B \rangle$$

with

$$\omega_B(s_0) = \mathbf{0}, \quad \delta_B(s_0, \mathbf{a}) = s_0.$$

The map $\mu(r_0) = \mu(r_1) = s_0$ is a homomorphism $A \rightarrow B$. But there is no homomorphism $B \rightarrow A$: any such map must send s_0 to the start state r_0 , and then

$$\mu(\delta_B(s_0, \mathbf{a})) = \mu(s_0) = r_0$$

would have to equal

$$\delta_A(r_0, \mathbf{a}) = r_1,$$

which is impossible. So homomorphism is not symmetric, and therefore not an equivalence relation.

7.67. **Prove homomorphism $\mu : M \rightarrow M /_{E_M}$ exists.**

Proof. Define $\mu : S \rightarrow S_{E_M}$ by $\mu(s) = [s]_{E_M}$.

We verify the homomorphism properties.

(i) $\mu(s_0) = [s_0]_{E_M}$, which is the start state of the quotient.

(ii) For every state s and input $\mathbf{a}$,

$$\mu(\delta(s, \mathbf{a})) = [\delta(s, \mathbf{a})]_{E_M} = \delta_{E_M}([s]_{E_M}, \mathbf{a}) = \delta_{E_M}(\mu(s), \mathbf{a}).$$

(iii) For every state s and input $\mathbf{a}$,

$$\omega_{E_M}(\mu(s), \mathbf{a}) = \omega_{E_M}([s]_{E_M}, \mathbf{a}) = \omega(s, \mathbf{a}).$$

Thus μ is a homomorphism from M onto $M /_{E_M}$. □ □

7.68. **Prove homomorphism $\mu : M \rightarrow M /_{E_M}$ exists for Moore machines.**

Proof. Define $\mu(s) = [s]_{E_M}$ from the state set of M onto the state set of the quotient.

We verify the homomorphism properties.

(i) $\mu(s_0) = [s_0]_{E_M}$ is the start state of the quotient.

(ii) For every state s and input $\mathbf{a}$,

$$\mu(\delta(s, \mathbf{a})) = [\delta(s, \mathbf{a})]_{E_M} = \delta_{E_M}([s]_{E_M}, \mathbf{a}) = \delta_{E_M}(\mu(s), \mathbf{a}).$$

(iii) For every state s ,

$$\omega_{E_M}(\mu(s)) = \omega_{E_M}([s]_{E_M}) = \omega(s).$$

Hence μ is a Moore homomorphism from M onto $M /_{E_M}$. □ □

7.69. **Limit time in state s_2 to 3 clock cycles.**

(a) Replace s_2 by three states s_{2a}, s_{2b}, s_{2c} , all with output $\langle R, R, R, G \rangle$. The old transition $s_1 \rightarrow s_2$ becomes $s_1 \rightarrow s_{2a}$. Then define

	$\langle 0, 0 \rangle$	$\langle 0, 1 \rangle$	$\langle 1, 0 \rangle$	$\langle 1, 1 \rangle$
s_{2a}	s_4	s_{2b}	s_4	s_{2b}
s_{2b}	s_4	s_{2c}	s_4	s_{2c}
s_{2c}	s_4	s_4	s_4	s_4

so the configuration $\langle R, R, R, G \rangle$ can last for one, two, or three cycles, but never four.

(b) Replace s_6 by three states s_{6a}, s_{6b}, s_{6c} , all with output $\langle G, G, R, R \rangle$. Every old edge entering s_6 now enters s_{6a} . Then define

	$\langle 0, 0 \rangle$	$\langle 0, 1 \rangle$	$\langle 1, 0 \rangle$	$\langle 1, 1 \rangle$
s_{6a}	s_5	s_7	s_{6b}	s_{6b}
s_{6b}	s_5	s_7	s_{6c}	s_{6c}
s_{6c}	s_5	s_7	s_5	s_7

which bounds the left-turn green in the same way.

- (c) To guarantee a *minimum* of two cycles and a *maximum* of four cycles for the left-turn green, replace s_6 by four states $s_{6a}, s_{6b}, s_{6c}, s_{6d}$, all outputting $\langle G, G, R, R \rangle$, and let

	$\langle 0, 0 \rangle$	$\langle 0, 1 \rangle$	$\langle 1, 0 \rangle$	$\langle 1, 1 \rangle$
s_{6a}	s_{6b}	s_{6b}	s_{6b}	s_{6b}
s_{6b}	s_5	s_7	s_{6c}	s_{6c}
s_{6c}	s_5	s_7	s_{6d}	s_{6d}
s_{6d}	s_5	s_7	s_5	s_7

so the first transition out of s_{6a} is forced, the machine may remain in the green left-turn configuration for two, three, or four cycles, and the fourth cycle is the last possible one.

7.70. Ensure east-west traffic gets green occasionally.

- (a) Without adding states, force every excursion away from s_0 to return to s_0 after one service. Concretely, remove the self-loops on s_2 and s_6 , and redefine

$$\delta(s_2, \langle i, j \rangle) = s_4 \quad \text{for all } i, j \in \{0, 1\},$$

$$\delta(s_6, \langle 0, 0 \rangle) = s_5, \quad \delta(s_6, \langle 1, 0 \rangle) = s_5, \quad \delta(s_6, \langle 0, 1 \rangle) = s_7, \quad \delta(s_6, \langle 1, 1 \rangle) = s_7,$$

and then force the yellow states back to east-west green:

$$\delta(s_4, \langle i, j \rangle) = \delta(s_5, \langle i, j \rangle) = \delta(s_7, \langle i, j \rangle) = s_0$$

for all i, j . Then the machine must revisit $\langle G, R, G, R \rangle$ regularly.

- (b) To obtain a genuine fairness controller, pair each phase state with a memory set $H \subseteq \{A, B\}$, where A means “the α left-turn lane has already had green” and B means “the β side-street lane has already had green” since the last visit to s_0 . Use states (s, H) with $s \in \{s_0, \dots, s_7\}$ and start state $(s_0, \emptyset)$. Whenever the machine enters the old state s_2 , replace H by $H \cup \{B\}$; whenever it enters the old state s_6 , replace H by $H \cup \{A\}$; whenever it returns to s_0 , reset the memory to $\emptyset$. The yellow states now dispatch according to the set of sensor-driven lanes that are still owed service:

- (i) from a yellow state that follows B -service, if α is active and $A \notin H$, go next to the A -branch instead of returning to s_0 ; otherwise return to s_0 ;
- (ii) from a yellow state that follows A -service, if β is active and $B \notin H$, go next to the B -branch instead of returning to s_0 ; otherwise return to s_0 .

Thus no sensor-controlled lane can receive a second green before every other active sensor-controlled lane has received a first one.

7.71. Prove: if two Moore machines are homomorphic, they are equivalent.

Proof. If $\mu: A \rightarrow B$ is a homomorphism, by Exercise 7.47(c), $f_A = f_B$. Hence $A \equiv B$. □ □

7.72. Prove $\mathcal{D}_\Sigma$ is closed under any FTD function $f: \Sigma^* \rightarrow \Sigma^*$.

Proof. Let $L \in \mathfrak{D}_\Sigma$, and choose a DFA

$$A = \langle \Sigma, Q, q_0, \delta_A, F \rangle$$

with $L(A) = L$. Let

$$M = \langle \Sigma, \Sigma, S, s_0, \delta_M, \omega_M \rangle$$

be a Mealy machine computing the FTD function f .

We build an NDFA

$$N = \langle \Sigma, Q \times S, \{\langle q_0, s_0 \rangle\}, \Delta, F \times S \rangle$$

that accepts exactly $f(L)$. Intuitively, while N reads an output word y , it guesses an input word x letter by letter; the S -component simulates the transducer on the guessed input, and the Q -component simulates the DFA A on that same guessed input.

For $\langle q, s \rangle \in Q \times S$ and $\mathbf{b} \in \Sigma$, define

$$\Delta(\langle q, s \rangle, \mathbf{b}) = \{\langle \delta_A(q, \mathbf{a}), \delta_M(s, \mathbf{a}) \rangle \mid \mathbf{a} \in \Sigma, \omega_M(s, \mathbf{a}) = \mathbf{b}\}.$$

So on reading an output symbol $\mathbf{b}$, the NDFA chooses any input symbol $\mathbf{a}$ that would cause the transducer, in state s , to print $\mathbf{b}$.

By induction on the input length, after reading a word y , the machine N can be in exactly those states

$$\langle \bar{\delta}_A(q_0, x), \bar{\delta}_M(s_0, x) \rangle$$

for which $f(x) = y$. Therefore

$$y \in L(N) \iff (\exists x \in \Sigma^*)(f(x) = y \wedge \bar{\delta}_A(q_0, x) \in F) \iff (\exists x \in L)(f(x) = y).$$

Hence

$$L(N) = f(L).$$

Because N is an NDFA, Theorem 4.1 implies that $L(N) \in \mathfrak{D}_\Sigma$. Thus $f(L) \in \mathfrak{D}_\Sigma$, so $\mathfrak{D}_\Sigma$ is closed under every FTD function f . □

Chapter 8

Regular and Context-Free Grammars — Solutions

8.1. Can strings like $abBAdBc$ be derived from the start symbol in a right-linear grammar?

No. In a right-linear grammar, every production has the form $A \rightarrow w$ or $A \rightarrow wB$ where $w \in \Sigma^*$ and B is a nonterminal. Thus any sentential form derived from the start symbol has the structure w or wB where all terminal symbols precede the single nonterminal (if any). A string like $abBAdBc$ contains nonterminals interspersed with and following terminals, which cannot occur in any right-linear grammar derivation.

8.2. Prove $P(n)$: for all $x \in \Sigma^n$, if $t_0 \xRightarrow{*} xt_j$ then $\bar{\delta}_A(t_0, x) = t_j$.

Base case ($n = 0$): The only string of length 0 is λ . The derivation $t_0 \xRightarrow{*} \lambda t_0$ uses zero steps, and $\bar{\delta}_A(t_0, \lambda) = t_0$ by definition, so $P(0)$ holds.

Inductive step: Assume $P(n)$ holds for all strings of length n . Let $x = ya$ where $y \in \Sigma^n$ and $a \in \Sigma$. If $t_0 \xRightarrow{*} ya \cdot t_j$, then the derivation must pass through some intermediate state: $t_0 \xRightarrow{*} y \cdot t_k \Rightarrow ya \cdot t_j$ for some t_k . By the inductive hypothesis applied to y , $\bar{\delta}_A(t_0, y) = t_k$. The final step $t_k \Rightarrow a \cdot t_j$ corresponds to a production $t_k \rightarrow at_j$ in the grammar (by construction of G_A), which corresponds to a transition $\delta(t_k, a) = t_j$ in A . Therefore $\bar{\delta}_A(t_0, ya) = \delta(\bar{\delta}_A(t_0, y), a) = \delta(t_k, a) = t_j$.

8.3. Regular expressions for the languages of:

(a) G_4 : Tracing the grammar, we get

$$\begin{aligned} S &= abA \cup bbB \cup ccV, \\ A &= bC \cup cX, \quad B = ab, \quad C = \lambda \cup cS, \\ V &= aV \cup cX, \quad X = bV \cup aaX. \end{aligned}$$

From

$$V = aV \cup cX,$$

Arden's lemma (Theorem 6.1) gives

$$V = a^*cX.$$

Substituting into the equation for X gives

$$X = \mathbf{ba}^* \mathbf{c} X \cup \mathbf{aa} X = (\mathbf{ba}^* \mathbf{c} \cup \mathbf{aa}) X.$$

There is no constant term, so the only solution is

$$X = \emptyset,$$

and therefore also $V = \emptyset$.

Thus only the S, A, B, C portion contributes:

$$S = \mathbf{ab} A \cup \mathbf{bbab}, \quad A = \mathbf{b} C \cup \mathbf{c} \emptyset = \mathbf{b} C, \quad C = \lambda \cup \mathbf{c} S.$$

Hence

$$A = \mathbf{b} \cup \mathbf{bc} S,$$

so

$$S = \mathbf{abb} \cup \mathbf{abbc} S \cup \mathbf{bbab}.$$

By Arden's lemma (Theorem 6.1),

$$L(G_4) = (\mathbf{abbc})^* (\mathbf{abb} \cup \mathbf{bbab}).$$

(b) G_5 : The language equations for the three nonterminals are

$$S_0 = \lambda \cup \mathbf{0} S_2 \cup \mathbf{1} S_1,$$

$$S_1 = \mathbf{0} S_1 \cup \mathbf{1} S_2,$$

$$S_2 = \mathbf{0} S_2 \cup \mathbf{1} S_0.$$

The equation for S_2 has the form required by Theorem 6.1, so

$$S_2 = \mathbf{0}^* \mathbf{1} S_0.$$

Substituting this into the equation for S_1 gives

$$S_1 = \mathbf{0} S_1 \cup \mathbf{10}^* \mathbf{1} S_0,$$

and another application of Theorem 6.1 gives

$$S_1 = \mathbf{0}^* \mathbf{10}^* \mathbf{1} S_0.$$

Now substitute both expressions into the equation for S_0 :

$$\begin{aligned} S_0 &= \lambda \cup \mathbf{00}^* \mathbf{1} S_0 \cup \mathbf{10}^* \mathbf{10}^* \mathbf{1} S_0 \\ &= \lambda \cup (\mathbf{0}^+ \mathbf{1} \cup \mathbf{10}^* \mathbf{10}^* \mathbf{1}) S_0. \end{aligned}$$

Thus

$$L(G_5) = (\mathbf{0}^+ \mathbf{1} \cup \mathbf{10}^* \mathbf{10}^* \mathbf{1})^*.$$

- Every derivation cycles: $Z \Rightarrow \mathbf{b}T \Rightarrow \mathbf{ba}Z \Rightarrow \dots$, and neither Z nor T has a terminal production, confirming $L(G_6) = \emptyset$.

- From B : $B = \mathbf{a}bB \cup \lambda$, so $L(B) = (\mathbf{a}b)^*$.
- From C : $C = \mathbf{c}C \cup \mathbf{c}a$, so $L(C) = \mathbf{c}^* \mathbf{c}a = \mathbf{c}^+ a$.
- From S : $S = \mathbf{a}S \cup \mathbf{a}bB \cup \mathbf{c}C = \mathbf{a}S \cup \mathbf{a}b(\mathbf{a}b)^* \cup \mathbf{c}^+ a$. Thus

SO

The regular expression is $\mathbf{a^*(ab)^+ \cup a^*c^+a}$.

Lemma 8.1 states that for the grammar G_A constructed from DFA A , $L(G_A) = L(A)$.

$$\begin{aligned} x \in L(G_A) &\stackrel{*}{\iff} t_0 \Rightarrow x \text{ in } G_A \\ &\iff (\exists t_j \in F) \ t_0 \stackrel{*}{\Rightarrow} x t_j \Rightarrow x \\ &\iff (\exists t_j \in F) \ \bar{\delta}_A(t_0, x) = t_j \\ &\iff \bar{\delta}_A(t_0, x) \in F \\ &\iff x \in L(A). \end{aligned}$$
☐

Use the construction of Lemma 8.2: states are the suffix-strings that arise from right-hand sides, the start state is $\langle S \rangle$, and the only final state is $\langle \lambda \rangle$.

- $$\{ \langle S \rangle, \langle A \rangle, \langle B \rangle, \langle C \rangle, \langle V \rangle, \langle W \rangle, \langle X \rangle, \langle \mathbf{ab}A \rangle, \langle \mathbf{b}A \rangle, \langle \mathbf{bb}B \rangle, \langle \mathbf{b}B \rangle, \langle \mathbf{cc}V \rangle, \langle \mathbf{c}V \rangle, \langle \mathbf{b}C \rangle, \langle \mathbf{c}X \rangle, \langle \mathbf{ab} \rangle, \langle \mathbf{b} \rangle, \langle \mathbf{c}S \rangle, \langle \mathbf{a}V \rangle, \langle \mathbf{aa} \rangle, \langle \mathbf{a}V \rangle$$

$$\delta(\langle C \rangle, \lambda) = \{\langle \lambda \rangle, \langle \mathbf{c}S \rangle\}, \quad \delta(\langle V \rangle, \lambda) = \{\langle \mathbf{a}V \rangle, \langle \mathbf{c}X \rangle\},$$

$$\delta(\langle W \rangle, \lambda) = \{\langle \mathbf{aa} \rangle, \langle \mathbf{aW} \rangle\}, \quad \delta(\langle X \rangle, \lambda) = \{\langle \mathbf{bV} \rangle, \langle \mathbf{aaX} \rangle\}.$$

The nonempty letter-transitions are the suffix moves, for example

$$\delta(\langle \mathbf{abA} \rangle, \mathbf{a}) = \{\langle \mathbf{bA} \rangle\}, \quad \delta(\langle \mathbf{bA} \rangle, \mathbf{b}) = \{\langle \mathbf{A} \rangle\},$$

$$\delta(\langle \mathbf{ab} \rangle, \mathbf{a}) = \{\langle \mathbf{b} \rangle\}, \quad \delta(\langle \mathbf{b} \rangle, \mathbf{b}) = \{\langle \lambda \rangle\},$$

and the same suffix rule handles every other string-state: if the current state is $\langle \mathbf{au} \rangle$ and the next input letter is $\mathbf{a}$, the machine moves to $\langle \mathbf{u} \rangle$; if the input letter does not match the first symbol, there is no transition. For example,

$$\delta(\langle \mathbf{bbB} \rangle, \mathbf{b}) = \{\langle \mathbf{bB} \rangle\}, \quad \delta(\langle \mathbf{ccV} \rangle, \mathbf{c}) = \{\langle \mathbf{cV} \rangle\}, \quad \delta(\langle \mathbf{aa} \rangle, \mathbf{a}) = \{\langle \mathbf{a} \rangle\}, \quad \delta(\langle \mathbf{a} \rangle, \mathbf{a}) = \{\langle \lambda \rangle\}.$$

(b) For G_5 , the corresponding automaton is simply the three-state DFA

$$\langle \mathbf{0}, \mathbf{1} \rangle, \{S_0, S_1, S_2\}, S_0, \delta, \{S_0\},$$

with

$$\delta(S_0, \mathbf{0}) = S_2, \quad \delta(S_0, \mathbf{1}) = S_1, \quad \delta(S_1, \mathbf{0}) = S_1, \quad \delta(S_1, \mathbf{1}) = S_2,$$

$$\delta(S_2, \mathbf{0}) = S_2, \quad \delta(S_2, \mathbf{1}) = S_0.$$

(c) For G_6 , the state set is

$$\{\langle \mathbf{Z} \rangle, \langle \mathbf{T} \rangle, \langle \mathbf{aZ} \rangle, \langle \mathbf{bT} \rangle, \langle \lambda \rangle\},$$

with start state $\langle \mathbf{Z} \rangle$, final state $\langle \lambda \rangle$, and nonempty transitions

$$\delta(\langle \mathbf{Z} \rangle, \lambda) = \{\langle \mathbf{aZ} \rangle, \langle \mathbf{bT} \rangle\}, \quad \delta(\langle \mathbf{T} \rangle, \lambda) = \{\langle \mathbf{aZ} \rangle\},$$

$$\delta(\langle \mathbf{aZ} \rangle, \mathbf{a}) = \{\langle \mathbf{Z} \rangle\}, \quad \delta(\langle \mathbf{bT} \rangle, \mathbf{b}) = \{\langle \mathbf{T} \rangle\}.$$

There is no path to $\langle \lambda \rangle$, so the language is empty.

(d) For G_7 , the state set is

$$\{\langle \mathbf{S} \rangle, \langle \mathbf{B} \rangle, \langle \mathbf{C} \rangle, \langle \mathbf{aS} \rangle, \langle \mathbf{abB} \rangle, \langle \mathbf{bB} \rangle, \langle \mathbf{cC} \rangle, \langle \mathbf{ab} \rangle, \langle \mathbf{b} \rangle, \langle \mathbf{ca} \rangle, \langle \mathbf{a} \rangle, \langle \lambda \rangle\},$$

with start state $\langle \mathbf{S} \rangle$, final state $\langle \lambda \rangle$, and nonempty λ -transitions

$$\delta(\langle \mathbf{S} \rangle, \lambda) = \{\langle \mathbf{aS} \rangle, \langle \mathbf{abB} \rangle, \langle \mathbf{cC} \rangle\},$$

$$\delta(\langle \mathbf{B} \rangle, \lambda) = \{\langle \mathbf{abB} \rangle, \langle \lambda \rangle\}, \quad \delta(\langle \mathbf{C} \rangle, \lambda) = \{\langle \mathbf{cC} \rangle, \langle \mathbf{ca} \rangle\},$$

and nonempty letter-transitions

$$\delta(\langle \mathbf{aS} \rangle, \mathbf{a}) = \{\langle \mathbf{S} \rangle\}, \quad \delta(\langle \mathbf{abB} \rangle, \mathbf{a}) = \{\langle \mathbf{bB} \rangle\}, \quad \delta(\langle \mathbf{bB} \rangle, \mathbf{b}) = \{\langle \mathbf{B} \rangle\},$$

$$\delta(\langle \mathbf{cC} \rangle, \mathbf{c}) = \{\langle \mathbf{C} \rangle\}, \quad \delta(\langle \mathbf{ab} \rangle, \mathbf{a}) = \{\langle \mathbf{b} \rangle\}, \quad \delta(\langle \mathbf{b} \rangle, \mathbf{b}) = \{\langle \lambda \rangle\},$$

$$\delta(\langle \mathbf{ca} \rangle, \mathbf{c}) = \{\langle \mathbf{a} \rangle\}, \quad \delta(\langle \mathbf{a} \rangle, \mathbf{a}) = \{\langle \lambda \rangle\}.$$

8.6. Right-linear grammars for the given languages.

- (a) All words in $\{a, b, c\}^*$ *not* containing two consecutive **bs**:

$$G = \langle \{S, A\}, \{a, b, c\}, S, \{S \rightarrow aS|bA|cS|\lambda, A \rightarrow aS|cS|\lambda\} \rangle$$

State S = “last symbol was not **b**” (or start); state A = “last symbol was **b**”. From A , reading another **b** is disallowed.

- (b) All words *containing* two consecutive **bs**:

$$G = \langle \{S, A, B\}, \{a, b, c\}, S, \{S \rightarrow aS|cS|bA, A \rightarrow bB|aS|cS, B \rightarrow aB|bB|cB|\lambda\} \rangle$$

State S means “no consecutive **bs** have been seen and the last symbol was not **b**”; state A means “no consecutive **bs** have been seen, but the last symbol was **b**”; state B means “two consecutive **bs** have occurred.” Once B is reached, any remaining string may be generated.

- (c) All words with $|w|_a = |w|_b$: This language is not regular (it includes $\{a^n b^n\}$), so no right-linear grammar exists.
- (d) All words with an even number of **as**:

$$G = \langle \{S, A\}, \{a, b, c\}, S, \{S \rightarrow bS|cS|aA|\lambda, A \rightarrow bA|cA|aS\} \rangle$$

State S records that an even number of **as** has been generated so far, and state A records that an odd number has been generated. The **b**- and **c**-productions preserve parity, while each **a** switches parity.

- (e) All words *not* ending in **b**:

$$G = \langle \{S, A\}, \{a, b, c\}, S, \{S \rightarrow aS|bA|cS|a|c|\lambda, A \rightarrow aS|bA|cS|a|c\} \rangle$$

(Equivalently, $G = \langle \{S, A\}, \dots, \{S \rightarrow (a|b|c)^*(a|c)|\lambda\} \rangle$.) State S records that the string generated so far is empty or currently ends in a symbol other than **b**; state A records that it currently ends in **b**. Only S may stop by λ , and both states may stop after adding one final **a** or **c**.

- (f) All words containing no **cs**:

$$G = \langle \{S\}, \{a, b, c\}, S, \{S \rightarrow aS|bS|\lambda\} \rangle$$

State S generates exactly the words over $\{a, b\}$ and may stop at any point by using $S \rightarrow \lambda$; there is no **c**-production.

8.7. Left-linear grammars for the languages in Exercise 8.6.

A left-linear grammar has productions of the form $A \rightarrow w$ or $A \rightarrow Bw$.

- (a) No two consecutive **bs**:

$$G = \langle \{S, A\}, \{a, b, c\}, S, \{S \rightarrow Sa|Sc|Ab|\lambda, A \rightarrow Sa|Sc|\lambda\} \rangle$$

Here S generates the strings with no consecutive **bs**, while A generates those whose leftmost symbol is a single **b**.

(b) Containing two consecutive **bs**:

$$G = \langle \{S, A, B\}, \{a, b, c\}, S, \{S \rightarrow Sa \mid Sc \mid Ab, A \rightarrow Sa \mid Sc \mid Bb, B \rightarrow Ba \mid Bb \mid Bc \mid \lambda\} \rangle$$

Here S generates all strings that already contain two consecutive **bs**. State A generates strings whose leftmost symbol is one **b** away from creating the required pair, and B generates the arbitrary prefix to the left of that pair.

(c) Same number of **as** as **bs**: no left-linear grammar exists. By Theorem 8.2, every left-linear grammar generates a FAD language, and the corresponding part of Exercise 8.6 is not FAD.

(d) Even number of **as**:

$$G = \langle \{S, A\}, \{a, b, c\}, S, \{S \rightarrow Sb \mid Sc \mid Aa \mid \lambda, A \rightarrow Sa \mid Ab \mid Ac\} \rangle$$

Here S generates strings with an even number of **as**, while A generates strings with an odd number of **as**. Appending **b** or **c** preserves the parity, and appending **a** changes it.

(e) Not ending in **b**:

$$G = \langle \{S, A\}, \{a, b, c\}, S, \{S \rightarrow Aa \mid Ac \mid \lambda, A \rightarrow Aa \mid Ab \mid Ac \mid \lambda\} \rangle$$

Here S generates exactly the target strings: either λ , or a string whose final symbol is **a** or **c**. State A generates the arbitrary prefix that may precede that final non-**b** symbol.

(f) No **cs**:

$$G = \langle \{S\}, \{a, b, c\}, S, \{S \rightarrow Sa \mid Sb \mid \lambda\} \rangle$$

Here S generates exactly the words over $\{a, b\}$ and may stop at any point; there is no production that appends **c**.

8.8. Complete the inductive portion of the proof of Theorem 8.8.

Let $G = \langle \Omega, \Sigma, S, P \rangle$ already be in the normal form of Theorem 8.7, and let $G^0 = \langle \Omega \cup \{Z\}, \Sigma, Z, P^0 \rangle$ be the grammar constructed in Theorem 8.8.

The needed induction is on the number of productions used after the initial step $Z \rightarrow S$.

Claim: For every $A \in \Omega$ and every $x \in \Sigma^+$,

$$A \xRightarrow{*}_{G^0} x \quad \text{if and only if} \quad A \xRightarrow{*}_G x.$$

We prove the claim by induction on the number of productions in the derivation.

Base: A one-step derivation in G^0 must use a production $A \rightarrow a$. By construction of P^0 , there is some $B \in \Omega$ such that $A \rightarrow aB \in P$ and $B \rightarrow \lambda \in P$. Hence in G we have $A \Rightarrow aB \Rightarrow a$. Conversely, if $A \xRightarrow{*}_G x$ in two steps, then necessarily $x = a$, with productions $A \rightarrow aB$, $B \rightarrow \lambda$, so $A \rightarrow a$ is in P^0 and $A \xRightarrow{*}_{G^0} a$.

Step: Assume the claim holds for derivations using at most n productions, and suppose $A \xRightarrow{*}_{G^0} x$ uses $n+1$ productions.

If the first production is $A \rightarrow aB$, then

$$A \xRightarrow{*}_{G^0} aB \xRightarrow{*}_{G^0} x,$$

so $x = \mathbf{a}y$ and $B \Rightarrow_{G^0}^* y$ uses n productions. By the induction hypothesis, $B \Rightarrow_G^* y$, and therefore $A \Rightarrow_G^* \mathbf{a}B \Rightarrow_G x$.

The other possibility is that the first production is $A \rightarrow \mathbf{a}$; then $x = \mathbf{a}$ and this is already the base case.

For the converse direction, suppose $A \Rightarrow_G^* x$ and the derivation uses $n + 2$ productions. The first production cannot be $A \rightarrow \lambda$, since $x \in \Sigma^+$. If the first production is $A \rightarrow \mathbf{a}B$, and the remainder is $B \Rightarrow_G^* y$ with $x = \mathbf{a}y$, then either $y \in \Sigma^+$, in which case the induction hypothesis gives $B \Rightarrow_{G^0}^* y$ and hence $A \Rightarrow_{G^0}^* \mathbf{a}B \Rightarrow_{G^0} x$, or else $y = \lambda$, in which case $B \rightarrow \lambda \in P$ and therefore $A \rightarrow \mathbf{a} \in P^0$, so $A \Rightarrow_{G^0}^* \mathbf{a} = x$.

Now apply the claim with $A = S$. Any terminal derivation in G^0 either is $Z \Rightarrow \lambda$ (which occurs exactly when $S \rightarrow \lambda \in P$) or begins $Z \Rightarrow S$ and then follows a derivation covered by the claim. Hence $L(G^0) = L(G)$. $\square$

8.9. Complete the inductive portion of the proof of Theorem 8.5.

Let $G_* = \langle \Omega \cup \{Z\}, \Sigma, Z, P_* \rangle$ be the grammar of Theorem 8.5. The theorem says that derivations using exactly n productions of the form $A \rightarrow xZ$ generate exactly the words in $L(G)^n$.

Claim: For every $n \in \mathbb{N}$ and every $w \in \Sigma^*$,

$$Z \Rightarrow_{G_*}^* w \text{ using exactly } n \text{ productions of the form } A \rightarrow xZ$$

if and only if $w \in L(G)^n$.

We prove this by induction on n .

Base ($n = 0$): The only terminal derivation from Z that uses no production of the form $A \rightarrow xZ$ is $Z \Rightarrow \lambda$. Hence the words generated are exactly $L(G)^0 = \{\lambda\}$.

Step: Assume the claim holds for n , and consider a derivation using exactly $n + 1$ productions of the form $A \rightarrow xZ$. Since the start symbol Z appears only in the productions $Z \rightarrow \lambda$ and $Z \rightarrow S$, such a derivation must begin

$$Z \Rightarrow S \Rightarrow_{G_*}^* uZ \Rightarrow_{G_*}^* uv,$$

where the portion $S \Rightarrow_{G_*}^* uZ$ uses exactly one production $A \rightarrow xZ$. Replacing that final production by the corresponding production $A \rightarrow x$ from G shows that $u \in L(G)$. The remainder $Z \Rightarrow_{G_*}^* v$ uses exactly n productions of the form $A \rightarrow xZ$, so by the induction hypothesis $v \in L(G)^n$. Therefore $w = uv \in L(G) \cdot L(G)^n = L(G)^{n+1}$.

Conversely, let $w \in L(G)^{n+1}$. Then $w = uv$ for some $u \in L(G)$ and $v \in L(G)^n$. Choose a derivation $S \Rightarrow_G^* u$ in G . Its last step uses some production $A \rightarrow x$ in P ; in G_* this production has been replaced by $A \rightarrow xZ$, so the same derivation yields $S \Rightarrow_{G_*}^* uZ$. By the induction hypothesis, $Z \Rightarrow_{G_*}^* v$ using exactly n productions of the form $A \rightarrow xZ$. Thus

$$Z \Rightarrow S \Rightarrow_{G_*}^* uZ \Rightarrow_{G_*}^* uv = w$$

uses exactly $n + 1$ such productions. Hence the words generated in this way are exactly the words in $L(G)^{n+1}$. $\square$

8.10. Use the efficient algorithm of Theorem 8.3 to find regular expressions for $L(G_5)$, $L(G_6)$, $L(G_7)$ in Exercise 8.3.

The efficient algorithm eliminates one variable at a time using Theorem 6.1.

G_5 : Equations: $S_0 = \mathbf{0}S_2 \cup \mathbf{1}S_1 \cup \lambda$, $S_1 = \mathbf{0}S_1 \cup \mathbf{1}S_2$, $S_2 = \mathbf{0}S_2 \cup \mathbf{1}S_0$. Solve for S_2 from third equation: $S_2 = \mathbf{0}^* \mathbf{1}S_0$. Substitute into S_1 : $S_1 = \mathbf{0}S_1 \cup \mathbf{10}^* \mathbf{1}S_0$, so $S_1 = \mathbf{0}^* \mathbf{10}^* \mathbf{1}S_0$. Substitute S_1 and S_2 into S_0 : $S_0 = \mathbf{0} \cdot \mathbf{0}^* \mathbf{1}S_0 \cup \mathbf{1} \cdot \mathbf{0}^* \mathbf{10}^* \mathbf{1}S_0 \cup \lambda = (\mathbf{0}^+ \mathbf{1} \cup \mathbf{10}^* \mathbf{10}^* \mathbf{1})S_0 \cup \lambda$. By Theorem 6.1,

$$L(G_5) = (\mathbf{0}^+ \mathbf{1} \cup \mathbf{10}^* \mathbf{10}^* \mathbf{1})^*.$$

G_6 : Equations: $Z = \mathbf{a}Z \cup \mathbf{b}T$, $T = \mathbf{a}Z$. Substitute: $Z = \mathbf{a}Z \cup \mathbf{ba}Z = (\mathbf{a} \cup \mathbf{ba})Z \cup \emptyset$. By Theorem 6.1, $Z = (\mathbf{a} \cup \mathbf{ba})^* \cdot \emptyset = \emptyset$. So $L(G_6) = \emptyset$.

G_7 : Equations: $S = \mathbf{a}S \cup \mathbf{ab}B \cup \mathbf{c}C$, $B = \mathbf{ab}B \cup \lambda$, $C = \mathbf{c}C \cup \mathbf{ca}$. Solve $B = (\mathbf{ab})^*$. Solve $C = \mathbf{c}^* \mathbf{ca}$. Substitute: $S = \mathbf{a}S \cup \mathbf{ab}(\mathbf{ab})^* \cup \mathbf{c} \cdot \mathbf{c}^* \mathbf{ca} = \mathbf{a}S \cup (\mathbf{ab})^+ \cup \mathbf{c}^+ \mathbf{a}$. By Theorem 6.1:

$$L(G_7) = \mathbf{a}^* ((\mathbf{ab})^+ \cup \mathbf{c}^+ \mathbf{a}) = \mathbf{a}^* (\mathbf{ab})^+ \cup \mathbf{a}^* \mathbf{c}^+ \mathbf{a}.$$

8.11. Left-linear version of Theorem 8.3 and Lemmas 8.4–8.5.

(a) Let

$$G = \langle \{S_1, S_2, \dots, S_n\}, \Sigma, S_1, P \rangle$$

be a left-linear grammar, and for each nonterminal S_i let X_{S_i} be the set of terminal strings derivable from S_i . Then

$$X_{S_k} = E_k \cup X_{S_1} A_{1k} \cup X_{S_2} A_{2k} \cup \dots \cup X_{S_n} A_{nk}, \quad k = 1, 2, \dots, n,$$

where E_k is the union of all terminal strings x that appear in productions of the form $S_k \rightarrow x$, and A_{jk} is the union of all terminal strings x that appear in productions of the form $S_k \rightarrow S_j x$.

(b) The left-linear analogue of Lemma 8.4 is: if

$$X = E \cup XA,$$

then EA^* is always a solution, and any other solution Y must satisfy $EA^* \subseteq Y$.

The left-linear analogue of Lemma 8.5 is obtained by eliminating the last unknown on the right. If

$$X_n = E_n \cup X_1 A_{1n} \cup \dots \cup X_{n-1} A_{(n-1)n} \cup X_n A_{nn},$$

then the compatible minimal choice for X_n is

$$X_n = (E_n \cup X_1 A_{1n} \cup \dots \cup X_{n-1} A_{(n-1)n}) A_{nn}^*.$$

Substituting this into the first $n - 1$ equations yields

$$\hat{E}_i = E_i \cup E_n A_{nn}^* A_{ni}, \quad \hat{A}_{ji} = A_{ji} \cup A_{jn} A_{nn}^* A_{ni},$$

for $i, j = 1, 2, \dots, n - 1$.

(c) For G_2 in Example 8.12, the language equations are

$$X_S = X_A \mathbf{baa}, \quad X_A = \lambda \cup X_A \mathbf{a}.$$

By the left-linear version of Theorem 6.1,

$$X_A = \lambda \mathbf{a}^* = \mathbf{a}^*.$$

Therefore

$$L(G_2) = X_S = X_A \mathbf{baa} = \mathbf{a}^* \mathbf{baa}.$$

8.12. Grammar Q for binary numbers with decimal point.

(a) A correct NDFA is

$$A = \langle \{\mathbf{0}, \mathbf{1}, \cdot\}, \{I, D, F\}, I, \delta, \{F\} \rangle,$$

where

$$\begin{aligned} \delta(I, \mathbf{0}) &= \{I, D\}, & \delta(I, \mathbf{1}) &= \{I, D\}, \\ \delta(D, \cdot) &= \{F\}, & \delta(F, \mathbf{0}) &= \{F\}, & \delta(F, \mathbf{1}) &= \{F\}, \end{aligned}$$

and all other transitions are empty. The state D means “the last digit before the decimal point has just been read.” This accepts strings such as **011.** and **101.11**, but not strings beginning with a dot.

(b) Right-linear equations:

$$\begin{aligned} I &= \mathbf{0}I \cup \mathbf{1}I \cup \mathbf{0}.F \cup \mathbf{1}.F \\ F &= \lambda \cup \mathbf{0}F \cup \mathbf{1}F \end{aligned}$$

(c) Solving: $F = (\mathbf{0} \cup \mathbf{1})^*$ by Theorem 6.1 applied to $F = (\mathbf{0} \cup \mathbf{1})F \cup \lambda$. Substituting into I : $I = (\mathbf{0} \cup \mathbf{1})I \cup (\mathbf{0} \cup \mathbf{1}).(\mathbf{0} \cup \mathbf{1})^*$. By Theorem 6.1, $I = (\mathbf{0} \cup \mathbf{1})^+ . (\mathbf{0} \cup \mathbf{1})^*$.

8.13. Right-linear grammars for the given regular expressions.

(a) $(\mathbf{a} \cup \mathbf{b})\mathbf{c}^*(\mathbf{d} \cup (\mathbf{ab})^*)$:

$$\begin{aligned} G &= \langle \{S, A, B, C\}, \{\mathbf{a}, \mathbf{b}, \mathbf{c}, \mathbf{d}\}, S, P \rangle \\ P &= \{S \rightarrow \mathbf{a}A | \mathbf{b}A, A \rightarrow \mathbf{c}A | \mathbf{d} | \mathbf{ab} | \lambda, B \rightarrow \mathbf{b}C, C \rightarrow \mathbf{a}B | \lambda\} \end{aligned}$$

State A handles $\mathbf{c}^*$ and the choice of $\mathbf{d}$ or $(\mathbf{ab})^*$; state B after seeing $\mathbf{a}$ in the $(\mathbf{ab})^*$ part; state C after $\mathbf{ab}$.

(b) $(\mathbf{a} \cup \mathbf{b})^* \mathbf{a}(\mathbf{a} \cup \mathbf{b})^*$ (strings containing at least one $\mathbf{a}$):

$$G = \langle \{S, A\}, \{\mathbf{a}, \mathbf{b}\}, S, \{S \rightarrow \mathbf{aS} | \mathbf{bS} | \mathbf{a}A, A \rightarrow \mathbf{a}A | \mathbf{b}A | \lambda\} \rangle$$

State S generates the arbitrary prefix before the required occurrence of $\mathbf{a}$; the production $S \rightarrow \mathbf{a}A$ supplies that required occurrence. State A then generates the arbitrary suffix and may terminate by $A \rightarrow \lambda$.

8.14. Left-linear grammars for the same expressions.

- (a) $(a \cup b)c^*(d \cup (ab)^*)$: Reverse the order of derivation.

$$G = \langle \{S, A, B\}, \{a, b, c, d\}, S, \{S \rightarrow Ad|B, A \rightarrow Ac|a|b, B \rightarrow Cb|A, C \rightarrow Ba\} \rangle$$

Here A generates $(a \cup b)c^*$, so the production $S \rightarrow Ad$ gives the d branch. The other branch starts with $S \rightarrow B$, then $B \rightarrow A$ gives the $(ab)^0$ case, while the cycle $B \rightarrow Cb \rightarrow Bab$ adds one more copy of (ab) on the right.

- (b) $(a \cup b)^*a(a \cup b)^*$:

$$G = \langle \{S, A\}, \{a, b\}, S, \{S \rightarrow Sa|Sb|Aa, A \rightarrow Aa|Ab|\lambda\} \rangle$$

Here A generates the arbitrary prefix, and the production $S \rightarrow Aa$ places the required a after that prefix. The productions $S \rightarrow Sa$ and $S \rightarrow Sb$ append the arbitrary suffix on the right.

8.15. Algorithm to convert right-linear to left-linear grammar.

- (a) **Algorithm:**

- i. From the right-linear grammar G , construct the NDFA A_G of Lemma 8.2.
- ii. Reverse all arrows in A_G and interchange initial and final states. As in the reverse construction used in Exercise 5.20 and cited in the proof of Theorem 8.2, the resulting machine accepts $L(G)^r$.
- iii. Apply the subset construction from Definition 4.5 to obtain a DFA for $L(G)^r$.
- iv. Apply Lemma 8.1 to that DFA to get a right-linear grammar H for $L(G)^r$.
- v. Reverse every production in H . By Lemma 8.3, the resulting grammar is left-linear and generates

$$L(H)^r = (L(G)^r)^r = L(G).$$

- (b) Applying this to G_4 : first construct A_{G_4} from Lemma 8.2, then reverse it, determinize, and apply Lemma 8.1. The resulting right-linear grammar for $L(G_4)^r$ is

$$H = \langle \{R, U, V, W, X, L, M, N, P\}, \Sigma, R, P_H \rangle,$$

where P_H contains

$$\begin{aligned} R &\rightarrow bU \\ U &\rightarrow bV \mid aW \\ V &\rightarrow aL \\ W &\rightarrow bX \\ X &\rightarrow bL \\ L &\rightarrow \lambda \mid cM \\ M &\rightarrow bN \\ N &\rightarrow bP \\ P &\rightarrow aL. \end{aligned}$$

Reversing every production now yields a left-linear grammar for $L(G_4)$:

$$\begin{aligned}
 R &\rightarrow U\mathbf{b} \\
 U &\rightarrow V\mathbf{b} \mid W\mathbf{a} \\
 V &\rightarrow L\mathbf{a} \\
 W &\rightarrow X\mathbf{b} \\
 X &\rightarrow L\mathbf{b} \\
 L &\rightarrow \lambda \mid M\mathbf{c} \\
 M &\rightarrow N\mathbf{b} \\
 N &\rightarrow P\mathbf{b} \\
 P &\rightarrow L\mathbf{a}.
 \end{aligned}$$

8.16. Algorithm to convert a regular grammar to one generating the complement.

- (a) Construct an automaton for $L(G)$ from grammar G using Lemma 8.2, and then, if necessary, convert it to an equivalent DFA A by Definition 4.5.
- (b) Compute the complement DFA A^c (as outlined in the proof of Theorem 5.1) by keeping the same states, start state, and transition function as A , and replacing the final-state set F by $S - F$.
- (c) Construct a right-linear grammar G^c from A^c . Then $L(G^c) = \overline{L(G)}$.

8.17. Algorithm to determine whether $L(G) = \emptyset$ for a regular grammar G .

- (a) Build the automaton A_G from G .
- (b) Compute the connected state set S^c from the start state by the successive C_i construction of Definition 3.10. The assertion proved in Exercise 3.53 justifies that these C_i sets produce exactly the states reachable by strings of bounded length, so the iteration is the textbook reachability algorithm.
- (c) If no final state is reachable, $L(G) = \emptyset$; otherwise $L(G) \neq \emptyset$.

This terminates since the automaton has finitely many states.

8.18. Algorithm to determine whether $L(G)$ is infinite for a regular grammar G .

- (a) Build the automaton A_G from G .
- (b) Find all states reachable from the start state.
- (c) Find all states from which a final state is reachable.
- (d) Check whether the intersection of these two sets (the “useful” states) contains any cycle (e.g., by DFS looking for back edges).
- (e) $L(G)$ is infinite iff a cycle exists among the useful states.

8.19. Algorithm to determine whether two right-linear grammars G_1 and G_2 generate the same language.

(a) Construct DFAs $A_1 = \langle \Sigma, S_1, s_{0_1}, \delta_1, F_1 \rangle$ and $A_2 = \langle \Sigma, S_2, s_{0_2}, \delta_2, F_2 \rangle$ from G_1 and G_2 , using Lemma 8.2 followed by Definition 4.5 when determinization is needed. Rename states if necessary so that $S_1 \cap S_2 = \emptyset$.

(b) Form the composite DFA

$$A = \langle \Sigma, S_1 \cup S_2, s_{0_1}, \delta, F_1 \cup F_2 \rangle,$$

where $\delta(s, \mathbf{a}) = \delta_1(s, \mathbf{a})$ for $s \in S_1$ and $\delta(s, \mathbf{a}) = \delta_2(s, \mathbf{a})$ for $s \in S_2$.

(c) Use the Chapter 3 algorithm for computing E_A (Corollary 3.5).

(d) Answer yes exactly when $s_{0_1} E_A s_{0_2}$. If the two start states are state-equivalent in the composite machine, every input word either reaches final states in both component machines or reaches nonfinal states in both; if they are not equivalent, some word is accepted by exactly one component machine.

8.20. Determine $L(H)$ for grammar H .

The grammar is $H = \langle \{A, B, S\}, \{\mathbf{a}, \mathbf{b}, \mathbf{c}\}, S, \{S \rightarrow \mathbf{aSBc}, S \rightarrow \lambda, SB \rightarrow \mathbf{bS}, \mathbf{cB} \rightarrow \mathbf{Bc}\} \rangle$.

This grammar generates λ , and for every $n \geq 2$ it generates $\mathbf{a}^n \mathbf{b}^n \mathbf{c}^n$. For example,

$$S \Rightarrow \mathbf{aaSBcBc} \Rightarrow \mathbf{aaSBBcc} \Rightarrow \mathbf{aabSBcc} \Rightarrow \mathbf{aabbScc} \Rightarrow \mathbf{aabbcc}.$$

In general, after n uses of $S \rightarrow \mathbf{aSBc}$ we have

$$\mathbf{a}^n \mathbf{S} \mathbf{b}^n \mathbf{c}^n.$$

Repeatedly use $\mathbf{cB} \rightarrow \mathbf{Bc}$ to move one B left until it is adjacent to S , then apply $SB \rightarrow \mathbf{bS}$. Doing this n times yields

$$\mathbf{a}^n \mathbf{b}^n \mathbf{S} \mathbf{c}^n,$$

and then $S \rightarrow \lambda$ gives $\mathbf{a}^n \mathbf{b}^n \mathbf{c}^n$.

However, $\mathbf{abc}$ is *not* generated: after one use of $S \rightarrow \mathbf{aSBc}$ and then $S \rightarrow \lambda$ we obtain $\mathbf{aBc}$, and there is no rule that can eliminate that lone B . Thus the nonempty terminal strings begin at length 6.

Therefore

$$L(H) = \{\lambda\} \cup \{\mathbf{a}^n \mathbf{b}^n \mathbf{c}^n \mid n \geq 2\}.$$

8.21. What is wrong with the concatenation construction $G^\bullet = \langle \Omega_1 \cup \Omega_2 \cup \{Z\}, \Sigma, Z, P_1 \cup P_2 \cup \{Z \rightarrow S_1 \cdot S_2\} \rangle$?

The production $Z \rightarrow S_1 S_2$ has two nonterminals on the right-hand side. In a right-linear grammar, all productions must have the form $A \rightarrow w$ or $A \rightarrow wB$ (a string of terminals followed by at most one nonterminal). The production $S_1 S_2$ violates this since S_1 is a nonterminal followed by S_2 (also nonterminal), with no terminal prefix. This is a context-free production, not a right-linear one, so the resulting grammar is not right-linear and cannot directly represent a language in $\mathfrak{G}_\Sigma$.

8.22. Prove $\mathfrak{G}_\Sigma$ is closed under concatenation.

- (a) Construction: Let $G_1 = \langle \Omega_1, \Sigma, S_1, P_1 \rangle$ and $G_2 = \langle \Omega_2, \Sigma, S_2, P_2 \rangle$ be right-linear grammars with $\Omega_1 \cap \Omega_2 = \emptyset$. Define:

$$G^* = \langle \Omega_1 \cup \Omega_2, \Sigma, S_1, P^* \rangle$$

where P^* is obtained from P_1 by replacing each terminal production $A \rightarrow w$ (where $A \in \Omega_1$) with $A \rightarrow wS_2$, plus all productions from P_2 unchanged.

- (b) Proof that $L(G^*) = L(G_1) \cdot L(G_2)$: If $u \in L(G_1)$ and $v \in L(G_2)$, then $S_1 \xRightarrow{*} u$ in G_1 via some derivation ending with $A \rightarrow u'$ (terminal production). In G^* , this becomes $S_1 \xRightarrow{*} u' S_2 \xRightarrow{*} u' v = uv$. Conversely, any derivation in G^* must use rules from P_1 until reaching a production $A \rightarrow wS_2$, producing some prefix, then switch to P_2 . $\square$

8.23. Construct a λ -NFA A' with one start and one final state from an NFA A without λ -transitions.

Let $A = \langle \Sigma, S, S_0, \delta, F \rangle$ be an NFA without λ -transitions.

Add two new states i and f , where i will be the unique start state and f the unique final state. Define

$$A' = \langle \Sigma, S \cup \{i, f\}, \{i\}, \delta', \{f\} \rangle,$$

where

$$\delta'(s, \mathbf{a}) = \delta(s, \mathbf{a}) \quad (s \in S, \mathbf{a} \in \Sigma),$$

$$\delta'(i, \lambda) = S_0,$$

and for every old final state $p \in F$ we add the λ -transition

$$f \in \delta'(p, \lambda).$$

There are no other new transitions.

Every accepting path of A becomes an accepting path of A' by first taking a λ -move from i to one of the states in the start set S_0 , following the old transitions of A , and then taking a final λ -move into f . Conversely, any accepting path in A' must begin with one of the λ -moves from i to a state in S_0 , use only old transitions while reading the input, and end with a λ -move from an old final state into f . Hence

$$L(A') = L(A),$$

and A' has exactly one start state and exactly one final state.

8.24. Complete the proof of Lemma 8.2.

- (a) Inductive statement: For all $n \geq 0$ and all $x \in \Sigma^n$ and all nonterminals A : $A \xRightarrow{*} x$ in G iff $\bar{\delta}_A(A, x) \in F$.
- (b) Proof by induction on n : **Base** ($n = 0$): $A \xRightarrow{*} \lambda$ iff $A \rightarrow \lambda \in P$ iff $A \in F$ in the NFA iff $\bar{\delta}(A, \lambda) = A \in F$. **Step**: Assume the result holds for length n . For $x = ay$ ($|y| = n$): $A \xRightarrow{*} ay$ iff $(\exists B) A \rightarrow \mathbf{a}B \in P$ and $B \xRightarrow{*} y$ iff $(\exists B) B \in \delta(A, \mathbf{a})$ and (by IH) $\bar{\delta}(B, y) \in F$ iff $\bar{\delta}(A, ay) \in F$. $\square$

8.25. Complete the proof of Lemma 8.3.

(a) Inductive statement: for every $n \geq 0$, every pair of nonterminals A, B , and every $x \in \Sigma^*$,

$$A \Rightarrow_G^{*n} Bx \iff A \Rightarrow_{G^r}^{*n} x^r B.$$

(b) Proof by induction on n .

Base ($n = 0$): A zero-step derivation has the form $A \Rightarrow^{*0} A$, so $B = A$ and $x = \lambda$. Then also

$$A \Rightarrow_{G^r}^{*0} \lambda^r A = A.$$

Step: Assume the statement true for n productions. Suppose

$$A \Rightarrow_G^{*n+1} By.$$

Then the first production used in G has the form

$$A \rightarrow Cz$$

for some $C \in \Omega \cup \{\lambda\}$ and some $z \in \Sigma^*$, and after that

$$C \Rightarrow_G^{*n} Bx$$

with $y = zx$. By the induction hypothesis,

$$C \Rightarrow_{G^r}^{*n} x^r B.$$

Since G^r contains the reversed production

$$A \rightarrow z^r C,$$

we obtain

$$A \Rightarrow_{G^r} z^r C \Rightarrow_{G^r}^{*n} z^r x^r B = (xz)^r B = y^r B.$$

The converse implication is identical with the roles of G and G^r reversed. Therefore the statement holds for all n . In particular, terminal derivations are reversed, so

$$L(G^r) = L(G)^r.$$

□

8.26. Reasons for the assertions in the second half of the proof of Theorem 8.2.

The second half of Theorem 8.2 begins with a language L generated by a right-linear grammar.

- (a) Since L is generated by a right-linear grammar, Theorem 8.1 implies that $L \in \mathfrak{D}_\Sigma$.
- (b) Exercise 5.20 (or 6.36) shows that $\mathfrak{D}_\Sigma$ is closed under reverse, so $L^r \in \mathfrak{D}_\Sigma$.
- (c) Because $\mathfrak{D}_\Sigma = \mathfrak{G}_\Sigma$, there is a right-linear grammar G such that $L(G) = L^r$.

(d) Lemma 8.3 says that G^r is a left-linear grammar and that

$$L(G^r) = L(G)^r.$$

Since $L(G) = L^r$, this becomes

$$L(G^r) = (L^r)^r = L.$$

(e) Therefore G^r is a left-linear grammar that generates L . This is the “corresponding left-linear grammar” claimed in the theorem.

8.27. Formal inductive proofs.

(a) $L(G_1) = \{a^n baa \mid n \in \mathbb{N}\}$ (Example 8.7):

Claim: For all $n \geq 0$, $S \Rightarrow^* a^n baa$ in G_1 . **Base** ($n = 0$): $S \Rightarrow baa$ (production $S \rightarrow baa$). **Step:** If $S \Rightarrow^* a^n baa$, then $S \Rightarrow^* aS \Rightarrow^* a \cdot a^n baa = a^{n+1} baa$. Conversely, any derivation from S either uses $S \rightarrow baa$ (giving $\lambda \cdot baa$) or $S \rightarrow aS$ (giving a longer string by IH). So $L(G_1) = \{a^n baa\}$. $\square$

(b) $L(G_8) = \{a^n baa \mid n \in \mathbb{N}\}$ (Example 8.9): **Claim:** For every $n \geq 0$, $Z \Rightarrow^* a^n baa$ in G_8 .

Base ($n = 0$): $Z \Rightarrow baa$.

Step: If $Z \Rightarrow^* a^n baa$, then $Z \Rightarrow^* aZ \Rightarrow^* a^{n+1} baa$.

Conversely, any derivation from Z either uses $Z \rightarrow baa$ immediately or begins with $Z \rightarrow aZ$. Repeating the second choice n times and then using $Z \rightarrow baa$ yields exactly the string $a^n baa$, and there is no other way to terminate a derivation. Therefore $L(G_8) = \{a^n baa \mid n \in \mathbb{N}\}$. $\square$

8.28. Mixed grammar C with both right-linear and left-linear productions.

$C = \langle \{S, A, B\}, \{0, 1\}, S, \{S \rightarrow 0A \mid 1B \mid 0 \mid 1 \mid \lambda, A \rightarrow S0, B \rightarrow S1\} \rangle$.

(a) $L(C) = \{w \in \{0, 1\}^* \mid w = w^r\}$ — the set of all palindromes over $\{0, 1\}$.

Derivation: $S \Rightarrow 0A \Rightarrow 0S0$. Each step wraps the current sentential form between matching symbols. The grammar generates all even-length and odd-length palindromes.

(b) No. $L(C) = \{w \mid w = w^r\}$ is not regular; it requires unbounded memory to match the first and last symbols. Thus $L(C)$ is not FAD.

(c) The definition of regular grammars should *not* be expanded to include such mixed grammars, because the resulting class of languages would no longer coincide with the regular languages. Mixed grammars generate languages (like palindromes) that are context-free but not regular.

8.29. Importance of $\Omega_1 \cap \Omega_2 = \emptyset$ in Theorem 8.4.

(a) **Why important:** If $\Omega_1 \cap \Omega_2 \neq \emptyset$, then a shared nonterminal A would have its productions from both P_1 and P_2 merged. Derivations starting from S_1 could use P_2 productions for A and vice versa, producing strings not in $L(G_1) \cup L(G_2)$. **Example:** Let G_1 have $\{S_1 \rightarrow aA, A \rightarrow b\}$ and G_2 have $\{S_2 \rightarrow cA, A \rightarrow d\}$. If A is shared, the union grammar with $Z \rightarrow S_1 \mid S_2$ would derive aA and then use $A \rightarrow d$ from G_2 , producing $ad \notin L(G_1) \cup L(G_2)$.

(b) **Why possible:** We can always rename nonterminals in G_2 (choosing fresh names) to make $\Omega_1 \cap \Omega_2 = \emptyset$ without changing the language $L(G_2)$. This is a standard renaming/relabeling argument.

8.30. If A_G is disconnected, what does this say about G ?

If A_G (the N DFA constructed from G) is disconnected, it means some states (nonterminals) are not reachable from the start state. This means the corresponding nonterminals in G are *useless*: they cannot appear in any derivation from the start symbol. The portions of G defining those nonterminals contribute nothing to $L(G)$ and can be removed (along with all productions involving the useless nonterminals) without affecting the language generated.

8.31. Apply Lemma 8.1 to the automata in Figure 8.4.

Applying Lemma 8.1 means: use one nonterminal per state, use the start state as start symbol, and add one production for each labeled transition; final states also get λ -productions.

(a) Figure 8.4(a):

$$\begin{aligned} r_0 &\rightarrow ar_3 \mid br_1 \mid \lambda \\ r_1 &\rightarrow ar_0 \mid br_2 \\ r_2 &\rightarrow ar_1 \mid br_3 \\ r_3 &\rightarrow ar_2 \mid br_0. \end{aligned}$$

(b) Figure 8.4(b):

$$\begin{aligned} q_0 &\rightarrow aq_1 \mid bq_0 \mid cq_0 \mid \lambda \\ q_1 &\rightarrow aq_0 \mid bq_0 \mid cq_1. \end{aligned}$$

(c) Figure 8.4(c):

$$\begin{aligned} S_0 &\rightarrow aS_1 \mid bS_2 \\ S_1 &\rightarrow aS_3 \mid bS_2 \\ S_2 &\rightarrow aS_2 \mid bS_2 \mid \lambda \\ S_3 &\rightarrow aS_0 \mid bS_1. \end{aligned}$$

(d) Figure 8.4(d):

$$\begin{aligned} t_0 &\rightarrow at_1 \mid bt_2 \\ t_1 &\rightarrow at_0 \mid bt_2 \mid \lambda \\ t_2 &\rightarrow at_3 \mid bt_3 \\ t_3 &\rightarrow at_1 \mid bt_1 \mid \lambda. \end{aligned}$$

(e) Figure 8.4(e):

$$\begin{aligned} S &\rightarrow ar \mid br \mid bt \mid cq \mid \lambda \\ r &\rightarrow aS \mid bt \mid cq \\ q &\rightarrow ar \mid br \mid cq \mid \lambda \\ t &\rightarrow ar \mid cq. \end{aligned}$$

8.32. Restate Lemma 8.1 for NDFAs.

(a) **Restated Lemma:** Given an N DFA

$$A = \langle \Sigma, S, S_0, \delta, F \rangle,$$

define

$$G_A = \langle S \cup \{Z\}, \Sigma, Z, P \rangle$$

where $Z \notin S$, by adding

- a production $Z \rightarrow s$ for every start state $s \in S_0$,
- a production $s \rightarrow \mathbf{a}t$ for every $t \in \delta(s, \mathbf{a})$,
- a production $s \rightarrow t$ for every $t \in \delta(s, \lambda)$,
- a production $s \rightarrow \lambda$ for every final state $s \in F$.

Then $L(G_A) = L(A)$.

- (b) **Proof:** The productions from Z choose exactly one initial state from the NDFA start set S_0 . Every later production records one NDFA move: letter transitions become productions of the form $s \rightarrow \mathbf{a}t$, λ -transitions become unit productions $s \rightarrow t$, and final states get the production $s \rightarrow \lambda$. Thus a derivation in G_A corresponds exactly to a path in A , and the derivation ends in a terminal word precisely when the corresponding path ends in a final state. Therefore $L(G_A) = L(A)$.

- (c) Applied to Figure 8.5:

- i. Figure 8.5(a):

$$Z \rightarrow t_0, \quad t_0 \rightarrow \mathbf{a}t_0 \mid \mathbf{c}t_0 \mid \lambda.$$

- ii. Figure 8.5(b):

$$\begin{aligned} Z &\rightarrow q_1 \mid q_0 \\ q_1 &\rightarrow \mathbf{a}q_0 \mid \mathbf{b}q_1 \mid \lambda \\ q_0 &\rightarrow \mathbf{b}q_1 \mid \mathbf{c}q_0. \end{aligned}$$

- iii. Figure 8.5(c):

$$\begin{aligned} Z &\rightarrow r_0 \mid r_1 \\ r_0 &\rightarrow \mathbf{a}r_0 \mid \mathbf{a}r_1 \mid \mathbf{c}r_1 \mid \lambda \\ r_1 &\rightarrow \mathbf{b}r_0. \end{aligned}$$

- iv. Figure 8.5(d):

$$\begin{aligned} Z &\rightarrow S_0 \\ S_0 &\rightarrow \mathbf{a}S_1 \\ S_1 &\rightarrow \mathbf{a}S_2 \mid \mathbf{b}S_3 \\ S_2 &\rightarrow \lambda \\ S_3 &\rightarrow \mathbf{b}S_1. \end{aligned}$$

- v. Figure 8.5(e):

$$\begin{aligned} Z &\rightarrow S_1 \mid S_2 \\ S_1 &\rightarrow \mathbf{a}S_1 \mid \mathbf{a}S_2 \\ S_2 &\rightarrow \mathbf{a}S_1 \mid \mathbf{b}S_1 \mid \lambda. \end{aligned}$$

- vi. Figure 8.5(f):

$$\begin{aligned} Z &\rightarrow S_0 \\ S_0 &\rightarrow \mathbf{a}S_1 \\ S_1 &\rightarrow S_0 \mid \mathbf{b}S_2 \\ S_2 &\rightarrow \mathbf{c}S_3 \\ S_3 &\rightarrow S_1 \mid \lambda. \end{aligned}$$

vii. Figure 8.5(g):

$$\begin{aligned} Z &\rightarrow S_0 \\ S_0 &\rightarrow \mathbf{a}S_1 \mid \lambda \\ S_1 &\rightarrow \mathbf{a}S_0 \mid \mathbf{b}S_3 \mid \mathbf{c}S_2 \\ S_2 &\rightarrow \mathbf{c}S_1 \\ S_3 &\rightarrow \mathbf{b}S_1. \end{aligned}$$

8.33. **Context-free grammars for languages over $\Sigma = \{\mathbf{a}, \mathbf{b}\}$.**

(a) L_1 = words where last letter matches first letter:

$$G_1 : S \rightarrow \mathbf{a}X\mathbf{a} \mid \mathbf{b}X\mathbf{b} \mid \mathbf{a} \mid \mathbf{b}, \quad X \rightarrow \mathbf{a}X \mid \mathbf{b}X \mid \lambda$$

The empty word is excluded here, since it has no first or last letter.

(b) L_2 = odd-length words where first letter matches center letter: Let $n = 2k + 1$.

$$\begin{aligned} G_2 : S &\rightarrow \mathbf{a} \mid \mathbf{b} \mid \mathbf{a}A\mathbf{a} \mid \mathbf{a}A\mathbf{b} \mid \mathbf{b}B\mathbf{a} \mid \mathbf{b}B\mathbf{b}, \\ A &\rightarrow \mathbf{a} \mid \mathbf{a}A\mathbf{a} \mid \mathbf{a}A\mathbf{b} \mid \mathbf{b}A\mathbf{a} \mid \mathbf{b}A\mathbf{b}, \\ B &\rightarrow \mathbf{b} \mid \mathbf{a}B\mathbf{a} \mid \mathbf{a}B\mathbf{b} \mid \mathbf{b}B\mathbf{a} \mid \mathbf{b}B\mathbf{b} \end{aligned}$$

The nonterminal A generates all odd-length words whose center letter is $\mathbf{a}$, and B does the same for center letter $\mathbf{b}$. The productions for S force the first letter to match that center letter.

(c) L_3 = words where last letter matches none of the other letters:

$$G_3 : S \rightarrow X\mathbf{a} \mid X\mathbf{b}, \quad \text{where } X \text{ generates strings not ending in } \mathbf{a} \text{ (resp. } \mathbf{b})$$

Since $\Sigma = \{\mathbf{a}, \mathbf{b}\}$, “last letter matches none of the others” means all other letters differ. This is regular: last letter $\mathbf{a}$, all others $\mathbf{b}$; or last letter $\mathbf{b}$, all others $\mathbf{a}$: $L_3 = \mathbf{b}^*\mathbf{a} \cup \mathbf{a}^*\mathbf{b}$. So $G_3 : S \rightarrow B\mathbf{a} \mid A\mathbf{b} \mid \mathbf{a} \mid \mathbf{b}, B \rightarrow B\mathbf{b} \mid \lambda, A \rightarrow A\mathbf{a} \mid \lambda$.

(d) L_4 = even-length words where two center letters match:

$$G_4 : S \rightarrow \mathbf{a}S\mathbf{a} \mid \mathbf{a}S\mathbf{b} \mid \mathbf{b}S\mathbf{a} \mid \mathbf{b}S\mathbf{b} \mid \mathbf{a}\mathbf{a} \mid \mathbf{b}\mathbf{b}$$

(e) L_5 = odd-length words where center letter matches none of the others: Since $\Sigma = \{\mathbf{a}, \mathbf{b}\}$, if the center letter is $\mathbf{a}$ then every other letter must be $\mathbf{b}$, and if the center letter is $\mathbf{b}$ then every other letter must be $\mathbf{a}$. Therefore

$$L_5 = \{\mathbf{b}^k\mathbf{a}\mathbf{b}^k \mid k \in \mathbb{N}\} \cup \{\mathbf{a}^k\mathbf{b}\mathbf{a}^k \mid k \in \mathbb{N}\},$$

which is context-free but not regular:

$$G_5 : S \rightarrow A \mid B, \quad A \rightarrow \mathbf{b}A\mathbf{b} \mid \mathbf{a}, \quad B \rightarrow \mathbf{a}B\mathbf{a} \mid \mathbf{b}$$

(f) Regular languages: L_1 is regular; L_2 is not regular; L_3 is regular ($\mathbf{b}^*\mathbf{a} \cup \mathbf{a}^*\mathbf{b}$); L_4 is not regular; L_5 is not regular.

8.34. Context-free grammars for the given languages.

- (a) $L = \{x \in \{a, b\}^* \mid |x|_a < |x|_b\}$:

$$G: S \rightarrow E b E \mid E b S, \quad E \rightarrow a E b E \mid b E a E \mid \lambda$$

Here E generates exactly the strings with equally many a 's and b 's. The nonterminal S inserts one or more extra b 's between balanced pieces, so every derived string has strictly more b 's than a 's, and every such string can be decomposed in that way.

- (b) $G = \{x \in \{a, b\}^* \mid |x|_a \geq |x|_b\}$:

$$\begin{aligned} S &\rightarrow E \mid A \\ E &\rightarrow a E b E \mid b E a E \mid \lambda \\ A &\rightarrow E a E \mid E a A. \end{aligned}$$

Again E generates the balanced strings. The nonterminal A inserts one or more extra a 's between balanced pieces, so it generates exactly the strings with more a 's than b 's. Therefore S generates exactly the strings with $|x|_a \geq |x|_b$.

- (c) $K = \{w \in \{0, 1\}^* \mid w = w^r\}$ (palindromes): $S \rightarrow 0S0 \mid 1S1 \mid 0 \mid 1 \mid \lambda$.

- (d) $\Phi = \{a^j b^k c^m \mid j \geq 3, k = m\}$: $S \rightarrow a a a A T, A \rightarrow a A \mid \lambda, T \rightarrow b T c \mid \lambda$.

8.35. More context-free grammars.

- (a) $L_1 = \{x \in \{a, b\}^* \mid |x|_a = 2|x|_b\}$:

$$S \rightarrow \lambda \mid S a S a S b S \mid S a S b S a S \mid S b S a S a S$$

Each nonempty production inserts exactly two a 's and one b , with recursive pieces before, between, and after them, so all generated strings satisfy $|x|_a = 2|x|_b$. Conversely, any nonempty word satisfying this count condition can be split around two occurrences of a and one occurrence of b so that the four intervening pieces satisfy the same count condition; induction on the length of the word then gives a derivation from S .

- (b) $L_2 = \{x \in \{a, b\}^* \mid |x|_a \neq |x|_b\}$:

$$\begin{aligned} S &\rightarrow A \mid B \\ E &\rightarrow a E b E \mid b E a E \mid \lambda \\ A &\rightarrow E a E \mid E a A \\ B &\rightarrow E b E \mid E b B. \end{aligned}$$

Here E generates the words with equally many a 's and b 's. Then A inserts one or more extra a 's between balanced pieces, so it generates exactly the words with more a 's than b 's. The productions for B do the mirror-image job with extra b 's, so B generates exactly the words with more b 's than a 's.

- (c) Postfix expressions over $\{A, B, +, -\}$: $S \rightarrow A \mid B \mid SS+ \mid SS-$.

- (d) Parenthesized infix expressions over $\{A, B, +, -, (,)\}$: $S \rightarrow A \mid B \mid (S + S) \mid (S - S)$.

8.36. Context-sensitive grammars for the given languages.

- (a) $\Gamma = \{w2w \mid w \in \{0,1\}^*\}$: A pure context-sensitive grammar for Γ is

$$G_\Gamma = \langle \{S, A, B\}, \{0, 1, 2\}, S, P \rangle,$$

where

$$\begin{aligned} P = \{ & S \rightarrow 2, S \rightarrow 0AS, S \rightarrow 1BS, \\ & A0 \rightarrow 0A, A1 \rightarrow 1A, B0 \rightarrow 0B, B1 \rightarrow 1B, \\ & A2 \rightarrow 20, B2 \rightarrow 21 \}. \end{aligned}$$

The productions $S \rightarrow 0AS$ and $S \rightarrow 1BS$ generate a sentential form $M2w$, where the marker string M records the symbols of w in the same left-to-right order. The productions $A2 \rightarrow 20$ and $B2 \rightarrow 21$ convert the rightmost remaining marker into the next symbol of the left copy, while the swapping rules move the other markers right until they too meet the 2. For example,

$$S \Rightarrow 0AS \Rightarrow 01ABS \Rightarrow 01AB2 \Rightarrow 01A21 \Rightarrow 0A121 \Rightarrow 01201.$$

Thus $L(G_\Gamma) = \Gamma$.

- (b) $\Phi = \{b^{2^j} \mid j \in \mathbb{N}\} = \{b, bb, bbbb, \dots\}$: A pure context-sensitive grammar for Φ is

$$G_\Phi = \langle \{S, L, D, R\}, \{b\}, S, P \rangle,$$

where

$$P = \{S \rightarrow b, S \rightarrow LR, L \rightarrow LD, L \rightarrow D, Db \rightarrow bbD, DR \rightarrow bR, DR \rightarrow bb\}.$$

Starting from $S \rightarrow LR$, repeated use of $L \rightarrow LD$ followed by $L \rightarrow D$ produces $D^j R$. The rightmost D changes R into one b by $DR \rightarrow bR$ (or into bb if it is the last D). Each remaining D then moves rightward through the current block of b 's by means of $Db \rightarrow bbD$, doubling the length of that block. Hence each D multiplies the number of b 's by 2, and a sentential form with j copies of D yields exactly b^{2^j} . The first derivations are

$$S \Rightarrow b, \quad S \Rightarrow LR \Rightarrow DR \Rightarrow bb,$$

and

$$S \Rightarrow LR \Rightarrow LDR \Rightarrow DDR \Rightarrow DbR \Rightarrow bbDR \Rightarrow bbbb.$$

The only way to begin a derivation for a string longer than b is to choose $S \rightarrow LR$. After $L \rightarrow LD$ has been used $j - 1$ times and $L \rightarrow D$ has removed the L , the sentential form is $D^j R$. In any terminal derivation, each of the first $j - 1$ occurrences of D must eventually meet R and use $DR \rightarrow bR$, while the last occurrence must use $DR \rightarrow bb$; using $DR \rightarrow bb$ earlier leaves a D with no possible terminal completion. Each earlier D crosses the current block of b 's by $Db \rightarrow bbD$, thereby doubling that block before passing control to the next D . Thus $D^j R$ yields exactly b^{2^j} , and every $j \geq 1$ is obtained by the corresponding number of $L \rightarrow LD$ steps. The separate rule $S \rightarrow b$ gives the $j = 0$ case, so the grammar generates all and only the strings in Φ .

8.37. Show G (context-sensitive) is not equivalent to G' in Example 8.3.

The two grammars do *not* generate the same language.

In G'' from Example 8.3 we have

$$S \Rightarrow aSBc \Rightarrow aTBc \Rightarrow abTc \Rightarrow abc,$$

so

$$abc \in L(G'').$$

But $abc \notin L(G)$. In the grammar G , after one use of $S \rightarrow aSBc$ the only way to remove the remaining S is $S \rightarrow \lambda$, which gives

$$aBc.$$

That sentential form is stuck: there is no occurrence of SB to which $SB \rightarrow bS$ can be applied, and there is no occurrence of cB to which $cB \rightarrow Bc$ can be applied. Hence no derivation in G produces abc .

Therefore

$$abc \in L(G'') \setminus L(G),$$

so G is not equivalent to G'' .

8.38. Context-free grammars for the given languages.

- (a) $L_1 = a^*(b \cup c)^* \cap \{x \mid |x|_a = |x|_b + |x|_c\}$: Strings in L_1 are of the form $a^n b^j c^k$ where $n = j + k$:

$$G_1 : S \rightarrow aSb \mid aSc \mid \lambda$$

Each step adds one a at the left and one b or c at the right.

- (b) $L_2 = \{a^i b^j c^k \mid i + j = k\}$:

$$G_2 : S \rightarrow aSc \mid bTc \mid \lambda, \quad T \rightarrow bTc \mid \lambda$$

- (c) $L_3 = \{x \in \{a, b, c\}^* \mid |x|_a + |x|_b = |x|_c\}$:

$$G_3 : S \rightarrow aSc \mid cSa \mid bSc \mid cSb \mid SS \mid \lambda$$

Every production preserves the difference $|x|_c - (|x|_a + |x|_b)$: the four wrapping productions add one c and one non- c symbol, SS concatenates two already balanced strings, and $S \rightarrow \lambda$ supplies the empty balanced string. Hence the grammar generates only strings in L_3 .

Conversely, let $x \in L_3$. If a proper nonempty prefix of x is already balanced, split $x = uv$ at the first such prefix and derive u and v by induction, then use $S \rightarrow SS$. If no proper prefix is balanced, the first and last symbols must be one c and one symbol from $\{a, b\}$; removing those two symbols leaves a shorter balanced string, which is handled by one of $S \rightarrow aSc$, $S \rightarrow bSc$, $S \rightarrow cSa$, or $S \rightarrow cSb$. Thus all required strings are generated. For example, $S \Rightarrow cSb \Rightarrow ccSab \Rightarrow ccab$.

8.39. Prove $L(G') = L(G) \cup \{\lambda\}$ (Definition 8.4).

Definition 8.4 constructs G' from G by adding a new start symbol Z with $Z \rightarrow S \mid \lambda$ (where S was the original start). Then: $x \in L(G')$ iff $Z \xRightarrow{*} x$ in G' . $Z \rightarrow \lambda$ gives $\lambda \in L(G')$. $Z \xRightarrow{*} x$ gives $x \in L(G) \subseteq L(G')$. Conversely, any derivation from Z begins with either $Z \rightarrow \lambda$ or $Z \rightarrow S$, so $L(G') \subseteq L(G) \cup \{\lambda\}$. Hence $L(G') = L(G) \cup \{\lambda\}$. $\square$

8.40. **Prove** $L(G') = L(G) \cup \{\lambda\}$ (**Definition 8.6**).

Definition 8.6 constructs

$$G' = \langle \Omega \cup \{Z\}, \Sigma, Z, P \cup \{Z \rightarrow \lambda, Z \rightarrow S\} \rangle$$

from the pure context-free grammar $G = \langle \Omega, \Sigma, S, P \rangle$.

If $x \in L(G)$, then $S \xRightarrow{*}_G x$, so in G' we have

$$Z \Rightarrow S \xRightarrow{*} x,$$

hence $x \in L(G')$. Also $Z \Rightarrow \lambda$, so $\lambda \in L(G')$. Therefore

$$L(G) \cup \{\lambda\} \subseteq L(G').$$

Conversely, let $x \in L(G')$. Any derivation from the new start symbol Z must begin with either $Z \rightarrow \lambda$ or $Z \rightarrow S$, because these are the only productions having Z on the left. If the first step is $Z \rightarrow \lambda$, then $x = \lambda$. If the first step is $Z \rightarrow S$, the rest of the derivation uses only productions from P , so $S \xRightarrow{*}_G x$ and therefore $x \in L(G)$. Thus

$$L(G') \subseteq L(G) \cup \{\lambda\}.$$

Hence

$$L(G') = L(G) \cup \{\lambda\}.$$

□

8.41. **Properties of the normal form-preserving construction.**

- (a) If G is right-linear (Theorem 8.8), the naive star construction $Z \rightarrow S \mid ZZ \mid \lambda$ does *not* preserve right-linearity: $Z \rightarrow ZZ$ places a nonterminal in a non-final position, violating the form $A \rightarrow wB$ or $A \rightarrow w$. A right-linear Kleene-star construction avoids ZZ by folding the restart into the accepting productions of G : whenever $A \rightarrow w$ is a terminal production of G , add $A \rightarrow wS$ (to restart) and $A \rightarrow w$ (to halt). This keeps all productions right-linear.
- (b) Example:

$$G_1 = \langle \{S_1\}, \{\mathbf{a}, \mathbf{b}\}, S_1, \{S_1 \rightarrow \mathbf{a}\} \rangle, \quad G_2 = \langle \{S_2\}, \{\mathbf{a}, \mathbf{b}\}, S_2, \{S_2 \rightarrow \mathbf{b}\} \rangle.$$

Both grammars are in the Theorem 8.8 normal form: every production is of the form $A \rightarrow \mathbf{a}$ or $A \rightarrow \mathbf{b}$, and there are no contracting productions. But the union construction of Theorem 8.4 gives

$$G^\cup = \langle \{Z, S_1, S_2\}, \{\mathbf{a}, \mathbf{b}\}, Z, \{Z \rightarrow S_1, Z \rightarrow S_2, S_1 \rightarrow \mathbf{a}, S_2 \rightarrow \mathbf{b}\} \rangle.$$

The productions $Z \rightarrow S_1$ and $Z \rightarrow S_2$ are unit productions, not of the form $A \rightarrow \mathbf{a}B$ or $A \rightarrow \mathbf{a}$, so $G^\cup$ is not in the normal form of Theorem 8.8.

- (c) The $G^\bullet$ construction for concatenation in Example 8.18 is not normal form-preserving in general, since it requires replacing terminal productions $A \rightarrow w$ with $A \rightarrow wS_2$, which is a right-linear production but may violate other normal form constraints.

8.42. **Left-linear grammars for union, concatenation, and Kleene closure.**

Given left-linear grammars $G_1 = \langle \Omega_1, \Sigma, S_1, P_1 \rangle$ and $G_2 = \langle \Omega_2, \Sigma, S_2, P_2 \rangle$ with $\Omega_1 \cap \Omega_2 = \emptyset$:

- (a) **Union:** Let $H_1 = G_1^r$ and $H_2 = G_2^r$. By Lemma 8.3 these are right-linear grammars with

$$L(H_1) = L(G_1)^r, \quad L(H_2) = L(G_2)^r.$$

Apply the right-linear union construction of Theorem 8.4 to obtain a right-linear grammar $H^\cup$ for

$$L(H_1) \cup L(H_2) = (L(G_1) \cup L(G_2))^r.$$

Then $(H^\cup)^r$ is a left-linear grammar for $L(G_1) \cup L(G_2)$.

- (b) **Concatenation:** Again let $H_i = G_i^r$. Apply the right-linear concatenation construction to H_2 and H_1 (in that order) to obtain a right-linear grammar $H^\bullet$ such that

$$L(H^\bullet) = L(H_2) \cdot L(H_1) = L(G_2)^r L(G_1)^r = (L(G_1) \cdot L(G_2))^r.$$

Then $(H^\bullet)^r$ is a left-linear grammar for $L(G_1) \cdot L(G_2)$.

- (c) **Kleene closure:** Let $H_1 = G_1^r$, and apply the right-linear Kleene-star construction of Theorem 8.5 to get a right-linear grammar H_* for

$$L(H_*) = L(H_1)^* = (L(G_1)^*)^r.$$

Then $(H_*)^r$ is a left-linear grammar for $L(G_1)^*$.

Chapter 9

Normal Forms and Pumping Lemmas — Solutions

9.1. Characterize the parse trees of left-linear grammars.

In a left-linear grammar, every production has the form $A \rightarrow w$ or $A \rightarrow Bw$, where B is a nonterminal and $w \in \Sigma^*$. Thus the parse trees are the mirror image of the right-linear trees discussed in Example 9.2. More precisely:

- Whenever a node is expanded by a production of the form $A \rightarrow Bw$, its leftmost child is the only child labeled by a nonterminal; all remaining children are terminals.
- Following these leftmost nonterminal children produces a single “left spine” of the tree. Every nonterminal in the parse tree lies on this spine.
- The last nonterminal on the spine is expanded by a rule $A \rightarrow w$, so the bottom of the tree consists entirely of terminal leaves.

So left-linear parse trees are maximally left-branching, just as right-linear parse trees are maximally right-branching.

9.2. Context-free grammars for the following languages.

- (a) $\{a^n b^n c^m d^m \mid n, m \in \mathbb{N}\}$:

$$S \rightarrow AB, \quad A \rightarrow aAb \mid \lambda, \quad B \rightarrow cBd \mid \lambda.$$

Here A generates the balanced $a^n b^n$ block and B generates the independent balanced $c^m d^m$ block. The start symbol concatenates these two independently generated pieces.

- (b) $\{a^i b^j c^j d^i \mid i, j \in \mathbb{N}\}$:

$$S \rightarrow aSd \mid T \mid \lambda, \quad T \rightarrow bTc \mid \lambda.$$

Here S accounts for the outer matching a/d symbols, and then hands control to T for the middle matching b/c symbols. The rule $S \rightarrow \lambda$ is the case $i = j = 0$.

- (c) $\{a^n b^n c^m d^m\} \cup \{a^i b^j c^j d^i\}$:

$$S \rightarrow S_1 \mid S_2$$

where S_1 generates the first language (as in part (a)) and S_2 generates the second (as in part (b)). The two productions from S are the grammar-level union construction.

9.3. Unambiguous context-free grammars.

- (a) i. For $\{a^n b^n c^m d^m\}$: The grammar $S \rightarrow AB$, $A \rightarrow aAb \mid \lambda$, $B \rightarrow cBd \mid \lambda$ is already unambiguous (every string has a unique parse).
 ii. For $\{a^i b^j c^j d^i\}$: The grammar $S \rightarrow aSd \mid T \mid \lambda$, $T \rightarrow bTc \mid \lambda$ is unambiguous.
 iii. For the union $L_1 \cup L_2$, it is possible to make the grammar unambiguous by splitting the union into disjoint pieces:

$$L_1 \cup L_2 = L_1 \cup \{a^i b^j c^j d^i \mid i < j\} \cup \{a^i b^j c^j d^i \mid i > j\}.$$

An unambiguous grammar is

$$S \rightarrow S_1 \mid S_< \mid S_>,$$

where S_1 is the unambiguous grammar from part (i), and

$$S_< \rightarrow abS_<cd \mid U, \quad U \rightarrow bUc \mid bc,$$

$$S_> \rightarrow aS_>d \mid aVd, \quad V \rightarrow abVcd \mid \lambda.$$

The $S_<$ branch generates exactly the case $j > i$, the $S_>$ branch generates exactly the case $i > j$, and the S_1 branch generates L_1 . These three subsets are disjoint, so the grammar is unambiguous.

- (b) The statement is *false*. The example from part (i)(iii) already shows that two unambiguous context-free languages can have a union that is still unambiguous. On the other hand, Theorem 9.1 shows that inherently ambiguous context-free languages do exist, and the standard example

$$\{a^i b^j c^k \mid i = j\} \cup \{a^i b^j c^k \mid j = k\}$$

shows that the union of two unambiguous context-free languages can fail to be unambiguous.

- (c) No. Since there are unambiguous context-free languages whose union is not unambiguous, $\mathcal{U}_\Sigma$ is not closed under union.

9.4. State and prove the inductive result needed in Theorem 9.2.

Let $G' = G_{(A_G^{\sigma d})}$ be the grammar built in Theorem 9.2 from the deterministic automaton $A_G^{\sigma d}$.

Inductive statement: For every state q of $A_G^{\sigma d}$ and every word $x \in \Sigma^n$, there is at most one derivation in G' from the nonterminal q that yields x .

Proof: By induction on n .

Base ($n = 0$): The only possible word is λ . In the grammar obtained from a deterministic automaton, the only way to derive λ from q is via the production $q \rightarrow \lambda$, which exists exactly when q is a final state. Hence there is at most one such derivation.

Step: Assume the statement true for words of length n , and let $x = ay$ with $|y| = n$. Any derivation of x from q must begin with a production of the form

$$q \rightarrow ar,$$

where $r = \delta(q, \mathbf{a})$. Since the machine is deterministic, there is at most one such state r , hence at most one possible first production. After that first step, the rest of the derivation must derive y from r , and by the induction hypothesis there is at most one way to do so. Therefore there is at most one derivation of x from q .

Applying this to the start state shows that every word in $L(G')$ has a unique derivation tree, so G' is unambiguous. $\square$

9.5. **Algorithm to compute $B^u = \{C \mid B \Rightarrow^* C\}$ for each nonterminal B in P^u .**

Consider the unit-production graph: nodes are nonterminals, directed edge $B \rightarrow C$ for each unit production $B \rightarrow C$ in P^u . Then B^u is the set of nonterminals reachable from B in this graph (including B itself).

Algorithm: For each nonterminal B , compute B^u by BFS or DFS from B in the unit-production graph:

- (a) Initialize $B^u = \{B\}$.
- (b) While there exists $C \in B^u$ with a unit production $C \rightarrow D \in P^u$ and $D \notin B^u$: add D to B^u .
- (c) Repeat until no new nonterminals are added.

This is a standard graph reachability computation and terminates since $|\Omega|$ is finite.

9.6. **Closure of $\mathcal{C}_\Sigma$ under concatenation.**

- (a) **What is wrong:** The symbol $\cdot$ in $Z \rightarrow S_1 \cdot S_2$ is meta-notation for concatenation, not a symbol of the grammar. A context-free production must have a right side in $(\Omega \cup \Sigma)^*$, so the proposed rule is not literally a valid production.
- (b) **Fix:** Replace the invalid production with a construction named for concatenation. If Ω_1 and Ω_2 are not disjoint, first rename nonterminals so that they are. When G_1 and G_2 are pure context-free grammars, define

$$G' = \langle \Omega_1 \cup \Omega_2 \cup \{C\}, \Sigma, C, P_1 \cup P_2 \cup \{C \rightarrow S_1 S_2\} \rangle,$$

where $C \notin \Omega_1 \cup \Omega_2 \cup \Sigma$. This is again pure, since the added right side is nonempty. If either original grammar is non-pure only because it uses the special start rules $Z_i \rightarrow \lambda$ and $Z_i \rightarrow S_i$, use the corresponding pure core and add $C \rightarrow S_i$ when the other language contains λ . Finally, add the special rules $Z \rightarrow \lambda$ and $Z \rightarrow C$ only when both languages contain λ . The simpler single production $C \rightarrow S_1 S_2$ is already enough for the pure case and fulfills the exercise's requested repair.

- (c) **Proof:** In the pure case, if $w \in L(G')$, then the first step must be $C \Rightarrow S_1 S_2$. The symbols from S_1 can then derive some $u \in L(G_1)$, and the symbols from S_2 can derive some $v \in L(G_2)$, so $w = uv \in L(G_1) \cdot L(G_2)$. Conversely, if $w = uv$ with $u \in L(G_1)$ and $v \in L(G_2)$, then $C \Rightarrow^* S_1 S_2 \Rightarrow uv = w$. Thus $L(G') = L(G_1) \cdot L(G_2)$.

The non-pure cases add only the productions needed to represent concatenations where one factor is λ ; the special start rules are used exactly when both factors may be λ . Therefore the resulting grammar is a valid context-free grammar for $L(G_1) \cdot L(G_2)$. $\square$

9.7. $\{a^i b^j c^k \mid i, j, k \in \mathbb{N}, i + j = k\}$ is context free.

Grammar: $S \rightarrow aSc \mid T, T \rightarrow bTc \mid \lambda$.

Proof: Any derivation produces $a^i \cdot b^j c^j \cdot c^i = a^i b^j c^{i+j}$. Taking $k = i + j$, we get exactly the language. $\square$

9.8. Ambiguous right-linear grammar and its disambiguation.

(a) $G = \langle \{S, A, B\}, \{a\}, S, \{S \rightarrow A, S \rightarrow B, A \rightarrow aaA, A \rightarrow \lambda, B \rightarrow aaaB, B \rightarrow \lambda\} \rangle$. The string $aaaaaa = a^6$ can be derived as $S \Rightarrow A \Rightarrow^* aaA \Rightarrow^* aaaaA \Rightarrow^* aaaaaa$ (via A : three aa steps) or as $S \Rightarrow B \Rightarrow^* aaaB \Rightarrow^* aaaaaa$ (via B : two aaa steps). So G is ambiguous.

(b) $L(G) = \{w \mid |w| \equiv 0 \pmod{2}\} \cup \{w \mid |w| \equiv 0 \pmod{3}\} = \{w \mid 2 \mid |w| \text{ or } 3 \mid |w|\}$. An unambiguous right-linear grammar is obtained from the DFA whose states record $|w| \pmod{6}$. Let the non-terminals be $S_0, S_1, S_2, S_3, S_4, S_5$, where S_i corresponds to remainder i . The start symbol is S_0 , and the accepting remainders are 0, 2, 3, 4. The grammar is

$$\begin{aligned} S_0 &\rightarrow \lambda \mid aS_1, \\ S_1 &\rightarrow aS_2, \\ S_2 &\rightarrow \lambda \mid aS_3, \\ S_3 &\rightarrow \lambda \mid aS_4, \\ S_4 &\rightarrow \lambda \mid aS_5, \\ S_5 &\rightarrow aS_0. \end{aligned}$$

This grammar is unambiguous because at each nonterminal there is at most one production beginning with a , so each word determines a unique path through the six remainders. A derivation ends with λ exactly in the states whose remainders are divisible by 2 or 3, so the grammar generates precisely $L(G)$.

9.9. Grammar for regular expressions, modified.

(a) Grammar that strips outermost parentheses:

$$S \rightarrow R \mid R \cdot R \mid R \cup R$$

where R is the original grammar from Example 9.3:

$$R \rightarrow a \mid b \mid c \mid \epsilon \mid \emptyset \mid (R \cdot R) \mid (R \cup R) \mid R^*.$$

The new start symbol S therefore generates all old words, and in addition it allows the outermost parentheses of a binary expression to be omitted.

(b) Grammar allowing extraneous parentheses:

$$A \rightarrow (A) \mid a \mid b \mid c \mid \epsilon \mid \emptyset \mid (A \cdot A) \mid (A \cup A) \mid A^*.$$

The rule $A \rightarrow (A)$ allows any expression to be wrapped in an extra pair of parentheses, and repeated use of that rule gives arbitrarily many extraneous pairs.

9.10. **Leftmost and rightmost derivations for $((a^* \cdot b) \cup (c \cdot d)^*)$.**

(a) **Leftmost derivation** (always expand leftmost nonterminal):

$$\begin{aligned}
 R &\Rightarrow (R \cup R) \\
 &\Rightarrow ((R \cdot R) \cup R) \\
 &\Rightarrow ((R^* \cdot R) \cup R) \\
 &\Rightarrow ((a^* \cdot R) \cup R) \\
 &\Rightarrow ((a^* \cdot b) \cup R) \\
 &\Rightarrow ((a^* \cdot b) \cup (R \cdot R)^*) \\
 &\Rightarrow ((a^* \cdot b) \cup (c \cdot R)^*) \\
 &\Rightarrow ((a^* \cdot b) \cup (c \cdot d)^*)
 \end{aligned}$$

(b) **Rightmost derivation** (always expand rightmost nonterminal):

$$\begin{aligned}
 R &\Rightarrow (R \cup R) \\
 &\Rightarrow (R \cup (R \cdot R)^*) \\
 &\Rightarrow (R \cup (R \cdot d)^*) \\
 &\Rightarrow (R \cup (c \cdot d)^*) \\
 &\Rightarrow ((R \cdot R) \cup (c \cdot d)^*) \\
 &\Rightarrow ((R \cdot b) \cup (c \cdot d)^*) \\
 &\Rightarrow ((R^* \cdot b) \cup (c \cdot d)^*) \\
 &\Rightarrow ((a^* \cdot b) \cup (c \cdot d)^*)
 \end{aligned}$$

9.11. **Prove $L(G) = L(G')$ (Theorem 9.5) by induction on derivation steps.**

For every production $A \rightarrow \alpha_1 \alpha_2 \cdots \alpha_n$ of G , the grammar G' contains a replacement set that begins with $A \rightarrow \alpha_1 Y_{k1}$ and ends with $Y_{k,n-2} \rightarrow \alpha_{n-1} \alpha_n$ (with terminal symbols replaced by the new symbols X_a when necessary). Those replacement rules together produce exactly the same strings as the original rule.

Claim: If $S \xRightarrow{*}_G^m x$ with $x \in \Sigma^*$, then $S \xRightarrow{*}_{G'} x$.

Proof: By induction on m .

Base ($m = 1$): Then $S \rightarrow x$ is a single production of G . If it is already in CNF form, the same production is in G' . Otherwise, the replacement set for that production derives x in G' .

Step: Assume the claim true for derivations of length m . Suppose

$$S \xRightarrow{*}_G \alpha \xRightarrow{*}_{G'}^m x.$$

Replace the first production $S \rightarrow \alpha$ by its corresponding replacement set in G' . This produces the same sentential form α , except possibly that some terminal symbols have been replaced by the new nonterminals X_a , each of which immediately derives the corresponding terminal. After these finitely many extra steps, the remainder of the original derivation can be copied by the induction hypothesis. Hence $S \xRightarrow{*}_{G'} x$.

Conversely, every derivation in G' can be compressed to one in G : each new symbol Y_{ki} occurs in only one replacement set, so any maximal block of steps using Y_{ki} symbols is simply the expanded form of one original production of G . Replacing each such block by the corresponding production of G yields a derivation of the same terminal string in G .

Therefore $L(G) = L(G')$. □

9.12. **For CNF grammar G and $x \in L(G)$, $x \neq \lambda$: number of productions in any derivation of x is $2|x|$.**

Proof: In the chapter's definition of CNF, a nonempty derivation begins with the start step $Z \rightarrow S$. After that initial step, every production is either of the form $A \rightarrow BC$ or $A \rightarrow a$.

If x has length n , then exactly n terminal productions are needed, one for each terminal symbol in x . Before those terminal productions can be applied, the derivation must create exactly n nonterminals ready to become those terminals. Starting from the single nonterminal S , each production $A \rightarrow BC$ increases the number of nonterminals by 1, so exactly $n - 1$ productions of the form $A \rightarrow BC$ are required.

Thus the total number of productions is

$$1 + (n - 1) + n = 2n = 2|x|,$$

where the initial 1 is the production $Z \rightarrow S$. □

9.13. **Bounds on parse tree depth for CNF grammars.**

- (a) **Lower bound:** Let $|x| = n > 0$. Ignoring the optional initial rule $Z \rightarrow S$, the parse tree below S is a binary tree with exactly n terminal leaves, because every nonterminal production in CNF has the form $A \rightarrow BC$ or $A \rightarrow d$. A binary tree of depth h has at most 2^h leaves, so $2^h \geq n$ and therefore

$$h \geq \lceil \log_2 n \rceil.$$

If one also counts the initial edge $Z \rightarrow S$, this lower bound increases by 1.

- (b) **Upper bound:** The deepest possible binary parse tree with n terminal leaves is the maximally skewed one, where each internal node has one terminal child and one child that continues the derivation. Such a tree has depth $n - 1$ below S . Including the optional initial rule $Z \rightarrow S$, the full CNF parse tree has depth at most n .

Summary: With the full CNF tree rooted at Z ,

$$\lceil \log_2 |x| \rceil + 1 \leq \text{depth} \leq |x|.$$

If the initial $Z \rightarrow S$ edge is ignored, the bounds become $\lceil \log_2 |x| \rceil \leq \text{depth} \leq |x| - 1$.

9.14. **Convert grammars to Chomsky Normal Form.**

- (a) $\langle \{S, B, C\}, \{a, b, c\}, S, \{S \rightarrow aB, S \rightarrow abC, B \rightarrow bc, C \rightarrow c\} \rangle$:

Introduce terminal nonterminals: $T_a \rightarrow a$, $T_b \rightarrow b$, $T_c \rightarrow c$. - $S \rightarrow aB$ becomes $S \rightarrow T_a B$ (already binary CNF). - $S \rightarrow abC$ becomes $S \rightarrow T_a D_1$ where $D_1 \rightarrow T_b C$. - $B \rightarrow bc$ becomes $B \rightarrow T_b T_c$. - $C \rightarrow c$ becomes $C \rightarrow T_c$ — but $C \rightarrow T_c$ is a unit production! Replace: wherever C appears as a target, substitute. Since $C \rightarrow c$ and C does not appear in the right side of B or S

(only in $S \rightarrow \mathbf{ab}C$), we can just keep $C \rightarrow T_c$ and note it will be eliminated: replace $S \rightarrow T_a D_1$ (where $D_1 \rightarrow T_b C$) by substituting C : $D_1 \rightarrow T_b T_c$.

Final CNF:

$$S \rightarrow T_a B \mid T_a D_1, \quad D_1 \rightarrow T_b T_c, \quad B \rightarrow T_b T_c, \quad T_a \rightarrow \mathbf{a}, \quad T_b \rightarrow \mathbf{b}, \quad T_c \rightarrow \mathbf{c}.$$

(b) $\langle \{S, A, B\}, \{\mathbf{a}, \mathbf{b}, \mathbf{c}\}, S, \{S \rightarrow \mathbf{c}BA, S \rightarrow B, A \rightarrow \mathbf{c}B, A \rightarrow \mathbf{Abb}S, B \rightarrow \mathbf{aaa}\} \rangle$:

Step 1: Remove unit production $S \rightarrow B$: add all B -rules to S : $S \rightarrow \mathbf{aaa}$. Step 2: Introduce terminal symbols: $T_a \rightarrow \mathbf{a}$, $T_b \rightarrow \mathbf{b}$, $T_c \rightarrow \mathbf{c}$. Step 3: Break long right-hand sides: - $S \rightarrow \mathbf{c}BA$: $S \rightarrow T_c E_1$, $E_1 \rightarrow BA$. - $S \rightarrow \mathbf{aaa}$: $S \rightarrow T_a E_2$, $E_2 \rightarrow T_a T_a$. - $A \rightarrow \mathbf{c}B$: $A \rightarrow T_c B$. - $A \rightarrow \mathbf{Abb}S$: $A \rightarrow A E_3$, $E_3 \rightarrow T_b E_4$, $E_4 \rightarrow T_b S$. - $B \rightarrow \mathbf{aaa}$: $B \rightarrow T_a E_2$ (reuse E_2).

Final CNF:

$$\begin{aligned} S &\rightarrow T_c E_1 \mid T_a E_2 \\ E_1 &\rightarrow BA, \quad E_2 \rightarrow T_a T_a \\ A &\rightarrow T_c B \mid A E_3 \\ E_3 &\rightarrow T_b E_4, \quad E_4 \rightarrow T_b S \\ B &\rightarrow T_a E_2 \\ T_a &\rightarrow \mathbf{a}, \quad T_b \rightarrow \mathbf{b}, \quad T_c \rightarrow \mathbf{c} \end{aligned}$$

(c) $\langle \{R\}, \{\mathbf{a}, \mathbf{b}, \mathbf{c}, (,), \epsilon, \emptyset, \cup, \cdot, *\}, R, \{R \rightarrow \mathbf{a|b|c|}\epsilon|\emptyset|(R \cdot R)|(R \cup R)|R^*\} \rangle$:

Introduce terminal nonterminals for each terminal symbol: $T_a, T_b, T_c, T_\epsilon, T_\emptyset, T_((, T_), T_\cup, T_\cdot, T_*$.
 $R \rightarrow \mathbf{a|b|c|}\epsilon|\emptyset$: Each of these is a single-terminal production $R \rightarrow a$ (with $|a| = 1$), which is already in CNF form.

$R \rightarrow (R \cdot R)$: Break as $R \rightarrow T_((D_1, D_1 \rightarrow RD_2, D_2 \rightarrow T_\cdot D_3, D_3 \rightarrow RT_)$. $R \rightarrow (R \cup R)$: Break as $R \rightarrow T_((D_4, D_4 \rightarrow RD_5, D_5 \rightarrow T_\cup D_6, D_6 \rightarrow RT_)$. $R \rightarrow R^*$: $R \rightarrow RD_7, D_7 \rightarrow T_*$ —but $D_7 \rightarrow T_*$ is a unit production; substitute: $R \rightarrow RT_*$.

Final CNF:

$$\begin{aligned} R &\rightarrow \mathbf{a|b|c|}\epsilon|\emptyset \mid T_((D_1 \mid T_((D_4 \mid RT_* \\ D_1 &\rightarrow RD_2, \quad D_2 \rightarrow T_\cdot D_3, \quad D_3 \rightarrow RT_ \\ D_4 &\rightarrow RD_5, \quad D_5 \rightarrow T_\cup D_6, \quad D_6 \rightarrow RT_ \\ T_((&\rightarrow (, \quad T_() \rightarrow), \quad T_\cdot \rightarrow \cdot, \quad T_\cup \rightarrow \cup, \quad T_* \rightarrow * \end{aligned}$$

(d) $\langle \{T\}, \{\mathbf{a}, \mathbf{b}, \mathbf{c}, \mathbf{d}, -, +\}, T, \{T \rightarrow \mathbf{a|b|c|d|T-T|T+T}\} \rangle$:

$T \rightarrow \mathbf{a|b|c|d}$: Already in CNF (terminal productions). $T \rightarrow T - T$: $T \rightarrow TD_-$ where $D_- \rightarrow T_- T$ and $T_- \rightarrow -$. $T \rightarrow T + T$: $T \rightarrow TD_+$ where $D_+ \rightarrow T_+ T$ and $T_+ \rightarrow +$.

Final CNF:

$$T \rightarrow \mathbf{a|b|c|d} \mid TD_- \mid TD_+, \quad D_- \rightarrow T_- T, \quad D_+ \rightarrow T_+ T, \quad T_- \rightarrow -, \quad T_+ \rightarrow +.$$

9.15. Convert grammars to Greibach Normal Form.

- (a) $\langle \{S_1, S_2\}, \{a, b, c, d, e\}, S_1, \{S_1 \rightarrow S_2 S_1 e, S_1 \rightarrow S_2 b, S_2 \rightarrow S_1 S_2, S_2 \rightarrow c\} \rangle$:

Applying the substitutions from Theorem 9.6 gives the following equivalent GNF grammar (with one new nonterminal S_3):

$$\begin{aligned} S_1 &\rightarrow cS_1e \mid cb \mid cS_1eS_3 \mid cbS_3, \\ S_2 &\rightarrow c \mid cS_1eS_2 \mid cbS_2 \mid cS_1eS_3S_2 \mid cbS_3S_2, \\ S_3 &\rightarrow cS_1e \mid cb \mid cS_1eS_3 \mid cbS_3 \\ &\quad \mid cS_1eS_2S_1e \mid cbS_2S_1e \mid cS_1eS_3S_2S_1e \mid cbS_3S_2S_1e \\ &\quad \mid cS_1eS_2b \mid cbS_2b \mid cS_1eS_3S_2b \mid cbS_3S_2b \\ &\quad \mid cS_1eS_2S_1eS_3 \mid cbS_2S_1eS_3 \mid cS_1eS_3S_2S_1eS_3 \mid cbS_3S_2S_1eS_3 \\ &\quad \mid cS_1eS_2bS_3 \mid cbS_2bS_3 \mid cS_1eS_3S_2bS_3 \mid cbS_3S_2bS_3. \end{aligned}$$

Every production now begins with the terminal c , so the grammar is in GNF.

- (b) $\langle \{S_1, S_2, S_3\}, \dots, \{S_1 \rightarrow S_3 S_1, S_1 \rightarrow S_2 a, S_2 \rightarrow be, S_3 \rightarrow S_2 c\} \rangle$:

$S_2 \rightarrow be$ (already in GNF). $S_3 \rightarrow S_2 c \rightarrow bec$ (substitute): $S_3 \rightarrow bec$. $S_1 \rightarrow S_3 S_1 \rightarrow becS_1$ and $S_1 \rightarrow S_2 a \rightarrow bea = bea$. GNF: $S_1 \rightarrow becS_1 \mid bea$, $S_2 \rightarrow be$, $S_3 \rightarrow bec$.

- (c) $\langle \{S_1, S_2, S_3\}, \dots, \{S_1 \rightarrow S_1 S_2 c, S_1 \rightarrow dS_3, S_2 \rightarrow S_1 S_1, S_2 \rightarrow a, S_3 \rightarrow S_3 e\} \rangle$:

As printed, this grammar generates no terminal strings at all. The nonterminal S_3 has only the production $S_3 \rightarrow S_3 e$, so S_3 is unproductive. Hence the production $S_1 \rightarrow dS_3$ cannot lead to a terminal word. The other production for S_1 is $S_1 \rightarrow S_1 S_2 c$, which still contains S_1 on the left, so it cannot terminate either. Therefore

$$L(G) = \emptyset.$$

An equivalent grammar in pure Greibach normal form is simply

$$\langle \{S_1\}, \{a, b, c, d, e\}, S_1, \emptyset \rangle.$$

9.16. Prove there exists a CFG G'' equivalent to G' (with added $A \rightarrow \lambda$ rules).

G' is obtained from G by adding rules $A \rightarrow \lambda$ for various nonterminals A . We want to show that there is an equivalent grammar G'' in which all of those lambda productions have been removed, except possibly the special start rule $Z \rightarrow \lambda$.

Proof: Apply the standard lambda-elimination construction to G' . First identify all *nullable* nonterminals, that is, all A for which $A \Rightarrow^* \lambda$ in G' . For each production in which nullable nonterminals occur on the right, add all productions obtained by deleting any chosen subset of those nullable occurrences, except that we do not create an empty right-hand side unless it is the special start production $Z \rightarrow \lambda$. After all such replacement productions have been added, delete the old productions $A \rightarrow \lambda$.

Every derivation in G' can now be simulated in the new grammar G'' : whenever G' uses a nullable nonterminal and later erases it by $A \rightarrow \lambda$, the grammar G'' simply chooses the corresponding shortened production from the start. Conversely, every shortened production of G'' represents a derivation in G' in which the omitted nullable nonterminals are first produced and then erased. Thus G'' generates exactly the same language as G' , except that the only remaining lambda production, if needed, is the allowable special rule $Z \rightarrow \lambda$.

Therefore an equivalent grammar G'' always exists. $\square$

9.17. **Generalization of Lemma 9.1.**

Proof: The generalized replacement is to replace

$$X \rightarrow \gamma B \alpha$$

by the family

$$X \rightarrow \gamma \beta_1 \alpha, \dots, X \rightarrow \gamma \beta_m \alpha.$$

Any derivation in G that uses $X \rightarrow \gamma B \alpha$ and then expands B by some rule $B \rightarrow \beta_i$ can be replaced by the single step $X \rightarrow \gamma \beta_i \alpha$. All other productions are unchanged, so every word of $L(G)$ still belongs to $L(G')$.

Conversely, each new production $X \rightarrow \gamma \beta_i \alpha$ in G' can be simulated in G by first using $X \rightarrow \gamma B \alpha$ and then $B \rightarrow \beta_i$. Hence the two grammars generate the same language. $\square$

9.18. $P = \{d^p \mid p \text{ prime}\}$ **is not context free.**

- (a) **Pumping theorem proof:** Assume P is context free with pumping constant n . Take $s = d^p$ where $p > n$ is prime. By the pumping lemma, $s = uvwxy$ with $|vwx| \leq n$, $|vx| \geq 1$, and $uv^iwx^iy \in P$ for all $i \geq 0$.
 Let $|vx| = m$ where $1 \leq m \leq n$. Then $uv^iwx^iy = d^{p+(i-1)m}$. For $i = 0$: d^{p-m} . For $i = 2$: d^{p+m} .
 Choose $i = p + 1$: the resulting string has length $p + pm = p(1 + m)$. Since $p \geq 2$ and $m \geq 1$, $p(1 + m)$ is composite (product of two integers each ≥ 2). So $d^{p(1+m)} \notin P$, contradiction. $\square$
- (b) **Using nonregularity:** The language P is a language over the single-letter alphabet $\{d\}$. If P were context free, then Theorem 9.15 would imply that P is regular, because every context-free language over a one-letter alphabet is regular. But P is known to be nonregular. This contradiction shows that P is not context free. $\square$

9.19. $\Gamma = \{w2w \mid w \in \{0,1\}^*\}$ **is not context free.**

Assume Γ is context free with pumping constant n . Choose $s = 0^n 1^n 2 0^n 1^n \in \Gamma$. Then $|s| = 4n + 1$.

By the pumping lemma: $s = uvwxy$, $|vwx| \leq n$, $|vx| \geq 1$, all pumped strings in Γ .

Since $|vwx| \leq n$, the substring vwx can meet at most one boundary of the five-block word

$$0^n | 1^n | 2 | 0^n | 1^n.$$

We consider the possibilities.

- If vwx does *not* contain the separator **2**, then it lies entirely in the left half $0^n 1^n$ or entirely in the right half $0^n 1^n$, possibly crossing the internal boundary between **0**'s and **1**'s. Pumping changes one side of the separator but leaves the other side unchanged. Therefore the resulting word cannot be of the form $w'2w'$.
- If vwx does contain the separator **2**, then there is only one occurrence of **2** in s . If **2** appears in vx , then pumping with $i = 0$ or $i = 2$ changes the number of **2**'s, so the pumped word is not in Γ .
- The only remaining possibility is that **2** lies in w but not in vx . Then pumping changes symbols on one side of **2** while leaving the other side and the separator itself unchanged. Again the two halves around the separator are no longer identical, so the pumped word is not in Γ .

In every case some pumped word lies outside Γ , contradicting the pumping lemma. Therefore Γ is not context free. $\square$

9.20. $\Psi = \{ww \mid w \in \{0,1\}^*\}$ is not context free.

Assume Ψ is context free with pumping constant n . Choose $s = 0^n 1^n 0^n 1^n \in \Psi$. $|s| = 4n$.

By the pumping lemma: $s = uvwxy$, $|vwx| \leq n$, $|vx| \geq 1$, all pumped strings in Ψ .

Since $|vwx| \leq n$, vwx is contained within a span of n consecutive characters. The string s has the pattern $0^n 1^n 0^n 1^n$. The middle two blocks meet at position $2n$; the window vwx cannot span from the first 0^n -block to the second 0^n -block.

In all cases, pumping (or de-pumping with $i = 0$) changes the length or composition of one half of s without matching change in the other half, so the result cannot have the form $w'w'$. Contradiction. $\square$

9.21. $\Xi = \{b^{2^j} \mid j \in \mathbb{N}\}$ is not context free.

Assume Ξ is context free with pumping constant n . Choose j such that $2^j > n$, and let $s = b^{2^j}$.

By the pumping lemma: $s = uvwxy$, $|vwx| \leq n$, $|vx| = m \geq 1$. Then $|uv^iwx^iy| = 2^j + (i-1)m$ for each $i \geq 0$.

For $i = 2$: length $= 2^j + m$. We need $2^j + m$ to be a power of 2. But $2^j < 2^j + m \leq 2^j + n < 2^{j+1}$ (since $m \leq n < 2^j$). So $2^j + m$ is strictly between 2^j and 2^{j+1} , which means it is not a power of 2. Contradiction. $\square$

9.22. $\Phi = \{a^{j^2} \mid j \in \mathbb{N}\}$ is not context free; application to Φ' .

(a) **Φ is not context free:** Assume Φ is context free with pumping constant n . Let $j > n$; take $s = a^{j^2}$.

By the pumping lemma: $s = uvwxy$, $|vwx| \leq n$, $|vx| = m \geq 1$. Then pumped strings have length $j^2 + (i-1)m$. For $i = 2$: length $j^2 + m$. We need $j^2 + m$ to be a perfect square. Choose $j > n/2$ so that $n < 2j$; then $j^2 < j^2 + m \leq j^2 + n < j^2 + 2j + 1 = (j+1)^2$. So $j^2 + m$ is strictly between two consecutive perfect squares, hence not a perfect square. Contradiction. $\square$

(b) **Φ' not context free via homomorphism:** Define $h : \{b, c, d\}^* \rightarrow \{a\}^*$ by $h(b) = a$ and $h(c) = h(d) = \lambda$. If Φ' were a CFL, then $h(\Phi') = \Phi$ would also be a CFL (CFLs are closed under homomorphism), contradicting part (a). Hence $\Phi' \notin \mathcal{C}_\Sigma$.

(c) **Direct Ogden's lemma proof:** Assume Φ' is context free, and let p be the constant from Ogden's lemma. Choose the word

$$s = bc^{p^2}d^p \in \Phi',$$

and mark all p^2 occurrences of c . Ogden's lemma gives a decomposition $s = uvwxy$ such that vx contains at least one marked position, and vwx contains at most p marked positions.

Let $\alpha = |vx|_c$ and $\beta = |vx|_d$. Because vx contains a marked position, $\alpha \geq 1$. Also $\alpha \leq p$ and $\beta \leq p$. Pump once upward, to uv^2wx^2y . Its number of c 's is $p^2 + \alpha$, while its number of d 's is $p + \beta$.

If $\beta = 0$, then the number of d 's stays equal to p while the number of c 's changes, so the relation $|x|_c = (|x|_d)^2$ fails.

If $\beta \geq 1$, then

$$(p + \beta)^2 - p^2 = 2p\beta + \beta^2 > p \geq \alpha,$$

so

$$p^2 + \alpha < (p + \beta)^2.$$

Again the relation $|x|_c = (|x|_d)^2$ fails. In either case, $uv^2wx^2y \notin \Phi'$, contradicting Ogden's lemma. Therefore Φ' is not context free.

- (d) **Pumping theorem:** Yes. The ordinary pumping theorem also works. Assume Φ' is context free, and let n be its pumping constant. Take

$$s = \mathbf{bc}^{n^2}\mathbf{d}^n \in \Phi'.$$

Write $s = uvwxy$ with $|vwx| \leq n$ and $|vx| \geq 1$.

If vx contains the initial $\mathbf{b}$, then pumping down ($i = 0$) removes that only $\mathbf{b}$, so $uv^0wx^0y \notin \Phi'$. Otherwise, vx lies entirely in the $\mathbf{c}^*\mathbf{d}^*$ portion of the word. Let $\alpha = |vx|_c$ and $\beta = |vx|_d$. Then $\alpha + \beta \geq 1$ and $\alpha + \beta \leq n$.

If $\alpha = 0$, pumping up changes the number of $\mathbf{d}$'s while leaving the number of $\mathbf{c}$'s fixed, so the quadratic relation fails.

If $\beta = 0$, pumping up changes the number of $\mathbf{c}$'s while leaving the number of $\mathbf{d}$'s fixed, so the quadratic relation fails.

If both α and β are positive, then pumping up gives $n^2 + \alpha$ occurrences of $\mathbf{c}$ and $n + \beta$ occurrences of $\mathbf{d}$. But

$$(n + \beta)^2 - n^2 = 2n\beta + \beta^2 > n \geq \alpha,$$

so $n^2 + \alpha \neq (n + \beta)^2$. Thus the pumped word is not in Φ' .

In every case some pumped word lies outside Φ' , contradicting the pumping theorem. So it is possible to prove directly from the pumping theorem that Φ' is not context free.

- 9.23. $L = \{y \in \{0, 1\}^* \mid |y|_0 = |y|_1\}$ is context free.

Grammar: $S \rightarrow 0S1 \mid 1S0 \mid SS \mid \lambda$.

Proof: By induction, any string with equal numbers of 0s and 1s is derivable, and all derivations yield strings with equal counts. So $L(G) = L$. $\square$

- 9.24. **Formal recursive definition of the path in Theorem 9.9 (pumping lemma).**

- (a) Let $d(v)$ denote the number of distinguished positions in the subtree rooted at a node v . Define the path recursively as follows.

- **Boundary:** If v is a leaf, stop.
- **Rule:** If v has two children v_ℓ and v_r , choose the child with the larger value of d . If $d(v_\ell) = d(v_r)$, choose either one.

Starting at the root and repeating this rule yields a path ending at a distinguished leaf.

- (b) Because the chosen child always contains at least half of the distinguished positions of its parent, a node that is k levels above the end of the path still has at least 2^k distinguished positions in its subtree. Now choose the *highest* node N on the path whose subtree contains no more than n distinguished positions. Such a node exists because the final leaf contains exactly one

distinguished position. Its parent (if any) contains more than n distinguished positions, so N must still contain more than $n/2 = 2^{|\Omega|}$ distinguished positions. Hence the portion of the path inside the subtree rooted at N still has more than $|\Omega|$ interior nodes, and some nonterminal label must repeat there. Let the higher and lower occurrences of that repeated label be R .

If the higher R derives νRx , and the lower R derives w , then

$$S \Rightarrow^* uRy \Rightarrow^* uvRxy \Rightarrow^* uvwxy.$$

The subtree rooted at the lower R contains a distinguished position, so w contains one. The higher R lies inside the subtree rooted at N , so νwx contains no more than n distinguished positions. And since the two occurrences of R are distinct, at least one distinguished position lies in the part of the higher subtree outside the lower one, so νx contains a distinguished position. Replacing the lower R -subtree by repeated copies of the higher one now gives the pumped strings uv^iwx^iy , exactly as required in Theorem 9.9. $\square$

9.25. $\mathcal{C}_\Sigma$ is closed under $\cup$ via direct grammar construction.

Let $G_1 = \langle \Omega_1, \Sigma, S_1, P_1 \rangle$ and $G_2 = \langle \Omega_2, \Sigma, S_2, P_2 \rangle$ with $\Omega_1 \cap \Omega_2 = \emptyset$. Choose $Z \notin \Omega_1 \cup \Omega_2$ and define:

$$G^\cup = \langle \Omega_1 \cup \Omega_2 \cup \{Z\}, \Sigma, Z, P_1 \cup P_2 \cup \{Z \rightarrow S_1, Z \rightarrow S_2\} \rangle.$$

Then $w \in L(G^\cup)$ iff $Z \Rightarrow^* w$ iff $Z \Rightarrow^* S_i \Rightarrow^* w$ for $i = 1$ or $i = 2$ iff $w \in L(G_1) \cup L(G_2)$. $\square$

9.26. Closure properties of $\mathfrak{X}_\Sigma$ (languages that are NOT context free).

(a) **Union:** Not closed. Let

$$A = \{a^{2^j} \mid j \in \mathbb{N}\}.$$

Exercise 9.21 shows that A is not context free. Let $B = a^* - A$. If B were context free, then by Theorem 9.15 it would be regular, and therefore $A = a^* - B$ would also be regular, contrary to Exercise 9.21. Hence $B \in \mathfrak{X}_\Sigma$ as well. But

$$A \cup B = a^*,$$

which is regular, hence context free. Therefore $\mathfrak{X}_\Sigma$ is not closed under union.

(b) **Complement:** Not closed. Lemma 9.4 gives a context-free language L whose complement is not context free. Let $X = \overline{L}$. Then $X \in \mathfrak{X}_\Sigma$, but $\overline{X} = L \notin \mathfrak{C}_\Sigma$. So $\mathfrak{X}_\Sigma$ is not closed under complement.

(c) **Intersection:** Not closed. Let

$$A = \{a^{2^j} \mid j \in \mathbb{N}\} \quad \text{and} \quad B = a^* - A.$$

As in part (a), both A and B belong to $\mathfrak{X}_\Sigma$, but

$$A \cap B = \emptyset,$$

which is context free. Hence $\mathfrak{X}_\Sigma$ is not closed under intersection.

(d) **Kleene closure:** Not closed. Let

$$A = \{a^{2^j} \mid j \in \mathbb{N}\}.$$

Then $A \in \mathfrak{X}_\Sigma$. Since $a = a^{2^0} \in A$, every unary word belongs to A^* , and therefore

$$A^* = a^*.$$

Thus A^* is regular, so $\mathfrak{X}_\Sigma$ is not closed under Kleene closure.

(e) **Concatenation:** Not closed. Let

$$A' = A \cup \{\lambda\}, \quad B' = B \cup \{\lambda\},$$

where A and B are as in part (a). If A' were context free, then by Theorem 9.15 it would be regular; removing the single word λ would show that A is regular, contradiction. So $A' \notin \mathfrak{X}_\Sigma$.

If B' were context free, then Theorem 9.15 would make B' regular. Removing the single word λ would then show that $B = B' - \{\lambda\}$ is regular, contradiction. Hence $B' \notin \mathfrak{X}_\Sigma$.

Now every word a^n belongs to $A'B'$: if $a^n \in A$, then $a^n = a^n \lambda$ with $a^n \in A'$ and $\lambda \in B'$; if $a^n \notin A$, then $a^n \in B$ and $a^n = \lambda a^n$ with $\lambda \in A'$ and $a^n \in B'$. Hence $A'B' = a^*$, which is context free. Therefore $\mathfrak{X}_\Sigma$ is not closed under concatenation.

9.27. $\mathfrak{C}_\Sigma$ is closed under reversal.

Let $G = \langle \Omega, \Sigma, S, P \rangle$ generate L . Define $G^r = \langle \Omega, \Sigma, S, P^r \rangle$ where $P^r = \{A \rightarrow \alpha^r \mid A \rightarrow \alpha \in P\}$ (reverse the right-hand side of each production).

Claim: $L(G^r) = L^r$.

Proof: We prove a stronger statement: for every nonterminal $A \in \Omega$ and every terminal string $x \in \Sigma^*$,

$$A \Rightarrow_G^* x \iff A \Rightarrow_{G^r}^* x^r.$$

We prove the forward implication by induction on the derivation length of $A \Rightarrow_G^* x$.

Base: If the derivation has one step, then $A \rightarrow x$ is a production of G . By definition, $A \rightarrow x^r$ is a production of G^r , so $A \Rightarrow_{G^r}^* x^r$.

Step: Suppose the first production is

$$A \rightarrow X_1 X_2 \cdots X_k$$

in G , where each X_i is either a terminal or a nonterminal, and suppose the rest of the derivation yields

$$X_1 X_2 \cdots X_k \Rightarrow_G^* x_1 x_2 \cdots x_k = x,$$

where each x_i is the terminal string ultimately produced from X_i (possibly a single terminal if $X_i \in \Sigma$). By the induction hypothesis, each nonterminal X_i derives x_i^r in G^r , while terminals already equal their own one-letter reversals. Since G^r contains the production

$$A \rightarrow X_k X_{k-1} \cdots X_1,$$

we obtain

$$A \Rightarrow_{G^r}^* X_k X_{k-1} \cdots X_1 \Rightarrow_{G^r}^* x_k^r x_{k-1}^r \cdots x_1^r = (x_1 x_2 \cdots x_k)^r = x^r.$$

Conversely, suppose $x^r \in L(G^r)$. Applying the forward implication to the reversed grammar G^r , and using $(G^r)^r = G$, gives $x \in L(G)$. Taking $A = S$, we conclude that

$$x \in L(G) \iff x^r \in L(G^r),$$

so $L(G^r) = L^r$. □

9.28. **Determine if $\mathcal{C}_\Sigma$ is closed under operator T ($T(L) = \{w^r w \mid w \in L\}$).**

Claim: T does not preserve $\mathcal{C}_\Sigma$ in general; i.e., there exists $L \in \mathcal{C}_\Sigma$ such that $T(L) \notin \mathcal{C}_\Sigma$.

Counterexample: Let

$$L = \{a^n b a^n \mid n \geq 0\}.$$

This language is context free; for example, it is generated by $S \rightarrow aSa \mid b$. Since every word of L is a palindrome, we have

$$T(L) = \{a^n b a^{2n} b a^n \mid n \geq 0\}.$$

We claim $T(L) \notin \mathcal{C}_\Sigma$. Suppose instead that $T(L)$ is context free, and let N be its pumping constant. Choose

$$s = a^N b a^{2N} b a^N \in T(L).$$

Write $s = uvwxy$ with $|vwx| \leq N$ and $|vx| \geq 1$.

Because $|vwx| \leq N$, the substring vwx can contain at most one of the two b 's. Thus it lies entirely within one block, or else it crosses just one boundary between adjacent blocks.

- If vwx lies entirely within the leftmost a^N block, then pumping changes only that block. The rightmost a -block remains of length N , so the pumped word cannot have the form $a^n b a^{2n} b a^n$.
- If vwx lies entirely within the middle a^{2N} block, then pumping changes only the middle block. The outer blocks still both have length N , so the middle block should still have length $2N$, but it does not.
- If vwx lies entirely within the rightmost a^N block, the pumped word changes only that rightmost block. The leftmost a -block remains of length N , so the pumped word cannot have the required form $a^n b a^{2n} b a^n$, in which the two outer a -blocks have the same length.
- If vwx contains one of the b 's, then pumping with $i = 0$ or $i = 2$ either changes the number of b 's or changes one adjacent a -block while leaving the other outer a -block unchanged. In either event, the pumped word is not of the form $a^n b a^{2n} b a^n$.

Every possible placement of vwx leads to a contradiction. Therefore $T(L)$ is not context free, and $\mathcal{C}_\Sigma$ is not closed under T . □

9.29. **$\mathcal{C}_{\{a,b\}}$ is not closed under relative complement.**

Disproof: Let $\Sigma = \{a, b\}$ and let

$$K = \{a^n b^n a^n \mid n \in \mathbb{N}\}.$$

By the pumping-lemma argument used in the next exercise, K is not context free. Its complement $\Sigma^* - K$ is context free: it is the union of the regular language $\Sigma^* - a^* b^* a^*$ with the strings $a^i b^j a^k$ for which $i \neq j$ or $j \neq k$, and those inequality cases are context free.

Now take

$$L_1 = \Sigma^*, \quad L_2 = \Sigma^* - K.$$

Both L_1 and L_2 are context free, but

$$L_1 - L_2 = K,$$

which is not context free. Hence $\mathcal{C}_{\{a,b\}}$ is not closed under relative complement.

The same conclusion also follows from the closure properties: if $\mathcal{C}_{\{a,b\}}$ were closed under relative complement and we know it is closed under union, then it would be closed under complement (since $\overline{L_1} = \Sigma^* - L_1$, and Σ^* is CFL). But part (a) of the next exercise shows that $\mathcal{C}_{\{a,b\}}$ is not closed under complement. Hence $\mathcal{C}_{\{a,b\}}$ is *not* closed under relative complement. $\square$

9.30. $\mathcal{C}_\Sigma$ is not closed under intersection or complement.

(a) Let

$$L_1 = \{a^i b^j a^j \mid i, j \in \mathbb{N}\}, \quad L_2 = \{a^i b^i a^j \mid i, j \in \mathbb{N}\}.$$

Both are context free. For example, L_1 is generated by $S \rightarrow AB$, $A \rightarrow aA \mid \lambda$, $B \rightarrow bBa \mid \lambda$, and L_2 is generated by $S \rightarrow CD$, $C \rightarrow aCb \mid \lambda$, $D \rightarrow aD \mid \lambda$. Their intersection is

$$L_1 \cap L_2 = \{a^n b^n a^n \mid n \in \mathbb{N}\},$$

which is not context free (by the pumping lemma). So $\mathcal{C}_{\{a,b\}}$ is not closed under intersection.

If $\mathcal{C}_{\{a,b\}}$ were closed under complement, then $\overline{L_1}$ and $\overline{L_2}$ would be context free, and $\overline{\overline{L_1} \cup \overline{L_2}} = L_1 \cap L_2$ would be context free (by De Morgan and closure under union and complement). But $L_1 \cap L_2$ is not CFL. Contradiction. So $\mathcal{C}_{\{a,b\}}$ is not closed under complement.

(b) For $|\Sigma| > 1$, choose two distinct symbols $\alpha, \beta \in \Sigma$. Let $h : \{a, b\}^* \rightarrow \Sigma^*$ be the homomorphism with $h(a) = \alpha$ and $h(b) = \beta$. Applying h to the two languages from part (a) gives context-free languages over Σ :

$$h(L_1) = \{\alpha^i \beta^j \alpha^j \mid i, j \in \mathbb{N}\}, \quad h(L_2) = \{\alpha^i \beta^i \alpha^j \mid i, j \in \mathbb{N}\}.$$

But their intersection is

$$h(L_1) \cap h(L_2) = \{\alpha^n \beta^n \alpha^n \mid n \in \mathbb{N}\},$$

which is not context free by the same pumping-lemma argument. Therefore $\mathcal{C}_\Sigma$ is not closed under intersection whenever $|\Sigma| > 1$.

If $\mathcal{C}_\Sigma$ were closed under complement, then by De Morgan's law it would also be closed under intersection, since it is already closed under union. This contradiction shows that $\mathcal{C}_\Sigma$ is not closed under complement either. $\square$

9.31. Automaton-based method for Theorem 9.3.

In Theorem 9.3, each nonterminal B gives an NDFA A^B , and all of these machines share the same transition relation; only the start state changes. Instead of examining them one at a time, we can combine them into one automaton whose states are subsets of $\Omega \cup \{\omega\}$.

Define the DFA

$$D = \langle \{1\}, \wp(\Omega \cup \{\omega\}), \{S\}, \Delta, F \rangle,$$

where

$$\Delta(T, \mathbf{1}) = \bigcup_{A \in T} \delta(A, \mathbf{1}),$$

and F is the collection of subset states that contain ω .

The meaning of a subset state T is: after some number of grammar-expansion steps, the nonterminals in T are exactly the ones that can occur in the corresponding automaton search. Then:

- Reachability from the start state $\{S\}$ identifies the nonterminals that are accessible from S .
- A nonterminal B can derive a terminal string exactly when some state in D reachable from $\{B\}$ lies in F , that is, contains ω .
- After removing nonterminals that fail one of those tests, the same reachability check detects the remaining disconnected items.

So the subset-labeled automaton provides the same information as the family of machines A^B , but packages the computation into one finite automaton over $\wp(\Omega \cup \{\omega\})$.

9.32. Grammars in GNF with at most two symbols on the right.

- (a) Such grammars are *not* a normal form for all context-free languages. If a grammar is in GNF and no right side has more than two symbols, then every production has the form

$$A \rightarrow \mathbf{a} \quad \text{or} \quad A \rightarrow \mathbf{a}B.$$

That is exactly the form of a right-linear grammar. Therefore every language generated by such a grammar is regular. But $\{\mathbf{a}^n \mathbf{b}^n \mid n \in \mathbb{N}\}$ is context free and not regular, so it cannot be generated by any grammar of this restricted type.

- (b) By the same observation, the grammars that do satisfy this restriction are precisely the right-linear grammars. Hence they generate exactly the regular languages. $\square$

9.33. Determine if L^* must be regular for *any* collection L over Σ .

The statement is *false* as written.

Take

$$L = \{\mathbf{a}^n \mathbf{b}^n \mid n \geq 1\}.$$

Then

$$L^* = \{x_1 x_2 \cdots x_k \mid k \geq 0, x_i \in L\}.$$

Assume for contradiction that L^* is regular. The language $\mathbf{a}^* \mathbf{b}^*$ is regular, so

$$L^* \cap \mathbf{a}^* \mathbf{b}^*$$

would also be regular.

But

$$L^* \cap \mathbf{a}^* \mathbf{b}^* = L.$$

Indeed, any concatenation of two or more nonempty words from L has at least two alternations between $\mathbf{a}$'s and $\mathbf{b}$'s, so it cannot lie in $\mathbf{a}^* \mathbf{b}^*$. Thus the only words of L^* that belong to $\mathbf{a}^* \mathbf{b}^*$ are the single-block words $\mathbf{a}^n \mathbf{b}^n$.

Therefore L would be regular, contrary to the standard result that $\{\mathbf{a}^n \mathbf{b}^n \mid n \geq 1\}$ is not regular. So L^* need not be regular.

9.34. If $|\Sigma| = 1$, is $\mathcal{C}_\Sigma$ closed under complementation?

Yes. If $|\Sigma| = 1$, say $\Sigma = \{a\}$, and $L \in \mathcal{C}_\Sigma$, then Theorem 9.15 implies that L is regular. The regular languages are closed under complementation, so $\bar{L}$ is also regular. Every regular language is context free, hence $\bar{L} \in \mathcal{C}_\Sigma$.

Therefore $\mathcal{C}_\Sigma$ is closed under complementation when $|\Sigma| = 1$. □

9.35. $\{a^n b^n c^m \mid n, m \in \mathbb{N}\}$ is context free.

Grammar: $S \rightarrow AB$, $A \rightarrow aAb \mid \lambda$, $B \rightarrow cB \mid \lambda$.

A generates $\{a^n b^n \mid n \in \mathbb{N}\}$ and B generates c^* . So $L(G) = \{a^n b^n c^m \mid n, m \in \mathbb{N}\}$. □

9.36. Use Ogden's lemma to prove $\{a^k b^n c^m \mid k \neq n \wedge n \neq m\}$ is not context free.

Assume $L = \{a^k b^n c^m \mid k \neq n, n \neq m\}$ is context free with Ogden constant N .

Choose $p > N$ and let $s = a^{p!+p} b^p c^{p!+p} \in L$ (since $p!+p \neq p$ and $p \neq p!+p$). Mark all p occurrences of b in s .

By Ogden's lemma: $s = uvwxy$ where vw contains at most $N \leq p$ marked positions, vx contains at least 1 marked position, and all pumped strings $uv^i wx^i y \in L$.

Let $m = |vx|_b \geq 1$ (the number of b s in vx ; at least 1 since vx contains a marked symbol). Choose $i - 1 = p!/m$, which is a positive integer since $m \leq N < p$ implies $m \leq p - 1$ and hence $m \mid p!$. The pumped string has b -count equal to $p + (i - 1)m = p + p! = p! + p$.

Since $|vw| \leq N < p$ and the b -block has exactly p symbols, the window vw cannot reach both the a -block on the left and the c -block on the right simultaneously. Let $s_a = |vx|_a$ and $s_c = |vx|_c$. Then either $s_a = 0$ (window does not touch the a -block) or $s_c = 0$ (window does not touch the c -block).

- If $s_a = 0$: the a -count remains $p! + p = b$ -count, so $k = n = p! + p$, violating $k \neq n$. Contradiction.
- If $s_c = 0$: the c -count remains $p! + p = b$ -count, so $n = m = p! + p$, violating $n \neq m$. Contradiction.

In every case the pumped string lies outside L , contradicting Ogden's lemma. □

Chapter 10

Pushdown Automata — Solutions

10.1. **Show by induction:** $\bar{\delta}(s_0, x) = t \Leftrightarrow \langle s_0, x, \mathbb{C} \rangle \vdash^* \langle t, \lambda, \mathbb{C} \rangle$.

Claim ($P(n)$): For every state s, t and every $x \in \Sigma^n$, $\bar{\delta}(s, x) = t$ iff $\langle s, x, \mathbb{C} \rangle \vdash^* \langle t, \lambda, \mathbb{C} \rangle$.

Base ($n = 0$): $\bar{\delta}(s, \lambda) = s$ and $\langle s, \lambda, \mathbb{C} \rangle \vdash^* \langle s, \lambda, \mathbb{C} \rangle$ by zero moves. Both sides therefore give $t = s$. ✓

Step: Assume $P(n)$. Let $x = \mathbf{a}y$, where $\mathbf{a} \in \Sigma$ is the first letter and $|y| = n$, matching the recursive form of Definition 1.11iii.

($\Rightarrow$): If $\bar{\delta}(s, \mathbf{a}y) = t$, then by Definition 1.11iii,

$$\bar{\delta}(\delta(s, \mathbf{a}), y) = t.$$

Let $q = \delta(s, \mathbf{a})$. The PDA corresponding to the DFA has the one-step move

$$\langle s, \mathbf{a}y, \mathbb{C} \rangle \vdash \langle q, y, \mathbb{C} \rangle.$$

By the induction hypothesis applied from state q ,

$$\langle q, y, \mathbb{C} \rangle \vdash^* \langle t, \lambda, \mathbb{C} \rangle.$$

Chaining the moves gives

$$\langle s, \mathbf{a}y, \mathbb{C} \rangle \vdash^* \langle t, \lambda, \mathbb{C} \rangle.$$

($\Leftarrow$): If

$$\langle s, \mathbf{a}y, \mathbb{C} \rangle \vdash^* \langle t, \lambda, \mathbb{C} \rangle,$$

then the first move must read the initial input letter $\mathbf{a}$ and move to $q = \delta(s, \mathbf{a})$:

$$\langle s, \mathbf{a}y, \mathbb{C} \rangle \vdash \langle q, y, \mathbb{C} \rangle \vdash^* \langle t, \lambda, \mathbb{C} \rangle.$$

By the induction hypothesis, $\bar{\delta}(q, y) = t$. Since $q = \delta(s, \mathbf{a})$, Definition 1.11iii gives

$$\bar{\delta}(s, \mathbf{a}y) = \bar{\delta}(\delta(s, \mathbf{a}), y) = \bar{\delta}(q, y) = t.$$

□

10.2. **Define a one-state DPDA P'_1 with $\Lambda(P'_1) = \{a^n b^n \mid n \geq 1\}$.**

Because the acceptance condition is $\Lambda(P'_1)$, the machine must accept via *empty stack*, not via final state. A one-state DPDA with the required language is

$$P'_1 = \langle \{a, b\}, \{Z, S, A\}, \{t\}, t, \delta, Z, \emptyset \rangle,$$

where the only nonempty transitions are

$$\delta(t, a, Z) = \langle t, S \rangle,$$

$$\delta(t, a, S) = \langle t, SA \rangle,$$

$$\delta(t, b, S) = \langle t, \lambda \rangle,$$

$$\delta(t, b, A) = \langle t, \lambda \rangle.$$

After reading a^n ($n \geq 1$), the stack is SA^{n-1} : the symbol S marks the transition from the a -phase to the b -phase, and the A 's remember the remaining $n - 1$ copies. The first b removes S , and the remaining b 's remove the A 's one by one. Thus

$$\langle t, a^n b^n, Z \rangle \vdash^* \langle t, \lambda, \lambda \rangle \quad (n \geq 1).$$

No other words can empty the stack: λ leaves the initial symbol Z in place, a word beginning with b has no move, and once the machine has started reading b 's it can no longer read an a . Hence $\Lambda(P'_1) = \{a^n b^n \mid n \geq 1\}$.

10.3. **PDA P'_2 : proofs and language.**

$P'_2 = \langle \{a, b\}, \{S, C\}, \{t\}, t, \delta, S, \{t\} \rangle$ with $\delta(t, a, S) = \{\langle t, SC \rangle, \langle t, C \rangle\}$, $\delta(t, b, C) = \{\langle t, \lambda \rangle\}$, others empty.

(a) **Inductive proof:** ($\forall i \in \mathbb{N}$): if $\langle t, a^i, S \rangle \vdash^* \langle t, \lambda, \alpha \rangle$ then $\alpha = SC^i$ or $\alpha = C^i$.

Base ($i = 0$): $\langle t, \lambda, S \rangle \vdash^* \langle t, \lambda, S \rangle$; here $\alpha = S = SC^0$. ✓

Step: Assume the result for i . Reading a^{i+1} from S : first move on (a, S) either pushes SC or C .

Case 1: Push SC . Then $\langle t, a^{i+1}, S \rangle \vdash^* \langle t, a^i, SC \rangle$. Now process a^i from S : by IH, $\langle t, a^i, S \rangle \vdash^* \langle t, \lambda, \beta \rangle$ where $\beta \in \{SC^i, C^i\}$. So $\langle t, a^i, SC \rangle \vdash^* \langle t, \lambda, \beta C \rangle \in \{SC^{i+1}, C^{i+1}\}$.

Case 2: Push C . Then $\langle t, a^{i+1}, S \rangle \vdash^* \langle t, a^i, C \rangle$. Processing a^i from C : there are no moves (neither $\delta(t, a, C)$ nor $\delta(t, \lambda, C)$ are defined for reading a), so this branch is a dead end unless $i = 0$.

So $\alpha \in \{SC^{i+1}, C^{i+1}\}$. □

(b) **Inductive proof:** ($\forall i \in \mathbb{N}$): if $\langle t, x, C^i \rangle \vdash^* \langle t, \lambda, \beta \rangle$ then $x = b^i$.

Base ($i = 0$): $\langle t, x, \lambda \rangle$ can only have $x = \lambda$ (stack empty, no moves possible). So $x = \lambda = b^0$. ✓

Step: Assume for i . From $\langle t, x, C^{i+1} \rangle$: the top of the stack is C . The only move consuming C reads b : $\delta(t, b, C) = \langle t, \lambda \rangle$. So x must start with b : $x = bx'$ and $\langle t, bx', C^{i+1} \rangle \vdash^* \langle t, x', C^i \rangle$. By IH, $x' = b^i$. Hence $x = b^{i+1}$. □

(c) P'_2 accepts by empty stack (the final-state component $\{t\}$ is not used for acceptance here; a string is accepted iff the machine empties its stack while in state t with empty input). From (a), processing a^n from start symbol S can yield stack C^n (by choosing $\delta(t, a, S) = \langle t, SC \rangle$ at

each of the first $n - 1$ steps and then $\langle t, C \rangle$ at the last, so the final stack is C^n . From (b), the stack C^n is emptied by reading exactly b^n . Thus:

$$L(P'_2) = \{a^n b^n \mid n \geq 1\}.$$

For $n = 0$: the initial configuration $\langle t, \lambda, S \rangle$ has S still on the stack with no moves available, so the stack is never emptied and $\lambda \notin L(P'_2)$. Hence $L(P'_2) = \{a^n b^n \mid n \geq 1\}$.

10.4. $L = \{a^i b^j c^k \mid i, j, k \in \mathbb{N}, i + j = k\}$.

- (a) **PDA via final state:** Use a single counting symbol A : push one A for each a or b , then pop one A for each c . A machine is

$$P = \langle \{a, b, c\}, \{Z, A\}, \{q_0, q_1, q_2, q_f\}, q_0, \delta, Z, \{q_0, q_f\} \rangle,$$

where the nonempty transitions are

$$\begin{aligned} \delta(q_0, a, Z) &= \langle q_0, AZ \rangle, & \delta(q_0, a, A) &= \langle q_0, AA \rangle, \\ \delta(q_0, b, Z) &= \langle q_1, AZ \rangle, & \delta(q_0, b, A) &= \langle q_1, AA \rangle, \\ \delta(q_0, c, A) &= \langle q_2, \lambda \rangle, & \delta(q_1, b, A) &= \langle q_1, AA \rangle, \\ \delta(q_1, c, A) &= \langle q_2, \lambda \rangle, & \delta(q_2, c, A) &= \langle q_2, \lambda \rangle, \\ \delta(q_2, \lambda, Z) &= \langle q_f, Z \rangle. \end{aligned}$$

State q_0 handles the a -block, q_1 the b -block, and q_2 the c -block. The string λ is accepted because q_0 is final; otherwise, acceptance occurs exactly when all pushed A 's have been matched by c 's.

- (b) **PDA via empty stack:** Use the same three reading states and replace the last move by $\delta(q_2, \lambda, Z) = \langle q_2, \lambda \rangle$. To include λ , also allow $\delta(q_0, \lambda, Z) = \langle q_0, \lambda \rangle$.
- (c) **Counting automaton:** Yes. In fact the PDA in part (a) already uses only one genuine counting symbol A , so it is a counting automaton.
- (d) **DPDA:** Yes. At each stage there is only one possible action: push during the a - and b -blocks, then pop during the c -block. Thus the machine above can be made deterministic.
- (e) **Grammar from PDA** (using Definition 10.7): Applying Definition 10.7 to part (a) and deleting useless variables yields the simpler equivalent grammar

$$S \rightarrow aSc \mid T, \quad T \rightarrow bTc \mid \lambda.$$

Here S accounts for one outer a together with one final c , and T accounts for the remaining middle b/c matching. Thus a derivation first chooses i uses of $S \rightarrow aSc$ and then j uses of $T \rightarrow bTc$, producing exactly $a^i b^j c^{i+j}$.

10.5. $L = \{x \in \{a, b, c\}^* \mid |x|_a + |x|_b = |x|_c\}$.

- (a) **PDA via final state:** Use two stack symbols A (surplus of a/b) and B (surplus of c) to handle symbols in any order.

P : single reading state q , final state f , bottom marker Z :

$$\begin{array}{lll} \delta(q, \mathbf{a}, Z) = \langle q, AZ \rangle, & \delta(q, \mathbf{a}, A) = \langle q, AA \rangle, & \delta(q, \mathbf{a}, B) = \langle q, \lambda \rangle, \\ \delta(q, \mathbf{b}, Z) = \langle q, AZ \rangle, & \delta(q, \mathbf{b}, A) = \langle q, AA \rangle, & \delta(q, \mathbf{b}, B) = \langle q, \lambda \rangle, \\ \delta(q, \mathbf{c}, Z) = \langle q, BZ \rangle, & \delta(q, \mathbf{c}, B) = \langle q, BB \rangle, & \delta(q, \mathbf{c}, A) = \langle q, \lambda \rangle. \end{array}$$

When $\mathbf{a}$ or $\mathbf{b}$ is read with B on top, the pending $\mathbf{c}$ -surplus is cancelled, and vice versa. Accept by final state: $\delta(q, \lambda, Z) = \langle f, Z \rangle$ (fires when input exhausted and stack balanced).

- (b) **PDA via empty stack:** Pop Z as well. Add $\delta(q, \lambda, Z) = \langle q, \lambda \rangle$ (the stack becomes empty when balanced).
- (c) **Counting automaton:** Yes. One counter is enough to store the current difference between $|x|_{\mathbf{a}} + |x|_{\mathbf{b}}$ and $|x|_{\mathbf{c}}$, while the finite control records which side currently has the surplus.
- (d) **DPDA:** Yes. The two-symbol construction in part (a) is deterministic: for each (q, σ, X) triple there is at most one transition, since every input symbol and every stack top symbol determine a unique action. The only potential conflict is the λ -transition $\delta(q, \lambda, Z) = \langle f, Z \rangle$, which must not coexist with other q -transitions on Z . This is resolved by moving the λ -acceptance to a separate final state reached only when input is exhausted; in practice, the construction is deterministic for all reachable configurations. Hence a DPDA for L exists.
- (e) **Grammar from PDA:** Applying Definition 10.7 to part (a) and simplifying gives an equivalent grammar such as

$$S \rightarrow \mathbf{aSc} \mid \mathbf{cSa} \mid \mathbf{bSc} \mid \mathbf{cSb} \mid SS \mid \lambda.$$

Each production preserves the invariant $|x|_{\mathbf{a}} + |x|_{\mathbf{b}} = |x|_{\mathbf{c}}$, and every nonempty balanced word can be split at some point into one balanced word or into a balanced word bracketed by one of the four possible outer pairs.

10.6. Closure of $\mathfrak{P}_{\Sigma}$ and $\mathcal{A}_{\Sigma}$ under inverse homomorphism.

- (a) **$\mathfrak{P}_{\Sigma}$ is closed under inverse homomorphism:** Given a context-free language L over Σ and a homomorphism $h: \Delta^* \rightarrow \Sigma^*$, we want $h^{-1}(L) = \{w \in \Delta^* \mid h(w) \in L\}$ to be a CFL.

Proof: Construct a PDA for $h^{-1}(L)$ from the PDA P for L . The new PDA P' uses a buffer: for each input symbol $a \in \Delta$, it replaces a with $h(a)$ in a buffer and then simulates P on $h(a)$ before reading the next input symbol. The new PDA processes $h(a)$ letter-by-letter from the buffer. Since h maps to a finite string, this is a finite process per input symbol. Thus P' accepts exactly $h^{-1}(L)$, which is a CFL.

- (b) **$\mathcal{A}_{\Sigma}$ (DCFL) is closed under inverse homomorphism:** Given a DCFL L and a homomorphism h , $h^{-1}(L)$ is also a DCFL.

Proof: Let P be a DPDA for L . Build a new DPDA P' whose finite control stores, for each input symbol $a \in \Delta$, which letter of the fixed word $h(a)$ is currently being simulated. When P' reads a , it feeds the symbols of $h(a)$ to the transition function of P one by one before reading the next input symbol. Because each $h(a)$ is fixed and because P is deterministic, there is never a choice of how to process this buffered block. Thus P' is deterministic and accepts exactly those words x for which $h(x) \in L$, namely the words of $h^{-1}(L)$. $\square$

10.7. A finite language not recognized by any one-state PDA via final state.

Example: $L = \{ab\}$.

Proof: Every one-state PDA P with state set $\{t\}$, start state t , and final states $\{t\}$ trivially accepts λ : the initial configuration has P already in the (unique) final state with empty input, so λ is accepted regardless of the stack content. Since $\lambda \notin L = \{ab\}$, no one-state PDA via final state can recognize L .

More generally, any finite language that does not contain λ is not recognizable by any one-state PDA via final state, since such a machine always accepts λ . $\square$

10.8. $L = \{a^n b^n c^m d^m \mid n, m \in \mathbb{N}\}$.

- (a) **PDA via final state:** Push A on a ; pop A on b ; push B on c ; pop B on d ; accept when balanced. States: $\{q_0, q_1, q_2, q_f\}$. Initial state q_0 , final state q_f . Stack alphabet $\{Z, A, B\}$. $\delta(q_0, a, Z) = \langle q_0, AZ \rangle$, $\delta(q_0, a, A) = \langle q_0, AA \rangle$, $\delta(q_0, b, A) = \langle q_1, \lambda \rangle$, $\delta(q_1, b, A) = \langle q_1, \lambda \rangle$. After reading b^n , the machine is in state q_1 with Z on top of the stack. Then: $\delta(q_1, c, Z) = \langle q_2, BZ \rangle$, $\delta(q_2, c, B) = \langle q_2, BB \rangle$, $\delta(q_2, d, B) = \langle q_2, \lambda \rangle$, $\delta(q_2, d, Z) = \langle q_f, Z \rangle$. Also $\delta(q_0, \lambda, Z) = \langle q_1, Z \rangle$ to handle $n = 0$, and $\delta(q_1, \lambda, Z) = \langle q_2, Z \rangle$ for $m = 0$ (with appropriate final states).
- (b) **PDA via empty stack:** Use the same reading transitions as in part (a), but accept by emptying the bottom marker instead of entering q_f . Thus the final-state set is empty, and the bottom-marker moves are

$$\begin{aligned}\delta(q_0, \lambda, Z) &= \{\langle q_1, Z \rangle, \langle q_0, \lambda \rangle\}, \\ \delta(q_1, \lambda, Z) &= \{\langle q_2, Z \rangle, \langle q_1, \lambda \rangle\}, \quad \delta(q_2, \lambda, Z) = \{\langle q_2, \lambda \rangle\}.\end{aligned}$$

The first move from q_0 handles $n = 0$, the first move from q_1 handles the transition to the c/d phase when the a/b count is balanced, and the pop moves accept exactly when the input has been consumed and the relevant count is balanced.

- (c) **DPDA:** Yes. The DPDA deterministically pushes A s, then pops A s, then pushes B s, then pops B s. Each step is deterministic.
- (d) **Counting automaton:** Yes. A single stack symbol suffices: use it first to count the a 's against the b 's and, after the stack returns to Z , reuse the same symbol to count the c 's against the d 's.
- (e) **Grammar from PDA** (Definition 10.7): After applying Definition 10.7 to part (b) and deleting useless variables, we get the equivalent grammar

$$S \rightarrow AB, \quad A \rightarrow aAb \mid \lambda, \quad B \rightarrow cBd \mid \lambda.$$

Here A generates the first balanced block $a^n b^n$, B generates the second balanced block $c^m d^m$, and S concatenates those two independent pieces.

10.9. **Prove by induction (Theorem 10.2):** $\langle s, x, S \rangle \vdash^* \langle s, \lambda, \beta \rangle \Leftrightarrow S \Rightarrow^* x\beta$ as a leftmost derivation.

Claim ($P(n)$): For all $n \geq 0$ and strings x, β : $\langle s, x, S \rangle \vdash^* \langle s, \lambda, \beta \rangle$ in n moves iff $S \Rightarrow^* x\beta$ in n leftmost-derivation steps.

Base ($n = 0$): $\langle s, \lambda, S \rangle \vdash^* \langle s, \lambda, S \rangle$ (zero moves) iff $x = \lambda, \beta = S$, and $S \Rightarrow^* \lambda \cdot S$ (zero steps). $\checkmark$

Step: Assume $P(n)$ for all shorter computations. A computation of $n+1$ steps: $\langle s, ay, S \rangle \vdash \langle s, y, \gamma \rangle \vdash^* \langle s, \lambda, \beta \rangle$ (first move reads a , pushes γ for top of S). By the construction of the PDA (Theorem 10.2), this first step corresponds to applying a grammar production $S \rightarrow a\gamma$ (GNF production, starting with terminal a). By $P(n)$, $\langle s, y, \gamma \rangle \vdash^* \langle s, \lambda, \beta \rangle$ iff $\gamma \Rightarrow^* y\beta$ as leftmost derivation. So the full computation corresponds to $S \Rightarrow a\gamma \Rightarrow^* ay\beta = x\beta$. $\square$

10.10. Grammar for regular expressions: convert to GNF, find corresponding PDA.

Grammar: $R \rightarrow a|b|c|\epsilon|\emptyset|(R \cdot R)|(R \cup R)|R^*$.

- (a) **GNF conversion:** Since pure Greibach normal form only requires the first symbol on the right to be terminal, the only problem is the left-recursive rule $R \rightarrow R^*$. Introduce a new nonterminal Y generating one or more copies of $*$:

$$Y \rightarrow^* |^* Y.$$

Then an equivalent PGNF grammar is formed by combining the above rule with

$$\begin{aligned} R \rightarrow a|b|c|\epsilon|\emptyset|(R \cdot R)|(R \cup R) \\ |aY|bY|cY|\epsilon Y|\emptyset Y|(R \cdot R)Y|(R \cup R)Y. \end{aligned}$$

This simply replaces the old rule $R \rightarrow R^*$ by attaching a nonempty string of stars through Y .

- (b) **PDA from GNF grammar** (Definition 10.6): Let

$$P = \langle \Sigma, \Gamma, \{s\}, s, \delta, R, \emptyset \rangle,$$

where

$$\Sigma = \{a, b, c, (,), \epsilon, \emptyset, \cup, \cdot, *\}$$

and $\Gamma = \Sigma \cup \{R, Y\}$. The nonempty transitions are exactly those dictated by Definition 10.6:

$$\begin{aligned} \delta(s, a, R) &= \{\langle s, \lambda \rangle, \langle s, Y \rangle\}, \\ \delta(s, b, R) &= \{\langle s, \lambda \rangle, \langle s, Y \rangle\}, \\ \delta(s, c, R) &= \{\langle s, \lambda \rangle, \langle s, Y \rangle\}, \\ \delta(s, \epsilon, R) &= \{\langle s, \lambda \rangle, \langle s, Y \rangle\}, \\ \delta(s, \emptyset, R) &= \{\langle s, \lambda \rangle, \langle s, Y \rangle\}, \\ \delta(s, (, R) &= \{\langle s, R \cdot R \rangle, \langle s, R \cup R \rangle, \langle s, R \cdot R \rangle Y, \langle s, R \cup R \rangle Y\}, \\ \delta(s, *, Y) &= \{\langle s, \lambda \rangle, \langle s, Y \rangle\}, \end{aligned}$$

and for each terminal $\tau \in \Sigma$,

$$\delta(s, \tau, \tau) = \{\langle s, \lambda \rangle\}.$$

This PDA accepts via empty stack.

- (c) **PDA accepting via final state** (Theorem 10.4): Apply Theorem 10.4 to the machine in part (b). Using fresh bottom markers U and V , we obtain

$$P_f = \langle \Sigma, \Gamma \cup \{U, V\}, \{s, f\}, s, \delta_f, V, \{f\} \rangle,$$

where δ_f agrees with δ on all old stack symbols, and the additional nonempty transitions are

$$\delta_f(s, \lambda, V) = \{\langle s, RU \rangle\}, \quad \delta_f(s, \lambda, U) = \{\langle f, U \rangle\}.$$

There are no other moves involving U or V , and no moves out of f . By Theorem 10.4, $L(P_f) = \Lambda(P)$.

10.11. $L = \{a^i b^j c^j d^i \mid i, j \in \mathbb{N}\}$.

- (a) **PDA via final state:** Use A 's to remember the outer count i and B 's to remember the middle count j . A deterministic choice of states is

$$P = \langle \{a, b, c, d\}, \{Z, A, B\}, \{q_0, q_1, q_2, q_3\}, q_0, \delta, Z, \{q_0, q_2, q_3\} \rangle,$$

with the following nonempty transitions:

$$\begin{aligned} \delta(q_0, a, Z) &= \langle q_0, AZ \rangle, & \delta(q_0, a, A) &= \langle q_0, AA \rangle, \\ \delta(q_0, b, Z) &= \langle q_1, BZ \rangle, & \delta(q_0, b, A) &= \langle q_1, BA \rangle, \\ \delta(q_0, d, A) &= \langle q_3, \lambda \rangle, & \delta(q_1, b, B) &= \langle q_1, BB \rangle, \\ \delta(q_1, c, B) &= \langle q_2, \lambda \rangle, & \delta(q_2, c, B) &= \langle q_2, \lambda \rangle, \\ \delta(q_2, d, A) &= \langle q_3, \lambda \rangle, & \delta(q_3, d, A) &= \langle q_3, \lambda \rangle. \end{aligned}$$

State q_0 is final, so λ is accepted. State q_2 is final so that the case $i = 0$ gives $b^j c^j$, and state q_3 is final so that the case $j = 0$ gives $a^i d^i$. In the general case, the machine pushes A 's for the initial a 's, then stacks B 's above them for the b -block, removes the B 's on the c -block, and finally removes the A 's on the d -block.

- (b) **PDA via empty stack:** Apply Theorem 10.5 to the PDA from part (a). This produces an equivalent PDA that accepts via empty stack.
- (c) **DPDA:** Yes. The language has a fixed block structure: $a^i b^j c^j d^i$, so the DPDA deterministically processes each block. The stack remembers i (via A s) and j (via B s overlaid on top). This is deterministic.
- (d) **Counting automaton:** No. A counting automaton has only one stack symbol. After reading $a^i b^j$, its stack is just an undifferentiated pile of identical markers. To accept this language it would need to know where the j markers for the middle comparison end and where the i markers for the outer comparison begin, so that the c 's can be matched against the b 's first and the d 's against the a 's afterward. With only one kind of stack symbol, that boundary cannot be marked.
- (e) **Grammar from PDA** (Definition 10.7): Applying Definition 10.7 to part (b) and simplifying yields the equivalent grammar

$$S \rightarrow aSd \mid T, \quad T \rightarrow bTc \mid \lambda.$$

Here S preserves the outer a/d count while T handles the middle b/c count. After i uses of $S \rightarrow aSd$ and j uses of $T \rightarrow bTc$, the derived word is $a^i b^j c^j d^i$.

10.12. Find G_{P_3} from Example 10.5 using Definition 10.7.

Since Definition 10.7 applies to PDAs that accept by empty stack, first convert P_3 to an equivalent empty-stack PDA by Theorem 10.5. Applying Definition 10.7 to that machine and deleting useless variables yields an equivalent grammar for the language of Example 10.5:

$$Z \rightarrow X \mid Y,$$

$$X \rightarrow aXb \mid ab, \quad Y \rightarrow aaYb \mid aab.$$

Here X generates $\{a^n b^n \mid n \geq 1\}$ and Y generates $\{a^{2m} b^m \mid m \geq 1\}$, so Z generates exactly the language recognized by P_3 .

10.13. Prove by induction (Theorem 10.3): $A^{st} \Rightarrow^* x \Leftrightarrow \langle s, x, A \rangle \vdash^* \langle t, \lambda, \lambda \rangle$.

Claim ($P(n)$): For all n , for all states s, t , stack symbol A , string x : $A^{st} \Rightarrow^* x$ in n steps iff $\langle s, x, A \rangle \vdash^* \langle t, \lambda, \lambda \rangle$ in n moves.

Base ($n = 1$): $A^{st} \rightarrow a$ (terminal production from grammar of Definition 10.7) corresponds to $\delta(s, a, A) \ni \langle t, \lambda \rangle$, giving $\langle s, a, A \rangle \vdash \langle t, \lambda, \lambda \rangle$. ✓

Step: Assume $P(k)$ for all $k < n$. A derivation $A^{st} \Rightarrow^* x$ of length n begins with $A^{st} \Rightarrow aB_1^{s_0 s_1} B_2^{s_1 s_2} \dots B_m^{s_{m-1} t}$ (using a production from $\delta(s, a, A) \ni \langle s_0, B_1 \dots B_m \rangle$). Then each $B_i^{s_{i-1} s_i}$ independently derives x_i in fewer than n steps. By IH, $\langle s_{i-1}, x_i, B_i \rangle \vdash^* \langle s_i, \lambda, \lambda \rangle$. Chaining: $\langle s, ax_1 \dots x_m, A \rangle \vdash^* \langle s_0, x_1 \dots x_m, B_1 \dots B_m \rangle \vdash^* \langle t, \lambda, \lambda \rangle$. So $x = ax_1 \dots x_m$ and $\langle s, x, A \rangle \vdash^* \langle t, \lambda, \lambda \rangle$. □

10.14. $L = \{a^n b^n c^m d^m\} \cup \{a^i b^j c^j d^i\}$.

- (a) **PDA via final state:** Use nondeterminism to choose which language the input belongs to. P : from initial state, nondeterministically branch to handle either $a^n b^n c^m d^m$ (Exercise 10.8) or $a^i b^j c^j d^i$ (Exercise 10.11).
- (b) **PDA via empty stack:** Start with a new initial state and make a λ -choice to the empty-stack PDA from Exercise 10.8 or the one from Exercise 10.11. This is the standard PDA construction for union.
- (c) **DPDA:** No. The nondeterministic choice in part (a) is doing real work. To accept words from

$$\{a^n b^n c^m d^m \mid n, m \geq 0\} \cup \{a^i b^j c^j d^i \mid i, j \geq 0\}.$$

the machine would need two incompatible stack disciplines during the middle of the input. For the first sublanguage it should effectively discharge the a/b comparison and then reuse the stack to compare the c -block with the d -block. For the second sublanguage it must preserve the outer a -count all the way through the middle of the word while simultaneously comparing the b -block with the c -block. That is exactly the kind of branch choice that an NPDA can make by a λ -move and a DPDA cannot. Hence no DPDA exists.

- (d) **Counting automaton:** No. The union still contains the sublanguage

$$\{a^{n+1} b^n c^n d^{n+1} \mid n \geq 0\},$$

which lies only in the second half. Accepting that family already requires the machine to preserve the outer **a**-count while separately matching the middle **b/c** count. As in Exercise 10.11(d), a one-symbol stack cannot mark the boundary between those two pieces of information.

(e) **Grammar from PDA:** An equivalent grammar is

$$\begin{aligned} S &\rightarrow AB \mid C, \\ A &\rightarrow \mathbf{aAb} \mid \lambda, & B &\rightarrow \mathbf{cBd} \mid \lambda, \\ C &\rightarrow \mathbf{aCd} \mid T, & T &\rightarrow \mathbf{bTc} \mid \lambda. \end{aligned}$$

10.15. **Find G_{P_G} from Example 10.6 using Definition 10.7.**

Since P_G has only one state s , the nonterminals are Z , S^{ss} , $\mathbf{a}^{ss}$, and $\mathbf{b}^{ss}$. Definition 10.7 gives

$$\begin{aligned} Z &\rightarrow S^{ss}, \\ S^{ss} &\rightarrow \mathbf{a}S^{ss}\mathbf{b}^{ss} \mid \mathbf{ab}^{ss}, \\ \mathbf{b}^{ss} &\rightarrow \mathbf{b}. \end{aligned}$$

The variable $\mathbf{a}^{ss}$ is useless and may be deleted. Renaming S^{ss} as S and $\mathbf{b}^{ss}$ as $\mathbf{b}$ gives the familiar grammar

$$S \rightarrow \mathbf{aSb} \mid \mathbf{ab}.$$

10.16. **Prove (Theorem 10.4):** $\langle s, xy, \alpha \rangle \vdash^* \langle s, y, \beta \rangle$ in P iff $\langle s, xy, \alpha Y \rangle \vdash^* \langle s, y, \beta Y \rangle$ in P_f .

Proof by induction on the number of moves:

Base (0 moves): Both sides are trivially equivalent (same configuration).

Step: If $\langle s, xy, \alpha \rangle \vdash^* \langle s', x'y, \alpha' \rangle \vdash^* \langle s, y, \beta \rangle$ in P : The first move in P corresponds to $\delta_P(s, a, A) \ni \langle s', \gamma \rangle$ (reading a or λ , popping A , pushing γ). In P_f , the same move is available: δ_{P_f} includes all moves of δ_P plus new moves for the marker Y . The marker Y sits below α and is not affected by moves on α . By IH on the remaining $k-1$ moves, the result follows for P_f with Y appended. $\square$

10.17. **Prove (Theorem 10.5): same correspondence for P_λ .**

Proof by induction on the number of moves: The construction of P_λ keeps all old moves of P on the old stack symbols and adds the new bottom marker Y only below the old stack.

Base (0 moves): The two configurations are identical except for the extra marker Y below the stack:

$$\langle s, xy, \alpha \rangle = \langle s, y, \beta \rangle \iff \langle s, xy, \alpha Y \rangle = \langle s, y, \beta Y \rangle.$$

Step: Suppose the correspondence holds for computations of k moves. If

$$\langle s, xy, \alpha \rangle \vdash^* \langle s', x'y, \alpha' \rangle \vdash^* \langle t, y, \beta \rangle$$

is a computation in P , then the first move reads either a letter or λ , replaces the top old stack symbol by some old stack string, and does not inspect anything below that top symbol. The same first move is present in P_λ , so

$$\langle s, xy, \alpha Y \rangle \vdash^* \langle s', x'y, \alpha' Y \rangle.$$

By the induction hypothesis applied to the remaining k moves,

$$\langle s', x' y, \alpha' Y \rangle \stackrel{*}{\vdash} \langle t, y, \beta Y \rangle.$$

Thus the computation in P gives the corresponding computation in P_λ .

Conversely, any move of P_λ while an old stack symbol is on top is one of the old moves of P . During a computation from αY to βY , the marker Y is never the top symbol until all of the old stack above it has been processed, so deleting the untouched Y from every configuration gives exactly the corresponding computation of P from α to β . Hence

$$\langle s, xy, \alpha \rangle \stackrel{*}{\vdash} \langle t, y, \beta \rangle \iff \langle s, xy, \alpha Y \rangle \stackrel{*}{\vdash} \langle t, y, \beta Y \rangle.$$

□

10.18. $\{x \in \{\mathbf{a}, \mathbf{b}, \mathbf{c}\}^* \mid |x|_{\mathbf{a}} = |x|_{\mathbf{b}} \wedge |x|_{\mathbf{b}} > |x|_{\mathbf{c}}\}$ **is not context free.**

Proof using closure properties:

Let $K = \{x \mid |x|_{\mathbf{a}} = |x|_{\mathbf{b}} \wedge |x|_{\mathbf{b}} > |x|_{\mathbf{c}}\}$.

Suppose $K \in \mathfrak{P}_\Sigma$. Consider the regular language $R = \mathbf{a}^* \mathbf{b}^* \mathbf{c}^*$ (strings of the form $\mathbf{a}^i \mathbf{b}^j \mathbf{c}^k$).

$K \cap R = \{\mathbf{a}^n \mathbf{b}^n \mathbf{c}^k \mid n > k\}$.

If K were a CFL, then $K \cap R$ would be a CFL (by Theorem 10.6: CFL $\cap$ regular = CFL). We show this is impossible.

Assume

$$H = \{\mathbf{a}^n \mathbf{b}^n \mathbf{c}^k \mid n > k\}$$

is context free, with pumping constant p . Choose

$$z = \mathbf{a}^p \mathbf{b}^p \mathbf{c}^{p-1} \in H.$$

By the pumping lemma, write $z = uvwx$ with $|vwx| \leq p$, $|vx| \geq 1$, and $uv^i wx^i y \in H$ for every $i \geq 0$.

The substring vwx has length at most p , so it cannot meet both the $\mathbf{a}/\mathbf{b}$ boundary and the $\mathbf{b}/\mathbf{c}$ boundary. If vx contains symbols from only one of the three blocks, pumping with $i = 0$ changes exactly one of the three counts. Changing only the $\mathbf{a}$ -count or only the $\mathbf{b}$ -count destroys the equality of the first two counts; changing only the $\mathbf{c}$ -count and pumping upward with $i = 2$ gives at least p copies of $\mathbf{c}$ while the common $\mathbf{a}$ - and $\mathbf{b}$ -count remains p , so $n > k$ fails.

It remains to consider the cases where vx crosses one boundary. If vx touches the $\mathbf{a}/\mathbf{b}$ boundary, then pumping with $i = 0$ either removes unequal numbers of $\mathbf{a}$'s and $\mathbf{b}$'s, destroying equality, or removes the same positive number of each. In the latter case the common count becomes at most $p - 1$, while the $\mathbf{c}$ -count remains $p - 1$, so $n > k$ again fails. If vx touches the $\mathbf{b}/\mathbf{c}$ boundary and contains a $\mathbf{b}$, pumping changes the $\mathbf{b}$ -count while the $\mathbf{a}$ -count stays p , destroying equality. If it contains only $\mathbf{c}$'s, pumping upward gives the same $n > k$ failure as above.

In every possible decomposition, some pumped word lies outside H , contradicting the pumping lemma. Therefore H is not context free.

So K is not a CFL.

□

10.19. Two-tape automaton.

- (a) **Transition function:** Figure 10.6 is a PDA-like device with one input tape and *two stack tapes*. A suitable transition function is

$$\delta : S \times (\Sigma \cup \{\lambda\}) \times \Gamma_1 \times \Gamma_2 \longrightarrow \text{finite subsets of } S \times \Gamma_1^* \times \Gamma_2^*.$$

Thus a move depends on the current state, the next input symbol (or λ), and the two current top-of-stack symbols, and it chooses a new state together with the strings that replace those two top symbols.

- (b) **Two-tape automaton for $\{a^n b^n c^n \mid n \geq 1\}$:** Use the exact strategy described just before Figure 10.6: store the initial a 's on tape 1, match them against the b 's while storing the b 's on tape 2, and then match the c 's against tape 2.

Let tape 1 use symbols Z_1, A and tape 2 use symbols Z_2, B . Take states q_0, q_1, q_2, q_f , with q_f final, and define the nonempty transitions by

$$\begin{aligned}\delta(q_0, a, Z_1, Z_2) &= \{\langle q_0, AZ_1, Z_2 \rangle\}, \\ \delta(q_0, a, A, Z_2) &= \{\langle q_0, AA, Z_2 \rangle\}, \\ \delta(q_0, b, A, Z_2) &= \{\langle q_1, \lambda, BZ_2 \rangle\}, \\ \delta(q_1, b, A, B) &= \{\langle q_1, \lambda, BB \rangle\}, \\ \delta(q_1, c, Z_1, B) &= \{\langle q_2, Z_1, \lambda \rangle\}, \\ \delta(q_2, c, Z_1, B) &= \{\langle q_2, Z_1, \lambda \rangle\}, \\ \delta(q_2, \lambda, Z_1, Z_2) &= \{\langle q_f, Z_1, Z_2 \rangle\}.\end{aligned}$$

All other transitions are empty. The first tape counts the a 's, the b -phase simultaneously removes one A from tape 1 and places one B on tape 2, and the c -phase removes the B 's. The machine reaches q_f exactly when both stacks are back to their bottom symbols after all input has been consumed, so it accepts precisely $\{a^n b^n c^n \mid n \geq 1\}$.

10.20. $\{a^n b^n c^n\}$ is not context free; part (b) proof.

- (a) **$\{a^n b^n c^n\}$ is not CFL:** Assume it is, with pumping constant N . Take $s = a^N b^N c^N$. By the pumping lemma, $s = uvwxy$ with $|vwx| \leq N$ and $|vx| \geq 1$.

Since $|vwx| \leq N$, the substring vwx cannot span all three blocks. So v and x together involve at most two of $\{a, b, c\}$. Pumping changes the count of at most two symbols while leaving the third unchanged. But all three counts must be equal, so some pumped word $uv^i wx^i y$ with $i \neq 1$ fails this equality. Contradiction. $\square$

- (b) **Proof:** The statement in part (b) is true. Let

$$K = \{x \in \{a, b, c\}^* \mid |x|_a = |x|_b = |x|_c\}.$$

Suppose, for contradiction, that K is context free. The language

$$R = a^+ b^+ c^+$$

is regular, so $K \cap R$ would be context free. But

$$K \cap R = \{a^n b^n c^n \mid n \geq 1\},$$

which part (a) proves is not context free. This contradiction proves that K is not context free. $\square$

10.21. DPDA constructions and non-DCFL example.

- (a) DPDA for $\{c^n b^m \mid n \geq 1, n = m\} \cup \{a^n b^m \mid n \geq 1, n = 2m\}$:

Let

$$P = \langle \{a, b, c\}, \{Z, A\}, \{q_0, q_c, q_{a,1}, q_{a,0}, q_b, q_f\}, q_0, \delta, Z, \{q_f\} \rangle.$$

The first symbol chooses the branch deterministically.

For the $c^n b^n$ branch, push one A for each c and then pop one A for each b :

$$\begin{aligned} \delta(q_0, c, Z) &= \langle q_c, AZ \rangle, \\ \delta(q_c, c, A) &= \langle q_c, AA \rangle, \\ \delta(q_c, b, A) &= \langle q_b, \lambda \rangle, \\ \delta(q_b, b, A) &= \langle q_b, \lambda \rangle, \\ \delta(q_b, \lambda, Z) &= \langle q_f, Z \rangle. \end{aligned}$$

For the $a^n b^m$ branch with $n = 2m$, the machine records the parity of the number of a 's: state $q_{a,1}$ means an odd number of a 's has been seen since the last push, and state $q_{a,0}$ means an even number.

$$\begin{aligned} \delta(q_0, a, Z) &= \langle q_{a,1}, Z \rangle, \\ \delta(q_{a,1}, a, Z) &= \langle q_{a,0}, AZ \rangle, \\ \delta(q_{a,1}, a, A) &= \langle q_{a,0}, AA \rangle, \\ \delta(q_{a,0}, a, A) &= \langle q_{a,1}, A \rangle, \\ \delta(q_{a,0}, b, A) &= \langle q_b, \lambda \rangle. \end{aligned}$$

Every second a pushes one A , so after reading a^n the stack contains exactly $n/2$ copies of A . The b -phase then pops one A per b . Acceptance occurs only when the input ends and the stack returns to Z , so this branch accepts exactly $\{a^{2m} b^m \mid m \geq 1\}$.

- (b) Define $h : \{a, b, c\}^* \rightarrow \{a, b\}^*$ by $h(a) = a$, $h(b) = b$, $h(c) = a$. Then: $h(\{c^n b^m \mid n = m\}) = \{a^n b^n \mid n \geq 1\}$ and $h(\{a^n b^m \mid n = 2m\}) = \{a^n b^m \mid n = 2m\}$. $h(L) = \{a^n b^n\} \cup \{a^n b^m \mid n = 2m\}$. The discussion immediately before Theorem 10.9 identifies this union as a context-free language that is *not* deterministic context free. Hence this homomorphism takes a DCFL to a non-DCFL, as required.

10.22. Ogden's lemma: $\{a^k b^n c^m \mid k \neq n \wedge n \neq m\}$ is not CFL.

Assume $L = \{a^k b^n c^m \mid k \neq n, n \neq m\}$ is CFL with Ogden constant N .

Choose $p > N$. Let $s = a^{p!+p} b^p c^{p!+p} \in L$ (since $p! + p \neq p$). Mark all b -symbols.

By Ogden's lemma: $s = uvwxy$ with vw containing at most N marked positions, vx has at least one marked position, all pumped strings in L .

Since all marks are on b s, and $|vw| \leq N \leq p$, vw lies within the b -block (possibly with some a s or c s on the boundary). Since vx has at least one b , $|vx|_b \geq 1$; let $|vx|_b = j$ ($1 \leq j \leq N$).

For the pumped string uv^iwx^iy : the count of **bs** is $p + (i - 1)j$. The counts of **as** and **cs** may also change if v or x extend into the **a** or **c** blocks.

Case 1: vx contains only **bs**. Then $k = p! + p$ and $m = p! + p$ for all i . Choose i such that $p + (i - 1)j = p! + p$: then $(i - 1)j = p!$, so $i - 1 = p!/j$. Since $j \leq N \leq p - 1$, $j \mid p!$, so i is an integer. The pumped string has $|b| = p! + p = k = m$, violating $k \neq n$ or $n \neq m$. Contradiction.

Case 2: vx spans the **a/b** boundary. Let $|vx|_a = \ell \geq 0$ and $|vx|_b = j \geq 1$ (at least one marked **b**). Choose $i - 1 = p!/j$ (an integer since $j \leq N < p$). Then the pumped string has $n = p + p! = p! + p$ and $m = p! + p$ (the **c**-block is untouched), so $n = m$, violating $n \neq m$. Contradiction.

Case 3: vx spans the **b/c** boundary. Let $|vx|_b = j \geq 1$ and $|vx|_c = r \geq 0$. Choose $i - 1 = p!/j$. The pumped string has $n = p! + p$ and $k = p! + p$ (the **a**-block is untouched), so $k = n$, violating $k \neq n$. Contradiction. $\square$

10.23. **Prove (Theorem 10.6) by induction.**

Theorem 10.6 states that if P_1 is a PDA and A_2 is a DFA (or regular language), then their “intersection” machine $P^\cap$ (Product machine) satisfies the correspondence stated.

Proof by induction on the number of moves:

Base (0 moves): $\langle \langle s_1, s_2 \rangle, y, \alpha \rangle \vdash^* \langle \langle s_1, s_2 \rangle, y, \alpha \rangle$ (trivially). Here $t_1 = s_1$, $t_2 = s_2$, $x = \lambda$, so $\bar{\delta}_2(s_2, \lambda) = s_2 = t_2$. $\checkmark$

Step: If the product machine takes $k + 1$ moves from $\langle \langle s_1, s_2 \rangle, xy, \alpha \rangle$: the first move reads some a (or λ) and moves to $\langle \langle s'_1, s'_2 \rangle, \cdot, \cdot \rangle$. In the product machine, this corresponds to a joint transition: P_1 moves from s_1 and A_2 reads a to move from s_2 . By IH on k moves, the remaining computation in the product machine corresponds to $P^\cap$ and $\bar{\delta}_2$. The A_2 component is precisely $\bar{\delta}_2(s_2, x)$ after reading x . $\square$

10.24. **Prove: DPDA P has equivalent DPDA P' with total transitions, no empty stack, scans full input.**

- (a) **Never empty stack:** First place a permanent bottom marker **\$** under the original start symbol. Introduce a new start state s_0 with the single move $\delta(s_0, \lambda, \$) = \langle s, Z\$ \rangle$, where s is the old start state and Z the old initial stack symbol. Whenever the old machine would have popped its last stack symbol, the new machine instead reaches a designated state whose stack top is **\$**. Thus the stack is never empty.
- (b) **Always scans full input:** Add an endmarker **¢** to the input alphabet. Replace every old accepting halt by a move into an accepting scan state q_{acc} , and every rejecting halt by a move into a rejecting scan state q_{rej} . In both scan states, on every ordinary input symbol the machine simply moves right and leaves the stack unchanged; on **¢** it stops. Hence every computation reads the entire input before it halts.
- (c) **Total transitions:** Now fill in whatever cases remain undefined, but in a way that respects the DPDA rules. If for some pair (q, X) there is already a λ -move, then no input move may be added there; the existing λ -move already supplies the unique next step. If there is no λ -move for (q, X) , then for each missing input symbol add a transition to q_{rej} that preserves the stack. If neither input moves nor a λ -move existed for (q, X) , add the single λ -move to q_{rej} . Thus every reachable configuration has a next move, and no input/ λ conflict is introduced. These three repairs preserve the accepted language and keep the machine deterministic, so the resulting DPDA has all the required properties. $\square$

10.25. $\mathcal{A}_\Sigma$ (DCFL) is closed under complementation.

Using Exercise 10.24: For a DPDA P (accepting via final state), P can be made total (all transitions defined), never emptying stack, and always scanning full input. Under these conditions:

Every computation on every input string either ends in a final state (accept) or a nonfinal state (reject). There are no infinite loops or premature halts.

Construct P' by swapping final and nonfinal states of P : a string is accepted by P' iff it is rejected by P . The main issue (mentioned in the hint) is λ -moves cycling through both final and nonfinal states: since Exercise 10.24 ensures the DPDA is well-behaved (complete, non-emptying, full-input scanning), after processing any input string, the computation ends deterministically in exactly one state. Swapping final/nonfinal gives the complement. $\square$

10.26. **Example:** $\mathcal{A}_\Sigma$ not closed under concatenation.

Let

$$L_1 = \{0^i 1^i 2^k \mid i, k \geq 0\} \quad \text{and} \quad L_2 = \{0^i 1^k 2^k \mid i, k \geq 0\}.$$

Both are DCFLs, but $L_1 \cup L_2$ is not. Now define

$$L_3 = aL_1 \cup L_2.$$

L_3 is a DCFL, because the initial a tells the DPDA whether to use the L_1 -routine or the L_2 -routine. The regular language a^* is certainly a DCFL. Suppose $\mathcal{A}_\Sigma$ were closed under concatenation. Then $a^* L_3$ would be a DCFL, and therefore so would

$$L_4 = a^* L_3 \cap a0^* 1^* 2^*.$$

But

$$L_4 = aL_1 \cup aL_2.$$

If a DPDA recognized L_4 , we could recognize $L_1 \cup L_2$ by simulating that DPDA after an imaginary initial a . Since $L_1 \cup L_2$ is not a DCFL, L_4 cannot be a DCFL, and so $\mathcal{A}_\Sigma$ is not closed under concatenation.

10.27. **Example:** $\mathcal{A}_\Sigma$ not closed under Kleene closure.

Let

$$L_1 = \{0^i 1^i 2^k \mid i, k \geq 0\}, \quad L_2 = \{0^i 1^k 2^k \mid i, k \geq 0\}, \quad L_3 = aL_1 \cup L_2,$$

and let

$$L_5 = \{a\} \cup L_3.$$

L_5 is a DCFL. Suppose $\mathcal{A}_\Sigma$ were closed under Kleene closure. Then L_5^* would be a DCFL, and hence so would

$$L_5^* \cap a0^* 1^* 2^*.$$

The only strings of L_5^* that have the form $a0^* 1^* 2^*$ are

$$\{a\} \cup aL_1 \cup aL_2.$$

Intersecting further with the regular complement of $\{a\}$ would leave $aL_1 \cup aL_2$, which is not a DCFL by the proof of Exercise 10.26. Therefore L_5^* cannot be a DCFL, and $\mathcal{A}_\Sigma$ is not closed under Kleene closure.

10.28. $\{c a^n b^m \mid n = m\} \cup \{a^n b^m \mid n = 2m\}$ is a DCFL.

A DPDA can distinguish the two cases by the first symbol. On the c -branch, it first consumes the initial c and then uses the standard DPDA for $\{a^n b^n\}$: push one A per a and pop one per b . On the a -branch, it uses the parity-counting construction from Exercise 10.20(a): every second a pushes one A , and each b pops one A . Thus the second branch accepts exactly $\{a^{2m} b^m \mid m \geq 1\}$. Since the first symbol determines the branch uniquely, the whole machine is deterministic. $\square$

10.29. $L_1 - R_2$ is context free if L_1 is CFL and R_2 is regular.

- (a) **Via product machine:** Extend the intersection proof (Theorem 10.6). Construct a product machine where the PDA simulates L_1 and the DFA simulates R_2 ; modify final states to accept when the PDA is in a final state and the DFA is in a *nonfinal* state. This gives $L_1 \cap \overline{R_2} = L_1 - R_2$.
- (b) **Via closure:** $L_1 - R_2 = L_1 \cap \overline{R_2}$. Since regular languages are closed under complement, $\overline{R_2}$ is regular. Since CFLs are closed under intersection with regular languages (Theorem 10.6), $L_1 \cap \overline{R_2}$ is a CFL. $\square$

10.30. $L_1 \cup R_2$ is context free if L_1 is CFL and R_2 is regular.

- (a) **Via modified product machine:** Accept when the PDA is in a final state (for L_1) OR the DFA is in a final state (for R_2). This is a product machine with final states $F_1 \times S_2 \cup S_1 \times F_2$.
- (b) **Via closure:** $L_1 \cup R_2$: since R_2 is regular, it is also a CFL. CFLs are closed under union: $L_1 \cup R_2 \in \mathfrak{P}_\Sigma$. $\square$

10.31. If L_1 is DCFL and R_2 is regular, $R_2 - L_1$ is DCFL.

$R_2 - L_1 = R_2 \cap \overline{L_1}$. Since L_1 is DCFL, $\overline{L_1}$ is DCFL (Exercise 10.25). Since R_2 is regular, and the discussion immediately after Theorem 10.6 notes that the product construction remains deterministic when the PDA factor is deterministic, $R_2 \cap \overline{L_1}$ is a DCFL. $\square$

10.32. $\{w2w^r\}$ is DCFL; $\{ww^r\}$ is not DCFL.

- (a) $\{w2w^r\}$ is DCFL: The DPDA pushes all symbols before 2 onto the stack, then pops matching symbols for each character after 2 (reversed). Since 2 marks the center deterministically, the DPDA knows exactly when to switch from push to pop mode. $\square$
- (b) $\{ww^r\}$ is not DCFL: Let

$$E = \{ww^r \mid w \in \{0,1\}^*\}.$$

Assume E were a DCFL. By Exercise 10.13, DCFLs are closed under inverse homomorphism. Define a homomorphism

$$h(a) = 01, \quad h(b) = 1010, \quad h(c) = 0101,$$

and let $R = c^+ b^+ \cup a^+ b^+$. For $x = c^n b^m$ we have

$$h(x) = (01)^{2n} (10)^{2m},$$

which is an even palindrome iff $n = m$. For $x = a^n b^m$ we have

$$h(x) = (01)^n (10)^{2m},$$

which is an even palindrome iff $n = 2m$. Therefore

$$R \cap h^{-1}(E) = \{c^n b^n \mid n \geq 1\} \cup \{a^n b^m \mid n \geq 1, n = 2m\}.$$

But Exercise 10.20 and the discussion immediately before Theorem 10.9 identify this language as not deterministic context free. This contradiction shows that E is not a DCFL. $\square$

10.33. Non-closure of $\mathcal{A}_\Sigma$ under various operations.

(a) $L_1 \cdot L_2$ need not be DCFL: Example from Exercise 10.26.

(b) $L_1 - L_2$ need not be DCFL: Let

$$L_1 = \{a^n b^n c^m \mid n, m \geq 0\} \quad \text{and} \quad K = \{a^m b^n c^n \mid m, n \geq 0\}.$$

Both L_1 and K are DCFLs. By Exercise 10.25, $\sim K$ is also a DCFL. Now

$$L_1 - (\sim K) = L_1 \cap K = \{a^n b^n c^n \mid n \geq 0\},$$

which is not even context free. So difference does not preserve $\mathcal{A}_\Sigma$.

(c) L_1^* need not be DCFL: Exercise 10.27.

(d) L_1^r need not be DCFL: let

$$L = \{a2^j 1^i 0^i \mid i, j \geq 0\} \cup \{b2^j 1^j 0^i \mid i, j \geq 0\}.$$

This is a DCFL, because the first symbol determines which deterministic routine to use. But

$$L^r = \{0^i 1^i 2^j a \mid i, j \geq 0\} \cup \{0^i 1^j 2^j b \mid i, j \geq 0\}.$$

If L^r were a DCFL, then a DPDA for L^r could inspect the final symbol a or b and thereby be turned into a DPDA for

$$\{0^i 1^i 2^j \mid i, j \geq 0\} \cup \{0^i 1^j 2^j \mid i, j \geq 0\},$$

because a DPDA for L^r could wait until it read the last symbol: if that last symbol were a , it would erase it and continue exactly as a DPDA for $\{0^i 1^i 2^j \mid i, j \geq 0\}$; if the last symbol were b , it would erase it and continue as a DPDA for $\{0^i 1^j 2^j \mid i, j \geq 0\}$. That would yield a DPDA for the displayed union, contradicting the standard non-DCFL example discussed before Theorem 10.9. So reversal does not preserve $\mathcal{A}_\Sigma$.

10.34. Closure of $\mathfrak{P}_\Sigma$ and $\mathcal{A}_\Sigma$ under quotient.

(a) $\mathfrak{P}_\Sigma$ is not closed under quotient: Let

$$L = \{a^n b^n c^{n+m} d^m \mid n, m \geq 0\} \quad \text{and} \quad K = \{c^m d^m \mid m \geq 0\}.$$

The language K is context free. So is L : a PDA pushes one marker for each a , and on each b replaces one such marker by a marker that must later be matched by a c . After those forced c 's have been read, any additional c 's are counted against the final d 's. Now

$$L/K = \{a^n b^n c^n \mid n \geq 0\}.$$

Indeed, every $a^n b^n c^n$ is in the quotient by taking the suffix $\lambda \in K$. Conversely, if $x \in L/K$, then $xy \in L$ for some $y = c^m d^m \in K$. Since every word of L has the form $a^n b^n c^{n+r} d^r$, the suffix $c^m d^m$ can only begin after exactly the first n c 's; otherwise the number of c 's preceding the d -block would exceed the number of trailing d 's. Hence $x = a^n b^n c^n$. But $\{a^n b^n c^n \mid n \geq 0\}$ is not context free, so $\mathfrak{P}_\Sigma$ is not closed under quotient.

- (b) $\mathcal{A}_\Sigma$ is **not closed under quotient**: The same counterexample works for deterministic context-free languages, because both L and K are accepted by DPDAs. For K , use the standard DPDA for $\{c^m d^m\}$. For L , use the deterministic phase structure described above:

a -phase $\rightarrow$ b -phase $\rightarrow$ forced c -phase $\rightarrow$ extra c -phase $\rightarrow$ d -phase.

Thus $L, K \in \mathcal{A}_\Sigma$, but

$$L/K = \{a^n b^n c^n \mid n \geq 0\}$$

is not even context free. Therefore $\mathcal{A}_\Sigma$ is not closed under quotient.

10.35. Closure under operator b (Theorem 5.11).

- (a) $\mathfrak{P}_\Sigma$ is **not** closed under b . Let

$$A = \{a^m b^m c^n \#\# d^{3n} \mid m, n \geq 0\}.$$

A is context free; for example,

$$S \rightarrow TU, \quad T \rightarrow aTb \mid \lambda, \quad U \rightarrow cUddd \mid \#\#.$$

Now intersect A^b with the regular language $a^* b^* c^* \#$. If $x = a^i b^j c^k \#$ belongs to this intersection, then some y of the same length satisfies $xy \in A$. The only possible choice is

$$xy = a^n b^n c^n \#\# d^{3n}$$

for some n , so $i = j = k = n$. Conversely, every $a^n b^n c^n \#$ does arise this way. Hence

$$A^b \cap a^* b^* c^* \# = \{a^n b^n c^n \# \mid n \geq 0\},$$

which is not context free. Therefore $\mathfrak{P}_\Sigma$ is not closed under b .

- (b) $\mathcal{A}_\Sigma$ is **not** closed under b . Let

$$A = \{a^n b^n c^m \#\# d^{3m} \mid n, m \geq 0\}.$$

This language is accepted by a DPDA: deterministically match the a 's against the b 's, then push three markers for each c , read the fixed delimiter $\#\#$, and pop one marker for each d . Thus $A \in \mathcal{A}_\Sigma$, but A^b is not even context free by part (a). So $\mathcal{A}_\Sigma$ is not closed under b .

10.36. Closure under operator Y (Theorem 5.7).

- (a) $\mathfrak{P}_\Sigma$ is closed under Y . If P is a PDA for L , build a new PDA with two copies of each state. In the first copy it simulates P normally, but once during the computation it may read one input letter and move to the second copy without changing the simulated stack. That single move represents the deleted letter. After that point the second copy continues to simulate P exactly. The new PDA therefore accepts precisely those strings obtained by deleting exactly one letter from a string of L .

(b) $\mathcal{A}_\Sigma$ is **not** closed under Y . Let

$$L = \{w2w^r \mid w \in \{0,1\}^*\}.$$

By Exercise 10.30(a), L is a DCFL. If we delete exactly one symbol from a word of L , the resulting string has no **2** exactly when the deleted symbol was the middle **2**. Hence

$$Y(L) \cap \{0,1\}^* = \{ww^r \mid w \in \{0,1\}^*\}.$$

By Exercise 10.30(b), the language on the right is not a DCFL. Since $\mathcal{A}_\Sigma$ is closed under intersection with regular sets, $Y(L)$ cannot be a DCFL. So $\mathcal{A}_\Sigma$ is not closed under Y .

10.37. Closure under prefix operator P (Exercise 5.16).

- (a) $\mathfrak{P}_\Sigma$ is **closed under P** : Let M be a PDA for L . Build a new PDA M' with two copies of each state of M . In the first copy, M' simulates M on the actual input. At any moment, M' may take a λ -move to the second copy. In that second copy, every transition of M that originally read a symbol of Σ is replaced by a λ -transition with the same stack effect, and the original λ -moves are kept as λ -moves. Thus, once the real input has been exhausted, M' may non-deterministically *guess* a continuation and simulate M on that guessed suffix. Therefore M' accepts exactly those x for which some y satisfies $xy \in L$, that is, exactly $P(L)$.
- (b) $\mathcal{A}_\Sigma$ is **closed under P** : Let P be a DPDA that has been prepared so that every computation reads the entire input, never empties the bottom stack marker, and halts only after the last input symbol has been processed. For each stack symbol X , use two tagged copies X^{live} and X^{dead} . While simulating P , the new machine maintains on the stack not just the current stack symbols but also, for each stack position, whether the subconfiguration beginning at that position can still be extended to an accepting computation. Because the original machine is deterministic, this tag is forced by the current state and the tagged symbols that replace X in the next move. The machine therefore updates the tags deterministically whenever it pushes or pops. After the input ends, the simulated configuration is live exactly when the topmost tag is live, and that is exactly the condition $x \in P(L)$. Hence a DPDA can accept the prefix language.

10.38. Closure under operator F (Exercise 5.19).

(a) $\mathfrak{P}_\Sigma$ is **not closed under F** . Let

$$K = \{a^i b^j c^k \mid i = j\} \cup \{a^i b^j c^k \mid j = k\}.$$

K is context free, but by Exercise 10.23 its complement over $\{a,b,c\}^*$ is not context free. Introduce a new symbol $\$$ and let

$$B = \{x\$ \mid x \in \{a,b,c\}^*\} \cup \{x\$\$ \mid x \in K\}.$$

B is context free. A word $x\$$ is in $F(B)$ exactly when $x \notin K$, because it fails to be maximal precisely when it can be extended to $x\$\$$. Therefore

$$F(B) \cap \{a,b,c\}^* \$ = \{x\$ \mid x \notin K\}.$$

If $F(B)$ were context free, then intersecting with the regular language $\{a,b,c\}^* \$$ and deleting the final $\$$ would give $\sim K$ as a CFL, contradiction.

- (b) $\mathcal{A}_\Sigma$ **is closed under F** : Start with a DPDA prepared so that it reads the whole input, never empties the bottom stack marker, and halts only after the last input symbol has been processed. For each input word x , let C_x be the unique configuration reached after all of x has been scanned. Then $x \in F(L)$ exactly when two conditions hold: C_x is accepting, and C_x is not *prolific*, meaning that no nonempty continuation from C_x can still lead to acceptance. A DPDA can track this property by tagging stack symbols as prolific or nonprolific and propagating those tags through each push and pop transition. The topmost tag then records whether the current accepting configuration has any strict extension that is also accepted. The machine accepts exactly when the current configuration is accepting and the topmost tag says “non-prolific.” So $\mathcal{A}_\Sigma$ is closed under F .

Chapter 11

Turing Machines — Solutions

11.1. How is a Turing machine like an elevator?

Analogy:

- **States** correspond to *floors* (the elevator's current position/state).
- **Input** corresponds to *button presses* (the sequence of requested floors).
- The elevator transitions from floor to floor based on button requests, just as the Turing machine transitions from state to state based on input symbols.

Essential missing component: An elevator has only *finite, bounded memory* — it can remember which buttons have been pressed (a finite set), but it cannot write new symbols to an unbounded tape. The critical component preventing an elevator from modeling a general computing device is the *unbounded read/write tape*. Without it, the elevator can only implement a finite state machine (regular language computation), not a Turing machine. The ability to store and retrieve arbitrarily large amounts of information is essential for general computation.

11.2. Turing machine for palindromes $L = \{w \mid w = w^r\}$ over $\Sigma = \{a, b, c\}$.

(a) One-tape, one-head TM:

Strategy: Repeatedly match the leftmost and rightmost unprocessed symbols.

1. Mark the leftmost symbol a (replace with $\bar{a}$); move right to the rightmost unmarked symbol.
2. If the rightmost symbol matches a , mark it and move left; continue.
3. If the rightmost symbol does not match, reject.
4. When all symbols are marked (or only the center symbol for odd-length), accept.

States: $\{q_{\text{start}}, q_a, q_b, q_c, q_{\text{right}}, q_{\text{check},a}, q_{\text{check},b}, q_{\text{check},c}, q_{\text{return}}, q_{\text{accept}}, q_{\text{reject}}\}$.

Transitions are given schematically. For each $d \in \{a, b, c\}$,

$$\delta(q_{\text{start}}, d) = (q_d, \bar{d}, R),$$

which marks the leftmost unprocessed symbol and records it in the state. In state q_d , move right over unmarked and marked input symbols until the first blank is reached; then move one square left into $q_{\text{check},d}$. In $q_{\text{check},d}$, accept the pair if the scanned unmarked symbol is

d by writing $\bar{d}$ and moving left to q_{return} ; reject if it is a different unmarked input symbol. State q_{return} moves left to the left end of the tape and then returns to q_{start} . If q_{start} finds no unmarked input symbol, it enters q_{accept} .

- (b) **Linear Bounded Automaton (LBA)**: Since the tape head never needs to go beyond the input boundaries (matching leftmost/rightmost within the input), the TM above is already an LBA. The computation uses only the n tape cells containing the input.
- (c) **Nondeterminism/multiple tapes**: A two-tape TM copies the input to tape 2, reverses tape 2, and compares tape 1 with tape 2 symbol by symbol — $O(n)$ time. A nondeterministic TM can guess the center and verify matching from both ends simultaneously.

11.3. TM recognizing $\{a^n \mid n \text{ is a perfect square}\}$ over $\Sigma = \{a\}$.

Strategy: The k -th perfect square is k^2 . Note $(k+1)^2 = k^2 + (2k+1)$: successive perfect squares differ by successive odd numbers 1, 3, 5, 7, ...

Algorithm:

1. If input is λ : accept (since $0 = 0^2$).
2. Mark the first a and subtract 1; this corresponds to $1^2 = 1$.
3. Then subtract 3; if exhausted, accept (that was $2^2 = 4$). If not enough, reject.
4. Then subtract 5; if exhausted accept ($3^2 = 9$). Etc.
5. Subtract $2k+1$ at each step for $k = 0, 1, 2, \dots$; accept if the tape is exactly emptied, reject if running short.

The TM maintains the current step k (in unary, using a separate tape region or using a scratch area) and repeats the subtraction of $2k+1$. It accepts when exactly k^2 symbols have been processed.

11.4. TM for languages over $\Sigma = \{a, b, c\}$.

- (a) $\{a^k b^n c^m \mid k \neq n \wedge n \neq m\}$:

Strategy: Count $|x|_a$, $|x|_b$, $|x|_c$ separately and verify $|x|_a \neq |x|_b$ and $|x|_b \neq |x|_c$.

A practical approach:

1. Verify the input has the form $a^k b^n c^m$ (check symbol ordering); reject if not.
 2. Pair off as and bs : erase one a and one b in each pass. If they run out simultaneously, $k = n$; reject.
 3. Restore the tape, then pair off bs and cs : erase one remaining b and one remaining c in each pass. If they run out simultaneously, $n = m$; reject.
 4. If both checks show inequality, accept.
- (b) $\{x \mid |x|_a \neq |x|_b \wedge |x|_b \neq |x|_c\}$ (arbitrary mixing):
- Strategy**: use two separate matching rounds, with fresh marks in each round, so that the b 's are still available for the second comparison.
1. In round 1, repeatedly mark one unmarked a as A and one unmarked b as B . If one kind runs out before the other, then $|x|_a \neq |x|_b$ is confirmed. If they run out together, reject.
 2. Restore all B 's to ordinary b 's.

3. In round 2, repeatedly mark one unmarked **b** as *B* and one unmarked **c** as *C*. If one kind runs out before the other, then $|x|_b \neq |x|_c$ is confirmed. If they run out together, reject.
4. Accept iff both checks confirm inequality.

11.5. **Machine M with $L(M) = L_1(M) = L_2(M) = L_3(M) = \{x \in \{a, b, c\}^* \mid |x|_a = |x|_b = |x|_c\}$.**

(a) **Construction:** First let M deterministically test whether the input has equally many **a**'s, **b**'s, and **c**'s:

1. Scan from the left to find the first unmarked **a**. If there is none, check whether any unmarked **b** or **c** remains. If not, the test succeeds; otherwise it fails.
2. Mark that **a**, continue right to find the first unmarked **b**, and mark it. If no such **b** exists, the test fails.
3. Continue right to find the first unmarked **c** and mark it. If no such **c** exists, the test fails.
4. Return to the left end and repeat.

Then:

1. If the test succeeds, erase the whole tape, write a single **Y**, position the head on that **Y**, and halt in a final state.
2. If the test fails, do *not* halt: enter a simple two-state loop that moves right and left forever on blanks.

For a yes-instance, the machine halts in a final state, with **Y** somewhere on the tape and in fact as the rightmost nonblank symbol. Thus the input belongs to all four sets $L(M), L_1(M), L_2(M), L_3(M)$. For a no-instance, the machine never halts, so it belongs to none of them. Therefore all four acceptance notions define exactly the required language.

(b) **Is this always possible?:** Yes, for *every* Turing-acceptable language L . Let A be any Turing machine with $L(A) = L$. Build a new machine M that simulates A , but modifies only the accepting behavior:

1. If A reaches an accepting halt, then M erases the tape, writes a single **Y**, positions the head on that **Y**, enters a final state, and halts.
2. If A would reject, or if A never accepts, then M simply never halts (for example, it can enter the same two-state blank-tape loop used in part (a)).

Then $x \in L$ exactly when M halts, exactly when it halts with **Y** on the tape, exactly when it halts with **Y** at the right end of the tape, and exactly when it halts in a final state. Hence

$$L(M) = L_1(M) = L_2(M) = L_3(M) = L.$$

11.6. $L = \{ww \mid w \in \{a, b, c\}^*\}$.

(a) **One-tape TM:** Strategy:

1. Sweep right across the input and write its length n in binary in a scratch region of the same tape to the right of the word. If n is odd, reject.
2. Replace that counter by $m = n/2$.
3. For each $i = 1, 2, \dots, m$, use a second binary counter on the scratch region to locate the i th symbol of the word and remember whether it is **a**, **b**, or **c**.

4. Using the counters again, locate the $(m + i)$ th symbol and compare it with the remembered symbol. If they differ, reject.
 5. If all m comparisons succeed, accept.
- (b) **LBA:** Let the input length be n . An LBA can use exactly the n input cells and marked versions of the input symbols. First, repeatedly mark the leftmost unmarked symbol as belonging to the left half and the rightmost unmarked symbol as belonging to the right half. If the final left mark has no right-hand mate, then n is odd and the machine rejects. Otherwise the two halves each have $n/2$ cells. Repeatedly compare the leftmost unprocessed symbol of the left half with the leftmost unprocessed symbol of the right half, marking both as processed and rejecting on a mismatch. The machine accepts when every pair agrees. All marks overwrite original input cells, so no cell outside the length- n input region is scanned.
- (c) **Multiple tapes:** A two-tape TM copies the first half of the input to tape 2, then compares tape 2 with the second half of tape 1 symbol by symbol. A nondeterministic TM can guess the midpoint and verify.

11.7. Lexicographic ordering Turing machines.

Given $\Sigma = \{a_1, \dots, a_n\}$ with lexicographic order based on base- n numbering.

- (a) **Successor TM:** Machine M_{succ} maps each word to the next word in the bijective base- n ordering described in the exercise.
- Algorithm: If the tape is blank (that is, the word is λ), write a_1 . Otherwise process the word from right to left. Find the rightmost symbol a_i with $i < n$; replace it with a_{i+1} and replace every symbol to its right with a_1 . If every symbol is a_n , so the word is a_n^k , replace it by a_1^{k+1} . This is exactly the next word in the stated order.
- (b) **Generator TM:** The initial blank tape already represents the first word λ . Apply M_{succ} to obtain a_1 , then repeatedly erase the current word and replace it by its successor. Thus the visible word sequence is

$$\lambda, a_1, a_2, \dots, a_n, a_1 a_1, \dots$$

in lexicographic order.

- (c) **Enumerator TM:** Again the blank tape represents the first word λ . Keep the current word in the rightmost block of nonblank cells. To append the next word, copy that rightmost block to a fresh work area, run M_{succ} on the copy, and leave the result to the right of one blank separator. Repeating this appends each successive word while leaving the earlier ones untouched.
- (d) **Deterministic simulation of NDTM:** Given a nondeterministic TM N , simulate it deterministically using the lexicographic ordering of computation paths. At step k , enumerate all possible k -step computation paths (sequences of transitions) using the lexicographic order, and simulate each one. If any path reaches an accepting state, accept. This gives a deterministic simulation (though potentially much slower). M_{succ} generates the next computation path to try.

11.8. Semi-infinite tape Turing machine.

A *semi-infinite tape* has a left boundary (leftmost cell = cell 1) and extends indefinitely to the right.

- (a) **Two-track simulation:** Store the nonnegative cells $0, 1, 2, \dots$ of the old tape on the upper track and the negative cells $-1, -2, -3, \dots$ on the lower track in reverse order. Thus column 0 contains the old cell 0 on the upper track and a special boundary marker on the lower track, while column $k \geq 1$ contains old cell k on the upper track and old cell $-k$ on the lower track.

The finite control remembers whether the simulated head is on the upper or lower track. Moving right on the old nonnegative side means moving one column right on the upper track; moving left from old cell 0 switches to the lower track in column 1; moving right from old cell -1 switches back to the upper track in column 0; and all other moves on the negative side are simulated by moving one column left or right on the lower track as appropriate.

- (b) **Equivalence proof:** Induct on the number of simulated steps. The induction invariant is that the two tracks encode exactly the same tape contents as the original bi-infinite tape, and that the simulated head position and state are correctly represented by the current column together with the remembered track. Each move of the old machine changes only one scanned cell and shifts the head by one position, and the semi-infinite simulation reproduces exactly that effect. Therefore the two machines accept and reject the same words. $\square$

11.9. TM recognizing $\{a^n \mid n \text{ is a power of } 2\}$.

Strategy: Repeatedly check if n is divisible by 2 and halve it until reaching 1.

Algorithm:

1. If input is a ($n = 1 = 2^0$), accept.
2. If input has odd length > 1 , reject.
3. Sweep left to right, marking every other a with $\bar{a}$. If n is odd (and > 1) an unmarked a remains at the end — reject. Otherwise, replace all unmarked a s with blanks; the $\bar{a}$ s form the halved string. Repeat.
4. Repeat until one symbol remains; accept.

11.10. Three-head TM for $\{x \mid |x|_a = |x|_b = |x|_c\}$.

Design: All three heads start at the leftmost character.

Strategy: - Head 1 scans for a -symbols (moving right, skipping b and c). - Head 2 scans for b -symbols. - Head 3 scans for c -symbols.

One synchronization step advances each head, in turn, to its next unmatched symbol of the appropriate type or to the right end of the input. After all three advances are complete, accept if all three heads reached the end; reject if exactly one or two heads reached the end; otherwise begin the next synchronization step.

No head ever needs to move left: Each head only moves right (it scans past symbols of other types, looking for its own). Since the heads never backtrack, none needs to move left. $\square$

11.11. Every CFL is Turing-acceptable.

Proof (Lemma 11.1): Let L be a CFL with PDA $P = \langle \Sigma, \Gamma, S, s_0, \delta, Z_0, F \rangle$.

Construct a TM M that simulates P : - Tape 1: input string x . - Tape 2: stack of P (top of stack at left). - M simulates each PDA move: reads next input symbol (or λ), looks up δ , pops the stack top,

pushes the new stack string (nondeterministically if $|\delta| > 1$). - If the PDA reaches a final state with empty input, M accepts.

Since the PDA is nondeterministic, M is a nondeterministic TM. A deterministic TM can simulate such a machine by enumerating all finite computation paths in lexicographic order and checking them one by one.

Thus $L = L(P) = L(M) \in \mathfrak{T}_{\Sigma}$. □

11.12. Every type 1 language is a LBL.

Proof (Theorem 11.3): Let $G = \langle \Sigma, \Omega, Z, P \rangle$ be a context-sensitive grammar. We follow the construction discussed just before Theorem 11.3, namely the two-tape machine used in the proof of Theorem 11.2.

On input x of length n , tape 1 holds x and is never altered. Tape 2 starts with just the start symbol Z . The finite control nondeterministically chooses a production from P and a position in the current sentential form on tape 2, checks whether the left side of that production occurs at that position, and if so replaces that occurrence by the right side by shifting the surrounding tape contents as needed.

The only extra observation needed for a type 1 grammar is that, once the machine has begun deriving a nonempty word, the length of the sentential form can never decrease. Therefore any branch on which tape 2 ever becomes longer than n can be abandoned immediately, because it can never later shrink back to the target word x . Thus every surviving computation keeps tape 2 within n cells.

If $x = \lambda$, the only successful derivation is the special initial production $Z \rightarrow \lambda$. If $x \neq \lambda$, the machine accepts exactly when some sequence of guessed productions makes tape 2 agree with tape 1. Thus this two-tape construction is bounded by a linear amount of tape, and hence is a nondeterministic LBA. Therefore every type 1 language is a LBL. □

11.13. Convert LBA M to three-track strict LBA.

- (a) **New alphabets:** - Track 1 carries the old tape alphabet Γ , not merely the input alphabet Σ . - Track 2 marks the left end of the original word. - Track 3 marks the right end of the original word.

It is convenient to compress these three tracks into a single alphabet

$$\Gamma' = \Gamma \cup \{\bar{d}, \underline{d} \mid d \in \Gamma\},$$

where $\bar{d}$ means that d is currently in the leftmost cell of the word and $\underline{d}$ means that d is currently in the rightmost cell. For words of length at least 2, these two markers occur in different cells.

- (b) **New transitions:** For any old transition $\delta(s, a) = (s', a', d)$ in M : - If the old move stays inside the word, copy it directly, preserving any overbar or underline markers on the end cells. - If the old machine would move left off the word to scan the left delimiter, replace that excursion by a short detour through a new state that records “the left delimiter is being scanned.” Because the delimiter can only return to the word in one way, this detour is unique. - Do the symmetric construction on the right end of the word for moves that would have scanned the right delimiter.

In other words, the strict machine does *not* reject when the old machine would have stepped onto a delimiter; instead, it simulates the unique delimiter-scan-and-return behavior without ever leaving the marked end cells.

- (c) **Proof of equivalence** (for $|x| \geq 2$): By induction on the number of steps: - Every ordinary move of M is matched by the corresponding move of M' while preserving the marked end cells. - Every two-step excursion of M from an end cell onto a delimiter and back is replaced by the corresponding unique boundary detour in M' . - Hence the two machines visit corresponding configurations inside the word and accept exactly the same words.

So for every input of length at least 2, the strict three-track machine accepts exactly when the original LBA does. $\square$

11.14. Handle strings of length 1; strict LBA for type 1 languages.

- (a) **Strings of length 1**: For each tape symbol d , add a special symbol $\bar{d}$ meaning “this cell is both the leftmost and the rightmost cell.” The modified strict LBA duplicates every transition that previously referred to $\bar{d}$ or $\underline{d}$, replacing those marked symbols by $\bar{d}$ whenever the word has length 1. In this way the machine still knows that the scanned cell is simultaneously the left and right boundary cell.
- (b) **Strict LBA accepting λ** : If the initial scan is of a blank cell, interpret that as the empty input and have the machine decide immediately whether λ should be accepted. After that special start-up case, the strict machine uses three tracks: one track stores the tape symbols, one marks the left end of the input, and one marks the right end. The transition function is changed so that whenever the old machine would step onto an endmarker and then return, the new machine performs the corresponding detour entirely inside the marked input cells. Thus every type 1 language can be recognized by a strict LBA in this slightly relaxed sense. $\square$

11.15. Prove (Theorem 11.4) by induction: TM computation iff LBA grammar derivation.

Induction statement: For every $k \geq 0$,

$$\alpha s \beta \vdash^* \gamma t \omega$$

if and only if there exist $i, j, m, n \in \mathbb{N}$ such that

$$[\#^i \alpha s \beta \#^j] \Rightarrow^* [\#^m \gamma t \omega \#^n],$$

where the derivation uses only class 2 productions.

Base case ($k = 0$): $\alpha s \beta \vdash^0 \alpha s \beta$, and likewise

$$[\#^i \alpha s \beta \#^j] \Rightarrow^0 [\#^i \alpha s \beta \#^j].$$

Induction step: Assume the statement holds for k . Suppose

$$\alpha s \beta \vdash^{* k+1} \mu u v.$$

Then there is an intermediate configuration $\gamma t \omega$ with

$$\alpha s \beta \vdash^{*k} \gamma t \omega \vdash^{*} \mu u v.$$

By the induction hypothesis,

$$[\#^i \alpha s \beta \#^j] \Rightarrow^{*k} [\#^m \gamma t \omega \#^n]$$

for suitable i, j, m, n . The final Turing-machine move is produced by one transition of M , and Definition 11.8 includes exactly one corresponding class 2 production. Applying that production yields

$$[\#^m \gamma t \omega \#^n] \Rightarrow^{*} [\#^p \mu u v \#^q]$$

for some $p, q \in \mathbb{N}$, depending only on whether the head has just moved onto a new blank cell. Hence

$$[\#^i \alpha s \beta \#^j] \Rightarrow^{*k+1} [\#^p \mu u v \#^q].$$

The converse is identical: every class 2 production encodes one legal move of M , so a derivation of length $k + 1$ factors into a derivation of length k followed by one TM move. Therefore the claim holds for all k . $\square$

11.16. General induction statement for Theorem 11.5.

Theorem 11.5 establishes that every LBL is a type 1 language.

General induction statement: Fix a strict LBA B and a nonempty input word of length at least 2, written $\alpha x b$. For each configuration C of B , let $\phi(C)$ be the corresponding sentential form in which the scanned tape symbol carries the current state as a subscript, while the first and last tape symbols retain their overbar and underline markers. Then for every $k \geq 0$ and every configuration C ,

$$s_0 \alpha x b \vdash^{*k} C \quad \text{iff} \quad \bar{a}_0 x \underline{b} \Rightarrow^{*k} \phi(C),$$

where only class 2 productions are used.

Base case ($k = 0$): The initial configuration is exactly $s_0 \alpha x b$, and its encoding is $\bar{a}_0 x \underline{b}$.

Induction step: Assume the claim for k . If

$$s_0 \alpha x b \vdash^{*k+1} C',$$

choose C with

$$s_0 \alpha x b \vdash^{*k} C \vdash^{*} C'.$$

By the induction hypothesis,

$$\bar{a}_0 x \underline{b} \Rightarrow^{*k} \phi(C).$$

Definition 11.9 gives one class 2 production for each possible move of the strict LBA, including the boundary cases where the scanned symbol is marked with an overbar or underline. Hence the last move $C \vdash^{*} C'$ is mirrored by one derivation step $\phi(C) \Rightarrow^{*} \phi(C')$. The converse direction is the same, since every class 2 production encodes a legal move of B and nothing else. This is the induction statement needed in the proof of Theorem 11.5. $\square$

11.17. Type 0 language Kleene closure.

- (a) **Flaw in** $G_* = \langle \Sigma, \Omega \cup \{W\}, W, P \cup \{W \rightarrow \lambda, W \rightarrow WW, W \rightarrow Z\} \rangle$:

The difficulty is that the two occurrences of Z generated by $W \rightarrow WW$ are placed *adjacent* to one another. Since the productions of a type 0 grammar may inspect several neighboring symbols at once, a production from P can then act across the boundary between the two intended copies. So the construction can generate strings that are not concatenations of words from $L(G)$.

- (b) **Example:** Let

$$G = \langle \{a, b\}, \{Z, A, B\}, Z, \{Z \rightarrow AB, A \rightarrow \lambda, B \rightarrow b, BA \rightarrow a\} \rangle.$$

For a single copy of Z , the rule $BA \rightarrow a$ is unusable, so $L(G) = \{b\}$. But in the naive construction,

$$W \Rightarrow WW \Rightarrow ZZ \Rightarrow ABAB,$$

and now the middle substring BA straddles the boundary between the two copies. Applying $BA \rightarrow a$ there gives

$$ABAB \Rightarrow AaB \Rightarrow ab.$$

Since $ab \notin \{b\}^*$, the naive construction is wrong.

- (c) **Correct** G_* : A separator alone is insufficient if it may be erased before its neighboring derivations are complete. Instead, simulate each copy of G using a tagged copy of every symbol. For each $X \in \Sigma \cup \Gamma$, choose a fresh nonterminal $\hat{X}$, and for a word $\alpha = X_1 \cdots X_r$ write $\hat{\alpha} = \hat{X}_1 \cdots \hat{X}_r$. Let

$$\hat{P} = \{\hat{\alpha} \rightarrow \hat{\beta} \mid \alpha \rightarrow \beta \in P\}.$$

Choose fresh nonterminals W and $\#$, and define

$$G_* = \langle \Sigma, \{\hat{X} \mid X \in \Sigma \cup \Gamma\} \cup \{W, \#\}, W, P_* \rangle,$$

where P_* contains $\hat{P}$ together with

$$W \rightarrow \lambda \mid \hat{Z} \mid \hat{Z}\#W, \quad \hat{a} \rightarrow a \quad (a \in \Sigma),$$

and the sole separator-removal rules

$$a\#b \rightarrow ab \quad (a, b \in \Sigma).$$

The rule $a\# \rightarrow a$ is deliberately absent: the final copied word is created by $W \rightarrow \hat{Z}$, so no trailing separator has to be erased. An internal separator can disappear only when a terminal symbol occurs on both sides. Even if the rest of the right-hand copy has not yet finished, its first untagged terminal symbol blocks every production in $\hat{P}$, whose symbols are all tagged. Thus tagged productions cannot cross either a separator or an untagged terminal symbol.

- (d) **Proof:** Let

$$w = w_1 w_2 \cdots w_k \quad \text{with each } w_i \in L(G) - \{\lambda\}.$$

For $k > 0$, start with $W \rightarrow \hat{Z}\#W$ exactly $k-1$ times and then use $W \rightarrow \hat{Z}$. In each tagged block, use $\hat{P}$ to simulate a derivation $Z \xRightarrow{*}_G w_i$, then apply $\hat{a} \rightarrow a$ to expose the terminal letters of w_i .

Once terminal symbols appear on both sides of each boundary, the rules $a\#b \rightarrow ab$ remove the resulting separators, giving w . The zero-factor derivation gives λ . Since $(L(G) - \{\lambda\})^* = L(G)^*$, this proves $L(G)^* \subseteq L(G_*)$.

Conversely, consider a terminal derivation from W . Every tagged block is created by one of the two nonempty alternatives for W . In a terminal result, the tagged nonterminals in each block have been eliminated by rules from $\hat{P}$, and every remaining tagged terminal has been converted by a rule $\hat{a} \rightarrow a$. If the tags are erased from this sequence of steps, each $\hat{P}$ step is a production of G and each conversion step changes nothing, so the resulting nonempty terminal word belongs to $L(G)$.

A separator can be removed only by $a\#b \rightarrow ab$. At that moment the untagged letters a and b remain between the two copies, so no tagged production can ever cross their former boundary, even if an unfinished tagged substring occurs farther to the right. Therefore every terminal word derived from W is a finite concatenation of words in $L(G)$, and $L(G_*) \subseteq L(G)^*$. Hence $L(G_*) = L(G)^*$. $\square$

11.18. $\mathfrak{T}_\Sigma$ is closed under various operations.

- (a) **Homomorphism:** Let M accept L , and let $h : \Sigma \rightarrow \Gamma^*$ be a homomorphism. On input $y \in \Gamma^*$, enumerate the words of Σ^* in lexicographic order (Exercise 11.7), and dovetail the simulations of M on those words. Whenever one simulation accepts some x , compute $h(x)$ and compare it with y ; if $h(x) = y$, accept. If $y \in h(L)$, some $x \in L$ with $h(x) = y$ will eventually be found, so $h(L) \in \mathfrak{T}_\Sigma$.
- (b) **Inverse homomorphism:** On input x , compute $h(x)$ and simulate M on $h(x)$. Then the new machine accepts exactly the words in $h^{-1}(L)$.
- (c) **Concatenation:** Let M_1 and M_2 accept L_1 and L_2 . On input y , list the finitely many splits $y = uv$. Simulate every associated pair (M_1, u) and (M_2, v) in round-robin fashion: at stage t , perform one more step of each simulation that has not halted. Accept as soon as both simulations for one split have accepted. If $y \in L_1 L_2$, the accepting split eventually gets enough simulated steps for both machines to accept; if no split works, the machine need not halt. Thus this semidecides $L_1 L_2$.
- (d) **Substitution:** Let s be a substitution with $s(\mathbf{a}) \in \mathfrak{T}_\Sigma$ for each $\mathbf{a} \in \Sigma$, and let M accept L . On input y , enumerate all pairs consisting of a word $x = \mathbf{a}_1 \cdots \mathbf{a}_k \in \Sigma^*$ and a factorization $y = y_1 \cdots y_k$ (allowing some $y_i = \lambda$). For each such pair, dovetail the computation of M on x together with the computations of machines for $s(\mathbf{a}_i)$ on y_i . Accept when one pair makes all these machines accept. That happens exactly when $y \in s(L)$. Hence $\mathfrak{T}_\Sigma$ is closed under substitution. $\square$

11.19. Equivalence of the acceptance modes L_1 , L_2 , and L_3 .

- (a) $L_1(A_1)$ to $L_2(A_2)$: Let $A_1 = \langle \Sigma, \Gamma, S, s_0, \delta \rangle$ and suppose $L = L_1(A_1)$. Form A_2 by adjoining a short clean-up routine with new states q_L, q_R, h' . Every old transition that formerly entered the halt state h is redirected to q_L . In q_L , the machine sweeps to the left end of the nonblank portion of the tape; in q_R , it erases everything while moving right until the first blank; then it writes a single $\mathbf{Y}$ and enters h' . Thus A_2 halts with $\mathbf{Y}$ on the tape exactly on the inputs for which A_1 halts at all. Hence $L_2(A_2) = L_1(A_1) = L$.

- (b) $L_2(A_2)$ **to** $L_3(A_3)$: Let A_2 be a machine with $L = L_2(A_2)$. Again redirect each old transition to the halt state into a new inspection state. That routine scans the tape once to determine whether any Y occurs. If one is found, it erases the tape, writes a single Y , and halts. If no Y is found, it halts with some other final tape contents, for example a single N . Therefore A_3 halts with Y at the right end of the tape exactly when A_2 would have halted with Y somewhere on the tape. So $L_3(A_3) = L_2(A_2) = L$.
- (c) $L_3(A_3)$ **to** $L_1(A_1)$: Let A_3 be a machine with $L = L_3(A_3)$. Redirect every transition entering the halt state of A_3 to a new verification routine. The routine scans right to the first blank cell, steps left, and checks the last nonblank tape symbol. If that symbol is Y , it enters a new halt state; otherwise it enters a simple infinite loop. Hence A_1 halts exactly when A_3 halts with Y at the right end of its tape. Therefore $L_1(A_1) = L_3(A_3) = L$. $\square$

11.20. $\mathcal{D}_\Sigma$ (context-sensitive languages) is closed under the same operations.

For a fixed input, an LBA has only finitely many configurations, so any LBA can be modified to halt by rejecting as soon as a configuration repeats. Also, for context-sensitive languages the standard closure statements for homomorphism and substitution are understood in the *nonerasing* case. That is the interpretation used in parts (a) and (d).

- (a) **Homomorphism**: Let $h : \Sigma \rightarrow \Gamma^+$ be nonerasing and let $L \in \mathcal{D}_\Sigma$. By Corollary 11.2 there is a halting LBA B for L . On input $y \in \Gamma^*$, enumerate all $x \in \Sigma^*$ with $h(x) = y$. Because h is nonerasing, every such x has length at most $|y|$, so only finitely many words need be checked. Run B on each of them and accept iff one is accepted. This halts on every input, so $h(L) \in \mathcal{D}_\Sigma$.
- (b) **Inverse homomorphism**: On input x , first compute $h(x)$ and then run the halting machine for L on that word. This composed machine halts because the preliminary computation of $h(x)$ is finite and the machine for L halts on every input. It accepts exactly when $h(x) \in L$, which is exactly when $x \in h^{-1}(L)$. Therefore $h^{-1}(L) \in \mathcal{D}_\Sigma$.
- (c) **Concatenation**: Given halting machines for L_1 and L_2 , list all splits $y = uv$ of the input and test each one. Since the list of splits is finite and both machines halt on every input, this decides $L_1 L_2$.
- (d) **Substitution**: Let s be a nonerasing substitution with $s(a) \in \mathcal{D}_\Sigma$ for each $a \in \Sigma$. On input y , enumerate all factorizations $y = y_1 \cdots y_k$ into nonempty pieces and all words $x = a_1 \cdots a_k \in \Sigma^k$ of the same length k . Because the pieces are nonempty, only finitely many such choices exist. Test whether $x \in L$ and $y_i \in s(a_i)$ for every i ; accept iff (at least) one choice works. Hence $\mathcal{D}_\Sigma$ is closed under nonerasing substitution. $\square$

Chapter 12

Decidability — Solutions

12.1. Verify assertions in proof of Theorem 12.1 concerning Theorem 2.7.

The point used in Theorem 12.1 is this: if A accepts any string at all, then A accepts one of length $< n$, where $n = \|S\|$.

Let

$$A = \langle \Sigma, S, s_0, \delta, F \rangle$$

and suppose $x \in L(A)$ with $|x| \geq n$. By Theorem 2.7, $L(A)$ is the union of classes of a finite-rank right congruence; in particular, we may use R^A , whose rank is at most $\|S\| = n$. Write

$$x = a_1 a_2 \cdots a_m$$

and consider the $n + 1$ prefixes

$$\lambda, a_1, a_1 a_2, \dots, a_1 a_2 \cdots a_n.$$

Since R^A has at most n classes, two of these prefixes must be related by R^A . So we can write

$$x = uvw$$

with $v \neq \lambda$ and $u R^A uv$. Because R^A is a right congruence,

$$uw R^A uvw = x.$$

Now $L(A)$ is a union of R^A -classes, and $x \in L(A)$, so $uw \in L(A)$ as well. Thus we have found a shorter accepted string. Repeating this shortening process eventually produces an accepted string of length $< n$.

That is exactly the assertion needed in the proof of Theorem 12.1. □

12.2. Prove Lemma 12.1.

Let

$$M = \{x \mid x \in L(A) \wedge |x| \geq n\}.$$

If $M \neq \emptyset$, then the set

$$\{|x| \mid x \in M\} \subseteq \mathbb{N}$$

is nonempty, so by the well-ordering principle it has a least element. Hence M contains a string of minimal length; call it x_m .

It remains to prove that $|x_m| < 2n$. Suppose, for contradiction, that $|x_m| \geq 2n$. Since $x_m \in L(A)$ and $|x_m| \geq n$, the pumping lemma (Theorem 2.3) gives a factorization

$$x_m = uvw$$

with

$$|uv| \leq n, \quad |v| > 0,$$

and

$$uv^i w \in L(A) \quad \text{for all } i \in \mathbb{N}.$$

Take $i = 0$. Then

$$uw \in L(A).$$

Also,

$$|uw| = |x_m| - |v| \geq |x_m| - n \geq n,$$

since $|v| \leq |uv| \leq n$. Thus $uw \in M$. But $|v| > 0$, so

$$|uw| < |x_m|,$$

contradicting the minimality of x_m .

Therefore $|x_m| < 2n$. □

12.3. Number of nonfinal states in minimal DFA $\leq$ nonfinal states in any machine B with $L(B) = L$.

Proof: Let M be the minimal DFA for L . By Nerode's theorem, the states of M correspond exactly to the classes of R_L . In particular, the nonfinal states of M correspond exactly to the classes C for which every string in C is *not* in L .

For each such nonfinal class C , choose a representative string $x_C \notin L$. Now let

$$B = \langle \Sigma, S_B, s_B, \delta_B, F_B \rangle$$

be any DFA with $L(B) = L$. Since $x_C \notin L$, the state

$$\bar{\delta}_B(s_B, x_C)$$

must be nonfinal.

We claim that different nonfinal classes of R_L must lead to different states of B . Suppose $C_1 \neq C_2$ but

$$\bar{\delta}_B(s_B, x_{C_1}) = \bar{\delta}_B(s_B, x_{C_2}).$$

Since C_1 and C_2 are distinct Nerode classes, there exists a string z such that exactly one of $x_{C_1}z$ and $x_{C_2}z$ belongs to L . But starting from the same state of B , reading the same suffix z must give the same accept/reject answer for both strings, a contradiction.

Thus distinct nonfinal Nerode classes determine distinct nonfinal states of B . Therefore the number of nonfinal states in M is less than or equal to the number of nonfinal states in B . □

12.4. Decidability of $L(A_1) \subseteq L(A_2)$.

Algorithm:

- (a) Construct DFA A_2^c for $\overline{L(A_2)}$ (complement: swap final/nonfinal states of A_2 's complete DFA).
- (b) Construct DFA $A^\cap$ for $L(A_1) \cap \overline{L(A_2)}$ (product machine).
- (c) $L(A_1) \subseteq L(A_2)$ iff $L(A_1) \cap \overline{L(A_2)} = \emptyset$ iff $L(A^\cap) = \emptyset$ (decidable by Theorem 12.1). $\square$

12.5. Decidability of $L(A)$ is cofinite.

$L(A)$ is cofinite iff $\overline{L(A)}$ is finite.

Algorithm:

- (a) Construct DFA A^c for $\overline{L(A)}$ (complement).
- (b) $\overline{L(A)}$ is finite iff $L(A^c)$ is finite iff the DFA A^c (after removing unreachable states) has no cycle reachable from the start state that also leads to a final state (Theorem 12.2 analog).
- (c) Check for cycles in A^c among useful states (BFS/DFS). Decidable. $\square$

12.6. Decidability: does $L(A)$ contain any string of length > 1228 ?

Algorithm: Build the DFA

$$B = \langle \Sigma, \{r_0, r_1, \dots, r_{1229}\}, r_0, \delta_B, \{r_{1229}\} \rangle,$$

where for every $a \in \Sigma$,

$$\delta_B(r_i, a) = r_{\min\{i+1, 1229\}}.$$

Then

$$L(B) = \{x \in \Sigma^* \mid |x| \geq 1229\},$$

that is, the strings of length greater than 1228.

Now form the product DFA $A^\cap$ for

$$L(A) \cap L(B).$$

Then

$$L(A) \text{ contains a string of length } > 1228$$

iff

$$L(A^\cap) \neq \emptyset.$$

By Theorem 12.1, emptiness for DFAs is decidable. Hence this question is decidable. $\square$

12.7. Decidability: does A accept any even-length strings?

Algorithm:

- (a) Construct DFA A_{even} that recognizes $\{x \mid |x| \text{ is even}\}$ (a simple 2-state DFA alternating between even/odd states).
- (b) Construct product DFA $A^\cap$ for $L(A) \cap L(A_{\text{even}})$.
- (c) A accepts an even-length string iff $L(A^\cap) \neq \emptyset$. Decidable by Theorem 12.1. $\square$

12.8. Decidability: R_1 and R_2 represent complementary languages.

Algorithm:

- (a) Convert regular expressions R_1 and R_2 to DFAs A_1 and A_2 .
- (b) Check $L(A_1) = \overline{L(A_2)}$: equivalently, check $L(A_1) \subseteq \overline{L(A_2)}$ and $\overline{L(A_2)} \subseteq L(A_1)$.

(c) By Exercise 12.4, each subset relation is decidable. Equivalently, check $L(A_1) \cup L(A_2) = \Sigma^*$ and $L(A_1) \cap L(A_2) = \emptyset$.

(d) Both conditions are decidable (using complement, intersection, and emptiness). □

12.9. Decidability: R_1 and R_2 describe common strings.

Algorithm: $L(R_1) \cap L(R_2) \neq \emptyset$?

(a) Convert R_1, R_2 to DFAs A_1, A_2 .

(b) Compute the product DFA $A^\cap$ for $L(A_1) \cap L(A_2)$.

(c) Check $L(A^\cap) \neq \emptyset$ (decidable by Theorem 12.1). □

12.10. Decidability: is there a DFA with < 31 states accepting $L(R_1)$?

Algorithm:

(a) Convert R_1 to a DFA A (via NFA construction).

(b) Minimize A to get A_{min} .

(c) There is a DFA with < 31 states accepting $L(R_1)$ iff $|A_{min}| < 31$.

(d) Count the states of A_{min} : decidable. □

12.11. Decidability: is there a DFA with > 31 states accepting $L(R_1)$?

One-step algorithm: Yes, always. If A_{min} has k states, then we can construct a DFA with $k + \ell$ states for any $\ell > 0$ by adding dead (sink) states. If $k \leq 31$, add $32 - k$ unreachable dead states. If $k > 31$, A_{min} itself already has > 31 states.

Therefore, for *any* regular language, there always exists a DFA with > 31 states accepting it. The answer is **always yes**. □

12.12. Decidability: $\exists \lambda$ -NFA with ≤ 1 final state accepting R .

Algorithm:

(a) Use the usual λ -NFA form of the structural construction in Lemma 6.2. The resulting regular-expression-to-automaton correspondence is recorded in Corollary 6.1 and summarized by Kleene's Theorem 6.5. Give every subexpression a fresh entry state and a fresh exit state; the union, concatenation, and star cases join these pairs with λ -moves. The exit state for the complete expression is the only final state.

(b) Thus every regular expression has a λ -NFA with exactly one final state that accepts its language. The answer is always Yes. □

12.13. Decidability: $\exists$ NFA (without λ -moves) with ≤ 1 final state accepting $L(A)$.

Let $L = L(A)$. We will show that such an NFA exists iff either

$$\lambda \notin L,$$

or else

$$\lambda \in L \quad \text{and} \quad L \cdot L \subseteq L.$$

Necessity: Suppose

$$N = \langle \Sigma, S, s_0, \delta, \{f\} \rangle$$

is an NDFA without λ -moves that accepts L and has at most one final state.

If $\lambda \notin L$, there is nothing more to prove. So assume $\lambda \in L$. Then the start state itself must be final, and since there is at most one final state we have

$$f = s_0.$$

Therefore every accepted word labels a path from s_0 back to s_0 . If $x, y \in L$, we may follow a path for x from s_0 back to s_0 and then a path for y from s_0 back to s_0 . Hence

$$xy \in L.$$

So whenever $\lambda \in L$, we must have $L \cdot L \subseteq L$.

Sufficiency when $\lambda \notin L$: Since regular languages are closed under reversal, construct a DFA for L^r and reverse all its arrows. This gives an NDFA M for L that has one final state q and possibly several start states collected in a set I . Because $\lambda \notin L$, the state q is not a member of I .

Now replace the set I by a single new start state s , and for each $a \in \Sigma$ define

$$\delta(s, a) = \bigcup_{i \in I} \delta_M(i, a).$$

Leave all other transitions unchanged and keep q as the unique final state. Every accepting path in the old machine is simulated by first choosing the same first step from s , so the new machine accepts exactly L . Thus the answer is Yes whenever $\lambda \notin L$.

Sufficiency when $\lambda \in L$ and $L \cdot L \subseteq L$: Start again with the reversed machine M above. Now $q \in I$. Make q the only start state, and redefine its outgoing transitions by

$$\delta'(q, a) = \bigcup_{i \in I} \delta_M(i, a).$$

All other transitions remain as in M , and q is still the unique final state.

Every word of L is accepted by this new machine, because its first move can imitate the first move from whichever old start state in I was used before. Conversely, any accepting path in the new machine starts at q and ends at q . Whenever the path returns to q , split it there. Each resulting segment is a word that was already accepted by the old multi-start machine, hence is in L . Repeated application of $L \cdot L \subseteq L$ shows that the whole input word also belongs to L . So this new machine again accepts exactly L and has one final state.

Decision procedure:

- (a) Check whether $\lambda \in L(A)$ by seeing whether the start state of A is final.
- (b) If $\lambda \notin L(A)$, answer Yes.
- (c) If $\lambda \in L(A)$, construct an automaton for $L(A) \cdot L(A)$ and use Exercise 12.4 to decide whether

$$L(A) \cdot L(A) \subseteq L(A).$$

This inclusion is exactly the required condition, so the question is decidable. □

12.14. **Decidability of $R_1 = R_2$ (same language).**

Algorithm:

- (a) Convert R_1, R_2 to DFAs A_1, A_2 .
- (b) Apply the DFA equivalence algorithm of Theorem 12.3 to A_1 and A_2 . That algorithm constructs the composite DFA and computes its state-equivalence relation, then compares the two start states.
- (c) Answer yes exactly when the two start states are state-equivalent in that composite DFA. This is equivalent to $L(A_1) = L(A_2)$, and hence to $L(R_1) = L(R_2)$. $\square$

12.15. **Decidability: R_1 and R_2 generate the same right congruence ($R_{L_1} = R_{L_2}$).**

Algorithm: Convert R_1 and R_2 to minimal DFAs

$$M_1 = \langle \Sigma, S_1, s_1, \delta_1, F_1 \rangle, \quad M_2 = \langle \Sigma, S_2, s_2, \delta_2, F_2 \rangle.$$

By Exercise 3.40, since these machines are minimal,

$$R_{L_1} = R^{M_1}, \quad R_{L_2} = R^{M_2}.$$

For each $s \in S_1$ and $t \in S_2$, form the two DFAs

$$B_{1,s} = \langle \Sigma, S_1, s_1, \delta_1, \{s\} \rangle, \quad B_{2,t} = \langle \Sigma, S_2, s_2, \delta_2, \{t\} \rangle.$$

Each has exactly one final state, and

$$L(B_{1,s}) = \{x \mid \bar{\delta}_1(s_1, x) = s\}, \quad L(B_{2,t}) = \{x \mid \bar{\delta}_2(s_2, x) = t\}.$$

Because the minimal machines are connected, these languages are precisely the equivalence classes of R^{M_1} and R^{M_2} , respectively. Apply Theorem 12.3 to every pair $B_{1,s}, B_{2,t}$. Answer Yes exactly when every class found from M_1 is equal to exactly one class found from M_2 , and conversely. This is a finite collection of applications of Theorem 12.3, so it decides whether $R_{L_1} = R_{L_2}$. $\square$

12.16. **Prove Theorem 12.5.**

Theorem 12.5 states that it is decidable whether two right-linear grammars

$$G_1 = \langle \Omega_1, \Sigma, S_1, P_1 \rangle, \quad G_2 = \langle \Omega_2, \Sigma, S_2, P_2 \rangle$$

are equivalent.

Use Lemma 8.2 to build NDFAs A_{G_1} and A_{G_2} for the two grammars. Convert these NDFAs to DFAs A_1 and A_2 . Then

$$L(G_1) = L(G_2)$$

iff

$$L(A_1) = L(A_2).$$

By Theorem 12.3, equivalence of DFAs is decidable. Therefore equivalence of the two right-linear grammars is decidable as well. $\square$

12.17. **Efficient algorithm for** $\{\bar{\delta}(s_0, y) \mid y \in \Sigma^*, n \leq |y| < 2n\}$.

Algorithm: Compute the sets

$$T_i = \{\bar{\delta}(s_0, y) \mid y \in \Sigma^i\}$$

recursively.

(a) Set $T_0 = \{s_0\}$.

(b) For $i = 0, 1, \dots, 2n - 2$, compute

$$T_{i+1} = \{\delta(s, a) \mid s \in T_i, a \in \Sigma\}.$$

(c) Return

$$T_n \cup T_{n+1} \cup \dots \cup T_{2n-1}.$$

Each T_i is exactly the set of states reached by strings of length i , so the returned union is the desired set. At stage i , each state in T_i requires $\|\Sigma\|$ transition-table lookups, one for each input letter; since $T_i \subseteq S$, there are at most $n\|\Sigma\|$ lookups at that stage. The procedure halts because it performs only $2n - 1$ finite computations. $\square$

12.18. **Would intersecting** $\{\bar{\delta}(s_0, y) \mid 5n \leq |y| < 6n\}$ **with** F **always work?**

Answer: Yes, it would always produce the correct answer.

Justification: If

$$\{\bar{\delta}(s_0, y) \mid 5n \leq |y| < 6n\} \cap F \neq \emptyset,$$

then some accepted string has length at least $5n$, hence at least n , and the argument used in Theorem 12.2 shows immediately that $L(A)$ is infinite.

For the converse, assume $L(A)$ is infinite. Then Theorem 12.2 tells us that there is an accepted string x with

$$n \leq |x| < 2n.$$

Apply the pumping lemma to x , writing

$$x = uvw$$

with

$$|uv| \leq n, \quad |v| > 0,$$

and

$$uv^i w \in L(A) \quad \text{for all } i \in \mathbb{N}.$$

Choose the least i such that

$$|uv^i w| \geq 5n.$$

Since $|v| \leq n$, minimality of i gives

$$|uv^i w| < 5n + n = 6n.$$

Thus some accepted string has length in the range

$$5n \leq |y| < 6n.$$

So this alternative interval also works. $\square$

12.19. **Decidability of equivalence of two Mealy machines.**

Let

$$M_i = \langle \Sigma, \Gamma, S_i, s_{0_i}, \delta_i, \omega_i \rangle \quad (i = 1, 2).$$

Define the directed product graph to have vertex set $S_1 \times S_2$, start vertex (s_{0_1}, s_{0_2}) , and, for each $a \in \Sigma$, an a -labeled edge

$$(q_1, q_2) \longrightarrow (\delta_1(q_1, a), \delta_2(q_2, a)).$$

Algorithm:

- (a) Perform breadth-first search from (s_{0_1}, s_{0_2}) , examining each reachable pair (q_1, q_2) once.
- (b) For each such pair and each $a \in \Sigma$, compare $\omega_1(q_1, a)$ and $\omega_2(q_2, a)$. If they differ, answer No.
- (c) Otherwise, place the endpoint $(\delta_1(q_1, a), \delta_2(q_2, a))$ of the defined a -edge on the search queue unless it was already examined.

Because the defined product graph has $\|S_1\| \cdot \|S_2\|$ vertices, this search halts. It answers Yes exactly when every pair reached by the same input word produces the same next output symbol on every possible next input, which is precisely equivalence of the two Mealy machines. $\square$

12.20. **Decidability of equivalence of two Moore machines.**

Algorithm: Use the product graph on pairs of states.

- (a) Start from the pair of initial states and first compare their output symbols.
- (b) For every reachable pair (q_1, q_2) , compare $\omega_1(q_1)$ and $\omega_2(q_2)$.
- (c) If a mismatch is found, the machines are not equivalent.
- (d) Otherwise follow matching input symbols to successor pairs and continue.

The product graph is finite, so this procedure halts. It is correct because Moore outputs are determined by the states themselves. $\square$

12.21. **Decide whether R represents strings of length > 28 , without finite automata.**

Algorithm: Compute a capped maximum-length function

$$m(R) \in \{-\infty, 0, 1, \dots, 29\},$$

where $m(R) = 29$ means “ R represents some string of length > 28 .” Define it recursively:

- (a) $m(\emptyset) = -\infty$.
- (b) $m(\lambda) = 0$.
- (c) $m(a) = 1$ for $a \in \Sigma$.
- (d) $m(R_1 \cup R_2) = \max\{m(R_1), m(R_2)\}$.
- (e) For concatenation:

$$m(R_1 R_2) = \begin{cases} -\infty & \text{if } m(R_1) = -\infty \text{ or } m(R_2) = -\infty, \\ 29 & \text{if } m(R_1) = 29 \text{ or } m(R_2) = 29 \text{ or } m(R_1) + m(R_2) > 28, \\ m(R_1) + m(R_2) & \text{otherwise.} \end{cases}$$

(f) For star:

$$m(R_1^*) = \begin{cases} 0 & \text{if } L(R_1) \subseteq \{\lambda\}, \\ 29 & \text{otherwise.} \end{cases}$$

Then R represents some string of length > 28 iff $m(R) = 29$. Since this recursion follows the finite parse tree of R , it always halts. $\square$

12.22. **Decide whether $L(G)$ contains a string of length > 28 for right-linear grammar G .**

Algorithm (without finite automata): Regard the grammar as a weighted directed graph. For each production of the form

$$A \rightarrow xB$$

place an edge $A \rightarrow B$ of weight $|x|$, and for each production

$$A \rightarrow x$$

place an edge from A to a terminal sink $\perp$ of weight $|x|$.

First discard useless nonterminals by the method of Theorem 12.6. Now there are two possibilities.

- (a) There is a reachable productive cycle whose total weight is positive. Then the cycle can be traversed arbitrarily many times before exiting to $\perp$, so $L(G)$ contains arbitrarily long strings, and in particular a string of length > 28 .
- (b) No such cycle exists. Then, after collapsing zero-weight cycles, the useful part of the graph is acyclic. The maximum length of a string derivable from each nonterminal can then be computed by dynamic programming in reverse topological order:

$$\ell(A) = \max\{|x|, |x| + \ell(B)\},$$

taking the maximum over productions $A \rightarrow x$ and $A \rightarrow xB$. The answer is Yes iff $\ell(S) > 28$.

Every step is finite, so the question is decidable. $\square$

12.23. **Prove $Z_0 \subseteq Z_1 \subseteq \dots \subseteq Z_n \subseteq \dots \subseteq \Omega$.**

In the proof of Theorem 12.6 (decidability of $L(G) = \emptyset$ for CFG G), Z_i is the set of nonterminals reachable from the start symbol S in i or fewer derivation steps.

$Z_0 = \{S\}$ (only the start symbol). $Z_{i+1} = Z_i \cup \{B \mid \exists A \in Z_i, \exists \alpha : A \rightarrow \alpha, B \in \alpha\}$ (all nonterminals appearing in right-hand sides of productions of nonterminals in Z_i).

Proof: $Z_0 \subseteq Z_1$ since $Z_1 = Z_0 \cup (\dots) \supseteq Z_0$. By induction: if $Z_k \subseteq Z_{k+1}$, then $Z_{k+1} \subseteq Z_{k+2}$ since any nonterminal in Z_{k+1} is reachable in at most $k+1$ steps, hence is reachable in at most $k+2$ steps. The chain is bounded above by Ω (which is finite). $\square$

12.24. **If $Z_m = Z_{m+1}$, then Z_m is the set of all nonterminals reachable from S .**

Proof: If $Z_m = Z_{m+1}$, then no new nonterminals are added in step $m+1$. This means: every nonterminal appearing in the right-hand side of any production of any nonterminal in Z_m is already in Z_m .

Claim: Z_m is closed under reachability, i.e., every nonterminal reachable from S is in Z_m .

Suppose B is reachable from S (i.e., $S \xRightarrow{*} \alpha B \beta$ for some α, β). The shortest derivation has some length k . By monotonicity ($Z_0 \subseteq Z_1 \subseteq \dots$), $B \in Z_k$. Since $Z_m = Z_{m+1} = Z_{m+2} = \dots$ (the sequence stabilizes), $B \in Z_m$ for all $k \leq m$ already. If $k > m$, then by the stabilization, every nonterminal reachable in k steps is already in Z_m . So $Z_m =$ all reachable nonterminals. $\square$

12.25. **Prove** $\exists m \in \mathbb{N} : Z_m = Z_{m+1}$.

Proof: The sequence $Z_0 \subseteq Z_1 \subseteq \dots \subseteq \Omega$ is a non-decreasing chain of subsets of Ω . Since $|\Omega|$ is finite, the chain can increase at most $|\Omega|$ times. Therefore there exists $m \leq |\Omega|$ such that $Z_m = Z_{m+1}$ (the chain stabilizes). $\square$

12.26. **Definition of W_i and monotonicity.**

- (a) In the proof of Theorem 12.6, W_i is the set of nonterminals that can derive a terminal string in i or fewer steps: $W_0 = \emptyset$. $W_{i+1} = W_i \cup \{A \mid \exists (A \rightarrow \alpha) \in P : \text{every symbol in } \alpha \in \Sigma \cup W_i\}$. (A nonterminal A can derive a terminal string in $\leq i+1$ steps if some production of A has a right-hand side consisting only of terminals and nonterminals already known to derive terminal strings.)
- (b) **Proof of monotonicity:** $W_0 = \emptyset \subseteq W_1$ (trivially). By induction: if $W_k \subseteq W_{k+1}$, then for any $A \in W_{k+1}$: A derives a terminal string in $\leq k+1$ steps, hence in $\leq k+2$ steps as well, so $A \in W_{k+2}$. Thus $W_{k+1} \subseteq W_{k+2}$. The chain is bounded by Ω . $\square$

12.27. **If $W_m = W_{m+1}$, then W_m is the set of all nonterminals producing terminal strings.**

Proof: If $A \xRightarrow{*} w \in \Sigma^*$ in k steps, then $A \in W_k$. Since $W_m = W_{m+k}$ for all $k \geq 0$ (by stabilization), $A \in W_m$. Conversely, every $A \in W_m$ derives a terminal string by definition. So W_m is precisely the set of productive nonterminals. $\square$

12.28. **Prove** $\exists m \in \mathbb{N} : W_m = W_{m+1}$.

Proof: By the definition of the sets W_i ,

$$W_0 \subseteq W_1 \subseteq W_2 \subseteq \dots \subseteq \Omega.$$

Whenever $W_i \neq W_{i+1}$, at least one new nonterminal of the finite set Ω has been added. Since Ω has only finitely many elements, this can happen only finitely many times. Therefore for some m no new symbol is added, so $W_m = W_{m+1}$. $\square$

12.29. **Decidability questions about NDFA A with λ -moves.**

- (a) **Exists string with no complete path:** Build the equivalent DFA A^d . For any string x , the state of A^d reached by x is exactly the set of states of A at the ends of complete paths labeled x . Thus there is a string with no complete path iff the empty subset is reachable in A^d .
- (b) **Exists string with a path to a nonfinal state:** In A^d , search for a reachable subset that contains a nonfinal state of A . Such a subset exists iff some string has at least one complete path ending in a nonfinal state.
- (c) **Exists accepted string with all paths leading to final states:** This asks whether there is a reachable subset Q of A^d such that

$$Q \neq \emptyset, \quad Q \subseteq F.$$

Then x is accepted (because Q is nonempty and every element is final), and every complete path on x ends in a final state.

- (d) **All accepted strings have all complete paths to final states:** This fails exactly when some reachable subset Q of A^d satisfies

$$Q \cap F \neq \emptyset \quad \text{and} \quad Q - F \neq \emptyset.$$

So the property is decidable by checking whether such a reachable subset exists.

- (e) **All strings have unique paths:** Construct a finite product machine whose states record two simultaneous computations of A on the same input, together with a bit indicating whether the two computations have diverged. Then there exists a string with two distinct complete paths iff a state with “diverged = yes” is reachable in this product machine. Hence uniqueness of paths is decidable.

12.30. Decidability for two DFAs A_1 and A_2 .

- (a) **Decidability of homomorphism existence:** A homomorphism $h : A_1 \rightarrow A_2$ (of DFAs) is a state map preserving start state, transitions, and taking final states to final states. Check all functions $h : S_1 \rightarrow S_2$: finitely many ($|S_2|^{|S_1|}$). For each, check the homomorphism conditions. Decidable (finite check).
- (b) **Decidability of isomorphism existence:** An isomorphism requires h to be bijective (so $|S_1| = |S_2|$). Check all bijections: $(|S_1|)!$ many. For each, verify the isomorphism conditions. Decidable (finite check for finite S_1).
- (c) **Decidability of > 3 isomorphisms:** Enumerate all bijections $h : S_1 \rightarrow S_2$. For each one, check whether it preserves the start state, transitions, and final states. Count how many bijections pass these tests. There are at most $(|S_1|)!$ candidates, so the procedure halts. Answer yes exactly when the final count exceeds 3. Hence the question is decidable. $\square$

12.31. Decidability: R_1 describes infinitely many strings (no DFA construction).

Algorithm (structural induction on R_1): - $R_1 = \emptyset$: finite (empty). No. - $R_1 = \lambda$ or $R_1 = a$: finite. No. - $R_1 = R \cup R'$: infinite iff R or R' is infinite. - $R_1 = R \cdot R'$: infinite iff (R is infinite and $L(R')$ is nonempty) or ($L(R)$ is nonempty and R' is infinite). - $R_1 = R^*$: infinite iff $L(R)$ contains a nonempty string.

Each check is computable by structural recursion on R_1 . Terminates since R_1 is finite. $\square$

12.32. Decidability of Mealy machine equivalent to Moore machine.

Let M be the given Mealy machine and A the given Moore machine. By Definition 7.19, construct the corresponding Mealy machine M_A from A . Theorem 7.7 gives

$$f_{M_A} = f_A,$$

where the translation functions are those of Definition 7.18. Therefore

$$M \text{ is equivalent to } A \iff M \text{ is equivalent to } M_A.$$

Algorithm:

- (a) Construct M_A using Definition 7.19.
- (b) Apply the product-graph procedure from Exercise 12.19 to M and M_A .

The construction and the search are finite, so this decides the question. □

12.33. **Decidability:** $L(R_2)$ properly contains $L(R_1)$ ($L(R_1) \subset L(R_2)$).

Algorithm:

- (a) Convert R_1, R_2 to DFAs.
- (b) Check $L(R_1) \subseteq L(R_2)$ (decidable, Exercise 12.4).
- (c) Check $L(R_1) \neq L(R_2)$ (decidable, Exercise 12.14 or 12.15).
- (d) $L(R_1) \subset L(R_2)$ iff both conditions hold. □

12.34. **Efficient method for $L(A) = \emptyset$ for NDFA A , without DFA conversion.**

- (a) **Method:** BFS/DFS directly on the NDFA A (viewed as a graph). Start from the start set S_0 , compute its λ -closure, then follow transitions. $L(A) = \emptyset$ iff no final state is reachable from any state in S_0 (via any sequence of transitions and λ -moves). Reachability on the finite graph of A is enough; there is no need to construct the exponential subset DFA.
- (b) **Method for infinite $L(A)$:** First find all states reachable from the initial λ -closure and all states from which some final state is reachable. Keep only these useful states. Then compute the strongly connected components of this useful subgraph. The language is infinite iff some useful component contains a cycle with at least one non- λ transition. Such a cycle can be pumped to produce arbitrarily many different accepted strings; if no such cycle exists, every accepting path has bounded input length. Hence the question is decidable without converting to a DFA. □

12.35. **Decidability:** $L(M)$ is a regular set for DPDA M .

Answer: This question is decidable, but the proof is substantially deeper than the earlier decidability arguments in the chapter. The standard regularity algorithm for DPDAs can be outlined as follows.

- (a) Put M into an equivalent normal form in which the effects of entering and later leaving a stack symbol can be summarized by a finite collection of control-state pairs.
- (b) Compute these finite summaries for all reachable stack symbols and use the DPDA equivalence procedure to identify summaries with the same residual input language.
- (c) Test the recurrent stack-growth patterns. If they yield infinitely many pairwise distinct residual languages, then $L(M)$ is not regular. Otherwise the finitely many residual languages are the states of an equivalent DFA, which the procedure constructs.

This procedure always terminates because the normal-form summaries are finite descriptions and the equivalence tests used in the recurring cases are decidable. It answers Yes exactly when the constructed finite collection of residual languages recognizes $L(M)$.

Caution: it is *not* enough to ask whether this particular DPDA happens to keep its stack bounded, since a DPDA can use unbounded stack space and still recognize a regular language. □

12.36. **Algorithm for path-finding in graphs (Theorem 12.9).**

- (a) **Appropriate algorithm:** BFS (breadth-first search) from the source node to find all reachable nodes and shortest paths. For directed graphs (as in the PDA product machine):
- Initialize a queue with the start node; mark it visited.
 - At each step, dequeue a node, enqueue all unvisited neighbors, mark them visited.
 - Terminate when the queue is empty or the target is found.

Time complexity: $O(|V| + |E|)$ where V is vertices and E is edges.

- (b) **More efficient recursive algorithm:** DFS with memoization. $\text{Reach}(v, t)$: returns true if t is reachable from v . - If $v = t$: return true. - If v is marked: return false. - Mark v ; for each neighbor u of v : if $\text{Reach}(u, t)$: return true. - Return false.

This avoids reprocessing nodes and runs in $O(|V| + |E|)$ time. $\square$

12.37. $\mathfrak{H}_\Sigma$ **(halting-TM languages) closed under union, intersection, concatenation, reversal.**

- (a) **Union:** Given halting TMs $M_1 (L_1)$ and $M_2 (L_2)$, build M for $L_1 \cup L_2$: run M_1 and M_2 in parallel (interleaved simulation). Since both halt on all inputs, M also halts. M accepts iff M_1 or M_2 accepts. $\square$
- (b) **Intersection:** Same parallel simulation; accept iff both accept. Both halt, so M halts. $\square$
- (c) **Concatenation:** M for $L_1 \cdot L_2$ on input x : enumerate all splits $x = uv$, run M_1 on u and M_2 on v . Since both halt, each check halts. There are $|x| + 1$ splits (finitely many), all decidable. Accept if any split works. $\square$
- (d) **Reversal:** M for L_1^r on input x : compute x^r (finitely computable), run M_1 on x^r . M_1 halts on x^r ; so M halts. Accept iff M_1 accepts x^r . $\square$

12.38. **Halting TM T' equivalent to $L_2(T)$ for halting TM T .**

$L = L_2(T)$: T halts on all inputs and L consists of strings for which T halts with $\mathbf{Y}$ on the tape.

Construct T' :

- Simulate T on input w .
- T halts (since it halts on all inputs by assumption).
- If T halted with $\mathbf{Y}$ somewhere on the tape: erase the tape, write w then $\mathbf{Y}$ after w on an otherwise blank tape; halt in the accepting state.
- If T halted with no $\mathbf{Y}$: erase the tape, write w then $\mathbf{N}$ after w ; halt in a rejecting state.

T' satisfies all three conditions: (1) halts on all input (since T does and the post-processing is finite); (2) writes $\mathbf{Y}$ after accepted words; (3) writes $\mathbf{N}$ after rejected words. And $L(T') = L_2(T) = L$. $\square$

12.39. **Machine $M_{\mathfrak{M}}$ and application to Theorem 12.15.**

- (a) **Proof:** Assume there is a machine $M_{\mathfrak{M}}$ that decides membership in X , and that every encoding in X represents a halting Turing machine. Define a halting Turing machine D as follows on input x :

- i. Use $M_{\mathfrak{M}}$ to test whether $x \in X$.
- ii. If $x \notin X$, reject.
- iii. If $x \in X$, let T_x be the Turing machine encoded by x , simulate T_x on input x , and do the opposite: accept iff T_x rejects x , and reject iff T_x accepts x .

This simulation halts because every machine encoded by a member of X is halting. So D is itself a halting Turing machine.

Now suppose $L(D) \in \mathfrak{M}$. Then there is some encoding $d \in X$ whose machine T_d recognizes $L(D)$. Consider what D does on input d .

- If $d \in L(D)$, then by the definition of D , the machine T_d rejects d , contradicting $L(T_d) = L(D)$.
- If $d \notin L(D)$, then by the definition of D , the machine T_d accepts d , again contradicting $L(T_d) = L(D)$.

Thus $L(D) \notin \mathfrak{M}$. So there exists a halting Turing machine that recognizes a language not in $\mathfrak{M}$. $\square$

- (b) **Application to Theorem 12.15:** Let X be the set of strings that encode valid context-sensitive grammars by the scheme of the next exercise. Membership in X is decidable. Also, each member of X represents a halting Turing machine, because Theorem 12.9 gives an algorithm for deciding membership in the language of a context-sensitive grammar. Therefore part (a) applies to the class $\mathfrak{M} = \mathfrak{D}_{\Sigma}$ of context-sensitive languages, and yields a halting Turing machine whose language is not context sensitive. That is exactly Theorem 12.15.

12.40. Encoding context-sensitive grammars.

- (a) **Encoding scheme:** Assume 2 terminal symbols $\{a, b\}$ and arbitrarily many nonterminals $N_1, N_2, N_3, \dots$

Encoding: - Terminals: $a \mapsto 00$, $b \mapsto 01$. - Nonterminal $N_i \mapsto 1^i 0$ (unary encoding with terminator). - Productions: $\alpha \rightarrow \beta$ encoded as $\bar{\alpha} \# \# \bar{\beta}$ where $\bar{x}$ is the encoding of x symbol by symbol. - Grammar: list of productions separated by $\# \# \#$, preceded by the start symbol encoding.

This encoding handles an unbounded number of nonterminals via unary encoding and terminates on any finite grammar.

- (b) **Algorithm to decide valid CSG encoding:**

- i. Parse the encoding to extract the alphabet, nonterminal list, start symbol, and productions.
- ii. Check that: (a) terminal symbols are from $\{a, b\}$; (b) nonterminals are distinct and properly encoded; (c) each production $\alpha \rightarrow \beta$ has $|\beta| \geq |\alpha|$ (context-sensitive condition), except possibly $Z \rightarrow \lambda$.
- iii. Verify the syntactic structure of the encoding (valid separators, well-formed strings).
- iv. The TM accepts valid encodings and rejects invalid ones. Terminates in polynomial time in the encoding length. $\square$

12.41. Undecidability of $L(X) = \emptyset$.

- (a) **Arbitrary Turing machines:** Undecidable. Given a Turing machine T and a word w , build a machine $M_{T,w}$ that ignores its own input, simulates T on w , and accepts iff that simulation halts. Then

$$L(M_{T,w}) = \emptyset$$

iff T does *not* halt on w . So a decider for emptiness would solve the halting problem.

- (b) **Halting Turing machines:** Still undecidable. Given an arbitrary Turing machine T and word w , build a halting Turing machine $H_{T,w}$ that on input x simulates T on w for exactly $|x|$ steps and accepts iff T halts within that many steps. Then $H_{T,w}$ halts on every input. Also,

$$L(H_{T,w}) = \emptyset$$

iff T never halts on w . So deciding emptiness for halting Turing machines would decide the halting problem.

- (c) **Context-sensitive grammars:** By Corollary 11.2, context-sensitive grammars and LBAs define the same class of languages. So it is enough to prove part (d): if emptiness were decidable for context-sensitive grammars, it would also be decidable for LBAs by first converting the LBA to an equivalent grammar.
- (d) **Linear bounded automata:** Given T and w , build an LBA $B_{T,w}$ that on input x of length n simulates T on w for exactly n steps and accepts iff T halts within those n steps. This can be done by a linear bounded automaton because in n steps the simulated machine can visit at most $n + |w|$ tape cells, and $|w|$ is a fixed constant; the finitely many very short inputs can be handled separately in the finite control.

Then

$$L(B_{T,w}) = \emptyset$$

iff T never halts on w . So emptiness for LBAs is undecidable, and therefore emptiness for context-sensitive grammars is undecidable as well. $\square$

12.42. Undecidability of $L(X) = \Sigma^*$.

- (a) **Arbitrary TMs:** Undecidable. Given T and w , build a machine $M'_{T,w}$ that ignores its input, simulates T on w , and accepts iff T halts. Then

$$L(M'_{T,w}) = \Sigma^*$$

iff T halts on w . So universality for arbitrary Turing machines is undecidable.

- (b) **Halting TMs:** Undecidable. Given T and w , define a halting Turing machine $U_{T,w}$ that on input x simulates T on w for exactly $|x|$ steps and accepts iff T has *not* halted within that many steps. Then $U_{T,w}$ halts on all inputs, and

$$L(U_{T,w}) = \Sigma^*$$

iff T never halts on w . So universality for halting Turing machines is undecidable.

- (c) **Context-sensitive grammars:** By Corollary 11.2, it is enough to prove the corresponding statement for LBAs.

- (d) **Linear bounded automata:** Given T and w , build an LBA $C_{T,w}$ that on input x of length n simulates T on w for exactly n steps and accepts iff T has *not* halted within those n steps. The simulation uses only linear space: the machine keeps x on one track, uses a counter of size $O(\log n)$ to count from 0 up to n , and stores on the remaining track space a length- $O(n)$ window describing the first n tape cells that T could have visited in its first n steps. Since a machine running for only n steps cannot move farther than n cells from its start position, this bounded work area is sufficient. Hence $C_{T,w}$ is indeed an LBA. Now

$$L(C_{T,w}) = \Sigma^*$$

iff T never halts on w . Hence universality is undecidable for LBAs, and therefore also for context-sensitive grammars.

- (e) **Context-free grammars:** Undecidable. Use the standard reduction from the Post correspondence problem. From an instance P , construct a context-free grammar G_P that generates every well-formed candidate encoding except an encoding of a solution of P ; malformed encodings and candidates with a first mismatch are generated directly. Thus

$$L(G_P) = \Sigma_P^* \iff P \text{ has no solution.}$$

The construction of G_P is effective, so a universality procedure for context-free grammars would decide the Post correspondence problem. Hence no such procedure exists.

- (f) **Pushdown automata:** Undecidable. Given a context-free grammar G , Theorem 10.2 effectively constructs a pushdown automaton P_G with $\Lambda(P_G) = L(G)$. Therefore a universality procedure for pushdown automata would decide the context-free-grammar case just proved. $\square$

12.43. Set E of encodings of TMs halting on λ .

- (a) $E \in \mathfrak{T}_\Sigma$: Yes. The TM that recognizes E works as follows: on input $\langle T \rangle$, simulate T on λ . If T halts, accept. If T does not halt, keep simulating forever. Therefore E is Turing acceptable.
- (b) $E \in \mathfrak{H}_\Sigma$: No. If $E \in \mathfrak{H}_\Sigma$, then E would be decidable. But E is exactly the halting problem on input λ , which is undecidable. So $E \notin \mathfrak{H}_\Sigma$.
- (c) $E \in \mathfrak{D}_\Sigma$: No. By Corollary 12.3, every context-sensitive language belongs to $\mathfrak{H}_\Sigma$. Since $E \notin \mathfrak{H}_\Sigma$, it certainly cannot belong to $\mathfrak{D}_\Sigma$.

12.44. Set N of encodings of TMs that do NOT halt on λ .

- (a) $N \in \mathfrak{T}_\Sigma$: No. Here

$$N = \overline{E}.$$

If both E and $\overline{E}$ belonged to $\mathfrak{T}_\Sigma$, then we could dovetail the two semidecision procedures and decide E , which would put E in $\mathfrak{H}_\Sigma$. But Exercise 12.41(b) showed that $E \notin \mathfrak{H}_\Sigma$. Therefore $N \notin \mathfrak{T}_\Sigma$.

- (b) $N \in \mathfrak{H}_\Sigma$: No. If N were in $\mathfrak{H}_\Sigma$, then both E and $N = \overline{E}$ would be recursive languages. A recursive language and its complement are both decidable, so E would be decidable. But the non-halting problem is undecidable. Therefore $N \notin \mathfrak{H}_\Sigma$.

- (c) $N \in \mathfrak{D}_\Sigma$: No. Again, $\mathfrak{D}_\Sigma \subseteq \mathfrak{H}_\Sigma$, and we have just seen that $N \notin \mathfrak{H}_\Sigma$. Hence $N \notin \mathfrak{D}_\Sigma$. $\square$

12.45. **Decidability questions about equivalence classes of $R_{L(A)}$.**

- (a) **At least one equivalence class is finite:** Minimize A to obtain the connected DFA

$$M = \langle \Sigma, S, s_0, \delta, F \rangle.$$

Its states correspond to the classes of $R_{L(A)}$. For a state $q \in S$, the corresponding class is

$$I(M, q) = \{x \in \Sigma^* \mid \bar{\delta}(s_0, x) = q\}.$$

This set is infinite iff some directed cycle reachable from s_0 can also reach q : traversing such a cycle any number of times gives infinitely many different strings in $I(M, q)$. Conversely, if no reachable cycle can reach q , then only finitely many paths lead from s_0 to q , so $I(M, q)$ is finite.

Therefore at least one class is finite iff there is some state q not reachable from any reachable cycle. This is decidable by graph search.

- (b) **All equivalence classes are finite:** No. By Definition 1.1, Σ is nonempty, so Σ^* is infinite. The relation $R_{L(A)}$ has only finitely many equivalence classes because $L(A)$ is FAD (equivalently, the reduced connected DFA is isomorphic to $A_{R_{L(A)}}$ by Theorem 3.1). If all of those finitely many classes were finite, then their union Σ^* would be finite, a contradiction. Thus the one-step decision procedure always answers No. $\square$

12.46. **Decide $L(G_1) \subset L(G_2)$ for right-linear grammars G_1, G_2 .**

Algorithm:

- (a) Convert G_1, G_2 to DFAs A_1, A_2 (via Lemma 8.1).
- (b) Check $L(A_1) \subseteq L(A_2)$ (decidable, Exercise 12.4).
- (c) Check $L(A_1) \neq L(A_2)$ (decidable, Exercise 12.14).
- (d) $L(G_1) \subset L(G_2)$ (proper subset) iff $L(A_1) \subseteq L(A_2)$ and $L(A_1) \neq L(A_2)$. $\square$